



AMD Generic Encapsulated Software Architecture (AGESA™) Interface Specification

Publication # 55483	Revision: 1.50
Issue Date: Aug 2020	

AMD Confidential—Advance Information
© 2016-2020 Advanced Micro Devices, Inc. All rights reserved.

The information contained herein is for informational purposes only, and is subject to change without notice. While every precaution has been taken in the preparation of this document, it may contain technical inaccuracies, omissions and typographical errors, and AMD is under no obligation to update or otherwise correct this information. Advanced Micro Devices, Inc. makes no representations or warranties with respect to the accuracy or completeness of the contents of this document, and assumes no liability of any kind, including the implied warranties of noninfringement, merchantability or fitness for particular purposes, with respect to the operation or use of AMD hardware, software or other products described herein. No license, including implied or arising by estoppel, to any intellectual property rights is granted by this document. Terms and limitations applicable to the purchase or use of AMD's products are as set forth in a signed agreement between the parties or in AMD's Standard Terms and Conditions of Sale.

Trademarks

AMD, the AMD Arrow logo, AGESA, and combinations thereof, are trademarks of Advanced Micro Devices, Inc.

PCIe and PCI Express are registered trademarks of PCI-SIG.

Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

Contents

List of Figures	6
List of Tables	7
List of Abbreviations	8
Revision History	9
1 Introduction	24
1.1 Terminology and Definitions	24
1.2 References	24
1.3 Delivery Package Directory Structure	25
1.3.1 Package—Single Family	26
2 Design and Concepts	28
2.1 Overview	28
2.2 UDK Environment	28
2.3 Configuration Parameters	29
2.4 Error Logging	33
2.5 Debug	33
2.6 Version label	34
2.7 Porting Guide & Topics	34
2.7.1 Porting The Development Environment	34
2.7.2 Porting Capsule Support	39
3 UEFI UDK API	43
3.1 FCH Drivers Module	43
3.1.1 USB Port Mapping	44
3.1.2 FCH Configuration Items	45
3.1.3 FCH_CONTROL_SERVICE_PROTOCOL	57
3.2 Promontory Drivers Module	59
3.2.1 The meaning of 'Auto'	66
3.2.2 Promontory Configuration Items	66
3.2.3 EFI_PT_GPIO_PPI	67
3.2.4 EFI_PT_GPIO_PROTOCOL	69
3.2.5 EFI_PT_GPIO_SMM_PROTOCOL	71
3.3 Error Log Driver Module	73
3.3.1 Error Log Error CLasses	74
3.3.2 PEI_AMD_ERROR_LOG_SERVICE_PPI	77
3.3.3 PEI_AMD_ERROR_LOG_SERVICE_READY_PPI	79
3.3.4 PEI_AMD_ERROR_LOG_FULL_PPI	80
3.3.5 PEI_AMD_ERROR_LOG_AVAILABLE_PPI	80
3.3.6 Error Log Services Protocol	81
3.3.7 AmdAcquireErrorLogWithFlagDxe()	82
3.3.8 AMD_ERROR_LOG_SERVICE_READY_PROTOCOL	83
3.3.9 AMD_ERROR_LOG_AVAILABLE_PROTOCOL	83
3.3.10 AMD_ERROR_LOG_FULL_PROTOCOL	84
3.4 Memory Driver	84
3.4.1 AMD_MEMORY_INFO_HOB_PPI	85

3.4.2 mMemPmuTrainingFailureHobReady.....	88
3.4.3 Mem Configuration Items.....	90
3.4.4 AMD_MEMORY_MBIST_TEST_RESULTS_PPI.....	91
3.4.5 AMD_MEMORY_INIT_COMPLETE_PPI.....	97
3.5 Core Complex Driver.....	116
3.5.1 The meaning of 'Auto'.....	117
3.5.2 CCX Configuration Items.....	117
3.5.3 AMD_ENABLE_UEFI_STACK2.....	127
3.5.4 AMD_DISABLE_UEFI_STACK2.....	128
3.5.5 DXE_AMD_CCX_MP_SERVICES_PREREQ_PROTOCOL.....	129
3.6 NBIO Driver.....	129
3.6.1 NBIO Configuration Items.....	130
3.6.2 DXIO Driver Common Structures.....	163
3.6.3 Published PPIs and Protocols.....	168
3.6.4 Consumed PPIs and Protocols.....	173
3.6.5 DXIO Complex Installation Macros.....	181
3.6.6 NBIO HotPlug Installation Macros.....	194
3.7 Data Fabric Driver.....	201
3.7.1 The meaning of 'Auto'.....	201
3.7.2 DF Configuration Items.....	202
3.7.3 FABRIC_RESOURCE_MANAGER_PROTOCOL.....	207
3.7.4 FABRIC_SOC_SPECIFIC_SERVICES_PROTOCOL [F17M30].....	214
3.7.5 FABRIC_NUMA_SERVICES_PROTOCOL FABRIC_NUMA_SERVICES2_PROTOCOL	215
3.8 Platform Security Processor Driver.....	221
3.8.1 PSP Configuration Items.....	221
3.8.2 PSP Protocol (DXE).....	222
3.8.3 PSP_RESUME_SERVICE_PROTOCOL.....	223
3.8.4 PSP_fTPM_PPI.....	225
3.8.5 PSP_fTPM_Protocol.....	228
3.8.6 AMD_APCB_SERVICE_PROTOCOL.....	232
3.8.7 PSP Mailbox Functions.....	253
3.8.8 AMD_PSP_FIRMWARE_VERSION_PROTOCOL [F19M00].....	259
3.9 RAS (Reliability, Availability and Serviceability).....	260
3.9.1 RAS Configuration Items.....	261
3.9.2 AMD_RAS_APEI_PROTOCOL.....	266
3.9.3 AMD_RAS_APEI2_PROTOCOL [F17M08].....	269
3.9.4 AMD_RAS_SMM_PROTOCOL.....	274
3.9.5 AMD_RAS_SMM2_PROTOCOL [F17M08].....	282
3.9.6 AMD_NBIO_PCIE_AER_PROTOCOL.....	291
4 AGESA Boot Loader (ABL).....	296
4.1 ABL Configuration.....	296
4.1.1 The meaning of 'Auto'.....	296
4.1.2 Memory Related Tokens.....	296
4.1.3 SPD_Info (Data).....	318
4.1.4 ABL Platform Specific Overrides Configuration Items.....	320
4.1.5 ABL Platform Timing Overrides Configuration Items.....	326
4.1.6 Console Controls.....	349
4.1.7 ABL Error Reporting Configuration Items.....	353
4.1.8 ABL MBIST Configuration Items.....	355
4.2 Selecting one of Multiple Boards.....	360
4.3 APCB V2 Components.....	365
4.3.1 Using multiple Board Support.....	365
4.4 APCB V3.0 Components.....	365
4.4.7 Platform APCB Porting.....	369

4.4.8 Managing Data.....	371
4.4.9 Multiple Board Support.....	374
5 Platform Topics and Services.....	378
5.1 Platform CMOS.....	378
5.2 Platform Writes to Flash.....	379
5.3 DDR4 Post Package Repair.....	379
5.4 Memory Context Restore [F17M10].....	381
5.5 Multi-Socket Designs.....	381
5.5.1 Normal Mode Macros.....	382
5.5.2 Compact Mode Macros.....	387
5.5.3 System Use Case Examples.....	389
5.6 APB Library.....	391
5.7 ACPI Library.....	393
5.7.1 ALIB Function 01: Report AC/DC State.....	393
5.7.2 ALIB Function 02: Performance Request.....	394
5.7.3 ALIB Function 03: PSPP Start/Stop.....	395
5.7.4 ALIB Function 04: Set Bus Width.....	396
5.7.5 ALIB Function 05: ALIB Initialize.....	396
5.7.6 ALIB Function 06: Hot Plug Request.....	397
5.7.7 ALIB Function 0A: Report Dock State.....	398
5.7.8 ALIB Function 0B: Battery Status.....	398
5.7.9 ALIB Function 0C: Dynamic Power and Thermal Configuration.....	399
5.7.10 AWAK Method: Apu Wake.....	403
5.7.11 APTS Method: Apu Sleep.....	403
5.8 Firmware Managed PCIe® Resets.....	404
6 Tools.....	407
6.1 APB Tool (V2).....	407
6.1.1 Running the APBTool.....	407
6.2 APB Tool (V3).....	408
6.3 AGESA Boot Loader Debug.....	408
6.3.1 ABL Error Codes.....	411
6.3.2 Mono-Directional Protocol.....	416
6.3.3 Bi-Directional Protocol.....	417
6.4 MakeFile Generator.....	418
6.5 ABL Version Checker.....	419
7 External MicroControllers.....	421
7.1 BMC via PCIe® Link.....	421
8 Support for Family 15h.....	424
8.1 The meaning of 'Auto'.....	424
8.2 Memory Driver for Family 15h.....	424
8.2.1 Configuration Items for Family15h (Memory Driver).....	424
8.3 Core Complex Driver for Family 15h.....	436
8.3.1 Configuration Items for Family 15h (CCX Driver).....	436
8.4 NBIO Driver for Family 15h.....	437
8.4.1 Configuration Items for Family 15h (NBIO Driver).....	437
8.4.2 PEI_AMD_NBIO_PCIE_COMPLEX_FM15_PPI.....	438
8.5 Data Fabric for Family 15h.....	439
8.5.1 Configuration Items for Family 15h (Data Fabric Driver).....	439

List of Figures

Figure 1. SOC Driver Launching.....	28
Figure 2. Setting the PCD values.....	31
Figure 3. Flow Diagram for Capsule operation.....	39

List of Tables

Table 1. Reference Documents.....	24
Table 2. Parameter Group.....	31
Table 3. Multi Param Groups.....	32
Table 4. USB Port Mapping.....	44
Table 5. FCH-Enablement Feature Availability.....	45
Table 6. OC Pin mapping.....	47
Table 7. USB PHY parameters.....	56
Table 8. USB PHY Lane Tuning Parameters.....	56
Table 9. Fatal Error Codes.....	74
Table 10. Alert Error Codes.....	75
Table 11. Bounds Check Error Codes.....	76
Table 12. Sensor coefficients.....	145
Table 13. PCIe PHY Tuning Parameters.....	191
Table 14. PCIe Port Tuning Parameters.....	193
Table 15. Memory Interleaving.....	311
Table 16. Pin Maps.....	320
Table 17. Pin Map Example 1.....	320
Table 18. Pin Map Example 2.....	321
Table 19. APB v3 File Summary.....	371
Table 20. Data Values Precedence.....	373
Table 21. CMOS storage usage.....	378
Table 22. ABL Error Codes.....	411
Table 23. ABL Bi-Directional Handshake Protocol.....	417

List of Abbreviations

Revision History

Date	Revision	Description
Aug 2020	1.50	
		Changed default for PcdAmdEnhancedPreferredIOMode to False.
		Added PCIe LclkDPMLevel control.
		Some updates to FCH controls for F19M00.
		Add controls to disable prefetchers .
		Added enabled for [F19M00] new instructions ERMS/FSRM .
		Added new PCD for SEV-SNP .
		Updated Down Core control for new Family.
		Deprecate DimmSensorResolution token for [F17M30][F19M00].
		Added: PcdPcieAriForwardingEnable , PcdCfgDffoDis , PcdCfgDfloDis , PcdAmdDlfCapEn , PcdAmdDlfExEn . Modified: PcdAmdNbIoRASControlV2 , PcdAmdPcieAerReportMechanism , PcdCfgApbDis , PcdAmdLcLoopbackWaitForAllActiveLanes .
		Updated CCX_PPIN_OPT_IN .
		Added MEM_PWRDOWNDLY , MEM_SPEED_DDR4 , HASH_BANK_COL_XOR , PMU_TRAIN_FFE_DDR4 , PMU_TRAIN_DFE_DDR4 .
		Added 'Availability table' to FCH Enablement configuration items.
		Added control for EnSpecStFill .
		New Protocol for NBIO_Services .
July 2020	1.48	
		Removed a VMPL control that was not needed.
		Update options for Double Refresh Rate.
		Added control to enable DIMM Self Healing .
		Added HSMP control.
		Added Self-Refresh Staggering control.
		Added new memory HOB for PMU Training results .
		Addition of NTB controls .
		Re-structured several PCD sections to alter the list format so as to prevent name wrapping.
		Added new PHY Tuning & Control controls.
		Duplicated DF interleaving control for new family implementation.
		Updated definition for DF Storage at Top .
May 2020	1.46	
		Addition of SST 2.1 PCD name changes.
		Added new PCDs for PCIe link tuning of presets.
		Updated Mem_Restore description.
		Added Boot Timer control.
		Added two Read-Only PCDs for TDP and Tjmax fused values.

Date	Revision	Description
		Added a PCB for DLWM enablement.
Apr 2020	1.44	
		Add reference for F17M30 RAS document.
		Added ROM3 region Base address control. Description of need in ROM Armor feature and new APCB token .
		Add new LegacyEnable IO range controls for the UART ports.
		Added Apic Mode control , CCD Mode Control and warning message for large systems having more than 256 cores.
		Update to SWCMDTHROTEN control.
		Update to SWCMDTHROTCYC control.
		Added new PCD to disable ASF slave .
		Added new Fatal error when mixing LRDIMM/RDIMM .
		Added new Protocol to read Firmware versions .
		Update options list for CCX_SEV_ASID_COUNT and marked for [F17M30][F19M00].
		Add Max Activity Count control.
		Add Memory refresh controls UrgRefLimit .
		Add CBS control token for Mem PowerDown enable.
		Add workaround option for F17M30 errata 1166 PcdPcieGen4x2AerCorrRcvErrBadTLPMaskEnable .
		Add action control for BIST failure APCB_TOKEN_UID_ACTION_ON_BIST_FAILURE .
		Deprecated PreferredIODevice , Platform Category .
		Several Token and PCD name updates ('Config' to 'UID') for F17M30 and F19M00. Noted in each entry.
Jan 2020	1.42	
		Updated memory context restore feature description.
		Added PcdPcieGen4x2AerCorrRcvErrBadTLPMaskEnable to enable workaround for errata 1166.
		Added PcdAmdPreferredIOBus and PcdAmdEnhancedPreferredIOMode .
		Clarify bit definition of FchRTDeviceEnableMap .
		Add notes about BoardID read process.
		New PCD control for PCIe Maximum Read Size .
		Clarification of MakeFileTool usage.
		PCDs for PCIe timing control.
		Added new design section on the Version string.
		Added new section on CMOS usage by the AGESA software.
Nov 2019	1.40	
		Removed Chapter 7 section on BMC via IO-ports.
		Added APCB control for Soldered down DRAM .
		Add a HotPlug control.
		Modified error code 0x4020 - Mixing ECC & non-ECC is not permitted.
		Update for NVDimm PCDs and guidance .
		Added token for pre-x86 TPM support.

Date	Revision	Description
		Clarify XGMI tools usage.
		Replaced PSP mailbox section with new AMD ROM Armor description.
		Added on/off PCD for CPPC feature.
		Added PCDs to NBIO and DF drivers.
Oct 2019	1.38	
		.Added Fabric Numa ServicesProtocol.
		Add definition for SATA lane PP_Invert_Polarity .
		Add APCB control for 'Group D' platforms.
		Added section on Memory Context Restore , updated the platform CMOS power topic and added a platform Flash Writes topic.
		Add MCTP PCDs to designate PCI address of the BMC.
		Added SPD optimized reads control.
		Added FCH controls for Uart and I2C IRQ control.
July 2019	1.36	
		Collected DIMM throttling controls to break-out paragraph. Added new controls.
		Remove deprecated ABL Error codes.
		Added section for eSPI PCD controls.
		Added APCB control for GPP clocks .
		Updated description for PostPackageRepair .
		Added PCD for PCIe lane vetting .
		Added APCB token for early PCIe-BMC link De-emphasis .
		Added PcdAmdFabricCcxAsNumaDomain .
		Added new sections PSP Config Items for PcdAmdPspApcbRecoveryEnable, and related Platform CMOS Power with references to/from affected features or options.
		Updated description for FCH_CONSOLE_OUT_BASIC_ENABLE Console Controls .
		Add new PCDs for PCIe global setting of RX Margin and ASPM mode.
		Add PCD for SpreadSpectrum .
		Add Protocol for XGMI Frequencies .
		Add new PCDs for Model 60h thermal management " STT ".
		ALIB Function 0Ch clarification.
		Default Fan Table updates.
April 2019	1.34	
		Updated some APCB names, removed 'BLDCFG'.
		Hotplug updates.
		Clarify PCIe_Reset_Control token.
		Clarify PCIe_Reset_Pin token.
		Add DXIO Port Params .
		Added PcdAmdPreferredIODevice .
		Added PCIe SRIS controls.

Date	Revision	Description
March 2019	1.32	
		Removed PCDAMDPcieLegacyReportMechanism as it is obsolete.
		Added parameter 'instanceId' to ApcbSetType .
		Add info to Delete PPR entry .
		Clarified porting guide Porting The Development Environment Build Time, step 4.
		clarified usage of Console Filtering .
		Clarified params for Dqs Drive Strength .
		Clarified PTO-UMC/PHY time when applied.
		Name changes 'Config' -> 'UID': MBIST_AGGR_STATIC_LANE_CTRL MBIST_AGGR_STATIC_LANE_SEL_ECC MBIST_AGGR_STATIC_LANE_SEL_L32 MBIST_AGGR_STATIC_LANE_SEL_U32
		Added more items to MemInitComplete structure.
		Added IO method for getting a board ID.
		Changed ABL PPR error 0x4007 to Warning class, from Fatal class.
		Added APB controls to specify the BMC link Alpha for early training : APCB_TOKEN_UID_BMC_SOCKET_NUMBER APCB_TOKEN_UID_BMC_DEVICE APCB_TOKEN_UID_BMC_FUNCTION APCB_TOKEN_UID_BMC_START_LANE APCB_TOKEN_UID_BMC_END_LANE APCB_TOKEN_UID_BMC_LINK_SPEED Also moved all BMC related controls to this common chapter.
		Added note about 'prefetchable' .
		Update several APB names for APB V3 changes: DIMMSENSORCONF. FORCEPWRDOWNTHROTN. DIMMSENSORRESOLUTION. DIMMSENSORUPPER. DIMMSENSORLOWER. DIMMSENSORCRITICAL. ENABLEPOWERDOWN. MEMORYALLCLOCKSON. MEMRESTORECTL. POST_PACKAGE_REPAIR_ENABLE. MEM_TSME_ENABLE. MEM_DATA_SCRAMBLE. MEM_DATA_POISON.

Date	Revision	Description
		Updated several MBIST APCB token names for APCB V3 changes: MEM_MBIST_AGGRESSORS_CHNL. MEM_MBIST_PATTERN_SELECT. MEM_MBIST_PATTERN_LENGTH. MEM_MBIST_READ_DATA_EYE_TIMING_STEP. MEM_MBIST_READ_DATA_EYE_VOLTAGE_STEP. MEM_MBIST_WRITE_DATA_EYE_TIMING_STEP. MEM_MBIST_WRITE_DATA_EYE_VOLTAGE_STEP. MEM_MBIST_TGT_STATIC_LANE_SEL_U32. MEM_MBIST_TGT_STATIC_LANE_SEL_L32. MEM_MBIST_WORST_CASE_GRAN. MEM_MBIST_TESTMODE. MEM_MBIST_AGGR_STATIC_LANE_VAL. MEM_MBIST_TGT_STATIC_LANE_VAL. MEM_MBIST_TGT_STATIC_LANE_CTRL. MEM_MBIST_TGT_STATIC_LANE_SEL_ECC.
		Updates for MemInitComplete parameters.
		Added APCB_TOKEN_UID_ENABLEMEMPSTATE.
		Added APCB Token: ENABLEBANKSWIZZLE.
		Clarifications to ErrLog Acquire .
		Added new ErrorLog Acquire functions to DXE and PEI.
		Add Data Fabric Driver error log entries for 1TB remap and NPS errors.
		Update of Hot Plug structures and function parameters.
		Deprecated APCB_TOKEN_UID_ENABLEZQRESET.
February 2019	1.30	
		Add details for ApcbGetDramPostPkgRepairEntries .
		Description Update for ApcbFlushData() ApcbFlushData .
		Move ApcbSetActiveInstance() to APCB 2.0-only section.
		Add board revision check to ID process of GetBoardID .
		Add new controls for Audio SSID .
		Removed PCDs for VoltageMargin .
		Added APCB_TOKEN_UID_DF3_XGMI2_LINK_CFG .
		Updated descriptions of PTO parameters.
		new tokens for baud rate and console control .
		Update Dimm Sensor settings.
		Split MBIST lane selects to 2x32bit values.
		Added BLDCFG_PCIE_RESET_PIN_SELECT to APCB token descriptions.
		.Added new Mem PCDs for MaxDiePerSocket/etc. Corrected a reference in APCB_TOKEN_CONFIG_USERTIMINGMODE .
		Add new PCD for NVDIMM GPIO FSR_REQ .

Date	Revision	Description
		Added APCB modification user example .
		Removed Token for Extended Mem Test.
		Added PCD for FCH Configuration Items Motherboard type.
		FCH Cold reset porting.
		Added token for NPS control.
		Added control PcdAfterResetDelay .
		Clarified parameters for ApcbGetType() .
		Added new PCD controls for I2C SDA hold time .
		Clarify PTO UMC controls.
		Clarify CBS linking process.
		Added SATA example for Phy tuning .

Date	Revision	Description
		Added several NBIO PCDs: • PcdCfgNbIoSsid • PcdCfgIommuSsid • PcdCfgPspccpSsid • PcdCfgXgbeSsid • PcdCfgNbifF0Ssid • PcdCfgNbifRCSsid • PcdCfgIommuSupport • PcdCfgAEREnable • PcdEarlyBmcLinkLaneNum • PcdAmdPowerSupplyIdleControl • PcdPcieLinkTrainingType • PcdCfgForcePcieGenSpeed • PcdDxioMajorRevision • PcdDxioMinorRevision • PcdHotplugI2cAddress • PcdHotplugSlotIndex • PcdAmdDisableIrqPoll • PcdAmdDlfsfcEn • PcdAmdRxMarginEnabled • PcdAmdNbIoReportEdbErrors • PcdCfgPcieLoopbackMode • PcdAmdLcLoopbackWaitForAllActiveLanes • RVPcdPspPolicy • PcdEfficiencyOptimizedMode • PcdCfgSystemConfiguration • PcdCfgApbDis • PcdCfgFixedSocPstate • PcdTelemetry_VddcrVddfull_Scale_Current • RVPcdTelemetryVddcrVddfullScale2Current • RVPcdTelemetryVddcrVddfullScale3Current • RVPcdTelemetryVddcrVddfullScale4Current • RVPcdTelemetryVddcrVddfullScale5Current • PcdTelemetry_VddcrSocfull_Scale_Current • PcdTelemetry_VddcrVddOffset • PcdTelemetry_VddcrSocOffset • PcdFastPptLimit • PcdSlowPptLimit • PcdSlowPptTimeConstant • PcdVrmLowPowerThreshold • PcdVrmSocLowPowerThreshold • PcdCfgThermCtlValue • PcdFMaxFrequency • TablePcdSbTsiAlertComparatorModeEn • PcdCfgPPT • PcdCfgPlatformTDP • PcdCfgPlatformPPT • PcdCfgPlatformTDC • PcdCfgPlatformEDC • PcdPcieEcrcEnablement • PcdPcieEcrcSeverityFatal • PcdAmdNbIoRASControlV2 • PcdAmdEdpcEnable • PcdAmdPcieSyncFloodOnFatal
		Clarifications RAS Driver's PCD section .
December 2018	1.28	

Date	Revision	Description
		Corrected typo in ABL_Log_Entry.IdInfo.
		Removed unused APCB tokens.
		FCH SATA config - removed 2 redundant SATA PCDs: PcdAmdSataEnable and PcdAmdSataEnable2. Only PcdSataEnable is needed.
		Add STTMinLimit PCD. Added reference for Thermal Design Guide . Added reference for Infrastructure RoadMap (IRM) .
		Improved definition of APCB_TOKEN_CONFIG_CCX_MIN_SEV_ASID .
		Updated controls for CCX SEV ASIDs APCB_TOKEN_UID_CCX_SEV_ASID_COUNT .
		Clarify the CpuDis value in FABRIC_RESOURCE_MANAGER_PROTOCOL .
		Align Error codes with AMD.h file in Error Log Driver .
		Added BLDCFG_DF_REMAP_AT_1TB .
		Add PcdAmdFabricSratBelow1MBReportingMode .
		Add APCB v3.0 Runtime Services in APCB_Services_Protocol .
		Added new routines supporting APCB v3.0: • ApcbPurgeAllTypes, • ApcbPurgeAllTokens, • ApcbFlushData (Updated), • ApcbAcquireMutex, • ApcbReleaseMutex, • ApcbGetTokenBool, • ApcbSetTokenBool, • ApcbGetToken8, • ApcbSetToken8, • ApcbGetToken16, • ApcbSetToken16, • ApcbGetToken32, • ApcbSetToken32, • ApcbGetType, • ApcbSetType,
		Add ABL log Filtering Controls feature.
		Updated Memory interleaving option for NUMA NPSx topologies.
		Updated ABL log filter control.
		Added PSP_MBoxLib2 for SPI locking and WhiteList feature.
		Moved APCB info to Chapter level. Added APCB V3 . Added sections for APCB V3 Porting and Data Management information.
		Changes to Board Getting Method .
		Add details about APCB 'Group' and 'Type'.
		Marked APCB_ExtOutputPortType settings as 'reserved'.
		Clarify use of PcdPcieEqSearchPreset .
		Clarify HotPlug mask parameter mFunctionMask .
		Added descriptor and notes about DXIO_PORT_DATA MiscControls. LinkComplianceMode . Added PCD: PcdPcieLinkComplianceModeAllPorts .
		Clarify USB port mapping.
		Remove mI2CGpioDeviceMappingExt from PCIE_HOTPLUG_INITIALIZER_FUNCTION .
		Added control for PcdPCIEExactMatchEnable .

Date	Revision	Description
June 2018	1.26	
		Added new DXIO controls PcdPcieEqSearchMode , PcdPcieEqSearchPreset ; Added new parameter DXIO_PORT_DATA_INITIALIZER_PCIE.mEqPreset .
		Updated the definition for ABL_MEM_ERROR_LRDIMM_MIXMFG .
		Resolve name conflict for DIMM_SPD_DATA_STRUCT .
		Added control for PcdCfgPcieLoopbackMode .
		Change default to FALSE for PcdAmdXgmiReadaptation .
		Added new control PcdAmdRRCEn replacing PcdAmdDisableIrqPoll .
		Added new control PcdCfgLcSchedRxEqEvalInt .
		Add Platform Memory Timing table PTO macros .
May 2018	1.24	
		Added Debug controls for PSP. APCB_TOKEN_CONFIG_PSP_TP_PORT , APCB_TOKEN_CONFIG_PSP_ERROR_DISPLAY , APCB_TOKEN_CONFIG_PSP_STOP_ON_ERROR .
		Added new error messages: ABL_MEM_ERROR_MIXED_X4_AND_X8_DIMM_IN_CHANNEL , ABL_MEM_ERROR_MIXED_3DS_AND_NON_3DS_DIMM_IN_CHANNEL , ABL_MEM_ERROR_MIXED_ECC_AND_NON_ECC_DIMM_IN_CHANNEL .
		Expanded definition of error message 0x4003: ABL_MEM_AGESE_MEMORY_TEST_ERROR
		Added memory test control. APCB_TOKEN_CONFIG_MEM_AGESE_EXTENDED_MEM_TEST_EN .
		Redefined Data-A definition of error message 0x4001: ABL_MEM_PMU_TRAIN_ERROR .
		Added list of ACPI and SMBios tables generated. CCX Driver
		Clarify APCB_NVDIMM_POWER.Configuration items: Enablement Category
		Update definition for GetSystemMemoryMap .
		Added verbosity control for ABL console out: APCB_TOKEN_CONFIG_FCH_CONSOLE_BASIC_OUT_ENABLE .
		Improve ABL status reporting. Added revision info and new structure elements to Memory_Init_Complete_PPI and new Memory Configuration HOB .
		Memory Driver updates for F17M30, added new SPD elements to rev4 of Memory Init Complete PPI .
		Update FCH controls. Added XHCI and SATA controller enables in enhancement category. Added include file list to Build-Time Support .
		Make Parity error controls available to customers PARITY_RETRY .
		Missing definitions for some memory parameters.
		Expand definition for ApcbUpdateCbsData() ApcbUpdateCbsData() .
		Updates to APCB services APCB get/set param .
		Added definition for ApcbAddDramPostPkgRepairEntry() DRRP_REPAIR_ENTRY .
		Clarify APCB PPR record entry content REPAIR_ENTRY .
		Added more Tokens for MBIST control to MBIST Cfg Items .
		Added clarification and examples for the mapping parameters in PSO macros .
		Added a reference for the PCIe spec and clarified Set AER fcn .
		Clarify parameters of ApcbReplaceType() .

Date	Revision	Description
April 2018	1.22	
		Corrected typo in ABL_Log_Entry.IdInfo. Added 'Processor Variant Tagging' to terminology section.
		Set NBIO PCDs for CRS to default of 0.NBIO Configuration Items.Performance.
		Increase the number of package repair entries from 32 to 64.Repair_Entry.
		Change default for PcdAmdPowerCeiling to value 0.PcdAmdPowerCeiling.
		Add PCDs for TDC and EDC for F17M08.PcdCfgTDC.
		Expand on Multi-APCB support. Added section Selecting an APCB.
		Add new RAS protocols for F17M08.RAS_APEI2 and RAS_SMM2.
February 2018	1.20	
		Updated error classes and added new memory error code #4033 for mis-matched LRDIMMs.ABL Error Codes
		Moved the Family 15h support information to new chapter Family 15h Support
		Add FCH protocols. FCH_Service_Protocol and new PCDs for USB port enables FCH PCD Enables
		Added PcdCfgDisableRcvrResetCycle to section NBIO PCDs for Performance Tuning.
January 2018	1.18	
		New PCDs for DPTC. - Removed pcdCurrentLimit, pcdMaximumCurrentLimit - Added PcdVrmSocCurrentLimit, PcdVrmCurrentLimit, PcdProchotlDeassertionRampTime, PcdStapmTimeConstant, PcdSustainedPowerLimit, PcdVrmMaximumCurrentLimit, PcdVrmSocMaximumCurrentLimit.
		Added APCB_TOKEN_CONFIG_FCH_SMBUS_SPEED to allow selecting the SMBus speed.
		Updates to Data Fabric Resource Manager definition.
		New DF control. Added PcdEfficiencyOptimizedMode
		Updates to the MBIST Data Eye descriptions for the PPI, HOB and configuration mode.
		Added USB 2.0 PHY parameter PCD: PcdUsbOemConfigurationTable.
		Add public PCD for Vmin Frequency to the NBIO driver.
		Add public PCD for PeApm feature - manage dGPU + iGPU power.
		New NBIO PCDs for ZP Hot Plug feature controls.
		Add PCDs for controlling power ceiling and performance cap.(CCX Power Management)
		Added note about supported memory frequencies to APCB_TOKEN_CONFIG_MEMORYBUSFREQUENCYLIMIT.
		Add PCD control to enable DDR4 ECC error retries.ABL Performance controls
		Added PCDs for XGMI links: PcdAmdNbIO SpcMode2P5GT & PcdAmdXgmiReadaptation.NBIO Performance Controls
		Add ECC Symbol size of 16 for Family 17h Model 30h.

Date	Revision	Description
		Add NBIO RAS PCDs. RAS Operational Parameters • PcdEgressPoisonSeverityHi • PcdEgressPoisonSeverityLo • PcdAmdMaskNbIoSyncFlood • PcdSyncFloodToApml • PcdAmdNbIoEgressPoisonMaskHi • PcdAmdNbIoEgressPoisonMaskLo • PcdAmdNbIoRASUcpMaskLo • PcdAmdNbIoRASUcpMaskHi • PcdSyshubWdtTimerInterval • PcdSlinkConvertReadResponseErrorsToOkay • PcdSlinkWriteResponseErrorHandling • PcdAmdLogPoisonDataFromSLink • PcdAmdPcieLegacyReportMechanism • PcdAmdPcieAerReportMechanism
		Add PCDs for CRS on Hot-Plug: PcdCfgPcieCrslimit & PcdCfgPcieCrslDelay. See NBIO PCDs - Tuning .
September 2017	1.16	
		BOTTOM_IO clarification. Wording changed in error log entry and added "known restrictions" sub paragraph to APGB Token definition.
		New event log entry for memory map errors. Added: • DF_FATAL_MEM_MAP_NOT_CREATED • DF_ALERT_PCI_MMIO_SIZE_MODIFIED
		Add S3 support PCD. Added PcdAmdAcpiS3Support to the CCX driver configuration items list.
		MBIST updates. Changes made to AMD_MEMORY_MBIST_TEST_RESULTS_PPI and its sub functions. • Removed PEI_DET_MBIST_DATA_EYE_RESULTS() • Removed ABL BldCfg items: • CPU_V_REF_DQ_RANGE • DRAM_V_REF_DQ_RANGE • HALT_ON_ERROR • Added MBIST_HOB definition
		Add optional SMM_Access2 protocol to CCX driver description.
		Clean up of memory parameters - Removed: APGB_TOKENS: • _MEMORYUDIMMCAPABLE • _MEMORYRDIMMCAPABLE • _MEMORYLRDIMMCAPABLE • _MEMORYLRDIMMCAPABLE • _MEMORYQUADRANKCAPABLE • _UMAMODE • _UMASIZE • _UMAABOVE4G • _UMAALIGNMENT • _LIMITMEMORYTOBELOW1TB • _ENABLEBANKSWIZZLE • _DRAMDOUBLEREFRESHRATE
		Added note to section 2.2.1.3 to state current tools being used.
		Added note to each configuration items chapter to define the meaning of 'auto'.
		Added HotPlug reference. Made some clarifications to the Hot-Plug descriptor macros.

Date	Revision	Description
August 2017	1.14	
		Added new section for "DXIO Published PPIs": - Added PEI_AMD_NBIO_PCIE_TRAINING_START_PPI - Added PEI_AMD_NBIO_PCIE_TRAINING_DONE_PPI
		Added ALIB function 0x0C_0x10 - dGPU Control.
		Added error codes for CCX and DF drivers to section 3.3.1
		Add a limit control for the Post Package Repair buffer size. Added PcdAmdMemCfgMaxPostPackageRepairEntries to section 4.3
		Updated the MMIO resources protocol. - Expanded from 1 to 4 functions. - Added IO support in addition to the MMIO region. - Changed protocol name to FABRIC_RESOURCE_MANAGER_PROTOCOL - replaced PCDs PcdAmdBitMapForMmioBelow4GOnSocket0 and PcdAmdBitMapForMmioBelow4GOnSocket1 with new PcdAmdMmioAbove4GLimit
		ARI/SRIOV support. Added PcdCfgPcieAriSupport to NBIO PCD list.
		Add more detail for memory interleaving settings. Updated description for APCB_TOKEN_CONFIG_CONFIG_DF_MEM_INTERLEAVING.
		Firmware Managed Resets. Added: - Reference to CPM document - new section in platform topics Firmware Managed PCIe® Resets - BLDCFG_PCIE_RESET_CONTROL to ABL configuration
June 2017	1.12	
		Various minor clarifications to APCB Library ApcbReplaceType.
		Added PCIe® Auto Speed Change parameter.
		Added some missing APCB parameters.
		Spec update for new CST PCD.
		Spec update for new SLIT PCDs. Added PcdAmdFabricSlitDegree, PcdAmdFabric2ndDegreeSlitDistance, PcdAmdFabric3rdDegreeSlitLocalDistance, PcdAmdFabric3rdDegreeSlitRemoteDistance.
		Removed PcdIvrsEnumAll.
		Update for new PMU results info in ABL error log.
		Updates to the ALIB Fcn 0x0C (DPTC) Command values.
May 2017	1.10	
		Add APCB_TOKEN_CONFIG_MEM_DATA_SCRAMBLE.
		Add AMD_NBIO_PCIE_AER_PROTOCOL.
		Add new alert codes for BIST: - CPU_EVENT_NON_CCX_BIST_FAILURE - CPU_EVENT_CCX_BIST_FAILURE Added new table for error codes to Error log driver.

Date	Revision	Description
		Add several missing APCB tokens. • APCB_TOKEN_CONFIG_MEM_DATA_POISON • APCB_TOKEN_CONFIG_ECCSYMBOLSIZE • APCB_TOKEN_CBS_DBG_MEM_RCD_PARITY_DDR4 • APCB_TOKEN_CBS_DBG_MEM_WRITE_CRC_ERROR_MAX_REPLAY_DDR4 • APCB_TOKEN_CBS_DBG_MEM_CTRLER_WR_CRC_EN_DDR4
		Provide ability to control HD audio via PCD: PcdCfgHdAudioEnable .
		Add several new PCDs for SPI bus speed control to FCH driver. • PcdResetMode • PcdResetSpiSpeed • PcdResetFastSpeed Removed clock control PCDs. • PcdGppClockRequest0 • PcdGppClockRequest1 • PcdGppClockRequest2 • PcdGppClockRequest3 • PcdSltGfxClockRequest0 • PcdSltGfxClockRequest1
		Added ABL error records (0x4002 - 0x404D) to the error table.
		New PCDs for ACPI table names. Added new name string PCDs for: CPU SSDT, ALIB SSDT, PT SSDT, CIDT, DRAT, SLIT, SRAT, IVRS to the NBIO driver section.
		Update the Memory_Complete_PPI. Updated the definition for ABL: DIMM_INFO_SMBUS
		Update definition for 'device' in APCB_DPPRCL_REPARI_ENTRY.
		Updated structures for AMD_NBIO_PCIE_AER_PROTOCOL.
		Updated the control parameter options for the ALIB DPTC, Fcn 0C.
		Added PcdAmdNumberOfPhysicalSocket to the CCX config items list.
		Added APCB token for MBIST debug output.
		Added new PSP Driver Protocol, AMD_APCB_SERVICE_PROTOCOL for modifying the APCB entries.
		Updated parameters of PSP_RESUME_REGISTER. Added text to better describe the function.
March 2017	1.8	
		Updated definitions for the MBIST feature.
		Added APCB XGMI coefficient settings
		Added place holder for ABL BeepCodeTable
		New PCDs for Overclocking limits
		Added NVDIMM PCD for Power_Source
March 2017	1.6	
		Added section 3.10 for RAS driver RAS (Reliability, Availability and Serviceability) .
		Added section 3.5.3.3 for Hot Plug Protocol and section 3.5.5 for the HotPlug macros .
		Add APCB entries for: Write CRC, RCD Parity and Recovery
		Add APCB entries for: TSME
		Update: APCB Package Integration

Date	Revision	Description
		Modified (split) PcdAmdAllocateMmioBelow4GOnThisDie into PcdAmdBitMapForMmioBelow4GOnSocket0 and PcdAmdBitMapForMmioBelow4GOnSocket1
January 2017	1.4	
		Added section 3.9 for ALIB ACPI Library .
		Added section 2.6.2 for Capsule Updates Porting Capsule Support .
		Added new DF APB Token in section 4.1.3 Memory Related Tokens .
		Modified sections 3.5.4, 3.5.5.2, 3.5.5.3 for various updates for new PHY data and added section 3.5.5.5 for the new PHY tuning macros PHY Tuning Macros .
		Added section 4.1.3 for new PCD value Memory Related Tokens .
		Corrected description in section 4.1.3 for Interleaving options Memory Related Tokens .
		Added PcdIvrsEnumAll in section 3.5.1 Platform Category .
		Added ABL Heartbeat feature to sections 4.2, 4.1.2 AGESA Boot Loader Debug .
		Added new NBIO PCD values in section 3.5.1 Enablement Category
September 2016	1.2	
		Updated the APB build process described in Running the APB Tool .
		Added new FCH PCDs for clocks control .
		Added note to Porting Guide - Build-Time .
		Added step to Porting Guide - Dev Check list .
		Deprecate APB_TOKEN_BOTTOMIO.
July 2016	1.0	
		Post Bring-Up modifications: • Replaced DXE_AMD_CCX_INIT_COMPLETE_PROTOCOL with new version DXE_AMD_MP_SERVICES_PREREQ_PROTOCOL • Updated the Porting Guide section • Added FABRIC_MMIO_MAP_MANAGER_PROTOCOL. • Added PcdAmdAllocateMmioBelow4GOnThisDie for the MMIO manager • Added ETHERNET_PORT_DATA to DXIO_PORT_DESCRIPTION • Updated NBIO 'PCIe' labels to 'DXIO_' • Updated DXIO driver structure definitions and moved them to a common file • Added Macro definitions for DXIO installation • Updated parameter for AmdAcquireErrorLogPei • Updated the package string, using Summit example • Added ACPI spec to list of references • Added section for Promontory (PT) driver • Updated Development check list in the Porting Guide section
April 2016	0.95	
		Updated ABL handshake protocol; added IdInfo block
		Updated porting guide section; fixed naming for AmdCalloutLib
		Update the AGESA version strings to unified package string .
		Added description for ArchV9 tool.
		Updated to MEMORY_INFO_HOB ; updated and added ABL config for memory .
		Added ABL version checking tool ; Added step to porting guide .
		Updated the instructions for the APB Tool .

Date	Revision	Description
		Added section for External Controllers communication.
March 2016	0.90	Changed title name to match PI release name; Added content.
February 2016	0.3.0	Initial NDA release.

1 Introduction

AMD Generic Encapsulated Software Architecture (AGESA™) Version 9, is a collection of program modules that are executed as part of a host platform BIOS. The program modules focus on the initialization of AMD silicon and are intended to be integrated with an IBV or open source code base.

AGESA V9 supports UDK platform code bases and is a modular design based on a layered BIOS approach.

This document describes the details of the AGESA V9 user interface, how to use it, and how to prepare the host BIOS for integrating the AGESA V9.

This document assumes familiarity with the UEFI Development Kit (UDK) .

1.1 Terminology and Definitions

Component	A piece of technology integrated into the SoC
Configuration Item	A user level control used to specify desired operation characteristics or to describe the motherboard hardware environment.
PCD	A UEFI data construct used to store a Configuration Item.
UEFI	Unified Extensible Firmware Interface
UDK	UEFI Development Kit
IBV	Independent BIOS Vendor
PPI	PEIM to PEIM Interface
Protocol	UEFI DXE module to DXE module interface
SoC	System On a Chip
SoC Driver	A PEIM or DXE module responsible for defining the components needed to operate the specific SoC
Component Driver	A PEIM or DXE module responsible for operating a piece of the SoC technology.

Processor Variant Tagging

Services that are available for select versions of the Processors will have a Family-Model tag. This will appear, for example, as "F17M10" which indicates Family 17h, Model 10h.

1.2 References

Table 1. Reference Documents

Short Name	PID	Title
BIOS and Software		
PPR	54945	<i>Processor Programming Reference (PPR) for AMD Family 17h Models 00h-0Fh Processors</i>
PPR	55803	<i>Processor Programming Reference (PPR) for AMD Family 17h Models 30h-3Fh Processors</i>
UEFI	v2.5	<i>UEFI Specification</i>
UEFI PEI	v1.2C	<i>UEFI Platform Initialization Specification</i>
UEFI Pkg	v1.0B	<i>UEFI Distribution Package Specification</i>
PSP Arch	55758	<i>AMD Platform Security Processor BIOS Architecture Design Guide for AMD Family 17h Processors</i>
ACPI	v6.1	<i>Advanced Configuration and Power Interface Specification</i>

Table 1. Reference Documents (continued)

Short Name	PID	Title
RAS	55987	<i>RAS Feature Enablement for AMD Family 17h Models 00h-0Fh</i>
RAS	56278	<i>RAS Feature Enablement for AMD Family 17h Models 30h-3Fh</i>
PkgRepair	1817.15A	<i>Proposed JEDEC DDR4 POST Package Repair</i>
CPM	55730	<i>Common Platform Module Implementation Guide</i> . Available in the PI at: PI\AmdCpmPkg\Addendum\Doc
PCIe<tm tmtype="reg "/>	3.1	<i>PCI Express Base Specification</i>
Numa NPSx	56338	<i>Socket SP3 Platform NUMA Topology AMD Family 17h, Model 30h-3Fh</i>
CPPC	56653	<i>AMD Collaborative Processor Performance Control (CPPC)</i>
NTB	56710	<i>Non-Transparent Bridging for Socket SP3 Family 17h Model 31h Processors</i>
Debug		
HDT	N/A	<i>Hardware Debug Tool—Hardware and software that interfaces to the JTAG and DB Ports to gain control of internal functions of the processor</i>
Platform		
MBDG	55414	<i>Socket SP3 Processor Motherboard Design Guide</i>
MBDG	55597	<i>FP5 Processor Motherboard Design Guide</i>
HotPlug	55996	Hot Plug Application Note
APML	55719	Advanced Platform Management Link Specification
ThDG	55595	<i>FP5 Thermal Design Guide for FP5 Processors</i>
ThDG	56483	<i>FP6 Thermal Design Guide for FP6 Processors</i>
IRM (FP5)	55593	<i>FP5 Infrastructure RoadMap</i>
IRM (SP3)	55418	<i>SP3 Infrastructure RoadMap</i>
NVDIMM	56333	NVDIMM-N Support for Socket SP3 Family 17h Models 30h-3Fh Processors Application Note

1.3 Delivery Package Directory Structure

AGESA is not a single monolithic binary. There are multiple driver components that relate to one of:

- Common infrastructure functions
- IP block technologies
- SoC products
- BIOS POST Phase (dependent upon the environment)

These component drivers work together to accomplish the proper silicon initialization.

There are several AGESA component drivers included in the directory structure. Many of these are internal drivers that have no user interaction. These will not be covered in this Interface specification. For details about these drivers, please see the embedded comments.

The term 'package' is rather generic and ubiquitous, generally meaning a collection of program files. For the purpose of this spec the term 'PI package' will be used to indicate the AGESA PI deliverable code set. The term 'Pkg' (abbreviation for "package") will be used to indicate a UEFI installable code module as defined in the UEFI specifications. The AGESA PI package contains two UEFI Pkgs, AmdModulePkg and AgesaPkg.

The AmdModulePkg is the AGESA pkg that contains the private AGESA drivers and interfaces between modules. The interfaces and modules in this pkg are intended for AMD internal use only and are subject to change without notice. Customers are discouraged from using the information in this pkg. The AGESA driver

INF files referenced the Agesa<FamilyA><Socket1>ModulePkg.dec files (containing private/AMD internal only interfaces for PPI/Protocols/GUIDs/PCDs/Library classes) for silicon specific interfaces. The AGESA driver INF also references the AgesaPkg.dec file (containing public generic AGESA interfaces for PPI/Protocols/GUIDs/PCDs/Library classes) for generic AGESA interfaces.

This pkg also contains the Agesa<FamilyA><Socket1>Pkg.dsc and the Agesa<FamilyA><Socket1>Pkg.fdf files that allow the platform BIOS to build the AGESA drivers.

The AgesaPkg is the AGESA public interface pkg. It contains the PPIs/Protocols and PCDs that are used to interface with the platform BIOS. Customer's platform BIOS interfaces will communicate with AGESA via this pkg.

Each of the AGESA PI packages will be designed to support one or more Families and/or Sockets.

1.3.1 Package—Single Family

The directory structure for an AGESA deliverable code set that supports only one family with one socket is defined as follows.

AgesaPkg

This package contains all of the public interfaces and build configuration files for the AGESA™ V9 support. Its contents include:

Addendum	Platform BIOS reference/sample code
Include	Files needed by other modules to interface with this one.
Agesa<FamilyA><Socket1>Pkg.fdf	
Agesa<FamilyA><Socket1>Pkg.dsc	
AgesaPkg.dec	AGESA interface configuration item definitions.

If the platform BIOS needs to support a single family with one socket, it will need to include the “Agesa<FamilyA><Socket1>Pkg.fdf” and “Agesa<FamilyA><Socket1>Pkg.dsc” in its platform build. The AGESA public interfaces can be referenced from AgesaPkg.dec.

AgesaModulePkg

This package contains all of the private interfaces and build configuration files for the AGESA™ V9 support. Its contents include, but is not limited to:

CCX	AGESA core complex driver
Debug	AGESA debug driver directory
Fabric	AGESA fabric driver directory
FCH	AGESA FCH driver directory
Include	
Guid	
Library	
Ppi	
Protocol	
Library	

Mem	AGESA Memory driver directory
MemHobInfo	AGESA Memory Hob Info directory
NBIO	AGESA NBIO driver directory
PSP	AGESA PSP driver directory
SOC	SOC driver directory
Agesa<Family><Socket>Pei	
Agesa<Family><Socket>Dxe	
Agesa<Family><Socket>Smm	
Universal	
Error Log	Error Log driver directory
Agesa<Family><Socket>ModulePkg.dec	

2 Design and Concepts

2.1 Overview

2.2 UDK Environment

V9 “AGESA Drivers” are compliant with the “Native UDK Drivers”. The operational flow is controlled by the UDK core dispatcher for PEI, DXE and SMM.

2.2.1 UDK driver hierarchy

The UDK environment dispatches the “AGESA Drivers”, evaluates dependencies, broadcasts notifications and publishes service PPIs. The AGESA drivers use the UDK notification and dependency systems to control the overall flow and rely on the UDK core dispatcher for execution ordering. Please see figure 2.

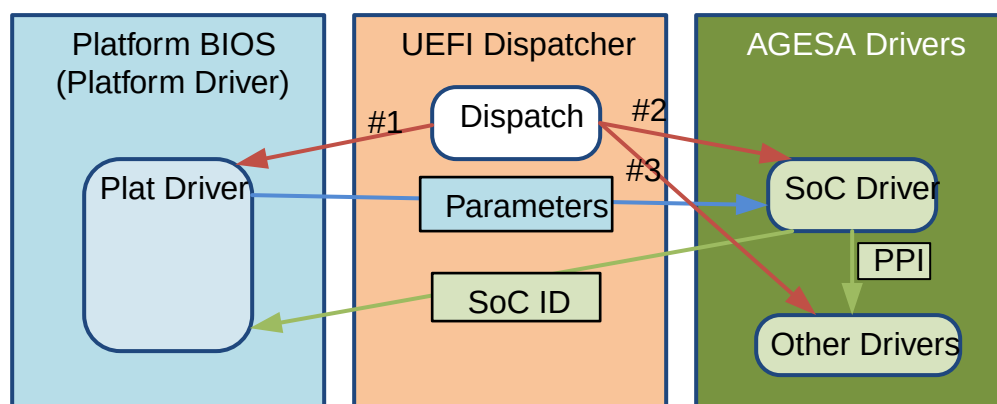


Figure 1. SOC Driver Launching

2.2.1.1 Process Flow

This section describes an example UDK execution flow:

1. At build-time, AGESA will define the PCD values for the AGESA parameters.
2. At build-time, the platform BIOS can specify platform specific values for the AGESA parameters using the build process – PCD value re-assignment.
3. At BIOS run-time, UDK dispatcher dispatches the first AGESA driver - the SOC driver.
4. The “SOC Driver” checks its “Silicon ID” to confirm that it matches to the SOC.
 - a. The driver publishes the list of PPIs/Protocols that each driver depends on to be dispatched.
 - b. In the case of a combo BIOS, multiple SoC drivers may be launched, but only one will pass this test and continue.
5. The SoC driver establishes the AGESA parameter values (more on this in the 2.5.4).
6. The SOC driver initialization completes.
7. The UDK dispatcher dispatches the other AGESA Drivers.
8. Each driver starts to execute after its specific PPI/Protocol dependencies from the SOC Driver PPI/Protocol list is published and its other dependencies are met.
9. If the “Driver” passes the test and is part of the SOC support team, it continues to execute else it aborts.

10. The initialization code will continue execution until it requires the use of a PCD.
11. If a “Driver” requires information from another “Driver”, it uses internal PPIs/Protocols to obtain the information or services.
12. The “Driver” can publish PPIs/Protocols for services when they become available to allow other “Drivers” to consume their services.
13. If an error is detected by the “Driver”, it reports it via a PPI/Protocol from the “Error Log Driver”.
14. The “Error Log Driver” collects the AGESA errors into an event log and publishes a PPI/Protocol to the platform BIOS when all errors are collected.
15. If the driver needs to issue a warm reset, it uses a PPI/Protocol from the “Warm Reset Driver”.
16. The “Driver” exits.

2.2.1.2 Driver Installation

All AGESA Drivers will be launched only if a PPI/Protocol that is required by that driver is published by the SOC driver. If this dependency is met along with its other dependencies, the UEFI phase dispatcher (PEI or DXE) will launch the driver. AGESA drivers may optionally use a “Hardware Identification Checks” to confirm that there is a match with the correct Hardware component(s). Driver(s) are dispatched and communicate with the platform BIOS to initialize the parameters. Each Driver/PEIM is designed to initialize a specific part of the SOC. The drivers were designed based on the following:

- A driver will not launch until its PPI/Protocol is published by the SOC driver and its optional “Hardware Identification” passes. If either identification fails, the driver will not be installed.
- Drivers are only allowed to use configuration parameters at POST time.
- Errors are logged and can be accessed via the “Error Log Driver” (see “Error Log Driver” sections for details)

2.2.1.3 Tools

The intent is for the “AGESA Package” and the “AGESA Drivers” to be compiled using standard EDK-II environment tools. At present the AGESA team is using Microsoft Visual Studio 2010 for compilation and validation.

2.3 Configuration Parameters

The AGESA™ V9 code set has many configuration items, called AGESA Parameters. These are provided so as to customize the operation for the specific platform. The full set of available configuration items are presented later in this specification. This section describes the method of how the configuration items values get processed in the build and POST times.

2.3.1 Overview

AGESA parameters can be set either by build-time options or by POST time code execution in the API. AGESA parameters can be modified via the API, but it is much faster and smaller code space to use the build time options.

If a platform wishes to expose some parameter value settings to the end user for shell level UI applet modification (e.g. SETUP fields), then the platform must provide separate storage for those UI control settings and implement POST time code to update the AGESA parameters via the API.

There is only one set of active AGESA parameters – those established during POST. These values are first populated by the build-time options and then allowed to be updated/modified dynamically by the platform via the API. Once the API call completes, no more changes can occur.

Each driver component of the package contributes PCD items to the list. These are categorized into four groups for easier location:

- Enablement Category - this category consists of items that control features - which are enabled, which are disabled.
- Platform Category - this category describes the platform environment. Items relating to physical characteristics of the platform and capabilities, such as layout, current capacity. Many of these items are required for the host BIOS to specify for proper operation of the system.
- Power Management Category - this category describes options related to power and thermal management of the system. This includes PState, CState and TDP limits; fan controls (temp thresholds and speed adjustments).
- Performance Category - This category is mostly optional; the defaults will support a working system. The items in this category are for fine tuning performance and operational parameters.

2.3.2 Usage Models

The parameters are flexible enough to support many platform usage models, from an isolated platform BIOS to a multi-purpose BIOS supporting a family of platforms.

2.3.3 Single Platform BIOS supporting a single platform

If a single platform BIOS supports only one platform, then no “AGESA Parameter Group” needs to be created. There is no need to provide the “AGESA Group Identification PPI/Protocol”. The platform BIOS can set the AGESA Parameters by directly referencing the AGESA PCDs.

There is a time component as to when the values may be changed. PCD values should be updated early, so that the new values are in place before the drivers reference those values.

2.3.4 Single platform BIOS supporting multiple platforms with same values

If a single platform BIOS is required to support multiple platforms with the same parameters, then each PCD can be provided with a different value by the platform BIOS build process. There is no need to provide the “AGESA Group Identification PPI/Protocol” or an “AGESA Parameter Group”.

2.3.5 Single platform BIOS supporting multiple platforms with different values

If a single platform BIOS is required to support multiple variations of the platform and one or more variations of the platforms require different parameter values to be defined at build, then the platform BIOS must provide multiple “AGESA Parameter Groups”. The AGESA will locate the “AGESA Group Identification PPI/Protocol”, so this PPI/Protocol must be installed. AGESA will use the “AGESA Group Identification” to determine the correct “AGESA Parameter Group”. Please see figure 7.

2.3.6 Multiple parameter value sets

The ‘normal’ for most platforms is to have one set of values for the AGESA parameters. For this use case no extra work is required for the platform. However, if the platform needs to support more than one configuration value set, then the platform will need to define multiple sets of AGESA parameters.

The build-time interface allows the platform BIOS to provide multiple value sets for the AGESA parameters. This can be useful for dealing with different board hardware variations in one BIOS implementation.

With this approach, a platform BIOS can create one BIOS image that can be used on different motherboards with different requirements (e.g. values) for the AGESA parameters.

To support this method, the platform BIOS is required to:

- Define separate build time option value sets for each variation and assign a unique label.
- Implement POST time code to identify and select which value set is to be used for this boot.

- Note: the platform BIOS may choose to save the value set choice into Non-Volatile storage (NVS) for use on later boots to compare for changes and/or improve boot times.
- The services used are dependent upon the environment. See the appropriate sections below.

2.3.7 UDK Configuration Parameters

This section will describe how to set the parameters within the UEFI-UDK environment. AGESA provides a build time interface that allows its configuration parameters to be configured using PCDs.

2.3.7.1 Design

If the platform BIOS needs to support multiple platforms with different parameter values at build-time, then one or more group of AGESA parameters can be created using groups called “AGESA Parameter Groups”. Each “AGESA Parameter Group” consists of configuration parameters and their corresponding values. Please see figure 2.

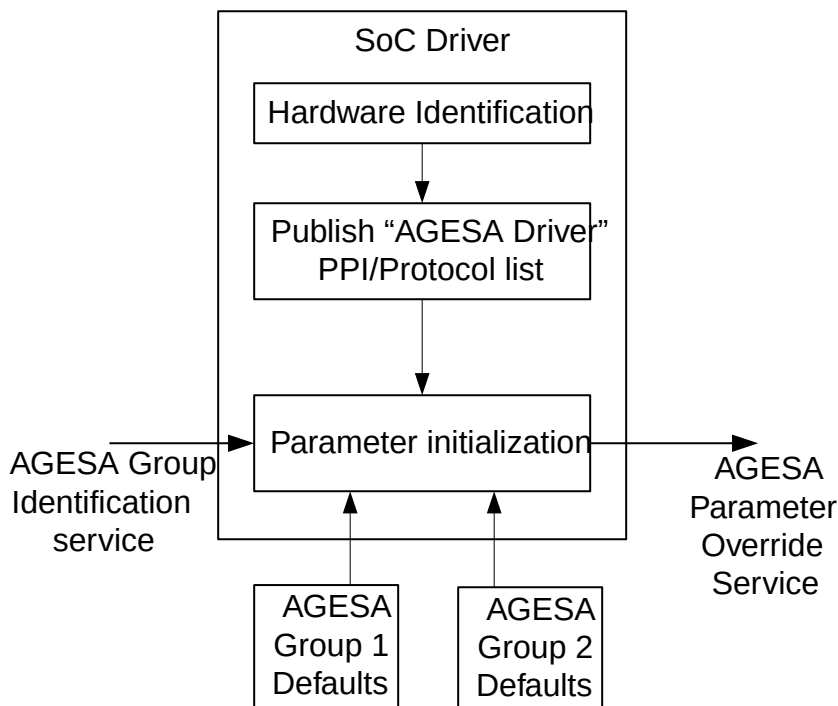


Figure 2. Setting the PCD values

2.3.7.2 AGESA Parameter Groups

The “AGESA Parameter Group” consists of multiple entries with configuration values. Each entry consists of a PCD GUID and a value. An “AGESA Parameter Group” can be used by one or more platforms. If there are more than one “AGESA Parameter Groups”, the platform BIOS must publish the “AGESA Group Identification PPI/Protocols” and provide a function that indicates which “AGESA Parameter Group” should be used by providing a unique “AGESA Group Identification” to each parameter group.

Table 2. Parameter Group

PCD GUID	Value
GUID 1	Value 1
GUID 2	Value 2
GUID 3	Value 3

Table 3. Multi Param Groups

Default Group 1		Default Group 2	
PCD GUID	Value	PCD GUID	Value
GUID 1	Value 1	GUID 1	Value 4
GUID 2	Value 2	GUID 2	Value 5
GUID 3	Value 3	GUID 3	Value 6

2.3.7.3 Boot Flow

1. At build-time, each “AGESA Parameter Group Entry” will be set with a unique GUID and value. At build-time, AGESA will define the PCD values for the AGESA parameters.
2. At build-time, the platform BIOS can specify platform specific values for the AGESA parameters by using the standard UEFI build process – PCD value re-assignment in the platform DSC files.
3. At run-time, UDK dispatcher dispatches the first AGESA driver - the SOC driver.
4. The “SOC Driver” checks its “Silicon ID” to confirm that it matches to the SOC.
5. The “SOC Driver” locates the “AGESA Group ID” PPI/Protocol which was published by the platform driver.
6. If the “AGESA Group ID” PPI/Protocol is not found, the “SOC Driver” will assume PCDs have been initialized at build-time.
7. If the “AGESA Group ID” PPI/Protocol is found, the “SOC Driver” will call a platform BIOS function to determine the “AGESA Group ID”.
8. The “SOC Driver” will use the “AGESA Group ID” to find the correct “AGESA Parameter Group”.
9. The “SOC Driver” reads the “AGESA Parameter Group” table entries and uses the “SetPcd” function to initialize all AGESA PCDs entries listed in the “AGESA Parameter Group”.
10. Once the PCDs are initialized, the “SOC Driver” publishes the list of specific PPI/Protocols to install the AGESA drivers.
11. The platform BIOS can register a notify callback to determine the SOC identification.
12. The “SOC Driver” publishes a PPI/Protocol indicating that the PCD values are ready to be modified by the platform BIOS.
13. The platform BIOS can register a notify callback to modify the AMD PCD values.
14. The platform BIOS driver modifies the values in the PCDs.
15. The SOC driver initialization completes.

2.3.7.4 Build-Time Support

At build-time the parameter values for each configuration can be modified by the platform BIOS.

AGESA configuration parameters are defined using PCDs, so their initial values will be specified by the AGESA modules and can be modified during the UDK build process by higher level modules; e.g. the platform module. This process uses the standard UEFI PCD process using definitions in the module .DEC and .DSC build control files.

If the platform BIOS needs to support multiple configuration parameter value sets, the platform BIOS will use the build time controls to establish “AGESA Parameter Group” value sets, one for each unique configurations.

The following tools will be used:

- This section is under construction.

2.3.7.5 Runtime Support

At runtime the values for each configuration parameter will be initialized from the build time values. Once the PCDs are set with final values, the Platform BIOS will have the opportunity to dynamically modify the values.

The Platform Driver can register a call-back to the dispatcher for activation when the SoC Driver publishes the “AGESA Parameter Override Ready PPI/Protocol”. At that time the Platform BIOS will get the call-back and can dynamically modify the configuration values.

2.4 Error Logging

The “Error Log Driver” communicates with the other AGESA component drivers and collects error information. When the log is full, or when all AGESA driver initialization is completed, a notification is sent to the platform BIOS that the log is ready for retrieval.

1. The “Error Log Driver” will provide an “Error Log Ready” service indicating when all AGESA errors have been collected.
2. The “Error Log Driver” will provide an “Error Log Full” service indicating that its internal log is full.

The “Error Log Driver” will install PPIs needed for:

1. The AGESA drivers to report errors, and
2. The Platform Driver to get notified about errors, and
 - a. When the error log is full, a “Error Log Full” PPI/Protocol is published.
 - b. A PPI will be published when all errors are collected.
3. The Platform Driver to read the error log Please see the PPI definitions in 3.1.

2.5 Debug

This section describes the debug features available to debug the AGESA drivers. Most of the debug controls are build time invocations only. The debug code will not be included into the build image if the debug options are disabled. Once included, POST time controls are available to enable/disable debug output by class.

The debug output is sent to IO port 80h by default.

Also note that the debug macros can be re-defined to direct the output to an alternate output path.

The types of debug features available are:

- General Flow Progression
 - Test Points (e.g. Port 80h codes)
- Sanity Checks
 - Parameter value checking via ASSERT()
- Process Tracing
 - Text string outputs from the core procedures

2.6 Version label

The AGESA™ code contains a version string that can be read from the driver binary at run-time. The version string is set as a 'Fixed-at-build' PCD by the name of `PcdAmdPackageString` and can be seen in the file: `AgesaModulePkg\AmdTools\ArchV9\Config\AgesaModulePkg.inc.dsc`. The version string is comprised of sections that provide the following information:

- Signature - this is a fixed string ("AGESA!V9") which can be used for searching the binary.
- separator - a null character (0x00).
- PI name string - a variant length string containing:
 - PI Name - a string identifying the PI release package (e.g. "RomePI", "RavenPI")
 - Socket Name - a string identifying the socket(s) supported by the PI release.
- separator - a 'space' character (0x20). This is not a hard rule and some releases may use a hyphen instead (0x2D).
- PI release number - a string containing the versioning number sequence. This is 4 numbers and/or letters separated by periods (e.g. "1.0.0.3"). Each digit position is one digit and uses the number-then-alpha sequence (0..9,A..Z).
 - position 1 - production release stream, 0 means pre-production.
 - position 2 - variant stream. This goes to >0 after the first production release. Typically means there are new features or processor support.
 - position 3 - Development state. Typical values are 3 = 1st prototype; 5 = all features enabled; 7 = pre-production; 9 = DVT testing ready.
 - position 4 - sequence in the stream, higher numbers are newer.
- terminator - null (0x00), standard string terminator.

Example:

The PCD declaration:

```
gEfiAmdAgesaModulePkgTokenSpaceGuid.PcdAmdPackageString|"AGESA!V9\0RenoirPI-FP6 0.0.5.0"
```

After the build will be converted to below structure and embedded as part of `AgesaModulePkg\Universal\Version\AmdVersionPei\AmdVersionPei` driver:

```
GLOBAL_REMOVE_IF_UNREFERENCED const UINT8 _gPcd_FixedAtBuild_PcdAmdPackageString[31] =
{65, 71, 69, 83, 65, 33, 86, 57, 0,
 82, 101, 110, 111, 105, 114, 80, 73, 45, 70, 80, 54,
 32, 48, 46, 48, 46, 53, 46, 48, 0};
```

2.7 Porting Guide & Topics

This section covers several topics related to porting the AGESA™ packages for your platform and how to integrate AGESA into the development environment. The AGESA™ package is designed to fit into the UEFI-UDK environment and build tools.

2.7.1 Porting The Development Environment

The Platform BIOS must integrate components of AGESA at build-time, during the build process and at run time.

2.7.1.1 Build-Time Support

The platform BIOS will specify values for the build-time parameters and perform runtime modifications of any AGESA Configuration parameters as described earlier in 2.5 Configuration Parameters.

Interface definition files (includes) are located in the release package under "**AgesaModulePkg\Include**". The various models of processor may need different include files as indicated in this table:

Processor	files to include
F17M00	FchRegistersTS.h
F17M10	FchRegistersSS.h
F17M30	FchRegistersHS.h
F15M60, F17M70	FchRegistersKN.h
(common to all)	FchRegistersCommon.h

The Platform porting must include:

1. The platform BIOS copies the AGESA parameter PCD names and sets platform specific values in the platform module .DSC build control file.
2. If the platform requires multiple parameter value sets, then
 - a. the platform module must define the various configuration sets and assign a unique identifier.
 - b. The platform module must provide the needed call-back routines and code to register those call-backs to the dispatcher.
3. Customize AGESA V9 Library instances. Example files are provided in the directory: AmdCpmPkg\Addendum\Oem\<PlatformName>. The host BIOS must provide customized versions of those libraries that implement the platform specific call-out functions required by the AGESA™ package.
 - Customize AgesaCcxPlatformLib to implement:
 - SaveApInitVector
 - RestoreContentVector
 - See: "AmdCpmPkg\Addendum\Oem\%board%\AgesaCcxPlatformLib"
 - The Platform porting engineer must implement a new library instance based on "AmdPspFlashAccLib" for the PSP DXE SMM driver to use. References:
 - The description can be found in: "AgesaModulePkg\Include\Library\AmdPspFlashAccLib.h".
 - Reference code can be found in: "AgesaPkg\Addendum\Psp\AmdPspFlashAccLibSmm\AmdPspFlashAccLibSmm.c"
 - (for Family 15h only) Customize AmdCallout lib to implement:
 - AgesaHookBeforeDramInit
 - AgesaReadSpd
 - AgesaHookBeforeExitSelfRefresh
 - AgesaSaveVolatileS3Info
 - AgesaSaveNonVolatileS3Info
 - See: "AmdCpmPkg\Addendum\Oem\%board%\Library\AmdCalloutLib"
 - Customize FchInitHookLib to implement:
 - FchPlatformPTPeiInit

- FchPlatformOemPeiInit
 - FchPlatformImcDxeInit
 - FchPlatformPTDxeInit
 - FchPlatformOemDxeInit
 - See: “AmdCpmPkg\Addendum\Oem\%board%\Library\FchInitHookLib”
4. Implement the PPIs and Protocols required by the AGESA™ package
 - (for Family 15h only) Implement and make public gAmdMemoryPlatformConfigurationPpiGuid to pass in the memory configuration table. See: “AmdCpmPkg\Addendum\Oem\%board%\Pei\PlatformMemoryConfigurationPei”.
 - Implement and make public gPspPlatformProtocolGuid which used for secure S3: Link to “AmdCpmPkg\Addendum\Oem\%board%\Dxe\PspPlatformDriver”.
 - Implement and make public gAmdCpmTablePpiGuid to provide Pci^(TM) Topology information (PeimPublicFunction.PcieComplexDescriptorPtr). Link to “AmdCpmPkg\Addendum\Oem\%board%\Pei\AmdCpmOemInitPei”.
 5. Override AGESA PCDs, as needed, to customize platform specific settings
 - Customize AGESA PCDs at build time: See: “AmdCpmPkg\Addendum\Oem\%board%\ AmdCpm\%board%\Pkg.dsc”.
 - Customize AGESA PCDs at runtime: See: “AmdCpmPkg\Addendum\Oem\%board%\Pei\AmdAgesaParameterGroupPei”.
 - Note that the FCH code enables all GPP and GFX Clocks and is then expecting the customer platform BIOS developer to work with their respective hardware to see which GPP and GFX clocks are not used on their respective platform. The platform developer then needs to set the FCH PCDs properly to turn off unused clocks.
 6. Update Platform driver INF files to include needed dependencies
 - Update Platform CpuMpServices Driver INF file to include a depex on “gAmdMpServicesPreReqProtocolGuid”. See section [DXE_AMD_CCX_MP_SERVICES_PREREQ_PROTOCOL](#).
 7. Use the AGESA tools to prepare for the build process:
 - Generate the build control files (see [ArchV9](#)), and
 - Check for version matching (see [ABLCheck](#)).

2.7.1.2 POST Time Support

Actions needed to be performed by the platform module include:

- Install the PSP platform protocol gPspPlatformProtocolGuid. This provides platform customizations information to the PSP. Sample code is in: AmdCpmPkg\Addendum\Oem\%board%\Dxe\PspPlatformDriver.
- For an S3 resume path, host BIOS code must issue a Software SMI for the FCH.
- The host BIOS must reserve the PEI FV in the memory area as Runtime Reserved type, so that the OS does not attempt to re-use the region. The PSP will cause the x86 cores to resume execution in this region.

2.7.1.3 Development Check List

The following is a checklist for the steps required to support AGESA V9 in the platform BIOS:

- Establish an EDK-II build environment; AGESA V9 only supports EDK-II.
- Add support for new build-time interface (single or multiple platforms).
- Add support for new run-time interface with new data structures.
- Platform Drivers need to support dispatched AGESA drivers.

- Support service PPIs/Protocols.
- Make use of SOC Identification PPIs/Protocol.
- Support AGESA Callouts Protocols/PPIs Services.
- Platform BIOS will need to add support for AGESA Notifications.
- Platform BIOS will need to meet AGESA external dependencies.

The following items presume a familiarity with the EDK-II environment. Some terms are specific to that environment.

- Use AgesaPkg/Include/AmdUefiStack.inc macro `AMD_ENABLE_UEFI_STACK2` to initialize temporary stack and heap space for use by PEI (see section for [AMD_ENABLE_UEFI_STACK2](#)). For example, in EDKII this macro could be called in PlatformSecLib: `_ModuleEntryPoint ()` called by UefiCpuPkg/SecCore.
- Use AgesaPkg/Include/AmdUefiStack.inc macro `AMD_ENABLE_UEFI_STACK2` to release temporary stack and heap space used by PEI (see section for [AMD_DISABLE_UEFI_STACK2](#)). For example, in EDKII this macro could be called in PlatformSecLib: `SecPlatformDisableTemporaryMemory ()` called by UefiCpuPkg/SecCore.
- UEFI code which produces the Resource Descriptor HOBs for the RAM resources should use the resources described by `gAmdMemoryInfoHobGuid` HOB (see section for [Memory Driver](#)) to produce the required HOBs.
- SPI ROM part, Firmware Volume Block, support is not provided as part of the AGESA PI.
- The Core Complex Driver (see section for [Core Complex Driver](#)) must be run after the platform code installs the ACPI processor records because it will produce an SSDT that scopes the processor objects. This will be the DXE driver for your architecture under `AgesaModulePkg/Ccx/`.
- The Driver which produces `EFI_MP_SERVICES_PROTOCOL` must have a Depex dependency expression added for `gAmdMpServicesPreReqProtocolGuid`. This will allow AGESA to start all cores and place them in a state for the `EFI_MP_SERVICES_PROTOCOL` to utilize them.
- AGESA does not have a driver which produces the `EFI_SMM_ACCESS2_PROTOCOL`. This needs to be developed for the platform SMM implementation.
- AGESA does not provide the SMM entry code which would be responsible for transitioning the processor from 16-bit SMM mode to 64-bit long mode and executing the UEFI SMM core. This needs to be developed for the platform SMM implementation.
- The processor requires the x86 cores to locate their Reset Vector in RAM. A patch update for the EDKII BaseTools has been incorporated to support this requirement.
 - The EDKII BaseTools GenFds build tool must include change “adb6ac256338 BaseTools/GenFv: Account for rebase of FV section containing VTF file”.
 - The platform FDF section which describes the firmware volume that will be copied to RAM by PSP must include the following `FvAlignment` parameters:
 - `FvBaseAddress`: Must equal the base address of the firmware volume used to relocate the XIP PEI drivers.
 - `FvForceRebase`: When set to TRUE, indicates to the BaseTools that the reset vector will be relocated based on the `FvBaseAddress` and the size of the firmware volume.
 - Customer needs to evaluate how the reset vector being in RAM could affect the layout of the BIOS image, modifications required for the SEC phase, and possible modifications required for the PEI phase of UEFI.

2.7.1.4 Upgrade Check List

The following is a list of changes that are required if the platform BIOS is upgrading from “AGESA Architecture 2008” (aka AGESA V5) to AGESA V9.

- **General Modifications**

The following items need to be removed or changed:

- All drivers sourced from the Architecture 2008 AGESA Package (“AmdAgesaPkg”) should be removed.
- Any platform library files that support “AmdAgesaPkg” should be removed.
- Support for “Unified Boot” needs to be added.
- All PPIs/Protocols/Guids referenced by Arch 2008 need to be removed.
- **Platform DSC file**

The following changes need to be made:

- Include AgesaModulePkg/Agesa<socket><program>ModulePkg.inc.dsc
- Optional: Include DebugLib|AgesaModulePkg/Library/DebugLibIdsDp/DebugLibIdsDp.inf is you wish to use the AMD implemented library, else you may use the standard EDK-II debuglib.
- **Platform FDF file**

The following changes need to be update in the “Platform FDF” file:

- PEI FV - Include AgesaModulePkg/Agesa<socket><program>ModulePkg.pei.inc.fdf
- DXE FV - Include AgesaModulePkg/Agesa<socket><program>ModulePkg.dxe.inc.fdf
- **Inf file Changes**

The following changes need to be updated in the any platform inf files:

- Packages
 - Remove any references to “AmdAgesaPkg/UefiUdk/Psp/PspxxxPkg/PspPkg.dec”.
 - Remove any references to “AmdAgesaPkg/AmdAgesaPkg.dec”.
 - Add reference to “AgesaModulePkg/Agesa<socket><program>ModulePkg.dec”.
- LibraryClasses
 - Remove references to “PspBaseLib”.
 - Add references to “AmdPspBaseLibV2”.
 - Remove references to “FchDxeLib”.
 - Add references to “FchDxeLibV9”.
- Dependency Expressions
 - Replace any dependencies on to gAmdPeiInitEarlyPpiGuid with gAmdCcxPeiInitCompletePpiGuid.
 - Replace any dependencies on to gAgesaMemPpiGuid with gAmdMemoryInitCompletePpiGuid.
 - Replace any dependencies on to gAmdPeiInitPostPpiGuid with gAmdMemoryInitCompletePpiGuid.
- **“C” and “H” Changes**

Any “C” or “H” files that reference AGESA header or “inc” files need to be modified:

- Replace references to “PspBaseLib.h” with “AmdPspBaseLibV2.h”.
- **“asm” Changes**

Any “asm” files that reference AGESA header need to be modified:

- Replace references to “uefimem.inc” with “AmdUefiStack.inc”.
- **AMD AGESA PSP Firmware stack Changes**

Any references to “AmdAgesaPkg/Firmwares/” should be changed to “AgesaModulePkg/Firmwares/xxx”
Where 'xxx' is the label for the product name or abbreviation.

2.7.1.5 Combination Builds

It is possible, and quite common, to build a BIOS that supports multiple SoCs at the same time. These are referred to as 'Combo builds'. Differences in the SoC architecture dictate how the firmware volume (FV) is constructed. The two architectures differ in their start-up process:

- **Classic** - The SoC initializes memory using the x86 core(s). Code starts execution at the classic reset vector (0xFFFF_FFF0).
- **Secured** - The SoC security processor initializes memory prior to releasing the x86 cores for execution. Code execution for the x86 cores begins at an address set by the security processor.

The UEFI code for the SEC and PEI phases is typically built for execute-in-place (XIP) operation, meaning the code modules have their relocation information removed during the build process and they are built to execute from a fixed address; but the secured model has a different address than the classic model. One solution is to build 2 PEI FVs with the same contents but designated to start with different base addresses at build time. A more optimized solution could pick up the specific PEIM modules in 2 different PEI FVs.

Below is an example in Project.fdf file, note the PEIM modules could be the same in [FV.RECOVERYFV] and [FV.RECOVERYFV2] as AGESA V9 supports different SOC drivers, or could be family specific to reduce BIOS image size.

```
[FD.%board%Pei]
...
$(FLASH_REGION_FV_RECOVERY_DRAM_OFFSET)|$(FLASH_REGION_FV_RECOVERY_DRAM_SIZE)
FV = RECOVERYFV2
[FD.%board%]
...
$(FLASH_REGION_FVMAIN_OFFSET)|$(FLASH_REGION_FVMAIN_SIZE)
FV = FVMAIN_COMPACT
$(FLASH_REGION_FV_RECOVERY_OFFSET)|$(FLASH_REGION_FV_RECOVERY_SIZE)
FV = RECOVERYFV
[FV.RECOVERYFV]
## List the PEIM modules for Recovery FV
[FV.RECOVERYFV2]
## List the PEIM modules for Recover FV2
```

2.7.2 Porting Capsule Support

'Capsule update' is the UEFI defined process for updating the BIOS flash ROM image in a secure manner. See section 7.5.3 Update Capsule of the UEFI specification. The process defines how data can be passed from a runtime (operating system [OS]–present) application back to the preboot phases (Pre-EFI Initialization [PEI], Driver Execution Environment [DXE], and Boot Device Selection [BDS]) for the purpose of altering the secure content of the flash ROM.

The update image is stored in DRAM; so, the key to make the process functional is to keep the DRAM content persist through the whole process and securely transferring control to the system BIOS for the Flash update. There are several way to keep the DRAM content persistent. The reliable, robust method is to make use of the ACPI S3 process. By sending the system into S3, using the memory retention capability of the S3 state, the capsule content can be preserved. The system is sent to S3 and woken up by the RTC timer or other ACPI event giving control to the system BIOS. In this section the details of the AGESA implementation are described.

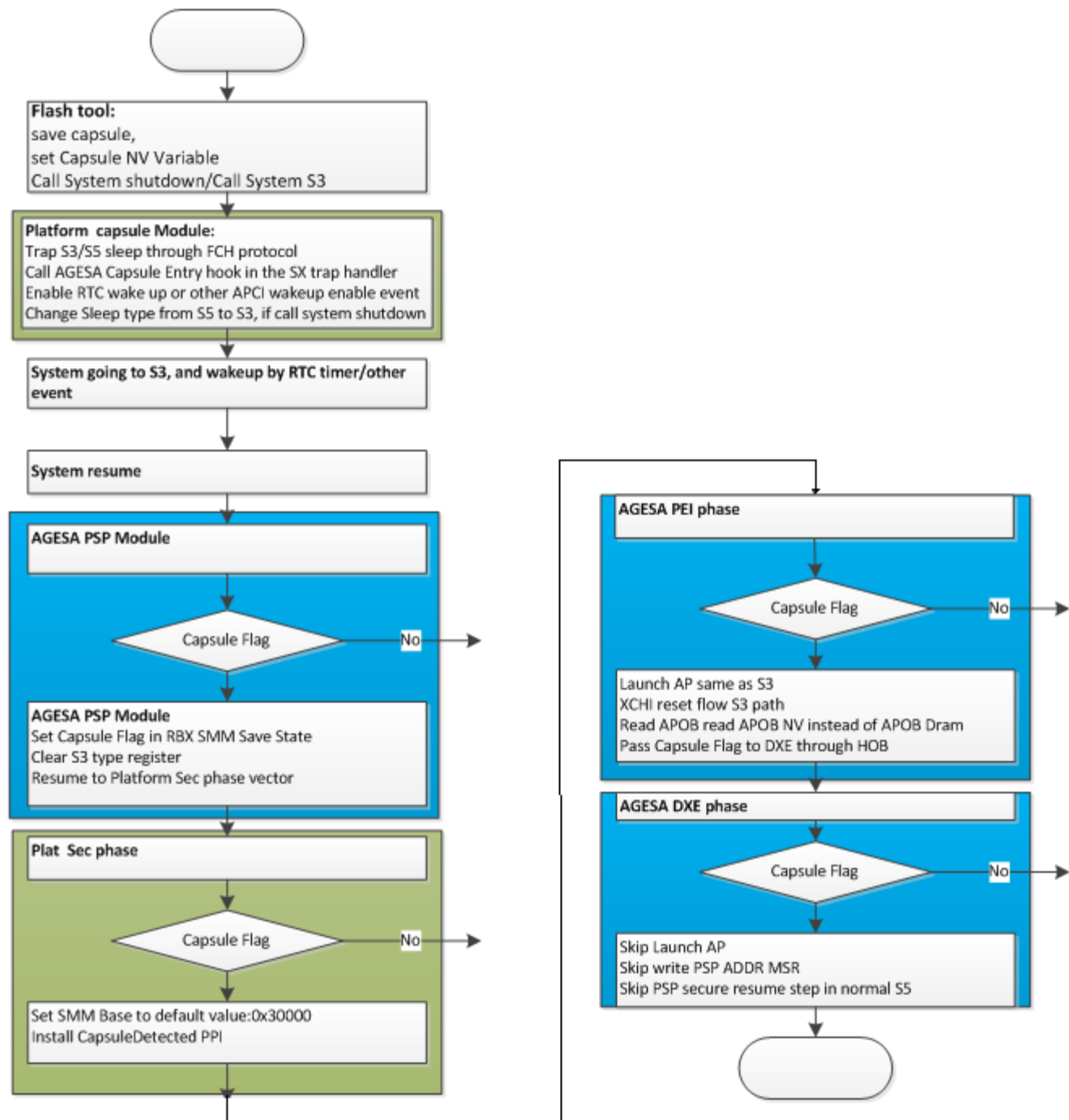


Figure 3. Flow Diagram for Capsule operation

Variances may exist by processor family.

The details for the steps in the general flow are:

1. During the S5 POST, system BIOS must register a sleep trap callback for processing the capsule.
2. In OS/Shell/DOS, the flash application saves the capsule data to DRAM.
3. Flash application calls system shutdown (S5), or S3 directly. Note, please keep SMI enable, or else it will impact below Sleep SMI trap.
4. The application's write to the sleep type register will be trapped, and the corresponding sleep SMI will be issued.

5. The AGESA FCH "FchSmmSxDispatchHandler" will start and dispatch all registered handlers, including the callback that was registered in step 1.
6. The callback routine starts execution and sets the sleep type to S3, enables the appropriate S3 wake up source and exits, waiting for the wake event.
7. The Sleep SMI completes, sending the system into the S3 state.
8. System wakes up due to the preset wake resource event.
9. The x86 starts running from the BspSmmResumeVector. It checks if the global variable is set indicating this wake is due to the capsule update process. If the capsule flag is set, RBX is set to 0xCA9501EF and the sleep type register is cleared. This causes the boot process to follow the S5 POST path instead of the S3/S5 resume path.
10. In SBIOS security phase, the system BIOS must detect capsule process is in progress by checking RBX for a value equal to 0xCA9501EF and redirect POST to follow the capsule path.
11. AGESA provides the Lib class AmdCapsuleLib in order to query capsule status.
12. AGESA does appropriate sequence by using AmdCapsuleGetStatus provided by AmdCapsuleLib.
13. (For Family 15h programs only) the system BIOS must set some AGESA parameters prior to the PEI driver starting:
 - PcdAmdMemCfgNvStorage - should be set as the address where the system set the AGESA store of memory S3 NV data.
 - PcdAmdMemCfgNvStorageSize - should be set to the size of the AGESA memory S3 NV data space.
 - PcdAmdMemCfgMemRestoreCtl - should be set to TRUE.
14. System continues to boot from PEI, to DXE, to BDS...
15. Update application processes the Capsule update per the defined UEFI process.

Further details of actions the platform BIOS must implement to enable this sequence:

- In step 1. during the S5 POST, system BIOS must register a sleep trap callback (CapsuleSxCallBackHandler) for processing the capsule. This handler is authored by the platform BIOS. Registration is done via the SMM Protocol " gEfiSmmXXXDispatch2ProtocolGuid.Register".

Example code for registering the callback:

```

Status = gSmst->SmmLocateProtocol (
    & gEfiSmmXXXDispatch2ProtocolGuid ,
    NULL,
    &SxDispatch
);
if (EFI_ERROR (Status)) {
    return Status;
}
SxRegisterContext.Type = SxS3;
SxRegisterContext.Phase = SxEntry;
SxRegisterContext.Order = 1;
// Register with Smm driver to intercept Sx transition and inform Psp
// via mailbox in that transition
IDS_HDT_CONSOLE_PSP_TRACE ("\tRegister Sleep SMI\n");
Status = SxDispatch->Register (
    SxDispatch,
    PspSxCallback,
    &SxRegisterContext,
    &SxHandle
);
if (EFI_ERROR (Status)) {
    return Status;
}

```

- In step 6, The CapsuleSxCallBackHandler starts execution. It must
 1. Locate & execute the AGESA published SMM protocol (gAmdCapsuleSmmHookProtocolGuid), and set the AmdCapsuleHookFlag.

- If the SMM Lock has been enabled, then set a the global variable AmdCapsuleHookFlag to value PSP_CAPSULE_HOOK_FLAG_SXINFO_CLRSMMMLCK.
 - Else, set a the global variable AmdCapsuleHookFlag to value PSP_CAPSULE_HOOK_FLAG_SXINFO_NOCLRSMMMLCK.
2. Enable the appropriate S3 wake up source to used for capsule.
 3. Set the sleep type to S3.
 4. Return and wait for the wake up event.
- In step 11, During the UEFI SEC phase, the system BIOS must detect that the capsule process is in process by checking RBX for a value equal to 0xCA9501EF. If the capsule process is indicated, then system BIOS must:
 1. Set SMMBase to default value.
 2. Install gCapsuleUpdateDetectedPpiGuid PPI (defined by UEFI spec).
 3. Call the function AmdCapsuleGetStatus - defined in AmdCapsuleLib.h.

3 UEFI UDK API

This section describes the API's for the UEFI-UDK environment. Each component driver will have three potential UEFI phases to execute within and therefore may have three separate name variations, one for each phase:

- PEI - “Agesa[Name]Peim”
- DXE – “Agesa[Name]DxeDriver”
- SMM – “Agesa[Name]SmmDriver”

Each will have its own PPI's or Protocols. These are defined in the next sections.

Package Configuration items

There are a few configuration items that are defined in the package but do not belong to a specific driver. These may be read by the host BIOS for informational purposes.

PcdAmdPackageString This is a string value indicating the release level of this AGESA™ package.

Example version string: "AGESA!V9\0SummitPI-AM4 0.0.5.0"

3.1 FCH Drivers Module

This driver module covers devices in the IOHUB and FCH integrated devices. Many of the API elements used to communicate with these devices are UEFI standards specified in the UEFI PI Specification, see reference: UEFI PI Specification. The following UEFI standard PPIs and Protocols are published by the FCH driver module:

PEI PPIs

- EFI_PEI_SMBUS2_PPI
- EFI_PEI_STALL_PPI
- EFI_PEI_RESET_PPI

DXE Protocols

- EFI_RESET_ARCH_PROTOCOL
- EFI_SMM_CONTROL2_PROTOCOL
- EFI_SMBUS_HC_PROTOCOL

SMM Protocols

- EFI_SMM_SW_DISPATCH2_PROTOCOL
- EFI_SMM_SX_DISPATCH2_PROTOCOL
- EFI_SMM_PERIODIC_TIMER_DISPATCH2_PROTOCOL
- EFI_SMM_USB_DISPATCH2_PROTOCOL
- EFI_SMM_GPI_DISPATCH2_PROTOCOL
- EFI_SMM_POWER_BUTTON_DISPATCH2_PROTOCOL
- EFI_SMM_IO_TRAP_DISPATCH2_PROTOCOL

The proprietary PPIs and Protocols published by the FCH driver are described in the next sections.

Local Terminology

Term	Title	Version	Data Rate
LS	Low Speed	1.x	1.5Mbps
FS	Full Speed	1.x	12Mbps
HS	High Speed	2.0	480Mbps
SS	Super Speed	3.0	5Gbps
SSP	Super Speed Plus	3.1	10Gbps

3.1.1 USB Port Mapping

The FCH Driver is responsible for initializing the USB controllers and ports regardless of where they are located in the chip architecture. Depending upon the Family/Model, USB ports may be located in:

- FCH USB - lanes located in the FCH IP block.
- IOHub - lanes located in the IO Hub IP block within a processor die.
- IOD - lanes located in a separate IO die on an MCM processor.

For the purpose of this software specification, all of the ports made available by the processor(s) in the system are numbered sequentially starting from Socket-0, Die-0 through Socket-N, Die-N; giving them a 'logical' port number. This 'logical' port number is used to index the various controls defined within the AGESA™ package. The following table is presented to relate logical port to the physical IP block providing those ports; however, the final reference should be the Processor Programming Reference and Functional Data Sheet for the processor under consideration.

Table 4. USB Port Mapping

FM	Pkg	Port0	Port1	Port2	Port3	Port4	Port5	Port6	Port7
F15M70	AM4	'EHCI' (USB2.x)				'xHCI' (USB3.x)			
F17M00	AM4	S0, D0,C0 - 'Ctrl0'							
	SP3	S0,D0,C0 - 'Ctrl0'		S0,D1,C0 - 'Ctrl1'		S1,D0,C0 - 'Ctrl2'		S1,D1,C0 - 'Ctrl3'	
	SP4	S0,D0,C0 - 'Ctrl0'		S0,D0,C0 - 'Ctrl1'					
	SP3r2	S0,D0,C0 - 'Ctrl0'				S1,D0,C0 - 'Ctrl1'			
F17M30	SP3	S0-IOD,C0 - 'Ctrl0'		S0-IOD,C1 - 'Ctrl1'		S1-IOD,C0 - 'Ctrl2'		S1-IOD,C1 - 'Ctrl3'	
F17M10	FP5	Container0 - 'Ctrl0'				Container1 - 'Ctrl1'			
F17M20	FP5	Controller0							

Note: syntax within the table: 'Sn,Dn,Cn' - address of the controller IP block by Socket, Die, Controller.

'CtrlN' - Software's reference name or controller index.

Some rules about USB ports:

- Reader should be familiar with the USB standards specification.
- Any port providing USB 3.x level service must also provide USB 2.0 service for backward compatibility.
- Any port can be disabled by turning off its USB3.x and USB 2.x capability, Or
- a group of ports can be disabled by turning off their controller.

- When a port or group of ports are disabled, that DOES NOT affect the logical port number of the remaining ports.

When reading the Processor Programming Reference, some helpful definitions:

Controller This is the core block with which the OS driver will interact to operate the USB port.

PHY This includes the analog lane driver and signal quality control.

Container This includes the port power management controls and services.

3.1.2 FCH Configuration Items

These are the configuration controls (PCDs) for the FCH driver.

3.1.2.1 FCH-Enablement Configuration Items

Content: 'Y' is supported, '-' is NOT supported, '?' support is unknown.

Table 5. FCH-Enablement Feature Availability

Feature				F19M50 CezannePI-FP6	F19M00 MilanPI-SP3	F17M90 VanGoghPI_FF3	F17M60 ComboAM4PI-v2	F17M60 RenoirPI-FP6	F17M30 RomePI-SP3	F17M30 Embedded RomePI-SP3	F17M30 CastlePeakPI-SP3r3	F17M20 EmbeddedPI-FP5	F17M20 PollockPI_FT5	F17M10 PicassoPI-FP5	F17M10 ComboAM4PI	F17M10 RavenPI-AM4-FP5	F17M00 PinnaclePI-AM4	F17M00 SnowyOwlPI-SP4	F17M00 NaplesPI-SP3	F17M00 SummitPI-AM4
SataEnable				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
SataEnable2				Y	Y	Y	Y	Y	Y	Y	Y	-	-	-	-	-	-	-	-	-
SataRasSupport				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	-	-	Y
HpetEnable				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Xhci0Enable				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
Usb3PortDisable				Y	-	Y	Y	Y	-	-	Y	-	Y	Y	Y	Y	Y	-	-	-
Usb2PortDisable				Y	-	Y	Y	Y	-	-	Y	-	Y	Y	Y	Y	Y	-	-	-
XhciOcPinSelect				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	-	-	-
XhciOcPolarity				Y	Y	-	Y	-	Y	Y	Y	-	-	-	-	-	Y	-	-	-
XhciECCDedErrRpt				-	Y	-	-	-	Y	Y	Y	-	-	-	-	-	Y	-	-	Y
SdConfig				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
SpreadSpectrum				Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y
DisableAsfSlave				-	Y	-	-	-	Y	Y	-	-	-	-	-	-	-	-	-	-
BootTimerEnable				-	Y	-	-	-	Y	Y	-	-	-	-	-	-	-	-	-	-
EspiOperatingFreq				Y	-	Y	Y	Y	-	-	-	-	Y	Y	Y	-	-	-	-	-
EspiIoMode				Y	-	Y	Y	Y	-	-	-	-	Y	Y	Y	-	-	-	-	-
EspiIoMmioDecode				Y	-	Y	Y	Y	-	-	-	-	Y	Y	Y	-	-	-	-	-

- PcdSataEnable [F17M00][F17M10][F17M20]**

Select whether or not the FCH Sata controller is active.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdSataEnable2 [F17M30][F19M00]**

There is one FCH per socket for these models; so, this control is used to select which FCH SATA controllers are active. there are 4 controllers per socket. This control is a bit-mapped list, each bit represents a controller; 0=disabled, 1=active.

Permitted Choices: (Type: Value)(Default: 0xFF)

- Bits 0-3: Socket 0, SATA controllers 0-3.
- Bits 4-7: Socket 1, SATA controllers 0-3.

- **PcdSataRasSupport [F19M00]**

Select whether or not the FCH Sata RAS feature is active.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdHpetEnable**

Select whether or not the ACPI Timer Device is active.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdXhci0Enable,**

PcdXhci1Enable,

PcdXhci2Enable,

PcdXhci3Enable

This group of controls select which XHCI controllers are active in the system. Note that different processor models have controllers on socket 1, please see [port mapping](#).

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - The controller on is active.
- FALSE - This controller is turned off.

- **PcdXhciUsb3PortDisable**

This item allow the host BIOS to select which USB ports will be active with USB 3.x capability in the system. Each bit in the value controls one USB port: bit0 for Port0, bit1 for Port1, etc.

Permitted Choices: (Type: Value)(Default: 0x00)

- bit n = 0 - the corresponding port has USB3.x capability Enabled.
- bit n = 1 - the corresponding port does not support USB3.x.

Refer to the lead-in chapter for [USB port mapping](#) details.

• **PcdXhciUsb2PortDisable**

This item allows the host BIOS to select which USB ports will be active with USB 2.0 capability in the system. Each bit in the value controls one USB port: bit0 for Port0, bit1 for Port1, etc.

Permitted Choices: (Type: Value)(Default: 0x00)

- bit n = 0 - the corresponding port has USB 2.0 capability Enabled.
- bit n = 1 - the corresponding port is Disabled.

Refer to the lead-in chapter for [USB port mapping](#) details.

• **PcdXhciOcPinSelect**

This item allows the customer to map the OC Pins to USB Ports. Each 4-bit field selects an OC pin (pin name "OCn", n=value in field). Each position in the array of 4-bit fields represents one USB port. Example: if bit[3:0] = 0, OC Pin0 will map to USB port0.

if bit [15:12] = 0x02, OC Pin2 will map to USB port3.

Permitted Choices: (Type: array of bit-packed Values)(Default: 0x00000000)

Table 6. OC Pin mapping

Bits	Port0 [3:0]	Port1 [7:4]	Port2 [11:8]	Port3 [15:12]	Port4 [19:16]	Port5 [23:20]	Port6 [27:24]	Port7 [31:28]
F17M00, F17M30	OCpinN	OCpinN	OCpinN	OCpinN	OCpinN	OCpinN	OCpinN	OCpinN
F17M10	OCpinN	OCpinN	OCpinN	OCpinN	OCpinN	OCpinN	N/A	N/A

Refer to the lead-in chapter for [USB port mapping](#) details and to the Product Processor Programming Reference and Socket Functional Data Sheet for the quantity of OC pins available.

• **PcdXhciOcPolarityCfgLow**

This item allows the customer to change OC signal polarity to be active low. This is universal and applies to all OC pins.

Permitted Choices: (Type: Boolean)(Default: FALSE)

At present, this control applies to Family17h Model 00h-2Fh.

- TRUE - OC pin is low when OC occurs.
- FALSE - OC pin is high when OC occurs.

• **PcdXhciECCDedErrRptEn**

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE - USB ecc SMI Disable.
- TRUE - USB ecc SMI Enable.

• **PcdSdConfig**

Selects the operational mode for the Secure Digital controller.

Permitted Choices: (Type: List)(Default: Disabled)

- Disabled
- SD 2.0 mode

- **PcdSpreadSpectrum [F17M30]**

This control enables the EMI control for Spread Spectrum feature for the clock source.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdAmdFchDisableAsfSlave [F17M00][F17M30][F19M00]**

This control will disable the ASF Slave device. This will prevent any contention for the bus with other devices. At this time, it is recommended that this control remain =TRUE.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - The Slave device is turned off.
- FALSE - the Slave device is active.

- **PcdBootTimerEnable [F17M00][F17M30][F19M00]**

Enable/Disable Boot Timer function in post time.

Permitted Values: (Type: Boolean)(Default: TRUE)

- False - Disable the boot timer.
- True - Boot Timer is used.

eSPI Enablement [F17M10][F17M18][F17M20]

- **PcdEspioOperatingFreq**

This control specifies a board restriction on the top frequency the eSPI channel should use.

Permitted Choices: (Type: Value)(Default: 0x0F)

- 0x00 - 16.7MHz is the top limit used on the board.
- 0x01 - 33MHz is the limit.
- 0x02 - 66MHz is the max the board can use.
- 0x0F - Auto. The FCH firmware will evaluate the host and device capability and choose the fastest common frequency.

- **PcdEspioIoMode**

This control specifies a board restriction on the transfer mode on the eSPI channel should use.

Permitted Choices: (Type: Value)(Default: 0x0F)

- 0x00 - Single IO mode is the top limit used on the board.
- 0x01 - Dual IO mode is the limit.
- 0x02 - Quad mode is the max the board can use.

- 0x0F - Auto. The FCH firmware will evaluate the host and device capability and choose the fastest common mode.

- **PcdEspiloMmioDecode**

The eSPI controller is capable decoding up to 4 IO ranges and upto 4 MMIO ranges for the device(s) present on the eSPI bus. The values used to select these ranges is device dependent. The ranges are specified in a PCD structure that has an effective structure is as follows:

```

struct {
    struct {
        UINT8      IoRangeEnable;
        UINT16     IoRangeBase;
        UINT8      IoRangeSize;
    } eSPI_IO_Ranges[3];
    struct {
        UINT8      MmioRangeEnable;
        UINT32     MmioRangeBase;
        UINT16     MmioRangeSize;
    } eSPI_MMIO_Ranges[3];
} PcdEspIoMmioDecode;

```

IoRangeEnable

Indicates if the range is active.

MmioRangeEnable

Permitted Choices: (Type: Boolean)(Default: False)

- False - range is not active.
- True - accesses to this range will be forwarded to the eSPI device.

IoRangeBase

This is the starting address on the IO bus for the decode range.

Permitted Choices: (Type: Value (Default: 0x0000))

IoRangeSize

This is the size, in bytes, of the range. Decode range is from Base to (Base+Size).

Permitted Choices: (Type: Value (Default: 0x0000))

MmioRangeBase

This is the starting address on the memory bus for the decode range.

Permitted Choices: (Type: Value (Default: 0x00000000))

MmioRangeSize

This is the size, in bytes, of the range. Decode range is from Base to (Base+Size).

Permitted Choices: (Type: Value (Default: 0x0000))

The PCD declaration is a byte packed array. For example:

[illegible]

Formatting rules for the PCD files force the above definition. For illustration, an equivalent C structure style declaration is provided below.

```
PcdEspIoMmioDecode =
{#eSPI decode ranges
    # 1,  ENABLE      - ESPI IO Decode Range
    # 2,3,BASE        - ESPI IO Decode Range base - support 4Ch/4Eh/4Fh 62h/66h 600h-6FFh
    # 4,  SIZE        - ESPI IO Decode Range size - #bytes-1
    { # IO decode ranges
        {0x01, {0x4C, 0x03}, 0x00}, # Enabled, base=4C:4F, size=3
        {0x01, {0x00, 0x06}, 0xFF}, # Enabled, base=0600:06FF, size=FF
        {0x00, {0x00, 0x00}, 0x00}, # Disabled
    }
```

```

    {0x01, {0x62, 0x00}, 0x04}, # Enabled, base=62:66, size=4
  }
  {# MMIO decode ranges
    # 1, ENABLE - ESPI MMIO Decode Range
    # 2, 3, 4, 5, BASE - ESPI MMIO Decode Range base
    # 6, 7, SIZE - ESPI MMIO Decode Range size
    { #ESPI MMIO decode ranges
      {0x01, {0x04, 0x03, 0x02, 0x01}, {0x02, 0x01}, # Enabled, base=0x01020304, size=0x0102
      {0x00, {0x00, 0x00, 0x00, 0x00}, {0x00, 0x00}, # Disabled
      {0x00, {0x00, 0x00, 0x00, 0x00}, {0x00, 0x00}, # Disabled
      {0x00, {0x00, 0x00, 0x00, 0x00}, {0x00, 0x00}, # Disabled
    }
  }
}

```

3.1.2.2 FCH-Platform Configuration Items

- **PcdFchOemBeforePciRestoreSwSmi**

This item defines the SMI command value sent by the host BIOS during the S3 resume sequence, to re-initialize the FCH registers. This must be issued before the platform driver restore function is started.

Permitted Choices: (Type: Value)(Default: 0xD3)

- **PcdFchOemAfterPciRestoreSwSmi**

This item defines the SMI command used by the host BIOS to signal the FCH driver that the platform driver has completed its restore function. This allows the FCH driver to perform some final FCH settings.

Permitted Choices: (Type: Value)(Default: 0xD4)

- **PcdFchOemEnableAcpiSwSmi**

Allows the host BIOS to set the SMI command value used by the OS to activate ACPI mode.

Permitted Choices: (Type: Value)(Default: 0xA0)

- **PcdFchOemDisableAcpiSwSmi**

Allows the host BIOS to set the SMI command value used by the OS to turn off ACPI mode.

Permitted Choices: (Type: Value)(Default: 0xA1)

- **PcdFchOemSpiUnlockSwSmi**

SMI command for setting the lock mode in the SPI controller. This will effectively provide a write protection to the SPI Flash ROM; however, write access to secondary SPI devices will also be blocked.

Permitted Choices: (Type: Value)(Default: 0xAA)

- **PcdFchOemSpiLockSwSmi**

SMI command used for releasing the SPI controller lock mode. All devices on the SPI bus will be writable.

Permitted Choices: (Type: Value)(Default: 0xAB)

- **PcdAmdFchCfgSmbus0BaseAddress**

Allows the host BIOS to set the base IO address used for the SMBus 0 controller. The SMBus1 controller base address is at a fixed offset from the base0 address. The size of this block is defined in the Processor Programmers Reference (PPR).

Permitted Choices: (Type: Value)(Default: 0x0B00)

- **PcdAmdFchCfgSioPmeBaseAddress**

This item sets the IO Base address for the SIO block of command registers. The size of this block depends on the SIO chosen.

Permitted Choices: (Type: Value)(Default: 0x0E00)

- **PcdAmdFchCfgAcpiPm1EvtBlkAddr**

Allows the host BIOS to specify the IO address for the ACPI PM1 register blocks as defined by ACPI spec.

Permitted Choices: (Type: Value)(Default: 0x0400)

- **PcdAmdFchCfgAcpiPm1CntBlkAddr**

Allows the host BIOS to specify the IO address for the ACPI PM1Cnt register blocks as defined by ACPI spec.

Permitted Choices: (Type: Value)(Default: 0x0404)

- **PcdAmdFchCfgAcpiPmTmrBlkAddr**

Allows the host BIOS to specify the IO address for the ACPI PM Timer as defined by ACPI spec.

Permitted Choices: (Type: Value)(Default: 0x0408)

- **PcdAmdFchCfgCpuControlBlkAddr**

Allows the host BIOS to specify the IO address for the ACPI CPU Control block as defined by ACPI spec.

Permitted Choices: (Type: Value)(Default: 0x0410)

- **PcdAmdFchCfgAcpiGpe0BlkAddr**

Allows the host BIOS to specify the IO address for the ACPI GPE0 register block as defined by ACPI spec.

Permitted Choices: (Type: Value)(Default: 0x0420)

- **PcdAmdFchCfgSmiCmdPortAddr**

Allows the host BIOS to set the IO address used for the ACPI SMI command port.

Permitted Choices: (Type: Value)(Default: 0xB0)

- **FchRTDeviceEnableMap [F17M30]**

This control selects which FCH devices are to be enabled during runtime. This is a bit-mapped value where each bit represents an FCH device. A bit=1 indicates the device is enabled/active; a bit=0 indicates the device is turned off. The definition for this item can be found in AgesaModulePkg\AgesaModuleFchPkg.dec. Bits marked as 'read-only' cannot be modified via this control.

Permitted Choices: (Type:value (bitmapped) Default:(0x00000000))

- Bit 0 - 3: reserved.
- BIT4: LPC.
- BIT5: I2C0.
- BIT6: I2C1.
- BIT7: I2C2.
- BIT8: I2C3.
- BIT9: I2C4.

- BIT10: I2C5.
- BIT11: UART0.
- BIT12: UART1.
- BIT13 - BIT15: Reserved.
- BIT16: UART2.
- BIT17: Reserved.
- BIT18 - BIT19: eMMC driver type. It is read-only:
 - 00b AMD eMMC Driver.
 - 01b MS SD Driver.
 - 10b MS eMMC Driver.
 - 11b reserved.
- BIT20 - BIT25: Reserved.
- BIT26: UART3.
- BIT27: eSPI. It is read-only.
- BIT28: eMMC. It is read-only.
- BIT29 - BIT31: Reserved.
- **PcdFchUart0Irq [F17M30]**
- PcdFchUart1Irq [F17M30]**
- PcdFchUart2Irq [F17M30]**
- PcdFchUart3Irq [F17M30]**
- PcdFchI2c0Irq [F17M30]**
- PcdFchI2c1Irq [F17M30]**
- PcdFchI2c2Irq [F17M30]**
- PcdFchI2c3Irq [F17M30]**
- PcdFchI2c4Irq [F17M30]**
- PcdFchI2c5Irq [F17M30]**

These controls allow the platform to specify which IRQs the devices should use. Between the UART and I2C ports, it is recommended that no two ports use the same IRQ. Control of which ports are enabled is handled by FchRTDeviceEnableMap.

Permitted Choices: (Type: value)(Default: as below)

- Uart 0 - IRQ3
- Uart 1 - IRQ4
- Uart 2 - IRQ3
- Uart 3 - IRQ4
- I2C 0 - IRQA
- I2C 1 - IRQB
- I2C 2 - IRQ4
- I2C 3 - IRQ6
- I2C 4 - IRQE
- I2C 5 - IRQF

- **FchUart0LegacyEnable [F17M30]**

FchUart1LegacyEnable

FchUart2LegacyEnable

FchUart3LegacyEnable

These controls allow the platform to specify the legacy IO ranges that are to be used by the UART ports in the processor. You can find these defined in file AgesaModulePkg\AgesaModuleFchPkg.dec.

Permitted Choices: (Type: value)(Default: Disabled)

- 0 - Disabled
- 1 - IO range 0x02E8 - 0x02EF
- 2 - IO range 0x02F8 - 0x02FF
- 3 - IO range 0x03E8 - 0x03EF
- 4 - IO range 0x03F8 - 0x03FF

- **PcdLpcSsid**

This item specifies the LPC bridge controller SSID. Some customers may wish to set a specific SSID so that their driver or application can invoke special features. A value of zero will indicate that the normal hardware fused value, should be used.

Permitted Choices: (Type: Value)(Default: 0x00000000)

- **PcdSdSsid**

Allows the host BIOS to set the PCI Sub-System ID value reported by the Secure Digital (SD) card controller.

Permitted Choices: (Type: Value)(Default: 0x00000000)

- **PcdSmbusSsid**

Allows the host BIOS to set the PCI Sub-System ID value reported by the SMBus controller. A value of zero will indicate that the normal hardware fused value should be used.

Permitted Choices: (Type: Value)(Default: 0x00000000)

- **PcdSataAhciSsid**

Allows the host BIOS to set the PCI Sub-System ID value reported by the SATA controller. A value of zero will indicate that the normal hardware fused value should be used.

Permitted Choices: (Type: Value)(Default: 0x00000000)

- **PcdSataClass**

Selects the operational mode for the SATA controller

Permitted Choices: (Type: List)(Default: AHCI_Mode)

- RAID mode Not available for [F19M00].
- AHCI mode
- AMD-AHCI mode (for Device ID of 0x7904)

- **PcdSataMultiDiePortESP [F17M30][F19M00]**

This control selects the SATA port hot plug capability. Note: 'ESP' is External SATA Port, also known as E-SATA in the SATA spec. 5.2.6 e-SATA. The hot-plug feature is described in SATA spec 5.2 Usage Models - hot plug support.

The control is a 64-bit value divided into bit fields. Each bit represents one SATA port as follows:

- Bit 0-7: Socket0, SATA0 controller, Ports 0-7
- Bit 8-15: Socket0, SATA1 controller, Ports 0-7
- Bit 16-23: Socket0, SATA2 controller, Ports 0-7
- Bit 24-31: Socket0, SATA3 controller, Ports 0-7
- Bit 32-39: Socket1, SATA0 controller, Ports 0-7
- Bit 40-47: Socket1, SATA1 controller, Ports 0-7
- Bit 48-55: Socket1, SATA2 controller, Ports 0-7
- Bit 56-63: Socket1, SATA3 controller, Ports 0-7

For each bit field, permitted Choices: (Type: Value)(Default: 0x00000000_00000000)

- 0b - hot plug feature is disabled.
- 1b - hot plug feature is enabled for the port.

• **PcdSataSgpioMultiDieEnable [F17M30][F19M00]**

This control specifies if each of the SATA controllers will use an SGPIO pin to show status/activity (e.g. via an LED). The value is split into bit fields as follows:

- bits 0-3: Socket 0 , SATA controllers 0-3.
- bits 4-7: Socket 1 , SATA controllers 0-3.

For each bit field, permitted Choices: (Type: Value)(Default: 0x00)

- 0b - No pin is used by this controller.
- 1b - SATA controller uses a SGPIO pin.

• **PcdSataRaidSsid**

Allows the host BIOS to set the PCI Sub-System ID value reported by the SATA controller when configured for the Raid operational mode. A value of zero will indicate that the normal hardware fused value should be used. This value will be used if the SataClass is set for Raid mode.

Permitted Choices: (Type: Value)(Default: 0x00000000)

• **PcdXhciSsid**

Allows the host BIOS to set the PCI Sub-System ID value reported by the USB-3 controller. A value of zero will indicate that the normal hardware fused value should be used.

Permitted Choices: (Type: Value)(Default: 0x00000000)

3.1.2.3 FCH-Power Management Configuration Items

• **PcdPwrFailShadow**

This item selects the action to take after a power failure has occurred.

Permitted Choices: (Type: List)(Default: Pwr_Restore)

- Pwr_Off - Always leave the unit off.
- Pwr_On - Always restart the unit.

- Pwr_Restore - Return the unit to the state prior to the power outage.

3.1.2.4 FCH-Performance Configuration Items

- **PcdResetSpiSpeed**

This option selects the clock speed for the SPI bus after a reset.

Permitted Choices: (Type: List)(Default: 16.66MHz)

- 66MHz
- 33MHz
- 22MHz
- 16.7MHz
- 100MHz
- 800KHz

- **PcdResetFastSpeed**

This option selects the clock speed for the 'fast read' command on the SPI bus after a reset.

Permitted Choices: (Type: List)(Default: 33MHz)

- 66MHz
- 33MHz
- 22MHz
- 16.7MHz
- 100MHz
- 800KHz

- **PcdResetMode**

This option selects the SPI bus mode after a reset. The original SPI bus used only 1 wire for information transfer; this is known as 'normal mode'. Later, to improve transfer speeds, additional lines were added and thus was added 'Dual mode' and 'Quad mode'. The numbers following the mode names below show how the information is transferred, e.g. the number of lines used. The first number is for the command, the 2nd for the address and the 3rd is for the data.

Permitted Choices: (Type: List)(Default: Normal read, 33MHz)

- Normal read (up to 33MHz)
- Dual IO (1-1-2)
- Quad IO (1-1-4)
- Dual IO (1-2-2)
- Quad IO (1-4-4)
- Normal Read (up to 66MHz)
- Fast Read commands

- **PcdAmdFchI2c0SdaHold**

PcdAmdFchI2c1SdaHold

PcdAmdFchI2c2SdaHold

PcdAmdFchI2c3SdaHold

PcdAmdFchI2c4SdaHold**PcdAmdFchI2c5SdaHold**

These PCDs allow a platform to set the required SDA hold time, in units of ic_clk period, for the I2C controllers.

Permitted Choices: (Type: Value)(Default: 0x0000_0001)

- bits[15:0] IC_SDA_TX_HOLD - Sets the required SDA hold time in units of ic_clk period, when I2C logic acts as a transmitter.
- bits[23:16] IC_SDA_RX_HOLD - Sets the required SDA hold time in units of ic_clk period, when I2C logic acts as a receiver.

- **PcdUsbOemConfigurationTable**

This parameter is a multi-entry table that allows the platform designer to specify USB PHY characteristics. At present, this is only supported in the Family 17 Models 10h processors.

(Type: Table)(Default: 0x00)

Table structure:

Table 7. USB PHY parameters

Field	Length	Offset	Description
Version	2	0	USB controller version
Length	1	2	Length of table (bytes)
Reserved	1	3	
USB2.0 PHY	54	4	Usb 2.0 Driving Strength. There're 9 fields for each port. Totally 6 ports on RV. See table below.
Removable	1	58	Device removable flag: bit0-> Port0, bit1-> Port1, etc.
Reserved	3	59	

Within the USB2.0 PHY field is one record of 9 values for each USB port. Each record contains the following fields:

Table 8. USB PHY Lane Tuning Parameters

Byte	Field	Range	Pwr-on	Description
0	COMPSTUNE	0-0x7	3h	Disconnect Threshold Adjustment.
1	SQRXTUNE	0-0x7	3h	Squelch Threshold Adjustment.
2	TXFSLSTUNE	0-0xF	3h	FS/LS Source Impedance Adjustment.
3	TXPREEMPAMPTUNE	0-0x3	0h	HS Transmitter Pre-Emphasis Current Control.
4	TXPREEMPULSETUNE	0-0x1	0h	HS Transmitter Pre-Emphasis Duration Control.
5	TXRISETUNE	0-0x3	1h	HS Transmitter Rise/Fall Time Adjustment.
6	TXVREFTUNE	0-0xF	3h	HS DC Voltage Level Adjustment.
7	TXHSXVTUNE	0-0x3	3h	Transmitter High-Speed Crossover Adjustment.
8	TXRESTUNE	0-0x3	1h	USB Source Impedance Adjustment.

The Family17h Model 10h APU contains 6 ports, so an example of the PCD declaration is as follows:

```
gEfiAmdAgesaModulePkgTokenSpaceGuid.PcdUsbOemConfigurationTable|{ \
#Version_Major, Minor, Length, Reserve
0x0D, 0x00, 0x3E, 0x00, \
#Usb 2.0 PHY Lane Parameters
```



```
#COMPDiS, SQRX, FSLs, PREEMPAMP, PREEMPULSE, RISE, VREF, HSXV, RES
0x03, 0x03, 0x03, 0x03, 0x00, 0x01, 0x06, 0x03, 0x01, \#Controller0 Port0
0x03, 0x03, 0x03, 0x03, 0x00, 0x01, 0x06, 0x03, 0x01, \#Controller0 Port1
0x03, 0x03, 0x03, 0x03, 0x00, 0x01, 0x06, 0x03, 0x01, \#Controller0 Port2
0x03, 0x03, 0x03, 0x03, 0x00, 0x01, 0x06, 0x03, 0x01, \#Controller0 Port3
0x03, 0x03, 0x03, 0x00, 0x00, 0x01, 0x03, 0x03, 0x01, \#Controller1 Port0
0x03, 0x03, 0x03, 0x03, 0x00, 0x01, 0x06, 0x03, 0x01, \#Controller1 Port1
0x00, 0x00, 0x00, 0x00} #DeviceRemovable, 3Reserved
```

• **PcdFchFullHardReset**

This item selects the action to be taken when the system requests a 'cold reset'.

Permitted Choices: (Type: Boolean)(Default: False)

- False - Assert Reset signals only. This will be quicker.
- True - place system in S5 state for 3 to 5 seconds. This causes power Good to de-assert. Return to operation will take longer.

• **PcdResetCpuOnSyncFlood**

This control specifies the action taken when a Sync Flood is detected.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- False - System halt when Sync Flood occurs.
- True - System reset when Sync Flood occurs.

SATA Device Sleep

The SATA specification provides for setting a device into a runtime sleep state (RTD3). The following controls enable device sleep (DevSlp) for two of the SATA controller ports and specify which ports are designated as the target devices. See Serial ATA Device Sleep (DevSleep) and Runtime D3 (RTD3) specification.

• **PcdSataDevSlpPort0**

PcdSataDevSlpPort1

These controls Enable/Disable DevSlp for a port of the SATA controller.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE - SATA DevSlp Port N is Disable.
- TRUE - SATA DevSlp Port N Enabled.

• **PcdSataDevSlpPort0Num**

PcdSataDevSlpPort1Num

This control specifies the SATA controller port that is to be enabled for DevSlp. Up to two ports can be so designated. This control is dependent upon the corresponding PcdSataDevSlpPortN control.

Permitted Choices: (Type: List)(Default: 0)

- 0 - Port0 of SATA controller is targeted to support DevSlp.
- 1:7 - Port [1:7] of SATA controller is targeted to support DevSlp.

3.1.3 FCH_CONTROL_SERVICE_PROTOCOL

Summary

Provides FCH control and service functions for platform use.

GUID

```
gFchControlServiceProtocolGuid = { 0xae8fa023, 0xa0b6, 0x4ebc,  
                                     {0xa3, 0xc7, 0x3d, 0xf3, 0xa6, 0xa8, 0x94, 0xe9} }
```

Protocol Interface Structure

```
typedef struct _FCH_CONTROL_SERVICE_PROTOCOL {  
    UINTN      Revision;  
    UINTN      FchRev;  
    FP_FCH_GET_ACPI_MMIO FpGetAcpiMmioBase;  
} FCH_CONTROL_SERVICE_PROTOCOL;
```

Members

Revision	Protocol revision.
FchRev	FCH revision
FpGetAcpiMmioBase	Function used to acquire the base address for the ACPI control blocks.

Description

This Interface protocol provides a service function to get FCH configuration information.

3.1.3.1 FpGetAcpiMmioBase()

Summary

Get ACPI MMIO base address of the FCH, by selected die.

Prototype

```
typedef EFI_STATUS (EFI_API *FP_FCH_GET_ACPI_MMIO) (  
    IN CONST FCH_CONTROL_SERVICE_PROTOCOL *This,  
    IN UINT8 FchDie,  
    OUT UINT64 *AcpiMmioBase  
);
```

Parameters

This	A pointer to the FCH_CONTROL_SERVICE_PROTOCOL instance.
FchDie	The Die number of which to inquire about MMIO.
AcpiMmioBase	A pointer to a caller allocated buffer where the current ACPI MMIO base address of FCH will be placed.

Description

This function is used to get the current ACPI MMIO Base address being used on selected FCH.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_UNSUPPORTED	The ACPI MMIO decoding is not supported on the selected FCH.

status code	meaning
EFI_INVALID_PARAMETER	A parameter is invalid.

3.2 Promontory Drivers Module

Summary

This Driver module provides support for the Promontory (PT) port extension component.

Description

The 'Promontory'(PT) is a 'port expansion device' that can be paired with the integrated FCH. The PT devices adds USB, SATA,PCIe® and GPIO extensions to the FCH. The Promontory drivers module works in tandem with the FCH Driver Module to implement support for these multi-function devices.

Configuration and options are controlled by parameters set in the interface structures. The structures are defined in FCHPT.h file and the default values for the parameters are declared in the file ResetDefTS.c. The customer is free to modify this file to alter the default settings instantiated at build time, or may choose to use the POST time call-out routine to actively modify the parameters. This path would be used for items that have SETUP option fields defined by the platform.

Related Definitions - Common Data Structure Definitions

FCH_PT

```
typedef _FCH_PT {
    PT_USB      PtUsb;
    PT_SATA     PtSata;
    FCH_PT_PCIE PtPcie;
    PT_USBPort  PtUsbPorts;
    PT_USBTx    PtUsbTxSettings;
    PT_SATA_Tx  PtSataTxSettings;
} FCH_PT;
```

PtUsb

General settings for the PT USB ports.

PtSata

General settings for the PT SATA ports.

PtPcie

Definition TBD

PtAddresses

Definition TBD

PtUsbPorts

Activation list for the PT USB ports.

PtUsbTxSettings

PHY transmitter settings for the PT USB ports.

PtSataTxSettings

PHY transmitter settings for the PT SATA ports.

PT_USB - PT Common USB Settings

```
typedef struct {
    UINT8 PTXhciGen1;
    UINT8 PTXhciGen2;
    UINT8 PTAOAC;
    UINT8 PTHW_LPM;
    UINT8 PTDbC;
    UINT8 PTXHC_PME;
    UINT8 PTSystemSpreadSpectrum;
    UINT8 Equalization4;
    UINT8 Redriver;
} PT_USB;
```

<i>PTXhciGen1</i>	This value determines if the Xhci enables Gen1 support. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTXhciGen2</i>	This value determines if the Xhci enables Gen2 support. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTAOAC</i>	This element specifies if the XHCI is to support the "Always On, Always Connected" (AOAC) feature. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTHW_LPM</i>	This element specifies if the XHCI is to support the hardware based Low Power Mode (LPM) feature. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTDbC</i>	This value determines if XHCI should enabled the Debug port support. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTXHC_PME</i>	This element specifies if the XHCI should support the "Power Management Event" (PME) for the USB ports. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTSystemSpreadSpectrum</i>	This value determines if PT enabled SpreadSpectrum support. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>Equalization4</i>	This element specifies if the XHCI should enable the alternate equalization table support for this platform. This requires the platform to modify the alternate table contents. This may be needed when designing for the use of non-standard connector. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - This option is active • FALSE - This option is turned off

Redriver

This element specifies if the XHCI should enable the re-driver support. When the USB trace length is longer than 4", the system needs to add a signal repeater ('re-driver'). The PT driver contains two tables to handle these two cases. This element specifies if the driver needs to use the re-driver table.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PT_SATA - PT Common SATA Settings

```
typedef struct {
    UINT8    PTSataPortEnable;
    UINT8    PTSataMode;
    UINT8    PTSataAggressiveDevSlpP0;
    UINT8    PTSataAggressiveDevSlpP1;
    UINT8    PTSataAggrLinkPmCap;
    UINT8    PTSataPscCap;
    UINT8    PTSataSscCap;
    UINT8    PTSataMsiCapability;
    UINT8    PTSataPortMdPort0;
    UINT8    PTSataPortMdPort1;
    UINT8    PTSataHotPlug;
} PT_SATA;
```

PTSataEnable

This element specifies if all of the SATA ports in the PT are to be active.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

PTSataMode

This value determines which PT SATA mode enabled.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - RAID mode enabled
- FALSE - AHCI mode enabled

PTSataAggressiveDevSlpP0

This value determines if SATA enabled Aggressive Device Sleep on Port0.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PTSataAggressiveDevSlpP1

This value determines if SATA enabled Aggressive Device Sleep on Port1.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

<i>PTSataAggrLinkPmCap</i>	This value determines if the SATA controller is to enable the Aggressive Link Power Management (ALPM) capability. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTSataPscCap</i>	This value determines if the SATA controller is to enable the partial state capability (PSC). Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTSataSscCap</i>	This value determines if the SATA controller is to enable the Slumber state capability (SSC). Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTSataMsiCapability</i>	This element specifies if the PT SATA controller is to use MSI protocol for interrupts. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
<i>PTSataPortMdPort0</i>	This value determines the SATA mode for Port 0. Permitted Choices: (Type: List)(Default: NO_LIMIT) • 0 - No limit • 1 - Fixed port speed as Gen1 • 2 - Fixed port speed as Gen2 • 3 - Fixed port speed as Gen3
<i>PTSataPortMdPort1</i>	This value determines the SATA mode for Port 1. Permitted Choices: (Type: List)(Default: NO_LIMIT) • 0 - No limit • 1 - Fixed port speed as Gen1 • 2 - Fixed port speed as Gen2 • 3 - Fixed port speed as Gen3
<i>PTSataHotPlug</i>	This value determines if the SATA controller is to enable the HotPlug capability. This feature is described in the SATA standard specification. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - This option is active • FALSE - This option is turned off

PT_USBPort - PT USB Port Enable/Disable

```
typedef struct {
    UINT8    PTUsb31P0;
    UINT8    PTUsb31P1;
    UINT8    PTUsb30P0;
    UINT8    PTUsb30P1;
    UINT8    PTUsb30P2;
    UINT8    PTUsb30P3;
    UINT8    PTUsb30P4;
    UINT8    PTUsb30P5;
    UINT8    PTUsb20P0;
    UINT8    PTUsb20P1;
    UINT8    PTUsb20P2;
    UINT8    PTUsb20P3;
    UINT8    PTUsb20P4;
    UINT8    PTUsb20P5;
} PT_USBPort;
```

PTUsb<cc><pp>

These elements enable the individual ports in the USB controllers. Where <cc> is the controller designator and <pp> indicates the port within the controller. E.g. PTUsb31P0 designates USB 3.1 controller, port 0.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This port is active
- FALSE - This port is turned off

PT_USBTx - PT USB Tx Structure

```
typedef struct {
    PT_USB31Tx    USB31Tx[2];
    PT_USB30Tx    USB30Tx[3];
    UINT8         USB20B2Tx00;
    UINT8         USB20B2Tx05;
    UINT8         USB20B3Tx1113;
    UINT8         USB20B3Tx1012;
    UINT8         USB20B4Tx0206;
    UINT8         USB20B4Tx0307;
    UINT8         USB20B5Tx0408;
    UINT8         USB20B5Tx0109;
} PT_USBTx;
```

These parameters are performance tuning values that set the drive strength of signals on the ports. Please refer to the Promontory specification for more details.

USB31Tx

Tuning values for the USB 3.1 controller ports.

USB30Tx

Tuning values for the USB 3.0 controller ports.

USB20B2Tx00

USB2.0 TX driving current for USB_HSDP/N[0]

Permitted Choices: 0..7 (Type: Value)(Default: 0x05)

USB20B2Tx05

USB2.0 TX driving current for USB_HSDP/N[5]

Permitted Choices: 0..7 (Type: Value)(Default: 0x05)

USB20B3Tx1113

USB2.0 TX driving current for USB_HSDP/N[13][11]

Permitted Choices: 0..7 (Type: Value)(Default: 0x05)

USB20B3Tx1012

USB2.0 TX driving current for USB_HSDP/N[12][10]

USB20B4Tx0206	Permitted Choices: 0..7 (Type: Value)(Default: 0x05) USB2.0 TX driving current for USB_HSDP/N[2][6]
USB20B4Tx0307	Permitted Choices: 0..7 (Type: Value)(Default: 0x05) USB2.0 TX driving current for USB_HSDP/N[3][7]
USB20B5Tx0408	Permitted Choices: 0..7 (Type: Value)(Default: 0x05) USB2.0 TX driving current for USB_HSDP/N[4][8]
USB20B5Tx0109	Permitted Choices: 0..7 (Type: Value)(Default: 0x05) USB2.0 TX driving current for USB_HSDP/N[1][9] Permitted Choices: 0..7 (Type: Value)(Default: 0x05)

PT_USB31Tx - PT USB31 Tx Structure

```

typedef struct {
    UINT8    USB31Gen1Swing;
    UINT8    USB31Gen2Swing;
    UINT8    USB31Gen1PreEmEn;
    UINT8    USB31Gen2PreEmEn;
    UINT8    USB31Gen1PreEmLe;
    UINT8    USB31Gen2PreEmLe;
    UINT8    USB31Gen1PreShEn;
    UINT8    USB31Gen2PreShEn;
    UINT8    USB31Gen1PreShLe;
    UINT8    USB31Gen2PreShLe;
} PT_USB31Tx;

```

These values are used for performance tuning. Please see the Promontory specification for further details.

USB31Gen1Swing	This value determines USB3.1 PCS_B1 genI swing Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x0F)
USB31Gen2Swing	This value determines USB3.1 PCS_B1 genII swing Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x0F)
USB31Gen1PreEmEn	This value determines USB3.1 PCS_B1 genI pre-emphasis enable Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
USB31Gen2PreEmEn	This value determines USB3.1 PCS_B1 genII pre-emphasis enable Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
USB31Gen1PreEmLe	This value determines USB3.1 PCS_B1 genI pre-emphasis level Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x04)
USB31Gen2PreEmLe	This value determines USB3.1 PCS_B1 genII pre-emphasis level

USB31Gen1PreShEn	Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x03) This value determines USB3.1 PCS_B1 genI pre-shoot enable Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - This option is active • FALSE - This option is turned off
USB31Gen2PreShEn	This value determines USB3.1 PCS_B1 genII pre-shoot enable Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
USB31Gen1PreShLe	This value determines USB3.1 PCS_B1 genI pre-shoot level Permitted Choices: 0x00..0x07 (Type: Value)(Default: 0x04)
USB31Gen2PreShLe	This value determines USB3.1 PCS_B1 genII pre-shoot level Permitted Choices: 0x00..0x07 (Type: Value)(Default: 0x01)

PT_USB30Tx - PT USB30 Tx Structure

```
typedef struct {
    UINT8    USB30Gen1Swing;
    UINT8    USB30Gen1PreEmEn;
    UINT8    USB30Gen1PreEmLe;
} PT_USB30Tx;
```

USB30Gen1Swing	This value determines USB3.0 PCS_B3 genI swing Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x0F)
USB30Gen1PreEmEn	This value determines USB3.0 PCS_B3 genI pre-emphasis enable Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
USB30Gen1PreEmLe	This value determines USB3.0 PCS_B3 genI pre-emphasis level Permitted Choices: 0x00..0x07 (Type: Value)(Default: 0x04)

PT_SATATx - PT SATA Tx Structure

```
typedef struct {
    UINT8    SATAGen1Swing;
    UINT8    SATAGen2Swing;
    UINT8    SATAGen3Swing;
    UINT8    SATAGen1PreEmEn;
    UINT8    SATAGen2PreEmEn;
    UINT8    SATAGen3PreEmEn;
}
```

```

        UINT8          SATAGen1PreEmLevel;
        UINT8          SATAGen2PreEmLevel;
        UINT8          SATAGen3PreEmLevel;
    } PT_SATATx;

```

These values are used for performance tuning. Please see the Promontory specification for further details.

SATAGen1Swing	SATA genI swing
SATAGen2Swing	Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x00) SATA genII swing
SATAGen3Swing	Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x04) SATA genIII swing
SATAGen1PreEmEn	Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x08) This value determines SATA genI pre-emphasis enable. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - This option is active • FALSE - This option is turned off
SATAGen2PreEmEn	This value determines SATA genII pre-emphasis enable. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
SATAGen3PreEmEn	This value determines SATA genIII pre-emphasis enable. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
SATAGen1PreEmLevel	SATA genI pre-emphasis level
SATAGen2PreEmLevel	Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x00) SATA genII pre-emphasis level
SATAGen3PreEmLevel	Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x02) SATA genIII pre-emphasis level Permitted Choices: 0x00..0x0F (Type: Value)(Default: 0x03)

3.2.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

3.2.2 Promontory Configuration Items

Enablement Category

-

Platform Category

-

Power Management Category

-

Performance Category

- **PcdPTCheckFwAfterWrite**

Select whether or not FW check mechanism is active.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

3.2.3 EFI_PT_GPIO_PPI**Summary**

Provide Promontory GPIO Initialize/read/write services PPI in PEI phase.

GUID

```
gEfiPeiPromotoryGpioPpiGuid =
    { 0x31859c50, 0x2845, 0x42da,
      {0x9f, 0x37, 0xe7, 0x18, 0x67, 0xe3, 0xe0, 0x5e } }
```

PPI Interface Structure

```
typedef struct _EFI_PT_GPIO_PPI {
    PTInitialGpioPei  PTInitialGpio;
    PTWriteGpioPei    PTWriteGpio;
    PTReadGpioPei     PTReadGpio;
} EFI_PT_GPIO_PPI;
```

Parameters

<i>PTInitialGpio</i>	Function to perform GPIO structures initialization. This routine also uses a call-out to the OEM for making modifications to selected parameters.
<i>PTWriteGpio</i>	Function to set the GPIO to a value.
<i>PTReadGpio</i>	Function to read the present state of a GPIO.

Description This Interface PPI is provides services to the PEI phase for handling the general GPIO pins on the Promontory component.

3.2.3.1 PTInitialGpio()**Summary**

Promontory provide GPIO Initialize PPI in PEI to initialize PT's GPIOs 0 to 7.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTInitialGpioPei)(
    IN      EFI_PEI_SERVICES    **PeiServices,
    IN      VOID                *PTGpioPtr
);
```

Parameters

PeiServices Standard UEFI pointer to the PEI services table.

PTGpioPtr Pointer to the platform GPIO table. This table is defined in the AMD Common Platform Module (CPM). Please see the documentation for that module.

Description

This function is used to initialize the GPIO services and GPIO states for the PEI phase.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The GPIO driver couldn't find the Promontory device in the system.

3.2.3.2 PTWriteGpio()

Summary

This function is used to set the output value of a GPIO.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTWriteGpioPei) (
    IN      EFI_PEI_SERVICES    **PeiServices,
    IN      UINT8               Pin,
    IN      UINT8               Value
);
```

Parameters

PeiServices Standard UEFI pointer to the PEI services table.

Pin This value determines which GPIO pin to be written. There are 8 general purpose GPIO pins on the promontory labeled 0..7. This parameter selects the pin (0..7) to be modified.

Value This value specifies the new output state for selected GPIO pin.

Permitted Choices: (Type: List)(Default: n/a)

- 0 - GPIO pin is set to logic low level.
- 1 - GPIO pin is set to logic High level.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
RETURN_UNSUPPORTED	The GPIO pin cannot support the write function due to its configuration doesn't define it as a GPIO. This may occur with multi-function pins when the alternate pin function is selected.

3.2.3.3 PTReadGpio()

Summary

This function is used to read the input status of a GPIO.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTReadGpioPei) (
    IN      EFI_PEI_SERVICES  **PeiServices,
    IN      UINT8              Pin,
    OUT     UINT8              Value
);
```

Parameters

PeiServices

Standard UEFI pointer to the PEI services table.

Pin

This value determines which GPIO pin to be read. There are 8 general purpose GPIO pins on the promontory labeled 0..7. This parameter selects the pin (0..7) to be read.

Value

This value reflects the status of the selected GPIO pin.

Permitted Choices: (Type: List)(Default: n/a)

- 0 - GPIO pin status is 0.
- 1 - GPIO pin status is 1.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
RETURN_UNSUPPORTED	The GPIO pin cannot support the read function due to its configuration doesn't define it as a GPIO. This may occur with multi-function pins when the alternate pin function is selected.

3.2.4 EFI_PT_GPIO_PROTOCOL

Summary

Provide Promontory GPIO Initialize/read/write Protocol in DXE phase.

GUID

```
gEfiPromontoryGpioProtocolGuid =
{ 0x0617bd2e, 0x01c1, 0x4432,
  {0xbb, 0xf6, 0x28, 0x42, 0x93, 0xcc, 0x0d, 0xb9 } }
```

PPI Interface Structure

```
typedef struct _EFI_PT_GPIO_PROTOCOL {
    PTInitialGpioDxe  PTInitialGpio;
    PTWriteGpioDxe    PTWriteGpio;
    PTReadGpioDxe     PTReadGpio;
```

```
} EFI_PT_GPIO_PROTOCOL;
```

Parameters

PTInitialGpio Function to perform GPIO structures initialization. This routine also uses a call-out to the OEM for making modifications to selected parameters.

PTWriteGpio Function to set the GPIO to a value.

PTReadGpio Function to read the present state of a GPIO.

Description This Interface protocol provides services to the DXE phase for handling the general GPIO pins on the Promontory component.

3.2.4.1 PTInitialGpioDxe()

Summary

Promontory provide GPIO Initialize protocol in DXE to initialize PT's GPIOs 0 to 7.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTInitialGpioDxe)(
IN      VOID                *PTGpioPtr
);
```

Parameters

PTGpioPtr Pointer to the platform GPIO table. This table is defined in the AMD Common Platform Module (CPM). Please see the documentation for that module.

Description

This function is used to initialize the GPIO services and GPIO states for the DXE phase.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The GPIO driver couldn't find the Promontory device in the system.

3.2.4.2 PTWriteGpioDxe()

Summary

This function is used to set the output value of a GPIO.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTWriteGpioDxe) (
IN      UINT8                Pin,
IN      UINT8                Value
);
```

Parameters

Pin This value determines which GPIO pin to be written. There are 8 general purpose GPIO pins on the promontory labeled 0..7. This parameter selects the pin (0..7) to be modified.

Value This value specifies the new output state for selected GPIO pin.

Permitted Choices: (Type: List)(Default: n/a)

- 0 - GPIO pin is set to logic low level.
- 1 - GPIO pin is set to logic High level.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
RETURN_UNSUPPORTED	The GPIO pin cannot support the write function due to its configuration doesn't define it as a GPIO. This may occur with multi-function pins when the alternate pin function is selected.

3.2.4.3 PTReadGpioDxe()

Summary

This function is used to read the input status of a GPIO.

Prototype

```
typedef
EFI_STATUS
(EFI_API *PTReadGpioDxe) (
    IN      UINT8      Pin,
    OUT     UINT8      Value
);
```

Parameters

Pin This value determines which GPIO pin to be read. There are 8 general purpose GPIO pins on the promontory labeled 0..7. This parameter selects the pin (0..7) to be read.

Value This value reflects the status of selected GPIO pin.

Permitted Choices: (Type: List)(Default: n/a)

- 0 - GPIO pin status is 0.
- 1 - GPIO pin status is 1.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
RETURN_UNSUPPORTED	The GPIO pin cannot support the read function due to its configuration doesn't define it as a GPIO. This may occur with multi-function pins when the alternate pin function is selected.

3.2.5 EFI_PT_GPIO_SMM_PROTOCOL

Summary

Provide Promontory GPIO read/write services protocol in SMM (Runtime) phase.

GUID

```
gEfiPromontoryGpioSmmProtocolGuid =
    { 0x9bb1624d, 0xf007, 0x4041,
      {0xb8, 0x45, 0x6b, 0xea, 0xd5, 0xb0, 0x7d, 0x34}}
```

PPI Interface Structure

```
typedef struct _EFI_PT_GPIO_SMM_PROTOCOL {
    PTWriteGpioSmm    PTWriteGpioSmm;
    PTReadGpioSmm     PTReadGpioSmm;
} EFI_PT_GPIO_SMM_PROTOCOL;
```

Parameters

PTWriteGpioSmm

Function to set the GPIO to a value.

PTReadGpioSmm

Function to read the present state of a GPIO.

Description This Interface protocol provides services to the SMM phase for handling the general GPIO pins on the Promontory component.

3.2.5.1 PTWriteGpioSmm()

Summary

This function is used to set the output value of a GPIO.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTWriteGpioDxe) (
    IN      UINT8          Pin,
    IN      UINT8          Value
);
```

Parameters

Pin

This value determines which GPIO pin to be written. There are 8 general purpose GPIO pins on the promontory labeled 0..7. This parameter selects the pin (0..7) to be modified.

Value

This value specifies the new output state for selected GPIO pin.

Permitted Choices: (Type: List)(Default: n/a)

- 0 - GPIO pin is set to logic low level.
- 1 - GPIO pin is set to logic High level.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
RETURN_UNSUPPORTED	The GPIO pin cannot support the write function due to its configuration doesn't define it as a GPIO. This may occur with multi-function pins when the alternate pin function is selected.

3.2.5.2 PTReadGpioSmm()

Summary

This function is used to read the input status of a GPIO.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *PTReadGpioDxe) (
    IN      UINT8      Pin,
    OUT     UINT8      Value
);
```

Parameters

Pin This value determines which GPIO pin to be read. There are 8 general purpose GPIO pins on the promontory labeled 0..7. This parameter selects the pin (0..7) to be read.

Value This value reflects the status of selected GPIO pin.

Permitted Choices: (Type: List)(Default: n/a)

- 0 - GPIO pin status is 0.
- 1 - GPIO pin status is 1.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
RETURN_UNSUPPORTED	The GPIO pin cannot support the read function due to its configuration doesn't define it as a GPIO. This may occur with multi-function pins when the alternate pin function is selected.

3.3 Error Log Driver Module

Summary

This module implements a error logging buffer and an API for the system BIOS to retrieve reported entries.

Description

The Error log driver provides the platform BIOS with a window into the outcome of the initialization processes. All warnings, errors and messages reported by the AGESA™ software are logged into the Error Log for later reference by the platform BIOS. Notifications are provided for events of special interest. This module's PEI driver will allocate buffer space for the log entries - the Error Log.

Related Definitions

This structure is common to the PEI and DXE error reporting modules.

ERROR_LOG_PARAMS

```
typedef struct {
    OUT     UINT32      ErrorClass;
    OUT     UINT32      ErrorInfo;
    OUT     UINT32      DataParam1;
    OUT     UINT32      DataParam2;
    OUT     UINT32      DataParam3;
    OUT     UINT32      DataParam4;
} ERROR_LOG_PARAMS;
```

ErrorClass The severity of this error, the values and meanings are aligned with the definition of AGESA_STATUS as found in the file \Include \AMD.h.

- AGESA_FATAL - A condition exists such that the boot process should not continue. The platform BIOS should indicate the error to the user then halt or re-start.
- AGESA_ALERT - The function could not accomplish the indicated outcome. For example, the user may request a certain feature, but the conditions in the platform do not presently support that feature.
- AGESA_BOUNDS_CHK - A parameter used by a user function call is out of bounds or otherwise not valid. The platform BIOS making the call should review their values. The operation of the function may not result in the expected behavior.

ErrorInfo

The unique error identifier, or error code value, zero means "no error".

DataParam1,
DataParam2,
DataParam3,
DataParam4

Data specific to the error.

3.3.1 Error Log Error CLasses

The following sections define the logged errors by Class.

3.3.1.1 Error Class: Fatal**Summary**

These reported errors indicate a condition under which the boot process should not continue. The platform BIOS should indicate the error to the user then halt or re-start.

Table 9. Fatal Error Codes

CodeValue	Title			
	Data A	Data B	Data C	Data D
0x00003002	ABL_ERROR_SLAVE_SYNC_COMMUNICATION_WITH_DATA_SENT_TO_MASTER_ERROR			
	-	-	-	-
0x00003003	ABL_ERROR_SLAVE_BROADCAST_COMMUNICATION_FROM_MASTER_TO_SLAVE_ERROR			
	-	-	-	-
0x00003004	ABL_MSG_SLAVE_INFORMS_MASTER_ERROR_INFO_MSG			
	-	-	-	-
0x00004034	ABL_MEM_ERROR_MIX_RDIMM_LRDIMM_IN_SOCKET [F17M30] - Mixing of LRDIMM and RDIMM is not permitted.			
	-	-	-	-
0x08030100	CPU_ERROR_HEAP_BUFFER_HANDLE_IS_ALREADY_USED			
	handle	-	-	-
0xDF410000	DF_FATAL_MEM_MAP_NOT_CREATED			

Table 9. Fatal Error Codes (continued)

CodeValue	Title			
	Data A	Data B	Data C	Data D
	Cause: 1 - Memory structures not present in heap 2 - Channel population info not present in heap 3 - All channels in the system are reporting no valid DIMMs present	-	-	-

3.3.1.2 Error Class: Alert**Summary**

Errors in the class 'Alert' indicate situations in which the function operation may not have produced the desired outcome. For example, a requested feature is not available due to the current system configuration.

Table 10. Alert Error Codes

CodeValue	Title			
	Data A	Data B	Data C	Data D
0x08000100	CPU_EVENT_NON_CCX_BIST_FAILURE			
	Socket	Die Number	CCD Number	BistData
0x08000101	CPU_EVENT_CCX_BIST_FAILURE			
	Socket	Die	CCX	-
0x08000102	CPU_EVENT_DOWN_CORE_FAILURE			
	This group of error events are logged when an issue with the down-core feature occurs. The cause of the failure is held in the Data A parameter. The Data B/C parameters may contain a bit mask; where each bit position represents a CCD/Core, 1=disable, 0=enable. For example, if down coring to a 1 core would be: 0x00FE, 2 cores would be 0x00FC.			
	CCD_INVALID_SELECTION - An unsupported number of CCDs was selected through PcdAmdCcdMode			
	0x001	[31:16]Requested # CCD [15:0]Actual # CCD	[31:16]Requested # Cores [15:0] Actual # Cores	-
	CCD_INVALID_CONFIG - The number of CCDs requested is less than current enabled CCDs in system.			
	0x003	[31:16]Requested # CCD [15:0]Actual # CCD	-	-
	CCD_DOWN_CORE_CONFIG_MET - System has successfully down CCDed to the requested number of CCDs.			
	0x004	-	-	-
	CORE_INVALID_SELECTION - An invalid/unsupported number of cores was selected through PcdAmdDownCoreMode.			
	0x005	-	-	-
	CORE_INVALID_CONFIG_FAILURE - The number of cores requested is less than current enabled cores in system.			
	0x007	[31:16]Requested # CCD [15:0]Actual # CCD	[31:16]Requested # Cores [15:0]Actual # Cores	-

Table 10. Alert Error Codes (continued)

CodeValue	Title			
	Data A	Data B	Data C	Data D
	CORE_DOWN_CORE_CONFIG_MET - System has successfully down cored to the requested number of cores.			
	0x008	[31:16]Requested # CCD [15:0]Actual # CCD	[31:16]Requested # Cores [15:0]Actual # Cores	-
	CORE_DOWN_CORE_FROM_XAPIC - see PcdAmdApicMode.			
	0x009	-	-	-
0xDF010000	DF_ALERT_BOTTOM_IO_MODIFIED The 32-bit address requested via token APCB_TOKEN_CONFIG_DF_BOTTOMIO was modified to match processor restrictions.			
	Requested Address	Actual address assigned	-	-
0xDF010002	DF_WARNING_NPS_INVALID_REQUEST Unable to fulfill the NPS setting that was requested.			
	NumaNodesPerSocket	Actual NPS setting used.	-	-
0xDF010001	DF_ALERT_1TB_REMAP_NOT_POSSIBLE 1TB of memory is present, however remap is not possible.			
	-	-	-	-
0xDF020000	DF_ALERT_PCI_MMIO_SIZE_MODIFIED The processor requires the number of busses to be a power of 2. The requested value was rounded up to the next power of 2.			
	requested size - size of PCI MMIO space requested via APCB token: APCB_TOKEN_CONFIG_DF_PCI_MMIO_SIZE.	effective size - amount of MMIO space allocated to contain the number of busses requested.		

3.3.1.3 Error Class: Bounds Check

Summary

These reported errors indicate a parameter used by a user function call is out of bounds or otherwise not valid. The platform BIOS making the call should review their values. The operation of the function may not result in the expected behavior.

Table 11. Bounds Check Error Codes

CodeValue	Title			
	Data A	Data B	Data C	Data D
0x08030100	CPU_ERROR_HEAP_BUFFER_HANDLE_IS_ALREADY_USED			
	handle	-	-	-
0x08020100	CPU_ERROR_HEAP_IS_FULL			
	handle that failed allocation	-	-	-

Table 11. Bounds Check Error Codes (continued)

CodeValue	Title			
	Data A	Data B	Data C	Data D
0x08040100	CPU_ERROR_HEAP_BUFFER_HANDLE_IS_NOT_PRESENT			
	handle	-	-	-

3.3.2 PEI_AMD_ERROR_LOG_SERVICE_PPI

Summary

Read error records from the log.

GUID

```
gAmErrorLogServicePpiGuid =
{0x7cdf73a2, 0x51a9, 0x4c0b,
{0xaa, 0x68, 0x3c, 0x46, 0x91, 0x75, 0x76, 0xf9}}
```

PPI Interface Structure

```
typedef struct _PEI_AMD_ERROR_LOG_SERVICE_PPI {
    AMD_AQUIRE_ERROR_LOG_PEI    AmdAquireErrorLogPei;
    AMD_AQUIRE_ERROR_LOG_WITH_FLAG_PEI    AmdAquireErrorLogWithFlagPei;
} PEI_AMD_ERROR_LOG_SERVICE_PPI;
```

Parameters

AmdAquireErrorLogPei

This function is used by the BIOS to read log entries.

AmdAquireErrorLogWithFlagPei

This function is used by the BIOS to read log entries with flag to Choose clear or keep the entries in the queue.

Description

This API is used to operate the logging services of the AGESA™ code base.

3.3.2.1 AMD_AQUIRE_ERROR_LOG_PEI()

Summary

Function to use to read the entries in the error log.

Prototype

```
typedef
EFI_STATUS
(EFI_API *AMD_AQUIRE_ERROR_LOG_PEI)(
    IN      PEI_AMD_ERROR_LOG_SERVICE_PPI    *This,
    OUT     ERROR_LOG_DATA_STRUCT *ErrorLogDataPtr
);
```

Parameters

This

UEFI PEI standard pointer to the PEI services function block.

ErrorLogDataPtr

Pointer to a data block that will be filled with the error log entries. The caller is responsible for allocating a data buffer of the proper size to hold the maximum of entries.

Description

This function is used to read the error log entries. All entries in the log are returned for each call to this function. The entries are kept in a circular queue, so it is possible to have entries overwritten by new entries. As entries are read by this function, the entry is removed from the queue. It is the responsibility of the caller to preserve any information it wishes to process at a later time point. This function clears the error log buffer, so it should be called once after the [PEI_AMD_ERROR_LOG_AVAILABLE_PPI](#) has been published.

Related Definitions

ERROR_LOG_DATA_STRUCT

```
typedef struct {
    OUT    UINT32          Count;
    OUT    ERROR_LOG_PARAMS ErrorLog_Param[TOTAL_ERROR_LOG_BUFFERS];
} ERROR_LOG_DATA_STRUCT;
```

Count

Actual count of Error Log entries found in the log queue.

ErrorLog_Param

This is an array of log entries containing data specific to the error.

ERROR_LOG_PARAMS

```
typedef struct {
    OUT    UINT32          ErrorClass;
    OUT    UINT32          ErrorInfo;
    OUT    UINT32          DataParam1;
    OUT    UINT32          DataParam2;
    OUT    UINT32          DataParam3;
    OUT    UINT32          DataParam4;
} ERROR_LOG_PARAMS;
```

ErrorClass

The severity of this error, the values and meanings are aligned with the definition of AGESA_STATUS.

ErrorInfo

The unique error identifier, zero means "no error".

DataParam1,

Data specific to the error. The error codes and parameter values are defined elsewhere in this specification.

DataParam2,

DataParam3,

DataParam4

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.3.2.2 AMD_AQUIRE_ERROR_LOG_WITH_FLAG_PPI()

Summary

Function to use to read or inspect entries in the error log.

Prototype

```
typedef
EFI_STATUS
```

```
(EFIAPI * AMD_AQUIRE_ERROR_LOG_WITH_FLAG_PEI)(
IN      PEI_AMD_ERROR_LOG_SERVICE_PPI *This,
OUT     ERROR_LOG_DATA_STRUCT *ErrorLogDataPtr,
IN      BOOLEAN ClearBuffer
);
```

Parameters

- This** UEFI PEI standard pointer to the PEI services function block.
- ErrorLogDataPtr** Pointer to a data block that will be filled with the error log entries. The caller is responsible for allocating a data buffer of the proper size to hold the maximum of entries.
- ClearBuffer** Indicated the user's desire to erase or keep the entries being read.
- TRUE - Erase the entries as they are read.
 - FALSE - Retain the error log. Nothing is erased.

Description

This function is used to read the error log entries. All entries in the log are returned for each call to this function. The entries are kept in a circular queue, so it is possible to have entries overwritten by new entries. As entries are read by this function, the entry is removed from, or kept in, the queue. It is the responsibility of the caller to preserve any information it wishes to process at a later time point.

Related Definitions

ERROR_LOG_DATA_STRUCT

```
typedef struct {
OUT  UINT32      Count;
OUT  ERROR_LOG_PARAMS ErrorLog_Param[TOTAL_ERROR_LOG_BUFFERS];
} ERROR_LOG_DATA_STRUCT;
```

- Count** Actual count of Error Log entries found in the log queue.
- ErrorLog_Param** This is an array of log entries containing data specific to the error. This is the same structure used in the [Aquire Error Log](#) function.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.3.3 PEI_AMD_ERROR_LOG_SERVICE_READY_PPI

Summary

Indicates that the log service is available.

GUID

```
gAmErrorLogServiceReadyPpiGuid =
{0x80a63347, 0xe22f, 0x48e1,
{0xb4, 0x50, 0x44, 0x7e, 0x2b, 0x6, 0x7, 0x96}}
```

PPI Interface Structure

```
typedef struct _PEI_AMD_ERROR_LOG_SERVICE_READY_PPI {
    UINTN    Revision;          ///< Revision Number
} PEI_AMD_ERROR_LOG_SERVICE_READY_PPI;
```

Parameters

Revision	Definition
-----------------	------------

Description This is a flag type PPI that is published by the module initialization process to indicate to other modules that the error log service is available.

3.3.4 PEI_AMD_ERROR_LOG_FULL_PPI

Summary

Indicates that the logging buffer is full.

GUID

```
gAmdErrorLogFullPpiGuid =
    {0xcbe31239, 0x532c, 0x4c27,
     0x8f, 0x4e, 0x65, 0x7b, 0x3c, 0x39, 0xa5, 0x56}}
```

PPI Interface Structure

```
typedef struct _PEI_AMD_ERROR_LOG_FULL_PPI {
    UINTN    Revision;          ///< Revision Number
} PEI_AMD_ERROR_LOG_FULL_PPI;
```

Parameters

Revision	Definition
-----------------	------------

Description This is a flag type PPI that is published to indicate that the error log buffer is full.

3.3.5 PEI_AMD_ERROR_LOG_AVAILABLE_PPI

Summary

Indicates that the log is available. All AMD drivers in PEI phase have completed when this PPI is published.

GUID

```
gAmdErrorLogAvailablePpiGuid =
    {0xb2b0fa81, 0x8b34, 0x4351,
     0xa3, 0x9c, 0x8f, 0x5a, 0x88, 0x60, 0x19, 0x47}}
```

PPI Interface Structure

```
typedef struct _PEI_AMD_ERROR_LOG_AVAILABLE_PPI {
    UINTN    Revision;          ///< Revision Number
} PEI_AMD_ERROR_LOG_AVAILABLE_PPI;
```

Parameters

Revision	Definition
-----------------	------------

Description This is a flag type PPI that is published by the module initialization process to indicate to other modules that the error log is ready to be acquired. Publication of this PPI indicates that all AGESA IP driver execution has completed. >

3.3.6 Error Log Services Protocol

Summary

This API is used to operate the logging services of the AGESA™ code base.

GUID

```

gAmdErrorLogServiceProtocolGuid =
    {0x7ef3f75c, 0xae1f, 0x46fc,
     {0x87, 0x86, 0xe, 0xb4, 0xf2, 0x81, 0xb2, 0x3d}}

```

Protocol Interface Structure

```

typedef struct _DXE_AMD_ERROR_LOG_SERVICES_PROTOCOL {
    UINTN Revision;
    AMD_AQUIRE_ERROR_LOG_DXE AmdAquireErrorLogDxe ;
    AMD_AQUIRE_ERROR_LOG_WITH_FLAG_DXE AmdAquireErrorLogWithFlagDxe;
}DXE_AMD_ERROR_LOG_SERVICES_PROTOCOL;

```

Members

Revision	Revision identifier for the protocol interface.
AmdAquireErrorLogDxe	Function for reading the error log contents.
AmdAquireErrorLogWithFlagDxe	Function for reading the error log contents with a flag to choose clear or keep the entries in the queue.

3.3.6.1 AmdAquireErrorLogDxe()

Summary

Function to use to read the entries in the error log.

Prototype

```

EFI_STATUS
EFIAPI
AmdAquireErrorLogDxe (
    IN DXE_AMD_ERROR_LOG_SERVICES_PROTOCOL *This,
    OUT ERROR_LOG_DATA_STRUCT *ErrorLogDataPtr
);

```

Parameters

This	UEFI standard pointer to the protocol block.
ErrorLogDataPtr	Pointer to a data block that will be filled with the error log entries. The caller is responsible for allocating a data buffer of the proper size to hold the maximum of entries.

Description

This function is used to read the error log entries. All entries in the log are returned for each call to this function. The entries are kept in a circular queue, so it is possible to have entries overwritten by new entries. As entries are read by this function, the entry is removed from the queue. It is the responsibility of the caller to preserve any information it wishes to process at a later time point.

Related Definitions

ERROR_LOG_DATA_STRUCT

```

typedef struct {

```

```
OUT    UINT32          Count;
OUT    ERROR_LOG_PARAMS ErrorLog_Param[TOTAL_ERROR_LOG_BUFFERS];
} ERROR_LOG_DATA_STRUCT;
```

Count Actual count of Error Log entries found in the log queue.

ErrorLog_Param This is an array of log entries containing data specific to the error.

ERROR_LOG_PARAMS

```
typedef struct {
OUT    UINT32          ErrorClass;
OUT    UINT32          ErrorInfo;
OUT    UINT32          DataParam1;
OUT    UINT32          DataParam2;
OUT    UINT32          DataParam3;
OUT    UINT32          DataParam4;
} ERROR_LOG_PARAMS;
```

ErrorClass The severity of this error, the values and meanings are aligned with the definition of AGESA_STATUS.

ErrorInfo The unique error identifier, zero means "no error".

DataParam 1,
DataParam 2,
DataParam 3,
DataParam 4 Data specific to the error. The error codes and parameter values are defined elsewhere in this specification.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.3.7 AmdAcquireErrorLogWithFlagDxe()

Summary

Function to use to read or inspect the entries in the error log.

Prototype

```
EFI_STATUS
EFIAPI
AmdAcquireErrorLogWithFlagDxe (
    IN    DXE_AMD_ERROR_LOG_SERVICES_PROTOCOL *This,
    OUT   ERROR_LOG_DATA_STRUCT               *ErrorLogDataPtr,
    IN    BOOLEAN                             ClearBuffer
);
```

Parameters

This UEFI standard pointer to the protocol block.

ErrorLogDataPtr Pointer to a data block that will be filled with the error log entries. The caller is responsible for allocating a data buffer of the proper size to hold the maximum of entries.

ClearBuffer Indicated the user's desire to erase or keep the entries being read.

- TRUE - Erase the entries as they are read.
- FLASE - Retain the error log. Nothing is erased.

Description

This function is used to read the error log entries. All entries in the log are returned for each call to this function. The entries are kept in a circular queue, so it is possible to have entries overwritten by new entries. As entries are read by this function, the entry is removed from, or kept in, the queue. It is the responsibility of the caller to preserve any information it wishes to process at a later time point.

Related Definitions

ERROR_LOG_DATA_STRUCT

```
typedef struct {
    OUT    UINT32      Count;
    OUT    ERROR_LOG_PARAMS ErrorLog_Param[TOTAL_ERROR_LOG_BUFFERS];
} ERROR_LOG_DATA_STRUCT;
```

Count

Actual count of Error Log entries found in the log queue.

ErrorLog_Param

This is an array of log entries containing data specific to the error.
This is the same structure used in the [Acquire Error Log Dxe](#) function.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.3.8 AMD_ERROR_LOG_SERVICE_READY_PROTOCOL

Summary

Indicates that the log service is available.

GUID

```
gAmdErrorLogServiceReadyProtocolGuid =
    {0x576732ae, 0xa8fd, 0x4ee2,
     {0xae, 0x42, 0x29, 0x8e, 0xcf, 0xd5, 0xad, 0x3a}}
```

Protocol Interface Structure

none

Description This is a flag type Protocol that is published by the module initialization process to indicate to other modules that the error log service is available.

3.3.9 AMD_ERROR_LOG_AVAILABLE_PROTOCOL

Summary

Indicates that the Error Log is available.

GUID

```
gAmdErrorLogAvailableProtocolGuid =
    {0x8444f699, 0xdb97, 0x4f27,
     {0xb1, 0x1, 0x3b, 0x7b, 0x88, 0x9a, 0xb9, 0xd8}}
```

Protocol Interface Structure

none

Description This is a flag type Protocol that is published by the module initialization process to indicate to other modules that the error log contents are available. All AMD drivers in DXE phase have completed when this Protocol is published.

3.3.10 AMD_ERROR_LOG_FULL_PROTOCOL

Summary

Indicates that the log queue is full.

GUID

```

gAmdErrorLogFullProtocolGuid =
{0x38c2bd90, 0x7bac, 0x47f2,
 0xb7, 0x25, 0x4, 0x9, 0xad, 0x7b, 0x8d, 0xe8}

```

Protocol Interface Structure

none

Description This is a flag type Protocol that is published by the module initialization process to indicate to other modules that the error log is full.

3.4 Memory Driver

Summary

This driver provides information about the memory found in the system and connects platform specific routines to the core procedures that perform the memory discovery and initialization.

The driver also provides an optional BIOS Memory Test (MBIST) to provide a set of hardware and software tools to perform a system memory test for AMD processor-based solutions. Testing can be divided into two main execution modes:

- DDR Interface Bus Test
- DDR Interface Connectivity Test

The supported tests leverage architectural awareness to stress the memory interface, detect errors, and identify failing components.

- Allows memory to be margined beyond failure without software crashing
- Allows multi-threading and increases testing bandwidth
- Leverages knowledge of hardware components
- Provides increased hardware control and event repeatability vs. software-based load stores
- Provides software flexibility for integration into platform BIOS

The MBIST test can be configured to perform certain tests via the APCB parameters, see [ABL MBIST](#) and the results can be obtained through the [MBIST PPI](#).

3.4.1 AMD_MEMORY_INFO_HOB_PPI

Summary

Indicator that the memory HOB has been published.

GUID

```
gAmdMemoryInfoHobPpiGuid =
{ 0xba16e587, 0x1d66, 0x41b7,
  { 0x9b, 0x52, 0xca, 0x4f, 0x2c, 0xad, 0xd, 0xc8 } };
```

PPI Interface Structure

```
typedef struct _AMD_MEMORY_INFO_HOB_PPI {
  UINT32    Revision;
} AMD_MEMORY_INFO_HOB_PPI;
```

Parameters

Revision

A number indicating the version of HOB that was published.

- 0x0001 - The system published the [AMD_MEMORY_INFO_HOB](#).
- 0x0400 - The system published the [AMD_MEM_CFG_INFO_HOB](#).

Description This is a flag type PPI used to indicate to the system that the memory HOB has been published. Once this PPI is published, a PIEM may then request the HOB from the PEI Services standard UEFI procedures.

3.4.1.1 AMD_MEMORY_INFO_HOB struct

The Memory Information HOB is generated by the memory discovery and initialization procedures and contains basic information about the memory configuration. A PPI is published to signal that it is available. The HOB can be located via the standard UEFI PEI Services. The GUID needed to locate the HOB and its structure are listed below.

GUID

```
gAmdMemoryInfoHobGuid =
{ 0x1bce3d14, 0xa5fe, 0x4a0b,
  { 0x9a, 0x8d, 0x69, 0xca, 0x5d, 0x98, 0x38, 0xd3 } }
```

AMD_MEMORY_INFO_HOB

```
typedef struct {
  UINT32    Version;
  BOOLEAN    AmdMemoryVddioValid;
  UINT16    AmdMemoryVddio;
  BOOLEAN    AmdMemoryVddpVddrValid;
  UINT8     AmdMemoryVddpVddr;
  BOOLEAN    AmdMemoryFrequencyValid;
  UINT32    AmdMemoryFrequency;
  UINT32    NumberOfDescriptor;
  AMD_MEMORY_RANGE_DESCRIPTOR  Ranges[1];
} AMD_MEMORY_INFO_HOB;
```

Version

Version of HOB structure. For this structure, the version value is =0x01.

AmdMemoryVddioValid

This field designates if the Vddio field is valid. Using a 'valid' boolean prevents use of a non-initialized value. The host BIOS must verify this flag before using the voltage value.

AmdMemoryVddio	Vddio Voltage used.
AmdMemoryVddpVddrValid	Designates if the VddpVddr voltage is valid. The host BIOS must verify this flag before using the voltage value.
AmdMemoryVddpVddr	VddpVddr voltage
AmdMemoryFrequencyValid	Designates if the Memory Frequency Valid. The host BIOS must verify this flag before using the Frequency value.
AmdMemoryFrequency	Memory Frequency.
NumberOfDescriptor	Number of descriptors in the following array.
Ranges	An array of Memory ranges descriptors.

Related Definitions

AMD_MEMORY_RANGE_DESCRIPTOR

```
typedef struct {
    UINT64 Base;
    UINT64 Size;
    UINT32 Attribute;
    UINT32 Reserved;
} AMD_MEMORY_RANGE_DESCRIPTOR;
```

Base	Base address of memory range.
Size	Size of memory range.
Attribute	Attribute of memory range. An Attribute indicates a type of range or characteristic of the block. Defined attributes include: • Available - This block is available for general use. • UMA - This block is reserved from the system and is assigned to the Graphics controller for its display buffer. • MMIO – This block is reserved from the system and available for MMIO resource mapping for the devices requiring MMIO address range below 4 GB. • Reserved - This block is reserved for the system. These areas will contain information such as the ACPI tables.

3.4.1.2 AMD_MEM_CFG_INFO_HOB struct

The Memory Configuration Information HOB is generated by the memory discovery and initialization procedures and contains the results of the configuration process. A PPI is published to signal that it is available. The HOB can be located via the standard UEFI PEI Services. The GUID needed to locate the HOB and its structure are listed below.

GUID

```
gAmdMemCfgInfoHobGuid = {0x13f03f37, 0xaf63, 0x4398,
                          {0x98, 0x4d, 0x54, 0xe2, 0xdf, 0xe7, 0x03, 0xa5}};
```

AMD_MEM_CFG_INFO_HOB

```
typedef struct _AMD_MEM_CFG_INFO_HOB {
    UINT32 Version;
```

```

/// Max. number of Socket/Die/Channel/Dimm supported.
UINT8  MaxSocketSupported;
UINT8  MaxDiePerSocket;
UINT8  MaxChannelPerDie;
UINT8  MaxDimmPerChannel;

/// Fixed Data
MEM_CFG_INFO_HOB MbistTestEnable;
MEM_CFG_INFO_HOB MbistAggressorEnable;
MEM_CFG_INFO_HOB MbistPerBitSlaveDieReport;
MEM_CFG_INFO_HOB DramTempControlledRefreshEn;
MEM_CFG_INFO_HOB UserTimingMode;
MEM_CFG_INFO_HOB UserTimingValue;
MEM_CFG_INFO_HOB MemBusFreqLimit;
MEM_CFG_INFO_HOB EnablePowerDown;
MEM_CFG_INFO_HOB DramDoubleRefreshRate;
MEM_CFG_INFO_HOB PmuTrainMode;
MEM_CFG_INFO_HOB EccSymbolSize;
MEM_CFG_INFO_HOB UEccRetry;
MEM_CFG_INFO_HOB IgnoreSpdChecksum;
MEM_CFG_INFO_HOB EnableBankGroupSwapAlt;
MEM_CFG_INFO_HOB EnableBankGroupSwap;
MEM_CFG_INFO_HOB DdrRouteBalancedTee;
MEM_CFG_INFO_HOB NvdimmPowerSource;
MEM_CFG_INFO_HOB OdtsCmdThrotEn;
MEM_CFG_INFO_HOB OdtsCmdThrotCyc;
/// Offsets
UINT16 DimmPresentMapOffset;
UINT16 ChipselIntlvOffset;
UINT16 DramEccOffset;
UINT16 DramParityOffset;
UINT16 AutoRefFineGranModeOffset;

// Dynamic (variant length) data arrays (appended following this structure)
/// Status reporting stuff
//UINT16 DimmPresentMap[MaxSocketSupported * MaxDiePerSocket];
//MEM_CFG_INFO_HOB ChipselIntlv[MaxSocketSupported * MaxDiePerSocket * MaxChannelPerDie];
//MEM_CFG_INFO_HOB DramEcc[MaxSocketSupported * MaxDiePerSocket];
//MEM_CFG_INFO_HOB DramParity[MaxSocketSupported * MaxDiePerSocket];
//MEM_CFG_INFO_HOB AutoRefFineGranMode[MaxSocketSupported * MaxDiePerSocket];
} AMD_MEM_CFG_INFO_HOB;

```

Most of the parameters used in this structure are identical to those listed for [AMD_MEMORY_INIT_COMPLETE_PPI](#). Please refer to that section for more details. Parameters unique to this structure are listed below.

Version

Version of HOB structure. For this structure, the version value is =0x0400.

DimmPresentMapOffset,
ChipselIntlvOffset,
DramEccOffset,
DramParityOffset,
AutoRefFineGranModeOffset

These entries are 'pointers' into the later section of the HOB. Since the target arrays are variant in length due to the topology of the system, these values indicate the offset from the beginning of the HOB where the array starts. The array format can be found in their name sakes in [AMD_MEMORY_INIT_COMPLETE_PPI](#).

Examples

Users can use the macro 'GET_HOB_DIMM_PRESENCE_OF', defined in AmdMemoryInfoHob.h, to obtain specific memory information, such as DIMM presence.

```

GET_MEM_CFG_INFO_HOB_ADDR macro prototype:
    GET_MEM_CFG_INFO_HOB_ADDR(HobStart, Offset)
GET_HOB_DIMM_PRESENCE_OF macro prototype:
    GET_HOB_DIMM_PRESENCE_OF(MemCfgInfoHob, DimmPresentMap, Socket, Die, Channel, Dimm) != 0)

```

Example: To test if Dimm0 of Socket 0, Die 1, Channel 2 is present, use:

```
Boolean DimmIsPresent;
```

```

UINT16 *DimmPresentMap;
DimmPresentMap = (UINT16 *) GET_MEM_CFG_INFO_HOB_ADDR (MemCfgInfoHob, DimmPresentMapOffset);
DimmIsPresent = GET_HOB_DIMM_PRESENCE_OF(MemCfgInfoHob, DimmPresentMap, 0, 1, 2, 0);

```

Example: To extract ECC information:

```

// Get Mem Config. Info. from HOB
HobData = GetFirstGuidHob(&gAmdMemCfgInfoHobGuid);
MemCfgInfoHob = (AMD_MEM_CFG_INFO_HOB *) GET_GUID_HOB_DATA(HobData);
// Get the address of Dram Ecc status and status code data
DramEcc = (MEM_CFG_INFO_HOB *) GET_MEM_CFG_INFO_HOB_ADDR (MemCfgInfoHob, DramEccOffset);
for (Socket = 0; Socket < MemCfgInfoHob->MaxSocketSupported; Socket++)
{
    for (Die = 0, Die < MemCfgInfoHob->MaxDiePerSocket; Die++)
    {
        Index = Socket * MemCfgInfoHob->MaxDiePerSocket *
            MemCfgInfoHob->MaxChannelPerDie + Die;
        // Print
        IDS_HDT_CONSOLE (MAIN_FLOW, "--DramEcc.Status.Enabled = %d (StatusCode = 0x%04x)\n",
            DramEcc[Index].Status.Enabled, DramEcc[Index].StatusCode);
    }
}

```

Related Definitions

MEM_CFG_INFO_HOB

```

typedef struct _MEM_CFG_INFO_HOB {
    union
    {
        {
            BOOLEAN    Enabled;
            UINT16     Value;
        }
        {
            Status;
            StatusCode;
        }
    } MEM_CFG_INFO_HOB;
}

```

See MEM_CFG_INFO structure defined in [AMD_MEMORY_INIT_COMPLETE_PPI](#) for more details.

3.4.2 mMemPmuTrainingFailureHobReady

Summary

Indicates that the PMU training results are available in the HOB.

GUID

```

gAmdMemPmuTrainingFailureHobReadyGuid =
{
    {0x21A0FD2D, 0xC959, 0x4BEC,
     {0x9D, 0x96, 0x6E, 0xF7, 0x26, 0xB7, 0xC1, 0xC9}}
};

```

PPI Interface Structure

```

STATIC EFI_PEI_PPI_DESCRIPTOR mMemPmuTrainingFailureHobReady =
{
    (EFI_PEI_PPI_DESCRIPTOR_PPI | EFI_PEI_PPI_DESCRIPTOR_TERMINATE_LIST),
    &gAmdMemPmuTrainingFailureHobReadyGuid,
    &mAmdMemPmuTrainingFailureHobReadyPpi
};

STATIC UINTN mAmdMemPmuTrainingFailureHobReadyPpi =
{

```



```

    0
};

```

Description

This is a flag type PPI used to indicate to the system that the PMU training process is complete and the gAmdMemPmuTrainingFailureHob has been published. Once this PPI is published, the user can use standard UEFI techniques to obtain the HOB location:

```

EFI_HOB_GUID_TYPE    *HobAddr;
HobAddr = GetFirstGuidHob (&gAmdMemPmuTrainingFailureHobGuid);

```

3.4.2.1 PMU_TRAINING_FAILURE_INFO_HOB

The PMU (PHY Management Unit) HOB is generated by the memory driver to contain the results of the PMU training process - a list of any failures. A PPI is published to signal that it is available. The HOB can be located via the standard UEFI PEI Services. The GUID needed to locate the HOB and its structure are listed below. This HOB is transferred to DXE phase and can also be accessed at that time.

GUID

```

gAmdMemPmuTrainingFailureHobGuid =
    {0x5EFAF260, 0x682B, 0x48E2,
     {0x94, 0xE0, 0xDB, 0xAC, 0x3C, 0xB8, 0xCC, 0x72}}
};

```

AMD_MEMORY_INFO_HOB

```

typedef struct {
    PMU_TRAINING_FAILURE_INFO_ENTRY  FailureEntry[80];
} PMU_TRAINING_FAILURE_INFO_HOB;

```

FailureEntry

An array of error records containing the PMU errors encountered. Scan the array starting at entry #0 until the first 'invalid' entry is found. An entry is 'valid' when the ErrorCode field is non-zero.

Related Definitions

AMD_MEMORY_RANGE_DESCRIPTOR

```

typedef struct {
    UINT32  SocketId:8;
    UINT32  UmcId:8;
    UINT32  TrainId2d:4;
    UINT32  TrainIdAttempt:4;
    UINT32  DramType:4;
    UINT32  Reserved:4;
    UINT32  ErrorCode;
    UINT32  Data[4];
} PMU_TRAINING_FAILURE_INFO_ENTRY;

```

SocketId

Socket index for the reporting component.

UmcId

Index of the Memory controller (UMC) reporting the error.

TrainId2d

Indicator if the type of training '1D' or '2D'.

Train1dAttempt	If PMU 1D training failed, PMU will retry four more times with different vref at CPU and DIMM side, Total five times. Train1dAttempt = 0 to 4 of the 1D training attempt.
DramType	Type of DRAM being trained. • 0 = RDIMM • 1 = for LRDIMM
ErrorCode	PMU error code. A value of zero indicated an invalid (unused) entry; e.g. the end of the list. For more information about these codes please contact your AMD representative.
Data	Error related data. Some error codes provide additional detail about the error in this array of 4 values.

3.4.3 Mem Configuration Items

Platform Category

The following Fixed PCDs have been defined starting with the release package for Family17h, Model 30h-3Fh. Since this package has backward support for the earlier model 00h-1Fh, there are two versions of each PCD (V1 & V2). These PCDs are presented to allow the platform to reduce the storage size required for the memory arrays. The default values match the architectural limits for the model and the platform should not use any larger value. When the platform is designed for a less-than-max configuration, these PCDs may be modified to reduce storage requirements.

- **PcdAmdMemMaxSocketSupportedV2 [F17M30][F19M00]**
Permitted Choices: (Type: Value)(Default: 2)
- **PcdAmdMemMaxDiePerSocketV2 [F17M30][F19M00]**
Permitted Choices: (Type: Value)(Default: 1)
- **PcdAmdMemMaxChannelPerDieV2 [F17M30][F19M00]**
Permitted Choices: (Type: Value)(Default: 8)
- **PcdAmdMemMaxDimmPerChannelV2 [F17M30][F19M00]**
Permitted Choices: (Type: Value)(Default: 2)
- **PcdAmdMemMaxSocketSupportedV1**
Permitted Choices: (Type: Value)(Default: 2 [F17M10])
- **PcdAmdMemMaxDiePerSocketV1**
Permitted Choices: (Type: Value)(Default: 4 [F17M10])
- **PcdAmdMemMaxChannelPerDieV1**
Permitted Choices: (Type: Value)(Default: 2 [F17M10])
- **PcdAmdMemMaxDimmPerChannelV1**
Permitted Choices: (Type: Value)(Default: 2 [F17M10])

Most Memory related items that are configurable by the host system are manifested as parameters to the ACPBTool and are described in [ABL Configuration](#). The below controls are used to describe operational choices.

NVDIMM controls

Full details of how the reset warning system works can be found in the [NVDIMM-N app note](#). General application steps are as follows:

1. Based on NVDIMM-N and platform design, select the NVDIMM-N power source by [APCB_TOKEN_UID_MEM_NVDIMM_POWER_SOURCE](#), at AGESA\AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\Include\ApcbCustomizedDefinitions.h
2. Based on platform design, select the FCH GPIO used by BIOS to trigger NVDIMM-N catastrophic save while cold/warm reset happening. See below.
3. The JedecNvdimmDxe driver installs a gEfiEventReadyToBootGuid callback. Near the end of POST the driver's callback will set the energy store policy and performs the Erase/Arm operation, preparing the system for the next NVDIMM-N save operation.
4. Enable trapping of CF9 warm and cold resets. See below. PcdNvdimmEnable must be set to TRUE on systems that support NVDIMM-N.

- **PcdNvdimmEnable [F17M30]**

Enables trapping of CF9 warm and cold resets. This control must be set to TRUE on systems that support NVDIMM-N.

Permitted Choices: (Type: Boolean) (Default: FALSE)

- FALSE - No reset trap. NVDimm save is disabled.
- TRUE - Trap the resets to perform NVDIMM save.

- **PcdNvdimmGpioForceSave [F17M30]**

This controls specifies the GPIO used by BIOS to trigger NVDIMM-N catastrophic save while cold/warm reset is happening (i.e. the GPIO used as the FSR_REQ signal).

Permitted Choices: (Type: value) (Default: 00)

- 00 - no GPIO is assigned.
- 01.. - This is the number of the GPIO that is assigned to the function. Example: value of 149(deciman) --> uses EGPI0149. Be careful of the decimal<->Hex conversions.

3.4.4 AMD_MEMORY_MBIST_TEST_RESULTS_PPI

Summary

This PPI signals that the MBIST HOB is ready for inspection and provides multiple access methods.

GUID

```
gAmdMbistPeiPpiGuid = {0x8eb674cd, 0xe982, 0x4089,
{ 0x9a, 0xcd, 0x95, 0x8, 0xc1, 0xa0, 0x9a, 0x8f}}
```

PPI Interface Structure

```
typedef struct _AMD_MEMORY_MBIST_TEST_RESULTS_PPI {
    UINT32      Revision;
    MBIST_TEST_STATUS      MbistTestStatus [MAX_SOCKETS][MAX_DIES_PER_SOCKET];
    MBIST_DATA_EYE      MbistDataEyeMargin[ MAX_SOCKETS][MAX_DIES_PER_SOCKET];
    PEI_GET_MBIST_TEST_RESULTS      GetMbistTestResults;
    PEI_GET_MBIST_DATA_EYE      GetMbistDataEyeMargin;
} AMD_MEMORY_MBIST_TEST_RESULTS_PPI;
```

Parameters

Revision	This identifies the version of the PPI output in order to maintain backward compatibility. Current PPI revision is 0x03.
MbistTestStatus	This stores the status outcome of the MBIST tests. One record for each die in the system, presented in a 2 dimensional array by socket# and die#.
MbistDataEyeMargin	This stores the measurement outcome of the MBIST Data Eye tests. One record for each die in the system, presented in a 2 dimensional array by socket# and die#.
GetMbistTestResults	This function can be called to extract the MBIST test results into a local buffer.
GetMbistDataEyeMargin	This function can be called to extract the MBIST Data Eye margining results into a local buffer.

Description

This is a flag type PPI used to indicate to the system that the memory MBIST results HOB has been published. Once this PPI is published, a PEIM may then use one of the access methods to evaluate the results. A DXE driver may choose to use the HOB directly by requesting the MBIST results HOB from the PEI Services using standard UEFI procedures. A PEIM may choose to use the function call to obtain a local copy or access the data from the PPI arrays.

Related Definitions

MBIST_TEST_STATUS

```
typedef struct _MBIST_TEST_STATUS {
    UINT8      ResultValid[MAX_MBIST_SUBTEST][MAX_CHANNELS_PER_DIE];
    UINT8      ErrorStatus[MAX_MBIST_SUBTEST][MAX_CHANNELS_PER_DIE];
} MBIST_TEST_STATUS;
```

This structure holds the results of the Interface Mode MBIST tests.

ResultValid	An array of flags indicating which bit in ErrorStatus is valid for each chip select, on each channel for each subtest. The corresponding chip select data is valid when the channel is active, has memory attached to that chip select, and the corresponding subtest has completed. Bits 3:0 correspond to chip select 3:0. Bits 7:4 are reserved. For each bit: 0 – Chip select was not tested. Results can be ignored. 1 – Chip select was tested. Results are valid.
--------------------	--

ErrorStatus

An Array of flags indicating the error status for each chip select for which testing is valid.

Bits 3:0 correspond to chip select 3:0. Bits 7:4 are reserved.

For each bit:

- 0 – Test passed.
- 1 – Test Failed on this chip select.

Subtest Indexes

The subtest field indicates the specific test to which the results apply:

- 0x00 - Single CS Read/Write Burst.
- 0x01 - Single CS Read/Write Burst with Turn around.
- 0x02 - Multiple CS Read Write Burst.
- 0x03 - Multiple CS Read Write Burst with Turn around.
- 0x04 - Address Line Connectivity Test.

MBIST_DATA_EYE

```
typedef struct _MBIST_DATA_EYE {
    MBIST_DATA_EYE_MARGIN_RECORD  MbistDataEyeRecord[MAX_CHANNELS_PER_DIE]
                                     [MAX_CHIPSELECTS_PER_CHANNEL];
} MBIST_DATA_EYE;
```

This structure holds the results of the Data Eye Mode MBIST tests.

MbistDataEyeRecord

An array of records that hold the margin results for each channel and chip select.

MBIST_DATA_EYE_MARGIN_RECORD

```
typedef struct _MBIST_DATA_EYE_MARGIN_RECORD {
    BOOLEAN          IsDataEyeValid;
    MBIST_DATA_EYE_MARGIN  DataEyeMargin;
} MBIST_DATA_EYE_MARGIN_RECORD;
```

IsDataEyeValid

Indicated if the Data Eye Record is Valid. Record entries may exist for channels that are not populated. 'Valid' records represent populated channels.

DataEyeMargin

Values that define the height and width of the data eye, aggregated for this chip select.

MBIST_DATA_EYE_MARGIN

```
typedef struct _MBIST_DATA_EYE_MARGIN {
    MBIST_MARGIN    ReadDqDelay;
    MBIST_MARGIN    ReadVref;
    MBIST_MARGIN    WriteDqDelay;
    MBIST_MARGIN    WriteVref;
} MBIST_DATA_EYE_MARGIN;
```

ReadDqDelay	Largest common Positive and Negative width across all data lines. All of the data lines are aggregated to find the largest possible common eye width. This also correlates to the data lane with the narrowest eye.
ReadVref	Largest common Positive and Negative Vref offset across all data lines. All of the Vref results are aggregated to find the largest common eye height.
WriteDqDelay	Largest common Positive and Negative width across all data lines.
WriteVref	Largest common Positive and Negative width across all data lines.

MBIST_MARGIN

```
typedef struct _MBIST_MARGIN {
    UINT8    PositiveEdge;
    UINT8    NegativeEdge;
} MBIST_MARGIN;
```

PositiveEdge	This is the positive margin for this element. This value is an integer representing the space to the edge of the data eye from the center line that was determined during memory training.
NegativeEdge	This is the negative margin for this element. This value is an integer representing the space to the edge of the data eye from the center line that was determined during memory training.

3.4.4.1 GetMbistTestResults()

Summary

Retrieve the MBIST results into the caller's local buffer.

Prototype

```
EFI_STATUS
(EFI_API * PEI_GET_MBIST_TEST_RESULTS) (
    IN CONST EFI_PEI_SERVICES    **PeiServices,
    IN UINT32                    *BufferSize,
    IN MBIST_TEST_STATUS          *MbistTestResults
);
```

Parameters

BufferSize	Size of buffer allocated by the caller to contain the output data.
MbistTestResults	Pointer to the buffer where the function is to store the outcome status of the MBIST tests.

Description

This function is used by the Platform BIOS to extract the MBIST test results for each channel with specific chip select and report it as per their platform policy.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_BUFFER_TOO_SMALL	The buffer provided by the caller is not large enough to contain the MBIST data. Please see the PPR for the installed processor to determine the correct size. It should be: sizeof (MBIST_TEST_STATUS) * MAX_SOCKETS * MAX_DIES_PER_SOCKET

3.4.4.2 GetMbistDataEyeMargin()

Summary

Retrieve the MBIST Data Eye margining results into the caller's local buffer.

Prototype

```
typedef EFI_STATUS (EFI_API * PEI_GET_MBIST_DATA_EYE) (
    IN CONST EFI_PEI_SERVICES **PeiServices,
    IN UINT32 *BufferSize,
    IN OUT MBIST_DATA_EYE *MbistDataEye
);
```

Parameters

BufferSize Size of buffer allocated by the caller to contain the output data.

MbistDataEye Pointer to the buffer where the function is to store the results of the MBIST Data Eye tests.

Description

This function is used by the Platform BIOS to extract the MBIST Data Eye margining results for each channel with specific chip select and report it as per their platform policy.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_BUFFER_TOO_SMALL	The buffer provided by the caller is not large enough to contain the MBIST data. Please see the PPR for the installed processor to determine the correct size. It should be: sizeof (MBIST_DATA_EYE_MARGIN_RECORD) * MAX_SOCKETS * MAX_DIES_PER_SOCKET

3.4.4.3 AMD_MEMORY_MBIST_HOB

Summary

This MBIST HOB provides the MBIST test results data for both the interface mode and the data eye mode test sets.

GUID

```
gAmdMemoryMbistHobGuid = { 0xfdd95c81, 0xea58, 0x4b51,
    { 0x99, 0x74, 0x82, 0xc1, 0x4a, 0x36, 0x55, 0xc7 } }
```

HOB Structure

MBIST HOB

```

typedef struct _AMD_MEMORY_MBIST_HOB {
    UINT8      NumSockets;
    UINT8      NumDiePerSocket;
    UINT8      NumSubTests;
    UINT8      NumChannelPerDie;
    MBIST_TEST_RESULTS MbstTestStatus [MAX_SOCKETS] [MAX_DIES_PER_SOCKET]
                                         [MAX_MBIST_SUBTEST] [MAX_CHANNELS_PER_DIE];
    MBIST_DEYE_MARGIN MbstDataEyeMargin [MAX_SOCKETS] [MAX_DIES_PER_SOCKET]
                                         [MAX_CHANNELS_PER_DIE] [MAX_CHIPSELECTS_PER_CHANNEL];
} AMD_MEMORY_MBIST_HOB;

```

<i>NumSockets</i>	It indicates the number of sockets supported on the system under test.
<i>NumDiesPerSocket</i>	Indicates the number of dies supported per socket on the system under test.
<i>NumSubTest</i>	Indicates the index of subtest to be executed on the system under test.
<i>NumChannelPerDie</i>	Indicates the number of channels per die supported on the system under test.
<i>MbstTestStatus</i>	A four dimensional matrix of the results.
<i>MbstDataEyeMargin</i>	A four dimensional array of data eye margin results.

MBIST Test Results

```

typedef struct _MBIST_TEST_RESULTS {
    UINT8      ResultValid;
    UINT8      ErrorStatus;
} MBIST_TEST_RESULTS;

```

<i>ResultValid</i>	An array of flags indicating which bit in ErrorStatus is valid for each chip select, on each channel for each subtest. The corresponding chip select data is valid when the channel is active, has memory attached to that chip select, and the corresponding subtest has completed. Bits 3:0 correspond to chip select 3:0. Bits 7:4 are reserved. For each bit: • 0 – Chip select was not tested. Results can be ignored. • 1 – Chip select was tested. Results are valid.
<i>ErrorStatus</i>	An Array of flags indicating the error status for each chip select for which testing is valid. Bits 3:0 correspond to chip select 3:0. Bits 7:4 are reserved. For each bit:

- 0 – Test passed.
- 1 – Test Failed on this chip select.

MBIST_DEYE_MARGIN

```
typedef struct _MBIST_DEYE_MARGIN {
    BOOLEAN      IsDataEyeValid;
    MBIST_HOB_MARGIN  ReadDqDelay;
    MBIST_HOB_MARGIN  ReadVref;
    MBIST_HOB_MARGIN  WriteDqDelay;
    MBIST_HOB_MARGIN  WriteVref;
} MBIST_DEYE_MARGIN;
```

<i>IsDataEyeValid</i>	Indicated if the Data Eye Record is Valid. Record entries may exist for channels that are not populated. 'Valid' records represent populated channels.
<i>ReadDqDelay</i>	Largest common Positive and Negative width across all data lines. All of the data lines are aggregated to find the largest possible common eye width. This also correlates to the data lane with the narrowest eye.
<i>ReadVref</i>	Largest common Positive and Negative Vref offset across all data lines. All of the Vref results are aggregated to find the largest common eye height.
<i>WriteDqDelay</i>	Largest common Positive and Negative width across all data lines. All of the data lines are aggregated to find the largest possible common eye width. This also correlates to the data lane with the narrowest eye.
<i>WriteVref</i>	Largest common Positive and Negative Vref offset across all data lines. All of the Vref results are aggregated to find the largest common eye height.

MBIST_HOB_MARGIN

```
typedef struct _MBIST_HOB_MARGIN {
    UINT8  PositiveEdge;
    UINT8  NegativeEdge;
} MBIST_HOB_MARGIN;
```

<i>PositiveEdge</i>	This is the positive margin for this element. This value is an integer representing the space to the edge of the data eye from the center line that was determined during memory training.
<i>NegativeEdge</i>	This is the negative margin for this element. This value is an integer representing the space to the edge of the data eye from the center line that was determined during memory training.

3.4.5 AMD_MEMORY_INIT_COMPLETE_PPI

Summary

A flag type PPI indicating the memory driver has completed and is providing results. This structure has changed as new features are added to new processor family members. Please refer to the Revision field.

GUID

```
gAmdMemoryInitCompletePpiGuid = {0xe54e6e84, 0x20b, 0x4081,
    {0x81, 0x56, 0x37, 0xe9, 0xc6, 0x2c, 0x1e, 0xd6}}
```

PPI Interface Structure

```
typedef struct _AMD_MEMORY_INIT_COMPLETE_PPI {
    UINT32      Revision;           // Revision = 0x0001
    UINT16      AmdBottomIo;
    UINT32      AmdMemoryBelow4gb;
    UINT32      AmdMemoryAbove4gb;
    UINT32      AmdMemoryBelow1Tb;
    UINT32      AmdTotalMemorySize;
    UINT32      HoleBase;
    UINT32      Sub4GCacheTop;
    UINT32      Sub1THoleBase;
    UINT32      SysLimit;
    UINT32      AmdMemoryFrequency;
    DIMM_VOLTAGE AmdMemoryVddIo;
    VDDP_VDDR_VOLTAGE AmdMemoryVddpVddr;
    AMD_MEMORY_UMA_INFO AmdGetUmaInfo;
    UINT32      DdrMaxRate;
    PEI_GET_SYSTEM_MEMORY_MAP GetSystemMemoryMap;
    PEI_GET_DDR_POST_PACKAGE_REPAIR GetDdrPostPackageRepairInfo;
    AMD_MEM_DIMM_SPD_DATA_STRUCT AmdDimmSpdInfo;
} AMD_MEMORY_INIT_COMPLETE_PPI;
```

```
typedef struct _AMD_MEMORY_INIT_COMPLETE_PPI {
    UINT32      Revision;           // Revision = 0x0002
    UINT16      AmdBottomIo;
    UINT32      AmdMemoryBelow4gb;
    UINT32      AmdMemoryAbove4gb;
    UINT32      AmdMemoryBelow1Tb;
    UINT32      AmdTotalMemorySize;
    UINT32      HoleBase;
    UINT32      Sub4GCacheTop;
    UINT32      Sub1THoleBase;
    UINT32      SysLimit;
    UINT32      AmdMemoryFrequency;
    DIMM_VOLTAGE AmdMemoryVddIo;
    VDDP_VDDR_VOLTAGE AmdMemoryVddpVddr;
    AMD_MEMORY_UMA_INFO AmdGetUmaInfo;
    UINT32      DdrMaxRate;
    PEI_GET_SYSTEM_MEMORY_MAP GetSystemMemoryMap;
} AMD_MEMORY_INIT_COMPLETE_PPI;
```

```
typedef struct _AMD_MEMORY_INIT_COMPLETE_PPI {
    UINT32      Revision;           // Revision = 0x0400
    UINT16      AmdBottomIo;
    UINT32      AmdMemoryBelow4gb;
    UINT32      AmdMemoryAbove4gb;
    UINT32      AmdMemoryBelow1Tb;
    UINT32      AmdTotalMemorySize;
    UINT32      AmdMemoryFrequency;
    DIMM_VOLTAGE AmdMemoryVddIo;
    VDDP_VDDR_VOLTAGE AmdMemoryVddpVddr;
    AMD_MEMORY_UMA_INFO AmdGetUmaInfo;
    UINT32      DdrMaxRate;
    PEI_GET_SYSTEM_MEMORY_MAP GetSystemMemoryMap;

    LIST_ENTRY   SpdDataListHead;

    /// Use below 4 items to be aware of number of Socket/Die/Channel/Dimm present.
    /// Those are filled in by the memory PEIMs.
    UINT8      MaxSocketSupported;
    UINT8      MaxDiePerSocket;
    UINT8      MaxChannelPerDie;
    UINT8      MaxDimmPerChannel;
```

```

/// Dynamic data
// Status reporting stuff
UINT16      *DimmPresentMap;
MEM_CFG_INFO *ChipselIntlv;
MEM_CFG_INFO *DramEcc;
MEM_CFG_INFO *DramParity;
MEM_CFG_INFO *AutoRefFineGranMode;

// Platform Tuning stuff
UINT8      *MemCtrllerProcOdtDddr4;
UINT8      *MemCtrllerRttNomDddr4;
UINT8      *MemCtrllerRttWrDddr4;
UINT8      *MemCtrllerRttParkDddr4;
UINT8      *MemCtrllerAddrCmdSetupDddr4;
UINT8      *MemCtrllerCsOdtSetupDddr4;
UINT8      *MemCtrllerCkeSetupDddr4;
UINT8      *MemCtrllerCadBusClkDrvStrenDddr4;
UINT8      *MemCtrllerCadBusAddrCmdDrvStrenDddr4;
UINT8      *MemCtrllerCsOdtCmdDrvStrenDddr4;
UINT8      *MemCtrllerCkeDrvStrenDddr4;

/// Fixed data
// Status reporting stuff
MEM_CFG_INFO MbistTestEnable;
MEM_CFG_INFO MbistAggressorEnable;
MEM_CFG_INFO MbistPerBitSlaveDieReport;
MEM_CFG_INFO DramTempControlledRefreshEn;
MEM_CFG_INFO UserTimingMode;
MEM_CFG_INFO UserTimingValue;
MEM_CFG_INFO MemBusFreqLimit;
MEM_CFG_INFO EnablePowerDown;
MEM_CFG_INFO DramDoubleRefreshRate;
MEM_CFG_INFO PmuTrainMode;
MEM_CFG_INFO EccSymbolSize;
MEM_CFG_INFO UEccRetry;
MEM_CFG_INFO IgnoreSpdChecksum;
MEM_CFG_INFO EnableBankGroupSwapAlt;
MEM_CFG_INFO EnableBankGroupSwap;
MEM_CFG_INFO DdrRouteBalancedTee;
MEM_CFG_INFO NvdimmPowerSource;
MEM_CFG_INFO OdtsCmdThrotEn;
MEM_CFG_INFO OdtsCmdThrotCyc;
} AMD_MEMORY_INIT_COMPLETE_PPI;

```

Parameters

Revision

Revision of the structure definition. This structure has changed as new features are added to new processor family members. Use this field to identify which structure is being presented.

- revision 0x0001 - was used by the Platform Initialization (PI) release packages supporting F17M00.
- revision 0x0002 - is used by PI packages supporting F17M08, F17M10 and F17M20.
- revision 0x0400 - is used by PI packages supporting F19M00, F17M30 and their backward compatible support for F17M00.

AmdBottomIo

Bottom of 32-bit IO space (8-bits).

AmdMemory Below 4gb

NV_BOTTOM_IO[7:0]=Addr[31:24]

AmdMemory Above 4gb

If not zero, the 32-bit top of cacheable memory.

AmdMemory Below 1Tb

If not zero, the 64-bit top of cacheable memory.

If not zero Base[47:16] (system address) of sub 1TB dram hole.

AmdTotal MemorySize

Limit[47:16] (system address).

AmdMemory Frequency

Memory clock

- DDR400_FREQUENCY
- DDR533_FREQUENCY
- DDR667_FREQUENCY
- DDR800_FREQUENCY
- DDR1066_FREQUENCY
- DDR1333_FREQUENCY
- DDR1600_FREQUENCY
- DDR1866_FREQUENCY
- DDR2100_FREQUENCY
- DDR2133_FREQUENCY
- DDR2400_FREQUENCY

AmdMemory VddIo

VDDIO for DIMM operation

- VOLT1_5
- VOLT1_35
- VOLT1_25
- VOLT1_2
- VOLT_UNSUPPORTED - No common voltage found.

AmdMemory VddpVddr

VDDP for DIMM operation.

AmdGetUmaInfo

UMA info

DdrMaxRate

The final maximum DRAM data transfer rate set for the current configuration.

GetSystem MemoryMap

Function to acquire a map of the system discovered memory.

GetDdrPostPackage RepairInfo

Function to acquire a pointer to the DDR4 POST Package Repair report. See the function definition below.

AmdDimmSpInfo

This parameter contains the memory SPD information that was discovered during the memory initialization. The SPD information is presented in an array, one element for each SPD. The structure header contains flag and address information about each SPD (e.g. DIMM) found in the system. These flags should be checked before using the data. To determine if a DIMM is present, check the entry in the structure for DimmPresent == TRUE.

Please note that the structure Socket, Channel and Dimm entries are logical numbers used by the memory initialization code and may not match the labels on the platform motherboard. To locate where the physical DIMM is installed in the system, first check that DimmPresent == TRUE, then use the "Address", "MuxChannel" and "MuxI2CAddress" to located a DIMM based on the platform schematics. This item is an UEFI LinkedList head that makes a node based data collection for the valid SPD Data records. Users can use the UEFI framework provided

SpdDataListHead

LinkedList library to get the SpdData as shown in the below code example:

```
if (!IsListEmpty (&MemInitCompleteData->SpdDataListHead))
{
    LIST_ENTRY *Node;
    AMD_MEM_DIMM_SPD_DATA_STRUCT *SpdData;

    Node = GetFirstNode (&MemInitCompleteData->SpdDataListHead);
    do
    {
        SpdData = (AMD_MEM_DIMM_SPD_DATA_STRUCT *)
Node;
        IDS_HDT_CONSOLE (MAIN_FLOW,
            "SPD data dump(Socket#%d::Channel#%d::Dimm#%d):\n",
            SpdData->DimmSpdInfo.SocketNumber,
            SpdData->DimmSpdInfo.ChannelNumber,
            SpdData->DimmSpdInfo.DimmNumber);
        DumpBuffer ((VOID *)SpdData->DimmSpdInfo.Data, 512);

        Node = GetNextNode (&MemInitCompleteData->SpdDataListHead, Node);
    } while (!IsNull (&MemInitCompleteData->SpdDataListHead, Node));
}
```

MaxSocketSupported

Indicates the number of sockets supported in this system. This is not an architectural design max, but the actual number used in this system. This, and the next three values, can be used for indexing into the memory status arrays. Examples provided below.

MaxDiePerSocket

Indicates the number of die per socket in this system.

MaxChannelPerDie

Indicates largest number of channels per die in the system.

MaxDimmPerChannel

Indicates largest number of DIMMs per channel in this system.

DimmPresentMap

An array of bit-packed values that records which memory DIMMs are presence in a system. Each value represents one die of the system and Bit[1:0] - Dimm[1:0] of Channel0, .. , Bit[15:14] - Dimm[1:0] of Channel7.

Users can use the macro "GET_DIMM_PRESENCE_OF", defined in AmdMemPpi.h, to get specific memory DIMM presence information.

```
GET_DIMM_PRESENCE_OF macro prototype:
GET_DIMM_PRESENCE_OF(This, DimmPresentMap,
    Socket, Die, Channel, Dimm)
```

The example: To get Dimm 0 presence of Socket 0, Die 1, Channel 2

```
DimmPresence =
GET_DIMM_PRESENCE_OF(MemInitCompleteData,
    MemInitCompleteData->DimmPresentMap,
    0, 1, 2, 0);
```

ChipselIntlv

An array indicating the resultant chipselect interleaving state and status for each channel, per Die, per Socket. ChipselIntlv[Socket][Die][Channel].

Note: For the meaning of the 'state and status', please refer to the definition of MEM_CFG_INFO below.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusOptionNotEnabled
- CfgStatusCsAddressingNotMatch
- CfgStatusDimmNotDetectedOrChannelDisabled
- CfgStatusNotPowerOf2EnabledCs

Associated AP CB token: [APCB_TOKEN_UID_ENABLECHIPSELECTINTLV](#)

DramEcc

An array indicating the resultant ECC feature state and status for each Die, per Socket. DramEcc[Socket][Die].

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusOptionNotEnabled

Associated AP CB token: [APCB_TOKEN_UID_ENABLEECCFEATURE](#)

DramParity

An array indicating the resultant ECC feature state and status for each Die, per Socket. DramParity[Socket][Die].

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusOptionNotEnabled
- CfgStatusRcdParityNotEnabled

Associated AP CB token: [APCB_TOKEN_UID_ENABLEPARITY](#).

AutoRefFine GranMode

An array indicating the resultant DIMM Sensor fine grain mode feature state and status for each Die, per Socket. AutoRefFineGranMode[Socket][Die].

Status type (value).

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride
- CfgStatusRefreshFix2XOnSelectiveODMDimms

Associated APCB token:

[APCB_TOKEN_UID_AUTOREFFINEGRANMOD](#).

MemCtrller ProcOdtDddr4

Programmed ProcOdt value during ABL initialization - a.k.a OdtDrvStrength

Status type (value: Bit[5:0]).

- 000_00_0 = High Impedance
- 000_00_1 = 480 ohms
- 000_01_0 = 240 ohms
- 000_01_1 = 160 ohms
- 001_00_0 = 120 ohms
- 001_00_1 = 96.0 ohms
- 001_01_0 = 80.0 ohms
- 001_01_1 = 68.6 ohms
- 011_00_0 = 60.0 ohms
- 011_00_1 = 53.3 ohms
- 011_01_0 = 48.0 ohms
- 011_01_1 = 43.6 ohms
- 111_00_0 = 40.0 ohms
- 111_00_1 = 36.9 ohms
- 111_01_0 = 34.3 ohms
- 111_01_1 = 32.0 ohms
- 111_11_0 = 30.0 ohms

MemCtrller RttNomDddr4

Programmed RttNom value during ABL initialization - a.k.a RttNom

Status type (value: Bit[2:0]).

- 0 0 0 - RTT_NOM Disable
- 0 0 1 - RZQ/4
- 0 1 0 - RZQ/2
- 0 1 1 - RZQ/6
- 1 0 0 - RZQ/1
- 1 0 1 - RZQ/5
- 1 1 0 - RZQ/3
- 1 1 1 - RZQ/7

MemCtrller RttWrDddr4

Programmed RttWr value during ABL initialization - a.k.a RttWr

Status type (value: Bit[2:0]).

MemCtrller RttParkDddr4

- 0 0 0 - Dynamic ODT Off
- 0 0 1 - RZQ/2
- 0 1 0 - RZQ/1
- 0 1 1 - Hi-Z
- 1 0 0 - RZQ/3
- 1 0 1 - Reserved
- 1 1 0 - Reserved
- 1 1 1 - Reserved

Programmed RttPark value during ABL initialization
- a.k.a RttPark

Status type (value: Bit[2:0]).

- 0 0 0 - RTT_PARK Disable
- 0 0 1 - RZQ/4
- 0 1 0 - RZQ/2
- 0 1 1 - RZQ/6
- 1 0 0 - RZQ/1
- 1 0 1 - RZQ/5
- 1 1 0 - RZQ/3
- 1 1 1 - RZQ/7

**MemCtrller
AddrCmdSetupDddr4**

programmed AddrCmdSetup value during ABL
initialization - a.k.a AddrCmdSetup.

Status type (value: 0...0x3F).

MemCtrller CsOdtSetupDddr4

Programmed CsOdtSetup value during ABL
initialization - a.k.a CsOdtSetup.

Status type (value: 0...0x3F).

MemCtrller CkeSetupDddr4

Programmed CkeSetup value during ABL
initialization - a.k.a CkeSetup.

Status type (value: 0...0x3F).

**MemCtrller
CadBusClkDrvStrenDddr4**

Programmed CadBusClkDrvStren value during ABL
initialization - a.k.a ClkStrength.

Status type (value: Bit[4:0]).

- 00000 = 120.0 Ohm
- 00001 = 60.0 Ohm
- 00011 = 40.0 Ohm
- 00111 = 30.0 Ohm
- 01111 = 24.0 Ohm
- 11111 = 20.0 Ohm

**MemCtrller
CadBusAddrCmdDrvStrenDddr4**

Programmed CadBusAddrCmdDrvStren value during
ABL initialization - a.k.a AddrCmdStrength.

Status type (value: Bit[4:0]).

- 00000 = 120.0 Ohm
- 00001 = 60.0 Ohm
- 00011 = 40.0 Ohm
- 00111 = 30.0 Ohm

MemCtrller**CsOdtCmdDrvStrenDddr4**

- 01111 = 24.0 Ohm
- 11111 = 20.0 Ohm

Programmed CsOdtCmdDrvStren value during ABL initialization - a.k.a CsOdtStrength.

Status type (value: Bit[4:0]).

- 00000 = 120.0 Ohm
- 00001 = 60.0 Ohm
- 00011 = 40.0 Ohm
- 00111 = 30.0 Ohm
- 01111 = 24.0 Ohm
- 11111 = 20.0 Ohm

MemCtrller CkeDrvStrenDddr4

Programmed CkeDrvStren value during ABL initialization - a.k.a CkeStrength.

Status type (value: Bit[4:0]).

- 00000 = 120.0 Ohm
- 00001 = 60.0 Ohm
- 00011 = 40.0 Ohm
- 00111 = 30.0 Ohm
- 01111 = 24.0 Ohm
- 11111 = 20.0 Ohm

MbistTestEnable

Final state and status of Mbist Test.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APB token: APB_TOKEN_CONFIG
_MEM_MBIST_TEST_ENABLE.

MbistAggressor Enable

Final state and status of Mbist Aggressor.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APB token: APB_TOKEN_CONFIG
_MEM_MBIST_AGGRESOR_ON.

MbistPerBit SlaveDieReport

Final state and status of Mbist Reporting.

Status type (BOOLEAN):

- TRUE : Enabled

***DramTempControlled
RefreshEn***

- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APCB token: APCB_TOKEN_CONFIG
_MEM_MBIST_PER_BIT_SLAVE_DIE_REPORT.
Final state and status of DRAM Temperature based
Refresh.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APCB token: [APCB_TOKEN_UID_
MEM_TEMP_CONTROLLED_REFRESH_EN](#).
Final state and status of User Timing Mode.

UserTimingMode

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusDdrRatePORLimited
- CfgStatusDdrRateUserTimingLimited
- CfgStatusDdrRateUserTimingSpecific
- CfgStatusDdrRateEnforcedDdrRate
- CfgStatusDdrRateMemoryBusFrequencyLimited

Associated APCB token: [APCB_TOKEN_UID_
USERTIMINGMODE](#).

UserTimingValue

Final state and status of User Timing Value.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusDdrRatePORLimited
- CfgStatusDdrRateUserTimingLimited
- CfgStatusDdrRateUserTimingSpecific
- CfgStatusDdrRateEnforcedDdrRate
- CfgStatusDdrRateMemoryBusFrequencyLimited

MemBusFreqLimit

Api_Mem_InitComplete.xml Associated APCB
token: [APCB_TOKEN_UID](#)
[_MEMCLOCKVALUE](#).

Final state and status of Memory Bus Frequency.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusDdrRatePORLimited
- CfgStatusDdrRateUserTimingLimited
- CfgStatusDdrRateUserTimingSpecific
- CfgStatusDdrRateEnforcedDdrRate
- CfgStatusDdrRateMemoryBusFrequencyLimited

Associated APCB token: [APCB_TOKEN_UID](#)
[_MEMORYBUSFREQUENCYLIMIT](#).

EnablePowerDown

Final state and status of Power Down feature.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APCB token: [APCB_TOKEN_UID](#)
[_ENABLEPOWERDOWN](#).

DramDouble RefreshRate

Final state and status of DRAM Double Refresh.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess

Associated APCB token: [APCB_TOKEN_UID](#)
[_DRAMDOUBLEREFRESHRATE](#).

PmuTrainMode

Final state and status of PMU training mode.

Status type (LIST)(Default: PMU_TRAIN_AUTO):

- PMU_TRAIN_1D - PMU 1D Training only.
- PMU_TRAIN_1D_2D_READ - PMU 1D and 2D Training read only.
- PMU_TRAIN_1D_2D_WRITE - PMU 1D and 2D Training write only.
- PMU_TRAIN_1D_2D - PMU 1D and 2D Training.

EccSymbolSize

- PMU_TRAIN_AUTO - Auto - PMU Training depends on configuration.

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APB token: [APCB_TOKEN_UID_PMUTRAINMODE](#).

Final state and status of the ECC symbol size.

Status type (Value):

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APB token: [APCB_TOKEN_UID_ECCSYMBOLSIZE](#).

UEccRetry

Final state and status of Uncorrectable ECC retries.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

Associated APB token: [APCB_TOKEN_UID_UECC_RETRY_DDR4](#).

IgnoreSpd Checksum

Final state and status of SPD checksum validation.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess

Associated APB token: [APCB_TOKEN_UID_IGNORESPDCHECKSUM](#).

EnableBank GroupSwapAlt

Final state and status of alternate Bank swapping.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride

EnableBank GroupSwap

Associated AP CB token: [APCB_TOKEN_UID_ENABLEBANKGROUPSWAPALT](#).

Final state and status of Bank swapping.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess
- CfgStatusCbsOptionOverride
- CfgStatusBankGroupSwapAltOverride

DdrRouteBalancedTee

Associated AP CB token: [APCB_TOKEN_CONFIG_ENABLEBANKGROUPSWAP](#).

Final state and status of DDR routing.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusSuccess

NvdimPowerSource

Associated AP CB token: [APCB_TOKEN_UID_DDDRROUTEBALANCEDTEE](#).

Final state and status of the power source for NVDIMMs.

Status type (Value):

- 1 - Device Managed Policy.
- 2 - Host Managed Policy.

Valid status code:

- CfgStatusSuccess

OdtCmdThrotEn

Associated AP CB token: [APCB_TOKEN_UID_MEM_NVDIMM_POWER_SOURCE](#).

Final state and status of DDR routing.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusOptionNotSupported

OdtCmdThrotCyc

Associated AP CB token: [APCB_TOKEN_UID_ODTSCMDTHROTEN](#).

Final state and status of DDR routing.

Status type (BOOLEAN):

- TRUE : Enabled
- FALSE : Disabled

Valid status code:

- CfgStatusOptionNotSupported

Associated APCB token: [APCB_TOKEN_UID_ODTSCMDTHROTCYC](#).

Description This is a flag type PPI, published by the memory initialization PEIM and also provides basic memory information.

Related Definitions

VDDP_VDDR_VOLTAGE

```
typedef struct _VDDP_VDDR_VOLTAGE {
    IN BOOLEAN IsValid;
    IN UINT8 Voltage;
} VDDP_VDDR_VOLTAGE;
```

IsValid

Indicates if data is valid.

Voltage

VDDP VDDR Voltage Value (VOLT1_05/ VOLT0_95)

AMD_MEMORY_UMA_INFO

```
typedef struct _AMD_MEMORY_UMA_INFO {
    UMA_MODE UmaMode;
    UINT32 UmaSize;
    UINT32 UmaBase;
} AMD_MEMORY_UMA_INFO;
```

This structure reports the location of the UMA block.

UmaMode

Uma Mode

- None
- Specified
- Auto

UmaSize

The size of shared graphics DRAM (16-bits)

NV_UMA_Size[31:0]=Addr[47:16]

UmaBase

Address where the UMA block starts.

AMD_MEM_DIMM_SPD_DATA_STRUCT

```
typedef struct {
    BOOLEAN AmdDimmSpdDataSupported;
    UINT8 Reserved[3];
    AMD_MEM_SPD_STRUCT DimmSpdInfo[AMD_MEM_PPI_MAX_SOCKETS_SUPPORTED *
                                    AMD_MEM_PPI_MAX_CHANNELS_PER_SOCKET *
                                    AMD_MEM_PPI_MAX_DIMMS_PER_CHANNEL];
```

```
} AMD_MEM_DIMM_SPD_DATA_STRUCT;
```

This structure holds the SPD data found in the system.

AmdDimmSpdDataSupported

Flag indicating that APOB DIMM SPD data is supported.

DimmSpdInfo

An array of structures containing the SPD information and contents as found in the system. The number of sockets, channels and DIMMs is specific to the CPU/APU product. Please refer to the PPR for the numbers.

AMD_MEM_DIMM_SPD_DATA_STRUCT_V2

```
typedef struct {
    LIST_ENTRY      Link;
    AMD_MEM_SPD_STRUCT DimmSpdInfo;
} AMD_MEM_DIMM_SPD_DATA_STRUCT_V2;
```

This new form uses the EFI Linked List library to hold the SPD information.

Link

Points to next node within the linked list.

DimmSpdInfo

The SPD data stored in this node.

AMD_MEM_SPD_STRUCT (revision=0x01)

```
typedef struct _AMD_MEM_SPD_STRUCT {
    UINT8    SocketNumber;
    UINT8    ChannelNumber;
    UINT8    DimmNumber;
    BOOLEAN  DimmPresent;
    UINT8    MuxPresent;
    UINT8    MuxI2CAddress;
    UINT8    MuxChannel;
    UINT32    Address;
    UINT32    SerialNumber;
    UINT8    Data[512];
} AMD_MEM_SPD_STRUCT;
```

AMD_MEM_SPD_STRUCT (revision=0x04)

```
typedef struct _AMD_MEM_SPD_STRUCT {
    UINT8    SocketNumber;
    UINT8    ChannelNumber;
    UINT8    DimmNumber;
    BOOLEAN  DimmPresent;
    UINT8    MuxPresent;
    UINT8    MuxI2CAddress;
    UINT8    MuxChannel;
    UINT32    Address;
    UINT32    SerialNumber;
    UINT8    Data[512];
} AMD_MEM_SPD_STRUCT;
```

This structure identifies a specific DIMM and reports its SPD content.

SocketNumber	Indicates the local socket number..
ChannelNumber	Indicates the local channel number
DimmNumber	Indicates the local DIMM number.
PageAddress	
DimmPresent	Indicates if the DIMM is present.
MuxPresent	Indicates if a I2C MUX is present.
MuxI2CAddress	Address of the I2C MUX on the I2C bus.
MuxChannel	Channel of the MUX on which this SPD is addressable.
Address	SMBus address of the DRAM SPD device.
SerialNumber	DIMM Serial Number as listed in the SPD.
Data	Buffer for 512 Bytes of SPD data from DIMM.

MEM_CFG_INFO

```
typedef struct _MEM_CFG_INFO {
    union
    {
        BOOLEAN      Enabled;
        UINT16       Value;
    }
    UINT16           Status;
    UINT16           StatusCode;
} MEM_CFG_INFO;
```

This structure is used to report the final state of a feature.

Status	This represents the final state of the feature after the ABL has finished its configuration process.
Enabled or Value	A feature may be a Boolean type or a value based type. Please refer to the associated APCB_TOKEN for feature settings.
StatusCode	The status code reports the result of the configuration process. This item will have one of the following values:

- CfgStatusSuccess - Indicating feature is enabled or configured value is set as the user desired.
- CfgStatusOptionNotEnabled - Indicating feature is not enabled by users input.
- CfgStatusDimmNotDetectedOrChannelDisabled - Indicating feature is not enabled due to Dimm not detected or Channel disabled.
- CfgStatusCbsOptionOverride - Indicating applicable CBS overriding happened.
- CfgStatusOptionNotSupported - Indicating feature control is not supported at this time.
- CfgStatusCsAddressingNotMatch - For chipselect interleaving, it indicates that interleaving is not enabled due to memory chipselect addressing does not match in the channel.
- CfgStatusNotPowerOf2EnabledCs - For chipselect interleaving, it indicates that interleaving is not enabled due to the number of enabled memory chipselects is not a power of 2.
- CfgStatusNotAllDimmEccCapable - For Ecc, it indicates that feature is not enabled because not all populated memory is ECC capable.

- CfgStatusRcdParityNotEnabled - For Rcd Parity, it indicates that feature is not enabled.
- CfgStatusRefreshFix2XOnSelectiveODMDimms - For DIMM Sensor fine grain mode, it indicates Refresh Fix 2x on selective memory.
- CfgStatusDdrRatePORLimited - For DDR rate, it indicates that DDR rate is limited by the DIMM population.
- CfgStatusDdrRateUserTimingLimited - For DDR rate, it indicates that the DDR rate is limited by the value specified by the user.
- CfgStatusDdrRateUserTimingSpecific - For DDR rate, it indicates that the DDR rate is forced to specific DDR rate specified by the user.
- CfgStatusDdrRateEnforcedDdrRate - For DDR rate, it indicates that the DDR rate is limited by the processor fusing.
- CfgStatusDdrRateMemoryBusFrequencyLimited - For DDR rate, it indicates that the DDR rate is limited by the maximum memory bus frequency supported by the processor.
- CfgStatusBankGroupSwapAltOverride - For Bank Group Swap enable, it indicates that the Bank Group Swap Enable function was replaced by the Bank Group Swap Alt setting.

3.4.5.1 PEI_GET_DDR_POST_PACKAGE_REPAIR() [F17M30]

Summary

Retrieve the DDR4 Post Package Repair Interface report.

Prototype

```
typedef EFI_STATUS (EFIAPI * PEI_GET_DDR_POST_PACKAGE_REPAIR) (
IN CONST EFI_PEI_SERVICES **PeiServices,
IN VOID **MemDdrPostPackageRepairPtr
);
```

Parameters

MemDdrPostPackageRepairPtr

Pointer to the repair report.

Description

The platform BIOS will locate “AMD_MEMORY_INIT_COMPLETE_PPI” and get the repair pointer. The pointer is the location of the output data structure APOB_DPPR_STRUCT which contains the repair output information from the ABL processing of the error list.

If the entries are valid, the platform BIOS can use the “APOB_DPPR_STRUCT. STRUCT.Channel” field to determine the repair results per channel. For each channel, the repair is defined with the same “Entry Number” provided in the APCB APCB_DPPRCL_REPAIR_ENTRY. Prior to consuming the information in the DPPRCL_REPAIR_REPORT_ENTRY, the platform BIOS must check the valid flag.

Note: Once the platform firmware has detected a successful hard PPR repair in the APOB, it should remove the associated repair from the repair list in the APCB.

Applicable to Family 17h Models 30h-3Fh.

Related Definitions

APOB_DPPR_STRUCT

```
typedef struct _APOB_DPPR_STRUCT {
    BOOLEAN PprResultsValid;
```

```

        UINT8
        UINT16
        APOB_DPPRCL_STRUCT
    } APOB_DPPR_STRUCT;
    Reserved;
    Reserved1;
    Channel[APOB_MAX_DPPRCL_ENTRIES];

```

PprResultsValid

Indicates if the PPR results are valid.

Channel

Array of report blocks, one per channel, indexed by the repair entry index.

APOB_DPPRCL_STRUCT

```

typedef struct _APOB_DPPRCL_STRUCT{
    DPPRCL_REPAIR_REPORT_ENTRY DppRclReportEntry[APOB_MAX_DPPRCL_ENTRY];
} APOB_DPPRCL_STRUCT;

```

DppRclReportEntry

Block of report entries for this channel. This is a fixed size block of entries.

DPPRCL_REPAIR_REPORT_ENTRY

```

typedef struct DPPRCL_REPAIR_REPORT_ENTRY {
    UINT32 Valid      :1;
    UINT32 Status     :8;
    UINT32 Type       :1;
    UINT32 Reserved   :22;
} DPPRCL_REPAIR_REPORT_ENTRY;

```

Short Description

Valid

Boolean flag designating a valid entry.

Status

- APOB_STATUS_REPAIR_FAIL - Repair failed
- APOB_STATUS_REPAIR_PASS - Repair was successful
- APOB_STATUS_REPAIR_RANK_MISS_MATCH_ERROR - Repair Error, Rank is valid, but did not match an installed rank
- APOB_STATUS_REPAIR_RANK_INVALID_ERROR - Repair Error, Rank is not valid
- APOB_STATUS_REPAIR_BANK_INVALID_ERROR - Repair Error, Bank is not valid
- APOB_STATUS_REPAIR_SOCKET_INVALID_ERROR - Repair Error, Socket is not valid
- APOB_STATUS_REPAIR_CHANNEL_INVALID_ERROR - Repair Error, Channel is not valid
- APOB_STATUS_REPAIR_DEVICE_INVALID_ERROR - Repair Error, Device is not valid
- APOB_STATUS_REPAIR_DEVICE_MISMATCH_ERROR - Repair Error, Device mismatch

Type

Type of repair made:

- APOB_DPPR_SOFT_REPAIR

- APOB_DPPR_HARD_REPAIR

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_UNSUPPORTED	This function is not supported by this version of the driver.

3.4.5.2 PEI_GET_SYSTEM_MEMORY_MAP()

Summary

Obtain a map of the system memory.

Prototype

```
typedef EFI_STATUS (EFI_API * PEI_GET_SYSTEM_MEMORY_MAP)
(
    IN CONST EFI_PEI_SERVICES    **PeiServices,
    IN UINT32                    *NumberOfHoles,
    IN UINT64                    *TopOfSystemMemory,
    IN MEMORY_HOLE_DESCRIPTOR    **MemHoleDescPtr
);
```

Parameters

PeiServices	A pointer to the PEI services.
NumberOfHoles	The number of holes in the system memory map.
TopOfSystemMemory	Top of system memory. This is the address of the last byte of physical memory.
MemHoleDescPtr	Pointer to an open ended array of MEMORY_HOLE_DESCRIPTORs.

Description

This procedure reports the size and 'shape' of the main system memory. Main system memory is assumed to start at physical address 0 and be contiguous through to the end of memory present with a few 'holes' cut into it for various compatibility needs. The MemHoleDesc array describes the holes.

Related Definitions

MEMORY_HOLE_DESCRIPTOR

```
typedef struct {
    UINT64          Base;
    UINT64          Size;
    MEMORY_HOLE_TYPES Type;
} MEMORY_HOLE_DESCRIPTOR;
```

Base	Full64 bit base address of the hole.
Size	Size in bytes of the hole.
Type	Hole type. • UMA - UC DRAM cycles. • MMIO - Cycles are sent out to IO. Only expect the 1 below 4GB. • PrivilegedDRAM - Read-only zero. No special cache considerations are needed. This region is not available to the Operating System.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_UNSUPPORTED	This function is not supported by this version of the driver.

3.5 Core Complex Driver

Summary

The Core Complex Driver (CCX) is responsible for starting the cores and configuring the power management systems.

Description

The Core Complex (CCX) driver is responsible for initializing the core silicon and providing some general services. There is one instance of this driver for each silicon family; so, you will see instances with minor name variances. While more than one instance may be present in the PEI Firmware Volume, the driver that pertains to the silicon family present in the platform will take control. The other(s) will abort during their module entry point.

The driver is comprised of two parts; one for PEI phase and one for the DXE phase. Each part will perform most of its actions during the module entry point at initial launch.

The PEI phase module will, as partial list:

- Perform early silicon initialization, setting specific register values.
- Load the core microcode.
- Establish the memory cache, set the MTRR cache regions.

The DXE phase module will, as a partial list:

- Prepare the Application Processor cores (APs) for execution. The standard UEFI MpServices may be used after this preparation is complete.
- Create ACPI table entries that pertain to the cores.
 - A power management related SSDT which is scoped into the platform BIOS declared processor objects.
 - P-state objects.
 - C-state objects.

Note: There are configuration items to control which objects are created, and to specify what the OEMID and OEMTableId used.

- SRAT table to declare the NUMA domains based on the level of DRAM interleaving employed.
- SLIT table to provide information on the time penalties associated with accessing memory on a different domain.
- proprietary extensions to the spec for HSA -
 - CRAT
 - CDIT

These provide similar information as the SRAT and SLIT, but provide more information about cache sharing.

- MSCT which provides information on the max amount of DRAM supported and information about the NUMA domains defined in the SRAT.
- Create SMBios table entries that pertain to the cores.

- type 4 (CPU info)
- type 7 (cache info)

Please note the configuration elements that pertain to these tables.

Produced PPIs

None.

Produced Protocols

DXE_AMD_MP_SERVICES_PREREQ_PROTOCOL

Optionally Produced Protocols

The SMM_ACCESS2 protocol is defined in the UEFI DXE (volume 4) specification. The CCX driver can be enabled to publish its own version of this protocol instead of relying on an IBV version. The CCX driver will check for the existence of a HOB of type EFI_SMRAM_HOB_DESCRIPTOR_BLOCK with a identifier matching gEfiSmmPeiSmramMemoryReserveGuid as defined by EdkCompatibilityPkg\Foundation\Framework\Guid\SmramMemoryReserve\SmramMemoryReserve.h. If this SMRAM HOB is present, then the CCX driver will publish its version of the SMM_ACCESS2 protocol.

3.5.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

3.5.2 CCX Configuration Items

These are driver level defined configuration items for the Core Complex (CCX). These will apply to ALL instances of a core complex.

3.5.2.1 CCX-Enablement Configuration Items

- **PcdAmdAgesaDmi**

The DMI table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiSlit**

The ACPI SLIT table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiSrat**

The ACPI SRAT table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiWhea**

The ACPI WHEA table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCrat**

The ACPI CRAT table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCdit**

The ACPI CDIT table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCpuSsdt**

The ACPI SSDT table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCpuSsdtPct**

The ACPI _PCT table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCpuSsdtPss**

The ACPI _PSS table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCpuSsdtXpss**

The ACPI XPSS table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCpuSsdtPsd**

The ACPI _PSD table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCpuSsdtPpc**

The ACPI _PPC table entry can be included or skipped in the output generation phase. This item specifies the platform preference for the production of this table.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced.
- FALSE - This table entry is skipped.

- **PcdAmdAcpiCstC1**

The ACPI _CST table entry can optionally include describing the C1 state via the monitor/mwait mechanism in addition to the C2 state in the output generation phase. This item specifies the platform preference for the

inclusion of this C1 state which has been shown to increase processor performance in certain operating systems.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This table entry will be produced if the hardware supports the feature.
- FALSE - This table entry is skipped.

• **PcdAmdEnableERMS [F19M00]**

This selects the state of the Enhanced Repeat Move String (ERMS) instruction capability in the processor. Generally, this should be left in its default state. See notes below and with the FSRM feature.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - The ERMS capability will be reflected in CPUID_Fn00000007_EBX_x00[9].
- FALSE - This instruction capability is disabled. Note: The FSRM=1 with ERMS=0 combination has been seen to cause a no-boot condition with some OS versions.

• **PcdAmdEnableFSRM [F19M00]**

This selects the state of the Fast Short Repeat Move String (FSRM) instruction capability in the processor. Generally, this should be left in its default state. See notes below and with the ERMS feature.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - The FSRM capability will be reflected in CPUID_Fn00000007_EDX_x00[4]. Note: FSRM is a new feature in [F19M00] and some OS implementations have a requirement that ERMS also be enabled when FSRM is enabled.
- FALSE - This instruction capability is disabled.

3.5.2.2 CCX-Platform Configuration Items

• **PcdAmdApicMode [F19M00]**

PcdAmdX2ApicMode [F17M30]

This control selects the Apic mode to be used. xAPIC mode supports upto 255 cores; x2 APIC mode supports much more than 255 cores. In this case, when selecting 'Auto', the system will choose the mode that best fits the number of active cores in the system. So, for a system with <= 255 cores, the xApic mode is chosen.

Permitted Choices: (Type: Value)(Default: Auto)

- Auto (0xFF)
- CompatibilityMode (0x00) - threads below 255 run in xAPIC with xAPIC ACPI structures and threads 255 and above run in x2 mode with x2 ACPI structures.
- xApicMode (0x01) - force legacy xApic mode. Note: If the system contains more than 255 active cores, then cores #256 - N will be disabled to allow xApic mode.
- x2ApicMode (0x02) - [F19M00] force x2Apic mode independent of thread count.

• **PcdAmdCcdMode**

This item is used to specify the number of CCDs that are desired to be enable in the system. Acceptable values are for this PCD are 0x01 through the max number available in the processor.

Permitted Values: (Type: Value)(Default: Auto)

- Auto(0x00) - The maximum CCDs provided by the processor will be enabled.

- $N \geq 1$ - The exact number of CCDs to be enabled in the system. If N is $< \text{Max}$ for the processor, the enabled CCDs will be the lowest order contiguous CCDs (#0..N).

- **PcdAmdDownCoreMode**

The ability to remove one or more cores from operation is supported in the silicon. It may be desirable to reduce the number of cores due to OS restrictions, or power reduction requirements of the system. This item allows the control of how many cores are running. This setting can only reduce the number of cores from those available in the processor. If the selection specifies more cores than are available in the processor, the setting is ignored.

Each core complex (a.k.a. Compute Unit) in the processor has two or more cores, each running 2 or more threads. The down-core option may be involved in two different ways:

- By_Complex - reductions are made first to reduce the number of core complexes. This is favorable for reducing the total power consumed.
- By_Core - reductions are made first to cores within a complex. This is favorable for maximizing performance by increasing the amount of L3 cache available to the remaining core(s).

In the option list, if the mode is not indicated, the default mode of operation is 'By_Core' reducing the number of Compute Units per complex first.

As new families of processors are introduced, the core architecture changes such that the option selection for this control changes to match the architecture. Please note the family-Model variations below.

Permitted Choices [F19M00]: (Type: List)(Default: AUTO) The '(x+y)' notation indicates the number of cores to remain in each CCX present. This is applied symmetrically to all sockets present.

- AUTO: CCX_DOWN_CORE_AUTO – No down coring. (All cores are enabled).
- TWO (2+0): CCX_DOWN_CORE_2_0 – Two cores remain, 2 on CCX0.
- THREE (3+0): CCX_DOWN_CORE_3_0 – Three cores remain, 3 on CCX0.
- FOUR (4+0): CCX_DOWN_CORE_4_0 – Four cores remain, 4 on CCX0.
- FIVE (5+0): CCX_DOWN_CORE_5_0 – Five cores remain, 5 on CCX0.
- SIX (6+0): CCX_DOWN_CORE_6_0 – Six cores remain, 6 on CCX0.
- SEVEN (7+0): CCX_DOWN_CORE_7_0 – Seven cores remain, 7 on CCX0.

Permitted Choices [F17M30]: (Type: List)(Default: AUTO) The '(x+y)' notation indicates the number of cores to remain in each CCX. This is applied symmetrically to all sockets present.

- Auto - no down coring is performed. All available cores are used.
- TWO (1 + 1) - Two cores remain after down coring, one in each CCX.
- FOUR (2 + 2) - Four cores remain, two in each CCX.
- SIX (3 + 3) - Six cores remain, 3 in each CCX.

Permitted Choices [F17M00][F17M10][F17M20]: (Type: List)(Default: DOWNCORE_NONE)

- DOWNCORE_NONE - Use all available cores
- DOWNCORE_SINGLE - just one core in one CU is left.
- DOWNCORE_DUAL - The lowest two cores in the lowest CU are left.
- DOWNCORE_TRI - the lowest 3 cores are left.
- DOWNCORE_FOUR - the lowest 4 cores are left.
- DOWNCORE_SIX - the lowest 6 cores are left.
- DOWNCORE_DUAL_BY_CORE - each CU is reduced in core count to leave 2 CUs each with one core.
- DOWNCORE_FOUR_BY_CORE - each CU is reduced in core count to leave 4 CUs each with one core.

• **PcdAmdAcpiS3Support**

This PCD is used to signal the system BIOS whether or not the installed processor in its current configuration can support the ACPI S3 sleep state. The PCD is declared at build time in a .DEC file. The CCX driver analyzes the current configuration during PEI phase and will reset this PCD to =FALSE if S3 cannot be supported. If S3 can be supported then the PCD is left in its original state.

The system BIOS should add this PCD to its criteria to determine whether or not it should enable the S3 feature. That evaluation is expected to occur in the DXE phase (after the EFI_ACPI_TABLE_PROTOCOL has been published).

- FALSE - The system should not be allowed to enter the S3 sleep state. The system BIOS should not declare the _S3 object to the operating system. Either the platform decided to not support S3 (set =FALSE in .DEC file) or the CCX driver determined the present configuration is not able to support the feature.
- TRUE - S3 is supported in the current configuration. The platform set the PCD to =TRUE in the .DEC file and it was not modified by the CCX driver.

• **PcdAmdAcpiTableHeaderOemId**

Set the OEM ID field in ACPI table outputs to this string. The string must conform to the ACPI rules for the OEM ID field.

Permitted Choices: (Type: Value)(Default: "AMD")

• **PcdAmdAcpiTableHeaderOemTableId**

Set the OEM TABLE ID field in ACPI table outputs to this string. The string must conform to the ACPI rules for the OEM TABLE ID field.

Permitted Choices: (Type: Value)(Default: "AmdTable")

• **PcdAmdAcpiCpuSsdtProcessorScopeInSb**

This element specifies whether the ACPI _PSS objects are defined in the system bus or processor scope.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - The objects will be under the _SB scope.
- FALSE - The objects will be under the _PR scope

• **PcdAmdAcpiCpuSsdtProcessorScopeName0**

PcdAmdAcpiCpuSsdtProcessorScopeName1

When creating the ACPI tables to describe the processor cores, the AGESA™ software will number the core sequentially from '00' to 'NN'. This leaves the first two characters of the ACPI name open for the customer to specify a desired prefix. These two elements set those two characters.

Permitted Choices:

- Scope name 0: (Type: Value/char)(Default: 'C'(0x43))
- Scope name 1: (Type: Value/char)(Default: '0'(zero)(0x30))

• **PcdAmdNumberOfPhysicalSocket**

This item informs the SMBios generation code as to how many physical processor sockets exist in the system and therefore how many Type 4 SMBios records to produce.

Permitted Choices: (Type: Value)(Default: 0x01)

- **PcdAmdSmbiosSocketDesignationSocket0**

PcdAmdSmbiosSocketDesignationSocket1

When creating the SMBios table entry, use this as the label for the processor socket. This should match the silkscreen label on the motherboard.

Permitted Choices: (Type: Value)(Default: "Unknown")

- **PcdAmdSmbiosSerialNumberSocket0**

PcdAmdSmbiosSerialNumberSocket1

When creating the SMBios table entry, use this as the value for the 'serial number' field for each processor.

Permitted Choices: (Type: Value)(Default: "Unknown")

- **PcdAmdSmbiosAssetTagSocket0**

PcdAmdSmbiosAssetTagSocket1

When creating the SMBios table entry, use this as the value for the 'Asset Tag' field for each processor.

Permitted Choices: (Type: Value)(Default: "Unknown")

- **PcdAmdSmbiosPartNumberSocket0**

PcdAmdSmbiosPartNumberSocket1

When creating the SMBios table entry, use this as the value for the 'Part Number' field for each processor.

Permitted Choices: (Type: Value)(Default: "Unknown")

- **PcdAmdCStateIoBaseAddress**

This item specifies a free block of 8 consecutive bytes of I/O ports that can be used to allow the CPU to enter C-States. This item should always be specified regardless of the C-State mode selected. The value of zero disables I/O C State transitions.

Permitted Choices: (Type: Value)(Default: 0x0413)

3.5.2.3 CCX-Power Management Configuration Items

- **PcdAmdCpbMode**

This item allows Core Performance Boost (CPB) to be forced to disabled, even if the hardware provides this feature. By default, CPB is enabled for processors that support it.

Permitted Choices: (Type: List)(Default: CpbModeAuto)

- CpbModeAuto - processors with CPB will have it enabled.
- CpbModeDisabled - force CPB to be disabled.

- **PcdAmdCStateMode**

This element specifies the C State operational mode. This can be used with processors which support deep sleep C states such as CC6.

Permitted Choices: (Type: List)(Default: 1)

- 0 - Disabled
- 1 - Use CState C6

• PcdAmdAgesaPstatePolicy

This element selects whether P-States should be forced to be independent, as reported by the ACPI _PSD object. This setting can improve performance for an OS which supports this feature. Different processors may have different preferences for this feature.

Permitted Choices: (Type: List)(Default: 0)

- 0 - PSD is reported as dependent or independent per processor preference.
- 1 - PSD will report dependent P-State control.
- 2 - PSD will report independent P-State control.

• PcdAmdPowerCeiling

Specifies a maximum power limit for the platform. This control is paired with PcdAmdAcpiCpuPerfPresentCap to control the value of the supported states reported in the ACPI PPC Table. The AGESA software will evaluate the defined P-States for the APU and trim off the top power states to achieve a P-State set where software P0 falls at or below the indicated power level.

This value is the integer number, in milliwatts, to be used as the TDP limit.

Permitted Choices: (Type: Value)(Default: 35000)

• PcdAmdAcpiCpuPerfPresentCap

Specifies the value of the highest power P-state to be listed in the ACPI PPC Table. This control is paired with, and simultaneously evaluated with PcdAmdPowerCeiling. It is limited by the value of PcdAmdPowerCeiling and the lowest power enabled P-state on the system. This control allows the user to separately restrict the PPC entries below what would be set by the PcdAmdPowerCeiling value.

This value is the P-State number to be used as the highest performance P-State as reported in the ACPI PPC table. This is a software P-State number as originally defined by the APU (e.g. before the power ceiling algorithm is applied).

Permitted Choices: (Type: Value)(Default: 0xFF)

3.5.2.4 CCX-Performance Configuration Items

The control of the prefetchers is a deep and complicated topic. Most platforms will not need to alter these controls from their default settings. Prefetcher tuning may be useful in improving overall performance for workloads with specific characteristics. The benefit of using a prefetcher is to have the code or data next to the current access fetched so as to have it available in cache for the next access, thus saving memory bus time. This presumes that accesses are in neighborhoods - e.g. code is sequential and/or data is in groups.

For example, workloads that do not have regular predictable patterns can benefit by turning off several of these prefetchers, where the prefetcher can cause additional stress on memory bandwidth and cache pollution (extra/ useless prefetches). Heavily random workloads with sparse data are a good example. Depending on the patterns, one or more of these controls will have varying affect as they have different affects on both bandwidth and what cache level (L1 or L2) might become polluted. In general, the L1 data prefetchers fetch lines into both the L1 data cache and the L2 cache, while the L2 data prefetchers fetch lines into the L2 cache.

Data Prefetcher	Description
L1 Stream	Uses history of memory access patterns to fetch additional sequential lines.
L1 Stride	Uses memory access history of individual instructions to fetch additional lines when each access is a constant distance from the previous.

Data Prefetcher	Description
L1 Region	Uses memory access history to fetch additional lines when the data access for a given instruction tends to be followed by other data accesses.
L2 Stream	Uses history of memory access patterns to fetch additional sequential lines.
L2 Stride	Uses memory access history of individual instructions to fetch additional lines when each access is a constant distance from the previous.
L2 Up/Down	Uses memory access history to determine whether to fetch the next or previous line for all memory accesses.

It is expected that anyone wishing to adjust these controls has a very good understanding of their workload and how the various prefetchers may affect the performance.

• **PcdAmdHardwarePrefetchMode**

This value provides for advanced performance tuning by controlling the hardware prefetcher setting. Use the recommended setting for best general performance, but this item can allow for optimizing specific workloads. The settings below are ordered from all enabled to all disabled and a specific disable setting implies all disable settings above it in the list as well. For example, disabling the L1 prefetcher implies that hardware prefetcher training on software prefetches is also disabled.

Permitted Choices: (Type: List)(Default: `HARDWARE_PREFETCHER_AUTO`)

- `HARDWARE_PREFETCHER_AUTO` - Use the recommended setting for the processor. In most cases, the recommended setting is enabled.
- `DISABLE_HW_PREFETCHER_TRAINING_ON_SOFTWARE_PREFETCHES` - Use the recommended setting for the hardware prefetcher, but disable training on software prefetches.
- `DISABLE_L1_PREFETCHER` - Use the recommended settings for the hardware prefetcher, but disable L1 prefetching and above.
- `DISABLE_L2_STRIDE_PREFETCHER` - Use the recommended settings for the hardware prefetcher, but disable the L2 stride prefetcher and above.
- `DISABLE_HARDWARE_PREFETCH` - Disable hardware prefetching.

• **PcdAmdSoftwarePrefetchMode**

This value provides for advanced performance tuning by controlling the software prefetch instructions. Use the recommended setting for best general performance, but this item can allow for optimizing specific workloads.

Permitted Choices: (Type: List)(Default: `SOFTWARE_PREFETCHES_AUTO`)

- `SOFTWARE_PREFETCHES_AUTO` - Use the recommended setting for the processor. In most cases, the recommended setting is enabled.
- `DISABLE_SOFTWARE_PREFETCHES` - Disable software prefetches (convert software prefetch instructions to NOP).

• **PcdAmdL1StreamPrefetcher**

See the section [above](#) for discussion on prefetchers.

Permitted Choices: (Type: Boolean)(Default: `TRUE`)

- `FALSE` - The prefetcher is disabled.
- `TRUE` - This prefetcher is running.

• **PcdAmdL1StridePrefetcher**

See the section [above](#) for discussion on prefetchers.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - The prefetcher is disabled.
- TRUE - This prefetcher is running.

- **PcdAmdL1RegionPrefetcher**

See the section [above](#) for discussion on prefetchers.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - The prefetcher is disabled.
- TRUE - This prefetcher is running.

- **PcdAmdL2StreamPrefetcher**

See the section [above](#) for discussion on prefetchers.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - The prefetcher is disabled.
- TRUE - This prefetcher is running.

- **PcdAmdL2UpDownPrefetcher**

See the section [above](#) for discussion on prefetchers.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - The prefetcher is disabled.
- TRUE - This prefetcher is running.

- **PcdAmdEnSpecStFill [F19M00]**

This control adjusts the performance of the speculative operations and changes the behavior of memory write operations. This may have performance benefits for a few very specialized workloads. The user is cautioned to use this with care.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - Use this setting for workloads that have multiple threads that are expected to update memory locations in close proximity temporally and spatially.
- TRUE - This setting gives the best performance for most workloads.

- **PcdAmdSmtMode**

This item selects the Symmetric Multi-Threading (SMT) mode of the Compute Units.

Permitted Choices: (Type: List)(Default: SMT_Auto)

- SMT_Disabled - the compute units are restricted to a single thread.
- SMT_Auto - the compute unit use all available threads.

- **PcdAmdFabricCcxAsNumaDomain**

Creates a layer of virtual domains on top of the physical domains in which each CCX is declared to be in its own domain.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Each CCX is declared to be in its own domain.

- FALSE - Use NPS settings for domain configuration.

- **PcdAmdFabricRoundRobinNumaDomainForCcx**

if PcdAmdFabricCcxAsNumaDomain is TRUE, this specifies if a round-robin scheme to return a single NUMA node from the list of nodes associated with a given Quadrant is used. See [NumaProxInfo\(\)](#).

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Use the Round-Robin scheme and report one node at a time.
- FALSE - Report all nodes at one time.

- **PcdAmdFabricSratBelow1MBReportingMode**

Historically, the System Resource Affinity ACPI Table (SRAT) has broken up the memory reported below 4GB such that the space from 0xA0000-0xFFFFF is not reported as DRAM. This item is used to control whether or not this region is declared as DRAM.

Permitted Choices: (Type: List)(Default: 0xFF)

- 0x00 - Do not report 0xA0000-0xFFFFF as DRAM (historical behavior)
- 0x01 - Report 0xA0000-0xFFFFF as DRAM
- 0xFF - Auto (will typically act as 0x00)

3.5.3 AMD_ENABLE_UEFI_STACK2

Summary

Establishes a stack during the SEC phase.

Prototype

```
OEM_SEGMENT1_START
;EnableStack      ; Host Environment Procedure
; Input: EBX - Return address
EnableStack PROC FAR PUBLIC
    AMD_ENABLE_UEFI_STACK2 STACK_AT_BOTTOM STACK_SIZE_64K
    jmp ebx ;Return to caller
EnableStack ENDP
OEM_SEGMENT1_END
```

Parameters

StackPosition

Specifies the position of the stack allocations within the memory zone.

- STACK_AT_BOTTOM - (Default) the stacks will be placed at the bottom, lower addresses, of the zone.
- STACK_AT_TOP - the stacks will be placed at the top, higher addresses, of the zone.

BspStackSize

Sets the size of the memory region allocated for data (e.g. stack/heap use).

- STACK_SIZE_64K - allocates 64K of data space (Default).
- STACK_SIZE_128K - allocated 128K of data space.
- STACK_SIZE_192K - allocated 192K of data space.
- STACK_SIZE_256K - allocated 256K of data space.
- STACK_SIZE_1M - allocated 1M of data space.

BspStackBase

NOTE: BspStackSize must be the same as the one declared in PspPlatformDriver.c (RESUME_BSP_STACK_SIZE)

Specifies the location where the stack should be allocated within the memory zone. Locations must be below 640K or above 1MB.

- STACK_AT_BOTTOM - (Default) the stacks will be placed at the bottom, lower addresses, of the zone.
- STACK_AT_TOP - the stacks will be placed at the top, higher addresses, of the zone.

EBX IN

Return Address - where to jump after macro completion. This register is preserved and can be used by the host BIOS for retaining a value of their preference.

SS:ESP OUT

Points to the private stack location for this processor core.

EAX OUT

Contains the AGESA_STATUS return code.

ECX OUT

Upon success, contains this processor core's stack size in byte.

EDX OUT

Contains the event sub-class for status return codes of higher severity than AGESA_SUCCESS.

Description

This procedure is native assembler source file and is “stackless” in its operation. Therefore the interface uses a register passing model.

The purpose is to initialize the processor and cache system to establish a viable stack region prior to main memory being available. The available pre-memory stack region is divided among the processor cores according to application needs. It then maps them to global address space and sets the processor MTRRs accordingly.

The procedure checks whether protected mode is enabled and if so, does not modify the Stack Segment register (SS). In this case the host environment must point SS to a GDT descriptor with base address = 0x0000_0000 before entering this procedure. The Stack Pointer (ESP) is set to contain a 32-bit memory offset. If Real Mode is being used, SS:ESP is set to point to a 16-bit execution-compatible Segment:Offset format. The upper 16 bits of ESP will be zeroes.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
AGESA_WARNING	CPU_EVENT_STACK_REENTRY - A stack is in use from a previous invocation of AMD_ENABLE_UEFI_STACK2. The stack will be reset to empty.
AGESA_FATAL	CPU_EVENT_UNKNOWN_PROCESSOR_FAMILY - The stack cannot be created because the processor family is unknown.
	CPU_EVENT_STACK_BASE_OUT_OF_BOUNDS - The base address specified is out of bounds
	CPU_EVENT_STACK_SIZE_INVALID - The stack size specified is invalid. 1MB stack size is only available for systems where DRAM is available in SEC phase. This excludes the Family 15h products.

3.5.4 AMD_DISABLE_UEFI_STACK2**Summary**

A macro used to tear down the stack.

Prototype

```
OEM_SEGMENT2_START
;DisableStack ; Host Environment Procedure
```



```

; Input: EBX - Return address
DisableStack PROC FAR PUBLIC
    AMD_DISABLE_UEFI_STACK2
    jmp ebx
DisableStack ENDP
OEM_SEGMENT2__END

```

Parameters

EAX OUT Contains the AGESA_STATUS return code

Description

This procedure is native assembler source file and is “stackless” in its operation. Therefore the interface uses a register passing model.

The purpose is to return the processor and cache system to a non-stack-enabled state.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.5.5 DXE_AMD_CCX_MP_SERVICES_PREREQ_PROTOCOL**Summary**

Protocol to indicate that the pre-requisites for the MP Services driver have been met.

GUID

```

gAndMpServicesPreReqProtocolGuid = {0x33f4458, 0x9c78, 0x42be,
    { 0x8e, 0x3d, 0x7b, 0xc6, 0x34, 0xb9, 0x5d, 0xe }}

```

Protocol Interface Structure

```

typedef struct _DXE_AMD_MP_SERVICES_PREREQ_PROTOCOL {
    UINTN    Revision;
} DXE_AMD_MP_SERVICES_PREREQ_PROTOCOL;

```

Members

Revision Current revision for the protocol.

Description The MP Services PreReq protocol is a flag type protocol that indicates to the host BIOS that the necessary initialization to allow the AP interprocessor interrupts to function properly is complete. The platform BIOS should add the MP Services PreReq guid to the DEPEX list of the MP Services driver.

3.6 NBIO Driver**Summary**

The NBIO Driver is responsible for the PCIe® Root Complex configuration.

Description

The NBIO Driver is comprised of several sub-components that are responsible for configuration of the northbridge IO related functions, including:

- Distributed IO crossbar (DXIO)

The high speed IO ports in the NorthBridge are capable of driving one of several types of physical connections. Each NB port can be configured to drive:

- PCIe® Root Complex
- SATA/SATA Express physical connection
- XGBe, GBe physical connection

This component is responsible for configuring the DXIO subsystem, the physical connections for devices that share the PHY connections, and internal GMI or GOP (GMI Over PCIe®) connections. The layout and assignment of the ports and devices is imported from the platform driver via the Root Complex PPI.

- IOMMU

The NBIO driver provides low level configuration and enablement of the IOMMUs, and provides the IVRS tables in ACPI to describe the IOMMUs to ACPI aware systems. The platform BIOS or PciHostBridge driver is responsible for IOMMU MMIO address initialization.

- System power, temperature and cooling management

The System Management component provides the interfaces for communicating with the platform driver for establishing power-related platform parameters. In addition to the platform power limits, there is a cooling fan controller using pulse-width modulation (PWM). There are multiple temperature set points (Low, Medium, High, and Critical) with configurable PWM settings for the fan speed at each temperature threshold. These values are specified using PCD parameters.

- NBIF bus interface to NBIO components

This component initializes the device registers for services used for internal communications.

3.6.1 NBIO Configuration Items

User controls provided for the customization of the driver for the specific platform and intended operations.

3.6.1.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

3.6.1.2 Read Only Elements

The following PCD items are for use by the system BIOS to know some default fuse settings of the processor currently present in the socket. Modifying these PCDs will have no effect in the system. They are for Read Only use.

- **PcdDxioMajorRevision [RO]**

- **PcdDxioMinorRevision [RO]**

These values are set by the AGESA™ to report to the platform BIOS the revision of DXIO firmware that is present in the system. This value is Read-Only, populated AGESA for consumption by platform BIOS.

Writes to this item will have no affect.

Permitted Choices: (Type: Value - [RO])

- **PcdAmdDefaultsocketTDP [F17M60][F19M50][RO]**

This value specifies the 'Default socket TDP' which is the fused value for the part. This value is set by the AGESA™ software to report the value to the platform BIOS. The PCD is declared as a 32-bit value with only the lower 16-bits being valid representing the default socket TDP, in Watts.

- **PcdAmdDefaultTJMax [F17M60][F19M50][RO]**

This value specifies the 'Maximum ASIC hotspot operating temperature' which is the fused value for the part. This value is set by the AGESA™ software to report the value to the platform BIOS. The PCD is declared as a 32-bit value with only the lower 16-bits being valid representing the Maximum ASIC hotspot operating temperature, in degrees Celcius.

3.6.1.3 Enablement Category

NBIO settings to select which features are to be enabled for this platform.

- **PcdIvrsControl**

This value controls whether AGESA will publish the IVRS table for IOMMUs.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - AGESA will publish the IVRS if IOMMUs are enabled.
- FALSE - No IVRS table will be published.

- **PcdAmdPcieSubsystemDeviceID**

This 16-bit value defines the subsystem device ID assigned to PCIe® bridges for Family 17h processors.

Permitted Choices: (Type: Value)(Default: 0x1453)

- **PcdAmdPcieSubsystemVendorID**

This 16-bit value defines the subsystem vendor ID assigned to PCIe® bridges for Family 17h processors.

Permitted Choices: (Type: Value)(Default: 0x1022)

- **PcdCfgACSEnable**

This value controls whether AGESA will enable Access Control Services (ACS). The PCIe® specification recommends that Advanced Error Reporting (AER) be enabled when ACS is enabled. For those platforms that support the [RAS](#) feature [F17M00][F17M30][F17M60], this recommendation will be enforced with a dependency that ACS cannot be enabled without AER also being enabled. On these platforms even though this PCD is set =TRUE, the code will decide to not enable ACS if [PcdCfgAEREnable](#) is not set =TRUE also.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - AGESA will enable ACS.
- FALSE - ACS option is turned off.

- **PcdCfgHdAudioEnable**

Select whether or not the NBIO High Definition (HD) audio Device is active.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active

- FALSE - This option is turned off

• PcdCfgPcieAriSupport

This value controls whether AGESA will enable Alternative Routing-ID Interpretation (ARI) support for PCIe® root ports and for any endpoints that support ARI. ARI devices interpret the PCI address as an 8-bit function number instead of a 3-bit function number and the device number is implied to be 0.

For Single-Root I/O Virtualization (SR-IOV) support this PCD should be set to TRUE. SR-IOV is a specification that allows a physical PCIe® device to be shared among different virtual machines in a virtual environment. The scope of SR-IOV goes beyond ARI, but ARI is a required feature for SR-IOV support. Please refer to the PCIe® specifications for more detail.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - AGESA will enable ARI in root ports and endpoints that support it. Note that this does not guarantee ARI will be active; the endpoint must also support ARI.
- FALSE - ARI support will remain disabled regardless of end-point support.

• PcdPcieAriForwardingEnable

This value determines if AGESA will enable Alternative Routing-ID Interpretation (ARI) Forwarding for each downstream port. The PCIe specification recommends that ARI Forwarding only be enabled by BIOS when an OS is present that supports ARI. This option allows for manual enable or disable of the feature by the platform owner.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - ARI Forwarding is enabled.
- FALSE - ARI Forwarding is disabled.

• PcdCfgPcieLoopbackMode [F17M00] PcdAmdLcLoopbackWaitForAllActiveLanes [F17M30] [F17M60][F19M00]

This configuration option enables a user to prevent the root port from entering loopback. For some use cases it is useful to prevent the root port from entering loopback mode.

Permitted Choices: (Type: List)(Default: Auto)

- Auto - AGESA will prevent the root port from entering loopback mode.
- FALSE - AGESA will prevent the root port from entering loopback mode.
- TRUE – AGESA will allow the root port to enter loopback mode.

• PcdCfgLcSchedRxEqEvalInt

This enables an optional mode during PCIe® initialization when switching to Gen 3 link speeds that has been shown to improve training results in certain configurations. Please consult your AMD representative before using any value other than the default.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0 – Disables this feature.
- 1 – Enables this feature.
- >1 - reserved.

• PcdPcieLinkComplianceModeAllPorts

Setting this control to TRUE will unconditionally put all PCIe® ports into Compliance mode.

Permitted Choices: (Type: BOOLEAN)(Default: FALSE)

- **PcdAmdDlSfcEn**

This Boolean enables the Data Link Feature (DLF) and Scalable Flow Control (SFC) on all Gen4 PCIe® ports.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE – DLF and SFC are turned off.
- TRUE – DLF and SFC are enabled.

- **PcdAmdRxMarginEnabled**

This Boolean is the control for receiver margin enablement.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE – RX Margin Feature is disabled
- TRUE – RX Margin Feature is enabled

- **PcdPcieRxMarginPersistenceAllPort**

When set, this value will be used as a global override for all ports. The PCDs are intended as a convenience for certain test cases when it can be useful to set all the ports at once. Setting this field, or the DXIO descriptors for 'Enable' may be useful when a re-timer is not present.

Permitted Choices: (Type: Value)(Default: Auto)

- 0: Disable - set all ports to disabled.
- 1: Enable - set all ports to enabled.
- 0xFF: Auto - Skip this override setting. The normal "per port" configuration for all ports in the DXIO descriptors config value that was populated by the customer will be the value used in AGESA.

- **PcdPcieLinkAspmAllPort**

When set, this value will be used as a global override for all ports. This PCD is intended as a convenience for certain test cases when it can be useful to set all the ports at once.

Permitted Choices: (Type: Value)(Default: Auto)

- 0: Disable - set all ports to disabled.
- 1: L0s - set all ports to use L0s.
- 2: L1 - set all ports to use L1s.
- 3: L0sL1 - set all ports to use L0sL1.
- 0xFF: Auto - Skip this override setting. The normal "per port" configuration for all ports in the DXIO descriptors config value that was populated by the customer will be the value used in AGESA.

- **PcdAmdNbioReportEdbErrors**

This value enables reporting of EDB errors from the PCIe® controller.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE – EDB errors are not reported by the hardware.
- TRUE – Bad EDB tokens are reported by the hardware.

- **PcdCfgPcieTbtSupport [F17M30]**

Control to support PCIe® 10-bit tag capability.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - AGESA will enable 10-bit tag for devices that indicate support.
- FALSE - 10-bit tag support is disabled.

• **PcdCfgCPPCMode (F17M30)**

Collaborative Processor Performance Control (CPPC) is a new feature added to ACPI 6.1 that allows the OS to more closely control the core(s) performance. Further details for this feature can be found in [ACPI specification](#) and [AMD CPPC overview](#). This control allows the platform to control the state of the feature.

Permitted Choices: (Type: Boolean)(Default: 'Auto')

- Auto - Use the default specified by the product fusing.
- Disabled - The feature is turned off.
- Enabled - The feature is operational in the system.

• **PcdAmdDlIfCapEn [F19M00]**

This value determines if AGESA will enable the Data Link Feature capability on all Gen4 PCIe® ports.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - Data Link Feature Capability is disabled.
- TRUE - Data Link Feature Capability is enabled.

• **PcdAmdDlIfExEn [F19M00]**

This value determines if AGESA will enable Data Link Feature Exchange on all Gen4 PCIe® ports. The PCIe specification explicitly provides this option so a root complex can disable the DL Feature Exchange transactions in the event a PCIe endpoint is present in a system that cannot respond properly to those packets.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - Data Link Feature Exchange is disabled.
- TRUE - Data Link Feature Exchange is enabled.

3.6.1.4 Platform Category

NBIO settings for describing the platform specific environment characteristics.

• **pcdWaitVidCompleteDisable**

This value determines how a voltage change completion is determined. For Voltage Regulator Modules (VRMs) that support the VOTF complete signal this value should be set to FALSE. This is the most efficient configuration, provided the VRM has this capability. For VRMs that do not have this capability, this value should be set to TRUE to signal that an internal timer should be used in the SVI logic.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Platform VRM supplies the VOTF signal to indicate voltage change is complete.
- FALSE - Use an internal timer.

• **PcdAmdCfgGnbHDAudioSSID**

Allows the host BIOS to set the PCI Sub-System ID value reported by the HD Audio controller. This is nBIF Function 6 for Family 17h processors. A value of zero will indicate that the normal hardware fused value should be used. Note: This value is also used for the Family 15h internal HD audio controller.

Permitted Choices: (Type: Value)(Default: 0x00000000)

- **PcdAmdCfgGnbIGPUAudioSSID**

Allows the host BIOS to set the PCI Sub-System ID value reported by the iGPU audio controller. This is nBIF Function 1 for Family 17h processors. A value of zero will indicate that the normal hardware fused value should be used.

Permitted Choices: (Type: Value)(Default: 0x00000000)

- **PcdAmdMCTPEnable [F17M30]**

Enables or disables MCTP support. The MCTP support applies special programming that manually route all unroutable VDMs to the BMC. The AGESA™ does not make any assumption about which bus/device/function in the enumerated PCI address map corresponds to the BMC; so, the platform must declare the address using the MCTP PCD PcdAmdMCTPMasterID in order to properly configure the underlying Data Fabric and NBIO registers.

Permitted Choices: (Type: Boolean) (Default: FALSE)

- True - MCTP support is enabled.
- False - MCTP support is disabled.

- **PcdAmdMCTPMasterID**

This 16-bit value specifies the PCI address of the MCTP master. This has no functionality if PcdAmdMCTPEnable is False.

Permitted Choices: (Type: Value) (Default: 0x602A)

MCTP Master ID = BMC bus/device/function (Bus << 8 | Device << 3 | Function)

- **PcdAmdAcpiCpuSsdtTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the CPU SSDT ACPI table.

Permitted Choices: (Type: String, up to 8chars)(Default: "AMD CPU")

- **PcdAmdAcpiAlibSsdtTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the ALIB SSDT ACPI table.

Permitted Choices: (Type: String, up to 8chars)(Default: "AMD ALIB")

- **PcdAmdAcpiPtSsdtTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the PT SSDT ACPI table.

Permitted Choices: (Type: String, up to 8chars)(Default: "AMD PT")

- **PcdAmdAcpiCditTableHeaderOemTableId** This value defines the 'OEM identifier' used when creating the CDIT ACPI table.

Permitted Choices: (Type: String, up to 8chars)(Default: "AMD CDIT")

- **PcdAmdAcpiCratTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the CRAT ACPI table.

Permitted Choices: (Type: String , up to 8chars)(Default: "AMD CRAT")

- **PcdAmdAcpiSlitTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the SLIT ACPI table.

Permitted Choices: (Type: String up to 8chars)(Default: "AMD SLIT")

- **PcdAmdAcpiSratTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the SRAT ACPI table.

Permitted Choices: (Type: String up to 8chars)(Default: "AMD SRAT")

- **PcdAmdAcpiIvrsTableHeaderOemTableId**

This value defines the 'OEM identifier' used when creating the IVRS ACPI table.

Permitted Choices: (Type: String, up to 8chars)(Default: "AMD IVRS")

- **PcdPcieGen4x2AerCorrRcvErrBadTLPMaskEnable [F17M30]**

Errata 1166 identifies a situation for F17M30 PCIe links that train to Gen4 speed and use two lanes (a x2 link). An optional workaround to avoid this situation causing extra OS overhead (reduced performance) is to mask the RCV_ERR and BADTLP link error from generating an AER interrupt. This control enables code that identifies the subject devices and forces the RCV_ERR and BADTLP mask bits to be enabled (=1) for these links.

Permitted Choices: (Type: Boolean) (Default: False)

- False - recoverable error is not masked and AER interrupt is presented to the OS.
- True - the link mask bits are set (=1) and no interrupt is sent to the OS.

- **PcdPcieDisGen3EQPhase [F17M30]**

Disable Gen3 EQ Phase2/3 PcdPcieDisGen3EQPhase - Setting this PCD to TRUE bypasses 8.0 GT/s Link Equalization phase 2 and phase 3. If these phases are skipped then the settings used in phase 1 will become the operational settings.

Permitted Choices: (Type: Boolean) (Default: False)

- True - Skip phases 2 and 3.
- False - perform full set of initialization phases. Phases are documented in the "Link Equalization Procedure for 8.0 GT/s and Higher Data Rates" section of the PCIE specification.

- **PcdPcieLinkBypassGen3EQ [F17M30]**

Bypass Gen 3 PcdPcieLinkBypassGen3EQ - Setting this PCD to TRUE bypasses the requesting Phase of 8.0 GT/s Link Equalization (Phase 2 in an Upstream Port or Phase 3 in a Downstream Port).

Permitted Choices: (Type: Boolean) (Default: False)

- True - Bypass Link Equalization requesting Phase.
- False - Perform Link Equalization requesting Phase. This is documented in the "Recovery.Equalization" section of the PCIE specification.

- **PcdPcieLinkBypassGen4EQ [F17M30]**

Bypass Gen 4 PcdPcieLinkBypassGen4EQ - Setting this PCD to TRUE bypasses the requesting Phase of 16.0 GT/s Link Equalization (Phase 2 in an Upstream Port or Phase 3 in a Downstream Port).

Permitted Choices: (Type: Boolean) (Default: False)

- True - Bypass Link Equalization requesting Phase.
- False - Perform Link Equalization requesting Phase. This is documented in the "Recovery.Equalization" section of the PCIE specification.
- **PcdPcieDisGen4EQPhase [F17M30]**

Disable Gen4 EQ Phase2/3PcdPcieDisGen4EQPhase - Setting this PCD to TRUE bypasses 16.0 GT/s Link Equalization phase 2 and phase 3. If these phases are skipped then the settings used in phase 1 will become the operational settings.

Permitted Choices: (Type: Boolean) (Default: False)

- True - Skip phases 2 and 3.
- False - perform full set of initialization phases.

Phases are documented in the "Link Equalization Procedure for 8.0 GT/s and Higher Data Rates" section of the PCIE specification.

- **PcdPcieGen4LaneEqDsTxPreset [F17M30]**

Downstream Tx Preset (Gen4) - This PCD value sets the Downstream Port's transmitter preset for initial operation at 16.0 GT/s. This is "Downstream Port 16.0 GT/s Transmitter Preset" field of the "16.0 GT/s Lane Equalization Control Register" documented in the PCIE specification.

This field can also be set **per port** through the:

PORT_PARAM (PP_GEN4_DS_TX_PRESET, <Value>)

macro where <Value> is the desired field setting. Example:

```
PORT_PARAMS_START
  PORT_PARAM (PP_GEN4_DS_TX_PRESET, 0x3)
PORT_PARAMS_END
```

Examples of the use of the PORT_PARAM macro can be found in: \AmdCpmPkg\Addendum\Oem\Diesel\Pei\AmdCpmOemInitPei\AmdCpmOemTable.c.

- **PcdPcieGen4LaneEqUsTxPreset [F17M30]**

Upstream Tx Preset (Gen4) This PCD value sets the transmitter preset value that the Downstream Port requests the other side to use for initial operation at 16.0 GT/s. This is the "Upstream Port 16.0 GT/s Transmitter Preset" field of the "16.0 GT/s Lane Equalization Control Register" documented in the PCIE specification.

This field can also be set **per port** through the:

PORT_PARAM (PP_GEN4_US_TX_PRESET, <Value>)

macro where <Value> is the desired field setting. Example:

```
PORT_PARAMS_START
  PORT_PARAM (PP_GEN4_US_TX_PRESET, 0x3)
PORT_PARAMS_END
```

Examples of the use of the PORT_PARAM macro are in: \AmdCpmPkg\Addendum\Oem\Diesel\Pei\AmdCpmOemInitPei\AmdCpmOemTable.c

3.6.1.4.1 SRIS/SRNS Configuration

The PCI Express (PCIe®) Base Specification allows implementations to employ architectures with separate Refclk sources for the transmitters (Tx) and receivers (Rx). However, the Base Specification does not define a method to ensure interoperability between components with separate Refclk sources that utilize independent spread spectrum clocking (SSC). The “Separate Refclk Independent SSC Architecture” (SRIS) ECN was created to provide technical details required to enable a design with separate Tx and Rx Refclk sources with independent SSC. The benefit of the ECN is that it can facilitate the enabling of a new class of PCIe applications (mainly involving cables) without needing complex system level solutions for EMI/RFI containment. Since the applications that take advantage of the SRIS ECN will be different form factors from the currently existing form factors, there will not be an interoperability requirement between the new form factors (with SRIS support) and the existing (non-SRIS or SRNS) form factors.

Controls used to manage this feature set are:

Controls used to manage this feature set are below: [F17M30]

- **PcdSrisEnableMode**

Permitted Choices: (Type: UINT8)(Default: 0xFF)

- 1: SRNS
- 2: SRIS
- 3: SRNS in SRIS
- 4: AutoDetect Mode
- 0xFF: Auto (Defaults to SRIS)

- **PcdSrisSkipInterval**

Controls SKP generation interval.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0: Disabled
- 1: Enabled

- **PcdSrisSrnsSkippInSris**

Controls if the Port generates SKP ordered sets with the SRNS SKP ordered set interval.

Permitted Choices: (Type: Value)(Default: 0xFF)

- 0x1 - Gen1
- 0x3 - Gen2 and Gen1
- 0x7 - Gen3 and Gen2 and Gen1
- 0xF (default) - Gen4 and Gen3 and Gen2 and Gen1

- **PcdSrisAutoDetectMode**

Controls the SKP ordered set interval selection method.

Permitted Choices: (Type: UINT8)(Default: 0x00)

- 0: Disabled
- 1: Enabled

3.6.1.5 Power Management Category

The following items are for user selection of NBIO settings to control power and thermal operations.

- **RVPcdPsppPolicy [F17M10][F17M20]**

- **PcdPsppPolicy [F17M60]**

- This value defines the initial PCIe® Speed and Power Policy (PSPP) feature. This feature manages the link speed of some devices based on bandwidth usage in order to reduce power consumption. The PSPP value can be modified during run-time via the ALIB functions.

- Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – PSPP is Disabled.
 - 1 – PSPP Performance Mode.
 - 2 – PSPP Balanced Mode.
 - 3 – PSPP Power Saving Mode.

- **pcdTtotalDevicePower**

- Specifies a maximum power limit for the processor. This indicates the capability of the thermal solution (e.g. heat sink) implemented on this platform to dissipate power. A value of zero means to use the fused value for the part.

- Permitted Choices: (Type: Value)(Default: 0x0000)

- This entry value expresses the power in milliwatts. For example, a TDP power limit of 35 watts is represented as: decimal 35,000.

- **PcdAmdcTDP**

- This value selects the maximum sustained power in Watts provided to the APU. 'Max' is the maximum value supported by the fusing of the part. NOTE: For more information please refer to the IRM spec.

- Permitted Choices: (Type: Value)(Default: 0x00)

- 0x00 – SMU will use the fused TDP value.
 - 0x01..N – specifies the TDP value in Watts, up to the 'Max'.

- **PcdCfgPlatformTDP [F17M30]**

- This value selects the maximum sustained power in Watts provided to the CPU. 'Max' is the maximum value supported by the fusing of the part. NOTE: For more information please refer to the IRM spec.

- Permitted Choices: (Type: Value)(Default: 0x00)

- 0x00 – SMU will use the fused TDP value.
 - 0x01..N – specifies the TDP value in milli-Watts, up to the 'Max'.

- **PcdVrmCurrentLimit**

- This value indicates the maximum current that the voltage regulators is capable of providing to the VDD of the processor on a continuous basis. A value of zero means to use the fused value for the part. The IRM specification refers to this as Thermal Design Current (TDC) [VDDCR_VDD TDC].

- Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in milliamperes. For example, a current limit of 12.6 amperes is represented as: decimal 12,600.

- **PcdCfgTDC [F17M00, F17M08]**

This value is the same as PcdVrmCurrentLimit but this is the label used by the F17M00 and F17M08 code sets.

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in Amperes. For example, a current limit of 40 amperes is represented as: decimal 40.

- **PcdCfgPlatformTDC [F17M30]**

This value is the same as PcdVrmCurrentLimit but this is the label used by the F17M30 code set.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0x00 - Use the fused default values.
- This a numeric value expressed in Amperes. For example, a current limit of 40 amperes is represented as: decimal 40.

- **PcdVrmMaximumCurrentLimit**

This value indicates the maximum current that the voltage regulator is capable of providing to the VDD pins of the processor for short amounts of time. The IRM specification refers to this as Engineering Design Current (EDC) [VDDCR_VDD EDC].

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in milliamperes. For example, a current limit of 14.5 amperes is represented as: decimal 14,500.

- **PcdCfgEDC [F17M00, F17M08]**

This value is the same as PcdVrmMaximumCurrentLimit but this is the label used by the F17M00 and F17M08 code sets.

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in Amperes. For example, a current limit of 40 amperes is represented as: decimal 40.

- **PcdCfgPlatformEDC [F17M30]**

This value is the same as PcdVrmMaximumCurrentLimit but this is the label used by the F17M30 code sets.

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in Amperes. For example, a current limit of 40 amperes is represented as: decimal 40.

- **PcdVrmSocCurrentLimit**

This value indicates the maximum current that the voltage regulators is capable of providing to the SOC pins of the processor on a continuous basis. A value of zero means to use the fused value for the part. The IRM specification refers to this as Thermal Design Current (TDC) [VDDCR_SOC TDC].

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in milliamperes. For example, a current limit of 12.6 amperes is represented as: decimal 12,600.

- **PcdFastPptLimit [F17M10][F17M20]**

PcdSlowPptLimit [F17M10][F17M20]

PcdCfgPlatformPPT [F17M30]

These values define the initial power limits for Package Power Tracking in mW. These values may be changes at run-time via the ALIB functions.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused values.
- XX – Specifies the override value.

- **PcdCfgPPT [F17M30]**

This value defines the power limits for Package Power Tracking in mW.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused values.
- XX – Specifies the override value.

- **PcdSlowPptTimeConstant [F17M10][F17M20]**

This value defines the time constant for “slow” PPT calculations.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused default values.
- XX – Specifies the override value.

- **PcdVrmLowPowerThreshold [F17M10][F17M20]**

PcdVrmSocLowPowerThreshold [F17M10][F17M20]

These values define the low power thresholds for the voltage regulators.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused default value.
- XX – Specifies the override value.

- **PcdCfgThermCtlValue [F17M10][F17M20]**

This value defines the Thermal Control value.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused default value.
- XX – Specifies the override value.

- **PcdFMaxFrequency [F17M10][F17M20]**

This value defines the limit on the FCLK frequency.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused default value.

- XX – Specifies the override value.

- **TablePcdSbTsiAlertComparatorModeEn [F17M10][F17M20]**

This Boolean selects the SB TSI Alert Comparator Mode.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE – Comparator mode enabled.
- FALSE – Comparator mode disabled.

- **PcdVrmSocMaximumCurrentLimit**

This value indicates the maximum current that the voltage regulator is capable of providing to the SOC pins of the processor for short amounts of time. A value of zero means to use the fused value for the part. The IRM specification refers to this as Engineering Design Current (EDC) [VDDCR_VDD EDC].

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in milliamperes. For example, a current limit of 14.5 amperes is represented as: decimal 14,500.

- **PcdSustainedPowerLimit**

This values specifies the sustained power limit that can be supported by the platform thermal solution and chassis thermal design. A value of zero means to use the fused value for the part. This limit is managed within the STAPM time constant.

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in milliwatts. For example, a power limit of 14.25 watts is represented as: decimal 14,250.

- **PcdStapmTimeConstant**

This value specifies the Time Constant used by the power monitor. This is a rolling average window of time used by the monitor. A value of zero means to use the fused value for the part. Passing in 200 will indicate the heat up time constant is 200 seconds.

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in seconds.

- **PcdAmdPowerSupplyIdleControl**

This value selects the idle operation as managed by the SMU. For APU this is the “CPUOFF” feature, while CPU refers to the feature as “PC6”.

Permitted Choices: (Type: Value)(Default: 0x0, Low Current Idle)

- 0 – Typical current idle
- 1 – Low current idle

- **PcdProchotlDeassertionRampTime**

This specifies the time to take to ramp the CCLK/GFXCLK clocks up to Fmax from Fmin when PROCHOT_L is deasserted. A value of zero means to use the default value (20ms). When PROCHOT_L is deasserted, a jump to boost could cause a large current in-rush. Some systems could experience a problem with their power supply. This control is provided to lengthen the ramp time and lessen the current in-rush.

Permitted Choices: (Type: Value)(Default: 0x0000)

This a numeric value expressed in milliseconds. For example, a deassertion time of 32 milliseconds is represented as: decimal 32.

- **PcdCfgPeApmEnable**

This value defines the enablement for Platform level power management. This feature manages power consumption between the APU and an AMD discrete GPU in the platform. **For optimal operation of the feature, the total power envelope available for APU and dGPU combined must be specified in the PPT Limit settings defined by PcdFastPptLimit and PcdSlowPptLimit** via the [ALIB function 0C](#).

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - PeApm is enabled if a discrete GPU is found
- FALSE - PeApm will not be enabled.

- **PcdSetSlowPPTLimitApuOnly**

[F17M18] This value specifies the maximum slow PPT value that the APU alone is allowed to consume in a platform that has PeAPM enabled. The traditional Slow PPT value becomes the combined platform limit for the APU and dGPU power. When PeAPM is disabled, this APU only slow PPT is not used.

This is a numeric value and must be in milliwatts. (Default: slow PPT value of the system configuration defined in the IRM.)

- **PcdCfgSystemConfiguration**

This value may be set to override the system configuration value as specified in the IRM specification for your APU.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use the fused default System Configuration.
- 1 – XX Identifies a specific system configuration to override the fused defaults.

- **PcdCfgApbDis [F17M30][F19M00]**

This value selects the APB Disable value for the SMU.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Clear ApbDis to SMU.
- 1 – Set ApbDis to SMU.

- **PcdCfgFixedSocPstate [F17M30]**

This value defines the target PState when ApbDis is set.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0-x – Specify a valid PState for the processor installed.

- **PcdTelemetry_VddcrVddfull_Scale_Current [F17M10][F17M20][F17M60]**

RVPcdTelemetryVddcrVddfullScale2Current [F17M10][F17M20]

RVPcdTelemetryVddcrVddfullScale3Current [F17M10][F17M20]

RVPcdTelemetryVddcrVddfullScale4Current [F17M10][F17M20]

RVPcdTelemetryVddcrVddfullScale5Current [F17M10][F17M20]

PcdTelemetry_VddcrSocfull_Scale_Current [F17M10][F17M20][F17M60]

These values define overrides for the specified current limits in mA.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused default value.
- XX – Specifies the override value.

- **PcdTelemetry_VddcrVddOffset [F17M10][F17M20][F17M60]**

PcdTelemetry_VddcrSocOffset [F17M10][F17M20][F17M60]

This value defines overrides for the specified voltage offsets in mV.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Use fused default value.
- XX – Specifies the override value.

3.6.1.5.1 Skin Temperature Tracking Feature

Skin Temperature Tracking (STT) was added in [F17M18] and expanded in [F17M60] as STT V2.0. New models are again extending to STT v2.1 for [F17M90] and [F19M50]. The new extensions are expanding the concept of the secondary heat source (Heat Source #2 - HS2); so, some of the below controls will undergo a name change (from 'GPU' to 'HS2') to reflect this broader concept. Older code bases may still use the 'GPU' name. The socket specific Thermal Design Guide (ThDG) contains the operational definition of the feature and many of these parameters. Values for temperatures being monitored are collected by an external APML master and passed to the APML slave (APU) on a periodic basis. This section details the method to declare the operational parameter settings. These settings may also be modified during run-time via the [ALIB controls](#).

PLEASE note: Declarations for these PCDs in the sample code will use initial settings that were determined for the Customer Reference Board. These settings should **NOT** be used on any other board. Customer designed boards **MUST** determine their own settings.

- **PcdSttMinLimit [F17M18]**

PcdSttMinPowerLimit [F17M60]

The System Temperature Tracking minimum value specifies the minimum sustained power that the APU power is allowed to reduce to in order to maintain hotspot temperatures within skin temperature limits. This is applicable only on platforms with system temperature tracking feature enabled.

STT_MinLimit is paired with PcdFastPptLimit (set via the [ALIB function 0C](#)) to form the low and high power limits for the cTDP operation range for the part. Both values can be obtained from the appropriate IRM. The default values for SST_MinLimit is the lowest cTDP for the PStates listed in the Power and Thermal Data Sheet. The Customer has the option to raise the STT_MinLimit to the minimum cTDP for the subset of parts targeted for their platform, or a value their testing has proven to maintain part thermal specifications. The STT_MinLimit might be raised to assure a minimum level of performance; but the customer is responsible to provide a sufficient thermal solution to assure the part can be maintained within its thermal specification. Additional information can be found in the infrastructure Thermal Design Guide.

Permitted Choices: (Type: Value)(Default: per ITM)

- 0 - Inactive, minimum power values are obtained from the Fuses.
- This a numeric value and must be in milliwatts.

- **PcdSttEnable [F17M60]**

This control enables the STT feature in the Model 60h. All STT controls for Model 60h depend on this feature enable.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active.
- FALSE - This option is turned off.

• **PcdSttPcbSensorCount [F17M60]**

This value specifies the number of motherboard sensors that are being used to report skin temperature calculation. There are two modes when STT is enabled and

Permitted Choices: (Type: List)(Default: UMA)

- 2 - UMA mode: The internal (iGPU) is being used so only 2 motherboard sensors are active; the remote and the APU (per the ThDG).
- 3 - APU+HS2 mode: A second heat source (HS2) is included in the STT calculations. The HS2 source is dependent upon the processor version and may be the dGPU, an SSD, 5G module or Prom Chip. All 3 motherboard sensors described by the ThDG are present and active.

It is the responsibility of the host BIOS to assure that this PCD is a valid value when STT is enabled else the feature will not work as expected.

• **PcdSttM1 [F17M60]**

PcdSttM2 [F17M60]

PcdSttM3 [F17M60]

PcdSttM4 [F17M60]

PcdSttM5 [F17M60]

PcdSttM6 [F17M60]

These values specify the characterized coefficients for the motherboard temperature sensors. There are two calculated values, three sensors and two coefficients for each sensor.

Characterized coefficients are derived from testing. Customers will have to run tests on their systems to determine these coefficients for their specific systems. The guideline to determine these coefficients is provided in the Thermal Design Guide (ThDG) for the Processor. Please see the appropriate TDG for the applicable infrastructure (e.g. FP6, AM4, FF3, etc.)

Table 12. Sensor coefficients

Sensor	APU calculation	HS2 calculation
Near APU	M1	M4
Near HS2	M3	M6
'Remote'	M2	M5

Permitted Choices: (Type: Value)(Default: 0x0000)

The Sensor coefficients are an encoded Q6.10 fractional number.

- Bit 15 - sign bit
- Bits 14:10 - integer portion
- bits 9:0 - fractional portion (is = value/1024)

- **PcdSttCApu [F17M60]**

- PcdSttCHS2 [F17M60]**

These values specify the 'characterized intercept' (see ThDG for explanation) to calculate APU or HS2 dependent skin temperature value.

Permitted Choices: (Type: Value)(Default: 0x0000)

These values are encoded as Signed Q6.10 fractional number representing the associated Intercept value.

- **PcdSttAlphaApu [F17M60]**

- PcdSttAlphaHS2 [F17M60]**

These values specify the 'characterized alpha-filter' value, between 0 and 1 for APU or HS2 dependent skin temperature

Permitted Choices: (Type: Value)(Default: 0x0000)

These values are encoded as Unsigned Q0.16 fractional number representing the associated alpha-filter value.

- Bits 15:0 - fractional portion (is = value/65536)

- **PcdSttSkinTemperatureLimitApu [F17M60]**

- PcdSttSkinTemperatureLimitHS2 [F17M60]**

These values specify the 'skin temperature limit' for APU or HS2 dependent skin temperature.

Permitted Choices: (Type: Value)(Default: 0x0000)

These values are an unsigned integer representing the associated skin temperature limit. Refer to the Thermal Design Guide for more details.

- **PcdSttErrorCoefficient [F17M60]**

This value specifies the 'proportion gain' control for APU Power or APU+dGPU power.

Permitted Choices: (Type: Value)(Default: 0x0000)

This value is encoded as Unsigned Q0.16 representing the proportion gain control.

- **PcdSttErrorRateCoefficient [F17M60]**

This value specifies the 'derivative gain' control for APU Power or APU+dGPU power.

Permitted Choices: (Type: Value)(Default: 0x0000)

This value is encoded as Unsigned Q0.16 representing the derivative gain control.

3.6.1.5.2 Fan Control table

These parameters define and control the operation of the system fan(s)

- **pcdFanTable Override**

This BOOLEAN value declares whether the SMU will use the BIOS-supplied fan table values, or the default fan table settings.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE – Use PWM or BIOS supplied fan table settings from PCDs

- FALSE – Use the default SMU fan table settings (which match the control defaults listed below).

- **PcdForceFan PwmEn**

This value controls whether the fan control should be forced to a fixed PWM value.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - The fan speed will be fixed at the PWM value provided by PcdForceFanPwm.
- FALSE - The fan speed will be controlled by the default fan table or the PCD override values.

- **PcdForceFanPwm**

This value controls the fixed PWM value when PcdForceFanPwmEn is set to true. A value between 0 and 100 (decimal) specifies the percentage of “on” time for the fan PWM.

Permitted Choices: (Type: Value)(Default: 0x00)

- **PcdFanTable Polarity**

This value identifies the polarity of the fan control signal used for PWM fan control.

Permitted Choices: (Type: Value)(Default: 0x0001)

- 0 – The fan control signal has a negative polarity (active low)
- 1 – The fan control signal has a positive polarity

- **pcdFanTable TemperatureLow**

This value indicates the temperature threshold at which the speed of the fan will be increased. The temperature is specified in degrees C. For each temperature threshold, there is also an associated PWM percentage value used to select the fan speed when the temperature exceeds the selected value. Hysteresis control is provided by another control.

Permitted Choices: (Type: Value)(Default: 0x0000)

- **pcdFanTable TemperatureMedium**

This value indicates the temperature threshold at which the speed of the fan will be increased. The temperature is specified in degrees C. For each temperature threshold, there is also an associated PWM percentage value used to select the fan speed when the temperature exceeds the selected value. Hysteresis control is provided by another control.

Permitted Choices: (Type: Value)(Default: 0x0032)

- **pcdFanTable TemperatureHigh**

This value indicates the temperature threshold at which the speed of the fan will be increased. The temperature is specified in degrees C. For each temperature threshold, there is also an associated PWM percentage value used to select the fan speed when the temperature exceeds the selected value. Hysteresis control is provided by another control.

Permitted Choices: (Type: Value)(Default: 0x0050)

- **pcdFanTable TemperatureCritical**

This value indicates the temperature at which the fan speed will be set to 100% on (full speed). The temperature is specified in degrees C.

Permitted Choices: (Type: Value)(Default: 0x0064)

- **pcdFanTable PwmLow**

This value indicates the fan speed for the associated temperature level (PWM cycle ratio). The value is specified in a percentage of “on” time from 0 to 100.

Permitted Choices: (Type: Value)(Default: 0x00)

- **pcdFanTable PwmMedium**

This value indicates the fan speed for the associated temperature level (PWM cycle ratio). The value is specified in a percentage of “on” time from 0 to 100.

Permitted Choices: (Type: Value)(Default: 0x00)

- **pcdFanTable PwmHigh**

This value indicates the fan speed for the associated temperature level (PWM cycle ratio). The value is specified in a percentage of “on” time from 0 to 100.

Permitted Choices: (Type: Value)(Default: 0x004C)

- **pcdFanTable Hysteresis**

This value indicates the temperature hysteresis in degrees C. The hysteresis value determines the amount of temperature drop below any given limit that will allow the fan speed to be reduced. For example, when the temperature exceeds pcdFanTableTemperatureHigh, the fan speed will be set according to pcdFanTablePwmHigh. The temperature must fall below (pcdFanTableTemperatureHigh – pcdFanTableHysteresis) in order for the fan speed to be set according to pcdFanTablePwmMedium.

Permitted Choices: (Type: Value)(Default: 0x05)

- **pcdFanTable PwmFrequency**

Selects the base frequency for the Pulse Width Modulation (PWM).

Permitted Choices: (Type: List)(Default: 0)

- 0 - 25kHz base frequency
- 1 - 100Hz base frequency

3.6.1.6 Performance Category

NBIO settings for fine tuning and operational choices.

- **PcdAmdNbpoisonConsumption**

This value controls whether AGESA will enable NBIO poison data consumption. When enabled, poison consumption will enable SyncFlood and link disablement when an APML error occurs.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - AGESA will enable poison consumption errors.
- FALSE - AGESA will not enable poison consumption errors.

- **PcdAmdDeterminismControlPerformance**

This value controls whether AGESA will enable determinism to control performance. NOTE: For more information please refer to the IRM spec.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - AGESA will enable 100% deterministic performance control
- FALSE - AGESA will not enable deterministic performance control

- **PcdAmdPSIDisable**

This value controls whether AGESA will disable the “Power State Indicate” control through SMU. NOTE: For more information please refer to the IRM spec.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - AGESA will disable PSI.
- FALSE - PSI option is turned on.

- **PcdOcDisable**

This value provides an option to disable overclocking capabilities.

Permitted Choices: (Type: Boolean)(Default: False)

- TRUE - Overclocking will not be permitted.
- FALSE - The system may use overclocking within the stated limits.

- **PcdOcVoltageMax**

This value defines the voltage limit that can be requested for overclocking. The value is in millivolts. If a value of 0 is specified then the OPN-dependent default value will be used. Example: To select a maximum voltage of 1.5V, this item should be set to 1500.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0x00 – Default fused value for the part will be used.
- 0x01..N – Voltage limit in milli-volts.

- **PcdOcFrequencyMax**

This value defines the frequency limit that can be requested for overclocking. The value is in megahertz (MHz). If a value of 0 is specified then the fused value will be used. Example: To select a maximum frequency of 3.6GHz, this PCD should be set to 3600.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0x00 – Default fused value for the part will be used.
- 0x01..N – Frequency limit in MHz.

- **PcdVminFrequency**

This value defines the frequency associated with Vmin voltage. The value is in megahertz (MHz). If a value of 0 is specified then the fused value will be used.

Example: To select a maximum frequency of 3.6GHz, this PCD should be set to 3600.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0x00 – Use default fused value
- 0x01..N – Frequency limit in MHz.

- **PcdCfgForcePcieGenSpeed**

This value defines an upper limit for PCIe® speed to apply to all PCIe ports. This value superceeds values set in the per-link settings in the DXIO_Port_Data structures.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Do not limit PCIe link speed
- 1 – Limit all PCIe links to Gen1
- 2 – Limit all PCIe links to Gen2
- 3 – Limit all PCIe links to Gen3

- **PcdAmdNbIOsPcMode5GT**

This control configures the data path width when the PCIe® link is running at **Gen2** speed. For Family 17h Models 00h see erratum #1080; customers are advised to read the erratum but not to change the default unless they are pretty sure they are having a PCIe® adapter training problem due to the 1SPC configuration. Family 17h Model 10h will run in 1 symbol mode to avoid this erratum.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - 1 symbol per Clock Data Path.
- TRUE - 2 symbols per Clock Data Path.

- **PcdAmdNbIOsPcMode2P5GT**

This control configures the data path width when the PCIe® link is running at **Gen1** speed. For Family 17h Models 00h see erratum #1080; customers are advised to read the erratum but not to change the default unless they are pretty sure they are having a PCIe® adapter training problem due to the 1SPC configuration. Family 17h Model 10h will run in 1 symbol mode to avoid this erratum.

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - 1 symbol per Clock Data Path.
- TRUE - 2 symbols per Clock Data Path.

- **PcdAmdXgmiReadaptation**

This option controls whether or not SMU FW performs XGMI Re-adaptation during BIOS POST. The phy used for xGMI links needs to adjust analog parameters to work at its optimum point. It is called 'adaptation'.

Once early in the boot process, DXIO FW adapts xGMI links; but as time passes, temperature/humidity changes the early boot analog parameters are not good enough. This causes bit error, and if not cured, eventually results in an MCE once certain a threshold is passed.

DXIO FW has a mechanism to overcome this, called “re-adaptation”: every 1ms, SMU calls a DXIO FW routine, checks for bit errors on the xGMI links. If it finds enough errors, it tries to repeat the adaption sequence. The xGMI link is brought down (for up to 400us); the re-adaptation is performed then the link is set back to normal. This task is defined as high priority in SMU scheduler and may take up to 2ms.

This re-adaptation process may cause some system instability in very specific cases (due to the time the link is not available). No issues have been observed and it is highly recommended to keep the feature ON. However, in certain cases, if this patch is found to destabilize the machine, the user may turn it OFF as a workaround. As turning the feature OFF may compromise the xGMI link quality/performance and cause MCE, users are recommended to report the case back to AMD and do not keep the feature turned OFF as a fix.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE - Only the initial adaptation process is used.
- TRUE - The adaptation process will be repeated as needed to avoid extra MCA errors.

• **PcdCfgDisableRcvrResetCycle**

During PCIe® training, if a PHY error occurs during equalization, the receiver will cause a reset to occur as an attempt to do error recovery.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE - Receiver reset circuit will be active.
- TRUE - Prevent resets from occurring during training. This may help with some cards or devices that are particularly sensitive to these resets. This affects the training cycle only and does not affect the ability of the system BIOS to issue a device level reset to the PCIe® device.

• **PcdAfterResetDelay**

This value specifies a delay, in mS, to be inserted between PCI reset de-assertion and start of training. Some devices, particularly NVMe drives, are problematic with longer delays. Other devices are problematic with short delays, such as a custom NIC found to require 500ms. This delay is inserted once for each socket. It only applies to the first PCIe port of each socket that has a GPIO reset ID specified in the platform topology. Generally the PCIe resets are connected to the PERST pin for one of the NBIOs in the chip, but the resets are controllable as GPIOs as well. It is assumed that the first listed entry is the one with the critical reset.

Permitted Choices: (Type: Value, milliseconds)(Default: 0x0)

- 0 – No delay inserted after the reset.
- XX – insert a delay of xx milliseconds.

• **PcdPcieEqSearchMode**

PCIe Link Equalization Mode. AMD PCIe links operating at Gen3 speed always perform equalization at the hardware level when training from Gen1 to Gen3. The equalization process involves the root port and the connected end point exchanging information with one another at the PHY level in order to obtain the best possible signal quality. The default

equalization mode will always be “Exhaustive” unless the user chooses to change it here.

Permitted Choices: (Type: Value)(Default: Exhaustive)

- DxioEqPresetSearchDirectional. This mode is supported by the hardware but AMD does not recommend using this mode.
- DxioEqPresetSearchExhaustive. The PHY performs a best fit algorithm using a predetermined range of coefficients for tuning its transmit and receive components. This is the recommended mode.
- DxioEqPresetSearchPreset. This equalization mode allows the user to directly control the coefficients used by the PHY for its tuning algorithm.

• **PcdPcieEqSearchPreset**

This control depends on PcdPcieEqSearchMode above. This control value will only be used when PcdPcieEqSearchMode == DxioEqPresetSearchPreset. In that circumstance the value of this control will be used as the preset constant for all PCIe ports. These values will take precedence over any value passed in the API structure DXIO_PORT_DATA_INITIALIZER_PCIE.mEqPreset

Permitted Choices: (Type: Value)(Default: 0xFF)

- 0..9 - valid preset constants.
- 0x0A .. 0xFE - reserved.
- 0xFF - per port assignment. AGESA will use the customer defined value passed in the DXIO API data structure.

• PcdPCIEExactMatchEnable

When ExactMatch is true, the algorithm that maps engines to PCIe ports will only map an engine to a port if the size of the engine matches the size of the port exactly. When exactMatch is false, the algorithm that maps engines to PCIe ports will allow an engine to be mapped to a port that is the same size or larger than the engine.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE - use any engine where port will fit.
- TRUE - Match port and engine sizes.

• PcdAmdPreferredIOBus [F17M30][F19M00]

Enables the performance improvement of a "preferred bus".

Permitted Choices: (Type: UINT16)(Default: 0xFFFF)

- 0x00 to 0xFF - Any valid PCI bus number, will enable the feature on the root complex that corresponds to that address.
- 0xFFFF - do not enable the feature

Note: For [F19M00] this control is superseded by the LclkDpmLevel control.

• PcdAmdEnhancedPreferredIOMode [F17M30][F19M00]

A preferred I/O enhancement is added to optimize performance on the selected root bridge. The enhanced preferred I/O mode can only be enabled if PcdAmdPreferredIOBus is also enabled. The combination of these two controls establishes one of the 8 root bridges to be the one 'preferred' bus.

Permitted Choices: (Type: Boolean)(Default: [F17M00] TRUE, [F19M00] FALSE)

- FALSE - disable the enhanced mode.
- TRUE - enable an enhanced mode which assures an LCLK value for best performance.

Note: For [F19M00] this control is superseded by the [PcdAmdNbIO LclkDpmLevel](#) control.

• PcdAmdNbIO LclkDpmLevel [F19M00]

This item selects the level of Dynamic Power Management (DPM) for the NBIO root bridges. This control is a 32-bit bit-mapped value with 8 bit fields each representing one of the NBIO root bridges. The PreferredIOMode and Bus controls establish one preferred bus; this control can establish preferred status to many root bridges.

```
// PcdAmdNbIO LclkDpmLevel BitField:
//   - Socket0 (NBIO0[3:0], NBIO1[7:4], NBIO2[11:8] NBIO3[15:12])
//   - Socket1 (NBIO0[19:16], NBIO1[23:20], NBIO2[27:24] NBIO3[31:28])
```

Permitted Choices within each bit field: (Type: Value)(Default: Auto - 0x0F)

- Auto - 0x0F - No special handling, the DPM feature is left intact for the optimum balance between power saving and performance. Recommended for most situations.
- Min - 0x00 - set the LCLK to the minimum frequency (300MHz). This disables the DPM feature and always saves the most power by running slower.
- Max - 0x02 - set the LCLK to the maximum frequency (593MHz). This disables the DPM feature and always runs the bridge at max performance. This may help workloads that tend to be quick bursts of data.

Note: the 'LCLK' is an internal NBIO clock used for the DPM feature. It is separate from the PCIe link clock(s); so, this option will not affect link speeds.

- **PcdAmdPreferredIODevice [Deprecated] [F17M30]**

In certain platform configurations it is possible to improve the performance of an endpoint by enabling "Preferred I/O" mode. This PCD is intended for use by platform BIOS software that provides the user interface to select a particular add-in device and references the PCI bus/device/function address of the add-in card when requesting Preferred I/O mode for the device.

See also PcdAmdPreferredIOBus and PcdAmdEnhancedPreferredIOMode. This control is not used when PcdAmdPreferredIOBus is active, and is now deprecated.

Permitted Choices: (Type: UINT32)(Default: 0x00)

- 0x00 - disable this feature.
- not=0 - PCI address of target device. Bit packed field:
 - [23:16] BusNum
 - [15:8] DevNum
 - [7:0] FunNum

- **PcdPcieLinkTrainingType**

This value selects the method of training for PCIe® ports. Single step training means that initial device training is done at the target operating speed. Dual step training means that initial training is done at Gen1 speed and then the link is transitioned to the target operating speed.

Permitted Choices: (Type: Value)(Default: 0x0)

- 0 – Enables single step training to the target operating speed
- 1 – Enables dual step training

- **PcdPcieLinkTxVetting [F17M30]**

This control is used to apply consistent emphasis across lanes. It enables lane TX adaptation adjustments during training on PCIe® links running Gen3 or Gen4. When set to TRUE, the DXIO firmware will train all lanes and then run an algorithm to compare adaptation values determined for each lane and make adjustments. When TX vetting is enabled, the DXIO firmware finds the most commonly occurring of each pre-emphasis equalization parameters (pre/post/main cursors) across the active lanes, applies the same value to all lanes, and retrain. This PCD affects all PCIe engines.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- **PcdPcieLinkRxVetting [F17M30]**

This control is used to enable lane RX adaptation adjustments during training on PCIe® links running Gen3 or Gen4. When set to TRUE, the DXIO firmware will train all lanes and then run an algorithm to compare adaptation values determined for each lane and make adjustments. When RX vetting is enabled, the DXIO firmware averages the RX adaptation values (DC gain, AC gain, IQ, pole) across all active lanes and then retrain using updated values. This PCD affects all PCIe engines.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- **PcdPcieDxioTimingControlEnable [F17M30]**

Controls whether or not the three PCIe training timing overrides below are leveraged by DXIO firmware. Note that all of the timers below have default values that are expected to work in any typical training scenario, but the timers are available for debug purposes or to address specific platform issues in special circumstances.

Permitted Choices: (Type: Boolean)(Default: FALSE)

Affected controls:

- PcdPCIELinkResetToTrainingTime
- PcdPCIELinkReceiverDetectionPolling
- PcdPCIELinkL0Polling

- **PcdPCIELinkResetToTrainingTime [F17M30]**

Time (in microseconds) from release of PERST# to the start of the LTSSM (training state machine). The default value of the internal timer controlled by this PCD is 1 microsecond. This control is enabled by PcdPcieDxioTimingControlEnable.

Permitted Choices: (Type: Value)(Default: 0x00)

- **PcdPCIELinkReceiverDetectionPolling [F17M30]**

Controls the timeout (in microseconds) from the beginning of PCIe training, i.e. when the AMD LTSSM begins, until receiver detection is successful or times out. Internally this timer defaults to 500 ms. This control is enabled by PcdPcieDxioTimingControlEnable.

Permitted Choices: (Type: Value)(Default: 0x00)

- **PcdPCIELinkL0Polling [F17M30]**

Timeout (in microseconds) for the link to be in L0 state after receiver detection is complete. Internally the timer controlled by this PCD defaults to 500 mS. This control is enabled by PcdPcieDxioTimingControlEnable.

Permitted Choices: (Type: Value)(Default: 0x00)

- **PcdForceGfxclkFrequency**

Specifies the forced GFXCLK frequency to the specified frequency [MHz].

Permitted Choices: (Type: Value)(Default: 0)

- 0 – Does not override the GFXCLK frequency.
- Non-Zero – Forces the GFXCLK frequency to the specified clock.

- **PcdGfxclkFmaxOverride**

Specifies the GFXCLK frequency max override value [MHz].

Permitted Choices: (Type: Value)(Default: 0)

- 0 – Does not override the GFXCLK frequency.
- Non-Zero – Forces the GFXCLK frequency to the specified clock.

- **PcdForceVddcrSocVoltage**

Specifies a forced voltage for the SOC [mV].

Permitted Choices: (Type: Value)(Default: 0)

- 0 – Does not override the SOC voltage.
- Non-Zero – Forces the SOC voltage to the specified value in millivolts.

- **PcdForceVddcrCpuVoltage**

Specifies a forced voltage for the CPU [mV].

Permitted Choices: (Type: Value)(Default: 0)

- 0 – Does not override the CPU voltage.
- Non-Zero – Forces the CPU voltage to the specified value in millivolts.

- **PcdAMDBoostFmax [F17M30]**

This value specifies the boost Fmax frequency limit to apply to all cores (MHz)

Permitted Choices: (Type: Value) (Default: 0x0)

- 0 ~ 0xFFFFFFFF : CPU cores limit on frequency, in MHz.

- **PcdAmdDeterminismMode [F17M30]**

This value can select the fused determinism or customized determinism

Permitted Choices: (Type: Value) (Default: 0x0)

- 0 - Use the fused Determinism.
- 1 - User can set customized Determinism.

- **PcdCfgMaxReadRequestSize [F17M30]**

This value controls the maximum Read Request size which a PCIe root port will support. The power up default is 512 bytes but this PCD can be used to increase as high as 4096 if an endpoint is present that supports the larger size.

Permitted Choices: (Type: List)(Default: 0xFF)

- 0x0 - 128 Bytes
- 0x1 - 256 Bytes
- 0x2 - 512 Bytes
- 0x3 - 1024 Bytes
- 0x4 - 2048 Bytes
- 0x5 - 4096 Bytes
- 0xFF - Used hardware default value

- **PcdCfgHSMPSupport [F17M30][F19M00]**

This value controls Host System Management Port (HSMP) interface to provide OS-level software with access to system management functions via a set of mailbox registers. Functionality of the HSMP interface is implemented by the power management firmware and documented in the [PPR](#).

Permitted Choices: (Type: List)(Default: 0xFF)

- 0x0 - Disable HSMP interface
- 0x1 - Enable HSMP interface support
- 0xFF - Used hardware default value

3.6.1.7 Hot Plug Feature

NBIO settings for PCIe® Hot Plug operations.

- **PcdAmdHotPlugHandlingMode [F17M30][F19M00]**

Selects hot plug mode.

Permitted Choices: (Type: Value)(Default: 0xFF/Auto)

- 0 - F17M00 Compatibility mode ([F17M30] only).
- 1 - OS-First.
- 3 - Firmware-First ([F17M30] Rev B0 Only).
- 0xFF (Auto) – for:
 - F17M30 Rev A0 = F17M00 Compatibility mode
 - F17M30 Rev B0 = OS-First
 - F19M00 = OS-First.

• **PcdAmdPresenceDetectSelectMode [F17M30]**

Selects hot plug presence detection mode.

Permitted Choices: (Type: Value)(Default: 0xFF/Auto)

- 0 - 'OR' – Presence detect is the logical OR of in-band presence detect and side-band presence detect.
- 1 - 'AND' - Presence detect is the logical AND of in-band presence detect and side-band presence detect.
- 0xFF (Auto) – use 'OR' mode.

• **PcdAmdHotPlugNVMEDefaultMaxPayload [F17M30]**

Sets default MAX_PAYLOAD_SIZE in DEVICE_CNTL register of unpopulated EnterpriseSSD hot plug ports.

Permitted Choices: (Type: Value)(Default: 0xFF)

- 0 - 128 Bytes
- 1 - 256 Bytes
- 2 - 512 Bytes
- 3 - 1024 Bytes
- 4 - 2048 Bytes
- 5 - 4096 Bytes
- 0xFF – default hardware setting.

• **PcdAmdHotPlugNVMESkipHPStatUpdate [F17M30]**

This options prevents sideband presence updates. This may be required in some specific configurations. Please refer to the [AMD Hot Plug Application Note](#).

Permitted Choices: (Type: Boolean)(Default: FALSE)

- False - Normal Mode (per spec).
- True - Skip sideband presence updates.

• **PcdAmdDisableSideband**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This control switch allows the firmware to ignore the physical presence (GPIO).

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - Will execute normal I2C sideband behavior (SMU reports physical presence to the PCIe® controller).

- TRUE - SMU does not report physical presence to the PCIe® controller. Presence_detect_state determined through PCIe® link inband detection. SMU does not report physical presence to the PCIe® controller. Presence_detect_state determined through PCIe® link inband detection.

• **PcdAmdDisableL1wa**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This switch allows a system to explore a limitation found in the PCIe® spec where a device can be removed when the link is in L1 (low power state) which causes the removal to be undetected to the PCIe® controller. Upon detection of drive removal (through physical presence), SMU will force a snapout of the link via LINK_RETRAIN. ('wa' -workaround)

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - SMU FW will enforce valid device presence status during a L1 hot remove via LINK_RETRAIN.
- TRUE - SMU FW will not enforce valid device presence during L1 hot removal.

• **PcdAmdDisableBridgeDis**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This switch allows BridgeDis programming based on physical presence.

Note: If PcdAmdIrqSetBridgeDis=FALSE, then there is no control of BridgeDis.

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - BridgeDis based on physical presence.
- TRUE - BridgeDis not controlled by physical presence.

• **PcdAmdRRCEn**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This switch determines if the ReceiverResetCycleEn bit is toggled during hot plug events.

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - ReceiverResetCycleEn remains unmodified during hot plug events..
- TRUE - ReceiverResetCycleEn is toggled during hot plug events.

• **PcdAmdIrqSetBridgeDis**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This switch allows BridgeDis to be based on PCIe® inband presence detect and DL_ACTIVE=1.

Note: When set to TRUE, PcdAmdDisableBridgeDis and PcdAmdRRCEn are forced to TRUE.

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - BridgeDis not based off of PCIe® inband presence detect and DL_ACTIVE=1.
- TRUE - BridgeDis based off of PCIe® inband presence detect and DL_ACTIVE=1.

• **PcdHotplugI2cAddress**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This switch defines the starting I2C address of the mux from the hot plug controller (SMU) to the PCA devices. Typically 0x70.

Permitted Choices: (Type: Byte Value)(Default: 0x0)

- **PcdHotplugSlotIndex**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. This switch defines if slot numbering for hotplug is 0 based or 1 based.

Permitted Choices: (Type: List)(Default: 0x0)

- 0 - 0 based slot numbering.
- 1 - 1 based slot numbering.

- **PcdPcieSyncReset**

This value is one of a group of controls provided to fine tune the Hot Plug feature operation. By default, PCIe® link training completes on each die before the links on the next die are trained. Setting this PCD to TRUE results in each die configuring the links for training but waiting until all dies are configured before releasing PCIe® RESET allowing all dies to train concurrently.

Permitted Choices: (Type: Boolean)(Default: False)

- FALSE - Disable concurrent training.
- TRUE - Enable concurrent training.

- **PcdCfgPcieCrsLimit**

This option is available to adjust the PCIe® CRS (Configuration sequence Retry Status) limit for hot plug ports in order to improve training results after a hot add. It is not expected that customers will need to adjust this value except in certain Hot-Plug systems. This setting controls how long the hot plug port retries the same configuration cycle before a timeout is signaled. This time duration is specified in 1.6 ms increments.

Permitted Choices: (Type: Value)(Default: 0x0000)

Example: a value of 6 indicates 9.6ms limit on retries.

- **PcdCfgPcieCrsDelay**

This option is available to adjust the PCIe® CRS (Configuration sequence Retry Status) retries for hot plug ports in order to improve training results after a hot add. It is not expected that customers will need to adjust this value except in certain Hot-Plug systems. This setting controls the delay between retries on the hot plug port. This time duration is specified in 1.6 ms increments.

Permitted Choices: (Type: Value)(Default: 0x0000)

Example: a value of 6 indicates 9.6ms between retries.

3.6.1.8 NTB - Non-Transparent Bridge Feature

NBIO settings for Non-Transparent Bridge (NTB) operations. For further information, please see the PPR and the [NTB document](#).

- **PcdCfgNTBEnable [F17M30]**

Selects to enable the NTB feature. This is the base PCD which needs to be enabled before using any of the other NTB related PCDs

Permitted Choices: (Type: Boolean)(Default: Disabled)

- False - Disable the NTB feature.
- TRUE - Enable NTB. This activates the following controls.

- **PcdCfgNTBLocation [F17M30]**

Socket and Die combination where to enable NTB for this link endpoint. The value of this PCD should be set to the Socket and Die pair on which NTB needs to be enabled.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Socket0 Die0.
- 1 – Socket0 Die1.
- 2 - Socket0 Die2.
- 3 - Socket0 Die3.
- 4 - Socket1 Die0.
- 5 - Socket1 Die1.
- 6 - Socket1 Die2.
- 7 - Socket1 Die3.

- **PcdCfgNTBPcieCoreSel [F17M30]**

PCIe Core on which to enable NTB for this end of the link. The value of this PCD should be set to the PCIe Core of the particular Socket and Die combination on which NTB needs to be enabled.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – PCIe Core 0.
- 16 – PCIe Core 1.

Values other than those listed above may result in undefined operations.

- **PcdCfgNTBMode [F17M30]**

Select NTB Mode. User can select whether they want to enable NTB as primary or secondary.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Disabled.
- 1 – NTB is primary.
- 2 - NTB is secondary.

- **PcdCfgNTBLinkSpeed [F17M30]**

Select Link Speed for the NTB. NTB can work at PCIe Gen1, Gen2, Gen3 and Gen4 speeds. The user can select the desired speed using this PCD.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Max Speed.
- 1 – Gen1.
- 2 - Gen2.
- 3 - Gen3.
- 4 - Gen4.

- **PcdCfgNTBP0P0**

Enable NTB on Socket-0 P0 Link. By default NTB is disabled. In case the users want to enable NTB on P0 link, they should set this PCD to True. This is the base PCD which needs to be enabled before using any of the other NTB related PCDs for link P0.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE – NTB is disabled.
- TRUE – NTB is enabled for S0-P0.

- **PcdCfgNTBStartLaneP0P0**

NTB Start Lane on Socket-0 P0 Link. NTB can be enabled on the highest width lanes (x4, x8, x16) on P0. This PCD allows the user to select the first lane of P0 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 00)

- 0 – x16 lane for NTB on P0..
- 1..15 - start lane for the link. See width below. Keep in mind the alignment and size requirements of the lane as described in the PPR.

- **PcdCfgNTBEndLaneP0P0**

NTB End Lane on Socket-0 P0 Link. This PCD allows the user to select the last lane of P0 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 15)

- 0..14 – Ending lane for the link. Keep in mind the alignment and size requirements for the lane as described in PPR.
- 15 – selects x16 lane for NTB on P0.

- **PcdCfgNTBLinkSpeedP0P0**

Link Speed for Socket-0 P0 Link. NTB can work at PCIe Gen1, Gen2, Gen3 and Gen4 speeds. The user can select the desired speed using this PCD.

Permitted Choices: (Type: Value)(Default: 0x03)

- 1 – Gen1.
- 2 - Gen2.
- 3 - Gen3.
- 4 - Gen4.

- **PcdCfgNTBModeP0P0**

Select NTB Mode. User can select whether they want to enable NTB as primary or secondary.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Disabled.
- 1 – NTB is primary.
- 2 - NTB is secondary.

- **PcdCfgNTBP0P1**

Enable NTB on Socket-0 P1 Link. By default NTB is disabled. In case the users want to enable NTB on P1 link, they should set this PCD to True. This is the base PCD which needs to be enabled before using any of the other NTB related PCDs for link P1.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE – NTB is disabled.
- TRUE – NTB is enabled for S0-P1.

- **PcdCfgNTBStartLaneP0P1**

NTB Start Lane on Socket-0 P1 Link. NTB can be enabled on the highest width lanes (x4, x8, x16) on P1. This PCD allows the user to select the first lane of P1 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 32)

- 32 – x16 lane for NTB on P1.
- 33..47 - start lane for the link. See width below. Keep in mind the alignment and size requirements of the lane as described in the PPR.

• **PcdCfgNTBEndLaneP0P1**

NTB End Lane on Socket-0 P1 Link. This PCD allows the user to select the last lane of P1 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 15)

- 32..46 – Ending lane for the link. Keep in mind the alignment and size requirements for the lane as described in PPR.
- 47 – selects x16 lane for NTB on P1.

• **PcdCfgNTBLinkSpeedP0P1**

Link Speed for Socket-0 P1 Link. NTB can work at PCIe Gen1, Gen2, Gen3 and Gen4 speeds. The user can select the desired speed using this PCD.

Permitted Choices: (Type: Value)(Default: 0x03)

- 1 – Gen1.
- 2 - Gen2.
- 3 - Gen3.
- 4 - Gen4.

• **PcdCfgNTBModeP0P1**

Select NTB Mode. User can select whether they want to enable NTB as primary or secondary.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Disabled.
- 1 – NTB is primary.
- 2 - NTB is secondary.

• **PcdCfgNTBP0P2**

Enable NTB on Socket-0 P2 Link. By default NTB is disabled. In case the users want to enable NTB on P2 link, they should set this PCD to True. This is the base PCD which needs to be enabled before using any of the other NTB related PCDs for link P2.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE – NTB is disabled.
- TRUE – NTB is enabled for S0-P2.

• **PcdCfgNTBStartLaneP0P2**

NTB Start Lane on Socket-0 P2 Link. NTB can be enabled on the highest width lanes (x4, x8, x16) on P2. This PCD allows the user to select the first lane of P2 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 80)

- 80 – x16 lane for NTB on P2.
- 81..95 - start lane for the link. See width below. Keep in mind the alignment and size requirements of the lane as described in the PPR.

- **PcdCfgNTBEndLaneP0P2**

NTB End Lane on Socket-0 P2 Link. This PCD allows the user to select the last lane of P2 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 95)

- 80..94 – Ending lane for the link. Keep in mind the alignment and size requirements for the lane as described in PPR.
- 95 – selects x16 lane for NTB on P2.

- **PcdCfgNTBLinkSpeedP0P2**

Link Speed for Socket-0 P2 Link. NTB can work at PCIe Gen1, Gen2, Gen3 and Gen4 speeds. The user can select the desired speed using this PCD.

Permitted Choices: (Type: Value)(Default: 0x03)

- 1 – Gen1.
- 2 - Gen2.
- 3 - Gen3.
- 4 - Gen4.

- **PcdCfgNTBModeP0P2**

Select NTB Mode. User can select whether they want to enable NTB as primary or secondary.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Disabled.
- 1 – NTB is primary.
- 2 - NTB is secondary.

- **PcdCfgNTBP0P3**

Enable NTB on Socket-0 P3 Link. By default NTB is disabled. In case the users want to enable NTB on P3 link, they should set this PCD to True. This is the base PCD which needs to be enabled before using any of the other NTB related PCDs for link P3.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE – NTB is disabled.
- TRUE – NTB is enabled for S0-P3.

- **PcdCfgNTBStartLaneP0P3**

NTB Start Lane on Socket-0 P3 Link. NTB can be enabled on the highest width lanes (x4, x8, x16) on P3. This PCD allows the user to select the first lane of P3 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 112)

- 112 – x16 lane for NTB on P3.
- 113..127 - start lane for the link. See width below. Keep in mind the alignment and size requirements of the lane as described in the PPR.

- **PcdCfgNTBEndLaneP0P3**

NTB End Lane on Socket-0 P3 Link. This PCD allows the user to select the last lane of P3 on which NTB will be enabled.

Permitted Choices: (Type: Value)(Default: 95)

- 112..126 – Ending lane for the link. Keep in mind the alignment and size requirements for the lane as described in PPR.
- 127 – selects x16 lane for NTB on P3.

- **PcdCfgNTBLinkSpeedP0P3**

Link Speed for Socket-0 P3 Link. NTB can work at PCIe Gen1, Gen2, Gen3 and Gen4 speeds. The user can select the desired speed using this PCD.

Permitted Choices: (Type: Value)(Default: 0x03)

- 1 – Gen1.
- 2 - Gen2.
- 3 - Gen3.
- 4 - Gen4.

- **PcdCfgNTBModeP0P3**

Select NTB Mode. User can select whether they want to enable NTB as primary or secondary.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0 – Disabled.
- 1 – NTB is primary.
- 2 - NTB is secondary.

3.6.2 DXIO Driver Common Structures

This section describes data structures common to the DXIO driver(s). These are shared by the family specific instances of the DXIO driver.

Related Definitions

The Host BIOS must create the DXIO Complex descriptor table. To help with this creation, the AGESA modules provides a set of macro definitions. These macro definitions are described later in the specification, see [DXIO Complex Installation Macros](#).

The host BIOS may find it desirable to update items in the DXIO_COMPLEX_DESCRIPTOR at POST based on the configuration of the platform at boot time. The data structure definitions can be found in AgesaPkg \Include\AmdPcieComplex.h and are listed below. By convention, the values described in the macro definitions are defined by the same name in a data structure, minus the “m” prefix. For example, mPortPresent in the macro definition would be represented in a data structure as “PortPresent”.

Please refer to the macro parameter definitions for detail about the entries.

DXIO_COMPLEX_DESCRIPTOR

```
typedef struct {
    IN      UINT32      Flags;
    IN      UINT32      SocketId;
    IN      PCIE_PORT_DESCRIPTOR *PciePortList;
    IN      VOID        *Reserved1;
    IN      VOID        *Reserved2;
```

```
} DXIO_COMPLEX_DESCRIPTOR;
```

The DXIO complex descriptor tables is an array of entries. There will be one complex descriptor for each node in the system containing a root bridge complex.

Flags	The high order bit indicates that this is the last item in the array.
SocketId	For multi-socket systems, this value identifies the socket associated with this DXIO_PORT_DESCRIPTOR list. Each socket in the system should provide a separate list of DXIO_PORT_DESCRIPTORs for that socket. This allows consistency in the lane numbers associated with each socket.
PciePortList	Pointer to array of PCIe® port descriptors, or is NULL. The Last element of the array will be a DESCRIPTOR_TERMINATE_LIST element.

DXIO_PORT_DESCRIPTOR

```
typedef struct {
    IN      UINT32      Flags;
    IN      PCIE_ENGINE_DATA EngineData;
    IN      PCIE_PORT_DATA Port;
    IN      ETHERNET_PORT_DATA EtherNet;
    IN      PHY_DATA    Phy;
} DXIO_PORT_DESCRIPTOR;
```

Each root complex may support one or more PCIe® ports. There will be one port descriptor entry in the array for each port supported by the system.

Flags	The high order bit indicates that this is the last item in the array.
EngineData	DXIO_ENGINE_DATA defines a device type and lanes associated with a given “engine” device.
Port	A structure describing port specific configuration info.
EtherNet	Ancillary data associated with a given EtherNet port.
Phy	Ancillary data for PHY programming customization. These are override values for tuning PHY parameters for PCIe® and SATA ports.

DXIO_ENGINE_DATA

```
typedef struct {
    IN      UINT8      EngineType;
    IN      UINT8      StartLane;
    IN      UINT8      EndLane;
    IN      UINT8      GpioGroupId;
    IN      UINT8      Reserved2;
    IN      UINT8      Reserved3;
    IN      UINT8      Reserved4;
} DXIO_ENGINE_DATA;
```

EngineType	Root complex ports can take on one of several characters. This item indicates which style this port is supporting. • 0 - Ignore engine configuration • 1 - PCIe® port (DxioPcieEngine) • 2 – SATA (DxioSataEngine)
-------------------	--

- 3 – XGBE (DxioXGBEEngine)
- 4 – GBE (DxioGBEEngine)

StartLane Start Lane ID (in reversed configurations StartLane > EndLane). Refer to lane descriptions and supported configurations in the BKDG.

EndLane End lane ID (in reversed configuration StartLane > EndLane). Refer to lane descriptions and supported configurations in the BKDG.

GpioGroupId Arbitrary number greater than 0 assigned by platform firmware indicating the GPIO identification which controls reset and status for this port. Each port with a unique GPIO set should have unique GpioGroupId assigned. All ports using the same GPIO to control reset should have the same GpioGroupId assigned. This identifier is used to communicate reset and status information bidirectionally between platform BIOS and DXIO firmware.

DXIO_PORT_DATA

```
typedef struct {
    IN      UINT8      PortPresent :1 ;
    IN      UINT8      Reserved1   :7 ;
    IN      UINT8      DeviceNumber :5 ;
    IN      UINT8      FunctionNumber:3 ;
    IN      UINT8      LinkSpeedCapability:2 ;
    IN      UINT8      AutoSpdChng  :2 ;
    IN      UINT8      Reserved2    :4 ;
    IN      UINT8      LinkAspm     :2 ;
    IN      UINT8      LinkAspmL1_1 :1 ;
    IN      UINT8      LinkAspmL1_2 :1 ;
    IN      UINT8      LinkAspmRsvd :4 ;
    IN      UINT8      LinkHotplug;
    IN      PCIE_PORT_MISC_CONTROL MiscControls;
    IN      APIC_DEVICE_INFO      ApicDeviceInfo;
    IN      PCIE_ENDPOINT_STATUS  EndpointStatus;
} DXIO_PORT_DATA;
```

PortPresent Enable PCIe® port for initialization.

DeviceNumber PCI Device number for port.

- 0 - Native port device number
- N - Port device number (See available configurations in the BKDG)

FunctionNumber Reserved.

LinkSpeedCapability PCIe® link speed

- 0 - Maximum supported by silicon
- 1 - Gen1
- 2 - Gen2
- 3 - Gen3

AutoSpdChng Setting for how to control speed change requests from the end-point.

- 0 – Keep the default definition
- 1 – Force auto speed changes disabled
- 2 – Force auto speed changes enabled

LinkAspm

ASPM control. (see AgesaPcieLinkAspm for additional options to control ASPM)

- bits 1:0 - LinkASPM
- bit 2 - LinkAspmL1_1
- bit 3 - LinkAspmL1_2
- bits 7:4 - reserved.

LinkHotplug

Hotplug control.

- HotplugDisabled - Hot plug is disabled.
- HotplugBasic - Basic hot plug is supported.
- HotplugServer - Server hot plug, standard PCIe® version, is supported.
- HotPlugServerSsd - Server hot plug for SolidStateDrives, version also known as NVMe (Non-Volatile Memory express), is supported.
- HotplugEnhanced - Enhanced hot plug is supported.

MiscControls**ApicDeviceInfo****EndpointStatus**

A structure of processing control flags.

IOAPIC device programming info

PCIe® endpoint (device connected to PCIe® port) status

- EndpointDetect - End point detected
- EndpointNotPresent - Endpoint not present (or connected). Used in case there is an alternative way to determine if the device is present on board or in slot. For example a GPIO can be used to determine device presence.

DXIO_APIC_DEVICE_INFO

```
typedef struct {
    IN      UINT8      LinkComplianceMode :1;
    IN      UINT8      LinkSafeMode       :2;
    IN      UINT8      SbLink              :1;
    IN      UINT8      ClkPmSupport        :1;
} PCIe_PORT_MISC_CONTROL;
```

LinkComplianceMode

Force port into compliance mode (device will not be trained, port outputs compliance pattern. See also [PcdPcieLinkComplianceModeAllPorts](#)).

LinkSafeMode

Safe mode PCIe capability. Used to limit the PCIe speed requested through PCIe_PORT_DATA::LinkSpeedCapability.

- 0 - port can advertize maximum supported capability.
- 1 - port limited to advertize capability and speed as PCIe Gen1.

SbLink

PCIe link type

- 0 - General purpose port
- 1 - Port connected to SB

ClkPmSupport

Clock Power Management Support

- 0 - Clock Power Management not configured.
- 1 - Clock Power Management configured according to PCIe device capability.

DXIO_APIC_DEVICE_INFO

```
typedef struct {
    IN      UINT8      GroupMap;
    IN      UINT8      Swizzle;
    IN      UINT8      BridgeInt;
} DXIO_APIC_DEVICE_INFO;
```

GroupMap

Group mapping for slot or endpoint device (connected to PCIe® port) interrupts .

- 0 - IGNORE THIS STRUCTURE AND USE RECOMMENDED SETTINGS.
- 1 - mapped to Grp 0 (Interrupts 0..3 of IO APIC redirection table).
- 2 - mapped to Grp 1 (Interrupts 4..7 of IO APIC redirection table).
- ...
- 8 - mapped to Grp 7 (Interrupts 28..31 of IO APIC redirection table).

Swizzle

Swizzle interrupt in the Group.

- 0 - ABCD
- 1 - BCDA
- 2 - CDAB
- 3 - DABC

BridgeInt

IOAPIC redirection table entry for PCIe® bridge interrupt

- 0 - Entry 0 of IO APIC redirection table.
- 1 - Entry 1 of IO APIC redirection table.
- ...
- 31 - Entry 31 of IO APIC redirection table.

ETHERNET_PORT_DATA

```
typedef struct {
    IN      ETH_PORT_PROPERTY0  EthPortProp0;
    IN      ETH_PORT_PROPERTY3  EthPortProp3;
    IN      ETH_PORT_PROPERTY4  EthPortProp4;
    IN      UINT32              PadMux0;
    IN      UINT32              PadMux1;
    IN      UINT32              MacAddressLo;
    IN      UINT32              MacAddressHi;
    IN      ETH_PORT_TXEQ       EthPortTxEq;
} ETHERNET_PORT_DATA
```

EthPortProp0	This is a bit packed value containing several operational flags. Please refer to the header files for further detail (AgesaPkg\Include\AmdPcieComplex.h).
EthPortProp3	This is a bit packed value containing several operational flags. Please refer to the header files for further detail.
EthPortProp4	This is a bit packed value containing several operational flags. Please refer to the header files for further detail.
PadMux0, PadMux1 MacAddressLo, MacAddressHi EthPortTxEq	

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.6.3 Published PPIs and Protocols

These are the PPIs and Protocols published by the DXIO driver for use by the system. The flag type PPIs are used to control synchronization and sequencing between PPI modules. The protocol is provided for the system to obtain information.

3.6.3.1 PEI_AMD_NBIO_PCIE_TRAINING_START_PPI

Summary

This PPI signals that the PCIe® training has started.

GUID

```
gAmdNbioPcieTrainingStartPpiGuid = {0xa99f2e33, 0x7f34, 0x4b5c,
    {0x83, 0xc1, 0x43, 0x92, 0x2f, 0x5b, 0x89, 0x58}}
```

PPI Interface Structure

```
typedef
struct _PEI_AMD_NBIO_PCIE_TRAINING_START_PPI {
    BOOLEAN TrainingStart;
} PEI_AMD_NBIO_PCIE_TRAINING_START_PPI;
```

Parameters

TrainingStart Always true, indicating that the PCI® e training has started.

Description This is a flag type PPI used to indicate to the system that PCIe® training has started. Once this PPI is published, a PIEM must not request access to PCIe® devices until the PEI_AMD_NBIO_PCIE_TRAINING_DONE_PPI has been published.

3.6.3.2 PEI_AMD_NBIO_PCIE_TRAINING_DONE_PPI

Summary

This PPI signals that the PCIe® training has completed.

GUID

```
gAmdNbIoPcieTrainingDonePpiGuid = {0x72166411, 0x442c, 0x4aab,  
    {0xa2, 0x60, 0x57, 0xd5, 0x84, 0xd3, 0x21, 0x39}}
```

PPI Interface Structure

```
typedef  
struct _PEI_AMD_NBIO_PCIE_TRAINING_DONE_PPI {  
    BOOLEAN TrainingComplete;  
} PEI_AMD_NBIO_PCIE_TRAINING_DONE_PPI;
```

Parameters

TrainingComplete Always true, indicating that the PCIe® training has completed.

Description This is a flag type PPI used to indicate to the system that PCIe® training has been completed. Once this PPI is published, a PIEM may then request access to PCIe® devices from the PEI Services using standard UEFI procedures.

3.6.3.3 AMD_NBIO_SERVICES_PROTOCOL

Summary

This protocol provides services to simplify the user task of relating the assigned PCI bus range to each NBIO root bridge on a platform

GUID

```
gAmdNbIoServicesProtocolGuid =  
    { 0xC4387568, 0x5AEF, 0x4E4D,  
      { 0x86, 0x70, 0x48, 0x2F, 0x22, 0x1E, 0x7C, 0xD7 } }  
};
```

Protocol Interface Structure

```
struct _AMD_NBIO_SERVICES_PROTOCOL {  
    AMD_NBIO_SERVICES_GET_SYSTEM_INFO      GetSystemInfo;  
    AMD_NBIO_SERVICES_GET_PROCESSOR_INFO    GetProcessorInfo;  
    AMD_NBIO_SERVICES_GET_ROOT_BRIDGE_INFO GetRootBridgeInfo;  
};
```

Members

GetSystemInfo A function to obtain total number of sockets, die, and root bridges on the platform.

GetProcessorInfo A function to return number of die and root bridges on a given socket.

GetRootBridgeInfo A function to return NBIO ID and PCI bus allocation per root bridge.

3.6.3.3.1 GetSystemInfo()

Summary

This function returns the total number of sockets, die, and root bridges on the platform.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *AMD_NBIO_SERVICES_GET_SYSTEM_INFO) (
IN      AMD_NBIO_SERVICES_PROTOCOL *This,
OUT     UINTN                      *NumberOfInstalledProcessors,
OUT     UINTN                      *TotalNumberOfDie,
OUT     UINTN                      *TotalNumberOfRootBridges
);
```

Parameters

<i>This</i>	This A pointer to this instance of the protocol.
<i>NumberOfInstalledProcessors</i>	The number of populated sockets on the platform.
<i>TotalNumberOfDie</i>	The combined number of die for all populated sockets on the platform.
<i>TotalNumberOfRootBridges</i>	The combined number of root bridges for all populated sockets on the platform.

Description

Typically this routine will be called only once to determine the number of populated sockets. The remaining functions in the protocol will then be called per-socket as needed to determine the desired root bridge ID and PCI bus assignments on each root bridge.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.

3.6.3.3.2 GetProcessorInfo()

Summary

This function returns the number of die and root bridges per socket.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *AMD_NBIO_SERVICES_GET_PROCESSOR_INFO) (
IN      AMD_NBIO_SERVICES_PROTOCOL *This,
IN      UINTN                      Socket,
OUT     UINTN                      *NumberOfDie,
OUT     UINTN                      *NumberOfRootBridges
);
```

Parameters

<i>This</i>	A pointer to this instance of the protocol.
<i>Socket</i>	The target of this query; a value 0 thru (NumberOfInstalledProcessors – 1) depending about which socket the user desires to obtain information.
<i>NumberOfDie</i>	The number of die on the socket specified above.
<i>NumberOfRootBridges</i>	The number of root bridges on the target socket.

Description

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.

3.6.3.3.3 GetRootBridgeInfo()

Summary

This function returns the NBIO ID and PCI bus allocation per root bridge.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *AMD_NBIO_SERVICES_GET_ROOT_BRIDGE_INFO) (
    IN      AMD_NBIO_SERVICES_PROTOCOL *This,
    IN      UINTN                      Socket,
    IN      UINTN                      Die,
    IN      UINTN                      Index,
    OUT     UINTN                      *NbIoPhysicalID,
    OUT     UINTN                      *NbIoLogicalID,
    OUT     UINTN                      *BusNumberBase,
    OUT     UINTN                      *BusNumberLimit,
    OUT     UINTN                      *PhysicalRootBridgeNumber
);
```

Parameters

<i>This</i>	A pointer to this instance of the protocol.
<i>Socket</i>	The value 0 thru (NumberOfInstalledProcessors – 1) depending on which socket the user is querying.
<i>Die</i>	The value 0 thru (NumberOfDie – 1) depending on which die the user is querying on the current socket.
<i>Index</i>	The value 0 thru (NumberOfRootBridges – 1) depending on which root bridge the user is querying on the current die.
<i>NbIoPhysicalID</i>	The silicon assigned unique ordinal ID of the root bridge specified by Index on the current socket and die.
<i>NbIoLogicalID</i>	The AGESA assigned logical ID of the root bridge specified by Index on the current socket and die.
<i>BusNumberBase</i>	Bus Number Base bits [7:0] for the current root bridge.
<i>BusNumberLimit</i>	Bus Number Limit bits [7:0] for the current root bridge.
<i>PhysicalRootBridgeNumber</i>	Reserved for future use.

Description

This routine will be called for each root bridge on the platform and return the Physical NBIO ID and assigned PCI bus number range for that particular root bridge.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.

Code Example

Below is a typical usage of the above functions to determine NBIO ID for each root bridge on a two socket F19M00 system.

```

VOID
NbIoRbEnum (    VOID    )
{
    EFI_STATUS Status;
    UINTN      Index;
    UINTN      Socket;
    UINTN      Die;
    UINTN      TotalNumberRootBridges;
    UINTN      NumberOfInstalledProcessors;
    UINTN      TotalNumberOfDie;
    UINTN      NumberOfDie;
    UINTN      NumberOfRootBridges;
    UINTN      NbioPhysicalID;
    UINTN      BusNumberBase;
    UINTN      BusNumberLimit;

    AMD_NBIO_SERVICES_PROTOCOL *NbIoServicesProtocol;

    Status = EFI_SUCCESS;

    // Locate NbIoServicesProtocol
    Status = gBS->LocateProtocol (&gAmdNbIoServicesProtocolGuid,
        NULL,
        (VOID **) &NbIoServicesProtocol);
    if(EFI_ERROR (Status)) return;

    IDS_HDT_CONSOLE (GNB_TRACE, "rbenum\n");

    // Number of sockets and rb's
    Status = NbIoServicesProtocol->GetSystemInfo (NbIoServicesProtocol,
        &NumberOfInstalledProcessors,
        &TotalNumberOfDie,
        &TotalNumberRootBridges);
    if(EFI_ERROR (Status)) return;

    IDS_HDT_CONSOLE (GNB_TRACE, "num_proc: %d, num_die: %d, num_rb: %d \n",
        NumberOfInstalledProcessors,
        TotalNumberOfDie,
        TotalNumberRootBridges);

    NumberOfDie = 0;
    NumberOfRootBridges = 0;
    for (Socket = 0; Socket < NumberOfInstalledProcessors; Socket++) {
        Status = NbIoServicesProtocol->GetProcessorInfo (NbIoServicesProtocol,
            Socket,
            &NumberOfDie,
            &NumberOfRootBridges);
        if (EFI_ERROR (Status)) return;

        IDS_HDT_CONSOLE (GNB_TRACE,
            "skt: %d, num_die/rb: %d/%d\n",
            Socket, NumberOfDie, NumberOfRootBridges);

        for (Die = 0; Die < NumberOfDie; Die++) {
            for (Index = 0; Index < NumberOfRootBridges; Index++) {
                Status = NbIoServicesProtocol->GetRootBridgeInfo(
                    NbIoServicesProtocol,
                    Socket, Die, Index,
                    &NbioPhysicalID, NULL,
                    &BusNumberBase, &BusNumberLimit,
                    NULL);
                if(EFI_ERROR (Status)) return;

                IDS_HDT_CONSOLE (GNB_TRACE,
                    "phys: %d, base: %08x, limit: %08x\n",
                    NbioPhysicalID, BusNumberBase, BusNumberLimit
                );
            }
        }
    }
}

```

3.6.4 Consumed PPIs and Protocols

This section describes the PPIs and Protocols 'consumed by' the NBIO driver. These are PPIs and Protocols provided by the host BIOS for use by the NBIO driver. The Host BIOS must provide these PPIs and Protocols in its code space and publish these PPIs and Protocols to the UEFI system for use by the NBIO driver.

These PPIs and Protocols describe the layout of the host platform PCIe® ports, lanes and devices within the platform design. The installations macros described in later sections can be used by the host platform to simplify the PCIe® complex descriptions.

3.6.4.1 PEI_AMD_NBIO_PCIE_COMPLEX_PPI

Summary

Call-out to the Host BIOS for services to obtain information about the PCIe® device assignment and layout.

GUID

```
#define AMD_NBIO_PCIE_COMPLEX_PPI_GUID
{0x324a4e15, 0x26ed, 0x4679, { 0xa9, 0xef, 0xba, 0x8a, 0x8f, 0xe7, 0x9a, 0xdb }}
```

PPI Interface Structure

```
typedef struct _PEI_AMD_NBIO_PCIE_COMPLEX_PPI {
    UINT32                      Revision;
    AMD_NBIO_PCIE_GET_COMPLEX_STRUCT  PcieGetComplex;
} PEI_AMD_NBIO_PCIE_COMPLEX_PPI;
```

Parameters

Revision	Revision identifier for this PPI interface.
PcieGetComplex	Function to obtain a pointer to the PCIe® complex descriptor tables.

Description

The Host BIOS must create and publish this PPI. The AGESA™ NBIO driver will use this PPI to obtain the platform dependent configuration information for the PCIe® ports, lanes and devices.

3.6.4.1.1 PcieGetComplex

Summary

Request a pointer to the PCIe® complex descriptor tables.

Prototype

```
typedef
EFI_STATUS
PcieGetComplex (
    IN      PEI_AMD_NBIO_PCIE_COMPLEX_PPI  *This,
    OUT     DXIO_COMPLEX_DESCRIPTOR        **UserConfig
);
```

Parameters

This	A pointer to this PPI.
UserConfig	Pointer to a caller provided storage space for a pointer to the PCIe® complex data structures.

Description

This function is used by the AGESA™ DXIO driver to obtain a pointer to the platform PCIe® complex descriptor tables. This procedure must be published by the platform driver. It must fill the caller provided pointer location with the address of the PCIe® complex table created for this platform.

Related Definitions

Please refer to the [DXIO common structures](#) section.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.6.4.2 NBIO_HOTPLUG_DESC_PROTOCOL

Summary

Call-out to the Host BIOS for services to obtain information about the optional PCIe® hotplug devices and GPIO extender connectivity.

GUID

```
gAmdHotplugDescProtocolGuid =
{0xe8d7e476, 0xedab, 0x4a80, {0x91, 0x19, 0xea, 0x5b, 0xcc, 0xc4, 0xc1, 0x95}}
```

Protocol Interface Structure

```
typedef struct _NBIO_HOTPLUG_DESC_PROTOCOL {
    UINT32                Revision;
    HOTPLUG_DESCRIPTOR    NbioHotplugDesc[];
} NBIO_HOTPLUG_DESC_PROTOCOL;
```

Members

Revision	Revision identifier for this Protocol interface.
NbioHotplugDesc	An array of structures describing the host BIOS HotPlug and GPIO mapping.

Description The Host BIOS must create and publish this Protocol if Server Hotplug ports are supported by the platform. The AGESA™ NBIO driver will use this Protocol to obtain the platform dependent configuration information for the server hotplug PCIe® ports and associated GPIO extenders. The NBIO driver will attempt to locate this Protocol via the UEFI services. If the protocol is not found then the NBIO driver will assume there are no HotPlug ports in the system.

3.6.4.2.1 HotPlug Structures

Summary

There are no functions in the HotPlug Protocol, just a pointer to the local Host BIOS copy of the description structures. The NBIO driver will access this structure to understand the platform topology. The initial structure is declared using the macros in [NBIO HotPlug Installation Macros](#); however, the host BIOS may choose to make POST time modifications. In which case, the Host BIOS will need to traverse these structures.

The hotplug descriptor table is an array of entries. There will be one hotplug descriptor to describe each server hotplug port in the system.

Related Definitions

HOTPLUG_DESCRIPTOR

```
typedef struct {
    UINT32          Flags;
    HOTPLUG_ENGINE_DATA Engine;
    PCIE_HOTPLUG_MAPPING Mapping;
    PCIE_HOTPLUG_FUNCTION Function;
    PCIE_HOTPLUG_RESET Reset;
    PCIE_HOTPLUG_GPIO Gpio;
} HOTPLUG_DESCRIPTOR;
```

Flags	The high order bit indicates that this is the last item in the array.
Engine	This structure identifies a specific slot in the platform.
Mapping	This structure describes logical information about the slot and its connection to the platform.
Function	This structure describes the GPIO extender connection to the server hotplug slot.
Reset	This structure describes for the link specifies the exact pin within the hot-plug I2C subsystem onto which the reset functionality is mapped.
Gpio	This structure describes stand-alone GPIO functionality in the GPIO expander. This functionality is reserved for future implementation.

Engine Configuration

```
typedef struct {
    IN    UINT8    StartLane;
    IN    UINT8    EndLane;
    IN    UINT8    SocketNumber;
    IN    UINT8    SlotNumber;
```

Short Description

StartLane	Start Lane ID (zero based). In a reversed configuration, (StartLane > EndLane). This value should correspond exactly to the definition the PCIe® topology.
EndLane	End lane ID (zero based). In a reversed configuration, (StartLane > EndLane). This value should correspond exactly to the definition the PCIe® topology.
SocketNumber	Socket Number from which this port is controlled.
SlotNumber	Unique identifier for the physical slot number of this port.

PCIe® Hotplug Mapping Descriptor

Each MAPPING descriptor contains information that identifies how a given hot-plug slot maps onto the APU device capabilities. Depending on whether the port is active, the <slotNum> MAPPING descriptor specifies a mapping to a particular PCIe® function/port in the socket. Additionally, the MAPPING descriptor also identifies the level of hot-plug capability, as well as the applicability of the RESET and GPIO descriptors. The MAPPING descriptor is always valid for an active hot-plug slot.

```
typedef struct {
    IN    UINT32    :3;
```

```

    IN    UINT32    Reserved0      :1;
    IN    UINT32    GpioDescriptorValid :1;
    IN    UINT32    ResetDescriptorValid:1;
    IN    UINT32    PortActive       :1;
    IN    UINT32    MasterSlaveAPU    :1;
    IN    UINT32    DieNumber         :4;
    IN    UINT32    ApertureId        :12;
    IN    UINT32    AlternateSlotNumber :6;
    IN    UINT32    PrimarySecondaryLink:1;
    IN    UINT32    Reserved1        :1;
} PCIE_HOTPLUG_MAPPING;

```

HotPlugFormat

Hotplug Format

- 0 - Simple Presence Detect
- 1 - PCIe® Express Module (Mode A). Mode A is the original AMD reference platform implementation.
- 2 - Enterprise SSD.
- 3 - PCIe® Express Module (Mode B). Mode B is a more industry standard implementation. The main difference between Mode A and Mode B is the GPIO expander pin assignments. Please refer to the Hot-Plug Application Note.

GpioDescriptorValid

GPIO Descriptor Valid

- 0 - This descriptor should be ignored.
- 1 - This descriptor is valid.

ResetDescriptorValid

Reset Descriptor Valid

- 0 - Reset is not supported from the I2C subsystem.
- 1 - Reset is supported from the I2C subsystem.

PortActive

This bit is essentially a valid marker indicating whether the hot-plug slot is implemented in the system and whether the information in the descriptor register is valid.

- 0 - This descriptor should be ignored.
- 1 - This descriptor is valid.

MasterSlaveAPU

This bit indicates the APU (Master or Slave die) the PCIe® device, that is mapped onto the hot-plug slot, resides. Note, the "Master" APU is the APU that hosts the I2C subsystem. The "Slave" APU communicates with the "Master" APU through a die-to-die communication link (DDCL).

- 0 - Connected to Master APU.
- 1 - Connected to Slave APU.

DieNumber

Indicates the Zeppelin Die number in the system.

ApertureId

Indicates the Aperture ID assigned to the PCIe® tile on the SMN network. This is used by the SMU to route the "SMU Status Transfer" to the correct PCIe® tile. This can also be seen as the PCIe® "core number", that contains PCIe® device that is mapped onto the hot-plug slot.

AlternateSlotNumber

When the descriptor is used for an Enterprise SSD form-factor, and the slot supports two PCIe® links, this field serves as a pointer to the PCIE_HOTPLUG_MAPPING<slotNum> descriptor associated with the 'other' PCIe® link. The primary descriptor shall point to the secondary descriptor, and the

secondary descriptor must point to the primary descriptor.
For a slot that only supports a single PCIe® link, the primary descriptor must set this field to zero.

PrimarySecondaryLink

When the descriptor is used for an Enterprise SSD form-factor, this bit specifies whether the descriptor is for the primary PCIe® device, or the secondary PCIe® device associated with the slot.

- 0 - This is the primary PCIe® device.
- 1 - This is the secondary PCIe® device.

PCIE_HOTPLUG_FUNCTION

Associated with each MAPPING descriptor, is a FUNCTION descriptor which indicates where the associated hot-plug functionality is mapped within the I2C subsystem. It is through the FUNCTION descriptor, that the exact pin (or group of pins) on a specific IO byte of a specific I2C GPIO device is identified to control the HotPlug functionality of the port. In addition, where a group of pins is allocated to the hot-plug functionality (based on the "HotPlugFormat" field of the corresponding MAPPING descriptor), a bitwise mask is provided to opt-out of specific functional elements of the specified hot-plug behavior, freeing the associated pin for other potential uses. The system supports a tree of I2C IO bus selectors and GPIO Expander devices.

```
typedef struct {          //[F17M00]
    IN  UINT32  I2CGpioBitSelector      :3;
    IN  UINT32  I2CGpioByteMapping      :3;
    IN  UINT32  I2CGpioDeviceMappingExt :2;
    IN  UINT32  I2CGpioDeviceMapping    :3;
    IN  UINT32  I2CDeviceType           :2;
    IN  UINT32  I2CBusSegment           :3;
    IN  UINT32  FunctionMask            :8;
    IN  UINT32  Reserved1               :8;
} PCIE_HOTPLUG_FUNCTION;
```

```
typedef struct {          // [F17M30]
    IN  UINT32  I2CGpioBitSelector      :3;
    IN  UINT32  I2CGpioByteMapping      :3;
    IN  UINT32  I2CGpioDeviceMapping    :5;
    IN  UINT32  I2CDeviceType           :2;
    IN  UINT32  I2CBusSegment           :5;
    IN  UINT32  FunctionMask            :8;
    IN  UINT32  Reserved1               :6;
} PCIE_HOTPLUG_FUNCTION;
```

I2CGpioBitSelector

I2C GPIO Bit Select: for a simple presence detect usage model, this field indicates which bit of which IO byte is used for the presence detect signal associated with this descriptor. For an Enterprise SSD form-factor, bits 0 and 1 must be 0 and bit 2 selects which nibble of the IO byte is used for the hot plug signals associated with this descriptor.

I2CGpioByteMapping

I2C GPIO IO Byte Mapping: indicates which IO byte of the I2C GPIO Expander device is used for the hot-plug signals associated with this descriptor.

I2CGpioDeviceMappingExt

I2CGpioDeviceMapping

These two fields combined form a 5-bit I2C GPIO Expander Device address. These are the lower 5 bits of

I2CDeviceType

the I2C 7-bit address for the I2C GPIO Expander device that is used to host the hot-plug signals associated with this descriptor.

I2C Device Type: indicates which of three I2C GPIO devices is used to host the hot-plug signals associated with this descriptor.

- 00 - PCA9539 address and register map; in support of legacy hot-plug implementations.
- 01 - PCA9535 address and register map; in support of newer hot-plug implementations.
- 10 - PCA9506 address and register map; in support of high slot-capacity implementations.
- 11 - reserved.

Note, PCA9535 and PCA9506 devices may be intermixed on the I2C interface. However, due to common addressing, a combined total of eight devices may be populated.

I2CBusSegment

I2C Bus Segment: in the context of an I2C subsystem constructed with PCA9545 I2C switches, this field specifies the downstream I2C bus segment, on one of two PCA9545 switches, on which the target I2C GPIO device is located. However, for I2C topologies that are not constructed with any PCA9545 devices, one value is reserved to identify that the I2C GPIO device does not reside behind any PCA9545 I2C switch. Note: I2C addresses below are the 7-bit address of the switch device.

For 3-bit Device Mappings: [F17M00]

- 000 - PCA9545, address=00, downstream I2C bus=00.(I2C address = 0x70)
- 001 - PCA9545, address=00, downstream I2C bus=01.(I2C address = 0x70)
- 010 - PCA9545, address=00, downstream I2C bus=10.(I2C address = 0x70)
- 011 - PCA9545, address=00, downstream I2C bus=11.(I2C address = 0x70)
- 100 - PCA9545, address=01, downstream I2C bus=00.(I2C address = 0x71)
- 101 - PCA9545, address=01, downstream I2C bus=01.(I2C address = 0x71)
- 110 - PCA9545, address=01, downstream I2C bus=10.(I2C address = 0x71)
- 111 - no PCA9545; I2C bus directly connected to APU socket.

For 5-bit Device Mappings: [F17M30]

- 01000 - PCA9545, address=01, downstream I2C bus=11.(I2C address = 0x71)

- 01001 - PCA9545, address=02, downstream I2C bus=00.(I2C address = 0x72)
- 01010 - PCA9545, address=02, downstream I2C bus=01.(I2C address = 0x72)
- 01011 - PCA9545, address=02, downstream I2C bus=10.(I2C address = 0x72)
- 01100 - PCA9545, address=02, downstream I2C bus=11.(I2C address = 0x72)
- 01101 - PCA9545, address=03, downstream I2C bus=00.(I2C address = 0x73)
- 01110 - PCA9545, address=03, downstream I2C bus=01.(I2C address = 0x73)
- 01111 - PCA9545, address=03, downstream I2C bus=10.(I2C address = 0x73)
- 10000 - PCA9545, address=03, downstream I2C bus=11.(I2C address = 0x73)

FunctionMask

Function Mask: when the descriptor is used for a multi-bit hot-plug functional format, this field can be used to opt-out of specific pin functionality. When an 8-bit GPIO group is allocated to the hot-plug function, all eight bits of this field are valid. When a 4-bit GPIO group is allocated to the hot-plug function, only the lower four bits of this field are valid. When valid, if bitN of this field is set, the sub-element of the hot-plug function that is mapped to the Nth IO bit of the byte/nibble is disabled. Once disabled via this field, the associated pin is made available for other potential purposes such as a RESET or GPIO capability.

PCIE_HOTPLUG_RESET

When PCIE_HOTPLUG_MAPPING<slotNum>.ResetDescriptorVld is set to 0, reset functionality is not tied to the hot-plug slot and must be furnished in some other manner. However, when the ResetDescriptorVld bit-field is set to 1, the RESET descriptor for the <slotNum> link specifies the exact pin within the hot-plug I2C subsystem onto which the reset functionality is mapped, enabling control of the reset functionality through a BIOS accessible API.

For F17M00:

```
typedef struct {
    IN    UINT32    Reserved0           :3;
    IN    UINT32    I2CGpioByteMapping :3;
    IN    UINT32    Reserved1           :2;
    IN    UINT32    I2CGpioDeviceMapping :3;
    IN    UINT32    I2CDeviceType       :2;
    IN    UINT32    I2CBusSegment       :3;
    IN    UINT32    ResetSelect         :8;
} PCIE_HOTPLUG_RESET;
```

For F17M30:

```
typedef struct {
    IN UINT32    Reserved0           :3;
    IN UINT32    I2CGpioByteMapping  :3;
    IN UINT32    I2CGpioDeviceMapping :5;
    IN UINT32    I2CDeviceType       :2;
    IN UINT32    I2CBusSegment       :5;
    IN UINT32    ResetSelect         :8;
    IN UINT32    Reserved1           :6;
} PCIE_HOTPLUG_RESET;
```

I2CGpioByteMapping

I2C GPIO IO Byte Mapping: indicates which IO byte of the I2C GPIO device is used for the hot-plug signals associated with this descriptor.

I2CGpioDeviceMapping

Indicates the lower bits of the 7-bit I2C address for the I2C GPIO Expander Device that is used to host the hot-plug signals associated with this descriptor.

I2CDeviceType

I2C Device Type: indicates which of three I2C GPIO devices is used to host the hot-plug signals associated with this descriptor.

- 00 - PCA9539 address and register map; in support of legacy hot-plug implementations.
- 01 - PCA9535 address and register map; in support of newer hot-plug implementations.
- 10 - PCA9506 address and register map; in support of high slot-capacity implementations.
- 11 - reserved.

Note, PCA9535 and PCA9506 devices may be intermixed on the I2C interface. However, due to common addressing, a combined total of eight devices may be populated.

I2CBusSegment

I2C Bus Segment: in the context of an I2C subsystem constructed with PCA9545 I2C switches, this field specifies the downstream I2C bus segment, on one of two PCA9545 switches, on which the target I2C GPIO device is located. However, for I2C topologies that are not constructed with any PCA9545 devices, one value is reserved to identify that the I2C GPIO device does not reside behind any PCA9545 I2C switch. Note: I2C addresses below are the 7-bit address of the switch device.

For 3-bit Device Mappings: [F17M00]

- 000 - PCA9545, address=00, downstream I2C bus=00. (I2C address = 0x70)
- 001 - PCA9545, address=00, downstream I2C bus=01. (I2C address = 0x70)
- 010 - PCA9545, address=00, downstream I2C bus=10. (I2C address = 0x70)
- 011 - PCA9545, address=00, downstream I2C bus=11. (I2C address = 0x70)
- 100 - PCA9545, address=01, downstream I2C bus=00. (I2C address = 0x71)

- 101 - PCA9545, address=01, downstream I2C bus=01.
(I2C address = 0x71)
- 110 - PCA9545, address=01, downstream I2C bus=10.
(I2C address = 0x71)
- 111 - no PCA9545; I2C bus directly connected to APU socket.

For 5-bit Device Mappings: [F17M30]

- 01000 - PCA9545, address=01, downstream I2C bus=11.
(I2C address = 0x71)
- 01001 - PCA9545, address=02, downstream I2C bus=00.
(I2C address = 0x72)
- 01010 - PCA9545, address=02, downstream I2C bus=01.
(I2C address = 0x72)
- 01011 - PCA9545, address=02, downstream I2C bus=10.
(I2C address = 0x72)
- 01100 - PCA9545, address=02, downstream I2C bus=11.
(I2C address = 0x72)
- 01101 - PCA9545, address=03, downstream I2C bus=00.
(I2C address = 0x73)
- 01110 - PCA9545, address=03, downstream I2C bus=01.
(I2C address = 0x73)
- 01111 - PCA9545, address=03, downstream I2C bus=10.
(I2C address = 0x73)
- 10000 - PCA9545, address=03, downstream I2C bus=11.
(I2C address = 0x73)

ResetSelect

Reset Select: this bit-field is used to specify which pin within the selected I2C IO byte (identified by the other fields in this descriptor) is used for the reset function associated with the <slotNum> PCIe® link. Since reset is a singular function, only one bit in this field may be set to one. When the Nth bit of this field is set to 1, the Nth bit of the selected I2C IO byte serves as the platform reset signal for the associated PCIe® link. Note, although this descriptor identifies a single bit, this field has been defined in this manner so as to be consistent with both the "FunctionMask" field in the FUNCTION descriptor and the "GPIOSelect" field in the GPIO descriptor.

3.6.5 DXIO Complex Installation Macros

The macros in this section are used to build the DXIO Complex data structures that describe the host platform environment.

AGESA provides a set of MACRO definitions to simplify the definition of the DXIO_PORT_DESCRIPTOR entries in the DXIO complex. Each entry requires only 2 macros. The first macro defines the type of “engine” and how it is connected, while the second macro defines more information specific to the type of engine.

3.6.5.1 DXIO_ENGINE_DATA_INITIALIZER()

Summary

This macro defines the device type associated with the engine and to which lanes it is connected.

Prototype

```
DXIO_ENGINE_DATA_INITIALIZER (
mType,           // The device type associated with this engine
mStartLane,      // The starting lane connected to this device
mEndLane,        // The ending lane connected to this device
mHotplug,        // The type of hotplug associated with this device
mGpioGroupId     // And identifier for a GPIO or group of GPIOs for this device
)
```

Parameters

mType

This value specifies the device type associated with this engine. The DXIO firmware supports the following engine types:

- DxioUnusedEngine – This engine will be ignored. This engine type can be used “disable” entries in the DXIO complex without resorting the table to remove the entry.
- DxioPcieEngine – Indicates that the associated lanes are connected to a PCIe® device or slot.
- DxioSATAEngine – Indicates that the associated lanes are connected to a SATA device or connector
- DxioEthernetEngine – Indicates that the associated lane is connected to one of the internal EtherNet controllers. Note that an EtherNet engine can only be a single lane, but the entire PHY group of lanes must be used for EtherNet or not used.

mStartLane

This value specifies the starting lane for this device. Note that a value for mStartLane that is greater than the mEndLane for the same device indicates that the link is connected in “reverse” and the DXIO subsystem will apply link reversal to the device. While link reversal may be auto-detected and configured, this manual method of applying link reversal may be useful for supporting endpoint devices that do not function well when reversed.

Lanes are generally numbered consecutively across a multi-die implementation. For example, if each die has 32 lanes, then Die 0 lanes are numbered 0-31, while Die 1 lanes are numbered 32-63, Die 2 lanes are numbered 64-95, etc. For a multi-socket platform, the lane numbers for Socket 1 restart at 0 and are numbered consecutively as they are in Socket 0. For this reason, Socket 1 is defined in a second port list.

mEndLane

This value specifies the ending lane for this device.

mHotplug

This value specifies the type of hotplug associated with a hotplug device. This value generally applies only to PCIe® devices, but is included in the engine data for future support of multi-protocol hotplug connections.

- DxioHotplugDisabled – This value indicates that this device does not support hotplug.

- **DxioHotplugBasic** – This value indicates that the device supports hotplug indicated by a single GPIO that indicates insertion/removal. This method allows the link to be powered down when a device is removed in order to minimize power consumption. It also requires additional runtime support from platform BIOS to detect an insertion/removal event and notify the DXIO subsystem through the ALIB ACPI library.
- **DxioHotplugServer** – This value indicates that the device supports “server” hotplug associated with a group of GPIOs that provide additional control. Details are TBD.
- **DxioHotplugEnhanced** – This value indicates that the device supports hotplug through an “always on and searching” mechanism. Insertion/removal is reported by the bridge interrupt to the operating system. This method does not require runtime support from the platform BIOS but is less power efficient than **DxioHotplugBasic**.

mGpioGroupId

This value specifies a unique identifier for a GPIO or group of GPIOs associated with this device. For example, if a device reset has a manual control through a GPIO, then this GPIO should be identified by this value. Note that the value does not have to represent the GPIO number specifically. It is simply a unique identifier for communicating requests between DXIO firmware and platform BIOS.

3.6.5.2 DXIO_PORT_DATA_INITIALIZER_PCIE()**Summary**

This group of four macros define data specific to the PCIe® type of device associated with the preceding engine. There are four nearly identical macros that cover different portions of the PCIe® fabric.

- **DXIO_PORT_DATA_INITIALIZER_PCIE** covers the traditional PCIe® ports
- **DXIO_PORT_DATA_INITIALIZER_CHIP** This marco identifies the location of the chipset extension chip (an IO chip external to the main SoC) within the PCIe® framework. If the chipset extension is present, it must be identified by this initializer. Only one “_CHIP” initializer is supported on the platform.
- **DXIO_PORT_DATA_INITIALIZER_PCIE_V2** covers the links adding the ability to assign a CLKREQ# signal to the end point device.
- **DXIO_PORT_DATA_INITIALIZER_PCIE_V3** covers the links adding the ability to specify how the link handles dynamic speed change requests from the end point device.

Links of all four types may exist in the system simultaneously, thus all four are needed to separate the configurations.

Prototype

```
DXIO_PORT_DATA_INITIALIZER_PCIE (
mPortPresent,           // Port presence indicator
mDevAddress,            // The desired device number for this PCIe device
mDevFunction,           // The desired function number for this PCIe device
mHotplug,               // The type of PCIe hotplug associated with this device
mMaxLinkSpeed,          // The maximum PCIe GenX speed desired for this device
mMaxLinkCap,            // The maximum PCIe GenX speed this device is capable of
mAspm,                  // The ASPM configuration for this device
mAspmL1_1,              // ASPM L1_1 support indicator
mAspmL1_2,              // ASPM L1_2 support indicator
mClkPmSupport,          // Clock Power Management support indicator for this device
mEqPreset               // port-specific Equalization Preset constant.
```

)

```

DXIO_PORT_DATA_INITIALIZER_CHIP (
mPortPresent,          // Port presence indicator
mDevAddress,           // The desired device number for this PCIe device
mDevFunction,          // The desired function number for this PCIe device
mHotplug,              // The type of PCIe hotplug associated with this device
mMaxLinkSpeed,         // The maximum PCIe GenX speed desired for this device
mMaxLinkCap,           // The maximum PCIe GenX speed this device is capable of
mAspm,                 // The ASPM configuration for this device
mAspmL1_1,             // ASPM L1_1 support indicator
mAspmL1_2,             // ASPM L1_2 support indicator
mClkPmSupport          // Clock Power Management support indicator for this device
)

```

```

DXIO_PORT_DATA_INITIALIZER_PCIE_V2 (
mPortPresent,          // Port presence indicator
mDevAddress,           // The desired device number for this PCIe device
mDevFunction,          // The desired function number for this PCIe device
mHotplug,              // The type of PCIe hotplug associated with this device
mMaxLinkSpeed,         // The maximum PCIe GenX speed desired for this device
mMaxLinkCap,           // The maximum PCIe GenX speed this device is capable of
mAspm,                 // The ASPM configuration for this device
mAspmL1_1,             // ASPM L1_1 support indicator
mAspmL1_2,             // ASPM L1_2 support indicator
mClkPmSupport,         // Clock Power Management support indicator for this device
mClkReq                // ASPM clock assignment
)

```

```

DXIO_PORT_DATA_INITIALIZER_PCIE_V3 (
mPortPresent,          // Port presence indicator
mDevAddress,           // The desired device number for this PCIe device
mDevFunction,          // The desired function number for this PCIe device
mHotplug,              // The type of PCIe hotplug associated with this device
mMaxLinkSpeed,         // The maximum PCIe GenX speed desired for this device
mMaxLinkCap,           // The maximum PCIe GenX speed this device is capable of
mAspm,                 // The ASPM configuration for this device
mAspmL1_1,             // ASPM L1_1 support indicator
mAspmL1_2,             // ASPM L1_2 support indicator
mClkPmSupport,         // Clock Power Management support indicator for this device
mClkReq,               // ASPM clock assignment
mAutoSpdChange         // Identifies special handling for upstream speed changes
)

```

Parameters

These macros share most of the same parameters.

mPortPresent

This value indicates whether this port is enabled or not.

Possible values are:

- DxioPortDisabled
- DxioPortEnabled

mDevAddress, ***mDevFunction***

These values indicate the desired device and function number to be assigned to the root port of this device. If the values are “0” then DXIO driver will assign the “default” device and function numbers associated with the port. The value will only be changed from the default if the default device and function are requested by another device in this list.

mHotplug

This value specifies the type of hotplug associated with a hotplug device. This value generally applies only to PCIe® devices, but is included in the engine data for future support of multi-protocol hotplug connections.

Permitted values are:

- DxioHotplugDisabled – This value indicates that this device does not support hotplug.
- DxioHotplugBasic – This value indicates that the device supports hotplug indicated by a single GPIO that indicates insertion/removal. This method allows the link to be powered down when a device is removed in order to minimize power consumption. It also requires additional runtime support from platform BIOS to detect an insertion/removal event and notify the DXIO subsystem through the ALIB ACPI library.
- DxioHotplugServer – This value indicates that the device supports “server” hotplug associated with a group of GPIOs that provide additional control. Details are TBD.
- DxioHotplugEnhanced – This value indicates that the device supports hotplug through an “always on and searching” mechanism. Insertion/removal is reported by the bridge interrupt to the operating system. This method does not require runtime support from the platform BIOS but is less power efficient than DxioHotplugBasic.

mMaxLinkSpeed

This value indicates the maximum PCIe® GenX speed to trained by this PCIe® port.

Permitted values are:

- DxioGenMaxSupported – use the maximum speed supported
- DxioGen1 – limit the speed to Gen1
- DxioGen2 – limit the speed to Gen2
- DxioGen3 – allow speeds up to Gen3

mMaxLinkCap

This value indicates the maximum PCIe® GenX speed to be advertised by this PCIe® port.

Permitted values are:

- DxioGenMaxSupported – use the maximum speed supported
- DxioGen1 – limit the speed to Gen1
- DxioGen2 – limit the speed to Gen2
- DxioGen3 – allow speeds up to Gen3

mAspm

This value specifies the supported ASPM link power management for this device.

Permitted values are:

- DxioAspmDisabled – ASPM is Disabled
 - DxioAspmL0s – PCIe® L0s link state is supported
 - DxioAspmL1 – PCIe® L1 link state is supported
 - DxioAspmL0sL1 – PCIe® L0s and L1 link states are supported
- ASPM control. (see mAspm above for additional option to control ASPM).

mAspmL1_1,* *mAspmL1_2

Permitted values are:

- 0 - Disabled
- 1 - Enabled

mClkPmSupport

This value indicates the state of Clock Power Management support for this device.

Permitted values are:

- DxioClkPmSupportDisabled – Disable
- DxioClkPmSupportEnabled – Enable

mEqPreset

This field specifies a per-port value that will be used as the starting preset when equalization occurs on the link during initial PCIe® link training.

Valid values are 0 to 9, inclusive.

Refer to the configuration control [PcdPcieEqSearchMode](#) for more details on how to use this field.

mClkReq

This value specifies the supported ASPM link power management for this device. Supported values are as follows:

- 0- NONE
- 1 - CLKREQ0 signal
- 2 - CLKREQ1 signal
- 3 - CLKREQ2 signal
- 4 - CLKREQ3 signal
- 5 - CLKREQG signal

mAutoSpdChange

This value indicates special handling for upstream speed change requests initiated by a PCIe® endpoint. In some cases it is desired to disable requests from endpoint devices for speed changes in order to maintain a reliable data stream at a lower data rate. By default auto speed change requests are supported for Gen2 and Gen3 devices and disabled for Gen1 devices.

- 0 – Keep the default definition
- 1 – Force auto speed changes disabled
- 2 – Force auto speed changes enabled

3.6.5.3 DXIO_PORT_DATA_INITIALIZER_SATA()**Summary**

This pair of macros define data specific to the SATA type of device associated with the preceding engine. There are two nearly identical macros that cover different portions of the SATA fabric.

- DXIO_PORT_DATA_INITIALIZER_SATA covers the legacy PCIe® ports
- DXIO_PORT_DATA_INITIALIZER_SATA_V2 covers the new channels supporting longer trace lengths.

Links of all both types may exist in the system simultaneously, thus both are needed to separate the configurations.

Prototype

```
DXIO_PORT_DATA_INITIALIZER_SATA(  
    mPortPresent  
)
```

```
DXIO_PORT_DATA_INITIALIZER_SATA_V2(  
    mPortPresent  
    mChannelType
```

)

Parameters

mPortPresent This value indicates whether this port is enabled or not.

Permitted values are:

- DxioPortDisabled
- DxioPortEnabled

mChannelType Specifies a trace length descriptor:

- 0 - Channel Tye not specified
- 1 - Channel Type is short trace
- 2 - Channel Type is long trace

3.6.5.4 DXIO_PORT_DATA_INITIALIZER_ENET ()**Summary**

This macro defines data specific to the eNet type of device associated with the preceding engine.

Prototype

```

DXIO_PORT_DATA_INITIALIZER_ENET (
mPortPresent,          // Port presence indicator
mPortNum,              // Unique port number for each port
mPlatConf,             // Platform Config
mMdioId,               // MDIO ID of the PHY associated with this port
mSuppSpeed,           // Ethernet speeds supported at the platform level
mConnType,            // Connection type
mMdioReset,           // MDIO Reset Type
mMdioGpioResetNum,    // MDIO Integrated GPIO Reset Number
mSfpGpioAdd,          // SFP+/MDIO Reset GPIO Address
mTxFault,             // TX_FAULT/MDIO Reset GPIO Number
mRS,                  // PCA9535 GPIO number used to control the SFP+ RS signal
mModAbs,              // PCA9535 GPIO number used to control the SFP+ Mod_ABS
mRxLoss,              // PCA9535 GPIO number used to control the SFP+ Rx_LOS
mSfpGpioMask,         // SFP+ sideband signals which are not supported
mSfpMux,              // Lower address of the PCA9545 I2C multiplexor
mSfpBusSeg,           // Downstream channel of the PCA9545 I2C multiplexor
mSfpMuxUpAdd,         // Upper address of the PCA9545 I2C multiplexor
mRedriverAddress,     // I2C or MDIO address used to control a platform level redriver
mRedriverInterface,   // Redriver Interface
mRedriverLane,        // Redriver Lane
mRedriverModel,       // Redriver Model
mRedriverPresent,     // Redriver Present
mPadMux0,             // PadMux 0
mPadMux1,             // PadMux 1
mMacAddressLo,        // The lower 32-bits of the MAC address
mMacAddressHi,        // The upper 16-bits of the MAC address
mTxEqPre,             // TX Equalization Pre
mTxEqMain,            // TX Equalization Main
mTxEqPost,            // TX Equalization Post
)

```

Parameters

mPortPresent This value indicates whether this port is enabled or not.

Permitted values are:

- DxioPortDisabled
- DxioPortEnabled

mPortNum Set this to a unique value for each integrated Ethernet port in the system. This value must be unique within and between nodes.

mPlatConf	Set this field based on the platform connectivity for the port Possible values are: • 0 = Reserved • 1 = 10G/1G Backplane • 2 = 2.5G Backplane • 3 = Soldered down 1000Base-T • 4 = Soldered down 1000Base-X • 5 = Soldered down NBase-T • 6 = Soldered down 10GBase-T • 7 = Soldered down 10GBase-R • 8 = SFP+ connector • 9-15 = Reserved
mMdioId	If an external PHY is connected using an MDIO sideband interface, set this field to the MDIO ID of the PHY associated with this port. If MDIO is not used by this port, set this to 0.
mSuppSpeed	Indicates the Ethernet speeds supported at the platform level. For example, if the controller is always connected over backplane to a 10G/1G switch port, then 10G and 1G support should be indicated. The actual speed of the port will be set by the driver based on auto negotiation or other means. For SFP+ connectors, 10G, 1G and 100M support should be indicated. Possible values are: • 1xxx b = 10G supported • x1xx b = 2.5G supported • xxx1 b = 100M supported
mConnType	Indicates additional information about the device connected to the processor's SERDES PHY. Possible values are: • 100b = Backplane connection. No sideband interface. • 010b = MDIO PHY • 001b = SFP+ connection (I2C sideband interface) • 000b = Port not used
mMdioReset	If an external MDIO PHY is used, this field indicates the method used by the software driver to control for the reset to that PHY. Possible values are: • 11b = Reserved • 10b = Integrated GPIO • 01b = I2C GPIO • 00b = None
mMdioGpioResetNum	If MDIO Reset Type is set to 10b, this field indicates the Ethernet integrated GPIO number used to control the reset.
mSfpGpioAdd	If an external MDIO PHY is used in combination with an I2C GPIO reset, this field indicates the lower I2C address of the

PCA9535 GPIO expander used to drive the reset. If this port is used as part of an SFP+ connection, this field indicates the lower I2C address of the PCA9535 GPIO expander used to drive the SFP+ sideband signals.

mTxFault Set this field to the PCA9535 GPIO number used to control either the SFP+ TX_FAULT signal or the MDIO reset associated with this port. If neither of those features are in use, set this to 0.

mRs Set this field to the PCA9535 GPIO number used to control the SFP+ RS signal. If this port is not used with an SFP+ connector, set this field to 0.

mModAbs Set this field to the PCA9535 GPIO number used to control the SFP+ Mod_ABS signal. If this port is not used with an SFP+ connector, set this field to 0.

mRxLoss Set this field to the PCA9535 GPIO number used to control the SFP+ Rx_LOS signal. If this port is not used with an SFP+ connector, set this field to 0.

mSfpGpioMask Set bits in this field to indicate SFP+ sideband signals which are not supported at the platform level. If this port is not used with an SFP+ connector, set this field to 0h.

Possible values are:

- 1xxxb = Rx_LOS not supported
- x1xxb = Mod_ABS not supported
- xx1xb = RS not supported
- xxx1 = TX_FAULT not supported

mSfpMux Set this field to indicate the lower address of the PCA9545 I2C multiplexor used to drive the SFP+ connector. If no multiplexing is used, set this field to 111b. If this port is not used with an SFP+ connector, set this field to 0.

Possible values are:

- 000b-011b = PCA9545 lower address
- 100b-110b = Reserved
- 111b = SFP+ directly connected to I2C

mSfpBusSeg Set this field to indicate the downstream channel of the PCA9545 I2C multiplexor used to drive the SFP+ connector. If no multiplexing is used, set this field to 111b. If this port is not used with an SFP+ connector, set this field to 0.

Possible values are:

- 000b-011b = PCA9545 lower address
- 100b-110b = Reserved
- 111b = SFP+ directly connected to processor I2C

mSfpMuxUpAdd Set this field to indicate the upper address of the PCA9545 I2C multiplexor used to drive the SFP+ connector. PCA9545 comes in 3 variants each with a different upper address.

Possible values are:

- 11100b = PCA9545A

- 11010b = PCA9545B
- 10110b = PCA9545C
- 11111b = SFP+ directly connected to processor I2C

If no multiplexing is used, set this field to 11111b. If this port is not used with an SFP+ connector, set this field to 0.

mRedriverAddress

This field encodes the I2C or MDIO address used to control a platform level redriver connected to this port. If no redriver is used, set this field to 0. If the redriver is connected using MDIO, the upper 2 address bits must be set to 0.

mRedriverInterface

This field indicates the control interface used by the redriver.

Possible values are:

- 0=MDIO
- 1=I2C

mRedriverLane

This field indicates the lane on which the redriver is connected to the processor port.

mRedriverModel

This field indicates the redriver model.

Possible values are:

- 000b=InPhi 4223
- 001b=InPhi 4227
- All other encodings reserved.

mRedriverPresent

Indicates if the redriver is to be used.

Possible values are:

- 0=Platform level redriver is not used
- 1=Platform level redriver is connected to the processor PHY

mPadMux0,***mPadMux1******mMacAddressLo,******mMacAddressHi***

These values specify the Sideband Interface Configuration as specified in the PPR.

These values specify the lower and upper 32-bits of the MAC address for this port.

Possible values are:

mTxEqPre,***mTxEqMain,******mTxEqPost***

These values specify the TX Equalization settings for the PHY. Values are TBD.

3.6.5.5 PHY Tuning Macros**Summary**

This trio of macros are used to specify override values for specific PHY parameters.

- PHY_PARAMS_START - This macro defines the beginning of a PHY parameter list within a port descriptor.
- PHY_PARAM - specifies an override value for a specific PHY parameter. There can be more than one of these macros uses in this block, but only one per PHY parameter.
- PHY_PARAMS_END - This macro defines the end of a PHY parameter list within a port descriptor.

These can be used with any PCIe® or SATA data port to fine tune the PHY operation. The ENET type data port has PHY valued within its structure so these separate macros are not needed for ENET ports and would be ignored if present.

Prototype

```
PHY_PARAM(  
    mPhyParam,  
    mPhyValue  
)
```

Parameters

mPhyParam

This is an enum that identifies a specific PHY parameter to override. Note that some values may be specified individually for each supported speed.

mPhyValue

Specifies the value to be assigned to the indicated PHY parameter.

rx_vref_ctrl	SSC_OFF_FRUG1	SSC_OFF_PHUG1
GEN1_txX_eq_pre	GEN2_txX_eq_pre	GEN3_txX_eq_pre
GEN1_txX_eq_main	GEN2_txX_eq_main	GEN3_txX_eq_main
GEN1_txX_eq_post	GEN2_txX_eq_post	GEN3_txX_eq_post
GEN1_txX_iboot_lv1	GEN2_txX_iboot_lv1	GEN3_txX_iboot_lv1
GEN1_txX_vboost_en	GEN2_txX_vboost_en	GEN3_txX_vboost_en
GEN1_Unsupported1	GEN2_Unsupported1	GEN3_rxX_adapt_afe_en
GEN1_Unsupported2	GEN2_Unsupported2	GEN3_rxX_adapt_dfe_en
GEN1_rxX_eq_att_lv1	GEN2_rxX_eq_att_lv1	GEN3_rxX_eq_att_lv1
GEN1_rxX_eq_vga1_gain	GEN2_rxX_eq_vga1_gain	GEN3_rxX_eq_vga1_gain
GEN1_rxX_eq_vga2_gain	GEN2_rxX_eq_vga2_gain	GEN3_rxX_eq_vga2_gain
GEN1_rxX_eq_ctle_pole	GEN2_rxX_eq_ctle_pole	GEN3_rxX_eq_ctle_pole
GEN1_rxX_eq_ctle_boost	GEN2_rxX_eq_ctle_boost	GEN3_rxX_eq_ctle_boost
GEN1_rxX_eq_dfe_tap1	GEN2_rxX_eq_dfe_tap1	GEN3_rxX_eq_dfe_tap1
GEN1_tx_vboost_lv1	GEN2_tx_vboost_lv1	GEN3_tx_vboost_lv1
GEN1_SSC_OFF_FRUG1	GEN2_SSC_OFF_FRUG1	GEN3_SSC_OFF_FRUG1
GEN1_SSC_OFF_PHUG1	GEN2_SSC_OFF_PHUG1	GEN3_SSC_OFF_PHUG1
GEN1_SSC_ON_FRUG1	GEN2_SSC_ON_FRUG1	GEN3_SSC_ON_FRUG1
GEN1_SSC_ON_PHUG1	GEN2_SSC_ON_PHUG1	GEN3_SSC_ON_PHUG1
GEN1_txX_pre_deemphasis	GEN2_txX_pre_deemphasis	GEN3_txX_pre_deemphasis
GEN1_txX_post_deemphasis	GEN2_txX_post_deemphasis	GEN3_txX_post_deemphasis

Reference: source file \AgesaPkg\Include\AmdPcieComplex.h : DXIO_PHY_PARAM_TYPE). This may, or may not be the full list specified by the PCIe specification, but these are the ones recognized by the AGESA software. The purpose of these parameters are beyond the scope of this document. The reader should refer to the PCIe Specification.

EXAMPLE: PCIe® Port Descriptor with PHY parameters

```
{ // GFX - x16 slot. Entry 0  
0,
```

```

DXIO_ENGINE_DATA_INITIALIZER (DxioPcieEngine, 24, 31,
DxioHotplugDisabled, 1), //DEVICE_ID_PCIE_X16_SLOT
DXIO_PORT_DATA_INITIALIZER_PCIE_V2 (
DxioPortEnabled, // Port Present
3, // Requested Device
1, // Requested Function
DxioHotplugDisabled, // Hotplug
DxioGenMaxSupported, // Max Link Speed
DxioGenMaxSupported, // Max Link Capability
DxioAspmL0sL1, // ASPM
DxioAspmDisabled, // ASPM L1.1 disabled
DxioAspmDisabled, // ASPM L1.2 disabled
DxioClkPmSupportDisabled, // Clock PM
DxioClkReqG // CLKREQ#
)
PHY_PARAMS_START
PHY_PARAM (GEN1_txX_eq_pre, 1),
PHY_PARAM (GEN1_txX_eq_main, 3),
PHY_PARAM (GEN1_txX_eq_post, 5),
PHY_PARAM (GEN1_txX_iboost_lvl, 7),
PHY_PARAM (GEN2_rxX_eq_vga1_gain, 9),
PHY_PARAM (GEN3_rxX_eq_att_lvl, 11),
PHY_PARAM (GEN2_rxX_eq_ctle_pole, 13)
PHY_PARAMS_END
},

```

EXAMPLE: SATA Port Descriptor with PHY parameters

```

{ // P0 - SATA port
0,
DXIO_ENGINE_DATA_INITIALIZER (DxioSATAEngine, 32, 39,
DxioHotplugDisabled, 1),
DXIO_PORT_DATA_INITIALIZER_SATA (DxioPortEnabled) // Port Present

PHY_PARAMS_START
PHY_PARAM (GEN3_txX_eq_main, 0x0),
PHY_PARAM (GEN3_txX_eq_post, 0xE),
PHY_PARAM (GEN3_txX_eq_pre, 0x0),
PHY_PARAM (GEN3_txX_pre_deemphasis, 1),
PHY_PARAM (GEN3_txX_post_deemphasis, 1),
PHY_PARAM (GEN2_txX_eq_main, 0x4),
PHY_PARAM (GEN2_txX_eq_post, 0x0),
PHY_PARAM (GEN2_txX_eq_pre, 0x0),
PHY_PARAM (GEN2_txX_pre_deemphasis, 1),
PHY_PARAM (GEN2_txX_post_deemphasis, 1),
PHY_PARAM (GEN1_txX_eq_main, 0xA),
PHY_PARAM (GEN1_txX_eq_post, 0x0),
PHY_PARAM (GEN1_txX_eq_pre, 0x0),
PHY_PARAM (GEN1_txX_pre_deemphasis, 1),
PHY_PARAM (GEN1_txX_post_deemphasis, 1)
PHY_PARAMS_END
},

```

NOTE: The values used for mPhyValue in the examples above are intended to demonstrate the format of the PHY_PARAM macro and do not in any way represent recommended or realistic values.

3.6.5.6 Port Param Macros [F17M30][F17M70]**Summary**

Introduced for Family17h Model 30h, this trio of macros are used to specify extended parameters for specific Ports.

- PORT_PARAMS_START - This macro defines the beginning of a Port parameter list within a port descriptor.
- PORT_PARAM - specifies an parameter value for a specific Port . There can be more than one of these macros uses in this block, but only one per Port parameter.
- PORT_PARAMS_END - This macro defines the end of a Port parameter list within a port descriptor.

These can be used with any SATA data port to fine tune the port operation.

Prototype

```

PORT_PARAM(
    mPortParam,
    mPortValue
)

```

Parameters

mPortParam

This is an enum that identifies a specific port parameter to set. Note that some values may be specified individually for each supported speed.

PP_INVERT_POLARITY

(provided for the below example)

- 1 = Invert RX polarity.
- 2 = Invert TX polarity.
- 3 = Invert RX and TX polarity.

mPortValue

Specifies the value to be assigned to the indicated Port parameter.

PP_PORT_PRESENT	PP_LINK_SPEED_CAP	PP_LINK_ASPM
PP_CLKREQ	PP_ASPM_L1_1	PP_ASPM_L1_2
PP_COMPLIANCE	PP_SAFE_MODE	PP_CHIPSET_LINK
PP_CLOCK_PM	PP_CHANNELTYPE	PP_TURN_OFF_UNUSED_LANES
PP_APIC_GROUPMAP	PP_APIC_SWIZZLE	PP_APIC_BRIDGEINT
PP_MASTER_PLL	PP_SLOT_NUM	PP_PHY_PARAM
PP_ESM	PP_CCIX	PP_INVERT_POLARITY
PP_GEN3_DS_TX_PRESET	PP_GEN3_DS_RX_PRESET_HINT	PP_GEN3_US_TX_PRESET
PP_GEN3_US_RX_PRESET_HINT	PP_GEN4_DS_TX_PRESET	PP_GEN4_US_TX_PRESET
PP_GEN3_FIXED_PRESET	PP_GEN4_FIXED_PRESET	PP_GEN2_DEEMPHASIS
PP_HOTPLUG_TYPE		

(Reference: source file \AgesaPkg\Include\AmdPcieComplex.h : DXIO_PORT_PARAM_TYPE). This may, or may not be the full list specified by the PCIe specification, but these are the ones recognized by the AGESA™ software. The purpose of these parameters are beyond the scope of this document. The reader should refer to the PCIe® Specification.

EXAMPLE: PCIe® Port Descriptor with Port parameters

```

{ // P0 - SATA port
0,
DXIO_ENGINE_DATA_INITIALIZER (DxioSATAEngine, 32, 39, DxioHotplugDisabled, 1),
DXIO_PORT_DATA_INITIALIZER_SATA (DxioPortEnabled) // Port Present

NO_PHY_PARAMS_DATA // Needed when no PHY params are used
PORT_PARAMS_START
    PORT_PARAM (PP_INVERT_POLARITY, 1)
PORT_PARAMS_END
}

```

EXAMPLE: SATA Port Descriptor with Port parameters

```

{ // P0 - SATA port

```

```

0,
DXIO_ENGINE_DATA_INITIALIZER (DxioSATAEngine, 32, 39, DxioHotplugDisabled, 1),
DXIO_PORT_DATA_INITIALIZER_SATA (DxioPortEnabled) // Port Present

PHY_PARAMS_START
    PHY_PARAM (GEN3_txX_eq_main, 0x0),
    PHY_PARAM (GEN3_txX_eq_post, 0xE),
    PHY_PARAM (GEN3_txX_eq_pre, 0x0),
    PHY_PARAM (GEN3_txX_pre_deemphasis, 1),
    PHY_PARAM (GEN3_txX_post_deemphasis, 1),
    PHY_PARAM (GEN2_txX_eq_main, 0x4),
    PHY_PARAM (GEN2_txX_eq_post, 0x0),
    PHY_PARAM (GEN2_txX_eq_pre, 0x0),
    PHY_PARAM (GEN2_txX_pre_deemphasis, 1),
    PHY_PARAM (GEN2_txX_post_deemphasis, 1),
    PHY_PARAM (GEN1_txX_eq_main, 0xA),
    PHY_PARAM (GEN1_txX_eq_post, 0x0),
    PHY_PARAM (GEN1_txX_eq_pre, 0x0),
    PHY_PARAM (GEN1_txX_pre_deemphasis, 1),
    PHY_PARAM (GEN1_txX_post_deemphasis, 1)
PHY_PARAMS_END

PORT_PARAMS_START
    PORT_PARAM (PP_INVERT_POLARITY, 1)
PORT_PARAMS_END
},

```

NOTE: The values used in the examples above are intended to demonstrate the format of the macro and do not in any way represent recommended or realistic values.

3.6.6 NBIO HotPlug Installation Macros

Summary

The macros in this section are used to build the NBIO HotPlug data structures that describe the host platform environment.

AGESA provides a set of MACRO definitions to simplify the definition of the HOTPLUG_DESCRIPTOR entries in the NBIO complex. Each entry requires only 2 macros. The first macro defines the type of “engine” and how it is connected, while the second macro defines more information specific to the type of engine.

An example implementation of the Server Hotplug Protocol and descriptor definition can be found in AgesaPkg\Addendum\Nbio\ServerHotplugDxe. This example provides the implementation of hotplug support for the AMD customer reference platform. In the simplest form, a single descriptor could be defined as shown below:

```

HOTPLUG_DESCRIPTOR    HotplugDescriptor[] = {
{
    DESCRIPTOR_TERMINATE_LIST,           // First, and Last, entry in the array
    // Flags - end-of-list
    HOTPLUG_ENGINE_DATA_INITIALIZER (80, 95, 0, 4)
    PCIE_HOTPLUG_INITIALIZER_MAPPING (HotplugExpressModule, 0, 0, 1, 1, 2, 0, 0)
    PCIE_HOTPLUG_INITIALIZER_FUNCTION (0, 0, 3, 1, 0, 0xFF)
    PCIE_HOTPLUG_INITIALIZER_NO_RESET()
    PCIE_HOTPLUG_INITIALIZER_NO_GPIO()
}
};

```

3.6.6.1 HOTPLUG_ENGINE_DATA_INITIALIZER()

Summary

This macro associates a platform's physical PCIe® device/slot to a platform label which may appear on the motherboard silkscreen. The platform label will be used in communications with the end user for information about this device/slot.

Prototype

```

HOTPLUG_ENGINE_DATA_INITIALIZER(
    mStartLane,           // Start lane
    mEndLane,             // End lane

```

```

    mSocketNumber,      // Socket number
    mSlotNumber         // Slot number to assign
)

```

Parameters

mStartLane	This value specifies the starting lane for this device. This value should match exactly with the StartLane assigned to this slot in the PCIe® topology. Lanes are generally numbered consecutively across a multi-die implementation. For example, if each Die has 32 lanes, then Die 0 lanes are numbered 0-31, while Die 1 lanes are numbered 32-63, Die2 lanes are numbered 64-95, etc. For a multi-socket platform, the lane numbers for Socket 1 restart at 0 and are numbered consecutively as they are in Socket 0. For this reason, Socket 1 is defined in a second port list.
mEndLane	This value specifies the ending lane for this device. This value should match exactly with the EndLane assigned to this slot in the PCIe® topology.
mSocketNumber	This value specifies the socket number to which this slot is connected. Valid values are 0 and 1.
mSlotNumber	This values specifies the platform's slot number assigned to this slot. The slot number is an identifier that will help the user identify the location of the physical slot. Valid values are 1 to 63.

3.6.6.2 PCIE_HOTPLUG_INITIALIZER_MAPPING ()

Summary

Macro to map the slot to a PCIe® port.

Prototype

```

PCIE_HOTPLUG_INITIALIZER_MAPPING (
    mHotPlugFormat,
    mGpioDescriptorValid,
    mResetDescriptorValid,
    mPortActive,
    mMasterSlaveAPU,
    mDieNumber,
    mAlternateSlotNumber,
    mPrimarySecondaryLink
)

```

Parameters

mHotplugFormat	Hot Plug Format: specifies which hot-plug form-factor the descriptor pertains to. The following is a list of currently supported form-factors: • 000: HotplugPresenceDetect. • 001: HotplugExpressModule (Mode A). Mode A is the original AMD reference platform implementation. • 010: HotplugEnterpriseSsd. • 011: HotplugExpressModule (Mode B). Mode B is a more industry standard implementation. The main difference between Mode A and Mode B is the GPIO expander pin assignments. Please refer to the Hot-Plug Application Note.
-----------------------	--

- 100 - 111: reserved.

Depending on the form-factor supported, certain fields within the descriptor will or will not apply, and/or may require utilize different numbers of bits.

mGpioDescriptorValid

GPIO Descriptor Valid. Indicates whether the GPIO descriptor for the link associated with the current descriptor is populated.

- 0 - indicates that the GPIO descriptor is not populated.
- 1 - indicates that the GPIO descriptor is populated and the information within is valid

mResetDescriptorValid

Reset Descriptor Valid: indicates whether the per-link reset for the link associated with the current descriptor is sourced from the I2C subsystem.

- 0 - indicates that the reset is not supported from the I2C subsystem.
- 1 - indicates that the reset is supported from the I2C subsystem as specified by the corresponding PCIE_HOTPLUG_RESET<slotNum> register is valid.

mPortActive

Port Active: this bit is essentially a valid marker indicating whether the hot-plug slot is implemented in the system and whether the information in the descriptor register is valid.

- 0 - indicates that the GPIO descriptor is not valid.
- 1 - indicates that the GPIO descriptor is valid

mMasterSlaveAPU

Master / Slave APU: this bit indicates the APU (Master or Slave die) the PCIe® device, that is mapped onto the hot-plug slot, resides. Note, the “Master” APU is the APU that hosts the I2C subsystem. The “Slave” APU communicates with the “Master” APU through a die-to-die communication link (DDCL).

- 0 - indicates that this slot is connected to the “Master” APU.
- 1 - indicates that this slot is connected to a “Slave” APU.

mDieNumber

Indicates the Zeppelin die number in the system. Valid values are 0 through 7.

mAlternateSlotNumber

Alternate Slot Number: when the descriptor is used for an Enterprise SSD form-factor, and the slot supports two PCIe® links, this field serves as a pointer to the PCIE_HOTPLUG_MAPPING<slotNum> descriptor associated with the ‘other’ PCIe® link. The primary descriptor shall point to the secondary descriptor, and the secondary descriptor must point to the primary descriptor. For a slot that only supports a single PCIe® link, the primary descriptor must set this field to zero. Valid values are 1 through 63.

mPrimarySecondaryLink

Primary / Secondary Link: when the descriptor is used for an Enterprise SSD form-factor, this bit specifies whether

the descriptor is for the primary PCIe® device, or the secondary PCIe® device associated with the slot.

- 0 - indicates that this slot is the Primary slot.
- 1 - indicates that this slot is the Secondary slot.

3.6.6.3 PCIE_HOTPLUG_INITIALIZER_FUNCTION()

Summary

This macro assigns I2C control data to the Hot plug slot. This data is used to control and handle the 'presence detection' GPIO for the slot.

Prototype [F17M00]

```
PCIE_HOTPLUG_INITIALIZER_FUNCTION (
    mI2CGpioBitSelector,
    mI2CGpioByteMapping,
    mAddrExt,
    mI2CGpioDeviceMapping,
    mI2CDeviceType,
    mI2CBusSegment,
    mFunctionMask
)
```

Prototype [F17M30]

```
PCIE_HOTPLUG_INITIALIZER_FUNCTION (
    mI2CGpioBitSelector,
    mI2CGpioByteMapping,
    mI2CGpioDeviceMapping,
    mI2CDeviceType,
    mI2CBusSegment,
    mFunctionMask
)
```

Parameters

mI2CGpioBitSelector

I2C GPIO Bit Select: for a simple presence detect usage model, this field indicates which bit of which IO byte is used for the presence detect signal associated with this descriptor. For an Enterprise SSD form-factor, only bit 2 is used and indicates which nibble of the IO byte is used for the hot-plug signals associated with this descriptor.

mI2CGpioByteMapping

I2C GPIO IO Byte Mapping: indicates which IO byte of the I2C GPIO device is used for the hot-plug signals associated with this descriptor. Valid values are 0 through 7, but only valid offsets within the I2C GPIO extender may be used.

mAddrExt

I2C GPIO Device Mapping Extended- indicates the optional upper two bits of the I2C address for the I2C GPIO Expander device that is used to host the hot-plug signals associated with this descriptor. These bits will be prepended to the DeviceMapping bits to form the 7-bit I2C address.

mI2CGpioDeviceMapping

I2C GPIO Device Mapping: indicates the lower {3 bits [F17M00], 5 bits [F17M30]} of the I2C 7-bit address for the I2C GPIO Expander device that is used to host the hot-plug signals associated with this descriptor.

mI2CDeviceType

I2C Device Type: indicates which of three I2C GPIO devices is used to host the hot-plug signals associated with this descriptor.

- 00: PCA9539 address and register map; in support of legacy hot-plug implementations.
- 01: PCA9535 address and register map; in support of newer hot-plug implementations.
- 10: PCA9506 address and register map; in support of high slot-capacity implementations.
- 11: reserved.

Note, PCA9535 and PCA9506 devices may be intermixed on the I2C interface. However, due to common addressing, a combined total of eight devices may be populated.

mI2CBusSegment

I2C Bus Segment: in the context of an I2C subsystem constructed with PCA9545 I2C switches, this field specifies the downstream I2C bus segment, on one of two PCA9545 switches, on which the target I2C GPIO device is located. However, for I2C topologies that are not constructed with any PCA9545 devices, one value is reserved to identify that the I2C GPIO device does not reside behind any PCA9545 I2C switch. Note: I2C addresses below are the 7-bit address of the switch device.

For 3-bit Device Mappings: [F17M00]

- 000 - PCA9545, address=00, downstream I2C bus=00. (I2C address = 0x70)
- 001 - PCA9545, address=00, downstream I2C bus=01. (I2C address = 0x70)
- 010 - PCA9545, address=00, downstream I2C bus=10. (I2C address = 0x70)
- 011 - PCA9545, address=00, downstream I2C bus=11. (I2C address = 0x70)
- 100 - PCA9545, address=01, downstream I2C bus=00. (I2C address = 0x71)
- 101 - PCA9545, address=01, downstream I2C bus=01. (I2C address = 0x71)
- 110 - PCA9545, address=01, downstream I2C bus=10. (I2C address = 0x71)
- 111 - no PCA9545; I2C bus directly connected to APU socket.

For 5-bit Device Mappings: [F17M30]

- 01000 - PCA9545, address=01, downstream I2C bus=11. (I2C address = 0x71)
- 01001 - PCA9545, address=02, downstream I2C bus=00. (I2C address = 0x72)

- 01010 - PCA9545, address=02, downstream I2C bus=01.(I2C address = 0x72)
- 01011 - PCA9545, address=02, downstream I2C bus=10.(I2C address = 0x72)
- 01100 - PCA9545, address=02, downstream I2C bus=11.(I2C address = 0x72)
- 01101 - PCA9545, address=03, downstream I2C bus=00.(I2C address = 0x73)
- 01110 - PCA9545, address=03, downstream I2C bus=01.(I2C address = 0x73)
- 01111 - PCA9545, address=03, downstream I2C bus=10.(I2C address = 0x73)
- 10000 - PCA9545, address=03, downstream I2C bus=11.(I2C address = 0x73)

mFunctionMask

When a hot plug port is configured as one of the multi-pin modes, i.e. HotplugExpressModule or HotplugEnterpriseSsd, this field provides the capability via a bitmask to disable a predefined pin function. For example, in HotplugExpressModule Mode A, pin 1 is the PWRFLT# input. By setting a function mask value of 0x02, the PWRFLT# function is disabled in the hot plug controller for that port.

3.6.6.4 PCIE_HOTPLUG_RESET()**Summary**

Used to connect a pin used for reset to the HotPlug slot.

Prototype

```
PCIE_HOTPLUG_INITIALIZER_RESET (
    mI2CGpioByteMapping,
    mI2CGpioDeviceMapping,
    mI2CDeviceType,
    mI2CBusSegment,
    mResetSelect
)
```

Alternate**PCIE_HOTPLUG_INITIALIZER_NO_RESET()**

This special case helper macro defines a NULL descriptor in cases where the reset descriptor is not valid, or not needed, for a given slot descriptor.

Prototype

```
PCIE_HOTPLUG_INITIALIZER_NO_RESET()
```

Parameters**mI2CGpioByteMapping**

I2C GPIO IO Byte Mapping: indicates which IO byte of the I2C GPIO device is used for the reset signal associated with this descriptor.

mI2CGpioDeviceMapping

I2C GPIO Device Mapping: indicates the lower {3 bits [F17M00], 5 bits [F17M30]} of the I2C 7-bit address for the I2C GPIO Expander device that is used to host the reset signal associated with this descriptor.

mI2CDeviceType

I2C Device Type: indicates which of three I2C GPIO devices is used to host the hot-plug signals associated with this descriptor.

- 00: PCA9539 address and register map; in support of legacy hot-plug implementations
- 01: PCA9535 address and register map; in support of newer hot-plug implementations
- 10: PCA9506 address and register map; in support of high slot-capacity implementations
- 11: reserved

mI2CBusSegment

I2C Bus Segment: in the context of an I2C subsystem constructed with PCA9545 I2C switches, this field specifies the downstream I2C bus segment, on one of two PCA9545 switches, on which the target I2C GPIO device is located. However, for I2C topologies that are not constructed with any PCA9545 devices, one value is reserved to identify that the I2C GPIO device does not reside behind any PCA9454 I2C switch. Note: I2C addresses below are the 7-bit address of the switch device.

For 3-bit Device Mappings: [F17M00]

- 000 - PCA9545, address=00, downstream I2C bus=00. (I2C address = 0x70)
- 001 - PCA9545, address=00, downstream I2C bus=01. (I2C address = 0x70)
- 010 - PCA9545, address=00, downstream I2C bus=10. (I2C address = 0x70)
- 011 - PCA9545, address=00, downstream I2C bus=11. (I2C address = 0x70)
- 100 - PCA9545, address=01, downstream I2C bus=00. (I2C address = 0x71)
- 101 - PCA9545, address=01, downstream I2C bus=01. (I2C address = 0x71)
- 110 - PCA9545, address=01, downstream I2C bus=10. (I2C address = 0x71)
- 111 - no PCA9545; I2C bus directly connected to APU socket.

For 5-bit Device Mappings: [F17M30]

- 01000 - PCA9545, address=01, downstream I2C bus=11. (I2C address = 0x71)
- 01001 - PCA9545, address=02, downstream I2C bus=00. (I2C address = 0x72)
- 01010 - PCA9545, address=02, downstream I2C bus=01. (I2C address = 0x72)

- 01011 - PCA9545, address=02, downstream I2C bus=10.(I2C address = 0x72)
- 01100 - PCA9545, address=02, downstream I2C bus=11.(I2C address = 0x72)
- 01101 - PCA9545, address=03, downstream I2C bus=00.(I2C address = 0x73)
- 01110 - PCA9545, address=03, downstream I2C bus=01.(I2C address = 0x73)
- 01111 - PCA9545, address=03, downstream I2C bus=10.(I2C address = 0x73)
- 10000 - PCA9545, address=03, downstream I2C bus=11.(I2C address = 0x73)

Note, PCA9535 and PCA9506 devices may be intermixed on the I2C interface. However, due to common addressing, a combined total of eight devices may be populated.

mResetSelect

Reset Select: this bit-field is used to specify which pin within the selected I2C IO byte (identified by the other fields in this descriptor) is used for the reset function associated with the <slotNum> PCIe® link. Since reset is a singular function, only one bit in this field may be set to one. When the Nth bit of this field is set to 1, the Nth bit of the selected I2C IO byte serves as the platform reset signal for the associated PCIe® link. Note, although this descriptor identifies a single bit, this field has been defined in this manner so as to be consistent with both the “FunctionMask” field in the FUNCTION descriptor and the “GPIOSelect” field in the GPIO descriptor.

3.7 Data Fabric Driver

Summary

The data fabric driver is responsible for initializing the data fabric hardware (formerly known as the processor northbridge or UNB).

Description

The data fabric driver has both a PEIM and DXE drivers. The initialization work is done in the module entry points. The PEI component collects topology information for internal use and the DXE component establishes power management and RAS functionality.

Produced Protocols

None.

3.7.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

3.7.2 DF Configuration Items

3.7.2.1 Enablement Category

- **PcdAmdSmee [F19M00]**

Secure Memory Encryption Extension (SMEE). TSME basically controls DataScrambleEn in the UMC. This is a hardware feature. SMEE which is also known as 'Secure Memory Encryption Extension' enables the privileged software to select individual memory pages for runtime encryption. Pages marked encrypted are automatically decrypted by hardware when read and encrypted when written to DRAM. For more information, see the Memory Encryption white papers and PPR description for MSR:SYS_CFG[23].

Permitted Choices: (Type: Boolean) (Default: FALSE)

- FALSE - the feature is disabled.
- TRUE - the SMEE feature is enabled.

3.7.2.2 Platform Category

- **PcdxGMIForceLinkWidth (F17M30)**

The value force xGMI link width.

Permitted Choices: (Type: Value) (Default: 0x00)

- 0 - x2 link.
- 1 - x8 link.
- 2 - x16 link.

- **PcdxGMIForceLinkWidthEn (F17M30)**

xGMI force link width control.

Permitted Choices: (Type: Value) (Default: 0x00)

- 0 - do not force the value in xGMIForceLinkWidth.
- 1 - force to the value in xGMIForceLinkWidth.

- **PcdxGMIMaxLinkWidth (F17M30)**

The value set the max xGMI link width.

Permitted Choices: (Type: Value) (Default: 0x00)

- 0 - x8.
- 1 - x16.

- **PcdxGMIMaxLinkWidthEn (F17M30)**

The max xGMI link width control enable.

Permitted Choices: (Type: Value) (Default: 0x00)

- 0 - Use default xGMI max supported link width.
- 1 - User can set custom xGMI max link width.

- **PcdCfgDLWMSupport [F17M30][F19M00]**

This value controls Dynamic Link Width Management (DLWM) feature. When the platform can support either an 8-lane or 16-lane xGMI operation, the dynamic adjustment feature can provide some power savings. This feature is described in the PPOG, section "xGMI - Connection Between Sockets".

Permitted Choices: (type: value)(Default: 0xFF)

- 0x0 - Force at fixed width as set by PcdxGMIForceLinkWidth.
- 0x1 - Enable dynamic link width adjustments.
- 0xFF - Used hardware default value - which is 'enabled'.

3.7.2.3 Power Management Category

-

3.7.2.4 Performance Category

- **PcdAmdFabricEccScrubRedirection**

DRAM ECC redirection is a data-protection feature. Redirection is a special ECC feature that enables the scrubber to immediately scrub any address in which a correctable error is discovered. The AMD recommended setting is to enable this feature. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

- **PcdAmdFabricDramScrubTime**

Number of hours to scrub the entire memory. AMD suggests ~24 hours. Please refer to the PPR/BKDG for a description of this feature.

Permitted Choices: (Type: Value)(Default: 24)

- **PcdAmdMmioAbove4GLimit**

Specifies the top address to be used for MMIO space allocations. No MMIO space will be used above this limit. Some devices have limits on the address space they are able to access. For example a device with a 40-bit address limitation can't use MMIO space above 1TeraByte (1T). By setting this PCD to 0x1000000000 (1T), MMIO space would not be allocated above 1T. The default is to use all of the space above the end of physical memory.

Permitted Choices: (Type: Value)(Default: 0xFFFF_FFFF_FFFF_FFFF)

- **PcdAmdBottomMmioReservedForDie0**

This entry specifies an address in the below 4GB space that starts a reserved MMIO region. The space from this address to the 0xFEC0_0000 address ('COMPAT' region) will be reserved for devices connected to Die0 to use as MMIO space. This address should be at the end of, or above the PCIe® general allocation zone (Base + size).

Permitted Choices: (Type: Value)(Default: 0xFE00_0000)

- **PcdAmdMmioSizePerDieForNonPciDevice**

Every Die has a non-PCI MMIO pool for items such as the PSP and IOMMU. This PCD defines the size of that pool. The default is 8MB per Die.

Permitted Choices: (Type: Value(32b))(Default: 0x800000M)

• **PcdAmdFabricResourceDefaultMap**

This control is provided to force the MMIO allocation to use the evenly allocated method regardless of any calls made to the MMIO reallocation functions.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - MMIO space will be allocated evenly over the present die.
- FALSE - User based re-distribution will be allowed.

• **PcdAmdFabricSlitDegree**

For CPUs that use multiple die or use multiple sockets. The NUMA distribution of memory devices affects the performance. The SLIT table tells the OS the relative distance of memory from the processor core, to allow it to select cores for tasks that are closer to the available memory so as to maximize performance. The number of unique distance values declared within the ACPI SLIT table has been found to affect performance within certain operating system environments. This item specifies the platform preference for how the SLIT distance values are to be created.

Permitted Choices: (Type: List)(Default: 0xFF)

- 0 - Honor the PPR definition and declare as many unique distance values as the hardware configuration warrants.
- 1 - Declare all domains as locally accessible (a distance value of 0xA for all system dies).
- 2 - Declare a maximum of 2 unique distance values.
 - Die local accesses will have a distance of 0xA.
 - All domains other than self are described by PcdAmdFabric2ndDegreeSlitDistance.
- 3 - Declare a maximum of 3 unique distance values.
 - Die local accesses will have a distance of 0xA.
 - Socket local domains other than self are described by PcdAmdFabric3rdDegreeSlitLocalDistance.
 - All accesses to the other sockets are described by PcdAmdFabric3rdDegreeSlitRemoteDistance.
- 0xFF - Declare a maximum of 3 unique distance values. Die local accesses will have a distance of 0xA. Socket local domains other than self are set to 0x10. All accesses to the other sockets are set to 0x20.

• **PcdAmdFabric2ndDegreeSlitDistance**

Specifies the SLIT distance to domains other than self if PcdAmdFabricSlitDegree is set to 2.

Permitted Choices: (Type: Value)(Default: 0x1C)

- 0x0A..0xFF - The SMP relative distance to use for all cores not on the same die.
- all other values - Reserved.

• **PcdAmdFabric3rdDegreeSlitLocalDistance**

Specifies the SLIT distance to 'socket local' domains other than self that are within the same socket, but not on the same die. This is used only when PcdAmdFabricSlitDegree is set to 3.

Permitted Choices: (Type: Value)(Default: 0x10)

- 0x0A..0xFF - The SMP relative distance to use for all cores not on the same die, but within the same socket.
- all other values - Reserved.

- **PcdAmdFabric3rdDegreeSlitRemoteDistance**

Specifies the SLIT distance to 'socket remote' domains that are in other sockets. This is used only when PcdAmdFabricSlitDegree is set to 3.

Permitted Choices: (Type: Value)(Default: 0x20)

- 0x0A..0xFF - The SMP relative distance to use for all cores not on the same socket.
- all other values - Reserved.

- **PcdEfficiencyOptimizedMode**

This control establishes the operational mode for the SMU QOS. The purpose is to improve overall performance per watt. However, some benchmarks may score better when this feature is disabled.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

- **PcdDfCstateEnable (F17M30)**

The value controls the DF C-states.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

- **PcdAmdFabricPeriodicDirRinse [F19M00]**

This control is part of the periodic scrub controls that are described in the PPR section "DRAM Scrub Rate Algorithm". The Periodic Directory Rinse (PDR) Algorithm uses this input to set the DramScrubLimitAddr.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Enable Periodic Directory Rinse using a DramScrubRate of 3.
- FALSE - DramScrubRate is a calculated number and will be less than 3.

- **PcdAmdCcxCfgPFEHEnable [F19M00]**

Permitted Choices: (Type: Boolean)(Default: FALSE)

- **PcdCfgDffoDis [F19M00]**

This value determines if AGESA will disable the DF PState Frequency Optimizer. The DFFO algorithm adjusts the DF P-state power setting based on frequency in certain cases. This field should only be set to true in special circumstances if advised by AMD.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0: DF PState CCLK effective Frequency Optimizer is enabled.
- 1: DF PState CCLK effective Frequency Optimizer is disabled. Careful of the double-negative here - 1, or True, means the optimizer is DISABLED.

- **PcdCfgDfloDis [F19M00]**

This value determines if AGESA will disable the DF PState Latency Optimizer. The DFLO algorithm ensures that the DF P-state will not drop when latency sensitive code is running. This field should only be set to true in special circumstances if advised by AMD.

Permitted Choices: (Type: Value)(Default: 0x00)

- 0: DF PState Latency Optimizer is enabled.
- 1: DF PState Latency Optimizer is disabled. Careful of the double-negative here - 1, or True, means the optimizer is DISABLED.

Secure Encrypted Virtualization and Secure Nested Paging

The next generation of SEV (Secure Encrypted Virtualization) is called SEV-SNP (Secure Nested Paging). AMD provides an topical information web site for customers to better understand this concept at this public site: <https://developer.amd.com/sev/> . Available at this site or by web search, is this intro to SEV-SNP white paper: <https://www.amd.com/system/files/TechDocs/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf>. Customers can also reference the DevHub site to read the Architecture Programmer's Manual (APM) Vol 2, section 15 'Secure Virtual Machine' and specifically section 15.36 'Secure Nested Paging (SEV-SNP)'.

The following controls enable and configure operation of the SEV-SNP feature.

- **PcdCfgSevSnpSupport [F19M00]**

This control selects the operation of the SEV-SNP feature.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- FALSE - Leave the SEV-SNP disabled.
- TRUE - Program SEV-SNP configuration setting and enable the feature.

- **PcdAmdSnpMemCover [F19M00]**

This control selects the operating mode of the Nested Paging (SNP) Memory and the Reverse MaP table (RMP).

The reason to set PcdAmdSnpMemCover manually is for performance. If pages are not covered by the RMP, then those additional hardware checks on each page are not performed. So a user could just cover the amount of memory needed for a VM by the RMP and make the rest unprotected/not covered pages. The RMP is used to ensure a one-to-one mapping between system physical addresses and guest physical addresses.

Permitted Choices: (Type: Value)(Default: 0)

- 0 - Disabled.
- 1 - Enabled (ENTIRE system memory).
- 2 - Custom.

- **PcdAmdSnpMemSize [F19M00]**

Specifies the SNP size of system memory (number of memory pages) to be covered by the RMP. This control applies only if PcdAmdSnpMemCover is set for 'Custom'.

Permitted Choices: (Type: Value)(Default: 0x0000_0000)

- 0x0000 .. 0x100000 - the number of pages to be covered by the RMP. This number is described in hex.

3.7.3 FABRIC_RESOURCE_MANAGER_PROTOCOL

Summary

This protocol provides services for managing the MMIO and IO space allocations.

GUID

```
gAmdFabricResourceManagerServicesProtocolGuid = {0xaf96f126, 0x64b6, 0x43dc,  
{0x9e, 0x6d, 0x76, 0x3f, 0xf1, 0x9b, 0xf3, 0x96}}
```

Protocol Interface Structure

```
struct _FABRIC_RESOURCE_MANAGER_PROTOCOL {  
    UINTN  
    FABRIC_ALLOCATE_MMIO  
    FABRIC_ALLOCATE_IO  
    FABRIC_GET_AVAILABLE_RESOURCE  
    FABRIC_REALLOCATE_RESOURCE_FOR_EACH_DIE  
    FABRIC_RESOURCE_RESTORE_DEFAULT  
    FABRIC_ENABLE_VGA_MMIO  
};  
  
Revision;  
FabricAllocateMmio;  
FabricAllocateIo;  
FabricGetAvailableResource;  
FabricReallocateResourceForEachDie;  
FabricResourceRestoreDefault;  
FabricEnableVgaMmio;
```

Members

Revision	Revision of the protocol.
FabricAllocateMmio	Pointer to a function for requesting a new MMIO allocation.
FabricAllocateIo	Pointer to a function for requesting a new IO allocation.
FabricGetAvailableResource	Pointer to function that returns the available MMIO/IO resources for each DIE.
FabricReallocateResourceForEachDie	Pointer to function to redistribute MMIO/IO resource based on user's request.
FabricResourceRestoreDefault	Pointer to function used to restore the resource allocation to the original settings.
FabricEnableVgaMmio	Pointer to function for the platform to specify where to locate the VGA MMIO block.

Description The processor may support up to 8 dies in a system, but there are only 16 MMIO register pairs. The system could run out of registers if every module is allowed to program MMIO allocations by themselves. To avoid this situation, AGESA provides this protocol/library. All DF MMIO/IO register pairs will be pre-initialized by AGESA in the SoC PEIM. All available MMIO space would be distributed evenly to all Dies. When PCI enumeration is finished, the PciHostBridge driver then knows how much PCI MMIO/IO space is needed for each die. If the equal distribution doesn't satisfy current hardware requirements, the PciHostBridge can call the FabricReallocateResourceForEachDie function to adjust the MMIO/IO size for each Die. Every Die has five MMIO pools and one IO pool. The MMIO pools are: non-prefetchable below 4G, non-prefetchable above 4G, prefetchable below 4G, prefetchable above 4G and non-PCI. The first 4 pools are for PciHostBridge driver, and the last pool is for the devices which are not standard PCI devices, such as PSP, IOMMU, IOAPIC, GPIO.

Note: 'Prefetchable' space is preferred since it allows for cacheing pre-fetches which improves speeds. A 'non-prefetchable' space is one that has side effects when read (causes some action) or the space does not support write merging.

Related Definitions

FABRIC_TARGET

```
typedef struct _FABRIC_TARGET {
    UINT16  PciBusNum:8;
    UINT16  SocketNum:2;
    UINT16  DieNum:5;
    UINT16  TgtType:1;
} FABRIC_TARGET;
```

This structure is used to identify the Root Complex that owns, or controls, this region.

PciBusNum PCI bus number to which this region applies.
SocketNum Socket number of the owner of this region.
DieNum Die number of the owner of the region.
TgtType Indicator of the target type. The caller can choose whether to use the PCI bus number to identify the owner of the MMIO region, or can specify the Socket/die number directly.

- 0 - TARGET_PCI_BUS - Use the PCI bus number to specify the owner of this region. The function will use the PciBusNum field to identify the root bridge in control of this region. The SocketNum and DieNum fields will be ignored.
- 1 - TARGET_DIE - Use the Socket, DIE numbers to specify the owner of this region. The function will use the SocketNum and DieNum fields to identify the root bridge in control of this region. The PciBusNum field will be ignored.

FABRIC_MMIO_ATTRIBUTE

```
typedef struct _FABRIC_MMIO_ATTRIBUTE {
    UINT8  ReadEnable:1;
    UINT8  WriteEnable:1;
    UINT8  NonPosted:1;
    UINT8  CpuDis:1;
    UINT8  :1; // Reserved
    UINT8  MmioType:3;
} FABRIC_MMIO_ATTRIBUTE;
```

This is a flags structure used to indicate the characteristics of the MMIO region. Most of the flags are outputs set by the function. The MmioType flag is an input that the caller must specify.

ReadEnable Indicates if the range is readable.

- TRUE - (1b) - The region will allow reads.
- FALSE - (0b) - The region will be hidden to reads.

WriteEnable Indicates if the range is writable.

- TRUE - (1b) - The region will allow writes.
- FALSE - (0b) - The region will be read-only.

NonPosted Indicates if transactions sent to this range are to be labeled as 'posted'. The term 'posted' is defined in the PCI specification and specifies the ordering of transactions on the PCIe® link. Most transactions should be 'posted' to maintain the strict Producer-Consumer ordering model. Please refer to the PCIe® specification for further details.

- TRUE - (1b) - The MMIO region transactions will be marked as non-posted.
- FALSE - (0b) - The transactions sent to the MMIO region will be marked as posted.

CpuDis

This allows core requests to be forced onto the bus as an IO-cycle. When CpuDis = 1 and the CPU core memory access uses the ReqIo=1, target addresses within this MMIO range are directed to the compatibility address space. Having accesses redirected to the IO compatibility space means that they will be forwarded to the system southbridge, which hosts the subtractive decode bus (normally LPC).

- TRUE - (1b) - The MMIO region allow IO-cycle conversion.
- FALSE - (0b) - The MMIO region will be strictly memory accesses regardless of the core request (ReqIo).

MmioType

Indicator whether the MMIO space should be placed above the 4GB address mark.

- MMIO_BELOW_4G - Non-Prefetchable MMIO
- MMIO_ABOVE_4G - Non-Prefetchable MMIO
- P_MMIO_BELOW_4G - Prefetchable MMIO
- P_MMIO_ABOVE_4G - Prefetchable MMIO
- NON_PCI_DEVICE_BELOW_4G - For non-discoverable devices, such as IOAPIC, GPIO, MP0/1 mailbox, IOMMU.

For Family 15h APUs, the above flags are also inputs stating the desired characteristics of the region.

FABRIC_ADDR_APERTURE

```
typedef struct _FABRIC_ADDR_APERTURE {
    UINT64  Base;
    UINT64  Size;
    UINT64  Alignment;
} FABRIC_ADDR_APERTURE;
```

Describes an allocated MMIO/IO resource.

Base	Definition
Size	Definition
Alignment	Alignment bit map. For example, 0x000F_FFFF means 1MB alignment.

FABRIC_RESOURCE_SIZE_FOR_EACH_DIE

```
typedef struct _FABRIC_RESOURCE_SIZE_FOR_EACH_DIE {
    FABRIC_ADDR_APERTURE NonPrefetchableMmioSizeAbove4G[];
    FABRIC_ADDR_APERTURE PrefetchableMmioSizeAbove4G[];
    FABRIC_ADDR_APERTURE NonPrefetchableMmioSizeBelow4G[];
    FABRIC_ADDR_APERTURE PrefetchableMmioSizeBelow4G[];
    FABRIC_ADDR_APERTURE Die0SecondNonPrefetchableMmioSizeBelow4G;
    FABRIC_ADDR_APERTURE Die0SecondPrefetchableMmioSizeBelow4G;
    FABRIC_ADDR_APERTURE IO[];
```

```
} FABRIC_RESOURCE_SIZE_FOR_EACH_DIE;
```

List of allocated resources for each die in the system. Each matrix is indexed by socket# first then Die#. Please refer to the AGESA.H file in the release package for definitions for the MAX_SOCKETS_SUPPORTED and MAX_DIES_PER_SOCKET values.

NonPrefetchable**MmioSizeAbove4G****Prefetchable MmioSizeAbove4G****NonPrefetchable****MmioSizeBelow4G****PrefetchableMmioSizeBelow4G****Die0Second NonPrefetchable****MmioSizeBelow4G****Die0Second Prefetchable****MmioSizeBelow4G****IO**

List of allocations presently established for this region type.

List of allocations presently established for this region type.

List of allocations presently established for this region type.

List of allocations presently established for this region type.

The Below 4G area has special considerations since the legacy PCIe® Config space may be located in the middle of the free space region creating two MMIO regions. The MMIO region below 4G is set by the TOM (Top of memory) register and the 4G address mark. PCIe® Config space is defined by the value in the EDK-II parameter PcdPciExpressBaseAddress and typically does not fill the entire region up to the 4G mark. If the PcdPciExpressBaseAddress value does not match with the value in TOM, a 'second' MMIO space is created in the region, below the PCIe® Config space.

See note above about 'second' MMIO region.

List of allocations presently established for this region type.

3.7.3.1 FabricAllocateMmio**Summary**

This function allocates MMIO regions for the processor cores.

Prototype

```
typedef EFI_STATUS (EFI_API *FABRIC_ALLOCATE_MMIO) (
    IN      FABRIC_RESOURCE_MANAGER_PROTOCOL *This,
    IN OUT  UINT64 *BaseAddress,
    IN OUT  UINT64 *Length,
    IN      UINT64 Alignment,
    IN      FABRIC_TARGET Target,
    IN OUT  FABRIC_MMIO_ATTRIBUTE *Attributes
);
```

Parameters**This**

A Pointer to this instance of the protocol.

BaseAddress

Starting address of the requested MMIO/IO range. Upon a successful return, this is the address of the allocated space.

Length

Length of the requested range. On an error return, this parameter will be set to the size of the largest available block.

Alignment	Alignment bit map. For example, 0x000F_FFFF means 1MB alignment.
Target	Identifies the owner of the requested space. This may be specified as a specific PCI bus or by socket/die numbers; see FABRIC_TARGET definition.
Attributes	Attributes of the requested MMIO range indicating whether it is readable/writable/non-posted/etc; see the definition of FABRIC_MMIO_ATTRIBUTE.

Description

At the first call to this routine, it will allocate MMIO regions from non-PCI space in the region below the 4GB address mark (32-bit addressing) and above the 4GB address mark (for 64-bit addressing). These regions will be marked with the attributes Read-writable, 'posted' and CpuDis=0. In the region above the 4GB address mark, all available space will be allocated to MMIO and the caller only needs to supply the Target and Length parameters. The routine will determine a location in the region for the requested size and return the BaseAddress for that space.

For Family 15h APUs, the caller must supply the desired BaseAddress and Length for the allocation and can also specify the desired attributes of the region.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ABORTED	Fatal error. The function could not allocate the MMIO space requested.
EFI_OUT_OF_RESOURCES	Not enough space remained, the largest size now available is returned in Length. This allows the user to discover the amount of space left and decide if they could accept a smaller MMIO space.

3.7.3.2 FabricAllocateIo

Summary

This function allocates IO regions for the processor cores.

Prototype

```
typedef EFI_STATUS (EFIAPI *FABRIC_ALLOCATE_IO) (
    IN      FABRIC_RESOURCE_MANAGER_PROTOCOL *This,
    OUT     UINT32                            *BaseAddress,
    IN OUT  UINT32                            *Length,
    IN      FABRIC_TARGET                     Target
);
```

Parameters

This	A Pointer to this instance of the protocol.
BaseAddress	Starting address of the requested IO range. Upon a successful return, this is the address of the allocated space.
Length	Length of the requested range. On an error return, this parameter will be set to the size of the largest available block.
Target	Identifies the owner of the requested space. This may be specified as a specific PCI bus or by socket/die numbers; see FABRIC_TARGET definition.

Description

The caller only needs to supply the Target and Length parameters. The routine will determine a location in the region for the requested size and return the BaseAddress for that space.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ABORTED	Fatal error. The function could not allocate the IO space requested.
EFI_OUT_OF_RESOURCES	Not enough space remained, the largest size now available is returned in Length. This allows the user to discover the amount of space left and decide if they could accept a smaller IO space.

3.7.3.3 FabricGetAvailableResource()

Summary

Returns a list of currently allocated resources.

Prototype

```
typedef EFI_STATUS (EFIAPI *FABRIC_GET_AVAILABLE_RESOURCE) (
    IN      FABRIC_RESOURCE_MANAGER_PROTOCOL *This,
    IN      FABRIC_RESOURCE_FOR_EACH_DIE    *ResourceForEachDie
);
```

Parameters

This A Pointer to this instance of the protocol.

ResourceForEachDie Pointer to a buffer where the function will place a matrix of information about the allocated resources. The buffer must be large enough to hold the matrix for the system.

Description

This function is used to read the current list of allocated resources; either for information or for preparation to make changes using the re-allocate function.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.7.3.4 FabricReallocateResourceForEachDie()

Summary

Re-distribute MMIO resource space.

Prototype

```
typedef EFI_STATUS (EFIAPI *FABRIC_REALLOCATE_RESOURCE_FOR_EACH_DIE) (
    IN      FABRIC_RESOURCE_MANAGER_PROTOCOL *This,
    IN      FABRIC_RESOURCE_FOR_EACH_DIE    *ResourceSizeForEachDie,
    IN      FABRIC_ADDR_SPACE_SIZE          *SpaceStatus
);
```

Parameters

This A Pointer to this instance of the protocol.

ResourceSizeForEachDie Pointer to a buffer containing the new desired allocation records.

SpaceStatus Structure indicating the desired allocations for the platform.

Description

If the current MMIO/IO distribution doesn't fit for the platform's desired configuration, the platform BIOS can call this function to adjust MMIO/IO size for each Die. The buffer can be initially populated with information obtained by a call to the Get Resources function. This function is ignored when the configuration value PcdAmdFabricResourceDefaultMap is set to TRUE.

Related Definitions

FABRIC_ADDR_SPACE_SIZE

```
typedef struct _FABRIC_ADDR_SPACE_SIZE {
    UINT32      IoSize;
    UINT32      IoSizeReqInc;
    UINT32      MmioSizeBelow4G;
    UINT32      MmioSizeBelow4GReqInc;
    UINT64      MmioSizeAbove4G;
    UINT64      MmioSizeAbove4GReqInc;
} FABRIC_ADDR_SPACE_SIZE;
```

IoSize IO size required by system resources.

IoSizeReqInc The amount needed over the current size.

MmioSizeBelow4G Below 4G MMIO size required by system resources.

MmioSizeBelow4GReqInc The amount needed over the current size.

MmioSizeAbove4G Above 4G MMIO size required by system resources.

MmioSizeAbove4GReqInc The amount needed over the current size.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.

3.7.3.5 FabricResourceRestoreDefault()

Summary

Return the allocations to the default system state - regress any user changes.

Prototype

```
typedef EFI_STATUS (EFI_API *FABRIC_RESOURCE_RESTORE_DEFAULT) (
    IN      FABRIC_RESOURCE_MANAGER_PROTOCOL *This
);
```

Parameters

This A pointer to this instance of the protocol.

Description

This function deletes any user requests, made through a call to the Redistribute Resources function and upon the next boot; the resources would be distributed base on the default strategy.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.7.3.6 FabricEnableVgaMmio()

Summary

Broadcast the home die location of the GFX to all die.

Prototype

```
typedef EFI_STATUS (EFI_API *FABRIC_ENABLE_VGA_MMIO) (
    IN      FABRIC_RESOURCE_MANAGER_PROTOCOL *This,
    IN      FABRIC_TARGET                    Target
);
```

Parameters

This A pointer to this instance of the protocol.
Target Indicates the location where the GFX MMIO space is allocated.

Description

This function helps the user to inform every Die where the GPU MMIO space is located. Caller provides the GPU location (target) and this function will program the VGA_EN controls on each die.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.7.4 FABRIC_SOC_SPECIFIC_SERVICES_PROTOCOL [F17M30]

Summary

This protocol provides services for fabric SOC specific services.

GUID

```
gAmdFabricSocSpecificServicesProtocolGuid = {0xc99f5067, 0xf8a1, 0x45c4,
    {0x9f, 0x95, 0x43, 0x35, 0x67, 0xe3, 0xef, 0xa7}}
```

Protocol Interface Structure

```
struct _FABRIC_SOC_SPECIFIC_SERVICES_PROTOCOL {
    UINTN      Revision;
    FABRIC_DXE_GET_XGMI_FREQ    FabricGetXgmiFreq;
};
```

Members

Revision Revision of the protocol.
FabricGetXgmiFreq Pointer to function that returns the xGMI link frequency for each xGMI link.

Description

This interface protocol provides various SoC specific details, such as a service function to get xGMI link information.

3.7.4.1 FabricGetXgmiFreq()

Summary

Get xGMI frequency for each link.

Prototype

```
typedef EFI_STATUS (EFI_API *FABRIC_DXE_GET_XGMI_FREQ) (
    IN      FABRIC_SOC_SPECIFIC_SERVICES_PROTOCOL *This,
    IN OUT  FABRIC_XGMI_FREQ_FOR_EACH_LINK        *XgmiFreqForEachLink
);
```

Parameters

This

UEFI standard pointer to this protocol.

XgmiFreqForEachLink

A pointer to a caller allocated array where the current xGMI link frequencies will be placed. This array must be large enough to contain the maximum number of entries possible for the Processor installed. See the Related Definitions section.

Description

This function is used to retrieve the current xGMI link frequencies.

Related Definitions

FABRIC_XGMI_FREQ_FOR_EACH_LINK

```
typedef struct _FABRIC_XGMI_FREQ_FOR_EACH_LINK {
    UINT32      XgmiFreq[MAX_XGMI_LINKS_SUPPORTED];
} FABRIC_XGMI_FREQ_FOR_EACH_LINK;
```

XgmiFreq

This array structure is used to hold the list of the current xGMI frequency for each link. Connected links will have a non-zero value. The platform may choose to not connect the maximum available links in favor of using some as PCIe® links.

Programming notes::

- The constant for MAX_XGMI_LINKS_SUPPORTED is defined in the package file: AgesaPkg/Include/Protocol/FabricXgmiInfoServicesProtocol.h
- the caller should ensure the array is zero filled prior to the call.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_ABORTED	The platform is a single socket system, therefore there are no xGMI links.

3.7.5 FABRIC_NUMA_SERVICES_PROTOCOL FABRIC_NUMA_SERVICES2_PROTOCOL

Summary

This protocol provides a collection of services that return information associated with non-uniform memory architecture (NUMA).

GUID

```
gAmdFabricNumaServicesProtocolGuid = {0xb750d611, 0x38ca, 0x4d38,
                                         {0x95, 0xf7, 0x9c, 0xe, 0x28, 0x1d, 0xf1, 0x9a}}

gAmdFabricNumaServices2ProtocolGuid = {0xa8ff2e64, 0xf319, 0x4af1,
                                         {0x9e, 0xc8, 0x18, 0x89, 0x89, 0xc3, 0x1e, 0x4a}};
```

Protocol Interface Structure

```
struct _FABRIC_NUMA_SERVICES_PROTOCOL { // [F17M00][F17M10][F17M20]
    UINTN Revision;
    FABRIC_NUMA_SERVICES_GET_DOMAIN_INFO GetDomainInfo;
    FABRIC_NUMA_SERVICES_DOMAIN_XLAT DomainXlat;
    FABRIC_NUMA_SERVICES_GET_MAX_DOMAINS GetMaxDomains;
};

typedef struct FABRIC_NUMA_SERVICES2_PROTOCOL { // [F17M30][F17M60]
    UINTN Revision;
    FABRIC_NUMA_SERVICES2_GET_DOMAIN_INFO GetDomainInfo;
    FABRIC_NUMA_SERVICES2_DOMAIN_XLAT DomainXlat;
    FABRIC_NUMA_SERVICES2_GET_PHYSICAL_DOMAIN_INFO GetPhysDomainInfo;
    FABRIC_NUMA_SERVICES2_GET_PROXIMITY_DOMAIN_INFO GetPxmDomainInfo;
};
```

Members

Revision	A number indicating the version or revision of the structure used in the protocol. - For F17M00 this value is 0x01. - For F17M30 this value is 0x03.
GetDomainInfo	A function to obtain information about a PCIe® root bridge.
DomainXlat	A function to translate a core's physical location to the appropriate NUMA domain.
GetMaxDomains	A function that returns the number of Proximity domains.
GetPhysDomainInfo	A function to gather information about the physical domains requested via NPS.
GetPxmDomainInfo	Proximity Domain information about a PCIe root-port bridge.

3.7.5.1 GetDomainInfo()

Summary

This function returns information about the NUMA domains.

Prototype

```
typedef
EFI_STATUS (EFI_API *FABRIC_NUMA_SERVICES_GET_DOMAIN_INFO) ( // [F17M00]
    IN    FABRIC_NUMA_SERVICES_PROTOCOL *This,
    OUT   UINT32 *NumberOfDomainsInSystem,
    OUT   DOMAIN_INFO **DomainInfo
);

typedef
EFI_STATUS (EFI_API *FABRIC_NUMA_SERVICES2_GET_DOMAIN_INFO) ( // [F17M30]
    IN    FABRIC_NUMA_SERVICES2_PROTOCOL *This,
    OUT   UINT32 *NumberOfDomainsInSystem,
    OUT   DOMAIN_INFO **DomainInfo,
    OUT   BOOLEAN *CcxAstNumaDomain
);
```


Parameters

This	A pointer to the FABRIC_NUMA_SERVICES2_PROTOCOL instance.
NumberOfDomainsInSystem	Number of unique NUMA domains.
DomainInfo	An array with information about each domain. Be careful to note the structure and type differences between F17M00 and F17M30.
CcxAsNumaDomain	• TRUE: each core complex is its own domain. • FALSE: physical mapping is employed.

Related Definitions

DOMAIN_INFO [F17M00]

```
typedef struct {
    DOMAIN_TYPE Type;
    union {
        SOCKET_INTLV_DOMAIN Socket;
        DIE_INTLV_DOMAIN Die;
        NO_INTLV_DOMAIN None;
    } Intlv;
} DOMAIN_INFO;
```

Type	Specifies the type of interleave used in the domain.
	• SocketIntlv - Interleave by socket. • DieIntlv - Interleave by Die. • NoIntlv - No interleaving.
Intlv	Structure holding the interleave information. Use the matching structure below.

DOMAIN_INFO [F17M30]

```
typedef struct {
    DOMAIN_TYPE2 Type;
    UINT32 SocketMap;
    UINT32 PhysicalDomain;
} DOMAIN_INFO2;
```

Type	Specifies the type of interleave used in the domain.
	• NumaDram - • NumaSLink -
SocketMap	Bitmap indicating physical socket location.
PhysicalDomain	Physical domain number.

SOCKET_INTLV_DOMAIN

```
typedef struct {
    UINT32 SocketCount;
    UINT32 SocketMap;
```

```
    } SOCKET_INTLV_DOMAIN;
```

SocketCount Socket Count.
SocketMap Socket Map.

DIE_INTLV_DOMAIN

```
typedef struct {  
    UINT32    DieCount;  
    UINT32    DieMap;  
} DIE_INTLV_DOMAIN;
```

DieCount Die Count
DieMap Die Map

NO_INTLV_DOMAIN

```
typedef struct {  
    UINT32    Socket;  
    UINT32    Die;  
} NO_INTLV_DOMAIN;
```

Socket Socket identifier (number).
Die Die number.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.

3.7.5.2 DomainXlat()

Summary

This function translates a core's physical location to the appropriate NUMA domain.

Prototype

```
typedef  
EFI_STATUS (EFIAPI *FABRIC_NUMA_SERVICES2_DOMAIN_XLAT) (  
    IN    FABRIC_NUMA_SERVICES2_PROTOCOL *This,  
    IN    UINTN                          Socket,  
    IN    UINTN                          Die,  
    IN    UINTN                          Ccd,  
    IN    UINTN                          Ccx,  
    OUT   UINT32                         *Domain  
);
```

Parameters

This A pointer to the FABRIC_NUMA_SERVICES2_PROTOCOL instance.

Socket	Socket to which the core is attached (zero based).
Die	Die within the socket to which the core is attached.
Ccd	Logical CCD the core is on.
Ccx	Logical core complex.
Domain	Domain to which the core belongs.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.

3.7.5.3 GetPhysicalDomainInfo()

Summary

This function returns information about the physical domains requested via NPS.

Prototype

```
typedef
EFI_STATUS (EFIAPI *FABRIC_NUMA_SERVICES2_GET_PHYSICAL_DOMAIN_INFO) (
    IN    FABRIC_NUMA_SERVICES2_PROTOCOL *This,
    OUT   UINT32                          *NumberOfPhysDomainsInSystem,
    OUT   PHYS_DOMAIN_INFO               **PhysDomainInfo,
    OUT   UINT32                          *PhysNodesPerSocket,
    OUT   UINT32                          *NumberOfSystemSLinkDomains
);
```

Parameters

<i>This</i>	A pointer to the FABRIC_NUMA_SERVICES2_PROTOCOL instance.
<i>NumberOfPhysDomainsInSystem</i>	Number of valid domains in the system.
<i>PhysDomainInfo</i>	An array with information about each physical domain.
<i>PhysNodesPerSocket</i>	Actual NPS as determined by ABL (not including SLink).
<i>NumberOfSystemSLinkDomains</i>	Number of domains describing SLink connected memory.

Related Definitions

Struc Name

CodeBlock

Element	Definition
---------	------------

Struc_Name

CodeBlock

Short Description

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.

3.7.5.4 GetPxmDomainInfo() [F17M30]

Summary

Returns Proximity Domain information about a PCIe® root-port bridge.

Prototype

```
typedef EFI_STATUS (EFI_API *FABRIC_NUMA_SERVICES2_GET_PROXIMITY_DOMAIN_INFO) (
    IN      FABRIC_NUMA_SERVICES2_PROTOCOL *This,
    IN      PCI_ADDR RootPortBDF,
    OUT     PXM_DOMAIN_INFO *PxmDomainInfo
);
```

Parameters

This A pointer to the FABRIC_NUMA_SERVICES2_PROTOCOL instance.

RootPortBDF The PCI Bus-Device-Function (BDF) address for the target root-port bridge. The structure PCI_ADDR is a standard definition in the AGESA.h file.

PxmDomainInfo A pointer to a structure returning associated NUMA node(s).

Description

This function returns Proximity Domain information about a PCIe root-port bridge.

For a given a root-bridge, specified by its BDF, this function identifies:

- the NBIO to which the root-bridge belongs,
- the Quadrant to which the NBIO belongs,
- and finally, the NUMA node (or nodes) associated with the Quadrant.

There are two cases to consider:

1. PcdAmdFabricCcxAsNumaDomain is disabled (FALSE): In this case the Quadrant is associated with a single NUMA node, and the NUMA node to which the Quadrant belongs is based on the NPS setting.
2. PcdAmdFabricCcxAsNumaDomain is enabled (TRUE): In this case the Quadrant may be associated with one or up to four NUMA nodes, depending on the number of CCXes in the Quadrant.

Based on PCD ([PcdAmdFabricRoundRobinNumaDomainForCcx](#)), this function implements a round-robin scheme to return a single NUMA node from the list of nodes associated with a given Quadrant. Otherwise, this function returns the complete list of nodes associated with a given Quadrant, allowing for platform-defined implementations.

Related Definitions

PXM_DOMAIN_INFO

This structure is used to hold a list of the NUMA node(s)/domains(s).

```
typedef struct {
    UINT32 Count;
```

```

    UINT32    Domain[MAX_PXM_VALUES_PER_QUADRANT];
} PXM_DOMAIN_INFO;

```

Count The number of Domains that are in the PXM_DOMAIN_INFO structure.

Domain An array of domain information. The value MAX_PXM_VALUES_PER_QUADRANT is defined for each product. For F17M30 its value is 0x04.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
Other	The requested operation could not be completed.

3.7.5.5 GetMaxDomains()

Summary

This function returns the number of proximity domains.

Prototype

```

typedef
UINT8 (EFI_API *FABRIC_NUMA_SERVICES_GET_MAX_DOMAINS) ( // [F17M00]
    IN          FABRIC_NUMA_SERVICES_PROTOCOL *This
);

```

Parameters

This Pointer to this instance of the protocol.

Return Value

UINT8 This value represents the number of proximity domains in the system.

3.8 Platform Security Processor Driver

Summary

The Platform Security Processor (PSP) handles the early initialization of memory and provides services to the host BIOS.

Description

The Platform Security Processor (PSP) is an embedded controller that executes first upon power-up. It is responsible for securing the BIOS code (validating the image), initializing the system memory, starting the x86 cores and transferring control to the BIOS image.

The PSP driver establishes the mailbox interface between the PSP & the X86 processor, provides the infrastructure for the Secure S3 process, and provides the fTPM bottom layer of services. The PSP driver provides these services to the host BIOS through DXE protocols defined below.

The PSP module receives configuration information via a AMD PSP Configuration Block (APCB) that is stored in the Flash ROM. It provides results and information via the AMD PSP Output Block (APOB) stored in memory. Platforms use the [ABL configuration tool](#) to create the APCB and defined APIs to read the contents of the APOB such as the [Memory Info HOB](#).

3.8.1 PSP Configuration Items

Enablement Category

• PcdAmdPspApcbRecoveryEnable

This feature selects the AP CB recovery support on x86 side. If or when an instance of a writable AP CB is determined to be invalid, the PSP driver will attempt a 'recovery' by copying the recovery instance of the AP CB (default values as indicated in the [APCB descriptor](#) files). Upon boot up, the ABL reads CMOS bytes 06/07 at index/data port of 0x72/0x73. If the CMOS flag reads anything else other than 0xA55A or 0x5555, the system boots in AP CB recovery mode, in which ABL consumes the recovery instances of AP CB. Otherwise it boots in normal/non-recovery mode.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - the AGESA PSP driver restores the AP CB instances from the recovery instance, writes 0xA55A to the CMOS location and triggers a reset. The next time the system powers up, ABL runs in normal/non-recovery mode.
- FALSE - the AGESA PSP driver writes 0x5555 to the CMOS location without restoring the AP CB instances or triggering a reset. In this mode the additional reset is avoided at the potential risk of the writeable AP CB instance being left corrupted forever.

This feature is affected by the platform CMOS power design. Please see [Platform CMOS power](#).

Platform Category

•

Power Management Category

•

Performance Category

•

3.8.2 PSP Protocol (DXE)

Summary

Host BIOS provided protocol that supplies required S3 resume operation data.

GUID

```
gPspPlatformProtocolGuid =
{0xccf14a29, 0x37e0, 0x48ad, { 0x90, 0x5, 0x1f, 0x89, 0x62, 0x2f, 0xb7, 0x98 }}
```

Protocol Interface Structure

```
typedef struct _PSP_PLATFORM_PROTOCOL {
    BOOLEAN                CpuContextResumeEnable;
    UINT8                  SwSmiCmdtoBuildContext;
    UINT32                  BspStackSize;
    UINT32                  ApStackSize;
    RSM_HANDOFF_INFO        *RsmHandOffInfo
} PSP_PLATFORM_PROTOCOL;
```

Members

CpuContextResumeEnable	Indicates the platform state of S3. This should be set to TRUE in all production BIOS. • TRUE: Enable Secure S3 • FALSE: Disable Secure S3
SwSmiCmdtoBuildContext	The SW SMI command value used by the host BIOS to tell the PSP Driver to save the CPU context.
BspStackSize	The size, in bytes, of the BSP Stack for PEI code during resume.
ApStackSize	The size, in bytes, of the AP Stack for PEI code during resume.
RsmHandOffInfo	Pointer to a structure containing the Secure S3 driver required customization information.

Description

This protocol provides platform customization information to the PSP driver. The host BIOS must create and publish this protocol. Through the information in this protocol structures, the host BIOS informs the AMD PSP driver how to operate the resume process on this platform.

Related Definitions

RSM_HANDOFF_INFO

```
typedef struct {
    UINT32  GdtOffset;
    UINT16  CodeSelector;
    UINT16  DataSelector;
    UINT32  RsmEntryPoint;
    UINT32  EdxResumeSignature;
} RSM_HANDOFF_INFO;
```

GdtOffset	Address of the GDT table. The GDT table is established and provide by the host BIOS. The secure S3 driver loads this GDT table before handing control over to the platform code.
CodeSelector	The index for the CODE Segment Selector when handing control to the platform code.
DataSelector	The index for the DATA Segment Selector when handing control to the platform code.
RsmEntryPoint	The address to which secure S3 driver will jump after it finishes its tasks. This address is the platform code resume entry point.
EdxResumeSignature	This is a value which is kept in EDX when handing control over to the platform code. This can be used to check if this is a secure S3 resume.

3.8.3 PSP_RESUME_SERVICE_PROTOCOL

Summary

Registers a driver with the SMM dispatcher.

GUID

```
gAmdPspResumeServiceProtocolGuid = {0x6c5ae0f9, 0xaad3, 0x47f8,
    { 0x8f, 0x59, 0xa5, 0x3a, 0x54, 0xce, 0x5a, 0xe2 }}
```

Protocol Interface Structure

```
typedef struct _PSP_RESUME_SERVICE_PROTOCOL {
    PSP_RESUME_REGISTER    Register;
    PSP_RESUME_UNREGISTER  UnRegister;
} PSP_RESUME_SERVICE_PROTOCOL ;
```

Members

Register	A function for registering a hook function with the SecureS3 dispatcher. The hook function will be called by the Secure S3 dispatcher during S3 or S0i3 exit.
UnRegister	A function for un-registering a hook function with the SecureS3 dispatcher.

Description

The AGESA™ modules provide the core SMI handler function. Device drivers may register a specialized handler for their device with the SMI handler for implementing their device special needs during resume. These device specific drivers will be called in registration order during the S3 resume process.

3.8.3.1 PSP_Resume_Register()

Summary

Register a function with the S3 resume dispatcher.

Prototype

```
typedef EFI_STATUS (EFI_API *PSP_RESUME_REGISTER) (
    IN      PSP_RESUME_SERVICE_PROTOCOL    *This,
    IN      PSP_RESUME_CALLBACK            CallbackFunction,
    IN OUT  VOID                           *Context,
    IN      UINTN                          CallbackPriority,
    IN      PSP_RESUME_CALLBACK_FLAG       Flag,
    OUT     EFI_HANDLE                     *DispatchHandle
)
```

Parameters

This	A pointer to the parent protocol structure.
CallbackFunction	The address location of the function to be registered.
Context	A pointer that will be passes to the function as a parameter. This should point to data that will be used by the function.
CallbackPriority	Desired priority in the dispatcher. CallbackPriority has 5 defined levels of priority, as below, from lowest level to highest level, and highest level will be dispatch 1st. When two function register with same priority, the last one registered will be dispatched 1st. • PSP_RESUME_CALLBACK_LOWEST_LEVEL • PSP_RESUME_CALLBACK_LOW_LEVEL • PSP_RESUME_CALLBACK_MEDIUM_LEVEL • PSP_RESUME_CALLBACK_HIGH_LEVEL • PSP_RESUME_CALLBACK_HIGHEST_LEVEL
Flag	A flag used to describe which core is allowed to run the registered handle, the supported value is defined as below: • PspResumeCallBackFlagBspOnly - Call Back function will only be executed on BSP.

- PspResumeCallBackFlagCorePrimaryOnly - Call Back function will only be executed on CorePrimary.
- PspResumeCallBackFlagCcxPrimaryOnly- Call Back function will only be executed on CcxPrimary.
- PspResumeCallBackFlagDiePrimaryOnly - Call Back function will only be executed on DiePrimary.
- PspResumeCallBackFlagSocketPrimaryOnly - Call Back function will only be executed on SocketPrimary.
- PspResumeCallBackFlagAllCores - Call Back function will be executed on all cores.

DispatchHandle

A pointer to a location where the registration function can place an identifier created for this function registration. This identifier will be needed when un-registering the function.

Description

This function is used by device drivers to register their call-back function with the core SMI handler function. The SMI handler will be executed at the very beginning of S3exit for both BSP and AP. For BSP it will be execute before handing execution to UEFI security phase. For AP it will be executed when AP is launched. The dispatch order is controlled by CallbackPriority, and the Flag parameter can be used to designate which core will execute the registered handler.

3.8.3.2 PSP_Resume_Unregister()**Summary**

Function called to remove a call-back function from the SMI handler's registration list.

Prototype

```
typedef EFI_STATUS (EFI_API *PSP_RESUME_UNREGISTER) (
    IN      PSP_RESUME_SERVICE_PROTOCOL  *This,
    IN      EFI_HANDLE                   DispatchHandle
);
```

Parameters**This**

A pointer to the parent protocol structure.

DispatchHandle

The identifier provided by the S3 Register function.

Description

This function is called to remove a device driver's call-back function from the SMI handlers registration list. The identifier must match that produced by the registration function. After this function call completes, the device driver's call-back function will no longer be invoked during an S3 resume sequence.

3.8.4 PSP_fTPM_PPI**Summary**

This PPI provides service functions for the bottom layer functionality of the firmware Trusted Program Module (fTPM)

GUID

```
gAmdPspFtpmPpiGuid = {0x91774185, 0xf72d, 0x467e,
                      {0x93, 0x39, 0xe0, 0x8, 0xdb, 0xae, 0xe, 0x14}}
```

PPI Interface Structure

```

Ctypedef struct _PSP_FTPM_PPI {
    FTPM_EXECUTE           Execute;
    FTPM_CHECK_STATUS      CheckStatus;
    FTPM_SEND_COMMAND      SendCommand;
    FTPM_GET_RESPONSE      GetResponse;
} PSP_FTPM_PPI;

```

Parameters

Execute A function to initiate a TPM command.

CheckStatus A function to retrieve the status of the TPM device.

SendCommand A function to send a command to the TPM device.

GetResponse A function to retrieve a command response from the TPM device.

Description

This PEI phase PPI is provide to allow the host BIOS to perform validation of code blocks. The fTPM is a firmware emulation of a TPM device. Commands and services can be performed as if communicating to a TPM discrete device.

The installation of this PPI indicates that the readiness of fTPM device, the TPM drivers can utilize this PPI to send TPM command to PSP simulated fTPM device

3.8.4.1 fTPM_Execute()

Summary

This function invokes a command to the fTPM.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *FTPM_EXECUTE) (
    IN      PSP_FTPM_PPI
    IN      VOID
    IN      UINTN
    IN OUT VOID
    IN OUT UINTN
    *This,
    *CommandBuffer,
    CommandSize,
    *ResponseBuffer,
    *ResponseSize
);

```

Parameters

This A pointer to the parent structure.

CommandBuffer A pointer to a buffer that contains the TPM command.

CommandSize The size, in bytes, of the TPM command buffer.

ResponseBuffer A pointer to an area where to place the TPM command response.

ResponseSize The size, in bytes, of the TPM response buffer.

Description

This function will send the command to the fTPM, wait for the command completion and retrieve the command result. The host BIOS does not need to perform these action separately.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.4.2 fTPM_Check_Status()

Summary

This function is used to retrieve the present status of the fTPM device.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *FTPM_CHECK_STATUS) (
    IN      PSP_FTPM_PPI      *This,
    IN OUT  UINTN             *FtpmStatus
);

```

Parameters

This A pointer to the parent structure.

FtpmStatus Pointer to a location where to store the fTPM information.

Description

This function retrieves the current status of the fTPM device. The host BIOS may use this to manually operate fTPM commands on its own.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.4.3 FTPM_Send_Command()

Summary

This function sends a command to the fTPM for execution.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *FTPM_SEND_COMMAND) (
    IN      PSP_FTPM_PPI      *This,
    IN      VOID               *CommandBuffer,
    IN      UINTN              CommandSize
);

```

Parameters

This A pointer to the parent structure.

CommandBuffer Pointer to a location that contains the fTPM command packet.

CommandSize The size, in bytes, of the TPM command buffer.

Description

This function sends the specified command packet to the fTPM. This routine does not wait for command completion or a response.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.4.4 fTPM_Get_Response()

Summary

This function waits for a fTPM command response.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *fTPM_GET_RESPONSE) (
    IN      PSP_fTPM_PPI          *This,
    IN OUT VOID                   *ResponseBuffer,
    IN OUT UINTN                  *ResponseSize
);
```

Parameters

This A pointer to the parent structure.

ResponseBuffer A pointer to an area where to place the TPM command response.

ResponseSize The size, in bytes, of the TPM response buffer.

Description

This function communicates with the fTPM device to check the status and will wait for the current command to complete. It will then retrieve the result and present the command response to the caller.

Related Definitions

Struc Name

CodeBlock

Element

Definition

Struc_Name

CodeBlock

Short Description

Element

Definition

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.5 PSP_fTPM_Protocol

Summary

This Protocol provides service functions for the bottom layer functionality of the firmware Trusted Program Module (fTPM)

GUID

```
gAmdPspFtpmProtocolGuid = { 0xac234e04, 0xb036, 0x476c,  
                             {0x91, 0x66, 0xbe, 0x47, 0x52, 0xa0, 0x95, 0x9}}
```

Protocol Interface Structure

```
typedef struct _PSP_FTPM_PROTOCOL {  
    FTPM_EXECUTE           Execute;  
    FTPM_CHECK_STATUS      CheckStatus;  
    FTPM_SEND_COMMAND      SendCommand;  
    FTPM_GET_RESPONSE      GetResponse;  
} PSP_FTPM_PROTOCOL;
```

Members

Execute	A function to initiate a TPM command.
CheckStatus	A function to retrieve the status of the TPM device.
SendCommand	A function to send a command to the TPM device.
GetResponse	A function to retrieve a command response from the TPM device.

Description

This DXE Protocol is provide to allow the host BIOS to perform validation of code blocks. The fTPM is a firmware emulation of a TPM device. Commands and services can be performed as if communicating to a TPM discrete device.

3.8.5.1 fTPM_Execute()

Summary

This function invokes a command to the fTPM.

Prototype

```
typedef  
EFI_STATUS  
(EFIAPI *FTPM_EXECUTE) (  
    IN      PSP_FTPM_PPI      *This,  
    IN      VOID              *CommandBuffer,  
    IN      UINTN             CommandSize,  
    IN OUT  VOID              *ResponseBuffer,  
    IN OUT  UINTN             *ResponseSize  
);
```

Parameters

This	A pointer to the parent structure.
CommandBuffer	A pointer to a buffer that contains the TPM command.
CommandSize	The size, in bytes, of the TPM command buffer.
ResponseBuffer	A pointer to an area where to place the TPM command response.
ResponseSize	The size, in bytes, of the TPM response buffer.

Description

This function will send the command to the fTPM, wait for the command completion and retrieve the command result. The host BIOS does not need to perform these action separately.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.5.2 fTPM_Check_Status()

Summary

This function is used to retrieve the present status of the fTPM device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *FTP_CHECK_STATUS) (
    IN      PSP_FTPM_PPI      *This,
    IN OUT  UINTN             *FtpmStatus
);
```

Parameters

This A pointer to the parent structure.
FtpmStatus Pointer to a location where to store the fTPM information.

Description

This function retrieves the current status of the fTPM device. The host BIOS may use this to manually operate fTPM commands on its own.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.5.3 FTPM_Send_Command()

Summary

This function sends a command to the fTPM for execution.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *FTP_SEND_COMMAND) (
    IN      PSP_FTPM_PPI      *This,
    IN      VOID              *CommandBuffer,
    IN      UINTN              CommandSize
);
```

Parameters

This A pointer to the parent structure.
CommandBuffer Pointer to a location that contains the fTPM command packet.
CommandSize The size, in bytes, of the TPM command buffer.

Description

This function sends the specified command packet to the fTPM. This routine does not wait for command completion or a response.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.5.4 fTPM_Get_Response()

Summary

This function waits for a fTPM command response.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *fTPM_GET_RESPONSE) (
    IN      PSP_fTPM_PPI          *This,
    IN OUT VOID                  *ResponseBuffer,
    IN OUT UINTN                 *ResponseSize
);
```

Parameters

This A pointer to the parent structure.

ResponseBuffer A pointer to an area where to place the TPM command response.

ResponseSize The size, in bytes, of the TPM response buffer.

Description

This function communicates with the fTPM device to check the status and will wait for the current command to complete. It will then retrieve the result and present the command response to the caller.

Related Definitions

Struc Name

CodeBlock

Element

Definition

Struc_Name

CodeBlock

Short Description

Element

Definition

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.6 AMD_APCB_SERVICE_PROTOCOL

Summary

This protocol provides the tools for the system BIOS to modify APCB entries. The Protocol structure has undergone some growth as Models mature. Versions 2 and 3 of this structure are introduced in the Platform Initialization Packages (AGESA_PI) starting with support for Family 17 Model 30 and the AM4ComboPI. Previous packages used the version 1 or version 1.5 structures as noted in the structure declaration below. Please refer to the files \AgesaModulePkg\Include\Protocol\AmdApcbProtocol.h in your PI package. All new PI packages will use the version 3 structure.

GUID

```
extern EFI_GUID gAmdApcbDxeServiceProtocolGuid;
```

Protocol Interface Structure

```
struct _AMD_APCB_SERVICE_PROTOCOL { //Version 1.0 [F17M10][F17M20]
    FP_SET_ACTIVE_APCB_INSTANCE      ApcbSetActiveInstance;
    FP_FLUSH_APCB_DATA               ApcbFlushData;
    FP_UPDATE_CBS_APCB_DATA          ApcbUpdateCbsData;
    FP_GET_CONFIG_PARAM              ApcbGetConfigParameter;
    FP_SET_CONFIG_PARAM              ApcbSetConfigParameter;
    FP_GET_CBS_PARAM                 ApcbGetCbsParameter;
    FP_SET_CBS_PARAM                 ApcbSetCbsParameter;
    FP_CLEAR_DRAM_POSTPKG_REPAIR_ENTRY ApcbClearDramPostPkgRepairEntry;
    FP_ADD_DRAM_POSTPKG_REPAIR_ENTRY  ApcbAddDramPostPkgRepairEntry;
    FP_REMOVE_DRAM_POSTPKG_REPAIR_ENTRY ApcbRemoveDramPostPkgRepairEntry;
};

struct _AMD_APCB_SERVICE_PROTOCOL { //Version 1.5 [F17M00]
    FP_SET_ACTIVE_APCB_INSTANCE      ApcbSetActiveInstance;
    FP_FLUSH_APCB_DATA               ApcbFlushData;
    FP_UPDATE_CBS_APCB_DATA          ApcbUpdateCbsData;
    FP_GET_CONFIG_PARAM              ApcbGetConfigParameter;
    FP_SET_CONFIG_PARAM              ApcbSetConfigParameter;
    FP_GET_CBS_PARAM                 ApcbGetCbsParameter;
    FP_SET_CBS_PARAM                 ApcbSetCbsParameter;
    FP_GET_DRAM_POSTPKG_REPAIR_ENTRY ApcbGetDramPostPkgRepairEntries;
    FP_CLEAR_DRAM_POSTPKG_REPAIR_ENTRY ApcbClearDramPostPkgRepairEntry;
    FP_ADD_DRAM_POSTPKG_REPAIR_ENTRY  ApcbAddDramPostPkgRepairEntry;
    FP_REMOVE_DRAM_POSTPKG_REPAIR_ENTRY ApcbRemoveDramPostPkgRepairEntry;
};

struct _AMD_APCB_SERVICE_PROTOCOL { //Version 2 [F17M00][F17M10][F17M20]
    //
    // Common APCB services
    //
    UINT32 Version;
    FP_SET_ACTIVE_APCB_INSTANCE      ApcbSetActiveInstance;
    FP_FLUSH_APCB_DATA               ApcbFlushData;
    FP_GET_DRAM_POSTPKG_REPAIR_ENTRY ApcbGetDramPostPkgRepairEntries;
    FP_CLEAR_DRAM_POSTPKG_REPAIR_ENTRY ApcbClearDramPostPkgRepairEntry;
    FP_ADD_DRAM_POSTPKG_REPAIR_ENTRY  ApcbAddDramPostPkgRepairEntry;
    FP_REMOVE_DRAM_POSTPKG_REPAIR_ENTRY ApcbRemoveDramPostPkgRepairEntry;

    FP_GET_CONFIG_PARAM              ApcbGetConfigParameter;
    FP_SET_CONFIG_PARAM              ApcbSetConfigParameter;
    FP_GET_CBS_PARAM                 ApcbGetCbsParameter;
    FP_SET_CBS_PARAM                 ApcbSetCbsParameter;
    FP_UPDATE_CBS_APCB_DATA          ApcbUpdateCbsData;
};

struct _AMD_APCB_SERVICE_PROTOCOL { // Version 3 [F17M30][F17M70]
    //
    // Common APCB services
    //
```



```

UINT32
FP_FLUSH_APCB_DATA
FP_GET_DRAM_POSTPKG_REPAIR_ENTRY
FP_CLEAR_DRAM_POSTPKG_REPAIR_ENTRY
FP_ADD_DRAM_POSTPKG_REPAIR_ENTRY
FP_REMOVE_DRAM_POSTPKG_REPAIR_ENTRY
//
// APCB 2.0 services
//
FP_SET_ACTIVE_APCB_INSTANCE
FP_GET_CONFIG_PARAM
FP_SET_CONFIG_PARAM
FP_GET_CBS_PARAM
FP_SET_CBS_PARAM
FP_UPDATE_CBS_APCB_DATA
//
// APCB 3.0 services
//
FP_ACQUIRE_MUTEX
FP_RELEASE_MUTEX
FP_GET_TOKEN_BOOL
FP_SET_TOKEN_BOOL
FP_GET_TOKEN_8
FP_SET_TOKEN_8
FP_GET_TOKEN_16
FP_SET_TOKEN_16
FP_GET_TOKEN_32
FP_SET_TOKEN_32
FP_GET_TYPE
FP_SET_TYPE
FP_PURGE_ALL_TOKENS
FP_PURGE_ALL_TYPES
};

```

```

Version;
ApcbFlushData;
ApcbGetDramPostPkgRepairEntries;
ApcbClearDramPostPkgRepairEntry;
ApcbAddDramPostPkgRepairEntry;
ApcbRemoveDramPostPkgRepairEntry;

ApcbSetActiveInstance;
ApcbGetConfigParameter;
ApcbSetConfigParameter;
ApcbGetCbsParameter;
ApcbSetCbsParameter;
ApcbUpdateCbsData;

ApcbAcquireMutex;
ApcbReleaseMutex;
ApcbGetTokenBool;
ApcbSetTokenBool;
ApcbGetToken8;
ApcbSetToken8;
ApcbGetToken16;
ApcbSetToken16;
ApcbGetToken32;
ApcbSetToken32;
ApcbGetType;
ApcbSetType;
ApcbPurgeAllTokens;
ApcbPurgeAllTypes;

```

Members

Version

Integer value indicating the structure version.

ApcbSetActiveInstance

Function to set the active instance of APCB.

ApcbFlushData

Function to flush APCB data back to the SPI ROM.

ApcbUpdateCbsData

Function to update CBS APCB data.

ApcbGetConfigParameter

Function to get an APCB configuration parameter.

ApcbSetConfigParameter

Function to set an APCB configuration parameter.

ApcbGetCbsParameter

Function to get an APCB CBS parameter.

ApcbSetCbsParameter

Function to set an APCB CBS parameter.

ApcbGetDramPostPkgRepairEntries

Function to retrieve DRAM Post Package Repair Entries.

ApcbClearDramPostPkgRepairEntry

Function to clear DRAM Post Package Repair Entries.

ApcbAddDramPostPkgRepairEntry

Function to add a DRAM Post Package Repair Entry.

ApcbRemoveDramPostPkgRepairEntry

Function to remove a DRAM Post Package Repair Entry.

ApcbAcquireMutex

Function to Acquire the Mutex.

ApcbReleaseMutex

Function to Release the Mutex.

ApcbGetTokenBool

Function to get an APCB BOOL token.

ApcbSetTokenBool

Function to set an APCB BOOL token.

ApcbGetToken8

Function to get an APCB UINT8 token.

ApcbSetToken8

Function to set an APCB UINT8 token.

ApcbGetToken16

Function to get an APCB UINT16 token.

ApcbSetToken16	Function to set an APCB UINT16 token.
ApcbGetToken32	Function to get an APCB UINT32 token.
ApcbSetToken32	Function to set an APCB UINT32 token.
ApcbGetType	Function to retrieve the data of a specified type.
ApcbSetType	Function to set the data of a specified type.
ApcbPurgeAllTokens	Function to purge all token/value pairs.
ApcbPurgeAllTypes	Function to purge the data of all types.

Description

A copy of the APCB is retained in memory as the 'shadow image'. The platform BIOS may have need to modify the contents of the APCB at POST time or RunTime. The system BIOS is recommended to make use of these services to retrieve or change APCB data elements. To reference this protocol, include AmdApcbProtocol.h and ApcbCommon.h in the drivers. Please note that any write operation occurs only in APCB shadow till ApcbFlushData() is invoked to write it back to the flash ROM.

To start an APCB operation sequence, first acquire the APCB mutex before any other APCB manipulation. Once the mutex is acquired, the type or token manipulation services may be used. Finally, the writeback service needs to be invoked to write the shadow copy back to the flash part. Once APCB access is no longer necessary, the mutex must be released. The APCB operation sequence has to be atomic. That means multiple masters should not be attempting simultaneous modifications. The mutex is provided to help avoid confusing operations. In DXE phase APCB services are only allowed to be invoked on the BSP. After ReadyToBoot, APCB services are only allowed to be invoked in SMI which are naturally not re-entrant.

An APCB token may be saved in different instances or purpose levels and can have instances of the token at multiple purpose levels. These purpose levels provide a hierarchy of priority such that a token entry at one purpose level can be added to override the same token value set at a lower purpose level. The classic example is a priority system such that ADMIN -> DEBUGGING -> NORMAL, which means something occurring at a higher priority level would override another at a lower one. If the user set a token to TRUE at the debugging level and set the same one to FALSE at the normal level, the token readout would be TRUE. The intent is that a token can be temporarily changed for debug or evaluation; but should eventually be migrated to the 'Normal' purpose.

APCB operations can be performed independently on each of the different purposes levels. Some of the APCB services require the consumer to provide the purpose, others return the purpose at which the token is found.

Some of the purposes (admin/debug/normal) can set their priority level via PCD variables. By default, ADMIN -> DEBUG -> NORMAL (e.g. High -> Medium -> Low), but the user can change it. For example if PcdAmdApcbPriorityLevelDebug is set to High and PcdAmdApcbPriorityLevelAdmin set to Medium, the operation in DEBUGGING realm will be prioritized over one in ADMIN (DEBUG -> ADMIN -> NORMAL). This can be done without actually changing the invocations in the executable code, but simply by changing the assignments of the PCDs.

PCD controls

PcdAmdApcbPriorityLevelAdmin	The assigned priority level for the 'Admin' purpose. Permitted Choices: (Type: List)(Default: APCB_PRIORITY_HIGH) • APCB_PRIORITY_HIGH • APCB_PRIORITY_MEDIUM • APCB_PRIORITY_LOW
PcdAmdApcbPriorityLevelDebug	The assigned priority level for the 'Debug' purpose.

Permitted Choices: (Type: List)(Default:
APCB_PRIORITY_MEDIUM)

- APCB_PRIORITY_HIGH
- APCB_PRIORITY_MEDIUM
- APCB_PRIORITY_LOW

PcdAmdApcbPriorityLevelNormal The assigned priority level for the 'Normal' purpose.

Permitted Choices: (Type: List)(Default: APCB_PRIORITY_LOW)

- APCB_PRIORITY_HIGH
- APCB_PRIORITY_MEDIUM
- APCB_PRIORITY_LOW

Related Definitions

APCB_TYPE_PURPOSE

```
typedef enum {
    APCB_TYPE_PURPOSE_HARD_FORCE,
    APCB_TYPE_PURPOSE_ADMIN,
    APCB_TYPE_PURPOSE_DEBUG,
    APCB_TYPE_PURPOSE_EVENT_LOGGING,
    APCB_TYPE_PURPOSE_NORMAL,
    APCB_TYPE_PURPOSE_DEFAULT,
} APCB_TYPE_PURPOSE;
```

This defines the default priority order of the purpose labels. Some of these can be altered by the above PCD controls.

Purpose

APCB Token instance retrieval order. When reading a token, the instance with the highest priority will be returned as the token value. Labels are in highest to lowest priority:

1. HARD_FORCE - (Highest priority) Force this value to always be used.
2. ADMIN - Administrator's set of values.
3. DEBUG - values used for temporary debugging.
4. EVENT_LOGGING - Instances used for Data Logging.
5. NORMAL - Instances intended for production.
6. DEFAULT - (lowest priority) Instances for tokens that have no platform specific setting.

3.8.6.1 ApcbSetActiveInstance()

Summary

This function sets the active APCB instance for subsequent editing.

Prototype

```
VOID
mApcbSetActiveInstance (
    IN      AMD_APCB_SERVICE_PROTOCOL    *This,
    IN      UINT8                        Instance
);
```

Parameters

This Pointer to the current instance of the parent protocol.
Instance Index of the instance targeted for modification.

Description

This function identifies a location in the APCB where subsequent modifications are to be performed. This sets the editing pointer in the APCB shadow copy. This function may not be available on all platforms, please check the return code.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_NOT_SUPPORTED	The function is not supported on this processor.

3.8.6.2 ApcbFlushData()

Summary

This function 'flushes' the APCB data back to SPI ROM.

Prototype

```
EFI_STATUS
mApcbFlushData (
    IN          AMD_APCB_SERVICE_PROTOCOL    *This
);
```

Parameters

This Pointer to the current instance of the parent protocol.

Description

This function writes any modified APCB shadow copy in DRAM back to the SPI flash ROM for use on the next boot cycle. To make sure the writeback is successful, the reserved space for the APCB instance in the flash part has to be greater than the size of the APCB shadow instance.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_UNSUPPORTED	The APCB shadow instance cannot be written back. Check that the shadow copy will fit into the reserved space in the Flash ROM.

3.8.6.3 ApcbGetConfigParameter()

Summary

This function extracts an APCB config data item.

Prototype

```
EFI_STATUS
mApcbGetConfigParameter (
    IN          AMD_APCB_SERVICE_PROTOCOL    *This,
    IN          UINT16                        TokenId,
    IN OUT     UINT32                        *SizeInByte,
    IN OUT     UINT64                        *Value
);
```

```
);
```

Parameters

This Pointer to the current instance of the parent protocol.

TokenId APCB token ID as defined by APCB_COMMON_CONFIG_ID in ApcbCommon.h

SizeInByte Size, in bytes, of the data block to be inserted, or replaced, into the APCB.

Value Pointer to a data block that will be used to contain the target element.

Description

This function is used to get the build configuration type instance of a single APCB token.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOND	This routine could not find the indicated token.
EFI_INVALID_PARAMETER	One of the parameters is not valid.

3.8.6.4 ApcbSetConfigParameter()**Summary**

This function sets the value of a APCB config data item.

Prototype

```
EFI_STATUS
mApcbSetConfigParameter (
    IN      AMD_APCB_SERVICE_PROTOCOL    *This,
    IN      UINT16                        TokenId,
    IN OUT  UINT32                        *SizeInByte,
    IN OUT  UINT64                        *Value
);
```

Parameters

This Pointer to the current instance of the parent protocol.

TokenId APCB token ID as defined by APCB_COMMON_CONFIG_ID in ApcbCommon.h

SizeInByte Size, in bytes, of the data block to be inserted, or replaced, into the APCB.

Value Pointer to a data block that will be used to update the target element.

Description

This function is used to set the build configuration type instance of a single APCB token.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOND	This routine could not find the indicated token.

status code	meaning
EFI_INVALID_PARAMETER	One of the parameters is not valid.

3.8.6.5 ApcbGetCbsParameter()

Summary

This function extracts a APCB CBS parameter.

Prototype

```
EFI_STATUS
mApcbGetCbsParameter (
    IN      AMD_APCB_SERVICE_PROTOCOL    *This,
    IN      UINT16                        TokenId,
    IN OUT  UINT32                        *SizeInByte,
    IN OUT  UINT64                        *Value
);
```

Parameters

This Pointer to the current instance of the parent protocol.

TokenId APCB token ID as defined by APCB_COMMON_CONFIG_ID in ApcbCommon.h

SizeInByte Size, in bytes, of the data block to be inserted, or replaced, into the APCB.

Value Pointer to a data block that will be used to contain the target element.

Description

This function is used to get the CBS type instance of an APCB-CBS token.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	This routine could not find the indicated token.
EFI_INVALID_PARAMETER	One of the parameters is not valid.

3.8.6.6 ApcbSetCbsParameter()

Summary

This function sets the value of an APCB CBS data item.

Prototype

```
EFI_STATUS
mApcbSetCbsParameter (
    IN      AMD_APCB_SERVICE_PROTOCOL    *This,
    IN      UINT16                        TokenId,
    IN OUT  UINT32                        *SizeInByte,
    IN OUT  UINT64                        *Value
);
```

Parameters

This Pointer to the current instance of the parent protocol.

TokenId	APCB token ID as defined by APCB_COMMON_CONFIG_ID in ApcbCommon.h
SizeInByte	Size, in bytes, of the data block to be inserted, or replaced, into the APCB.
Value	Pointer to a data element that will be used to update the target element.

Description

This function is used to set the CBS type instance of a single APCB-CBS token.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	This routine could not find the indicated token.
EFI_INVALID_PARAMETER	One of the parameters is not valid.

3.8.6.7 ApcbUpdateCbsData()**Summary**

This function updates the APCB data for CBS control instantiation phase.

Prototype

```
EFI_STATUS
mApcbUpdateCbsData (
    IN      AMD_APCB_SERVICE_PROTOCOL *This,
    IN      UINT8                      *ApcbTypeData,
    IN      UINT32                     SizeInByte,
    IN      BOOLEAN                    Internal
);
```

Parameters

This	Pointer to the current instance of the parent protocol.
ApcbTypeData	The APCB token/value pairs associated with CBS are organized as a token attribute stream followed by a value stream. The token attribute stream is an array of APCB_PARAM_ATTRIBUTE structs ended with {0, APCB_TOKEN_CONFIG_LIMIT, 0, 0}. The value stream is a series of bool/uint8/uint16/uint32/uint64 values placed back to back, usually defined by the macros BSBLN/BSU08/BSU16/BSU32/BSU64 which will divide the values into byte entries. A value in the value stream is associated with the token attribute at the same location of the token attribute stream, and the size of that value must match exactly the size defined by the associated token attribute.
SizeInByte	Size, in bytes, of the token attribute stream plus the value stream in bytes.
Internal	Flag indicating scope of the data. • TRUE - Update the internal CBS APCB data, • FALSE - Update the external CBS APCB data

Description

Whereas the Get and Set functions operate on a single APCB token at a time, this function provides the ability to batch process several tokens. This function is invoked in the CBS 'backend' code to update the APCB token(s)

modified by CBS (SETUP) changes. While not intended for mass customer use, this routine can be used with a group of token/value pairs to make multiple changes in one function.

Related Definitions

APCB_PARAM_ATTRIBUTE

```
typedef struct {
    UINT32      TimePoint:8;
    UINT32      Token:13;
    UINT32      Size:3;
    UINT32      Reserved:8;
} APCB_PARAM_ATTRIBUTE;
```

TimePoint

Specify when the parameter can be retrieved.

- APCB_TIME_POINT_NONE
- APCB_TIME_POINT_ANY
- APCB_TIME_POINT_BEFORE_DRAM_INIT
- APCB_TIME_POINT_AFTER_DRAM_SPD_TIMING_INIT
- APCB_TIME_POINT_AFTER_DRAM_DATASHEET_TIMING_INIT
- APCB_TIME_POINT_AFTER_DRAM_PHY_INIT
- APCB_TIME_POINT_AFTER_DRAM_INIT
- APCB_TIME_POINT_BEFORE_PMU_TRAINING
- APCB_TIME_POINT_AFTER_PMU_TRAINING
- APCB_TIME_POINT_AFTER_DRAM_FINALIZATION
- APCB_TIME_POINT_BEFORE_GMI_TRAINING
- APCB_TIME_POINT_AFTER_GMI_TRAINING
- APCB_TIME_POINT_AFTER_DF_FINAL_INIT
- APCB_TIME_POINT_BEFORE_DF_SPF_INIT
- APCB_TIME_POINT_BEFORE_DF_CREDIT_RELEASE
- APCB_TIME_POINT_DF_EARLY

Token

APCB token. Definitions can be found in the APCB.h include file.

Size

The actual size of the parameter - 1, in bytes.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.6.8 ApcbAddDramPostPkgRepairEntry()

Summary

This function adds a DRAM POST Package repair entry to the APCB.

Prototype

```
EFI_STATUS
mApcbAddDramPostPkgRepairEntry (
    IN      AMD_APCB_SERVICE_PROTOCOL  *This,
    IN      DRRP_REPAIR_ENTRY          *Entry
);
```

Parameters

This Entry Pointer to the current instance of the parent protocol.
DRAM POST Package repair entry to be added.

Description

This function adds a DRAM POST Package repair entry into the APCB. The repairs will be made by the ABL at the next boot.

Related Definitions

DPPRCL_REPAIR_ENTRY

```
typedef struct _DPPRCL_REPAIR_ENTRY {
    UINT32 Bank           :5;
    UINT32 RankMultiplier :3;
    UINT32 Device         :5;
    UINT32 ChipSelect     :2;
    UINT32 Column         :10;
    UINT32 RepairType     :1;
    UINT32 Valid          :1;
    UINT32 TargetDevice   :5;
    UINT32 Row            :18;
    UINT32 Socket         :3;
    UINT32 Channel        :3;
    UINT32 Reserverd1     :8;
} DPPRCL_REPAIR_ENTRY;
```

This structure identifies a point in the memory that is needing repair. Please note that a memory physical address can be translated into {CS, Bank, Row, Column} by using the [AMD_RAS_SMM_PROTOCOL.TranslateSysAddrToCS\(\)](#) function.

Bank This value incorporates the Bank Group and Bank Address components.

- Bit 4 - ??
- bits 3:2 - Bank Group.
- bits 1:0 - Bank Address or Bank Number.

RankMultiplier Rank multiplication was introduced with LRDIMMs and expands the number of ranks capable of being supported on a DIMM. Please see your DIMM manufacturer for more details.

Device Specifies the target DRAM device by width where all DRAM devices of the class will be repaired.

- 4 - Repair DIMMs with device width x4 only.
- 8 - Repair DIMMs with device width x8 only.
- 16 - Repair DIMMs with device width x16 only.
- 0x1F - Ignore device width and repair a specific device.

ChipSelect Chip Select controlling the repair location.

Column Column address where the repair is needed.

RepairType A 'Hard' will modify the DIMM instantiating a permanent and non-reversible repair. A soft repair will invoke a temporary repair that needs to be repeated at the next reset. It is recommended to use the Soft repair type.

- 0 - Hard repair.
- 1 - Soft repair.

Valid Indicates that the entry is valid and should be employed. An entry may be temporarily disabled by clearing this bit.

TargetDevice

When Device = 0x1F (not using a DRAM class), this entry identifies the target DRAM device by index. The index depends upon the DRAM device bit width.

DRAM width	DQ63:0	ECC, if present
x4	indexes 15:0	index 17:16
x8	indexes 7:0	index 8
x16	indexes 3:0	index 4

Row

Row address where the repair is needed.

Socket

Index of the socket [0..N], relative to the system, on which the DIMM is located.

Channel

Index of the channel [0..N], relative to socket, where the DIMM is located.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.6.9 ApcbGetDramPostPkgRepairEntries()**Summary**

This function retrieves DRAM POST Package repair entries from the APCB.

Prototype

```
EFI_STATUS
ApcbGetDramPostPkgRepairEntries (
    IN      AMD_APCB_SERVICE_PROTOCOL    *This,
    IN OUT  DRRP_REPAIR_ENTRY *Entry,
    IN OUT  UINT32              *NumOfEntries
);
```

Parameters**This**

Pointer to the current instance of the parent protocol.

Entry

This is a pointer to a local buffer that will be filled with a copy of the DRAM POST Package repair entries.

NumOfEntries

On entry, this is the number of DRAM Post Package Repair entries to be retrieved. On exit, this is the actual number of entries copied.

Description

This function is used to retrieve an array of DRAM Post Package Repair entries from the APCB. The retrieved entries are copied to the caller allocated buffer. The caller is responsible for freeing the buffer appropriately after using the data.

Related Definitions**DRRP_REPAIR_ENTRY**

```
typedef struct _DPPRCL_REPAIR_ENTRY {
    :
} DPPRCL_REPAIR_ENTRY;
```

This is the same structure used in [ApcbAddDramPostPkgRepair\(\)](#).

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_NOT_FOND	There were no entries found in the APCB.

3.8.6.10 ApcbClearDramPostPkgRepairEntry()**Summary**

This function clears the DRAM POST Package repair entries in the APCB.

Prototype

```
EFI_STATUS
mApcbClearDramPostPkgRepairEntry (
    IN          AMD_APCB_SERVICE_PROTOCOL  *This
);
```

Parameters

This Pointer to the current instance of the parent protocol.

Description

This function clears the DRAM POST Package repair entries from the APCB.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

3.8.6.11 ApcbRemoveDramPostPkgRepairEntry()**Summary**

This function removes a single DRAM POST Package repair entry from the APCB.

Prototype

```
EFI_STATUS
mApcbRemoveDramPostPkgRepairEntry (
    IN          AMD_APCB_SERVICE_PROTOCOL  *This,
    IN          *Entry
);
```

Parameters

This Pointer to the current instance of the parent protocol.

Entry This is a pointer to a local buffer that contains a copy of the DRAM POST Package repair entry to be removed.

Description

This function removes a DRAM POST Package repair entry from the APCB. The ABL will no longer attempt any repair for this location. This will take effect at the next boot. The structure must be populated so to exactly match the target entry. It is suggested that `ApcbGetDramPostPkgRepair()` be used to populate the structure.

Related Definitions

DRRP_REPAIR_ENTRY

```
typedef struct _DPPRCL_REPAIR_ENTRY {
    :
} DPPRCL_REPAIR_ENTRY;
```

This is the same structure used in [ApcbAddDramPostPkgRepair\(\)](#).

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_NOT_FOND	The indicated entry was not found in the APCB.

3.8.6.12 ApcbAcquireMutex()

Summary

Attempt to lock APCB resources.

Prototype

```
typedef EFI_STATUS (EFIAPI *FP_ACQUIRE_MUTEX) (
    IN      AMD_APCB_SERVICE_PROTOCOL *This
);
```

Parameters

This Pointer to the current protocol instance.

Description

This function is used to acquire the APCB mutex to lock resources prior to any APCB operation sequence. One APCB operation sequence cannot be initiated when the APCB resources are locked by another.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ACCESS_DENIED	APCB cannot be accessed at this time. This routine does not wait for the mutex to become available.

3.8.6.13 ApcbReleaseMutex()

Summary

Unlock the APCB resources.

Prototype

```
typedef EFI_STATUS (EFIAPI *FP_RELEASE_MUTEX) (
    IN      AMD_APCB_SERVICE_PROTOCOL *This
);
```

Parameters

This Pointer to the current protocol instance.

Description

This function is used to release the lock on the APCB resources mutex after an APCB operation sequence is finished. One APCB operation sequence cannot be initiated when the APCB resources are locked by another.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.

3.8.6.14 ApcbGetTokenBool()

Summary

Find a token and retrieve its boolean value.

Prototype

```
typedef EFI_STATUS (EFIAPI *FP_GET_TOKEN_BOOL) (
    IN      AMD_APCB_SERVICE_PROTOCOL  *This,
    OUT     UINT8                      *Purpose,
    IN      UINT32                     TokenId,
    OUT     BOOLEAN                    *bValue
);
```

Parameters

- This** Pointer to the current protocol instance.
- Purpose** Location where the routine will store the purpose level at which the token was found. (see definition at [protocol](#) level).
- TokenId** APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .
- bValue** Location where the routine will store the value of the token.

Description

This function is used to retrieve the boolean value associated with an APCB token in the APCB binary. It starts to search the token in the APCB instance of the highest priority level to that of the lowest, and returns the purpose at which the token is found.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The APCB token was not found.

3.8.6.15 ApcbSetTokenBool()

Summary

Set a boolean token value.

Prototype

```
typedef EFI_STATUS (EFIAPI *FP_SET_TOKEN_BOOL) (
    IN      AMD_APCB_SERVICE_PROTOCOL  *This,
    IN      UINT8                      Purpose,
    IN      UINT32                     TokenId,
    IN      BOOLEAN                    bValue
);
```

Parameters

This	Pointer to the current protocol instance.
Purpose	Purpose level for the token. (see definition at protocol level).
TokenId	APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .
bValue	The boolean value to be associated with the APCB Token ID.

Description

This function is used to associate a boolean value with an APCB token in the APCB instance mapped to the target purpose. If the token/value pair is already present at the target purpose, the associated value is updated. If the token/value pair is not present at the target purpose, a new token/value pair is added. This requires the space allocated for the APCB instance on the flash part to have at least 8 bytes free space.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_OUT_OF_RESOURCES	Not enough space to add the boolean value.

3.8.6.16 ApcbGetToken8()**Summary**

Retrieve the value of a token8

Prototype

```
typedef EFI_STATUS (EFI_API *FP_GET_TOKEN_8) (
    IN      AMD_APCB_SERVICE_PROTOCOL *This,
    OUT     UINT8 *Purpose,
    IN      UINT32 TokenId,
    OUT     UINT8 *Value8
);
```

Parameters

This	Pointer to the current protocol instance.
Purpose	Location where the routine will store the purpose level at which the token was found. (see definition at protocol level).
TokenId	APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .
Value8	Location where the routine will store the value of the token.

Description

This function is used to retrieve the UINT8 value associated with an APCB token in the APCB binary. It starts to search for the token in the APCB instance of the highest priority level to that of the lowest, and returns the purpose at which the token is found.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The APCB token was not found.

3.8.6.17 ApcbSetToken8()

Summary

Establish a value8 token.

Prototype

```
typedef EFI_STATUS (EFI_API *FP_SET_TOKEN_8) (
    IN      AMD_APCB_SERVICE_PROTOCOL  *This,
    IN      UINT8                      Purpose,
    IN      UINT32                     TokenId,
    IN      UINT8                      Value8
);
```

Parameters

This Pointer to the current protocol instance.

Purpose Purpose level for the token. (see definition at [protocol](#) level).

TokenId APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .

Value8 The value to be associated with the APCB Token ID.

Description

This function is used to associate a UINT8 value with an APCB token in the APCB instance mapped to the target purpose. If the token/value pair is already present in the target purpose, the associated value is updated. If the token/value pair is not present in the target purpose, a new token/value pair is added. This requires the space allocated for the APCB instance on the flash part to have at least 8 bytes free space.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_OUT_OF_RESOURCES	Not enough space to add the token value.

3.8.6.18 ApcbGetToken16()

Summary

Retrieve the value of a token16

Prototype

```
typedef EFI_STATUS (EFI_API *FP_GET_TOKEN_16) (
    IN      AMD_APCB_SERVICE_PROTOCOL  *This,
    OUT     UINT8                      *Purpose,
    IN      UINT32                     TokenId,
    OUT     UINT8                      *Value16
);
```

Parameters

This Pointer to the current protocol instance.

Purpose Location where the routine will store the purpose level at which the token was found. (see definition at [protocol](#) level).

TokenId APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .

Value16 Location where the routine will store the value of the token.

Description

This function is used to retrieve the UINT16 value associated with an APCB token in the APCB binary. It starts to search for the token in the APCB instance of the highest priority level to that of the lowest, and returns the purpose at which the token is found.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The APCB token was not found.

3.8.6.19 ApcbSetToken16()

Summary

Establish a value16 token.

Prototype

```
typedef EFI_STATUS (EFIAPI *FP_SET_TOKEN_16) (
    IN      AMD_APCB_SERVICE_PROTOCOL  *This,
    IN      UINT8                      Purpose,
    IN      UINT32                     TokenId,
    IN      UINT8                      Value16
);
```

Parameters

This Pointer to the current protocol instance.

Purpose Purpose level for the token. (see definition at [protocol](#) level).

TokenId APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .

Value16 The value to be associated with the APCB Token ID.

Description

This function is used to associate a UINT16 value with an APCB token in the APCB instance mapped to the target purpose. If the token/value pair is already present in the target purpose, the associated value is updated. If the token/value pair is not present in the target prurpose, a new token/value pair is added. This requires the space allocated for the APCB instance on the flash part to have at least 8 bytes free space.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_OUT_OF_RESOURCES	Not enough space to add the token value.

3.8.6.20 ApcbGetToken32()

Summary

Retrieve the value of a token32

Prototype

```
typedef EFI_STATUS (EFIAPI *FP_GET_TOKEN_32) (
```



```

IN    AMD_APCB_SERVICE_PROTOCOL    *This,
OUT   UINT8                        *Purpose,
IN    UINT32                       TokenId,
OUT   UINT8                        *Value32
);

```

Parameters

This Pointer to the current protocol instance.

Purpose Location where the routine will store the purpose level at which the token was found. (see definition at [protocol](#) level).

TokenId APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .

Value32 Location where the routine will store the value of the token.

Description

This function is used to retrieve the UINT32 value associated with an APCB token in the APCB binary. It starts to search for the token in the APCB instance of the highest priority level to that of the lowest, and returns the purpose at which the token is found.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The APCB token was not found.

3.8.6.21 ApcbSetToken32()**Summary**

Establish a value32 token.

Prototype

```

typedef EFI_STATUS (EFI_API *FP_SET_TOKEN_32) (
IN    AMD_APCB_SERVICE_PROTOCOL    *This,
IN    UINT8                        Purpose,
IN    UINT32                       TokenId,
IN    UINT8                        Value32
);

```

Parameters

This Pointer to the current protocol instance.

Purpose Purpose level for the token. (see definition at [protocol](#) level).

TokenId APCB token id value. This can be located in file: AgesaModulePkg\Include\ApcbCommon.h .

Value32 The value to be associated with the APCB Token ID.

Description

This function is used to associate a UINT32 value with an APCB token in the APCB instance mapped to the target purpose. If the token/value pair is already present in the target purpose, the associated value is updated. If the token/value pair is not present in the target prurpose, a new token/value pair is added. This requires the space allocated for the APCB instance on the flash part to have at least 8 bytes free space.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_OUT_OF_RESOURCES	Not enough space to add the token value.

3.8.6.22 ApcbGetType()

Summary

Locate a type in the APCB.

Prototype

```
typedef EFI_STATUS (EFI_API *FP_GET_TYPE) (
    IN  AMD_APCB_SERVICE_PROTOCOL *This,
    OUT UINT8 *Purpose,
    IN  UINT16 GroupId,
    IN  UINT16 TypeId,
    OUT UINT8 **DataBuf,
    OUT UINT32 *DataSize
);
```

Parameters

This	Pointer to the current protocol instance.
Purpose	Location where the routine will store the purpose level at which the type was found. (see definition at protocol level).
GroupId	Defines which component collection the Type belongs to. See Managing Data for more information.
TypeId	Defines the target data element. See Managing Data for more information.
DataBuf	Location where the routine will store the pointer to the target type data.
DataSize	Location where the routine will store the size of the target type data.

Description

This function is used to find the data (header not included) of a target type in the APCB binary. It starts to search the type in the APCB instance of the highest priority level to that of the lowest, and returns the purpose at which the token is found. The retrieved pointer references the buffer within the shadow APCB instance in DRAM.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The target type data was not found.

3.8.6.23 ApcbSetType()

Summary

Establish a type entry in the APCB.

Prototype

```
typedef EFI_STATUS (EFI_API *FP_SET_TYPE) (
    IN  AMD_APCB_SERVICE_PROTOCOL *This,
    IN  UINT8 Purpose,
    IN  UINT16 GroupId,
    IN  UINT16 TypeId,
    IN  UINT16 InstanceId,
    IN  UINT8 *DataBuf,
    IN  UINT32 DataSize
);
```

);

Parameters

This	Pointer to the current protocol instance.
Purpose	Location where the routine will store the purpose level at which the type was found. (see definition at protocol level).
GroupId	Defines which component collection the Type belongs. See Managing Data for more information.
TypeId	Defines the target data element. See Managing Data for more information.
InstanceId	The instance ID of the target type. Typically this will be 0x00, but there are certain cases where an element could have multiple instances. These will be noted in the specific data group-type. An example would be a memory parameter that may have one value for vendor A and another value for vendor B.
DataBuf	Location where the routine will find the target type data.
DataSize	The size of the target type data to establish.

Description

This function is used to set the data (header not included) of a target group-type-instance in the APCB mapped to the target purpose. The type header must be already present in the APCB before the service is called. The original type data will be removed and the new type data pointed to by DataBuf with the size of DataSize will be saved into the APCB shadow instance in DRAM. To purge the data of a target type, a 0 has to be passed as DataSize. A NULL can be optionally passed as DataBuf.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	The target type data was not found.

3.8.6.24 ApcbPurgeAllTokens()**Summary**

Remove APCB token/value pair entries

Prototype

```
typedef EFI_STATUS (EFI_API *FP_PURGE_ALL_TOKENS) (
    IN      AMD_APCB_SERVICE_PROTOCOL *This,
    IN      UINT8                      Purpose
);
```

Parameters

This	Pointer to the current protocol instance.
Purpose	Purpose of the current operation (see definition at protocol level).

Description

This function is used to remove all token/value pairs of the target purpose. Once this is done, no token at this purpose will be seen in APCB by ABL or any other components.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	Not all tokens have been purged.

3.8.6.25 ApcbPurgeAllTypes()

Summary

Remove APCB type entries.

Prototype

```
typedef EFI_STATUS (EFI_API *FP_PURGE_ALL_TYPES) (
    IN      AMD_APCB_SERVICE_PROTOCOL *This,
    IN      UINT8                      Purpose
);
```

Parameters

This Pointer to the current protocol instance.

Purpose Purpose of the current operation (see definition at [protocol](#) level).

Description

This function is used to remove all type data of the target purpose. The type headers are preserved with the size field updated.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_NOT_FOUND	Not all types have been purged.

3.8.6.26 Usage Example

Summary

One of the most common APCB operations is to get/set an APCB token, which is defined as a 32-bit hash ID in `ApcbV3TokenUid.h`. These are associated with a value of `BOOLEAN`, `UINT8`, `UINT16` or `UINT32`. These elementary data types can be easily manipulated with the following services:

- `ApcbGetTokenBool () / ApcbSetTokenBool ()`
- `ApcbGetToken8 () / ApcbSetToken8 ()`
- `ApcbGetToken16 () / ApcbSetToken16 ()`
- `ApcbGetToken32 () / ApcbSetToken32 ()`

A common form of revising an APCB token value looks this way:

```
If (Protocol.Version == APCB_PROTOCOL_VERSION_3_0)
{
    //
    // Make sure this is an APCB 3.0 Protocol instance
    //
    ApcbAcquireMutex (&Protocol);
    ApcbGetToken16 (&Protocol, &Purpose, TokenId, &Value16);
    if (Value16 needs to be modified) {
        ApcbSetToken16 (&Protocol, Purpose, TokenId, NewValue16);
    }
    ApcbFlushData (&Protocol);
    ApcbReleaseMutex (&Protocol);
}
```

```
}

```

ApcbGetToken16() returns the highest level purpose in which the token was found. ApcbSetToken16() takes purpose as an argument to indicate at which level the user would like the set operation to occur.

More complicated cases involve manipulating a custom APCB data type entry. To identify such an entry, the user needs to specify GroupId, TypeId and InstanceId instead of a simple APCB token UID, in ApcbGetType()/ApcbSetType(). The retrieved data buffer points directly to the entry data within the shadow copy of an APCB binary instance. To change the data of the entry the user needs to copy the returned buffer to a secondary one and then pass the modified secondary buffer to ApcbSetType() to replace the original data entry entirely. Likewise, ApcbFlushData() has to be invoked to write the shadow copies back to the flash part.

3.8.7 PSP Mailbox Functions

The PSP MailBox library is provided to the platform to support the use case of making special services available to the Platform SMM driver. These library routines must be integrated into, and be called from the platform SMM driver. See files:

- AgesaModulePkg\Include\Library\AmdPspRomArmorLib.h
- AgesaModulePkg\Library\AmdPspRomArmorLib\AmdPspRomArmorLib.c

At present, the AMD ROM Armor feature is available in F17M30.

Security Enhancements - AMD ROM Armor

AMD ROM Armor is a security feature that offers improved protections for the SPI ROM. AMD ROM Armor is an opt-in feature. AMD is deprecating the chipset based hardware ROM protection features and recommends AMD ROM Armor to be used by BIOS to protect the ROM.

When the BIOS begins executing, x86 Ring0 code will have full access to the SPI Controller. The BIOS can then choose to activate AMD ROM Armor to lockdown the SPI Controller so that only the BIOS's SMM handler will be able to interact with the SPI Controller over a secure communication channel with the PSP. When AMD ROM Armor is enabled, Ring0 and DMA will still have read access to the ROM via the direct mapped MMIO interface (traditionally found at FF00_0000 to FFFF_FFFF), but writes will be prohibited.

The AMD Rom Armor PSP MailBox library is provided to the platform to create a secure communication channel with the PSP for the AMD Rom Armor feature.

Platform Implementation Note

Newer AMD processors have implemented an additional ROM access region known as 'ROM3'. This special address space can support larger ROMs (up to 64 MB). By default (power-on reset) this address space is set to 0xFD_0000_0000 – 0xFD_03FF_FFFF (just under 1TB). Systems with very large memory spaces (>= 1TB) will encounter an address space conflict when the ROM Armor feature is enabled. For prevention of the conflict, an APCB Token was introduced to designate the address space location for the ROM3 region. The ROM3 region should be placed outside the locations containing memory.

Please see the definition for [APCB_TOKEN_UID_FCH_ROM3_BASE_HIGH](#)

3.8.7.1 PspEnterSmmOnlyMode()

Summary

Secure the SPI controller.

Prototype

```
EFI_STATUS
EFIAPI
```

```
PspEnterSmmOnlyMode (
    IN      SPI_COMMUNICATION_BUFFER  *SpiCommunicationBuffer
);
```

Parameters

SpiCommunicationBuffer

This is a pointer to the SMM->PSP communication buffer for sending SPI commands. The SMM->PSP communication buffer is allocated by the platform BIOS in the TSEG (SMM) space and should not be deallocated once created.

Description

This command is a request to the PSP to secure the SPI Controller, activating the AMD ROM Armor feature. This command can only be executed once. Once AMD ROM Armor is activated it cannot be disabled. Only a system reset will return AMD ROM Armor to its disabled state.

PSP will secure the SPI Controller so that only the PSP can touch it and establishes a SMM->PSP communication buffer for sending SPI commands.

The command must be executed within System Management Mode (SMM) before the EFI “Ready To Boot” event. It is the responsibility of the platform to create SMM->PSP communication buffer in the TSEG region.

Related Definitions

SPI_COMMUNICATION_BUFFER

```
#define PSP_MAX_SPI_CMD_SUPPORT      (4)
typedef struct {
    UINT8      ReadyToRun;
    UINT8      CommandCount;
    SPI_COMMUNICATION_RESULT SpiCommunicationResult;
    SPI_COMMAND SpiCommand[PSP_MAX_SPI_CMD_SUPPORT];
} SPI_COMMUNICATION_BUFFER;
```

ReadyToRun

Set to FALSE by x86 while the buffer is being constructed. Set to TRUE by x86 as the last step in building the communication buffer, just before x86 rings the PSP doorbell. Set to FALSE by PSP after the PSP copies the buffer from DRAM to private SRAM.

CommandCount

Number of commands to execute, Valid Values 1-4.

SpiCommunicationResult

Set to zero by x86 when the buffer is built. Atomically set to a non-zero value by the PSP to indicate the PSP has finished processing the requests in the communication buffer. The specific value written by the PSP provides per command results.

SpiCommand

An array of structures defining the SPI commands.

SPI_COMMUNICATION_RESULT

```
typedef union {
    struct {
        UINT16 Command0Result:4;
        UINT16 Command1Result:4;
        UINT16 Command2Result:4;
        UINT16 Command3Result:4;
    };
};
```

```

    } Field;
    UINT16 Value;
} SPI_COMMUNICATION_RESULT;

```

Parameters

Command0Result The result of Command[0..3]
Command1Result
Command2Result
Command3Result

Result values with special meaning:

- 0x0000 = (written by x86) PSP has not finished handling the request
- 0x1000 = PSP determined the request is malformed (invalid CommandCount, chipselect, BytesToRx/Tx, etc)
- 0x2000, 0x3000, 0x4000, ... , 0xF000 = Reserved for future errors

The result of Command values:

- 0 = Command not examined/processed
- 1 = Command completed successfully
- 2 = Execution Error (i.e. timeout)
- 3 = Command not allowed by Whitelist
- 4 = Command malformed
- 5-15 = reserved for future use

Examples of Generic Results:

- 0x0000 - PSP has not finished the request
- 0x0001 - PSP ran Command0 successfully, and is now idle
- 0x0111 - PSP ran Command0/1/2 successfully and is now idle
- 0x0031 - PSP ran Command0, but Command1 was blocked by whitelist

SPI_COMMAMD

```

#define PSP_MAX_SPI_DATA_BUFFER_SIZE    (72)
typedef struct {
    UINT8    ChipSelect;
    UINT8    Frequency;
    UINT8    BytesToTx;
    UINT8    BytesToRx;
    UINT8    OpCode;
    UINT8    Reserved[3];
    UINT8    Buffer[PSP_MAX_SPI_DATA_BUFFER_SIZE];
} SPI_COMMAND;

```

Parameters

ChipSelect Slave device select.

- 1 = CS1,
- 2 = CS2,
- all other values illegal.

Frequency Serial Peripheral Interface (SPI) bus speed.

- 0 = 66.66MHz,

- 1 = 33.33MHz,
- 2 = 22.22MHz,
- 3 = 16.66MHz,
- 4 = 100Mhz,
- 5 = 800KHz,
- all others illegal

BytesToTx

Bytes to Transmit. Valid range is 0-72 and does not include the SPI Opcode byte, but does include the address, dummy bytes, and data.

BytesToRx

Bytes to Receive from device, BytesToTx + BytesToRx <=72.

OpCode

The SPI Command OpCode (the first byte sent by the SPI controller).

Reserved

Reserved for future expansion.

Buffer

The remaining 0-72 bytes sent/received by the SPI controller. The SPI Controller will:

1. Assert the ChipSelect
2. Send the one byte OpCode
3. Send Buffer[0] to Buffer[BytesToTx-1] to the SPI device
4. Read BytesToRx bytes from the device into Buffer[BytesToTx] to Buffer[BytesToTx+BytesToRx-1]
5. Deassert the ChipSelect line.

SPI ROM Commands that include a target address send the address immediately after the OpCode (i.e. Buffer[0..2] or Buffer[0..3] depending if 24 or 32bit addresses are associated with the OpCode).

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_DEVICE_ERROR	The command not accepted by PSP.
EFI_UNSUPPORTED	Unsupported command.

3.8.7.2 EnforceWhitelist()**Summary**

Establish a list of permitted SPI bus commands.

Prototype

```
EFI_STATUS
EFIAPI
PspEnforceWhitelist (
    IN      SPI_WHITE_LIST  *SpiWhiteList
);
```

Parameters**SpiWhiteList**

This is a pointer to the SPI command white list buffer.

Description

The SPI whitelist consists of approved SPI commands, and address/data ranges for those commands. If the SMM handler requests a command not listed on the allowed list, the request will be rejected. Once an whitelist is set, it

cannot be modified or disabled. Only a system reset will return AMD ROM Armor to its disabled state. This command can only be executed once and only after PspEnterSmmOnlyMode() is called and before the EFI “Ready To Boot” event occurs. It is the responsibility of the platform to create SpiWhiteList structure in the TSEG (SMM) region of memory. The space can be released after the command has executed.

Related Definitions

SPI_WHITE_LIST

```
#define PSP_MAX_WHITE_LIST_CMD_NUM      (32)
#define PSP_MAX_WHITE_LIST_REGION_NUM   (16)
typedef struct {
    UINT8 AllowedCmdCount;
    UINT8 AllowedRegionCount;
    WHITE_LIST_ALLOWED_COMMAND WhiteList_AllowedCommands[PSP_MAX_WHITE_LIST_CMD_NUM];
    WHITE_LIST_ALLOWED_REGION WhiteList_AllowedRegions[PSP_MAX_WHITE_LIST_REGION_NUM];
} SPI_WHITE_LIST;
```

Parameters

AllowedCmdCount	Number of allowed SPI commands to execute, Valid values 0-32, and 255. The value 255 indicates that all SPI commands will be accepted (i.e. the whitelist is disabled).
AllowedRegionCount	Number of allowed access address regions. Valid values are 0-16.
WhiteList_AllowedCommands	An array of structures defining the allowed SPI commands to execute.
WhiteList_AllowedRegions	An array of structures defining allowed access address region to execute.

WHITE_LIST_ALLOWED_REGION

```
typedef struct {
    UINT32 StartAddress;
    UINT32 EndAddress;
} WHITE_LIST_ALLOWED_REGION;
```

Parameters

StartAddress	LSB must be 0x00, bit31 identifies a chipselect: 0=CS1, 1=CS2.
EndAddress	LSB must be 0xFF, StartAddress must be less than EndAddress.

WHITE_LIST_ALLOWED_COMMAND

```
typedef struct {
    UINT8 ChipSelect;
    UINT8 Frequency;
    UINT8 OpCode;
    UINT8 MinTx, MaxTx;
    UINT8 MinRx, MaxRx;
    UINT8 AddrChkMethod;
    UINT32 ImpactSize;
```

```
} WHITE_LIST_ALLOWED_COMMAND;
```

Parameters

ChipSelect

Slave device select.

- 0 = Allowed on all chip selects,
- 1 = CS1,
- 2 = CS2, all other values illegal.

Frequency

Serial Peripheral Interface (SPI) bus speed.

- 0=66.66MHz,
- 1=33.33MHz,
- 2=22.22MHz,
- 3=16.66MHz,
- 4=100Mhz,
- 5=800KHz, all others illegal

OpCode

An allowed commands opcode.

MinTx, MaxTx

The range of allowed transmit byte counts for this command (does not include opcode).

MinRx, MaxRx

The range of allowed Rx byte counts.

AddrChkMethod

- 0=No address verification performed
- 1=Treat Buffer[0-2] as a 24-bit address, and verify the entire ImpactZone of the command falls within one of the allowed regions
- 2=Treat Buffer[0-3] as a 32-bit address, and verify the entire ImpactZone of the command falls within one of the allowed regions

ImpactSize

The Impact Zone is the naturally aligned power of two sized block of addresses that may be impacted by a given SPI Command. For example, a sector erase command targeted at an address within a 64K block will impact every byte within that 64K block. Likewise a page program SPI command (i.e. write) may impact many bytes within the targeted 256/512 byte page due to page wrap-around, but no bytes outside the page. The ImpactSize field specifies the power of two size of the ImpactZone for this command. If VerifyAddress is zero (no checking) this field must also be zero, otherwise this field must be a power of two between 256 and 64MB (256, 512, ..., 67108864). NOTE: When setting this field, carefully examine your device's datasheet.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_DEVICE_ERROR	The command was not accepted by PSP.
EFI_UNSUPPORTED	Unsupported command.

3.8.7.3 PspExecuteSpiCommand()

Summary

Send request to PSP to execute an SPI command.

Prototype

```
EFI_STATUS
EFIAPI
PspExecuteSpiCommand();
```

Description

PspExecuteSpiCommand() alerts the PSP that there is a SPI request waiting in the SMM->PSP communication buffer. The SMM->PSP buffer is established when PspEnterSmmOnlyMode() was called. Software should setup the SPI commands in the communication buffer, zero the SpiCommunicationResult field, set the buffers ReadyToRun flag, then call PspExecuteSpiCommand() to tell the PSP to process the commands in the buffer. Software may then retrieve the results of the SPI commands when the PSP updates the SpiCommunicationResult field.

Command operations will have completed when this functions returns and the TSEG buffer will have been updated with the command results.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_UNSUPPORTED	The command is not supported. Make sure the SMM Only Mode has been established prior to this call.
EFI_DEVICE_ERROR	The command was not accepted by the PSP. Check SMM Only mode status and that the command is valid.

3.8.8 AMD_PSP_FIRMWARE_VERSION_PROTOCOL [F19M00]

Summary

This Protocol provides functions to retrieve versions labels for firmware entities.

GUID

```
gAmdPspFirmwareVersionProtocolGuid = {0x925b78d4, 0x6eb5, 0x4d4b,
                                         {0x82, 0xa0, 0x7e, 0xd4, 0xa4, 0x31, 0x16, 0xa0}}
```

Protocol Interface Structure

```
typedef struct _AMD_PSP_FIRMWARE_VERSION_PROTOCOL {
    GET_FIRMWARE_VERSION    GetFirmwareVersion;
} AMD_PSP_FIRMWARE_VERSION_PROTOCOL;
```

Members

GetFirmwareVersion Function to firmware versions.

3.8.8.1 GetFirmwareVersion()

Summary

This function can be used to retrieve the version title of various firmware entities.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *GET_FIRMWARE_VERSION) (
```

```

        IN      AMD_PSP_FIRMWARE_VERSION_PROTOCOL *This,
        IN      AMD_PSP_FIRMWARE_TYPE             FirmwareType,
        IN      AMD_PSP_FIRMWARE_LEVEL             FirmwareLevel,
        IN OUT  UINT32                             *FirmwareVersion
    );

```

Parameters

This

A pointer to this instance of the protocol.

FirmwareType

Specifies the firmware entity from which to retrieve its version information. AMD_PSP_FIRMWARE_TYPE is an enum type that defines which firmware entities are available.

- FirmwareTypePspBootLoader
- FirmwareTypePspRecoveryBootLoader
- FirmwareTypeSmu
- FirmwareTypeAegsaBootLoader
- FirmwareTypeApcb
- FirmwareTypeApob
- FirmwareTypeAppb

FirmwareLevel

Specifies the PSP directory level of the target binary. Refer to the [PSP BIOS Arch](#) spec for details. AMD_PSP_FIRMWARE_LEVEL is an enum type that defines which directory levels are available.

- FirmwareLevel1 - the PSP directory
- FirmwareLevel2 - the BIOS directory

Description

There are several different types of firmware entities in the system in different levels of PSP directory structure. This protocol can retrieve the version information of each firmware entity.

Status Codes Returned

status code	meaning
UEFI_SUCCESS	The function has completed successfully.
EFI_INVALID_PARAMETER	One of the parameters is not valid.
EFI_NOT_FOUND	This routine could not find the indicated firmware entity in the system.

3.9 RAS (Reliability, Availability and Serviceability)

Summary

This driver provides the configuration and services to operate a RAS feature set.

Description

This Driver is the entry point of the AMD RAS feature set. It performs the early configuration, allocates resources and installs the component protocols listed below.

The RAS sub-system uses features provided by many levels in the computer, including silicon, firmware, BIOS and the OS. Please make use of the references for the applicable Family-Model to gain a broader understanding of how these systems function: [PPR](#) and [RAS](#).

Related Definitions - Common Data Structure Defs

DIMM_INFO

```
typedef struct _DIMM_INFO {
    UINT8    ChipSelect;
    UINT8    Bank;
    UINT32    Row;
    UINT16    Column;
    UINT8    RankMul;
    UINT8    DimmId;
} DIMM_INFO;
```

ChipSelect	Chip select number, relative to the channel, that contains the target address.
Bank	Bank number, relative to the Chip Select, that contains the target address. Zero is the first bank.
Row	Row number within the bank of the row that contains the target address. Zero is the first row.
Column	Column number, within the bank, of the column that contains the target address. Zero is the first column.
RankMul	The Rank Multiplex that contains the address.
DimmId	Reserved. Not used at this time.

NORMALIZED_ADDRESS

```
typedef struct {
    UINT64    normalizedAddr;
    UINT8    normalizedSocketId;
    UINT8    normalizedDieId;
    UINT8    normalizedChannelId;
    UINT8    reserved;
} NORMALIZED_ADDRESS;
```

The Universal Memory controller (UMC) MCA registers use addresses that are relative to the memory controller. The first address they control is Local 0x00 regardless of where in the system memory map that memory is placed. This is the 'normalized' or UMC local relative address as seen in the UMC MCA ADDR MSRs.

normalizedAddr	UMC local Address.
normalizedSocketId	The socket on which the UMC is located.
normalizedDieId	The die on which the UMC is located.
normalizedChannelId	The channel which the UMC is controlling.

3.9.1 RAS Configuration Items

Note: the controls listed here may have further usage examples shown in the [RAS](#) specification. Please refer to that document for further details.

3.9.1.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

3.9.1.2 RAS Enablement Category

- **PcdAmdNbioRASControl**

Describes if the controls for the RAS features for PCIe® (AER) and SATA are to be enabled.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - NBIO RAS features are enabled.
- FALSE - This option is turned off

- **PcdCfgIommuSupport**

Selects the state of the IOMMU support firmware.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdCfgAEREnable**

Selects the state of the PCIe® Advanced Error Reporting (AER) support.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdPcieEcrcEnablement**

Specifies the state of the NBIO ECRC feature.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active.
- FALSE - This option is turned off.

- **PcdPcieEcrcSeverityFatal**

Specifies the severity of an ECRC error.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Errors are fatal.
- FALSE - Error are not fatal.

- **PcdAmdNbioRASControlV2 [F17M30][F19M00]**

This value is the master control for NBIO RAS support.

Permitted Choices: (Type: Value)(Default: [F17M30]:0x0, [F19M00]:0x01)

- 0 – Disabled
- 1 – MCA reporting

- 2 – Legacy Mode
- **PcdAmdEdpcEnable [F17M30]**
This value enables EDPC.
Permitted Choices: (Type: Value)(Default: 0x0)
 - 0 – EDPC is disabled.
 - 1 – EDPC is enabled.
- **PcdAmdPcieSyncFloodOnFatal [F17M30]**
This control selects the response of the PCIe® subsystem to fatal errors.
Permitted Choices: (Type: Value)(Default: TRUE)
 - FALSE – No sync flood is generated.
 - TRUE – Sync flood is generated on fatal errors.

3.9.1.3 RAS Platform Category

- **PcdMceSwSmiData**
When using Platform First Error Handling (PFEH) operation by the BIOS, this is the value used by the MCE sub-system to write to the software SMI command port to trigger a software SMI. The default is generally best, but it is allowed to be changed since this value could conflict with other handlers in the host system.
Permitted Choices: (Type: Value)(Default: 0x80)
- **PcdCfgNbIoSsid**
This 32-bit value defines the subsystem ID assigned to the NTB CCP controller.
Permitted Choices: (Type: Value)(Default: 0x0000)
 - 0x0000 - Use the default power-up setting.
 - !=0 - Value to set as the SSID.
- **PcdCfgIoMmuSsid**
This 32-bit value defines the subsystem ID assigned to the IOMMU controller.
Permitted Choices: (Type: Value)(Default: 0x0000)
 - 0x0000 - Use the default power-up setting.
 - !=0 - Value to set as the SSID.
- **PcdCfgPspccpSsid**
This 32-bit value defines the subsystem ID assigned to the PSP CCP controller.
Permitted Choices: (Type: Value)(Default: 0x0000)
 - 0x0000 - Use the default power-up setting.
 - !=0 - Value to set as the SSID.
- **PcdCfgXgbeSsid**
This 32-bit value defines the subsystem ID assigned to the internal XGBE controller.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0x0000 - Use the default power-up setting.
- !=0 - Value to set as the SSID.

- **PcdCfgNbifF0Ssid**

This 32-bit value defines the subsystem ID assigned to the NBIF0 controller.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0x0000 - Use the default power-up setting.
- !=0 - Value to set as the SSID.

- **PcdCfgNbifRCSsid**

This 32-bit value defines the subsystem ID assigned to the Root Controllers.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0x0000 - Use the default power-up setting.
- !=0 - Value to set as the SSID.

3.9.1.4 RAS Performance Category

- **PcdMcaErrThreshEn**

This value controls the MCA error thresholding counter to all MCA banks.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Enable the MCA Error Thresholding counter.
- FALSE - The counter is turned off, MCE interrupts will be raised on every occurrence.

- **PcdMcaErrThreshCount**

The value for setting the MCA Error Thresholding counter.

Permitted Choices: (Type: Value)(Default: 0x0FF5)

- **PcdNbioCorrectedErrThreshEn**

NBIO corrected error thresholding counter control.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Enable the NBIO Corrected Error Thresholding counter.
- FALSE - The counter is turned off.

- **PcdNbioCorrectedErrThreshCount**

The value for setting the NBIO corrected error thresholding counter.

Permitted Choices: (Type: Value)(Default: 0x0FF5)

- **PcdNbioDeferredErrThreshEn**

NBIO deferred error thresholding counter control.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Enable the NBIO Deferred Error Thresholding counter.
- FALSE - The counter is turned off.

- **PcdNbioDeferredErrThreshCount**

The value for setting the NBIO deferred error thresholding counter.

Permitted Choices: (Type: Value)(Default: 0x0FF5)

- **PcdEgressPoisonSeverityHi**

PcdEgressPoisonSeverityLo

These two values set the value for EGRESS POISON SEVERITY configuration. Each bit is associated with a logical IOHC Egress port such as a PCIe® RC. For each bit set to 1, the poison severity associated with that port is HIGH severity. For each bit set to 0 it is considered LOW severity. See the PPR section on "Data Poisoning". Each value must be a 32-bit number.

Permitted Choices: (Type: UINT32 Value) (Default: Hi 0x00030011, Lo 0x00000004)

- **PcdAmdMaskNbioSyncFlood**

This value may be used to mask SyncFlood caused by NBIO RAS options. When set to TRUE SyncFlood from NBIO is masked off. When set to FALSE, NBIO is capable of generating SyncFlood.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Prevent RAS SyncFlood
- FALSE - Allow SyncFlood errors

- **PcdSyncFloodToApmI**

This value is used to enable SyncFlood reporting to APML. When set to TRUE SyncFlood will be reported to APML. When set to FALSE that reporting will be disabled.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Use APML reporting
- FALSE - Do not report to APML

- **PcdAmdNbioEgressPoisonMaskHi**

PcdAmdNbioEgressPoisonMaskLo

These values set the enable mask for masking of errors logged in EGRESS_POISON_STATUS. For each bit set to 1, errors are masked off. For each bit set to 0, errors trigger response actions. See the PPR section on "Data Poisoning". Each value must be a 32-bit number.

Permitted Choices: (Type: UINT32 Value) (Default: Hi 0xFFFCFFFF, Lo 0xFFFFFFF5)

- **PcdAmdNbioRASUcpMaskHi**

PcdAmdNbioRASUcpMaskLo

These set the enable mask for masking of uncorrectable parity errors on internal arrays. For each bit set to 1, a system fatal error event is triggered for UCP errors on arrays associated with that egress port. For each bit set to 0, errors are masked off. See the PPR section on "Data Poisoning". Each value must be a 32-bit number.

Permitted Choices: (Type: UINT32 Value) (Default: Hi 0x00030000, Lo 0x00000004)

• PcdSyshubWdtTimerInterval

This value specifies the timer interval of the SYSHUB Watchdog timer.

Value is in milliseconds and must be within the range: 1-65535.

Permitted Choices: (Type: Value)(Default: 2600)

• PcdSlinkConvertReadResponseErrorsToOkay

This value specifies whether SLINK read response errors are converted to an “Okay” response. See the PPR section on “Data Poisoning”.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - read response errors are converted to “Okay” responses with data of all FFs.
- FALSE - read response errors are not converted

• PcdSlinkWriteResponseErrorHandling

This value specifies whether SLINK write response errors are converted to an “Okay” response. See the PPR section on “Data Poisoning”.

Permitted Choices: (Type: List)(Default: 0)

- 0, write response errors will be logged in the MCA.
- 1, write response errors will trigger an MCOMMIT error.
- 2, write response errors are converted to “Okay” responses.

• PcdAmdLogPoisonDataFromSLink

This value specifies whether poison data propagated from SLINK will generate a deferred error. See the PPR section on “Data Poisoning”.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - deferred errors are enabled.
- FALSE - errors are not generated.

• PcdAmdPcieAerReportMechanism [F17M30][F19M00]

This value selects the method of reporting AER errors from PCI Express.

Permitted Choices: (Type: List)(Default: [F17M30]:0, [F19M00]:1)

- 0 - indicates that the hardware will report the error through PFEH-based Firmware First reporting.
- 1 - allows “OS First” handling of the errors through generation of a system control interrupt (SCI).
- 2 - provides for “Firmware First” handling of errors through generation of a system management interrupt (SMI).

3.9.2 AMD_RAS_APEI_PROTOCOL**Summary**

This protocol provides services for creating the ACPI APEI table.

GUID

```
gAmdRasApeiProtocolGuid = {0xe9dbcc60, 0x8f93, 0x47ed,
```

{0x84, 0x78, 0x46, 0x78, 0xf1, 0x9f, 0x73, 0x4a}}

Protocol Interface Structure

```
typedef struct _AMD_RAS_APEI_PROTOCOL {
    MCA_ERROR_ADDR_TRANSLATE    McaErrorAddrTranslate;
    TRANSLATE_SYSADDR_TO_CS     TranslateSysAddrToCS;
    AMD_ADD_BOOT_ERROR_RECORD_ENTRY AddBootErrorRecordEntry;
    ADD_HEST_ERROR_SOURCE_ENTRY AddHestErrorSourceEntry;
} AMD_RAS_APEI_PROTOCOL;
```

Members

<i>McaErrorAddrTranslate</i>	Function to convert an MCA_ADDR_UMC address to system address and DIMM specific information.
<i>TranslateSysAddrToCS</i>	Function for converting a physical address to DIMM specific information.
<i>AddBootErrorRecordEntry</i>	Function to add a boot error record entry into the ACPI BERT table.
<i>AddHestErrorSourceEntry</i>	Function to add an error source record entry into the ACPI HEST table.

Description This protocol provides services for APEI table creation and RAS feature boot services. It depends upon the MP service protocol.

3.9.2.1 MCA_ERROR_ADDR_TRANSLATE()

Summary

Converts an MCA address MSR value into a chip/device location.

Prototype

```
typedef EFI_STATUS (EFI_API *MCA_ERROR_ADDR_TRANSLATE) (
    IN        NORMALIZED_ADDRESS *NormalizedAddress,
    OUT       UINT64               *SystemMemoryAddress,
    OUT       DIMM_INFO            *DimmInfo
);
```

Parameters

<i>NormalizedAddress</i>	This is the 'normalized' or UMC local relative address as seen in the UMC MCA ADDR MSRs.
<i>SystemMemoryAddress</i>	This return value indicates the physical system address related to this localized address.
<i>DimmInfo</i>	Returned structure that contains the DIMM information related to the given address.

Description

This function converts the given valid error address from an MCA ADDR MSR (UMC) register to system address with DIMM specific information.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.2.2 TRANSLATE_SYSADDR_TO_CS()

Summary

Converts a system physical address into a chip/device location.

Prototype

```
typedef EFI_STATUS (EFI_API *TRANSLATE_SYSADDR_TO_CS) (
    IN      UINT64      *SystemMemoryAddress,
    OUT     NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     DIMM_INFO    *DimmInfo
);
```

Parameters

SystemMemoryAddress

System Physical address to be translated.

NormalizedAddress

The function will place a normalized address into this structure which represents the location relative this the hosting memory controller.

DimmInfo

Pointer to a DIMM_INFO structure buffer (see the common structure definitions for the driver).

Description

This function converts the given system memory address into a normalized address indicating the SocketId, DieId, channelId for the memory controller that is responsible for the specified system address and also includes information to indicate the DIMM device. This function helps customer identify the physical location (device) that contains the address.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.2.3 AMD_ADD_BOOT_ERROR_RECORD_ENTRY()

Summary

Adds an entry to the ACPI BERT table.

Prototype

```
typedef EFI_STATUS (EFI_API *AMD_ADD_BOOT_ERROR_RECORD_ENTRY) (
    IN      UINT8      *ErrorRecord,
    IN      UINT32     RecordLen,
    IN      UINT8      ErrorType,
    IN      UINT8      SeverityType
);
```

Parameters

***ErrorRecord**

Error Record buffer. This record is defined in [ACPI](#) reference specification as Generic Error Data Entry and in the UEFI reference specification Appendix for Common Platform Error Record.

RecordLen

Error Record buffer length

ErrorType

Error type of the record

SeverityType Severity type of the record

Description

Use this function to add an error into the ACPI BERT table.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.
EFI_NOT_FOUND	Function failure. The ACPI BERT table could not be found.

3.9.2.4 ADD_HEST_ERROR_SOURCE_ENTRY()

Summary

Adds an entry to the ACPI HEST error source table.

Prototype

```
typedef EFI_STATUS (EFI_API *ADD_HEST_ERROR_SOURCE_ENTRY) (
    IN      UINT8    *pErrorRecord,
    IN      UINT32    RecordLen
);
```

Parameters

***pErrorRecord** Error Record buffer as defined in the ACPI reference specification in the Error Source section.

RecordLen Error Record buffer length, in bytes.

Description

This function provides the service to add an error source record entry into the ACPI HEST table.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully,
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.
EFI_NOT_FOUND	Function failure. The ACPI HEST table could not be found.

3.9.3 AMD_RAS_APEI2_PROTOCOL [F17M08]

Summary

This protocol provides services for creating the ACPI APEI table. Two versions of this protocol exist "APEI" and "APEI2". The second version adds some functionality so a new name is applied to preserve backward compatibility. Note the GUID difference and the new Protocol function.

GUID

```
gAmdRasApei2ProtocolGuid = {0xD8ABD054, 0x468F, 0x503C,
                             {0xD3, 0xA7, 0xE5, 0x80, 0xD5, 0x6B, 0x29, 0xB9}}
```

Protocol Interface Structure

```
typedef struct _AMD_RAS_APEI2_PROTOCOL {
    MCA_ERROR_ADDR_TRANSLATE      McaErrorAddrTranslate;
    TRANSLATE_SYSADDR_TO_CS       TranslateSysAddrToCs;
    AMD_ADD_BOOT_ERROR_RECORD_ENTRY AddBootErrorRecordEntry;
    ADD_HEST_ERROR_SOURCE_ENTRY    AddHestErrorSourceEntry;
    SEARCH_MCA_ERROR               SearchMcaError;
} AMD_RAS_APEI2_PROTOCOL;
```

Members

<i>McaErrorAddrTranslate</i>	Function to convert an MCA_ADDR_UMC address to system address and DIMM specific information.
<i>TranslateSysAddrToCs</i>	Function for converting a physical address to DIMM specific information.
<i>AddBootErrorRecordEntry</i>	Function to add a boot error record entry into the ACPI BERT table.
<i>AddHestErrorSourceEntry</i>	Function to add an error source record entry into the ACPI HEST table.
<i>SearchMcaError</i>	Function for performing a search for MCA errors through all banks from a specific thread.

Description This protocol provides services for APEI table creation and RAS feature boot services. It depends upon the MP service protocol. This protocol is introduced for F17M08 and will propagate to new processors.

3.9.3.1 MCA_ERROR_ADDR_TRANSLATE()

Summary

Converts an MCA address MSR value into a chip/device location.

Prototype

```
typedef EFI_STATUS (EFI_API *MCA_ERROR_ADDR_TRANSLATE) (
    IN      NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     UINT64               *SystemMemoryAddress,
    OUT     DIMM_INFO            *DimmInfo
);
```

Parameters

<i>NormalizedAddress</i>	This is the 'normalized' or UMC local relative address as seen in the UMC MCA ADDR MSRs.
<i>SystemMemoryAddress</i>	This return value indicates the physical system address related to this localized address.
<i>DimmInfo</i>	Returned structure that contains the DIMM information related to the given address.

Description

This function converts the given valid error address from an MCA ADDR MSR (UMC) register to system address with DIMM specific information.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.

status code	meaning
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.3.2 TRANSLATE_SYSADDR_TO_CS()

Summary

Converts a system physical address into a chip/device location.

Prototype

```
typedef EFI_STATUS (EFI_API *TRANSLATE_SYSADDR_TO_CS) (
    IN      UINT64      *SystemMemoryAddress,
    OUT     NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     DIMM_INFO    *DimmInfo
);
```

Parameters

SystemMemoryAddress

System Physical address to be translated.

NormalizedAddress

The function will place a normalized address into this structure which represents the location relative this the hosting memory controller.

DimmInfo

Pointer to a DIMM_INFO structure buffer (see the common structure definitions for the driver).

Description

This function converts the given system memory address into a normalized address indicating the SocketId, DieId, channelId for the memory controller that is responsible for the specified system address and also includes information to indicate the DIMM device. This function helps customer identify the physical location (device) that contains the address.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.3.3 AMD_ADD_BOOT_ERROR_RECORD_ENTRY()

Summary

Adds an entry to the ACPI BERT table.

Prototype

```
typedef EFI_STATUS (EFI_API *AMD_ADD_BOOT_ERROR_RECORD_ENTRY) (
    IN      UINT8      *ErrorRecord,
    IN      UINT32     RecordLen,
    IN      UINT8      ErrorType,
    IN      UINT8      SeverityType
);
```

Parameters

***ErrorRecord**

Error Record buffer. This record is defined in [ACPI](#) reference specification as Generic Error Data Entry and in the UEFI reference specification Appendix for Common Platform Error Record.

RecordLen Error Record buffer length
ErrorType Error type of the record
SeverityType Severity type of the record

Description

Use this function to add an error into the ACPI BERT table.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.
EFI_NOT_FOUND	Function failure. The ACPI BERT table could not be found.

3.9.3.4 ADD_HEST_ERROR_SOURCE_ENTRY()**Summary**

Adds an entry to the ACPI HEST error source table.

Prototype

```
typedef EFI_STATUS (EFI_API *ADD_HEST_ERROR_SOURCE_ENTRY) (
    IN      UINT8    *pErrorRecord,
    IN      UINT32    RecordLen
);
```

Parameters

***pErrorRecord** Error Record buffer as defined in the ACPI reference specification in the Error Source section.
RecordLen Error Record buffer length, in bytes.

Description

This function provides the service to add an error source record entry into the ACPI HEST table.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully,
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.
EFI_NOT_FOUND	Function failure. The ACPI HEST table could not be found.

3.9.3.5 SEARCH_MCA_ERROR()**Summary**

Search through all MCA error banks for the errors.

Prototype

```
typedef EFI_STATUS (EFI_API *SEARCH_MCA_ERROR) (
    IN OUT  RAS_MCA_ERROR_INFO_V2* RasMcaErrorInfo
);
```


Parameters

***RasMcaErrorInfo** A pointer to a local structure buffer that will hold the errors found.

Description

Searches the MCA error banks for the specified thread to locate the all reported MCA errors. Each search starts with MCA bank 0 and proceeds to the last bank. If no error is found, then this routine will return EFI_ERROR.

Related Definitions**CPU_INFO**

```
typedef struct _CPU_INFO {
    IN  UINTN      ProcessorNumber;
    OUT UINT8      SocketId;
    OUT UINT8      DieId;
    OUT UINT8      CoreId;
    OUT UINT8      ThreadID;
} CPU_INFO;
```

This structure identifies the specific core reporting an MCA error.

ProcessorNumber Processor on which to search for an MCA error. This is the cpu index number used for the start up of the processor as defined in the MP Services of the UEFI reference specification.

SocketId Logical CPU ID physical socket number.

DieId logic CPU ID physical Die number

CoreId logic CPU ID physical Core number

ThreadID Logic CPU ID physical Thread number when SMT enabled. The number will always return 0 when SMT disabled.

MCA_BANK_ERROR_INFO

```
typedef struct _MCA_BANK_ERROR_INFO {
    UINTN      McaBankNumber;
    MCA_STATUS_MSR      McaStatusMsR;
    MCA_ADDR_MSR      McaAddrMsR;
    MCA_MISC0_MSR      McaMisc0MsR;
    MCA_CONFIG_MSR      McaConfigMsR;
    MCA_IPID_MSR      McaIpidMsR;
    MCA_SYND_MSR      McaSyndMsR;
    MCA_DESTAT_MSR      McaDeStatMsR;
    MCA_ADDR_MSR      McaDeAddrMsR;
    MCA_MISC1_MSR      McaMisc1MsR;
} MCA_BANK_ERROR_INFO;
```

This structure contains the collected MCA register error reporting data.

McaBankNumber MCA bank number for the record

McaStatusMsR The content of the Status MSR for this bank

McaAddrMsR The content of the Address MSR for this bank

McaMisc0MsR The content of the Misc0MSR for this bank

McaConfigMsr	The content of the Config MSR for this bank
McaIpidMsr	The content of the Ipid MSR for this bank
McaSyndMsr	The content of the Synd MSR for this bank
McaDeStatMsr	The content of the DeStatus MSR for this bank
McaDeAddrMsr	The content of the DeAddress MSR for this bank
McaMisc1Msr	The content of the Misc1MSR for this bank

RAS_MCA_ERROR_INFO_V2

```
typedef struct _RAS_MCA_ERROR_INFO_V2 {
    CPU_INFO          CpuInfo;
    UINTN             McaBankCount;
    MCA_BANK_ERROR_INFO McaBankErrorInfo[SSP_MCA_MAX_BANK_COUNT];
} RAS_MCA_ERROR_INFO_V2;
```

Short Description

CpuInfo	Data to identify the specific CPU reporting the error.
McaBankCount	Number of MCA banks that have a valid MCA error.
McaBankErrorInfo	An array containing information about the error(s), with McaBankCount entries.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4 AMD_RAS_SMM_PROTOCOL

Summary

This protocol provides the RAS runtime support via SMI services.

GUID

```
gAmdRasSmmProtocolGuid = {0x4E41A9E3, 0x46AB, 0x1549,
                          {0x06, 0x44, 0xB6, 0xA5, 0x55, 0x29, 0x89, 0x77}}
```

Protocol Interface Structure

```
struct _AMD_RAS_SMM_PROTOCOL {
    MCA_ERROR_ADDR_TRANSLATE    McaErrorAddrTranslate;
    TRANSLATE_SYSADDR_TO_CS    TranslateSysAddrToCS;
    SET_SMI_TRIG_IOCYCLE       SetSmiTrigIoCycle;
    GET_ALL_LOCAL_SMI_STATUS    GetAllLocalSmiStatus;
    SEARCH_MCA_ERROR            SearchMcaError;
    RAS_SMM_EXIT_TYPE           RasSmmExitType;
    GET_SMM_SAVE_STATE_BASE     GetSmmSaveStateBase;
    SET_MCA_CLOAK_CFG           SetMcaCloakCfg;
    CLR_MCA_STATUS              ClrMcaStatus;
};
```

Members

<i>McaErrorAddrTranslate</i>	Function to convert an MCA_ADDR_UMC address to system address and DIMM specific information.
<i>TranslateSysAddrToCS</i>	Function for converting a physical address to DIMM specific information. This is the same function used in the DXE protocol.
<i>SetSmiTrioIoCycle</i>	Pointer to a function for set register SmiTrioIoCycle
<i>GetAllLocalSmiStatus</i>	Pointer to a function for get value of LocalSmiStatus from all threads
<i>SearchMcaError</i>	Pointer to a function for search MCA error through all banks from specific thread
<i>RasSmmExitType</i>	Pointer to a function for select the type of interrupt is generated upon SMM exit
<i>GetSmmSaveStateBase</i>	Pointer to a function for get SMM Save State base address
<i>SetMcaCloakCfg</i>	Pointer to a function for set PFEH_CLOAK_CFG MSR
<i>ClrMcaStatus</i>	Pointer to a function for clear MCA_STATUS

Description This protocol provides configuration service and SMM runtime services for use by the platform to support their RAS feature set.

3.9.4.1 MCA_ERROR_ADDR_TRANSLATE()**Summary**

Converts an MCA address MSR value into a chip/device location.

Prototype

```
typedef EFI_STATUS (EFI_API *MCA_ERROR_ADDR_TRANSLATE) (
    IN      NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     UINT64               *SystemMemoryAddress,
    OUT     DIMM_INFO            *DimmInfo
);
```

Parameters

<i>NormalizedAddress</i>	This is the 'normalized' or UMC local relative address as seen in the UMC MCA ADDR MSRs.
<i>SystemMemoryAddress</i>	This return value indicates the physical system address related to this localized address.
<i>DimmInfo</i>	Returned structure that contains the DIMM information related to the given address.

Description

This function converts the given valid error address from an MCA ADDR MSR (UMC) register to system address with DIMM specific information.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.2 TRANSLATE_SYSADDR_TO_CS()

Summary

Converts a system physical address into a chip/device location.

Prototype

```
typedef EFI_STATUS (EFI_API *TRANSLATE_SYSADDR_TO_CS) (
    IN      UINT64      *SystemMemoryAddress,
    OUT     NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     DIMM_INFO    *DimmInfo
);
```

Parameters

SystemMemoryAddress

System Physical address to be translated.

NormalizedAddress

The function will place a normalized address into this structure which represents the location relative this the hosting memory controller.

DimmInfo

Pointer to a DIMM_INFO structure buffer (see the common structure definitions for the driver).

Description

This function converts the given system memory address into a normalized address indicating the SocketId, DieId, channelId for the memory controller that is responsible for the specified system address and also includes information to indicate the DIMM device. This function helps customer identify the physical location (device) that contains the address.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.3 SET_SMI_TRIG_IOCYLE()

Summary

This function sets an IO cycle trap/trigger to generate an SMI upon an access to the specified address. This applies to all threads.

Prototype

```
typedef EFI_STATUS (EFI_API *SET_SMI_TRIG_IOCYLE) (
    IN      UINT64 SmiTrigIoCycleData
);
```

Parameters

SmiTrigIoCycleData

The IO address for which the trap should be set.

Description

This function sets a given value into MSR register SmiTrigIoCycle. Please reference the PPR MSRC001_0056 [SMI Trigger IO Cycle] register description for detail.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.4 GET_ALL_LOCAL_SMI_STATUS()**Summary**

Collect the SMI status from all threads.

Prototype

```
typedef EFI_STATUS (EFI_API *GET_ALL_LOCAL_SMI_STATUS) (
    IN          LOCAL_SMI_STATUS* pLocalSmiStatusList
);
```

Parameters

***pLocalSmiStatusList** A pointer to a local structure buffer to hold the status records for the threads. The buffer size should be (size of(LOCAL_SMI_STATUS) * #threads).

Description

Gather the SMI status flags for all threads. The local buffer will be filled with the SMI status from each core and/or thread.

Related Definitions**LOCAL_SMI_STATUS**

```
typedef union {
    struct {
        UINT64 IoTrapSts:4;          /// [3:0] IO Trap Status
        UINT64 :4;                   /// [7:4] Reserved
        UINT64 MceRedirSts:1;        /// [8] Machine check exception redirection status
        UINT64 :2;                   /// [10:9] Reserved
        UINT64 WrMsr:1;              /// [11] SMM due to a WRMSR of an MCE_STATUS
        UINT64 :4;                   /// [15:12] Reserved
        UINT64 SmiSrcLvtLgcy:1;       /// [16] SMI source is legacy LVT APIC entry
        UINT64 SmiSrcLvtExt:1;        /// [17] SMI source is APIC[530:500] LVT
        UINT64 SmiSrcMca:1;           /// [18] SMI source is MCA
        UINT64 :54;                   /// [63:19] Reserved
    } Field;
    UINT64 Value;
} LOCAL_SMI_STATUS;
```

Note: This structure definition is the same as the register SMMxFEC4 as defined in the PPR. Please refer to the PPR for further details.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.5 SEARCH_MCA_ERROR()

Summary

Search through all MCA error banks for the next error.

Prototype

```
typedef EFI_STATUS (EFI_API *SEARCH_MCA_ERROR) (
    IN OUT  RAS_MCA_ERROR_INFO* RasMcaErrorInfo
);
```

Parameters

***RasMcaErrorInfo** A pointer to a local structure buffer that will hold the next found error.

Description

Searches the MCA error banks for the specified thread to locate the next reported MCA error. Each search starts with MCA bank 0 and proceeds to the last bank. If no error is found, then this routine will return EFI_NOTFOUND.

Related Definitions

RAS_MCA_ERROR_INFO

```
typedef struct _RAS_MCA_ERROR_INFO {
    CPU_INFO          CpuInfo;
    AMD_MCA_BANK_NUM  McaBankNumber;
    MCA_STATUS_MSR    McaStatusMsR;
    MCA_ADDR_MSR      McaAddrMsR;
    MCA_CONFIG_MSR    McaConfigMsR;
    MCA_SYND_MSR      McaSyndMsR;
    MCA_MISC0_MSR     McaMisc0MsR;
} RAS_MCA_ERROR_INFO;
```

Short Description

CpuInfo	pointer to a CPU_INFO structure
McaBankNumber	MCA bank number that has the next valid MCA error.
McaStatusMsR	The content of the Status MSR for this bank.
McaAddrMsR	The content of the Address MSR for this bank.
McaConfigMsR	The content of the Config MSR for this bank.
McaSyndMsR	The content of the Synd MSR for this bank.
McaMisc0MsR	The content of the Misc0 MSR for this bank.

CPU_INFO

```
typedef struct _CPU_INFO {
    IN  UINTN      ProcessorNumber;
    OUT UINT8      SocketId;
    OUT UINT8      DieId;
    OUT UINT8      CoreId;
    OUT UINT8      ThreadID;
} CPU_INFO;
```

ProcessorNumber	Processor on which to search for an MCA error. This is the cpu index number used for the start up of the processor as defined in the MP Services of the UEFI reference specification.
SocketId	Logical CPU ID physical socket number.
DieId	logic CPU ID physical Die number
CoreId	logic CPU ID physical Core number
ThreadID	Logic CPU ID physical Thread number when SMT enabled. the number will always return 0 when SMT disabled.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.6 RAS_SMM_EXIT_TYPE()

Summary

This is function selects the type of event generated to the specified processor upon SMM exit.

Prototype

```
typedef EFI_STATUS (EFI_API *RAS_SMM_EXIT_TYPE) (
    IN      UINTN ProcessorNumber,
    IN      UINTN SmiExitType
);
```

Parameters

ProcessorNumber	This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.
SmiExitType	Select interrupt type to be generate. Available selections:

- GENERATE_THRESHOLD_LVT_ON_EXIT
- GENERATE_DEFERRED_LVT_ON_EXIT
- GENERATE_MCE_ON_EXIT

Please refer to the PPR section for "Event Percolation" for further detail.

Description

When Platform First Error Handling (PFEH) is active, all MCA errors will go into SMM mode. After local processing, the SMI handler must set the event type that will be returned to the system. This function will set the type of event for the specific processor that will trigger the OS error handler for further processing.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.7 GET_SMM_SAVE_STATE_BASE()

Summary

Retrieve the SMM Base address for the indicated CPU core.

Prototype

```
typedef EFI_STATUS (EFI_API *GET_SMM_SAVE_STATE_BASE) (
    IN      UINTN ProcessorNumber,
    OUT     UINT64* SmmSaveStateBase
);
```

Parameters

ProcessorNumber	This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.
*SmmSaveStateBase	A Pointer to a local buffer where to place the SMM base pointer.

Description

This function will locate and return the SMM Save State base address for the indicated processor. For further information about the Save State buffer, see the PPR.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.8 SET_MCA_CLOAK_CFG()

Summary

Set the error cloaking mask for a specific processor.

Prototype

```
typedef EFI_STATUS (EFI_API *SET_MCA_CLOAK_CFG) (
    IN      UINTN ProcessorNumber,
    IN      UINT64 CloakValue
);
```

Parameters

ProcessorNumber	This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.
CloakValue	Value to write to the cloaking MSR for the indicated processor.

Description

When Platform First Error Handling (PFEH) is active, the MCA registers are 'cloaked' from the OS - meaning the OS will not see them and cannot access them. This routine will set the cloaking mask value for the specified processor. If the firmware wished for the OS to handle an event/error, then the firmware will set the cloaking value so that the OS is permitted to see the error register. For further information on cloaking, please see the PPR.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.4.9 CLR_MCA_STATUS()**Summary**

Clears the indicated MCA status register.

Prototype

```
typedef EFI_STATUS (EFI_API *CLR_MCA_STATUS) (
    IN      UINTN      ProcessorNumber,
    IN      AMD_MCA_BANK_NUM McaBankNumber,
    IN      BOOLEAN     IsWrMsr
);
```

Parameters***ProcessorNumber***

This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.

McaBankNumber

Indicates which MCA bank number for which the status should be cleared. The MCA bank is as defined in the PPR. See enum variable AMD_MCA_BANK_NUM in the code files for specific values.

IsWrMsr

When Platform First Error Handling (PFEH) is active and an MCA bank is uncloaked, a WRMSR to the MCA_STATUS register in that bank triggers an immediate SMI. On an SMI generated due to a WRMSR, SMM code can interrogate and take action:

- to determine which MCA bank and register is being queried.
- to determine the value being written.
- Decide to perform the update, or not
- Decide if the MCA bank should be re-cloaked.

So the function needs to know if this is a WRMSR instruction performed from the OS or it is a clear MCA_STATUS request from the firmware.

- FALSE - write is from firmware
- TRUE - write from OS via WRMSR instruction trap.

This information is reflected in the SMMxFEC4 register.

Description

When Platform First Error Handling (PFEH) is active access to MCA registers will cause SMI; so, these accessed need to be filtered by routines such as this to avoid extra SMI occurrences.

Status Codes Returned

status code	meaning
EFIA_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5 AMD_RAS_SMM2_PROTOCOL [F17M08]

Summary

This protocol provides the RAS runtime support via SMI services. Two versions of this protocol exist "SMM" and "SMM2". The second version adds some functionality so a new name is applied to preserve backward compatibility. Note the GUID difference and the new Protocol functions.

GUID

```
gAmdRasSmm2ProtocolGuid = {0x0CA49B20, 0x4EEF, 0xBFCE,
                           {0x27, 0xE6, 0xCB, 0x90, 0x33, 0xEE, 0xD4, 0x29}}
```

Protocol Interface Structure

```
struct _AMD_RAS_SMM2_PROTOCOL {
    MCA_ERROR_ADDR_TRANSLATE        McaErrorAddrTranslate;
    TRANSLATE_SYSADDR_TO_CS         TranslateSysAddrToCS;
    SET_SMI_TRIG_IOC_CYCLE          SetSmiTrigIoCycle;
    GET_ALL_LOCAL_SMI_STATUS        GetAllLocalSmiStatus;
    SEARCH_MCA_ERROR                SearchMcaError;
    RAS_SMM_EXIT_TYPE               RasSmmExitType;
    GET_SMM_SAVE_STATE_BASE         GetSmmSaveStateBase;
    SET_MCA_CLOAK_CFG               SetMcaCloakCfg;
    CLR_MCA_STATUS                  ClrMcaStatus;
    MAP_SYMBOL_TO_DRAM_DEVICE        MapSymbolToDramDevice;
    SET_MCA_THRESHOLD               SetMcaThreshold;
};
```

Members

McaErrorAddrTranslate

Function to convert an MCA_ADDR_UMC address to system address and DIMM specific information.

TranslateSysAddrToCS

Function for converting a physical address to DIMM specific information. This is the same function used in the DXE protocol.

SetSmiTrigIoCycle

Function to set register SmiTrigIoCycle.

GetAllLocalSmiStatus

Function to get value of LocalSmiStatus from all threads.

SearchMcaError

Function to search for MCA errors through all banks from specific thread.

RasSmmExitType

Function to select the type of interrupt is generated upon SMM exit.

GetSmmSaveStateBase

Function to get SMM Save State base address.

SetMcaCloakCfg

Function to set PFEH_CLOAK_CFG MSR.

ClrMcaStatus

Function to clear MCA_STATUS.

MapSymbolToDramDevice

Function to map the symbol to a DRAM device.

SetMcaThreshold

Function to setup MCA threshold counter.

Description This protocol provides configuration service and SMM runtime services for use by the platform to support their RAS feature set. This protocol is introduced for F17M08 and will propagate to new processors.

3.9.5.1 MCA_ERROR_ADDR_TRANSLATE()

Summary

Converts an MCA address MSR value into a chip/device location.

Prototype

```

typedef EFI_STATUS (EFI_API *MCA_ERROR_ADDR_TRANSLATE) (
    IN      NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     UINT64               *SystemMemoryAddress,
    OUT     DIMM_INFO           *DimmInfo
);

```

Parameters

NormalizedAddress This is the 'normalized' or UMC local relative address as seen in the UMC MCA ADDR MSRs.

SystemMemoryAddress This return value indicates the physical system address related to this localized address.

DimmInfo Returned structure that contains the DIMM information related to the given address.

Description

This function converts the given valid error address from an MCA ADDR MSR (UMC) register to system address with DIMM specific information.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.2 TRANSLATE_SYSADDR_TO_CS()

Summary

Converts a system physical address into a chip/device location.

Prototype

```

typedef EFI_STATUS (EFI_API *TRANSLATE_SYSADDR_TO_CS) (
    IN      UINT64               *SystemMemoryAddress,
    OUT     NORMALIZED_ADDRESS *NormalizedAddress,
    OUT     DIMM_INFO           *DimmInfo
);

```

Parameters

SystemMemoryAddress System Physical address to be translated.

NormalizedAddress The function will place a normalized address into this structure which represents the location relative this the hosting memory controller.

DimmInfo Pointer to a DIMM_INFO structure buffer (see the common structure definitions for the driver).

Description

This function converts the given system memory address into a normalized address indicating the SocketId, DieId, channelId for the memory controller that is responsible for the specified system address and also includes information to indicate the DIMM device. This function helps customer identify the physical location (device) that contains the address.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.3 SET_SMI_TRIG_IOCYCLE()

Summary

This function sets an IO cycle trap/trigger to generate an SMI upon an access to the specified address. This applies to all threads.

Prototype

```
typedef EFI_STATUS (EFIAPI *SET_SMI_TRIG_IOCYCLE) (
    IN      UINT64 SmiTrigIoCycleData
);
```

Parameters

SmiTrigIoCycleData The IO address for which the trap should be set.

Description

This function sets a given value into MSR register SmiTrigIoCycle. Please reference the PPR MSRC001_0056 [SMI Trigger IO Cycle] register description for detail.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.4 GET_ALL_LOCAL_SMI_STATUS()

Summary

Collect the SMI status from all threads.

Prototype

```
typedef EFI_STATUS (EFIAPI *GET_ALL_LOCAL_SMI_STATUS) (
    IN      LOCAL_SMI_STATUS* pLocalSmiStatusList
);
```

Parameters

***pLocalSmiStatusList**

A pointer to a local structure buffer to hold the status records for the threads. The buffer size should be (size of(LOCAL_SMI_STATUS) * #threads).

Description

Gather the SMI status flags for all threads. The local buffer will be filled with the SMI status from each core and/or thread.

Related Definitions**LOCAL_SMI_STATUS**

```
typedef union {
    struct {
        UINT64 IoTrapSts:4;          ///< [3:0] IO Trap Status
        UINT64 :4;                   ///< [7:4] Reserved
        UINT64 MceRedirSts:1;        ///< [8] Machine check exception redirection status
        UINT64 :2;                   ///< [10:9] Reserved
        UINT64 WrMsr:1;              ///< [11] SMM due to a WRMSR of an MCE_STATUS
        UINT64 :4;                   ///< [15:12] Reserved
        UINT64 SmiSrcLvtLgcy:1;       ///< [16] SMI source is legacy LVT APIC entry
        UINT64 SmiSrcLvtExt:1;        ///< [17] SMI source is APIC[530:500] LVT
        UINT64 SmiSrcMca:1;           ///< [18] SMI source is MCA
        UINT64 :54;                  ///< [63:19] Reserved
    } Field;
    UINT64 Value;
} LOCAL_SMI_STATUS;
```

Note: This structure definition is the same as the register SMMxFEC4 as defined in the PPR. Please refer to the PPR for further details.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.5 SEARCH_MCA_ERROR()**Summary**

Search through all MCA error banks for the errors.

Prototype

```
typedef EFI_STATUS (EFI_API *SEARCH_MCA_ERROR) (
    IN OUT  RAS_MCA_ERROR_INFO_V2* RasMcaErrorInfo
);
```

Parameters***RasMcaErrorInfo**

A pointer to a local structure buffer that will hold the errors found.

Description

Searches the MCA error banks for the specified thread to locate the all reported MCA errors. Each search starts with MCA bank 0 and proceeds to the last bank. If no error is found, then this routine will return EFI_ERROR.

Related Definitions

CPU_INFO

```
typedef struct _CPU_INFO {
    IN  UINTN      ProcessorNumber;
    OUT UINT8      SocketId;
    OUT UINT8      DieId;
    OUT UINT8      CoreId;
    OUT UINT8      ThreadID;
} CPU_INFO;
```

This structure identifies the specific core reporting an MCA error.

ProcessorNumber	Processor on which to search for an MCA error. This is the cpu index number used for the start up of the processor as defined in the MP Services of the UEFI reference specification.
SocketId	Logical CPU ID physical socket number.
DieId	logic CPU ID physical Die number
CoreId	logic CPU ID physical Core number
ThreadID	Logic CPU ID physical Thread number when SMT enabled. The number will always return 0 when SMT disabled.

MCA_BANK_ERROR_INFO

```
typedef struct _MCA_BANK_ERROR_INFO {
    UINTN      McaBankNumber;
    MCA_STATUS_MSR      McaStatusMsr;
    MCA_ADDR_MSR      McaAddrMsr;
    MCA_MISC0_MSR      McaMisc0Msr;
    MCA_CONFIG_MSR      McaConfigMsr;
    MCA_IPID_MSR      McaIpidMsr;
    MCA_SYND_MSR      McaSyndMsr;
    MCA_DESTAT_MSR      McaDeStatMsr;
    MCA_ADDR_MSR      McaDeAddrMsr;
    MCA_MISC1_MSR      McaMisc1Msr;
} MCA_BANK_ERROR_INFO;
```

This structure contains the collected MCA register error reporting data.

McaBankNumber	MCA bank number for the record
McaStatusMsr	The content of the Status MSR for this bank
McaAddrMsr	The content of the Address MSR for this bank
McaMisc0Msr	The content of the Misc0MSR for this bank
McaConfigMsr	The content of the Config MSR for this bank
McaIpidMsr	The content of the Ipid MSR for this bank
McaSyndMsr	The content of the Synd MSR for this bank
McaDeStatMsr	The content of the DeStatus MSR for this bank
McaDeAddrMsr	The content of the DeAddress MSR for this bank
McaMisc1Msr	The content of the Misc1MSR for this bank

RAS_MCA_ERROR_INFO_V2

```
typedef struct _RAS_MCA_ERROR_INFO_V2 {
    CPU_INFO      CpuInfo;
    UINTN         McaBankCount;
    MCA_BANK_ERROR_INFO McaBankErrorInfo[SSP_MCA_MAX_BANK_COUNT];
} RAS_MCA_ERROR_INFO_V2;
```

Short Description

CpuInfo	Data to identify the specific CPU reporting the error.
McaBankCount	Number of MCA banks that have a valid MCA error.
McaBankErrorInfo	An array containing information about the error(s), with McaBankCount entries.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.6 RAS_SMM_EXIT_TYPE()

Summary

This function selects the type of event generated to the specified processor upon SMM exit.

Prototype

```
typedef EFI_STATUS (EFI_API *RAS_SMM_EXIT_TYPE) (
    IN      UINTN ProcessorNumber,
    IN      UINTN SmiExitType
);
```

Parameters

ProcessorNumber	This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.
SmiExitType	Select interrupt type to be generate. Available selections: • GENERATE_THRESHOLD_LVT_ON_EXIT • GENERATE_DEFERRED_LVT_ON_EXIT • GENERATE_MCE_ON_EXIT Please refer to the PPR section for "Event Percolation" for further detail.

Description

When Platform First Error Handling (PFEH) is active, all MCA errors will go into SMM mode. After local processing, the SMI handler must set the event type that will be returned to the system. This function will set the type of event for the specific processor that will trigger the OS error handler for further processing.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.7 GET_SMM_SAVE_STATE_BASE()

Summary

Retrieve the SMM Base address for the indicated CPU core.

Prototype

```
typedef EFI_STATUS (EFI_API *GET_SMM_SAVE_STATE_BASE) (
    IN      UINTN ProcessorNumber,
    OUT     UINT64* SmmSaveStateBase
);
```

Parameters

ProcessorNumber	This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.
*SmmSaveStateBase	A Pointer to a local buffer where to place the SMM base pointer.

Description

This function will locate and return the SMM Save State base address for the indicated processor. For further information about the Save State buffer, see the PPR.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.8 SET_MCA_CLOAK_CFG()

Summary

Set the error cloaking mask for a specific processor.

Prototype

```
typedef EFI_STATUS (EFI_API *SET_MCA_CLOAK_CFG) (
    IN      UINTN ProcessorNumber,
    IN      UINT64 CloakValue
);
```

Parameters

ProcessorNumber	This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.
CloakValue	Value to write to the cloaking MSR for the indicated processor.

Description

When Platform First Error Handling (PFEH) is active, the MCA registers are 'cloaked' from the OS - meaning the OS will not see them and cannot access them. This routine will set the cloaking mask value for the specified processor. If the firmware wished for the OS to handle an event/error, then the firmware will set the cloaking value so that the OS is permitted to see the error register. For further information on cloaking, please see the PPR.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.9 CLR_MCA_STATUS()

Summary

Clears the indicated MCA status register.

Prototype

```
typedef EFI_STATUS (EFI_API *CLR_MCA_STATUS) (
    IN      UINTN      ProcessorNumber,
    IN      AMD_MCA_BANK_NUM McaBankNumber,
    IN      BOOLEAN     IsWrMsr
);
```

Parameters

ProcessorNumber

This is the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES.

McaBankNumber

Indicates which MCA bank number for which the status should be cleared. The MCA bank is as defined in the PPR. See enum variable AMD_MCA_BANK_NUM in the code files for specific values.

IsWrMsr

When Platform First Error Handling (PFEH) is active and an MCA bank is uncloaked, a WRMSR to the MCA_STATUS register in that bank triggers an immediate SMI. On an SMI generated due to a WRMSR, SMM code can interrogate and take action:

- to determine which MCA bank and register is being queried.
- to determine the value being written.
- Decide to perform the update, or not
- Decide if the MCA bank should be re-cloaked.

So the function needs to know if this is a WRMSR instruction performed from the OS or it is a clear MCA_STATUS request from the firmware.

- FALSE - write is from firmware
- TRUE - write from OS via WRMSR instruction trap.

This information is reflected in the SMMxFEC4 register.

Description

When Platform First Error Handling (PFEH) is active access to MCA registers will cause SMI; so, these accessed need to be filtered by routines such as this to avoid extra SMI occurrences.

Status Codes Returned

status code	meaning
EFIA_SUCCESS	The function has completed successfully.
EFI_ERROR	Function failure. This is a standard EFI error, please see the UEFI references.

3.9.5.10 MAP_SYMBOL_TO_DRAM_DEVICE ()

Summary

Maps an ECC Symbol to a DRAM device based on DIMM device width, EccSymbolSize and EccBitInterleaving.

Prototype

```
typedef EFI_STATUS (EFI_API *MAP_SYMBOL_TO_DRAM_DEVICE) (
    IN      AMD_RAS_SMM2_PROTOCOL *This,
    IN      RAS_MCA_ERROR_INFO_V2 *RasMcaErrorInfo,
    IN      NORMALIZED_ADDRESS    *NormalizedAddress,
    IN      UINT8                  BankIndex,
    OUT     UINT32                  *DeviceStart,
    OUT     UINT32                  *DeviceEnd,
    OUT     UINT8                  *DeviceWidth
);
```

Parameters

This	UEFI SMM standard pointer to the SMM services function block.
RasMcaErrorInfo	A pointer to a structure buffer that will hold found errors.
NormalizedAddress	This is the 'normalized' or UMC local relative address as seen in the UMC MCA ADDR MSRs.
BankIndex	Offset of data in McaBankErrorInfo struct.
DeviceStart	First device where the symbol indicates an error.
DeviceEnd	Last device where the symbol indicates an error.
DeviceWidth	DIMM Device Width from SPD.

Description

This function converts an ECC symbol to a DRAM device range. This service id used for DDR Post Package Repair feature. Please reference Socket SP3 RAS Platform Specification and Implementation Guide for DDR Post Package Repair feature.

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	This is a standard EFI error, please see the UEFI reference.

3.9.5.11 SET_MCA_THRESHOLD ()

Summary

This is the function to set MCA threshold counter and interrupt type.

Prototype

```
Typedef EFI_STATUS (EFI_API *SET_MCA_THRESHOLD) (
    IN      UINTN      *ProcessorNumber,
    IN      UINTN      *McaBankNumber,
    IN      RAS_THRESHOLD_CONFIG *RasThresholdConfig,
    IN      BOOLEAN     OvrflwChk
);
```

Parameters

ProcessorNumber

A pointer to the cpu index number used for the MP services during start up. See the UEFI reference specification for MP_SERVICES. The function will setup through all threads when input pointer = NULL.

McaBankNumber

A Pointer to the MCA bank number for which the threshold need to set. The MCA bank is as defined in the PPR. The function will setup through all available banks in the thread when input pointer = NULL.

RasThresholdConfig OvrflwChk

A pointer to threshold configuration data structure. There are two use scenarios. One required unconditional setup threshold counter (Ex: boot time initialize). One only reset the threshold counter when MCA_MISCx[Ovrflw] = 1. (Ex: Runtime error handler)

- FALSE - Always reset ErrCnt.
- TRUE – Will check Ovrflw bit first before reset ErrCnt.

Description

This function is used to covert an ECC symbol to a DRAM device range.

Related Definitions

RAS_THRESHOLD_CONFIG

```
Typedef struct _RAS_THRESHOLD_CONFIG {
    BOOLEAN      ThresholdControl;
    UINT16       ThresholdCount;
    UINT8        ThresholdIntType;
} RAS_THRESHOLD_CONFIG;
```

ThresholdControl

Value for MCA_MISCx[CntEn]

ThresholdCount

Value for MCA_MISCx[ErrtCnt]

ThresholdControl

Value for MCA_MISCx[ThresholdIntType]

Status Codes Returned

status code	meaning
EFI_SUCCESS	The function has completed successfully.
EFI_ERROR	This is a standard EFI error, please see the UEFI reference.

3.9.6 AMD_NBIO_PCIE_AER_PROTOCOL

Summary

This protocol provides the ability to enable, configure or disable the PCIe® Advanced Error Reporting (AER) feature.

GUID

```
gAmdNbioPcieAerProtocolGuid=
{0xe48c773, 0x4445, 0x40d5,
 {0x9f, 0x11, 0x5f, 0x25, 0x6d, 0x19, 0xc1, 0x7b}}
}
```

Protocol Interface Structure

```
typedef struct _AMD_NBIO_PCIE_AER_PROTOCOL {
    AMD_NBIO_PCIE_AER_FEATURE    SetPcieAerFeature;
};
```

Members

SetPcieAerFeature Function used to Enable or disable the PCIe® Advanced Error Reporting feature.

Description Advanced Error Reporting (AER) is defined by the PCI Express specification to allow the system level hardware and software to gracefully handle errors in the PCIe® system. SetPcieAerFeature is used to control reporting of individual correctable/uncorrectable errors. It controls whether an individual uncorrectable error is reported by a function as a Non-fatal or Fatal error.

3.9.6.1 SetPcieAerFeature()

Summary

Function used to enable or disable the PCIe AER feature.

Prototype

```
typedef
EFI_STATUS
(EFI_API * AMD_NBIO_PCIE_AER_FEATURE) (
    IN  AMD_NBIO_PCIE_AER_PROTOCOL *This,
    IN  PCIE_PORT_AER_CONFIG       *PcieAerSetting
);
```

Parameters

This pointer to this instance of the protocol.
PcieAerSetting pointer to a structure used for setting the AER feature controls.

Description

This protocol provides Enable or disable control for the PCIe® Advanced Error Reporting feature. The purpose, usage and controls explanation can be found in the PCI Express Base Specification Revision 3.1

Related Definitions

PCIe_PORT_AER_CONFIG

```
typedef struct {
    UINT8    AerEnable;
    UINT8    PciBus;
    UINT8    PciDev;
    UINT8    PciFunc;
    PCIE_AER_CORRECTABLE_MASK    CorrectableMask;
    PCIE_AER_UNCORRECTABLE_MASK  UncorrectableMask;
    PCIE_AER_UNCORRECTABLE_SEVERITY    UncorrectableSeverity;
} PCIE_PORT_AER_CONFIG;
```

This structure is used to describe PCIe® AER Port Configuration.

AerEnable	General per-port enable, 0=disable 1=enable.
PciBus	PCI Bus Number.
PciDev	PCI Device Number.
PciFunc	PCI Function Number.
CorrectableMask	Per-port mask for correctable errors.
UncorrectableMask	Per-port mask for uncorrectable errors.
UncorrectableSeverity	Per-port severity configuration for uncorrectable errors.

PCIe_AER_CORRECTABLE_MASK

```
typedef union {
    struct {
        UINT32 BadTLPMask           :1;
        UINT32 BadDLLPMask         :1;
        UINT32 ReplayNumberRolloverMask :1;
        UINT32 ReplayTimerTimeoutMask :1;
        UINT32 AdvisoryNonFatalErrorMask :1;
    } Field;
    UINT32 Value;
} PCIe_AER_CORRECTABLE_MASK;
```

This structure is used to describe PCIe® Correctable Error Mask

BadTLPMask	Controls if the function can report a Bad TLP Error.
BadDLLPMask	Controls if the function can report a Bad DLLP Error.
ReplayNumberRolloverMask	Controls if the function can report a REPLAY_NUM Rollover Error.
ReplayTimerTimeoutMask	Controls if the function can report a Replay Timer Timeout Error.
AdvisoryNonFatalErrorMask	Controls if the function can report an Advisory Non-Fatal Error.
Value	A dummy label to be able to access the full bit-packed array as one 32-bit integer.

PCIe_AER_UNCORRECTABLE_MASK

```
typedef union {
    struct {
        UINT32 DataLinkProtocolErrorMask :1;
        UINT32 PoisonedTLPMask           :1;
        UINT32 CompletionTimeoutMask     :1;
        UINT32 CompleterAbortMask         :1;
        UINT32 UnexpectedCompletionMask   :1;
        UINT32 MalTlpMask                 :1;
        UINT32 ECRCErrormask              :1;
        UINT32 UnsupportedRequestErrorMask :1;
        UINT32 AcsViolationMask           :1;
    } Field;
    UINT32 Value;
} PCIe_AER_UNCORRECTABLE_MASK;
```

This structure is used to describe PCIe® Uncorrectable Error Mask.

DataLinkProtocolErrorMask	Controls if the function can report a Data Link Protocol Error.
PoisonedTLPMask	Controls if the function can report a Poisoned TLP Received Error.
CompletionTimeoutMask	Controls if the function can report a Completion Timeout Error.
CompleterAbortMask	Controls if the function can report a Completer Abort Error.
UnexpectedCompletionMask	Controls if the function can report an Unexpected Completion Error.
MalTlpMask	Controls if the function can report a Malformed TLP Error.
ECRCErrorMask	Controls if the function can report an ECRC Error.
UnsupportedRequestErrorMask	Controls if the function can report an Unsupported Request Error.
AcsViolationMask	Controls if the function can report an ACS Violation Error.
Value	A dummy label to be able to access the full bit-packed array as one 32-bit integer.

PCIE_AER_UNCORRECTABLE_SEVERITY

```
typedef union {
    struct {
        UINT32 DataLinkProtocolErrorSeverity :1;
        UINT32 PoisonedTLPSeverity :1;
        UINT32 CompletionTimeoutSeverity :1;
        UINT32 CompleterAbortSeverity :1;
        UINT32 UnexpectedCompletionSeverity :1;
        UINT32 MalTlpSeverity :1;
        UINT32 ECRCErrorSeverity :1;
        UINT32 UnsupportedRequestErrorSeverity :1;
        UINT32 AcsViolationSeverity :1;
    } Field;
    UINT32 Value;
} PCIE_AER_UNCORRECTABLE_SEVERITY;
```

This structure is used to describe PCIe® Uncorrectable Error Severity

DataLinkProtocolErrorSeverity	Controls how the function reports a Data Link Protocol Error.
PoisonedTLPSeverity	Controls how the function reports a Poisoned TLP Received Error.
CompletionTimeoutSeverity	Controls how the function reports a Completion Timeout Error.
CompleterAbortSeverity	Controls how the function reports a Completer Abort Error.
UnexpectedCompletionSeverity	Controls how the function reports an Unexpected Completion Error.
MalTlpSeverity	Controls how the function reports a Malformed TLP Error.
ECRCErrorSeverity	Controls how the function reports an ECRC Error.

UnsupportedRequestErrorSeverity

Controls how the function reports an Unsupported Request Error.

AcsViolationSeverity

Controls how the function reports an ACS Violation Error.

Value

A dummy label to be able to access the full bit-packed array as one 32-bit integer.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.
EFI_UNSUPPORTED	Function failure.

4 AGESA Boot Loader (ABL)

The AGESA™ Boot Loader (ABL) is firmware code that runs on the secured processor. It is responsible for configuring many of the system components from the secure environment so as to prevent any malicious activity. This includes:

- Data and control fabric
- Multi-socket links
- system memory DRAM
- I/O devices (PCIe, SATA, Ethernet)

After memory is initialized the x86 cores are released to execute from memory. For more information about the PSP and the ABL operation, please see {ref: [PSP BIOS Architecture spec](#)}.

These activities require lots of platform description parameters to inform the ABL about the platform design and layout. These descriptors are contained in the AGESA Platform Configuration Block (APCB). To limit any attempts to adversely affect the boot process, these parameters are not available for modification during the boot phase. They are built by the platform designers during the development phase and placed into the Flash ROM and signed for security.

Growth over time of the parameter set has caused the need for a re-vamp of the APCB structure and operation. Family 17h processors from models 00h through 2Fh have used the original APCB while Family 17h Model 30h introduces the new APCB V3.0; to summarize:

- APCB v2 (original) [F17M00][F17M08][F17M10][F17M18][F17M20]
- APCB v3.0 [F17M30], and beyond

4.1 ABL Configuration

The ABL platform configuration items are built into an APCB image through the [APCB Tool](#). There is also a POST time method provided for dynamic modifications to be made. See [APCB Services](#).

During the execution of the ABL, it is possible that errors may be detected. How the ABL responds to those errors can be configured by the platform and a monitoring device can be used to observe those errors. See [ABL debug](#) for these methodologies.

This section identifies and explains the configuration items available in the APCB.

4.1.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

4.1.2 Memory Related Tokens

ABL Configuration covers several components, but mostly the memory configurations. The controls are separated here by component and category.

- The Memory component deals with the Unified Memory Controller(s) (UMCs) in the system. These focus on the DRAM devices on their attached channels.
- The Data Fabric (DF) handles system wide features that span UMCs and possibly multiple Die and sockets.
- The PHY is the physical translator from the internal digital signals to the high frequency traces running to the DRAMs. As memory data rates increase, more tuning controls are provided at this layer to fine tune individual board for the best performance.

The APCB tokens provide the base configuration options (which apply to all UMCs uniformly); but, the **PSO** and **PTO** macros can apply overrides to these base settings using parameters to target specific subsets.

4.1.2.1 Memory Controller - Enablements Category

This section describes the external parameters that can be used to control ABL execution. Please see “AgesaPkg \Addendum\%board% \Apcb\ApcbData_<Platform>_ExtParams.c”

- **APCB_TOKEN_CONFIG_CONFIG_ENABLEBANKINTLV [F17M00][F17M10][F17M20]**

Specifies if the system should use DRAM bank (also known as chip-select) interleaving. This feature will spread memory accesses across the banks of memory within a channel.

If the build option is set to include the code for this feature, then this setting activates the feature. If the feature code is removed from the build, then this element has no effect.

The AMD recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: List)(Default: Enabled)

- Enabled—This option is active
- Disabled—This option is turned off
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_ENABLECHIPSELECTINTLV [F17M30][F19M00]**

Specifies if the system should use Chip Select. This feature will spread memory accesses across the ranks of memory within a channel.

Permitted Choices: (Type: List)(Default: Enabled)

- Enabled—This option is active
- Disabled—This option is turned off
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_ENABLEECCFEATURE [F17M30][F19M00]**
APCB_TOKEN_CONFIG_CONFIG_ENABLEECCFEATURE [F17M00][F17M10][F17M20]

This turns on the correction action and enables the ability of the MCA subsystem to report errors. It does not activate the MCA error report interrupts.

Permitted Choices: (Type: Boolean)(Default: Enabled)

- Enabled—This option is active
- Disabled—This option is turned off
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_ENABLEPARITY [F17M30][F19M00]**
APCB_TOKEN_CONFIG_ENABLEPARITY [F17M00][F17M10][F17M20]

Parity is an error-detection tool available on some DIMMs. This control specifies whether or not the memory controller should activate parity checking. The feature is invoked only if the DIMMs present are all capable of supporting the parity detection.

Permitted Choices: (Type: Boolean)(Default: Enabled)

- Enabled—This option is active
- Disabled—This option is turned off
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_MEMORYBUSFREQUENCYLIMIT [F17M30][F19M00]**
APCB_TOKEN_CONFIG_MEMORYBUSFREQUENCYLIMIT [F17M00][F17M10][F17M20]

This is the maximum memory clock at which the platform memory busses are capable of performing. This setting will restrict the frequency even if the DIMMs support a higher frequency. This platform restriction may occur due to layout difficulties.

Permitted Choices: (Type: List)(Default: DDR3200_FREQUENCY)

Please consult the PPR specification for the product in use for supported frequencies.

- DDR400_FREQUENCY
- DDR533_FREQUENCY
- DDR667_FREQUENCY
- DDR800_FREQUENCY
- DDR1066_FREQUENCY
- DDR1333_FREQUENCY
- DDR1600_FREQUENCY
- DDR1866_FREQUENCY
- DDR2100_FREQUENCY
- DDR2133_FREQUENCY
- DDR2400_FREQUENCY
- DDR2667_FREQUENCY
- DDR2933_FREQUENCY
- DDR3200_FREQUENCY
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_CBS_CMN_MEM_SPEED_DDR4 [F19M00]**

This token allows the memory clock to be further reduced from the platform maximum set by APCB_TOKEN_CONFIG_MEMORYBUSFREQUENCYLIMIT. This control will be ignored if it tries to set a speed greater than the platform limit. This could be used as a temporary reduction for testing with new devices.

Permitted Choices: (Type: List)(Default: Auto)

Please consult the PPR specification for the product in use for supported frequencies.

- Auto - use the limit set by APCB_TOKEN_CONFIG_MEMORYBUSFREQUENCYLIMIT.
- 400MHz - DDR800_FREQUENCY
- 667MHz - DDR1333_FREQUENCY
- 800MHz - DDR1600_FREQUENCY
- 933MHz - DDR1866_FREQUENCY
- 1067MHz - DDR2133_FREQUENCY

- 1200MHz - DDR2400_FREQUENCY
 - 1333MHz - DDR2667_FREQUENCY
 - 1467MHz - DDR2933_FREQUENCY
 - 1600MHz - DDR3200_FREQUENCY
 - 1633MHz - DDR3266_FREQUENCY
 - 1667MHz - DDR3333_FREQUENCY
 - 1700MHz - DDR3400_FREQUENCY
 - 1733MHz - DDR3466_FREQUENCY
 - 1767MHz - DDR3533_FREQUENCY
 - 1800MHz - DDR3600_FREQUENCY
- **APCB_TOKEN_UID_ENABLEMEMPSTATE [F17M30][F17M70]**
This value indicates whether to enable memory P-States.
Permitted Choices: (Type: Boolean)(Default: TRUE)
 - TRUE - Enable memory P-States.
 - FALSE - Disable memory P-States.
 - **APCB_TOKEN_UID_ENABLEBANKSWIZZLE [F17M30][F17M70][F19M00]**
Address swizzle is a performance fine-tuning element that swaps some address lines. See the BKDG for further description.
Permitted Choices: (Type: Boolean)(Default: TRUE)
 - TRUE - This option is active
 - FALSE - This option is turned off
 - **APCB_TOKEN_UID_ACTION_ON_BIST_FAILURE [F19M00]**
This token defines the action to be taken when there is a memory BIST failure.
Permitted Choices: (Type: Value) (Default: 0)
 - 0: Do nothing
 - 1: Use Down-CCD feature to disable CCDs and attempt to boot with a minimum configuration.

4.1.2.2 Memory Controller - Platform Category

This section describes the parameters that can be used to describe the platform specific selections for the Memory controller(s).

- **APCB_TOKEN_UID_MOTHERBOARD_TYPE_0 [F17M30]**
This control notifies the PSP about the type of motherboard the design sets for the platform. The PPR for F17M30 states different memory speed capabilities for Type 0 and Type 1 boards.
Permitted Choices: (Type: Boolean)(Default: False)
 - False - This board is not type 0, it is type 1.
 - True - This board is a type 0 motherboard.
- **PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT [F19M00]**

- **PSP_GET_BOARD_ID_FROM_IO_STRUCT [F19M00]**
- **APCB_TOKEN_UID_USERTIMINGMODE [F17M30][F19M00]**
APCB_TOKEN_CONFIG_USERTIMINGMODE [F17M00][F17M10][F17M20]

Allows the platform to select how the determination is made for the memory clock that is to be used in the system. (See also APCB_TOKEN_UID_MEMCLOCKVALUE.)

Permitted Choices: (Type: List)(Default: TIMING_MODE_AUTO)

- TIMING_MODE_AUTO—The AGESA™ software calculates the best memory clock rate.
- TIMING_MODE_LIMITED—The AGESA software calculates the best memory clock, restricted by a maximum user limit provided in APCB_TOKEN_UID_MEMCLOCKVALUE.
- TIMING_MODE_SPECIFIC—The platform specifies the clock rate, which is provided in APCB_TOKEN_UID_MEMCLOCKVALUE.
- TIMING_MODE_AUTO—ABL decides the best setting.

- **APCB_TOKEN_UID_MEMCLOCKVALUE [F17M30][F19M00]**
APCB_TOKEN_CONFIG_MEMCLOCKVALUE [F17M00][F17M10][F17M20]

This is the specified memory clock to be used in the system as determined by the selected mode. (See also APCB_TOKEN_UID_USERTIMINGMODE.)

Permitted Choices: (Type: List)(Default: Auto)

- DDR400_FREQUENCY
- DDR533_FREQUENCY
- DDR667_FREQUENCY
- DDR800_FREQUENCY
- DDR1066_FREQUENCY
- DDR1333_FREQUENCY
- DDR1600_FREQUENCY
- DDR1866_FREQUENCY
- DDR2100_FREQUENCY
- DDR2133_FREQUENCY
- DDR2400_FREQUENCY
- DDR2667_FREQUENCY
- DDR2933_FREQUENCY
- DDR3200_FREQUENCY
- Auto—ABL decides the best setting.

- **APCB_TOKEN_CONFIG_FCH_SMBUS_SPEED**

This control specifies the operational frequency of the SMBus. The frequency can be virtually any value as defined by the equation listed in the PPR. The platform designer should specify this value based upon the components designed onto the motherboard.

The user value presented here is multiplied by 4 then used as a divisor to 66MHz.

Permitted Choices: (Type: Value)(Default: 42d)

This is a decimal whole integer to be used in the PPR equation. For example:

The default value (42d): $\text{Freq} = 66\text{MHz} / (42 * 4) = 392\text{KHz}$

The value of 0xB0 (176d): $\text{Freq} = 66\text{MHz} / (176 * 4) = \sim 94\text{KHz}$

A value of 1: $\text{Freq} = 66\text{MHz} / (1 * 4) = 16.5\text{MHz}$

- **APCB_TOKEN_UID_IGNORESPDCHECKSUM [F17M30][F19M00]**
APCB_TOKEN_CONFIG_IGNORESPDCHECKSUM [F17M00][F17M10][F17M20]

When the checksum of the SPD record fails, the typical action is to drop the DIMM and not attempt to configure it into the system. The software indicates via return code that this condition has occurred. Under certain conditions, the platform can decide to accept the associated risk and choose to ignore the SPD checksum.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE—Ignore faulty SPD checksum and continue DIMM initialization
- FALSE—If the checksum of an SPD is found to be invalid, then the memory initialization process is terminated for that DIMM
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_SOLDERDOWNDRAM [F17M00][F17M10]**
APCB_ID_CONFIG_SOLDERDOWNDRAM [F17M30][F19M00]

This control specifies that the platform has a memory-down design. This type of platform will also need to specify the SPD content to describe the memory device timing. Please see [PSO-SPD Info](#).

- **APCB_TOKEN_UID_MEM_SPD_READ_OPTIMIZATION_DDR4 [F17M30][F19M00]**

This item configures the SPD Read Optimization. Because SPD reads via SMBUS is a very slow interface, this option allows the user to skip some unnecessary SPD reads to reduce the memory detection time.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - SPD reads are skipped for reserved fields and most of upper 256 Bytes.
- FALSE - Read all 512 SPD Bytes.

This control is not used for memory down systems. SPD checksums are not verified when this control is set to True.

- **APCB_TOKEN_UID_FCH_ROM3_BASE_HIGH [F17M30][F19M00]**

This token sets the base address for the ROM3 region. See the PPPR for "BAR_64MB_ROM3_High" at FCH::ITF::SPI. This control is used by systems enabling the [ROM Armor feature](#).

Permitted Choices: (Type: Value Int32)(Default: 0x0000_0000)

- 0x0000 - This will result in no change of the BAR register from the power-on default value of 0x0000_00FD. (ROM3 region will be 0x00FD_0000_0000 to 0x00FD_03FF_FFFF).
- 0x0??? - This value will specifically update the BAR register. e.g. if this token value is set to 0x07FC, the register will read as 0x7FC and the ROM3 region will be 0x07FC_0000_0000 to 0x07FC_03FF_FFFF.

- **APCB_TOKEN_CONFIG_MEMCONTEXTCTL [F17M10]**

Control switch to save memory context at the end of MemAuto.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE—AGESA will setup MemContext block before exit AmdInitPost
- FALSE—AGESA will not setup MemContext block. Platform is, expected to call S3Save laterPOST if it wants to use memory context restore feature.

- Auto—ABL decides the best setting.
- **APCB_TOKEN_UID_MEM_ADDRESS_HASH_BANK_2_COL_XOR [F19M00]**
This token sets a register field related to bank hashing modes. See PPR section "Address Hash Bank". This is mostly applicable for validation purpose. Customer should not change it. Internal use only by validation.
Permitted Choices: (Type: Value)(Default: 0x03F8)

4.1.2.3 Memory Controller - Power Management Category

This section describes the parameters that can be used to control the individual memory controller power management features.

- **APCB_TOKEN_UID_ENABLEPOWERDOWN [F17M30][F17M70][F19M00]**
APCB_TOKEN_CONFIG_ENABLEPOWERDOWN [F17M00][F17M10][F17M20]
This feature conserves power in the DIMMs by placing them into self-refresh when the memory on a channel is not actively being accessed. They quickly exit self-refresh upon the next processor access.
Permitted Choices: (Type: Boolean)(Default: TRUE)
 - TRUE—This option is active
 - FALSE—This option is turned off
 - Auto—ABL decides the best setting.
- **APCB_TOKEN_UID_MEM_SELF_REFRESH_EXIT_STAGGERING [F17M30][F19M00]**
This item enables LRDIMM Refresh Staggering by adding a value to SrsTiming0.TckSrx to delay Self-Refresh Exit by $.33 * Trfc$ or $0.25 Trfc * UMC$ Instance number modulo 4. The purpose is to prevent all of the DIMMs exiting self refresh at the same time, which may cause power surge or voltage droop.
Permitted Choices: (Type: List)(Default: Disabled)
 - Disabled — This option is turned off.
 - Trfc/3 - one third of the Trfc time.
 - Trfc/4 - one fourth of the Trfc time.
- **APCB_TOKEN_UID_CBS_CMN_MEM_CTRLER_PWR_DN_EN_DDR4 [F17M30][F19M00]**
For those who include the CBS feature, this item is the CBS control for enabling the memory Power Down. When this item is set to 'Auto', the resulting value is taken from the APCB_TOKEN_UID_ENABLEPOWERDOWN control.
Permitted Choices: (Type: Boolean)(Default: TRUE)
 - TRUE— Power Down is forced to be active.
 - FALSE—Power Down is forced to be turned off.
 - Auto—value is taken from APCB token APCB_TOKEN_UID_ENABLEPOWERDOWN.
- **APCB_TOKEN_UID_CBS_CMN_MEM_PWRDOWNDLY [F19M00]**
For those who include the CBS feature, this item is the CBS control set a delay for entry into DDR power down mode by specified a number of DRAM clocks to wait. Usage: Setting it to higher values will delay the entry into DRAM power-down mode. In lower DRAM bandwidth workloads, higher values may reduce average latency to DRAM at the cost of reduced DRAM power-down residency (less power saved).

Permitted Choices: (Type: Value)(Default: 0xFF - Auto)

- 1..N - The number of clocks to wait before entering Power Down Mode.
- 0xFF, Auto — Don't delay entry. Save as much power as possible.

• **APCB_TOKEN_CONFIG_MEMORYALLCLOCKSON [F17M00][F17M10][F17M20]
APCB_TOKEN_UID_MEMORYALLCLOCKSON [F17M30][F17M70]**

This is a power-usage control. To save power, unused memory clocks are disabled. Some platforms may not prefer to use this feature.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE—Enable all memclocks whether they are used or not.
- FALSE—Unused memclocks are disabled
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_MEM_NVDIMM_POWER_SOURCE [F17M30][F19M00]**

The NVDIMM DDR4 modules have a feature for a catastrophic save operation. Please see the Jedec spec for more details. The power source during this operation is selectable. This item allows the user to select the source.

Permitted Choices: (Type: List)(Default: DEVICE_MANAGED)

- DEVICE_MANAGED - the module is configured to use a locally attached Energy Source and will monitor the health of the Energy Source. Set this APCB token to 1 for this source type.
- HOST_MANAGED - the module is configured to use the 12V pin as the Energy Source. Set this APCB token to 2 for this source type.

DIMM Temperature controls

This group of controls are related to the DIMM temperature sensing and response systems in the controller and DIMM itself. Please refer to the PPR section on "DRAM Thermal Management and Bandwidth Capping" for more details.

• **APCB_TOKEN_UID_MEM_TEMP_CONTROLLED_REFRESH_EN**

This option selects the temperature controlled Refresh Mode.

Permitted Choices: (Type: BOOLEAN) (Default: Disabled)

- Enabled - This feature is enabled and operational.
- Disabled

• **APCB_TOKEN_UID_DIMMSENSORCONF [F17M30][F17M70][F19M00]
APCB_TOKEN_CONFIG_DIMMSENSORCONF [F17M00][F17M10][F17M20]**

New for DDR4 is a temperature sensor on the DIMM. There are a few registers that can be programmed by software. The definition of these registers can be found in your DIMM manufacturer's spec sheets which may use a device similar to the STTS2004. This options sets the DIMM Sensor Configuration register (Reg1).

Permitted Choices: (Type: Value)(Default: 0x0408)

• **APCB_TOKEN_UID_DIMMSENSORUPPER [F17M30][F17M70][F19M00]
APCB_TOKEN_CONFIG_DIMMSENSORUPPER [F17M00][F17M10][F17M20]**

This control sets a value for the new DDR4 DIMM temperature sensor upper threshold (Reg2).

Permitted Choices: (Type: Value)(Default: 80)

- **APCB_TOKEN_UID_DIMMSSENSORLOWER [F17M30][F17M70][F19M00]**
APCB_TOKEN_CONFIG_DIMMSSENSORLOWER [F17M00][F17M10][F17M20]

This control sets a value for the new DDR4 DIMM temperature sensor lower threshold (Reg3).

Permitted Choices: (Type: Value)(Default: 10)

- **APCB_TOKEN_UID_DIMMSSENSORCRITICAL [F17M30][F17M70][F19M00]**
APCB_TOKEN_CONFIG_DIMMSSENSORCRITICAL [F17M00][F17M10][F17M20]

This control sets a value for the new DDR4 DIMM temperature sensor critical threshold (Reg4).

Permitted Choices: (Type: Value)(Default: 95)

- **APCB_TOKEN_CONFIG_DIMMSSENSORRESOLUTION [F17M00][F17M10][F17M20]**
APCB_TOKEN_UID_DIMMSSENSORRESOLUTION [F17M30][F17M70][F19M00](Deprecated)

This control sets a value for the new DDR4 DIMM temperature sensor resolution (Reg8).

Permitted Choices: (Type: Value)(Default: 1)

- **APCB_TOKEN_UID_AUTOREFFINEGRANMODE [F17M30][F19M00]**

This options sets DIMM Sensor fine grain mode which controls the refresh rate when the DIMM sensor detects a 'hot' condition.

Permitted Choices: (Type: Value)(Default: 0)

- 0 - use 1x, normal, refresh.
- 1 - use 2x, double, refresh rate.
- 2 - use 4x, quad, refresh rate.

- **APCB_TOKEN_UID_ODTSCMDTHROTEN [F17M30][F19M00]**
APCB_TOKEN_CONFIG_ODTSCMDTHROTEN [F17M00][F17M10][F17M20]

This options enables ODTs CMD Throttle.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE—
- FALSE—
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_ODTSCMDTHROTCYC [F17M30][F19M00]**
APCB_TOKEN_CONFIG_ODTSCMDTHROTCYC [F17M00][F17M10][F17M20]

This options sets ODTs command cycle throttling by adding a delay or 'stretch' to the clock cycle.

Permitted Choices: (Type: Value)(Default: Auto)

-
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_SWCMDTHROTEN [F17M30][F19M00]**
APCB_TOKEN_CONFIG_SWCMDTHROTEN [F17M00][F17M10][F17M20]

This option enables the memory channel SW CMD Throttle feature (see PPR: SWCmdThrotEn). APCB V2 platforms use token APCB_TOKEN_CONFIG_SWCMDTHROTEN which is set by modifying the value of

BLDCFG_SW_CMD_THROT_EN. APB V3 platforms use APB_TOKEN_UID_SWCMDTHROTEN which is set by modifying the value of APB_TOKEN_UID_SWCMDTHROTEN.

Permitted Choices: (Type: Boolean)(Default: False)

- TRUE—Enabled.
 - FALSE—Disabled.
 - Auto—ABL decides the best setting.
- **APB_TOKEN_CONFIG_SWCMDTHROTCYC [F17M00][F17M10][F17M20]
APB_TOKEN_UID_SWCMDTHROTCYC [F17M30][F19M00]**

This options sets SW command Throttle Cycle (see PPR: SwCmdThrotCyc). APB V2 platforms use token APB_TOKEN_CONFIG_SWCMDTHROTCYC which is set by modifying the value of BLDCFG_SW_CMD_THROT_CYCLE. APB V3 platforms use APB_TOKEN_UID_SWCMDTHROTCYC which is set by modifying the value of APB_TOKEN_UID_SWCMDTHROTCYC.

Permitted Choices: (Type: Value)(Default: 0)

This control is enabled by APB_TOKEN_CONFIG_SWCMDTHROTEN

- **APB_TOKEN_UID_FORCEPWRDOWNTHROTEN [F1730][F17M70][F19M00]
APB_TOKEN_CONFIG_FORCEPWRDOWNTHROTEN [F17M00][F17M10][F17M20]**

This options enables Force Power Down Throttle Enable.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE— feature is enabled.
 - FALSE— feature is turned off.
 - Auto—ABL decides the best setting.
- **APB_TOKEN_UID_DRAMDOUBLEREFRESHRATE [F17M30][F19M00]**
- This is a performance optimization setting and indicates whether or not to refresh the memories at twice the rate calculated or as indicated in the SPD information. This faster frequency refresh is likely to be employed in cases where extended temperature support is desired or smaller geometry DRAM technologies are utilized.
- Permitted Choices: (Type: List)(Default: 0)
- 0 —Use the standard refresh rate. {Example: 7.8us}
 - 1 —Double the refresh rate (cut period in half). {Example: 3.9us}

4.1.2.4 Memory Controller - Performance Category

This section describes the external parameters that can be used to control ABL execution. Please see “AgesaPkg\Addendum\%board%\Apcb\ApcbData_<Platform>_ExtParams.c”

- **APB_TOKEN_UID_MEMHOLEREMAPPING [F17M30][F19M00] APB_TOKEN_CONFIG_MEMHOLEREMAPPING**

This options enables or disables the Memory Hole Remapping feature. DRAM memory is contiguous from 0 through 4GB or more; however, for PCI compatibility, a 'hole' is created just under the 4GB address mark to allow 32-bit address devices to locate their MMIO region for register access. The actual DRAM memory 'under' this hole will be lost. The remapping feature recovers this memory by making it accessible at high

addresses beyond the end of physical memory. This feature is most valuable to system with smaller amounts of DRAM. See also the Bottom IO setting.

Permitted Choices: (Type: List)(Default: Enable)

- Enable—Memory under the PCI MMIO 'hole' is Remapped to high addresses.
- Disable—Memory under the PCI MMIO 'hole' is lost.
- Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_ENABLEBANKGROUPSWAPALT [F17M30][F19M00]**
APCB_TOKEN_CONFIG_ENABLEBANKGROUPSWAPALT [F17M00][F17M10][F17M20]

This is an advanced performance setting. It specifies if the system should use DRAM Bank Group Swap mode. The AMD recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: List)(Default: Enabled)

- Disabled
- Enabled
- Auto – ABL decides the best setting.

- **APCB_TOKEN_UID_ENABLEBANKGROUPSWAP [F17M30][F19M00] APCB_TOKEN_CONFIG_ENABLEBANKGROUPSWAP [F17M00][F17M10][F17M20]**

This is an advanced performance setting. It specifies if the system should use DRAM Bank Group Swap mode. The AMD recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: List)(Default: Enabled)

- Disabled
- Enabled
- Auto – ABL decides the best setting.

- **APCB_TOKEN_UID_DDRROUTEBALANCEDTEE [F17M30][F19M00]**

- **APCB_TOKEN_CONFIG_ENABLEZQRESET [F17M00]**

Enable CPU reset. This is no longer used in F17M30.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE—
- FALSE—
- Auto—ABL decides the best setting.

- **APCB_TOKEN_CBS_CMN_MEM_ADDR_CMD_PARITY_RETRY_DDR4 [F17M30][F19M00]**

Specifies if Command Parity Retry is to be attempted.

Permitted Choices: (Type: List)(Default: Disabled)

- Enabled
- Disabled
- Auto—ABL will decide the best setting.

- **APCB_TOKEN_CBS_DBG_MEM_ADDR_CMD_PARITY_ERROR_MAX_REPLAY_DDR4 [F17M00][F17M10][F17M20]**

Specifies the Command Parity error maximum number of retries. This option is only valid if the retry feature is enabled, see: APCB_TOKEN_CBS_DBG_MEM_ADDR_CMD_PARITY_RETRY_DDR4

Permitted Choices: (Type: Value)(Default: 0x08)

- 0x04..0x3F number of retries to attempt.
- Auto—ABL will decide the best setting.

- **APCB_TOKEN_UID_RCD_PARITY**

Specifies whether or not the memory controller should activate parity checking on RDIMMs. The feature is invoked only if the RDIMMs present are all capable of supporting the parity detection.

Permitted Choices: (Type: List)(Default: Enabled)

- Enabled
- Disabled
- Auto—ABL will decide the best setting.

- **APCB_TOKEN_UID_CBS_CMN_MEM_CTRLER_WR_CRC_EN_DDR4**

Specifies if Write CRC feature is to be enabled in the memory controller and should be attempted. CRC errors could cause fatal system error or be retried. The processor implements a cyclic redundancy check (CRC) on write data packets, as per the JEDEC DDR4 standard. If a CRC error occurs, the command is replayed. This feature provides detection and recovery for transient errors on the bus. Enablement of this feature is optional, since enabling has a performance impact, and the number of replay attempts is configurable.

Permitted Choices: (Type: List)(Default: Disabled)

- Enabled - ABL automatically enables address/command parity replay also.
- Disabled
- Auto—ABL will decide the best setting.

- **APCB_TOKEN_CBS_CMN_MEM_WRITE_CRC_RETRY_DDR4**

Specifies if Write CRC Retries are to be attempted.

Permitted Choices: (Type: List)(Default: Disabled)

- Enabled
- Disabled
- Auto—ABL will decide the best setting.

- **APCB_TOKEN_UID_CBS_CMN_MEM_WRITE_CRC_ERROR_MAX_REPLAY_DDR4**

Specifies the Write CRC error maximum number of replays (retries). This option is only valid if the retry feature is enabled. Once the retries are exhausted, a system fatal error will be logged in the MCA.

Permitted Choices: (Type: value)(Default: 0x08)

- 0x04..0x08 - number of retries to attempt.
- Auto—ABL will decide the best setting.

- **APCB_TOKEN_UID_MEM_DATA_POISON [F17M30][F17M70][F19M00]**
APCB_TOKEN_CONFIG_MEM_DATA_POISON [F17M00][F17M10][F17M20]

Specifies if the memory data poisoning feature should be turned on. Data poison should be enabled by default on systems that supports ECC DIMMs. Poisoning cannot be enabled on a non-ECC system.

Permitted Choices: (Type: List)(Default: Auto)

- Enabled
 - Disabled
 - Auto—ABL will decide the best setting.
- **APCB_TOKEN_CONFIG_ECCSYMBOLSIZE**

Specifies the size of the ECC validation.

Permitted Choices: (Type: List)(Default: x8 symbols)

For F17M30, F19M00 (Default: x16 symbols)

- x4 symbols
- x8 symbols
- x16 symbols - only available on F17M30, F19M00

- **APCB_TOKEN_UID_UECC_RETRY_DDR4 [F17M30][F19M00]**

This option enables retries on uncorrected ECC errors. This token will set retries on all channels, UMCs, dies, and sockets.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- FALSE - Disable retries
- TRUE - Perform a tr-try when an Uncorrectable ECC error occurs on a DDR4 channel.

- **APCB_TOKEN_UID_DF_INVERT_DRAM_MAP**

Enabling this token causes the highest memory channels to get assigned the lowest addresses in the system.

Permitted Choices (Type: Boolean) (Default: FALSE)

- FALSE - Disable memory map inversion.
- TRUE - Use high channels for low address memory.

- **APCB_TOKEN_UID_MEM_MAXACTIVATECOUNT [F17M30][F19M00]**

This item allows the user to limit the DDR4 DIMM activity. Normally this would be set by the DIMM SPD byte 7 [3:0] Maximum Activity Count (MAC) field.

Permitted Choices: (Type: List)(Default: 0xff)

- 0 = Untested MAC
- 1 = 700 K
- 2 = 600 K
- 3 = 500 K
- 4 = 400 K
- 5 = 300 K
- 6 = 200 K
- 8 = Unlimited MAC

- 0xff = Auto (use DIMM SPD Byte 7 bit [3:0])
- **APCB_TOKEN_UID_MEM_URGREFLIMIT [F17M30][F19M00]**
This item allow the user to customize refresh controls. Please see the PPR, ref: UMC::CH::SpazCtrl::UrgRefLimit. Some customers may choose to use these new options as override to {1,1} or {1,2} to reduce the likelihood of TRRespass type exploits.
Permitted Choices: (Type: value)(Default: 6)
 - 1..6 - see PPR for details.
- **APCB_TOKEN_UID_MEM_SUBURGREFLOWERBOUND [F17M30][F19M00]**
This item allow the user to customize refresh controls. Please see the PPR, ref: UMC::CH::SpazCtrl::SubUrgRefLowerBound. Note: SubUrgRefLowerBound MUST be less than or equal to UrgRefLimit per PPR documentation.
Permitted Choices: (Type: value)(Default: 4)
 - 1..6 - see PPR for details.

4.1.2.5 PHY Tuning & Control

These controls specify how the physical layer component (PHY) is to be configured.

- **APCB_TOKEN_UID_PMUTRAINMODE [F17M30][F19M00]**
APCB_TOKEN_CONFIG_PMUTRAINMODE [F17M00][F17M10][F17M20]
For processors which provide a Phy Micro-controller Unit (PMU), this parameter selects the memory data bus training mode which the PMU will perform.
Permitted Choices: (Type: List)(Default: PMU_TRAIN_AUTO)
 - PMU_TRAIN_1D — perform 1D training only
 - PMU_TRAIN_1D_2D_READ — perform both 1D and 2D read only training
 - PMU_TRAIN_1D_2D — perform both 1D and 2D training
 - PMU_TRAIN_AUTO — The training mode will be automatically selected based on configuration
- **APCB_TOKEN_UID_DISPLAY_PMU_TRAIN_RESULTS [F17M30][F19M00]**
- **APCB_TOKEN_UID_MEM_CTRLER_PMU_TRAIN_FFE_DDR4 [F19M00]**
FFE is a Transmit Data equalization in the PHY that applies a modified drive level to the DQ Lanes to improve the signal at the Dram. Customers may want to disable FFE on certain DIMMs that have lots of margin to reduce power consumption.
Permitted Choices: (Type: List)(Default: AUTO)
 - Auto - the power up default is used (enabled).
 - Enable - the FFE service is active.
 - Disable - the FFE service is stopped.
- **APCB_TOKEN_UID_MEM_CTRLER_PMU_TRAIN_DFE_DDR4 [F19M00]**
DFE is a Phy Hardware feature for the receive Data DQ to improve read margin. Customers may want to disable DFE on certain DIMMs that have lots of margin to reduce power consumption.

Permitted Choices: (Type: List)(Default: AUTO)

- Auto - the powerup default is used (enabled).
- Enable - the DFE service is active.
- Disable - the DFE service is stopped.

Data Eye Adjustments

These are PHY parameters that are programmed to adjust the eye and margin. These APCB tokens will over-ride the default xGMI vetting algorithm and might lead to higher BER in platforms if over-ride values are not approved by AMD.

These only take effect for certain parts: Only if the part is a 4-CCD part and the core count is ≤ 32 is when these values are used.

- **APCB_TOKEN_UID_DXIO_VGA_API_ENABLE**

This is the main switch that enables the Pole, VGA, DC, & IQOFC phy over-ride parameters.

Permitted choices: (Type: Boolean)(Default: FALSE)

- FALSE - The feature is disabled.
- TRUE - The below overrides are active.

- **APCB_TOKEN_UID_DXIO_PHY_PARAM_POLE**

This controls the capacitive degeneration.

Permitted choices: (Type: Value)(Default: 0xFFFFFFFF (skip))

- 0x00000000~0xFFFFFFFF = (UINT32) Valid Range.
- 0xFFFFFFFF = skip.

- **APCB_TOKEN_UID_DXIO_PHY_PARAM_VGA**

Controls the CTLE gain.

Permitted choices: (Type: Value)(Default: 0xFFFFFFFF (skip))

- 0x00000000~0xFFFFFFFF = (UINT32) Valid Range.
- 0xFFFFFFFF = skip.

- **APCB_TOKEN_UID_DXIO_PHY_PARAM_DC**

Controls register degeneration.

Permitted choices: (Type: Value)(Default: 0xFFFFFFFF (skip))

- 0x00000000~0xFFFFFFFF = (UINT32) Valid Range.
- 0xFFFFFFFF = skip.

- **APCB_TOKEN_UID_DXIO_PHY_PARAM_IQOFC**

Shift the phase on x-axis.

Permitted choices: (Type: Value)(Default: 0xFFFFFFFF (skip))

- -4 ~ 4 = (INT32) Valid Range.
- 0x7FFFFFFF = skip.

4.1.2.6 Data Fabric - Enablements Category

This section describes the external parameters that can be used to control ABL execution. Please see “AgesaPkg \Addendum\%board% \Apcb\ApcbData_<Platform>_ExtParams.c”

• APCB_TOKEN_UID_DF_MEM_INTERLEAVING

Selects the type of interleaving desired for the system. Memory interleaving is the process of spreading accesses across various memory components in the system. Interleaving tends to improve overall throughput by running operations in parallel to the different memory controllers. Interleaving is only available on systems with more than one memory controller with populated memory devices.

Permitted Choices: (Type: List)(Default: Auto)

- DF_MEM_INTLV_SOCKET - [F17M00][F17M10][F17M20] spread accesses across all populated memory devices on all sockets in the system.
- DF_MEM_INTLV_DIE - [F17M00][F17M10][F17M20] spread accesses across all populated memory devices within a socket.
- DF_MEM_INTLV_CHANNEL - [F17M00][F17M10][F17M20] spread accesses across all populated memory devices by channels.
- DF_MEM_INTLV_NONE - Large scale interleaving is not used. However, bank interleaving within a DIMM may still be active. See APCB_TOKEN_CONFIG_CONFIG_ENABLEBANKINTLV.
- Auto - [F17M30] Interleaving is based upon the NUMA topology per NUMA Node Per Socket (NPSx). See reference [Socket SP3 NUMA Topology](#)
- Auto – [F17M00][F17M10][F17M20] ABL decides the best setting according to this chart:

Table 15. Memory Interleaving

Package	Channel Interleave	Die Interleave	Socket Interleave	Address Hashing	Interleave Bit
AM4	1	n/a	n/a	1	8
SP3	1	0	0	1	8
SP3r2-clients	1	1	n/a	0 ¹	8
SP3r2-server	1	0	0	1	8
APU w/iGPU ²	1	n/a	n/a	0	8
APU w/o iGPU	1	n/a	n/a	1	8

• APCB_TOKEN_UID_DF_DRAM_NPS

For the above Memory Interleaving option, this token specifies the number of NUMA nodes per socket.

Permitted Choices (Type: List) (Default: NPS1)

- NPS0
- NPS1 - 1 NUMA node per socket.
- NPS2 - use 2 NUMA nodes per socket.
- NPS4 - use 4 NUMA nodes per socket.

• APCB_TOKEN_UID_DF_DRAM_INTERLEAVE_SIZE

[F17M00][F17M10][F17M20] Specifies the block size, in bytes, to use for interleaving.

¹ Hashing is enabled by default if die-interleaving is forced off by a APCB option.

² Doesn't matter if a dGPU is in use or not.

Permitted Choices: (Type: List)(Default: DF_DRAM_INTLV_SIZE_256)

- DF_DRAM_INTLV_SIZE_256
- DF_DRAM_INTLV_SIZE_512
- DF_DRAM_INTLV_SIZE_1024—not valid when socket or die interleave enabled.
- DF_DRAM_INTLV_SIZE_2048—not valid when socket or die interleave enabled.
- Auto—ABL decides the best setting. See APCB_TOKEN_CONFIG_CONFIG_DF_MEM_INTERLEAVING.

• **APCB_TOKEN_UID_DF_MEM_INTERLEAVING_SIZE**

[F17M30][F19M00] Specifies the starting address bit used for interleaving.

Permitted Choices: (Type: List)(Default: DF_MEM_INTLV_SIZE_256BYTES)

- DF_MEM_INTLV_SIZE_256BYTES
- DF_MEM_INTLV_SIZE_512BYTES
- DF_MEM_INTLV_SIZE_1KB. see note 2.
- DF_MEM_INTLV_SIZE_2KB. see note 2.
- DF_MEM_INTLV_SIZE_4KB. see notes 1 & 2. ([F19M00] only).
- DF_MEM_INTLV_SIZE_AUTO—ABL uses = DF_MEM_INTLV_SIZE_256BYTES.

Notes:

1. [F19M00]When 6-channel interleaving is in use, only DF_MEM_INTLV_SIZE_2KB and DF_MEM_INTLV_SIZE_4KB are supported, any other interleave size selection will be forced to DF_MEM_INTLV_SIZE_2KB.
2. [F17M30][F19M00] If DF_MEM_INTLV_SIZE_1KB or DF_MEM_INTLV_SIZE_2KB or DF_MEM_INTLV_SIZE_4KB is selected, channel interleaving hash will be disabled.

• **APCB_TOKEN_CONFIG_CONFIG_DF_ENABLE_CHAN_INTLV_HASH**

[F17M00]Only valid when channel interleaving is enabled. Not valid on APUs. Recommended for systems with Probe Filter enabled. Please see the notes for DF_MEM_INTERLEAVING above .

Permitted Choices: (Type: List)(Default: Enabled)

- Disabled
- Enabled
- Auto—ABL decides the best setting. See APCB_TOKEN_UID_DF_MEM_INTERLEAVING.

• **APCB_TOKEN_UID_DF_PROBE_FILTER_ENABLE**

Probe Filter Enable—fuse controls part capability

Permitted Choices: (Type: List)(Default: Enabled)

- Enabled
- Disabled
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_DF_MEM_CLEAR**

This option controls whether or not system DRAM contents will be initialized to zero. The clear operation will vary in time duration proportionate to the size of DRAM memory. Avoiding the clear, could reduce overall boot time. Note that if DRAM ECC is enabled, this token has no effect as the memory clear process is required.

Permitted Choices: (Type: List)(Default: Auto)

- Enabled - DRAM memory will be cleared to zeroes.
- Disabled - DRAM will not be cleared.
- Auto - ABL decides the best setting

• **APCB_TOKEN_UID_DF_GMI_ENCRYPT**

This options enables GMI encryption.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE—Enabled.
- FALSE—Disabled.
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_DF_XGMI_ENCRYPT**

This options enables XGMI encryption.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE—Enabled.
- FALSE—Disabled.
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_DF_SAVE_RESTORE_MEM_ENCRYPT**

Choice to activate the C6 / DF save / restore regions.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE—Enabled.
- FALSE—Disabled.
- Auto—ABL decides the best setting.

• **APCB_TOKEN_UID_MEM_TSME_ENABLE [F17M30][F17M70][F19M00] APCB_TOKEN_CONFIG_MEM_TSME_ENABLE [F17M00][F17M10][F17M20]**

This item Selects the operational mode for TSME (Transparent Secure Memory Encryption).

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - ForeceEnc and AddrTweakEn will be set (see volume 4 of the PPR)
- FALSE - This option is turned off.

See also: Secure Memory Encryption Extension (SMEE).

• **APCB_TOKEN_UID_MEM_DATA_SCRAMBLE [F17M30][F17M70][F19M00] APCB_TOKEN_CONFIG_MEM_DATA_SCRAMBLE [F17M00][F17M10][F17M20]**

This item specifies whether or not the UMC Data is scrambled.

Permitted Choices: (Type: List)(Default: Auto)

- Enabled - Data scrambling is enabled (DataScrambleEn will be set. See volume 4 of the PPR)
- Disabled - Data scrambling is disabled.
- Auto - The ABL will decide the best setting.

4.1.2.7 Data Fabric - Platform Category

This section describes the parameters that can be used to describe system wide platform controls.

- **APCB_TOKEN_UID_DF_BOTTOMIO**

Bottom of 32-bit PCI MMIO space (Addr[31:24]). This item sets the address where the PCI MMIO 'hole' will start. The setting choice depends on the quantity of PCI device resources needed in the 32-bit address space. There may be alignment restrictions imposed by a particular processor. The AGESA™ software will perform the required alignment necessary for the processor.

Permitted Choices: (Type: Value)(Default: 0xE0)

Known restrictions:

- Family 17h, Models 00h-0Fh: Only bits 31:28 should be used to match with TOM2 granularity.
- Family 17h, Models 10h-1Fh: Only bits 31:28 should be used to match with TOM2 granularity.

- **APCB_TOKEN_UID_DF_PCI_MMIO_SIZE**

Raw size in bytes of the space used for PCI cfg MMIO. Lower 4 nibbles must be 0.

Permitted Choices: (Type: Value)(Default: 0x10000000)

Auto—ABL decides the best setting.

- **APCB_TOKEN_UID_DF_SYS_STORAGE_AT_TOP_OF_MEM**

[F17M00][F17M10][F17M20]Controls whether or not the private memory regions (PSP, SMU and CC6) are at the top of last DRAM pair, the top of first DRAM pair or distributed. Privileged memory space will be reserved by UEFI BIOS to prohibit OS from using it, if customers wants to consolidate all reserved memory to a single piece, they can choose to use CONSOLIDATED options.

Permitted Choices: (Type: List)(Default: DF_SYS_STORAGE_AT_TOP_OF_MEM_DISTRIBUTED)

- **DF_SYS_STORAGE_AT_TOP_OF_MEM_DISTRIBUTED** - For a system with 8-CCDs and 8 memory channels with full memory populated (no NVDIMM)
 - The privileged memory space of CCD0 and CCD1 will be consolidated and located at top of specific memory region - region 0.
 - The privileged memory space of CCD2 and CCD3 will be consolidated and located at top of specific memory region - region 1.
 - and so on through CCD6 and CCD7.

This the best from performance point of view, because the memory resource may be more local to CCDs than when consolidated.

- **DF_SYS_STORAGE_AT_TOP_OF_MEM_CONSOLIDATED** - For the 8-CCDs/8 memory channels with full memory populated (no NVDIMM) case:

- All CCDs' privileged memory region will be consolidated and located at top of last dram region

This is best from the OS point of view in that it creates fewer reserved regions and allows the OS to more freely allocate resources.

- **DF_SYS_STORAGE_AT_TOP_OF_1ST_MEM_CONSOLIDATED** - consolidate all reserved/privileged memory in first dram pair.
- **DF_SYS_STORAGE_AT_TOP_OF_MEM_AUTO**.

- **APCB_TOKEN_UID_CCX_SEV_ASID_COUNT [F17M30][F19M00]**

Secure Encrypted Virtualization (SEV) uses Address Identifiers (ASIDs) to label the various Virtual Machines (VMs) running in the system. The VMs may be using different features. This token specifies the maximum valid ASID, which affects the maximum system physical address space. 16TB of physical address space is available for systems that support 253 ASIDs, while 8TB is available for systems that support 509 ASIDs.

Permitted Choices: (Type: List) (Default: 0x01 - 509 ASIDs)

- 0x00 - Use 253 ASIDs (16TB physical address space).
- 0x01 - Use 509 ASIDs (8TB physical address space).
- 0x03 - Auto mode - will prefer 509 ASIDs.

• **APCB_TOKEN_UID_CCX_MIN_SEV_ASID [F17M30][F19M00]**

This control specifies, per the PPR, the minimum ASID value that must be used for a guest that has SEV enabled, but SEV-ES disabled. SEV VMs using ASIDs below this value must enable the SEV-ES feature. ASIDs from `APCB_TOKEN_UID_CCX_MIN_SEV_ASID` to `(APCB_TOKEN_UID_CCX_SEV_ASID_COUNT + 1)` can only be used with SEV only VMs. If this token is set to `(APCB_TOKEN_UID_CCX_SEV_ASID_COUNT + 1)`, all ASIDs are forced to be SEV-ES ASIDs.

Permitted Choices: (Type: Value)(Default: 0x0001)

• **APCB_TOKEN_UID_CCX_PPIN_OPT_IN [F19M00]**

This control is used to Enable / Disable the Protected Processor Inventory Number (PPIN) feature. Please see the PPR description for and `CPUID_Fn80000008_EBX [23:PPIN]`.

• **APCB_TOKEN_UID_PCIE_RESET_CONTROL [F17M00]**

This value sets the ABL control of the PCIe® Reset signal. See [Firmware Managed PCIe® Resets](#).

- TRUE—Enables ABL & Firmware control of PCIe® reset. The PCIE_RESET lines (one per die) are forced active (low) by the ABL. The x86 PCI driver will release the reset once it is ready to initialize the device.
- FALSE— Use only the standard hardware delay on the PCIE_RESET signal.

• **APCB_TOKEN_UID_FCH_GPP_CLK_MAP [F17M30][F19M00]**

This control allows the user to disable the GPP clocks at a very early ABL boot stage. Multiple clocks can be specified.

Permitted Choices: (Type: list) (Default: GPP_ALL_CLK_AUTO)

- GPP_ALL_CLK_ON - used for the condition where some of the CPU GPP CLK has been fused (set to default as OFF) for the specific OPN; but, the customer wants ALL GPP CLK(s) to be ON.
- GPP_ALL_CLK_AUTO - Keep the power-on reset value.
- S0_GPP0_CLK_OFF
- S0_GPP1_CLK_OFF
- S0_GPP2_CLK_OFF
- S0_GPP3_CLK_OFF
- S0_GPP4_CLK_OFF
- S1_GPP0_CLK_OFF
- S1_GPP1_CLK_OFF
- S1_GPP2_CLK_OFF
- S1_GPP3_CLK_OFF

- S1_GPP4_CLK_OFF

Examples:

Example 1. Socket 0 GPP1 clock and Socket 0 GPP2 clock OFF
APCB_TOKEN_UID_FCH_GPP_CLK_MAP (S0_GPP1_CLK_OFF | S0_GPP2_CLK_OFF)

Example 2. Socket 0 GPP0 clock and Socket 1 GPP2 clock OFF
APCB_TOKEN_UID_FCH_GPP_CLK_MAP (S0_GPP0_CLK_OFF | S1_GPP2_CLK_OFF)

- **APCB_TOKEN_UID_GROUP_D_PLATFORM [F17M30][F19M00]**

APCB request to provide for Group D platform selection. This control tells the SMU to make the appropriate IRM adjustments for a 'Group D' platform. F17M30 platforms are group D platforms (TRUE); whereas, F17M00 platforms are not (FALSE). See the [IRM](#) for further details.

Permitted Choices: (Type: Boolean) (Default: FALSE)

- **APCB_TOKEN_UID_PSP_MEASURE_CONFIG [F17M30]**

Reserved feature. Must be =0.

Permitted Choices: (Type: Value)(Default: 0)

4.1.2.8 Data Fabric - Power Management Category

This section describes the parameters that can be used to control system wide power management.

-

4.1.2.9 Data Fabric - Performance Category

This section describes the external parameters that can be used to control ABL execution. Please see “AgesaPkg \Addendum\%board% \Apcb\ApcbData_<Platform>_ExtParams.c”

- **APCB_TOKEN_UID_DF_REMAP_AT_1TB**

Under certain conditions, problems have been found using IOMMU when system DRAM is present just below the 1TB address boundary. The solution to date has been to make an IOMMU hole by marking multiple GB of DRAM as reserved. This variable controls whether or not the ABL tries to remap memory just below the 1TB boundary to be above the boundary. This solution allows the full contingent of DRAM to be available. It is worth noting that remapping is not possible in certain memory configurations. If the region that crosses the problematic boundary is based at system address 0x0000, remapping is not possible. As an example, if DRAM interleaving is set to 'Die' and each socket has 1TB of DRAM, remapping is not possible.

Permitted Choices: (Type: Boolean)(Default: False)

- True—Memory under the 1TB IOMMU 'hole' is Remapped to high addresses above the 1TB boundary.
- False—Memory under the 1TB IOMMU 'hole' is lost.

- **APCB_TOKEN_UID_DF3_XGMI2_LINK_CFG**

On some multi-socket designs, it is possible to connect only 3 of the 4 xGMI2 links between the processors in a 2P configuration. This frees up those lanes for additional PCI connectivity at the cost of interprocessor bandwidth. If 3 links are incorporated on the motherboard, this token must be set to a raw value of 0x01. All other values will result in the 4-link settings.

Permitted Choices: (Type: List)(Default: Auto)

- DF_XGMI_LINK_CFG_AUTO - Auto determination will likely use 4 links.
- DF_XGMI_LINK_CFG_3XGMILINKS - use 3 xGMI Links
- DF_XGMI_LINK_CFG_4XGMILINKS - use 4 xGMI Links

- **APCB_TOKEN_UID_DF_3LINK_MAX_XGMI_SPEED**

The maximum xGMI2 speed for a 3-link system is determined by this token, in addition to hardware limitation.

Permitted Choices: (Type: List)(Default: 13 Gbps)

- 6.4 Gbps
- 7.467 Gbps
- 8.533 Gbps
- 9.6 Gbps
- 10.667 Gbps
- 11 Gbps
- 12 Gbps
- 13 Gbps
- 14 Gbps
- 15 Gbps
- 16 Gbps
- 17 Gbps
- 18 Gbps
- 19 Gbps
- 20 Gbps
- 21 Gbps
- 22 Gbps
- 23 Gbps
- 24 Gbps
- 25 Gbps

- **APCB_TOKEN_UID_DF_4LINK_MAX_XGMI_SPEED**

The maximum xGMI2 speed for a 4-link system is determined by this token, in addition to any hardware limitation.

Permitted Choices: (Type: List)(Default: 13 Gbps)

- 6.4 Gbps
- 7.467 Gbps
- 8.533 Gbps
- 9.6 Gbps
- 10.667 Gbps
- 11 Gbps
- 12 Gbps
- 13 Gbps
- 14 Gbps
- 15 Gbps
- 16 Gbps
- 17 Gbps

- 18 Gbps
- 19 Gbps
- 20 Gbps
- 21 Gbps
- 22 Gbps
- 23 Gbps
- 24 Gbps
- 25 Gbps
- **APCB_TOKEN_UID_DF_CAKE_CRC_THRESH_PERF_BOUNDS**

This options specifies the performance penalty the customer is willing to accept in order to enable inter-die link CRC detection / protection in 0.00001% units. The parameter is 32-bits in length with an allowable range of 0 - 1,000,000 (decimal) which corresponds to [disabled - 10%]. A value of zero disables the feature. If there are no GMI or xGMI links in the system (single die system), the value is ignored.

Permitted Choices: (Type: Value)(Default: 100 (0.001%))

- **APCB_TOKEN_UID_DF_INVERT_DRAM_MAP**

Enabling this token causes the highest memory channels to get assigned the lowest addresses in the system

Permitted Choices (Type: Boolean) (Default: Disabled)

4.1.3 SPD_Info (Data)

This section describes the SPD data that is required to describe the platform layout of the memory bus. The data structure is intended to be an instance declaration containing the values that best describe the platform. There must be one structure instance for each DIMM slot or on-board bank.

There are two types of structures, one for a DIMM slot and one for a memory-down bank. Since a memory-down implementation does not include an SPD, the information structure must contain a valid image of an SPD to describe the memory chip characteristics.

An example of this file can be found in “AgesaPkg\Addendum\Apcb\ApcbData_<Platform>_SpdInfo.c”

DIMM_INFO_ARRAY

```
typedef struct _DIMM_ARRAY {
    UINT8      NumberOfEntries;
    _DIMM_INFO SlotDescription[];
    _SPD_DEF_STRUCT MemDownBank[];
} DIMM_INFO_ARRAY;
```

This is the parent structure for the file containing memory SPD information.

NumberOfEntries

A count of the number of structures in the following array.

SlotDescription

An array of structures describing the memory banks, one structure per DIMM Slot or memory-down bank.

MemDownBank

An unsized array of description structures for memory-down implementations. This array may be empty; to conserve storage space, entries only need to exist when memory down banks are implemented. Entries only need to exist for those banks that are memory-down; a DIMM slot bank does not need an entry in this array. Additionally, two or more memory-down banks can index to the same SPD entry if they use identical memory chips.

DIMM_INFO_SMBUS

```
typedef struct _DIMM_INFO {
    UINT8  DimmSlotPresent;
    UINT8  SocketId;
    UINT8  ChannelId;
    UINT8  DimmId;
    UINT8  DimmSmbusAddress;
    UINT8  I2CMuxAddress;
    UINT8  MuxControlAddress;
    UINT8  MuxChannel
} DIMM_INFO_SMBUS;
```

This structure is used to describe a single DIMM slot.

<i>DimmSlotPresent</i>	Indicates the type of memory bank. • TRUE - this bank is a DIMM slot • FALSE - this bank is a memory-down bank.
<i>SocketId</i>	Indicates the socket Id (0..N), relative to the system, to which this bank is attached.
<i>ChannelId</i>	Indicates the channel Id (0..N), relative to the socket, to which this bank is attached.
<i>DimmId</i>	Indicates the DIMM Id (0..N), relative to the channel, to which this bank is attached.
<i>DimmSmbusAddress</i>	For DIMM slots, this is the Smbus address of the SPD device for the slot. For memory-down banks, this is the structure index that further describes the memory chips. See below.
<i>I2CMuxAddress</i>	There is a limited number of available SMBus addresses for SPD devices. For large systems that have more memory slots than the SMBus limit, an I2C MUX must be installed on the motherboard. This entry specifies the SMBus address where the I2C Mux can be accessed. If no mux devices is implemented, this value should be set to 0xFF.
<i>MuxControlAddress</i>	This is the index for the register within the MUX device that selects the branch to be activated. If no mux devices is implemented, this value should be set to 0xFF.
<i>MuxChannel</i>	This value specifies which branch within the mux device should be selected to access this DIMM. If no mux devices is implemented, this value should be set to 0xFF.

SPD_DEF_STRUCT

```
typedef struct _SPD_DEF_STRUCT {
    TECHNOLOGY_TYPE  Technology;
    UINT8            Data[512];
} SPD_DEF_STRUCT;
```

Systems that use soldered down memory need to provide the memory component operational characteristics that would normally be included on a DIMM within a Serial Presence Device (SPD). This structure emulates the SPD and also contains some platform description values that permit the ABL to simulate an SPD device. This is an additional structure used for the special case of a memory-down bank; memory chips soldered directly to the motherboard. There must be one structure instance for each memory-down bank. The array is indexed by the DimmSmbusAddress element in the _DIMM_INFO structure for the bank.

Parameters

Technology	Indicates the type of technology supported (DDR4_TECHNOLOGY).
Data	This section contains the SPD data. This data must match the JEDEC definition for the stated memory technology type.

4.1.4 ABL Platform Specific Overrides Configuration Items

This section describes the platform specific override values and the macros used to create them. Please see “AgesaPkg\Addendum\%board%\Apcb\ApcbData_<Platform>_PsoOverride.c”

This configuration settings file contains a structured set of macros that create a data table used by the ABL to understand the platform memory topology. Each macro is described like a function below. Each macro has a 'matching-conditions' front half and a 'set-of-values' second half. If the system conditions match the 'matching-conditions', then the 'set-of-values' are applied.

Pin Mapping

Most of these macros have a data element in the set-of-values portion described as a map which is used to describe how the motherboard layout connects the Memory Controller pins to DIMM pins. This is needed for the software to understand the connections. Please be aware that these 'maps' define the connections used on the platform. The individual macros will analyze the connection map and decide which controller outputs can be disabled, or tri-stated.

A map may consist of 4 or 8 entries depending on the pin class. Each entry in the array represents a controller output control, first entry is the first pin (#0). The value of the entry describes to which DIMM pin(s) that memory controller output is connected. The value is bit-mapped meaning that each bit in the value represents a pin on the DIMM, thus allowing one controller output to manipulate one or more DIMM pins. LSb is the first DIMM pin (#0). The values should never indicate that any individual DIMM pin is connected to more than one controller output. Meaning that bit N in the values should =1 in only one entry.

Table 16. Pin Maps

Controller Pin	Map index	DIMM_xx_3	DIMM_xx_2	DIMM_xx_1	DIMM_xx_0	Entry value
Output_xx_00	0	no connect	no connect	no connect	connected	0x01
Output_xx_01	1	no connect	no connect	connected	no connect	0x02
Output_xx_02	2	no connect	connected	no connect	no connect	0x04
Output_xx_03	3	connected	no connect	no connect	no connect	0x08

Example 1: The platform has a single controller pin driving two DIMM pins. In this case Ctlr_xx_00 drives DIMM_xx_00 and DIMM_xx_02; while Ctlr_xx_01 drives DIMM_xx_01 and DIMM_xx_03:

Table 17. Pin Map Example 1

Controller Pin	Map index	DIMM_xx_3	DIMM_xx_2	DIMM_xx_1	DIMM_xx_0	Entry value
Output_xx_00	0	no connect	connected	no connect	connected	0x05
Output_xx_01	1	connected	no connect	connected	no connect	0x0A
Output_xx_02	2	no connect	no connect	no connect	no connect	0x00
Output_xx_03	3	no connect	no connect	no connect	no connect	0x00

The platform will specify this with the macro:

```
ODT_TRI_MAP(ANY_SOCKET, ANY_CHANNEL, 0x05, 0x0A, 0x00, 0x00)
```

Example 2: The platform has special routing needs and has not used controller pins sequentially from 0. In this case

- M[B,A]_CLK_H/L[4] --> DIMM CK0
- M[B,A]_CLK_H/L[2] --> DIMM CK1
- M[B,A]_CLK_H/L[3] --> DIMM CK2
- M[B,A]_CLK_H/L[5] --> DIMM CK3

Table 18. Pin Map Example 2

Controller Pin	Map index	DIMM_CK_3	DIMM_CK_2	DIMM_CK_1	DIMM_CK_0	Entry value
Output_xx_00	0	no connect	no connect	no connect	no connect	0x00
Output_xx_01	1	no connect	no connect	no connect	no connect	0x00
Output_xx_02	2	no connect	no connect	connected	no connect	0x02
Output_xx_03	3	no connect	connected	no connect	no connect	0x04
Output_xx_04	4	no connect	no connect	no connect	connected	0x01
Output_xx_05	5	connected	no connect	no connect	no connect	0x08
Output_xx_06	6	no connect	no connect	no connect	no connect	0x00
Output_xx_07	7	no connect	no connect	no connect	no connect	0x00

The platform will specify this with the macro:

```
MEMCLK_DIS_MAP(ANY_SOCKET, ANY_CHANNEL, 0x00, 0x00, 0x02, 0x04, 0x01, 0x08,  
0x00, 0x00)
```

4.1.4.1 MEMCLK_DIS_MAP (macro)

Summary

Macro is used to designate which memory clocks will be disabled.

Prototype

```
MEMCLK_DIS_MAP (SocketID, ChannelID,  
Bit0Map, Bit1Map, Bit2Map, Bit3Map,  
Bit4Map, Bit5Map, Bit6Map, Bit7Map)
```

Parameters

SocketID

Matching-Condition: Mask indicating the physical processor socket or sockets to which this control should be applied.

Possible values are:

- SOCKET0
- SOCKET1
- SOCKET2
- SOCKET3
- SOCKET4
- SOCKET5
- SOCKET6
- SOCKET7
- ANY_SOCKET

These values may be combined as follows: SOCKET0 + SOCKET2

ChannelID

Matching-Condition: A mask indicating the physical channel to which this control should be applied. Possible values are:

- CHA
- CHB
- CHC
- CHD
- ANY_CHANNEL

These values may be combined as follows: CHA + CHC

Bit0Map...Bit7Map

These bitmaps indicate how the memory clock pins connect to the DIMM ranks. See the map description in the main PSO chapter.

Description

The number of clock signals produced by the controller is stated in the BKDG. Disabling a clock signal may save power. Example pin names:

- M[B,A]_CLK_H/L[0]
- M[B,A]_CLK_H/L[1]
- M[B,A]_CLK_H/L[2]
- M[B,A]_CLK_H/L[3]
- M[B,A]_CLK_H/L[4]
- M[B,A]_CLK_H/L[5]
- M[B,A]_CLK_H/L[6]
- M[B,A]_CLK_H/L[7]

4.1.4.2 MEMCLK_CKE_TRI_MAP (macro)**Summary**

This macro is used to describe which memory clock signals should be tri-stated.

Prototype

```
CKE_TRI_MAP (SocketID, ChannelID, Bit0Map, Bit1Map, Bit2Map, Bit3Map)
```

Parameters

SocketID

Matching-Condition: Mask indicating the physical processor socket or sockets to which this control should be applied.

Possible values are:

- SOCKET0
- SOCKET1
- SOCKET2
- SOCKET3
- SOCKET4
- SOCKET5
- SOCKET6
- SOCKET7
- ANY_SOCKET

These values may be combined as follows: SOCKET0 + SOCKET2

ChannelID

Matching-Condition: A mask indicating the physical channel to which this control should be applied. Possible values are:

- CHA
- CHB
- CHC
- CHD
- ANY_CHANNEL

These values may be combined as follows: CHA + CHC

Bit0Map...Bit3Map

These bitmaps indicate how the memory clock pins connect to the DIMM ranks for the selected channel(s). See the map description in the main PSO chapter.

Description

The number of clock signals produced by the controller is stated in the BKDG. Placing a signal in Tri-state status may save power.

4.1.4.3 ODT_TRI_MAP (macro)**Summary**

This macro defines which ODT signals go to which DIMM pins.

Prototype

```
ODT_TRI_MAP (SocketID, ChannelID, Bit0Map, Bit1Map, Bit2Map, Bit3Map)
```

Parameters**SocketID**

Matching-Condition: Mask indicating the physical processor socket or sockets to which this control should be applied.

Possible values are:

- SOCKET0
- SOCKET1
- SOCKET2
- SOCKET3

- SOCKET4
- SOCKET5
- SOCKET6
- SOCKET7
- ANY_SOCKET

These values may be combined as follows: SOCKET0 + SOCKET2

ChannelID

Matching-Condition: A mask indicating the physical channel to which this control should be applied. Possible values are:

- CHA
- CHB
- CHC
- CHD
- ANY_CHANNEL

These values may be combined as follows: CHA + CHC

Bit0Map...Bit3Map

These bitmaps indicate how the controller ODT signals connect to the DIMM ranks.

Description

The number of On-Die Termination (ODT) signals produced by the controller is stated in the BKDG. Setting unused ODT signals to tri-state status may save power.

4.1.4.4 CS_TRI_MAP (macro)

Summary

This macro is used to designate how the memory CS signals are connected to the DIMM CS pins.

Prototype

```
CS_TRI_MAP (SocketID, ChannelID,
            Bit0Map, Bit1Map, Bit2Map, Bit3Map,
            Bit4Map, Bit5Map, Bit6Map, Bit7Map)
```

Parameters

SocketID

Matching-Condition: Mask indicating the physical processor socket or sockets to which this control should be applied.

Possible values are:

- SOCKET0
- SOCKET1
- SOCKET2
- SOCKET3
- SOCKET4
- SOCKET5
- SOCKET6
- SOCKET7
- ANY_SOCKET

These values may be combined as follows: SOCKET0 + SOCKET2

ChannelID

Matching-Condition: A mask indicating the physical channel to which this control should be applied. Possible values are:

- CHA
- CHB
- CHC
- CHD
- ANY_CHANNEL

Bit0Map...Bit7Map

These values may be combined as follows: CHA + CHC

These bitmaps indicate how the controller CS signals connect to the DIMM memory chip-selects. See the map description in the main PSO chapter.

Description

The number of chip select(CS) signals produced by the controller is stated in the BKDG. Setting unused chip selects signals to tri-state status may save power.

4.1.4.5 NUMBER_OF_DIMMS_SUPPORTED (macro)**Summary**

This macro defines how many DIMM slots the platform has designed onto the motherboard.

Prototype

```
NUMBER_OF_DIMMS_SUPPORTED (SocketID, ChannelID, NumberOfDimmSlotsPerChannel)
```

Parameters**SocketID**

Mask indicating the physical processor socket or sockets to which this control applies. Possible values are:

- SOCKET0
- SOCKET1
- SOCKET2
- SOCKET3
- SOCKET4
- SOCKET5
- SOCKET6
- SOCKET7
- ANY_SOCKET

These values may be combined as follows:
SOCKET0 + SOCKET2

ChannelID

A mask indicating the physical channel to which this control applies. Possible values are:

- CHA
- CHB
- CHC
- CHD
- ANY_CHANNEL

These values may be combined as follows: CHA
+ CHC

NumberOfDimmSlotsPerChannel

Specifies the number of DIMM slots per channel.

Description

This macro usually exists in the platform options file to specify the number of DIMM slots per channel in socket/channel basis. On a platform with soldered down DRAM only, SODIMM only, or soldered-down DRAM plus SODIMM, this macro will be used to specify total number of DIMMs supported on a channel, including slotted and soldered down DIMMs.

4.1.5 ABL Platform Timing Overrides Configuration Items

This section describes a set of macros used to do fine tuning of the memory timing parameters. They are divided into the following categories:

- Enablement
- Performance
- Power
- Platform
- Memory Discovery
- UMC Configuration
- PHY Configuration
- Training Inputs

These settings are considered an advanced topic and should only be used by those that fully understand their effects. The preferred method for customers to specify timing settings is through the [APCB v3](#) structures. These macros are not supported for the APCB v2 implementation.

Please see an example in “AgesaPkg\Addendum\%board% \Apcb \ApcbData_<Platform>_GID_0x1704_Type_PlatformTuningEntry.c”. This configuration settings file contains a structured set of macros that will create a data table used by the ABL to understand the platform memory topology. The macros are categorized by when the target timing value of the macro is to be altered; before or after the training process is complete.

These macros will apply new, or adjust present, settings when the current system configuration matches conditions indicated by the matching condition parameters. Not all macros will use all of the matching condition parameters. See the specific macro definition for which condition parameters apply. This allows separate values to be applied, for example, when one DIMM is populated on the channel than when there are two DIMMs populated on the channel.

These Timing macro (PTO) settings may co-exist with the Platform Specific Overrides (PSO) setting described in another chapter. The PTO settings will always take precedence.

At this time, only F17M30 supports these PTO macros.

Matching Condition Parameters

- SocketBitMap - A bit map that specifies to which CPU socket(s) this value is to be applied

Valid definitions:

- ANY_SOCKET - e.g. all sockets
- SOCKET0
- SOCKET1

- ChannelBitMap - A bit map that specifies to which memory channel(s) this value is to be applied. Note: multiple values can be specified by using an 'or' function: "(CHANNEL_A | CHANNEL_G)".

Valid definitions:

- ANY_CHANNEL
- CHANNEL_A
- CHANNEL_B
- CHANNEL_C
- CHANNEL_D
- CHANNEL_E
- CHANNEL_F
- CHANNEL_G
- CHANNEL_H
- DimmPerCh - To specify what kind of Memory DIMM population this value should be applied. Note, this is statement about the motherboard design, not the current population of DIMMs.

Valid definitions:

- 1 - One DIMM per channel
- 2 - Two DIMM per Channel
- SpeedBitMap - A bit map that specifies to which memory speed(s) this value is to be applied. Note: multiple values can be specified by using an 'or' function: "(DDR400 | DDR533)".

Valid definitions:

- ANY_SPEED
- DDR400
- DDR533
- DDR667
- DDR800
- DDR1066
- DDR1333
- DDR1600
- DDR1866
- DDR2133
- DDR2400
- DDR2667
- DDR2933
- DDR3200
- Width - To specify the memory device width for the value.

Valid definitions:

- 4 - Memory device width x4.
- 8 - Memory device width x8.
- 16 - Memory device width x16.
- 0xff - Any width device.
- VddIo - To specify the memory operating voltage being used when this value is to be applied.

Valid definitions:

- V1_2 - 1.200 volts
- Dimm0Rank Dimm1Rank - These two items specify the Rank type of the DIMM for which this value is to be applied.

Valid definitions:

- NP - Not populated
- DIMM_SR - Single Rank
- DIMM_DR - Dual Rank
- DIMM_QR - Qual Rank
- DIMM_LR - LRDIMM
- 0xFF - ignore Rank type (match any).
- MemPstate - To specify in which Memory Pstate this value is to be applied.

Valid definitions:

- MEMORY_PSTATE0
- MEMORY_PSTATE1
- MEMORY_PSTATE2
- MEMORY_PSTATE3
- 0xFF - ignore Pstate (match any).
- Dimm - To specify which DIMM is to have this value applied.

Valid definitions:

- 0 - DIMM 0
- 1 - DIMM 1
- DimmBitMap - To specify which DIMM(s) is to have this value applied. Note: multiple values can be specified by using an 'or' function: "(DIMM_0 | DIMM_3)".

Valid definitions:

- DIMM_0
- DIMM_1
- DIMM_2
- DIMM_3

Action Parameters

- Value - This parameter provides the numeric value for the target timing element. The *type* of the value is element dependent and may be Boolean, numeric or an address. See the specific macro definition for *type* and unit-of-measure information.
- Offset - For SPD data override, it specifies which SPD data offset is going to receive the value.

Valid definitions:

0 to 511 - For memory training, it specifies which PMU message block data offset is going to receive the value.

Valid definitions:

Depending on PMU message data block definitions

- Operation - To specify the operation for how to apply the value.

Valid definitions:

- OP_MINUS - Value is to be subtracted from the current value.

- OP_PLUS - Value is to be added to the current value.
- OP_OVERRIDE - Value is to replace the current value.
- Byte - To specify which byte lane will receive the value.

Valid definitions:

0 to 8 - Byte 0 to Byte 7 are data bytes - Byte 8 is ECC byte if applicable.

- Nibble - To specify which Nibble will receive the value.

Valid definitions:

0 to 1 - Nibble 0 or Nibble 1

- Dq - To specify which Dq will receive the value.

Valid definitions

: 0 to 8

4.1.5.1 Enablement Category

These macros will be evaluated and applied before the memory training process occurs.

Interleaving by Chipselect	Macro control to enable or disable Chipselect Interleaving. MAKE_PTO_CHIPSEL_INTLV_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_UID_ENABLECHIPSELECTINTLV or APCB_TOKEN_CONFIG_CONFIG_ENABLEBANKINTLV depending upon processor.
ECC feature for DRAM channels	Macro control to enable or disable the Dram ECC function. MAKE_PTO_ECC_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_UID_ENABLEECCFEATURE or APCB_TOKEN_CONFIG_CONFIG_ENABLEECCFEATURE depending upon processor.
Parity for Dram channels	Macro control to enable or disable Dram Parity function. MAKE_PTO_PARITY_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_UID_ENABLEPARITY or APCB_TOKEN_CONFIG_CONFIG_ENABLEPARITY depending upon processor
Mbist feature	Macro control to enable or disable of the Mbist feature. MAKE_PTO_MBIST_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean)

	• TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by BLDCFG_MEM_MBIST_TEST_ENABLE
Mbist Test Mode	Macro control to the operational mode of the MBIST test feature. MAKE_PTO_MBIST_TEST_MODE_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: enum) • action when not specified: state defined by BLDCFG_MEM_MBIST_TESTMODE • enum values - see the related token.
Mbist Aggressor	Macro control to aggressive traffic during the MBIST test. MAKE_PTO_MBIST_AGGRESSOR_ON_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by BLDCFG_MEM_MBIST_AGGRESSOR_ON
Mbist report per bit slave	Macro control to . MAKE_PTO_MBIST_PER_BIT_SLAVE_DIE_REPORT_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_CONFIG_MEM_MBIST_PER_BIT_SLAVE_DIE_REPORT
Post Package Repair	Macro control to support for the DDR4 feature for Post Package Repairs. MAKE_PTO_POST_PACKAGE_REPAIR_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_CBS_DBG_MEM_CTRLER_POST_PACKAGE_REPAIR_EN_DDR4
Post Package Repair All	Macro control to whether or not all banks of a DIMM are to be repaired. MAKE_PTO_POST_PACKAGE_REPAIR_ALL_BANKS_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_CBS_DBG_MEM_CTRLER_POST_PACKAGE_REPAIR_ALL_BANKS_DDR4

4.1.5.2 Performance Category

These macros will be evaluated and applied before the memory training process occurs.

Bank Swap by groups	Macro control to group level bank swapping. MAKE_PTO_BANK_GROUP_SWAP_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_CONFIG_ENABLEBANKGROUPSWAP
Command Throttling for ODTs	Macro control to how often commands may be sent. MAKE_PTO_ODTS_CMD_THROTTLE_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_UID_ODTSCMDTHROTEN.
ECC Symbol Size	Macro control to set the symbol size for the ECC feature. MAKE_PTO_ECC_SYMBOL_SIZE_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: number) • 0 - x4 • 1 - x8 • 2 - x16 • action when not specified: state defined by APCB_TOKEN_UID_ECCSYMBOLSIZE.
Retry Uncorrectable Error	Macro control to . MAKE_PTO_DDR4_UECC_RETRY_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_UID_UECC_RETRY_DDR4.
CRC for DDR4 writes	Macro control to enable CRC checking by the controller on all write operations. MAKE_PTO_WR_CRC_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, Value) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active. • FALSE - This option is turned off. • action when not specified: state defined by APCB_TOKEN_CBS_DBG_MEM_CTRLER_WR_CRC_EN_DDR4
CRC Retries of Writes	Macro control to whether or not retries will be replayed when a write CRC error is detected. MAKE_PTO_WR_CRC_RETRY_ENTRY(SocketBitMap, ChannelBitMap, Enable) Permitted Value Choices: (Type: Boolean) • TRUE - This option is active.

- FALSE - This option is turned off.
- action when not specified: state defined by
APCB_TOKEN_CBS_DBG_MEM_WRITE_CRC_RETRY_DDR4

Parity Retries Macro control to retries when a parity error is detected.
MAKE_PTO_ADDR_CMD_PARITY_RETRY_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: Boolean)

- TRUE - This option is active.
- FALSE - This option is turned off.
- action when not specified: state defined by
APCB_TOKEN_CBS_DBG_MEM_ADDR_CMD_PARITY_RETRY_DDR4

Parity for RCD DIMMs Macro control to enable the parity checking feature on registered DDR4 DIMMs.
MAKE_PTO_RCD_PARITY_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: Boolean)

- TRUE - This option is active.
- FALSE - This option is turned off.
- action when not specified: state defined by
APCB_TOKEN_CBS_DBG_MEM_RCD_PARITY_DDR4

Sensor use Fine Grain mode Macro control to how the DIMM sensor operates.
MAKE_PTO_AUTO_REFFINE_GRAN_MODE_ENTRY(SocketBitMap, ChannelBitMap, Value)
Permitted Value Choices: (Type: value) Values and action when this PTO control is not specified are defined by APCB_TOKEN_UID_AUTOREFFINEGRANMODE or APCB_TOKEN_CONFIG_AUTOREFFINEGRANMODE depending upon processor.

4.1.5.3 Power Category

These macros will be evaluated and applied before the memory training process occurs.

Power Down of DRAM Macro control to set the power saving state of the DIMMs.
MAKE_PTO_ENABLE_POWER_DOWN_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: Boolean)

- TRUE - This option is active.
- FALSE - This option is turned off.
- action when not specified: state defined by
[APCB_TOKEN_UID_ENABLEPOWERDOWN](#).

Power Down Mode Macro control to the method of granularity used for power down.
MAKE_PTO_ENABLE_POWER_DOWN_MODE_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: enum)

- action when not specified: state defined by [APCB_TOKEN_UID_POWERDOWNMODE](#).
- enum settings - see the definition of the related token.

- Clocks - all on** Macro control to specify that all MEMCLKs should be left enabled.
MAKE_PTO_ENABLE_TUNING_MEMORY_ALL_CLOCKS_ON_ENTRY(SocketBitMap, ChannelBitMap, Value)
Permitted Value Choices: (Type: Boolean)
- TRUE - This option is active.
 - FALSE - This option is turned off.
 - action when not specified: state defined by [APCB_TOKEN_UID_MEMORYALLCLOCKSON](#).
- Refresh by Temperature** Macro control to enable the temperature sensing for the DIMMs.
MAKE_PTO_TEMPERATURE_CONTROLLED_REFRESH_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, Value)
Permitted Value Choices: (Type: Boolean)
- TRUE - This option is active.
 - FALSE - This option is turned off.
 - action when not specified: state defined by [APCB_TOKEN_UID_MEM_TEMP_CONTROLLED_REFRESH_EN](#).

4.1.5.4 Platform Category

These macros will be evaluated and applied before the memory training process occurs.

- Speed limit of memory bus** Macro control to apply a cap on the memory bus frequency.
MAKE_PTO_MEM_BUS_LIMIT_ENTRY(SocketBitMap, ChannelBitMap, Value)
Permitted Value Choices: (Type: number, in MHz)
- action when not specified: state defined by [APCB_TOKEN_CONFIG_MEMORYBUSFREQUENCYLIMIT](#)
- Timing - Command Bus** Macro control to specify timings on the Dram Command Bus.
MAKE_PTO_CMD_BUS_TMGM_ENTRY(SocketBitMap, ChannelBitMap, DimmPerCh, SpeedBitMap, Width, VddIo, Dimm0Rank, Dimm1Rank, SlowMode, AddrCmdCtrl, CkeStrength, CsOdtStrength, AddrCmdStrength, ClkStrength)
- SlowMode** Permitted Choices: (Type: Boolean)
- TRUE - This option is active.
 - FALSE - This option is turned off.
- AddrCmdCtrl** This control allows fine tuning of the bus, please see the PPR section "DDR Command and Address Bus Configuration" for more details.
Permitted Choices: (Type: bit-packed value)
- [21:16] AddrCmdSetup : 0 ~ 0x3F
 - [13:8] CsOdtSetup : 0 ~ 0x3F
 - [5:0] CkeSetup : 0 ~ 0x3F
- CkeStrength**
CsOdtStrength These controls allow fine tuning of the PHY bus, please see the PPR section "Phy Drive Strength Programming" for more details.
Permitted Choices: (Type: value)

- AddrCmdStrength** • 00000 = 120.0 Ohm
- ClkStrength** • 00001 = 60.0 Ohm
- 00011 = 40.0 Ohm
- 00111 = 30.0 Ohm
- 01111 = 24.0 Ohm
- 11111 = 20.0 Ohm

Timing - Data Bus

Macro control to specify timings on the DRAM data bus.

MAKE_PTO_DATA_BUS_TM_ENTRY(SocketBitMap, ChannelBitMap, DimmPerCh, SpeedBitMap, Width, VddIo, Dimm0Rank, Dimm1Rank, RttNom, RttWr, RttPark, DqDrvStrength, DqsDrvStrength, OdtDrvStrength)

RttNom This and the following controls are dependent upon the memory DIMMs present and are described in the PPR section on "DDR Data Bus Configuration".

Permitted Choices: (Type: value)

Bits[2:0]:

- 0 0 0 RTT_NOM Disable
- 0 0 1 RZQ/4
- 0 1 0 RZQ/2
- 0 1 1 RZQ/6
- 1 0 0 RZQ/1
- 1 0 1 RZQ/5
- 1 1 0 RZQ/3
- 1 1 1 RZQ/7

RttWr Permitted Choices: (Type: value)

Bits[2:0]:

- 0 0 0 Dynamic ODT Off
- 0 0 1 RZQ/2
- 0 1 0 RZQ/1
- 0 1 1 Hi-Z
- 1 0 0 RZQ/3
- 1 0 1 Reserved
- 1 1 0 Reserved
- 1 1 1 Reserved

RttPark Permitted Choices: (Type: value)

Bits[2:0]:

- 0 0 0 RTT_PARK Disable
- 0 0 1 RZQ/4
- 0 1 0 RZQ/2
- 0 1 1 RZQ/6
- 1 0 0 RZQ/1

- 1 0 1 RZQ/5
- 1 1 0 RZQ/3
- 1 1 1 RZQ/7

DqDrvStrength Permitted Choices: (Type: value)

Bits[5:0]:

- 000000 - HiZ.(High Impedance)
- 000001 - 480.0
- 000010 - 240.0
- 000011 - 160.0
- 000110 - 120.0
- 000111 - 96.0
- 001010 - 80.0
- 001011 - 68.6
- 001110 - 60.0
- 001111 - 53.3
- 011010 - 48.0
- 011011 - 43.6
- 011110 - 40.0
- 111010 - 34.3
- 111011 - 32.0
- 111110 - 30.0
- 111111 - 28.2

DqsDrvStrength Permitted Choices: (Type: value)

Bits[5:0]:

- 000000 - HiZ.(High Impedance)
- 000001 - 480.0
- 000010 - 240.0
- 000011 - 160.0
- 000110 - 120.0
- 000111 - 96.0
- 001010 - 80.0
- 001011 - 68.6
- 001110 - 60.0
- 001111 - 53.3
- 011010 - 48.0
- 011011 - 43.6
- 011110 - 40.0
- 111010 - 34.3
- 111011 - 32.0
- 111110 - 30.0
- 111111 - 28.2

OdtDrvStrength Permitted Choices: (Type: value)

- 000_00_0 = High Impedance
- 000_00_1 = 480 ohms
- 000_01_0 = 240 ohms
- 000_01_1 = 160 ohms
- 001_00_0 = 120 ohms
- 001_00_1 = 96.0 ohms
- 001_01_0 = 80.0 ohms
- 001_01_1 = 68.6 ohms
- 011_00_0 = 60.0 ohms
- 011_00_1 = 53.3 ohms
- 011_01_0 = 48.0 ohms
- 011_01_1 = 43.6 ohms
- 111_00_0 = 40.0 ohms
- 111_00_1 = 36.9 ohms
- 111_01_0 = 34.3 ohms
- 111_01_1 = 32.0 ohms
- 111_11_0 = 30.0 ohms

4.1.5.5 Memory Discovery Category

These macros will be evaluated and applied before the memory training process occurs.

**SPD entry
override**

Macro control to perform a soft modification of the SPD data values. This macro may have multiple instances to make modifications to multiple SPD bytes.

MAKE_PTO_SPD_VALUE_OVERRIDE_ENTRY(SocketBitMap, ChannelBitMap, DimmBitMap,
Offset, Value)

Permitted Offset Choices: (Type: number)

- range: 0..511
- action when not specified: state defined by

Permitted Value Choices: (Type: number)

- see Jedec DDR4 specification for value range and units of measure.
- action when not specified: state defined by

**SPD
Checksum
Ignored**

Macro control to bypass faulty SPD checksums.

MAKE_PTO_SPD_CRC_IGNORE_ENTRY(SocketBitMap, ChannelBitMap, Dimm, Value)

Permitted Value Choices: (Type: Boolean)

- TRUE - This option is active.
- FALSE - This option is turned off.
- action when not specified: state defined by
[APCB_TOKEN_UID_IGNORESPDCHECKSUM](#).

4.1.5.6 UMC Category

These macros will be evaluated and applied to the memory controller timing registers. These controls set values familiar to those who deal with memory timing details and won't be further defined here. Please see the PPR and/or the applicable JEDEC specs.

Values set BEFORE training begins

ODT pattern - CS0	MAKE_PT0_ODT_PAT_CTRL_CS0_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value32)
ODT pattern - CS1	MAKE_PT0_ODT_PAT_CTRL_CS1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value32)
ODT pattern - CS2	MAKE_PT0_ODT_PAT_CTRL_CS2_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value32)
ODT pattern - CS3	MAKE_PT0_ODT_PAT_CTRL_CS3_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value32)
Impedance - TX EQ DQ	MAKE_PT0_TX_EQ_IMPEDANCE_DQ_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Impedance - TX DQ	MAKE_PT0_TX_IMPEDANCE_DQ_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Impedance - TX EQ HI DQ	MAKE_PT0_TX_EQ_HI_IMPEDANCE_DQ_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Drive Strength - TX ODT	MAKE_PT0_TX_ODT_DRV_STREN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Gear Down	MAKE_PT0_GEAR_DOWN_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Timing - 2T commands	MAKE_PT0_CMD_2T_ENABLE_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Timing - Trcdwr	MAKE_PT0_TRCDWR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trcdrd	MAKE_PT0_TRCDRD_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tras	MAKE_PT0_TRAS_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tcl	MAKE_PT0_TCL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trp	MAKE_PT0_TRP_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trtp	MAKE_PT0_TRTP_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trddlr	MAKE_PT0_TRRDDLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trrdl	MAKE_PT0_TRRDL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trrds	MAKE_PT0_TRRDS_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tfawdlr	MAKE_PT0_TFAWDLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tfawslr	MAKE_PT0_TFAWSLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)

Timing - Tfaw	MAKE_PTO_TFAW_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twtrl	MAKE_PTO_TWTRL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twtrs	MAKE_PTO_TWTRS_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tcwl	MAKE_PTO_TCWL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twr	MAKE_PTO_TWR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trcpage	MAKE_PTO_TRCPAGE_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trfc	MAKE_PTO_TRFC_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value16InMemClk)
Timing - Trfc2	MAKE_PTO_TRFC2_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value16InMemClk)
Timing - Trfc4	MAKE_PTO_TRFC4_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value16InMemClk)
Timing - Tstag	MAKE_PTO_TSTAG_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tstaglr	MAKE_PTO_TSTAGLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tstag2lr	MAKE_PTO_TSTAG2LR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tstag4lr	MAKE_PTO_TSTAG4LR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trdrdscdlr	MAKE_PTO_TRDRDSCDLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrwrscdlr	MAKE_PTO_TWRWRSCDLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrrdscdlr	MAKE_PTO_TWRRDSCDLR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)

Values applied AFTER training has completed

Control - DQ/DQS Receiver	MAKE_PTO_DQ_DQS_RCV_CTRL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Byte, Nibble, Value16)
Timing - Trdrdban	MAKE_PTO_TRDRDBAN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trdrdscl	MAKE_PTO_TRDRDSCL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Tdrdrsc	MAKE_PTO_TRDRDSC_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trdrdsd	MAKE_PTO_TRDRDSD_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trdrddd	MAKE_PTO_TRDRDDD_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrwrban	MAKE_PTO_TWRWRBAN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)

Timing - Twrwrscl	MAKE_PTO_TWRWRSCL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrwrsc	MAKE_PTO_TWRWRSC_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrwrsd	MAKE_PTO_TWRWRSCL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrwrdd	MAKE_PTO_TWRWRDD_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Trdwr	MAKE_PTO_TRDWR_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)
Timing - Twrrd	MAKE_PTO_TWRRD_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, ValueInMemClk)

4.1.5.7 PHY Configuration Category

These macros will be evaluated and applied to the memory PHY timing registers. These controls set values familiar to those who deal with memory timing details and won't be further defined here. Please see the PPR and/or the applicable JEDEC specs.

Values set BEFORE training begins

Command Address setup	MAKE_PTO_ADDR_CMD_SETUP_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
CS ODT setup	MAKE_PTO_CS_ODT_SETUP_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
CKE setup	MAKE_PTO_CKE_SETUP_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Drive Strength - CLK	MAKE_PTO_CLK_DRV_STREN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Drive Strength - CMD	MAKE_PTO_ADDR_CMD_DRV_STREN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Drive Strength - CS ODT	MAKE_PTO_CS_ODT_DRV_STREN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Drive Strength - CKE	MAKE_PTO_CKE_DRV_STREN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Timing - Write Preamble	Macro control to write timing. MAKE_PTO_WR_PREAMBLE_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value) Permitted Value Choices: (Type: number) • 0 - 1 Tck • 1 - 2 Tck
Timing - Read Preamble	Macro control to read timing. MAKE_PTO_RD_PREAMBLE_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value) Permitted Value Choices: (Type: number) • 0 - 1 Tck • 1 - 2 Tck

Values applied AFTER training has completed

Delay - RxEnable	MAKE_PTO_RX_ENABLE_DELAY_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Operation, Dimm, Byte, Nibble, Value)
Delay - Tx DQ	MAKE_PTO_TX_DQ_DELAY_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Operation, Dimm, Byte, Dq, Value)
Delay - Tx DQS	MAKE_PTO_TX_DQS_DELAY_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Operation, Dimm, Byte, Nibble, Value16)
Delay - Rx PB	MAKE_PTO_RX_PB_DELAY_ENTRY(SocketBitMap, ChannelBitMap, Operation, Dimm, Byte, Dq, Value)
Delay - Rx CLK	MAKE_PTO_RX_CLK_DELAY_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Operation, Dimm, Byte, Nibble, Value)
Vref - DAC0	MAKE_PTO_VREF_DAC_0_ENTRY(SocketBitMap, ChannelBitMap, Operation, Byte, Dq, Value)
Vref - DAC1	MAKE_PTO_VREF_DAC_1_ENTRY(SocketBitMap, ChannelBitMap, Operation, Byte, Dq, Value)
Vref - Global	MAKE_PTO_VREF_IN_GLOBAL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Operation, Value16)

4.1.5.8 Training Inputs Category

These macros will be evaluated and applied before the memory 1D training process occurs.

1D training - MR0	Macro control to . MAKE_PTO_1D_TRAIN_MR0_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value) Permitted Value Choices: (Type: Value) • value -??. • action when not specified: state defined by
1D training - MR1	Macro control to . MAKE_PTO_1D_TRAIN_MR1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value) Permitted Value Choices: (Type: Value) • value -??. • action when not specified: state defined by
1D training - MR2	Macro control to . MAKE_PTO_1D_TRAIN_MR2_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value) Permitted Value Choices: (Type: Value) • value -??. • action when not specified: state defined by
1D training - MR3	Macro control to . MAKE_PTO_1D_TRAIN_MR3_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value) Permitted Value Choices: (Type: Value) • value -??.

- action when not specified: state defined by

1D training - MR4

Macro control to .

MAKE_PTO_1D_TRAIN_MR4_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - MR5

Macro control to .

MAKE_PTO_1D_TRAIN_MR1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - MR6

Macro control to .

MAKE_PTO_1D_TRAIN_MR6_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - Vref Phy

Macro control to .

MAKE_PTO_1D_TRAIN_PHY_VREF_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - Sequence

Macro control to .

MAKE_PTO_1D_TRAIN_SEQUENCE_CTRL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - DFI

Macro control to .

MAKE_PTO_1D_TRAIN_DFI_MRL_MARGIN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - Custom Msg

Macro control to .

MAKE_PTO_1D_CUSTOM_MSG_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Offset, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 0**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL0_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 1**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 2**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL2_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 3**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL3_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 4**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL4_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 5**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL5_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**1D training -
ACSM 6**

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL6_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - ACSM 7

Macro control to .

MAKE_PTO_1D_TRAIN_ACSM_ODT_CTRL7_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

1D training - Alt CAS Lat

Macro control to .

MAKE_PTO_1D_TRAIN_ALT_WCAS_L_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

These macros will be evaluated and applied before the memory 2D training process occurs.

2D training - MR0

Macro control to .

MAKE_PTO_2D_TRAIN_MR0_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - MR1

Macro control to .

MAKE_PTO_2D_TRAIN_MR1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - MR2

Macro control to .

MAKE_PTO_2D_TRAIN_MR2_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - MR3

Macro control to .

MAKE_PTO_2D_TRAIN_MR3_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
MR4**

Macro control to .

MAKE_PTO_2D_TRAIN_MR4_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
MR5**

Macro control to .

MAKE_PTO_2D_TRAIN_MR1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
MR6**

Macro control to .

MAKE_PTO_2D_TRAIN_MR6_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
Vref Phy**

Macro control to .

MAKE_PTO_2D_TRAIN_PHY_VREF_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
Sequence**

Macro control to .

MAKE_PTO_2D_TRAIN_SEQUENCE_CTRL_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
DFI**

Macro control to .

MAKE_PTO_2D_TRAIN_DFI_MRL_MARGIN_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

**2D training -
ACSM 0**

Macro control to .

MAKE_PTO_2D_TRAIN_ACSM_ODT_CTRL0_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - ACSM 1

Macro control to .

MAKE_PT0_2D_TRAIN_ACSM_ODT_CTRL1_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - ACSM 2

Macro control to .

MAKE_PT0_2D_TRAIN_ACSM_ODT_CTRL2_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - ACSM 3

Macro control to .

MAKE_PT0_2D_TRAIN_ACSM_ODT_CTRL3_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - ACSM 4

Macro control to .

MAKE_PT0_2D_TRAIN_ACSM_ODT_CTRL4_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - ACSM 5

Macro control to .

MAKE_PT0_2D_TRAIN_ACSM_ODT_CTRL5_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

2D training - ACSM 6

Macro control to .

MAKE_PT0_2D_TRAIN_ACSM_ODT_CTRL6_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)

Permitted Value Choices: (Type: Value)

- value -??.
- action when not specified: state defined by

- 2D training - ACSM 7** Macro control to .
MAKE_PTO_2D_TRAIN_ACSM_ODT_CTRL7_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Permitted Value Choices: (Type: Value)
- value -??.
 - action when not specified: state defined by
- 2D training - Alt CAS Lat** Macro control to .
MAKE_PTO_2D_TRAIN_ALT_WCAS_L_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Permitted Value Choices: (Type: Value)
- value -??.
 - action when not specified: state defined by
- 2D training - Rx DFE** Macro control to .
MAKE_PTO_2D_TRAIN_RX2D_TRAIN_OPT_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Permitted Value Choices: (Type: Boolean)
- TRUE - This option is active.
 - FALSE - This option is turned off.
 - action when not specified: state defined by
- 2D training - Tx FFE** Macro control to .
MAKE_PTO_2D_TRAIN_TX2D_TRAIN_OPT_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Value)
Permitted Value Choices: (Type: Value)
- value -??.
 - action when not specified: state defined by
- 2D training - Custom Msg** Macro control to .
MAKE_PTO_2D_CUSTOM_MSG_ENTRY(SocketBitMap, ChannelBitMap, MemPstate, Dimm0Rank, Dimm1Rank, Offset, Value)
Permitted Value Choices: (Type: Value)
- value -??.
 - action when not specified: state defined by

4.1.5.9 Validity Groups

The concept of a validity group is a collection of PTO settings that apply collectively to a DIMM with group characteristics. The group characteristics are supplied at the beginning of a block of PTO macros and the block end is specified by a termination macro. More than one Validity block initiator can be supplied at the beginning to combine conditions.

The same matching conditions parameters apply as defined for the PTO macros.

- DRAM device Mfgr ID** This macro designates the beginning of a validity block to be applied when the memory devices on the DIMM are from a certain manufacturer.
MAKE_PTO_VALIDITY_DRAM_MFG_ID_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: Value) Hex value identifying the manufacturer. This code is assigned by the JEDEC organization.

DIMM Module Mfg ID This macro designates the beginning of a validity block to be applied when the DIMM is from a certain manufacturer.

MAKE_PTO_VALIDITY_MODULE_MFG_ID_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: Value) Hex value identifying the manufacturer. This code is assigned by the JEDEC organization.

DRAM Device Stepping This macro designates the beginning of a validity block to be applied when the memory devices on the DIMM are of a certain stepping.

MAKE_PTO_VALIDITY_DRAM_STEPPING_ENTRY(SocketBitMap, ChannelBitMap, Value)

Permitted Value Choices: (Type: Value) Hex value identifying the manufacturer's stepping. This code is assigned by the manufacturer.

Block Termination This macro designates the end of a validity block of PTO macros. This macro must appear at the end to properly terminate the grouping.

MAKE_PTO_VALIDITY_END()

No parameters are needed.

4.1.5.10 PTO Examples

This section provides a few examples of how to use the PTO macros to create a specialized tuning list.

Example #1:

To force a function or feature, such as the Chip Select Interleaving feature, to be disabled in all cases, the PlatformTuningEntry.c file should contain:

```
PTO_ENTRY PtoEntry[] = {
    MAKE_PTO_CHIPSEL_INTLV_ENTRY(ANY_SOCKET, ANY_CHANNEL, FALSE),

    MAKE_PTO_TERMINATE()    // Terminator
};
```

Example #2:

To force the chipselect interleaving feature to be disabled in all cases, and set memory CAS latency value to 0x16 for all socket's channelA but only when the memory pstate is 0 and DIMM0 is 'single rank' and DIMM1 not present, the PlatformTuningEntry.c file should contain:

```
PTO_ENTRY PtoEntry[] = {
    MAKE_PTO_CHIPSEL_INTLV_ENTRY(ANY_SOCKET, ANY_CHANNEL, FALSE),
    MAKE_PTO_TCL_ENTRY(ANY_SOCKET, CHANNEL_A, MEMORY_PSTATE0, DIMM_SR, NP, 0x16),

    MAKE_PTO_TERMINATE()    // Terminator
};
```

Example #3:

To force the chipselect interleaving feature to be disabled in all cases, and set memory CAS latency value to 0x18 for any socket's, channelB, but only when memory pstate is 1 and DIMM0 is single rank or dual rank (but not quad rank) and DIMM1 is also single rank or dual rank, the PlatformTuningEntry.c file should contain:

```
PTO_ENTRY PtoEntry[] = {
    MAKE_PTO_CHIPSEL_INTLV_ENTRY(ANY_SOCKET, ANY_CHANNEL, FALSE),
    MAKE_PTO_TCL_ENTRY(ANY_SOCKET, CHANNEL_B, MEMORY_PSTATE1, // socket, channel, Pstate
                       (DIMM_SR | DIMM_DR), (DIMM_SR | DIMM_DR), // Dimm0, Dimm1
                       0x18), // Value
    MAKE_PTO_TERMINATE() // Terminator
};
```

Example #4:

To enable command 2T setting for any socket, any channel, any Memory Pstate and any DIMM type; the PlatformTuningEntry.c file should contain:

```
PTO_ENTRY PtoEntry[] = {
    MAKE_PTO_CMD_2T_ENABLE_ENTRY(ANY_SOCKET, ANY_CHANNEL,
                                  0xFF, 0xFF, 0xFF, // Pstate, Dimm0, Dimm1
                                  TRUE), //Value
    MAKE_PTO_TERMINATE() // Terminator
};
```

Example #5:

This example has two groups (exclusive and non-exclusive). Its goals are:

1. disable chipselect interleaving for everywhere
2. set memory CAS latency value to 0x18 for any socket, channel B when memory pstate is 1 and DIMM0 is single or dual rank and DIMM1 is also single or dual rank
3. For any DIMM using DRAM devices that were manufactured by company 0x1234:
 - a. disable the ECC feature for a DIMM in channel A of any socket
 - b. override SPD data byte 13 of DIMM0 ChannelA Socket0 value to 0x0B

```
PTO_ENTRY PtoEntry[] = {
    // Non-exclusive
    MAKE_PTO_CHIPSEL_INTLV_ENTRY(ANY_SOCKET, ANY_CHANNEL, FALSE),
    MAKE_PTO_TCL_ENTRY(ANY_SOCKET, CHANNEL_B, MEMORY_PSTATE1, // socket, channel, Pstate
                       (DIMM_SR | DIMM_DR), (DIMM_SR | DIMM_DR), // Dimm0, Dimm1
                       0x18), // Value
    // Exclusive
    MAKE_PTO_VALIDITY_DRAM_MFG_ID_ENTRY(SOCKET0, CHANNEL_A, // Begin block
                                         0x1234), // If Mfgr 0x1234 only
    MAKE_PTO_ECC_ENTRY(ANY_SOCKET, CHANNEL_A, TRUE), // ECC=off
    MAKE_PTO_SPD_VALUE_OVERRIDE_ENTRY(SOCKET0, CHANNEL_A, // Dimm, byte, Value
                                       DIMM0, 13, 0x0B),
    MAKE_PTO_VALIDITY_END() // End block
    // Terminator
    MAKE_PTO_TERMINATE()
};
```

Example #6:

This example has two groups (exclusive and non-exclusive). Its goals are:

1. set memory CAS latency value to 0x21 for any socket, channel B when memory pstate is 0 and DIMM0 is single or dual rank and DIMM1 is dual rank,
2. For any DIMM using DRAM devices that were manufactured by company 0x1234
AND the DRAM parts are of stepping 0x11:

:

- a. set 1D training MR1 value to 0x3412 for any socket and any channel when memory pstate is 0

```

PTO_ENTRY PtoEntry[] = {
    // Non-exclusive
    MAKE_PTO_TCL_ENTRY(ANY_SOCKET, CHANNEL_B, MEMORY_PSTATE0,    // socket, channel, Pstate
                      (DIMM_SR | DIMM_DR), DIMM_DR),             // Dimm0, Dimm1
                      0x21),                                     // Value
    // Exclusive
    MAKE_PTO_VALIDITY_DRAM_MFG_ID_ENTRY(SOCKET0, CHANNEL_A,      // Begin block
                      0x1234),                                   // If Mfgr 0x1234 only

    MAKE_PTO_VALIDITY_DRAM_STEPPING_ENTRY(SOCKET0, CHANNEL_A,   // AND stepping=0x11
                      0x11),                                     // socket, channel
    MAKE_PTO_1D_TRAIN_MR1_ENTRY(ANY_SOCKET, ANY_CHANNEL,        // Pstate, Dimm0, Dimm1
                                MEMORY_PSTATE0, DIMM_SR, NP,    // Value
                                0x3412),                         // End block
    MAKE_PTO_VALIDITY_END()

    // Terminator
    MAKE_PTO_TERMINATE()
};

```

4.1.6 Console Controls

Platform Enables

These controls are used to activate the ABL console output which can be used to trace and debug the ABL operations. These are not enabled for production systems due to the time added to the boot process.

- **APCB_TOKEN_CONFIG_FCH_CONSOLE_OUT_ENABLE**

This option enables the ABL Console Output for debug. Data is sent to a specific port as defined by APCB_TOKEN_CONFIG_FCH_CONSOLE_OUT_SERIAL_PORT. Enabling console output will increase the boot times.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Enables the ABL Console Out messages.
- FALSE - This option is turned off and no output is generated.

- **APCB_TOKEN_UID_ABL_SERIAL_BAUD_RATE**

This control sets the baud rate of the selected serial port used for the console.

Permitted Choices: (Type: List)(Default: BAUD_RATE_115200)

- BAUD_RATE_2400
- BAUD_RATE_3600
- BAUD_RATE_4800
- BAUD_RATE_7200
- BAUD_RATE_9600
- BAUD_RATE_19200
- BAUD_RATE_38400
- BAUD_RATE_57600
- BAUD_RATE_115200

- **APCB_TOKEN_CONFIG_PSP_TP_PORT**

This option controls the debug output of Test Points to the debug port (typically IO port 80h).

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Test Points will be sent to debug output port.
- FALSE - No debug output is generated.

• **APCB_TOKEN_CONFIG_FCH_CONSOLE_OUT_SERIAL_PORT**

This option specifies the COM Port from a specific device which can be used.

Permitted Choices: (Type: List)(Default: 0)

- 0 - Use Super IO COM1 (IO Address 3F8) through LPC port of FCH.
- 1 - Use UART 0 of CPU/APU via MMIO
- 2 - Use UART 1 of CPU/APU via MMIO

• **APCB_TOKEN_CONFIG_PSP_ERROR_DISPLAY**

This option controls the action taken by the ABL when an error is detected. The ABL can log the error, stop immediately (used for debug) and report the error via a beep code or GPIO.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - ABL will perform all enabled actions.
- FALSE - No action is taken.

• **APCB_TOKEN_CONFIG_PSP_STOP_ON_ERROR**

This option controls the action taken by the ABL when an error of type AGESA_FATAL, AGESA_ERROR or AGESA_OC_FATAL is detected. The ABL will report the error and then can stop immediately (used for debug) or continue.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - ABL will stop immediately (hard loop) waiting for debugger break-in or reset.
- FALSE - No action is taken.

• **APCB_TOKEN_UID_PSP_ENABLE_DEBUG_MODE**

This token enables debug mode in PSP and SMU FW, allowing the system to hang when there's an error instead of a reset.

Permitted Choices: (Type: Value) (Default: TRUE - Production Mode)

- FALSE = Production Mode. System will reset on fatal error.
- TRUE = Debug Mode. System will hang allowing time for a debug user to view the issue.

• **APCB_TOKEN_UID_PSP_SYSHUB_WDT_INTERVAL**

This token the timer interval of the PSP SYSHUB Watchdogtimer. Value is in milliseconds and must be within the range: 1-65535.

Permitted Choices: (Type: Value)(Default: 2600)

Output Verbosity Controls

• **APCB_TOKEN_CONFIG_FCH_CONSOLE_BASIC_OUT_ENABLE**

This option controls the amount of data sent via the ABL Console Output. Limiting the quantity of output can balance the increase of boot time and needed debug information. This option takes effect only when `APCB_TOKEN_CONFIG_FCH_CONSOLE_OUT_ENABLE` is `=TRUE`.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Limits the ABL debug console output to 'basic' information. This includes:
 - Boot flow
 - Break Points
 - Error information
 - Platform tuning time points

This are the results of the processes occurring in the ABL.

- FALSE - Content is not limited. In addition to the Basic content output, ABL will also output information to track process activity including:
 - register writes/reads
 - status information
 - test information

• **APCB_TOKEN_UID_DISPLAY_PMU_TRAIN_RESULTS**

Control to enable/disable PMU training results display, currently, it supports to display the following PMU training results:

- Receive Enable Delay
- Read DQS to RxClk Delay
- Write DQS Delay
- Read DQ per-bit BDL delay
- VrefDAC0 control for DQ Receiver
- VrefDAC1 control for DQ Receiver (when DFE is enabled in DDR4)

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - PMU training results output is enabled.
- FALSE - No debug output is generated.

• **APCB_TOKEN_UID_MEM_TRAINING_HDTCTRL [F17M30][F19M00]**

The switch to control PMU message output

- `DETAILED_DBG_MSG` - Detailed debug messages are provided.
- `COARSE_DBG_MSG`- Coarse level debug messages are provided - enough to follow the flow.
- `STAGE_COMPLETION`- Display a message only when each training stage completes. Stages include ReadEnable, ReadDelay, WriteDelay, ReadLatency, WriteLeveling and more.
- `FIRMWARE_COMPLETION_MSG_ONLY` - display a message only when PMU Firmware completes.

Filtering Controls

Once the above controls are set to establish the output content, the filtering feature can be invoked to specify which segments of the output are 'interesting' to the viewer at this time. This can help focus the output data to the area of interest. These are set by the platform BIOS at build time by providing the following control values in file: "`AgesaPkg\Addendum\Apcb\%board%\Include\ApcbCustomizedDefinitions.h`". The platform porting

engineer will edit this file and make changes for the desired values. This file is then used by the APCBTool to construct the APCB image for the build.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_ENABLE**

Enable ABL Console output filter

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Global Enable of ABL console output filter.
- FALSE - Disable ABL console output filter. All data will appear.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_FLOW**

ABL Console out filter for "MEM FLOW".

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_SET_REG**

ABL Console out filter to enable "MEM SETREG".

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_GET_REG**

ABL Console out filter to enable "MEM GETREG".

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_STATUS**

ABL Console out filter to enable "MEM STATUS".

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_PMU**

ABL Console out filter to enable "MEM PMU".

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Filter is enabled and this data will be displayed. The level of detail for this data is controlled by the control: `APCB_TOKEN_UID_MEM_TRAINING_HDTCTRL`
- FALSE - Filter is disabled and this console output is removed from the log.

- **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_PMU_SRAM_READ**

ABL Console out filter to enable "MEM PMU SRAM READ"

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

• **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_PMU_SRAM_WRITE**

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

• **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_TEST_VERBOSE**

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

• **BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_MEM_BASIC_OUTPUT**

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Filter is enabled and this data will be displayed.
- FALSE - Filter is disabled and this console output is removed from the log.

4.1.7 ABL Error Reporting Configuration Items

The ABL Error log can be controlled by the platform BIOS at build time by providing the following control values. Please see “AgesaPkg\Addendum\%board%\Apcb\Standalone\ApcbData_<Platform>_ErrorReportingControl.c” – This section is still to be added to the tool.

• **ErrorLogPortReportingEnable**

Indicates if ABL will report errors via a port. This is the main switch to turn of/off the error reporting via output port feature.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Error logging will be reported via a port.
- FALSE - Error logging will not be reported via a port.

• **ErrorLogReportUsingHandshakeEnable**

This flag indicates if the ABL will use a handshake for the Error Log

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Error log reported using a handshake with the ErrorLogOutputPort and ErrorLogInputPort. This is a bi-directional handshake; so the ABL will not proceed until the debug host responds.
- FALSE - Error log reported using ErrorLogOutputPort only with each DWORD in log delayed by ErrorLogOutputDwordDelay. This is a mono-directional output stream; so the ABL is not blocked. The output delay is used to control overrun at the output device. Note: adding delay will slow down the display rate so that it is visible longer, but will also lengthen the boot time.

• **ErrorReportErrorBeepCodeEnable**

This flag indicates if the ABL will report errors via beep codes.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Error logging (beep codes) will be reported via the FCH speaker.
- FALSE - no beep codes will be used.

- **ErrorLogInputPort**

Input Port to receive ABL handshake control. This is only valid when using the bi-directional output mode.

Permitted Choices: (Type: Value)(Default: 0x00)

- **ErrorLogReportInputPortSize**

Indicates the size of the input port.

Permitted Choices: (Type: List)(Default: 8Bit)

- 8-bit port
- 16-bit port
- 32-bit port

- **ErrorLogOutputPort**

IO Port to use for ABL Error information output

Permitted Choices: (Type: Value)(Default: 0x80)

- **ErrorLogReportOutputPortSize**

Indicates the size of the output port.

Permitted Choices: (Type: List)(Default: 8Bit)

- 8-bit port
- 16-bit port
- 32-bit port

- **ErrorLogOutputDwordDelay**

Number of 10ns units to wait before sending the next Log Dword of information. This value is used for mono-directional and heart beat output modes.

Permitted Choices: (Type: Value)(Default: 15000)

- **ErrorStopOnFirstFatalErrorEnable**

Indicates if ABL should stop on the first fatal error.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Stop and report the first FATAL error.
- FALSE - Report all errors.

- **ErrorLogReportClearAcknowledgement**

Indicates if the ABL will clear acknowledgements during protocol

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE – Clear acknowledgements.

- FALSE – Do Not clear acknowledgements.

- **ErrorLogHeartBeatEnable**

Indicates if the ABL will provide periodic status to a port as a 'heart beat'.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE – Heartbeat Error log will be reported via a port.
- FALSE – No Heartbeat will be produced.

This feature is valid only when error logging is enabled. It can be used in either mono-directional or bi-directional handshake. It requires the input port, size and type to be defined.

4.1.8 ABL MBIST Configuration Items

This section describes the input controls for MBIST. Please see “AgesaPkg\Addendum\%board% \Apcb \ApcbData_<Platform>_MbistControl.c”

- **APCB_TOKEN_CONFIG_MEM_MBIST_TEST_ENABLE [F17M00][F17M10][F17M20]**
APCB_TOKEN_UID_MEM_MBIST_TEST_ENABLE [F17M30]

This option controls whether or not the MBIST tests are performed.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

- **APCB_TOKEN_CONFIG_MEM_MBIST_TESTMODE [F17M00]**

APCB_TOKEN_UID_MEM_MBIST_TESTMODE [F17M30][F17M70]

This option specifies the test mode in which MBIST is to operate. Either Interface Mode or Data Eye Mode May be chosen..

Permitted Choices: (Type: List)(Default: MBIST_TESTMODE_INTERFACE)

- MBIST_TESTMODE_INTERFACE – This mode has the intent of proving the electrical interface is working correctly. These tests will look for crosstalk between data lines and crosstalk between address lines.
- MBIST_TESTMODE_DATAEYE – This mode has the intent of identifying the data eye size or 'margin'. It does not alter the data eye center found during training. Output from this test can be used to draw graphical representations of the data eye.
- MBIST_TESTMODE_BOTH – First Run Interface Mode Tests, then Data Eye Tests

- **APCB_TOKEN_UID_MEM_MBIST_AGGRESOR_ON [F17M30][F19M00]**

APCB_TOKEN_CONFIG_MEM_MBIST_AGGRESOR_ON [F17M00][F17M10][F17M20]

This token turns on the Aggressor traffic during the Data Eye Test Mode by initiating constant memory read/write transactions on the opposing channel(s) of each die. This will maximize the occurrence of any crosstalk for easier detection. Further tuning can be achieved with the following controls.

Permitted Choices:(Type: Boolean)(Default: TRUE)

- TRUE – Enable Aggressor Traffic During Data Eye mode
- FALSE – Disable Aggressor Traffic During Data Eye mode

- **APCB_TOKEN_CONFIG_MEM_MBIST_AGGRESSORS_CHNL [F17M00]**

APCB_TOKEN_UID_MEM_MBIST_AGGRESSORS_CHNL [F17M30][F17M70]

This token helps configure the number of possible channels which can be set as an aggressor. The following options are supported in this case.

- Disable – No Aggressor is setup. - Default
- 1 Aggressor Channel – Setup 1 Aggressor Channel per Channel Pair.
- 3 Aggressor Channel – Setup 1 Channel as Test and 3 Channel as Aggressor.
- 7 Aggressor Channel – Setup 1 Channel as Test Channel and 7 Channel as Aggressor Channel.

- **APCB_TOKEN_UID_MEM_MBIST_PATTERN_LENGTH [F17M30][F17M70]**

This token configures the test pattern length. The current support length can be from 3-12. Where 3 is interpreted as 10^3 while 12 is 10^{12} pattern.

- **APCB_TOKEN_UID_MEM_MBIST_PATTERN_SELECT [F17M30][F17M70]**

This token configures what kind of test patten will be selected. It contains the following option.

- PRBS - Default
- SSO
- Both

- **APCB_TOKEN_CONFIG_MEM_MBIST_AGGR_STATIC_LANE_CTRL [F17M00]**

APCB_TOKEN_UID_MEM_MBIST_AGGR_STATIC_LANE_CTRL [F17M30][F17M70]

This token control for using aggressor static lanes include ECC lanes.

- Disable – Aggressor Static Lane configuration is disabled (Default)
- Enabled - Aggressor Static Lane configuration is enabled

- **APCB_TOKEN_CONFIG_MEM_MBIST_AGGR_STATIC_LANE_SEL_L32 [F17M00]**

APCB_TOKEN_CONFIG_MEM_MBIST_AGGR_STATIC_LANE_SEL_U32 [F17M00]
APCB_TOKEN_UID_MEM_MBIST_AGGR_STATIC_LANE_SEL_L32 [F17M30][F17M70]
APCB_TOKEN_UID_MEM_MBIST_AGGR_STATIC_LANE_SEL_U32 [F17M30][F17M70]

This pair of tokens set the DQ lanes mask for Aggressor Lanes. This is a 64-bit value split to be two 32-bit values (L32-lower bits, U32-upper bits) where each bit represents 1 DQ lane. Lanes represented with a 1, will be used as aggressors during the MBIST test.

- **APCB_TOKEN_CONFIG_MEM_MBIST_AGGR_STATIC_LANE_SEL_ECC [F17M00]**

APCB_TOKEN_UID_MEM_MBIST_AGGR_STATIC_LANE_SEL_ECC [F17M30][F17M70]

This token set the DQ lanes mask for ECC byte for Aggressor Lanes. This is 1 byte and each bit represents 1 DQ lane of ECC.

- **APCB_TOKEN_CONFIG_MEM_MBIST_AGGR_STATIC_LANE_VAL [F17M00][F17M10]
[F17M20]**

APCB_TOKEN_UID_MEM_MBIST_AGGR_STATIC_LANE_VAL [F17M30][F17M70]

This token determines the value which need to be set for Aggressor Static Lanes. The possible values are either 0 or 1. Default is set to 0.

- **APCB_TOKEN_CONFIG_MEM_MBIST_TGT_STATIC_LANE_CTRL [F17M00][F17M10][F17M20]**

APCB_TOKEN_UID_MEM_MBIST_TGT_STATIC_LANE_CTRL [F17M30][F17M70]

This token, if enable will enable the entries for target static lanes including ECC lanes. Disable – Aggressor Static Lane configuration is disabled (Default) Enabled - Aggressor Static Lane configuration is enabled

- **APCB_TOKEN_CONFIG_MEM_MBIST_TGT_STATIC_LANE_SEL_L32 [F17M00][F17M10][F17M20]**

APCB_TOKEN_CONFIG_MEM_MBIST_TGT_STATIC_LANE_SEL_U32 [F17M00][F17M10][F17M20]

APCB_TOKEN_UID_MEM_MBIST_TGT_STATIC_LANE_SEL_L32 [F17M30][F17M70]

APCB_TOKEN_UID_MEM_MBIST_TGT_STATIC_LANE_SEL_U32 [F17M30][F17M70]

This pair of tokens set the DQ lanes mask for Target Lanes. This is a 64-bit value split to be two 32-bit values (L32-lower bits, U32-upper bits) where each bit represents 1 DQ lane. Lanes represented with a 1, will be used as the target for the MBIST test.

- **APCB_TOKEN_CONFIG_MEM_MBIST_TGT_STATIC_LANE_SEL_ECC [F17M00][F17M10][F17M20]**

APCB_TOKEN_UID_MEM_MBIST_TGT_STATIC_LANE_SEL_ECC [F17M30][F17M70]

This token set the DQ lanes mask for ECC byte for Target Lanes. This is 1 byte and each bit represents 1 DQ lane of ECC.

- **APCB_TOKEN_CONFIG_MEM_MBIST_TGT_STATIC_LANE_VAL [F17M00][F17M10][F17M20]**
APCB_TOKEN_UID_MEM_MBIST_TGT_STATIC_LANE_VAL [F17M30][F17M70]

This token determines the value which need to be set for Aggressor Static Lanes. The possible values are either 0 or 1. Default is set to 0.

- **APCB_TOKEN_CONFIG_MEM_MBIST_DATA_EYE_TYPE**

This token determines what type of data eye we need to capture. The following option are supported. 1D Voltage Sweep – This option provide the best operating region of voltage for a given time for a specific DQ 1D Timing Sweep – This option provide the best operating region of a time given voltage is fixed for a given DQ 2D Full Data Eye – This option provide the best operating region for voltage and timing to determine the eye. Worst Case Margin only – This option gives the worst case possible range of voltage and timing for a possible operating range (Default).

- **APCB_TOKEN_CONFIG_MEM_MBIST_WORST_CASE_GRAN [F17M00][F17M10][F17M20]**
APCB_TOKEN_UID_MEM_MBIST_WORST_CASE_GRAN [F17M30][F17M70]

This token determines if granularity of the data capture for worst case margins. The possible choices are Per Chips Select - (Default) Per Nibble

- **APCB_TOKEN_CONFIG_MEM_MBIST_READ_DATA_EYE_VOLTAGE_STEP [F17M00][F17M10][F17M20]**

APCB_TOKEN_UID_MEM_MBIST_READ_DATA_EYE_VOLTAGE_STEP [F17M30][F17M70][F19M00]

This field is used in sweeping the voltage Vref for DDR4. This token determines the step size for the read data eye voltage parameter. For further detail, see the PPR description for UMC::Phy::VrefDac1.

Permitted choices are: (Type: List)(Default: 1)

- 1 - increment by one 'step unit' (as defined by the PPR) on each pass.
 - 2 - increment by 2 step units.
 - 4 - increment by 4 step units.
- **APCB_TOKEN_CONFIG_MEM_MBIST_READ_DATA_EYE_TIMING_STEP [F17M00][F17M10][F17M20]**

APCB_TOKEN_UID_MEM_MBIST_READ_DATA_EYE_TIMING_STEP [F17M30][F17M70][F19M00]

This token determined the step size for the read data eye timing parameter.

Permitted choices are: (Type: List)(Default: 1)

- 1 - increment by one 'step unit' (as defined by the PPR) on each pass.
 - 2 - increment by 2 step units.
 - 4 - increment by 4 step units.
- **APCB_TOKEN_CONFIG_MEM_MBIST_WRITE_DATA_EYE_VOLTAGE_STEP [F17M00][F17M10][F17M20]**
- APCB_TOKEN_UID_MEM_MBIST_WRITE_DATA_EYE_VOLTAGE_STEP [F17M30][F17M70][F19M00]**
- This field is used in sweeping the voltage Vref for DDR4. This token determines the step size for the write data eye voltage parameter. For further detail, see the PPR description for UMC::Phy::VrefDac0.
- Permitted choices are: (Type: List)(Default: 1)
- 1 - increment by one 'step unit' (as defined by the PPR) on each pass.
 - 2 - increment by 2 step units.
 - 4 - increment by 4 step units.
- **APCB_TOKEN_CONFIG_MEM_MBIST_WRITE_DATA_EYE_TIMING_STEP [F17M00][F17M10][F17M20]**
- APCB_TOKEN_UID_MEM_MBIST_WRITE_DATA_EYE_TIMING_STEP [F17M30][F17M70][F19M00]**

This token determined the step size for the write data eye timing parameter.

Permitted choices are: (Type: List)(Default: 1)

- 1 - increment by one 'step unit' (as defined by the PPR) on each pass.
 - 2 - increment by 2 step units.
 - 4 - increment by 4 step units.
- **APCB_TOKEN_CONFIG_MEM_MBIST_PER_BIT_SLAVE_DIE_REPORT [F17M00][F17M10][F17M20]**

APCB_TOKEN_CONFIG_MEM_MBIST_PER_BIT_SLAVE_DIE_REPORT [F17M30][F17M70]

This option selects the amount of ABL MBIST output per bit data to the console.

Permitted Choices: (Type: BOOLEAN) (Default: Disabled)

- Enabled - output MBIST per bit data of master and slave both to console.
- Disabled - output MBIST per bit data of just master die to console.

• Voltage Corners

This option selects the voltage corners used during the tests:

Permitted Choices: (Type: List)(Default: A)

- None - Perform Tests at nominal voltages only
- Nominal + VDDIO - Perform Tests at nominal voltage as well as +/- 5% VDDIO
- Nominal + Vref - Perform tests at nominal voltage as well as +/- 3% Vref
- All (Nominal + VDDIO + Vref) - Perform tests at Nominal voltage, +/- 5% VDDIO, and +/- 3% Vref.

• Exclude On Error

This option allows the user to exclude a bank (chipselect) if an error occurs during the MBIST test.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE – Exclude failed chip selects from the memory map.
- FALSE – Do not exclude failed chip selects from the memory map.

• DRAM Phy Timing Range

This option selects the DDR Phy Timing Range; +/- Percentage value, by which to vary DDR PHY Timing during Memory Interface Test

Permitted Choices: (Type: Value)(Default: 0x0000)

• DRAM Pattern Run Length

This option selects the Pattern run length per corner. A value or 1-255 sets a pattern run length per corner x 1M Quadwords

Permitted Choices: (Type: Value)(Default: 0x0000)

• APCB_TOKEN_UID_MEM_SELF_HEAL_BIST_ENABLE [F19M00]

New DDR4 DIMMs have the ability to 'Self Heal'. JEDEC has defined a standard where the DIMM performs a MBIST self test then perform a hard repair for areas found to be problematic. The BIOS can also perform a similar test targeting repairs with the Post Package Repair (PPR) process. This items enables a full memory test.

Caution: The testing will increase the boot time.

Permitted Choices: (Type: List)(Default: 0)

- 0 - Disabled
- 1 - BIOS Mem BIST, this tests the full memory after training. Failing memory will be repaired using soft or hard PPR depending on the PPC configuration. The test will take ~3 minutes per 16GB of installed memory.
- 2 - Self-Healing Mem BIST, this runs the JEDEC DRAM self healing, if the device and DIMM support the feature. The DRAM will do a hard repair for failing memory. The test will take ~10 seconds per memory rank per channel.
- 3 - BIOS and Self-Healing Mem BIST, this option runs both forms of the test sequentially.

- **APCB_TOKEN_UID_MEM_SELF_HEAL_BIST_TIMEOUT [F19M00]**

This token defines the timeout value when to abort the Self-Healing Mem BIST. The value provided is in milli-seconds (ms).

Permitted Choices: (Type: Value)(Default: 10000 (10sec))

4.2 Selecting one of Multiple Boards

When the BIOS ROM image contains support for different platforms or variations, a means to identify the specific APCB data for use is needed - identifying which board is in use. The ABL supports three standard methods to detect the board ID: EEPROM, SMBUS and GPIO. Depending on the method used, one of the following structures are initialized.

The data structure to support board ID detection is instantiated in `ApcbData_<SoC>_GID_0x1704_Type_BoardIdGettingMethod.c`. "if customer wants to support multiple APCB instances for different boards with one BIOS, set `APCB_MULTI_BOARD_SUPPORT=1` in environment variable when building BIOS otherwise default `APCB_MULTI_BOARD_SUPPORT` setting (=0) in `SetApcbBoardList.bat` will be chosen.

When determining the board ID, a call will be placed to the `ApcbData_ZP_GID_0x1704_Type_BoardIdGettingMethod.c` routine to acquire the board ID. This routine must be modified by the customer to choose one of the standard access methods to be used:

- EEPROM - Read the ID from an on-board EEPROM device on the I²C bus.
- SMBUS - Read the ID from an SMBUS device's register.
- GPIO - Read the ID from a group of GPIO lines.

The platform may also implement non-standard platform specific conditions or features. A bit at the top of the board ID is reserved to indicate this platform feature. The user must set the mask and APCB index values to accommodate the feature bit in the `ID_APCB_MAPPING` content.

Some rules to note about the "Board ID Getting Method" implementation:

1. Board ID detection must result in the same value for a given board across all reboot cycles and after BIOS update.
2. When using the run-time libraries to modify APCB content, caution must be taken to set the proper BoardID as it has higher selection priority than the default setting. Using a warm reset after such modifications, could result in an improper board selection. However;
3. a CMOS clear is provided to trigger APCB setting reset to make the HW response board ID change to work again as the higher priority setting from earlier board ID will be cleaned up.

Related Definitions

PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT

This structure is used when the Board ID is to be read from an EEPROM located on an I2C bus.

```
// [F17M00][F17M08][F17M10][F17M18][F17M20]
typedef struct _PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT {
    IN        UINT16        AccessMethod;    // = BOARD_ID_METHOD_EEPROM
    IN        UINT16        I2cCtrlr;
    IN        UINT16        SmbusAddr;
    IN        ID_APCB_MAPPING IdApcbMapping[];
} PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT;
```

```
// [F17M30][F17M70]
typedef struct _PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT {
```



```

IN      UINT16      AccessMethod;    // = BOARD_ID_METHOD_EEPROM
IN      UINT16      I2cCtrlr;
IN      UINT16      DeviceAddr;
IN      UINT16      BoardIdByteOffset;
IN      UNIT16      BoardRevByteOffset;
IN      ID_REV_APCB_MAPPING IdRevApcbMapping[];
} PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT;

```

AccessMethod

Common word for all structures, defines the content of the reset of the structure.

- BOARD_ID_METHOD_EEPROM - Board ID is stored in an EEPROM. Use this structure definition.
- BOARD_ID_METHOD_SMBUS - Board ID is stored in an SMBus device, use the PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT defined below.
- BOARD_ID_METHOD_GPIO - Board ID is set by GPIOs, use the PSP_GET_BOARD_ID_FROM_GPIO_STRUCT defined below.
- BOARD_ID_METHOD_IO - Board ID is read from an IO port, use the PSP_GET_BOARD_ID_FROM_IO_STRUCT defined below.
- 0xF: USER_CONFIG - Code written by the platform owner determines the board ID.

I2cCtrlr

I2C controller number where the device is attached: 0: I2C_0 ... 5: I2C_5

SmbusAddr

Device's address on the I2C bus.

DeviceAddr**BoardIdByteOffset**

Offset within the EEPROM to read for the BoardID (one byte).

BoardRevByteOffset

Offset within the EEPROM to read for the BoardRev (one byte). Note: a value of 0xFF indicates this feature is 'Not Applicable'.

IdApcbMapping

This is an array of Board ID and Board Revision to match with the values read from the EEPROM to select which APCB is to be used.

IdRevApcbMapping

PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT

This structure is used when the Board ID is to be read from an device located on an SMBus.

```

typedef struct _PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT {
IN      UINT16      AccessMethod;    // = BOARD_ID_METHOD_SMBUS
IN      UINT16      I2cCtrlr;
IN      UINT16      SmbusAddr;
IN      UINT16      RegIndex;
IN      ID_REV_APCB_MAPPING IdApcbMapping[];
} PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT;

```

Short Description**AccessMethod**

Common word for all structures, defines the content of the reset of the structure. See above.

I2cCtrlr

SMBus controller number where the device is attached: 0: SMBus_0 ... 1: SMBus_1

SmbusAddr	Device's address on the I2C bus.
RegIndex	The register index where the board ID can be read.
IdApcbMapping	This is an array of Board ID to match with the value read from the SMBus Device to select which APCB is to be used.

PSP_GET_BOARD_ID_FROM_GPIO_STRUCT

This structure is used when the Board ID is to be read from a group (one or more) of GPIO lines.

```
typedef struct _PSP_GET_BOARD_ID_FROM_GPIO_STRUCT {
IN      UINT16      AccessMethod;    // = BOARD_ID_METHOD_GPIO
      IN      UINT8      Gpio0;
      IN      UINT8      Gpio0IoMUX;
      IN      UINT8      Gpio0BankCtl;
      IN      UINT8      Gpio1;
      IN      UINT8      Gpio1IoMUX;
      IN      UINT8      Gpio1BankCtl;
      IN      UINT8      Gpio2;
      IN      UINT8      Gpio2IoMUX;
      IN      UINT8      Gpio2BankCtl;
      IN      UINT8      Gpio3;
      IN      UINT8      Gpio3IoMUX;
      IN      UINT8      Gpio3BankCtl;
      IN      ID_APCB_MAPPING IdApcbMapping[];
} PSP_GET_BOARD_ID_FROM_GPIO_STRUCT;
```

The code supporting this access method will read a maximum of 4 pins for board ID. Since the FCH GPIO pins are shared with other functions, an IOMUX register has to be configured correctly to make it a GPIO. Apart from that the GPIO Bank Control registers have to be configured for initialization. Please Refer to PPR for details on how to configure the IOMUX/GPIO registers. If any GPIO pins are not used, 0xFF has to be set in the GPIOx/IOMUXx/BankCtlx parameters for that pin. The GPIO pins are aggregated to form one 4-bit value, using GPIO[0] as the least significant bit and GPIO[3] as the most significant bit.

AccessMethod Common word for all structures, defines the content of the reset of the structure. See above.

Gpio[3..0] FCH GPIO number used for the board ID bit N.

Gpio0IoMUX[3..0] value to write to IOMUX to configure this GPIO pin.

Gpio0BankCtl[3..0] Value to write to GPIOBankCtl[23:16] to configure this GPIO pin.

IdApcbMapping This is an array of Board IDs to match with the value read from the GPIOs to select which APCB is to be used.

This structure is used when the Board ID is to be read from an IO port. [F17M30],[F17M70]

```
typedef struct _PSP_GET_BOARD_ID_FROM_IO_STRUCT {
IN      UINT16      AccessMethod;
IN      UINT16      ConfigPortWidth;
IN      UINT16      ConfigValue;
IN      UINT32      ConfigPortAddress;
IN      UINT16      DataPortWidth;
IN      UINT32      DataPortAddress;
IN      ID_APCB_MAPPING IdApcbMapping[];
```

```
} PSP_GET_BOARD_ID_FROM_IO_STRUCT;
```

The code supporting this access method will access the indicated IO port(s) to read a board ID value. The presumption is that this is an index-data IO port pair. First the ConfigValue is written to the ConfigPortAddress, followed by a read from the DataPortAddress. If the platform implementation is for only a single IO port, the ConfigPort and ConfigValue can be set to use the same IO address as the DataPort (assuming it is a Read-only port); or can be directed to a non-existent address where it will not cause issues (e.g. port 80h).

AccessMethod	Common word for all structures, defines the content of the reset of the structure. See above.
ConfigPortWidth	The width of Configuration IO Port used to activate the data. Permitted values: (8, 16)bits.
ConfigValue	The value written to Configuration IO Port for accessing the data.
ConfigPortAddress	The address of Configuration IO port. This is the standard x86 IO BUS address where the port is located.
DataPortWidth	The width of Data IO Port used to read the board ID value. Permitted values: (8, 16)bits.
DataPortAddress	The address of Data IO port where to read the board ID value.
IdApcbMapping	This is an array of Board IDs to match with the value read from the IO port to select which APCR is to be used.

----- ID_APCR_MAPPING

Short Description

These structures associate the encoded board ID/Rev with a single instance of APCR data blocks.

```
typedef struct {
    IN      UINT8    IdAndFeatureMask;
    IN      UINT8    IdAndFeatureValue;
    IN      UINT8    ApcbInstanceIndexForBoard;
} ID_APCR_MAPPING;
```

ID_REV_APCR_MAPPING

```
typedef struct {
    IN      UINT8    IdRevAndFeatureMask;
    IN      UINT8    IdAndFeatureValue;
    IN      UINT8    RevAndFeatureValue;
    IN      UINT8    ApcbInstanceIndexForBoard;
} ID_REV_APCR_MAPPING;
```

IdAndFeatureMask Mask applied to the ID value read from the board.

- Bit 6:0 - Id Mask Bits
- Bit7 - determines between normal or feature controlled Instance. This is a platform specific implementation.
- Bit7, 1 = User Controlled Feature Enabled, 0 - Normal Mode.

IdRevAndFeatureMask Mask applied to the ID and Rev values read from the board. Uses full 8-bit mask.

IDAndFeatureValue

After the mask is applied to the read ID value, the remaining value must be equal to this entry to be considered a match.

RevAndFeatureValue

After the mask is applied to the read Rev value, the remaining value must be equal to this entry to be considered a match.

ApcbInstanceIndexForBoard

If the ID, or ID and Rev, values match, this is corresponding APCB instance to use for this Board ID.

Instance Examples

The mask/value/instance array has to be properly configured to select the corresponding APCB instance ID. An instance of APCB is loaded only if (BoardID AND Mask) == Value).

```
// Read Board_ID from Smbus device

APCB_TYPE_HEADER    ApcbTypeHeader = {APCB_GROUP_PSP,
    APCB_PSP_TYPE_BOARD_ID_GETTING_METHOD,
    (sizeof(APCB_TYPE_HEADER) + sizeof (PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT)), 0, 0, 0 };
PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT BoardIdGettingMethod =
{
    BOARD_ID_METHOD_SMBUS,    // Smbus device method
    0,    // I2C controller number
    0x4e,    // Smbus address
    0,    // register index
    {    // Id_Apcb_Mapping
        {0x7, 0, 0},
        {0x7, 1, 0},
        {0x7, 2, 0},
        {0x7, 3, 0},
        {0x7, 4, 1},
        {0x7, 5, 1},
        {0x7, 7, 1}
    }
};

// Read Board_ID from FCH GPIOs

APCB_TYPE_HEADER    ApcbTypeHeader = {APCB_GROUP_PSP,
    APCB_PSP_TYPE_BOARD_ID_GETTING_METHOD,
    (sizeof(APCB_TYPE_HEADER) + sizeof (PSP_GET_BOARD_ID_FROM_GPIO_STRUCT)), 0, 0, 0 };
PSP_GET_BOARD_ID_FROM_GPIO_STRUCT BoardIdGettingMethod =
{
    BOARD_ID_METHOD_GPIO,    // GPIO method
    42,    // Gpio0
    1,    // Gpio0IoMux
    0,    // Gpio0BankCtl

    0xff,    // Gpio1
    0xff,    // Gpio1IoMux
    0xff,    // Gpio1BankCtl

    0xff,    // Gpio2
    0xff,    // Gpio2IoMux
    0xff,    // Gpio2BankCtl

    0xff,    // Gpio3
    0xff,    // Gpio3IoMux
    0xff,    // Gpio3BankCtl

    {    // Id_Apcb_Mapping
        {0x1, 0, 0},    // Regular -> Instance 0
        {0x1, 1, 1}    // DAP -> Instance 1
    }
}
```

};

4.3 APCB V2 Components

The APCB v2 is comprised of several pieces and tools. An understanding of these pieces and the supporting tools is needed for proper construction of the APCB images into the final ROM image. ABCP v2 uses these components:

- ABL firmware binaries
- APCB data files
- APCB data build tool: [APCB Tool](#)
- AGESA post/runtime services driver: [APCB Services](#)

4.3.1 Using multiple Board Support

When it is necessary to support multiple platforms with a single BIOS image the PSP will need to have APCB descriptions for each specific board. These multiple APCB instances will reside in the BIOS image simultaneously. For this type of build, the platform developer will need to create each APCB instance separately then combine them into one binary.

For a multiple instance APCB build, the APCB tools and code should be moved into separate directories.

1. Create the separate directory for each APCB to be built.
2. Pick an APCB package closest to the target platform based on the processor and DIMM configuration. The file names will include the program name, the package and the memory type.
3. Copy the example `ApcbData/`, `Include/` and `ApcbCreate.bat` to the new build folder
4. Copy `Inc/` under the APCB root directory to the new build folder
5. Copy `AgesaModulePkg\AMDTools\ApcbToolV2\Tools\` to the new build folder The new build folder for APCB should now look like below:

```
<NewDir>
├── External
│   ├── ApcbData
│   ├── Include
│   └── ApcbCreate.bat
├── Inc
└── Tools
```

6. Override `\External\Include\ApcbCustomizedDefinitions.h` with the platform specific settings
7. Override other source files under `\External\ApcbData\` if necessary
8. Run `ApcbCreate.bat` CLEAN BUILD to produce the APCB binary under the `\External\release\`
9. Embed the APCB binary into the BIOS image

4.4 APCB V3.0 Components

The APCB v3.0 is comprised of several pieces and tools. An understanding of these pieces and the supporting tools is needed for proper construction of the APCB images into the final ROM image. APCB v3 uses these components:

- ABL firmware binaries
- APCB data files
- APCB data build tool: [APCBv3 Tool](#)
- PSP BIOS Directory XML
- AGESA post/runtime services driver: [APCB Services](#)
- CBS linking

- APCB V3 binary view/edit tool

4.4.1 ABL Firmware binaries

The code that executes on the PSP processor is pre-compiled and signed for security reasons. They are located in the PI package at:

```
AgesaModulePkg\Firmwares\<SOC>\AgesaBootloader_U_prod_[Mcm_]SOC>.bin
```

Where 'SOC' is the abbreviation for the product solution, 'MCM' applies to certain large systems. Check with your AMD representative for more information. Examples are. Several files in this directory must be included into the ROM image and referenced by the PSP_Directory as described in [PSP Architecture specification](#). Examples for the AMD reference boards can be found in the AGESA PI releases. A useful file in this directory is:

```
AblPostCode.h
```

It contains the definitions of the POST codes used by the ABL and is very useful in tracking the ABL progress.

4.4.2 APCB data files

The files contained in this directory tree are used to construct the platform's APCB image. There are data files that contain the specific parameter settings for your platform and there are control files used to process the data files. The platform porting engineer must modify these files to match the needs of the platform.

```
AgesaPkg\Addendum\Apcb\<ProgramSolution>
```

Where 'ProgramSolution' is a string identifying your target platform. The AGESA PI package has several variations that can be used as starting samples. Pick the one that is the closest match to your platform and make a copy using your own platform name.

The ProgramSolution names typically vary by including the processor name, socket, memory technology and Board detection method. Examples are:

```
RomeSp3Rdimm, RomeSp3Udimm, RomeAm4Udimm
```

Files contained at this (top) directory level are:

ApcbCreate.bat This is the main script for building APCB data for the program solution. It needs to be modified to specify the list of APCB binary files to include into the image.

Subdirectory: Include

These are the common header files that will define the parameter settings used if no board specific values are given, e.g. the defaults.

```
<ProgramSolution>\Include\
```

Files contained in this directory are:

ApcbDefaults.h	Defaults for build configuration definitions of the APCB data for the program solution
ApcbAutoGen.h	APCB token data and settings. This file is auto-generated by CBS script from the CBS setup database. Platform BIOS is responsible to run CBS script and keep this file updated. See CBS linking below.
ApcbCustomizedDefinitions.h	Common custom settings for all customer boards of this ProgramSolution type. This is also a pattern file to copy to board specific sub-directories, see below.

ApcbCustomizedBoardDefinitions.h Common custom settings for a specific board. This is also a pattern file to copy to board specific sub-directories, see below.

Subdirectory: ApcbDataDefaultRecovery

This directory contains configuration data for all boards of this ProgramSolution type. It constitutes the recovery instance of APCB data and is also used as the platform set baseline settings. Modify this file to make the board list to match your common platform needs. Specific settings can be 'overridden' by using a platform specific sub-directory, see below.

<ProgramSolution>\ApcbDataDefaultRecovery\

Files contained in this directory are:

SetApcbBoardList.bat Board selection for APCB data for default and recovery binary. This file controls the list of boards to be included into the image. The list can have a single board, or more than one when multi-platform support is used. Modify the board list for the recovery APCB instance. The names added to the list are the sub-directory names for each board variant, see below.

Subdirectory: CRB_<CrbBoardName>

The files in this directory provide parameter settings specific to the named board. The setting values in these files will take precedence over the common and the default values.

<ProgramSolution>\ApcbDataDefaultRecovery\CRB_<CrbBoardName>

Where 'CrbBoardName' is the name given to the platform. Examples in the AGESA PI: 'EthanoX', 'Myrtle', 'Diesel'.

Files contained in this directory are:

ApcbCustomizedBoardDefinitions.h Settings specific to this board variant. Also, make sure to set the board mask for ApcbData_*.c. See section on supporting multiple board variances.

SetApcbDataFileList.bat This control file is called to add the needed files in this directory to the list of files to be used to create the APCB image. Modify this file to update the list of board specific APCB data files (files like: ApcbData_*.c) contained in this sub-directory.

Note: you can have multiple directories like this, one for each board variation that needs unique parameter settings from the other variations. Each directory will have a unique board name ('CrbBoardName') and each contains the above files, each modified to reflect the specific needs of the board variant.

Sub-Directory: ApcbDataUpdatable

This directory contains APCB data type files whose content tokens are allowed to be targeted by APCB service updates. These Data file settings are applied to all boards, and are the location where the data written by the APCB service is kept. These files are auto generated by a CBS script.

<ProgramSolution>\ApcbDataUpdatable\

The data in these files specifies token pairs (ID, value) divided by class (Boolean, 1Byte, 2Bytes, 4Bytes) and purpose (normal, debug).

Files contained in this directory are:

ApcbData_<SOC> _GID_0x3000_Type _Token<Class>.c	Contains the token/value pairs for the modifiable tokens (e.g. SETUP options)
ApcbData_<SOC> _GID_0x3000_Type _Token<Class>Debug.c	Debug data is used by CBS operation which is linked to an APCB token through ApcbAutoGen.h. Tokens which have a default setting other than 'Auto' will have a setting here.

Sub-Directory: ApcbDataEventLog

This directory contains APCB data for event logging. Optionally built per ProgramSolution since only some features support this type of data, such as: DDR post package repair.

<ProgramSolution>\ApcbDataEventLog\

Files contained in this directory are:

ApcbData_<SOC> _GID_0x1704_Type _DdrPostPackageRepair.c	Initial APCB data for DDR Post Package Repair event logging structure. This creates the storage buffer where the repair entries will be kept.
--	--

Sub-Directory: <Release>

Where 'Release' is the kind of build being performed. This is the output directory for the generated APCB binary files that will be included into the PSP BIOS directory.

<ProgramSolution>\<Release>

Files contained in this directory are:

APCB_SSP_D4_DefaultRecovery.bin	The platform settings. This includes the common settings and the board specific settings.
APCB_SSP_D4_Updatable.bin	The platform SETUP option token/value pairs.
APCB_SSP_D4_EventLog.bin	The logging space for certain features.

4.4.3 APCB build tool

The APCB binary build tool executable which generates APCB V3 compliant binary from the compiled APCB binary data files listed above. The tool is located at:

AgesaModulePkg\AMDTools\ApcbToolV3\Tools\ApcbToolV3.exe

4.4.4 PSP BIOS directory XML

Samples of the PSP directory XML with APCB binary integration information on BIOS Entry and Instance numbers can be found here:

AgesaModulePkg\AMDTools\NewPspKit\Sample\PspDirectory*.xml

Use these samples to create your platform specific file. This will be used by the PSPDirectoryTool for creating the PSP_Directory image. See also [PSP Architecture specification](#).

4.4.5 CBS linking

For CBS settings that are not 'auto', a script auto-generates this APCB token data and setting header file from the CBS database.

```
AmdCbsPkg\Library\Family\0x17\<SOC>\External\Resource<SOC>\ApcbAutoGen<SOC>.h
```

This file identifies the tokens to be connected with the CBS controls and should be used to replace ApcbAutoGen.h used by APCB binary data files in: AgesaPkg\Addendum\Apcb\<ProgramSolution>\Include\ApcbAutoGen<SOC>.h .

The script file used to generate this file is located in the CBS package. It is a CBS perl tool used to process a CBS XML file to generate the token and header files consumed by setup UI driver. Location:

```
AmdCbsPkg\Tools\CBSgenerate.pl
```

This build tool is typically included in the platform BIOS build process. AMD provides a batch file

```
AmdCbsPkg\Library\Family\0x17\<SoC>\External\xmlparse.bat
```

which can be used by the customer to integrate into the BIOS build process. Customer must set the environment variable "%PERL_PATH%" to indicate the location of the Perl interpreter on their development system.

4.4.6 APCB View/Edit Tool

A GUI based APCB V3 tool is provided for viewing and editing the APCB binary data. The APCB V3 header files need to be referenced for decoding the APCB binary content and the binary must match the header files from the same PI package.

```
AgesaModulePkg\AMDTools\ApcbV3BinTool\
```

Files contained in this directory are:

ApcbV3BinTool.exe

This is the APCB V3 GUI binary tool executable. It requires supporting files in the folder and sub-folder (ApcbV3Editor), APCB V3 header files, and the PspDirectory Tool kit within the AGESA PI package. Therefore, it is required to be executed in place within the AGESA package structure; or, optionally, use the below Generate_MemoryTuningKit.bat to help remove the requirement of execution in place.

Generate_MemoryTuningKit.bat

This is a packaging script that collects the necessary tool and database information needed to run the ApcbV3BinTool outside of the AGESA PI package. The script creates a folder called "MemoryTuningKit" for running as a standalone package.

4.4.7 Platform APCB Porting

The APCB v3.0 is built from several binary images created by the APCB tool set. The various directories and files contained therein are listed in [APCB V3 Components](#).

The steps below illustrate what actions need to be taken to customize a platform to use the component files.

Note: <SOC> refers to the processor name abbreviation. The first PI package to use the APCB V3.0 feature uses a name abbreviation of "SSP".

1. APCB_<SOC>_D4_DefaultRecovery.bin

- This binary keeps default APCB settings and is treated as the recovery copy.
- BIOS Entry 0x68, Instance 0 is the fixed location in PSP/BIOS Directory that should include this binary.
- Lists of build configuration data that are included in this binary are:
 - Structured data for each APCB Group/Type

- APB V3 UID tokens data (includes BOOL, 1-byte, 2-bytes, and 4-bytes type)
- Build time settings of all supported board (Multiple Board APB)
- In APB V3 operation, this binary is read-only at all times. No APB data write operation will be performed by APB V3 services.
- NOTE: Size of this image in PSP BIOS Directory XML should be limited to 0x2000 for current implementation.

2. APB_<SOC>_D4_Updatable.bin

- This binary keeps updated APB settings for all updatable data types.
- BIOS Entry 0x60, Instance 0 is the fixed location in PSP/BIOS Directory that should include this binary. This binary should also be included in Entry 0x68, Instance 8 as the recovery copy.
- List of updateable data settings includes:
 - APB V3 UID tokens data
- Note: Data of this binary are designed to apply all boards regardless which board is detected in a multiple board system.

3. APB_<SOC>_D4_EventLog.bin

- This binary keeps event log settings that gets logged by SMM services at runtime.
- Optionally, if event log feature is supported by the product. BIOS Entry 0x60, Instance 1 is the fixed location in PSP/BIOS Directory should include this binary.
- A recovery copy for this binary should be included at BIOS Entry 0x68, Instance 9 as fixed location in PSP/BIOS Directory.
- List of data are included in the binary
 - Initial structured data of data types for runtime data logging. E.g. DDR Post Package Repair data
- Note: Data in this binary are designed to apply all boards regardless which board is detected in a multiple board system.

4. APB_<SOC>_D4.bin - Deprecated

- This binary is a clone of APB_<SOC>_D4_DefaultRecovery.bin and is present only to help with APB V3 transition.
- This binary should not be used in APB V3 applicable SOC programs.

5. PSP BIOS Directory XML example

- Single level directory implementation should implement level 2 entries in two level directories implementation.
- Two level directories implementation example (ref: [PSP Architecture](#)) :

```
<BIOS_DIR Level="1" Base="2449408" Size="0x40000" SpiBlockSize=" 0x1000">
  <IMAGE_ENTRY Type="0x68" Instance="0x00"
    File="Build\Apb\Rome\APB_SSP_D4_DefaultRecovery.bin"
    Size="0x2000"/>
  <IMAGE_ENTRY Type="0x68" Instance="0x08"
    File="Build\Apb\Rome\APB_SSP_D4_Updatable.bin"
    Size="0x1000"/>
  <IMAGE_ENTRY Type="0x68" Instance="0x09"
    File="Build\Apb\Rome\APB_SSP_D4_EventLog.bin"
    Size="0x1000"/>
  ...
</BIOS_DIR>
<BIOS_DIR Level="2" Base="7254016" Size="0x40000" SpiBlockSize=" 0x1000">
  <IMAGE_ENTRY Type="0x68" Instance="0x00"
    File="Build\Apb\Rome\APB_SSP_D4_DefaultRecovery.bin"
    Size="0x2000"/>
  <IMAGE_ENTRY Type="0x68" Instance="0x08"
    File="Build\Apb\Rome\APB_SSP_D4_Updatable.bin"
    Size="0x1000"/>
  <IMAGE_ENTRY Type="0x68" Instance="0x09"
```

```

File="Build\Apcb\Rome\APCB_SSP_D4_EventLog.bin"
Size="0x1000"/>
<IMAGE_ENTRY Type="0x60" Instance="0x00"
File="Build\Apcb\Rome\APCB_SSP_D4_Updateable.bin"
Size="0x1000"/>
<IMAGE_ENTRY Type="0x60" Instance="0x01"
File="Build\Apcb\Rome\APCB_SSP_D4_EventLog.bin"
Size="0x1000"/>
...
</BIOS_DIR>

```

Table 19. APCB v3 File Summary

File Name	Req ³	Dir Lvl	Type	Inst.	R/W	Default for	Norm	Recov	Comment
APCB_<SOC>_D4_DefaultRecovery.bin	Y	1	0x68	0	RO	All Boards default	Y	Y	Build time image. Size limit is 0x2000
APCB_<SOC>_D4_Updateable.bin	Y	1	0x68	8	RO	CBS defaults	N	Y	Build time image
APCB_<SOC>_D4_EventLog.bin	N (*Y)	1	0x68	9	RO	Initial empty setting	N	Y	Build time image *Required, if normal operation copy is used
APCB_<SOC>_D4_DefaultRecovery.bin	Y	2	0x68	0	RO	All Boards default	Y	Y	Build time image. Size limit is 0x2000
APCB_<SOC>_D4_Updateable.bin	Y	2	0x60	0	RW	CBS defaults	Y	N	Will be updated at POST/Run time during BIOS configuration, CBS write operation
_Updateable (cont.)	Y	2	0x68	8	RO	CBS defaults	N	Y	Build time image
APCB_<SOC>_D4_EventLog.bin	N (*Y)	2	0x60	1	RW	Initial empty setting	Y	N	*Required for products with features using EventLog
_EventLog (cont.)	N (*Y)	2	0x68	9	RO	Initial empty setting	N	Y	Build time image *Required, if normal operation copy is used
APCB_<SOC>_D4.bin	N	n/a	n/a	n/a	n/a	All Boards default	N	N	This file should NOT be used anymore

4.4.8 Managing Data

APCB configuration parameters fall into two broad categories:

- APCB Tokens** An individual control used to modify a single characteristic. These will have a name starting with "APCB_TOKEN_" and will be defined in the interface spec.
- Control Structures** A group of controls related to a common feature or service. These items are built into a data structure in the APCB for the ABL to access. The structures are not visible, but the elements may be visible via a user parameter. These parameter names will start with "BLDCFG_" and are

3

Table columns:

-
-
-
-
-

Req - Required
Dir Lvl - Directory level
Inst. - Instance#
Norm - Normal mode
Recov - Recovery mode

generally not defined in the specification. If such a parameter is made visible to the user, it will be assigned an APCB_TOKEN.

Before progressing into the details, a few terms need to be defined to best understand the APCB internals.

- Groups** APCB Groups are a collection of parameters that share a common AGESA component (e.g. PSP, DF, Mem, FCH, GNB, CBS). Groups are comprised of Types. You can find Group definitions in file: AgesaPkg\Addendum\Apcb\Inc\SSP\ApcbDataGroups.h
- Types** APCB Types identify a data element, often a structure containing several parameters. The Type definitions can be found in files with names like Apcb<Group>Group.h, for example: AgesaPkg\Addendum\Apcb\Inc\SSP\ApcbMemGroup.h
- Tokens** APCB Tokens are individual configuration parameters. They have UUIDs (Unique Identifier) so that they can be located and individually modified; sometimes they are related to a PCD control. The token UUIDs are defined in file: AgesaPkg\Addendum\Apcb\Inc\SSP\ApcbV3TokenUid.h
- Purposes** APCB tokens can have several instances - one per 'purpose'. A 'purpose' declares a usage priority for the instance. This creates an ability to have a standard set of parameter values for a platform AND have an alternative sub-set of values for temporary debugging. This prevents the accidental oversight of not removing a debug only setting before production. As debug values are proven useful, they can be migrated to the standard set; then at production, the entire debug set of 'overrides' can be removed in one simple step.

The list of APCB 'Purposes' and their priority are defined in [PSP APCB Services](#).

Customizing Controls

To establish a customized value for a control that is already defined as a token or structure member, check these steps:

- Locate the customization file that does, or should contain the control. Possible locations and priority from high to low are:
 - <ProgramSolution>\ApcbDataDefaultRecovery\<BoardName>\ApcbCustomizedBoardDefinitions.h
 - <ProgramSolution>\Include\ApcbCustomizedBoardDefinitions.h
 - <ProgramSolution>\ApcbDataDefaultRecovery\<BoardName>\ApcbCustomizedDefinitions.h
 - <ProgramSolution>\Include\ApcbCustomizedDefinitions.h
- Locate APCB Token UUIDs and value definitions. They always occur in pairs, one for the UUID and one for the value assigned:
 - APCB_TOKEN_UUID_<Module>_<Control> - the UUID definition for the token.
 - APCB_TOKEN_UUID_<Module>_<Control>_VALUE - the value to be assigned to the token.

For example, the token name for the ECCEnable feature in the memory module is: ENABLEECCFEATURE so its UUID and value definition names are:

- APCB_TOKEN_UUID_ENABLEECCFEATURE
- APCB_TOKEN_UUID_ENABLEECCFEATURE_VALUE
- Add/remove/alter the macro in the file that establishes the token/value pair. The macros specify the <Type> as ('Boolean' | '1Byte' | '2Byte' | '4Byte'). There are two forms for the macros:
 - APCB_TOKEN_VAL_<Type> (TokenName)
 - This is the short form, you only have to provide the UUID and the macro will access the value by appending the '_VALUE' ending.
 - APCB_TOKEN_<Type> (TokenName, TokenValue)

- This is the long form requiring both the UID and the Value as parameters. This form does offer the option of using a hard-coded hex value for the value parameter.

The default value is declared in file:

```
AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\Include\ApcbDefaults.h
#define APCB_TOKEN_UID_ENABLEECCFEATURE_VALUE FALSE
```

The new token value is declared in a board specific file:

```
AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\Include\ApcbCustomizedDefinitions.h
#define APCB_TOKEN_UID_ENABLEECCFEATURE_VALUE TRUE
```

Then the new token instance is declared in a board specific file:

```
AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\ApcbDataDefaultRecovery
\CRB_EthanolX\ApcbData_SSP_GID_0x3000_Type-TokenBoolean.c
```

using the token macros as follows:

```
APCB_TOKEN_PAIR_BOOL TokenList[] = {
    ...
    APCB_TOKEN_VAL_BOOL (APCB_TOKEN_UID_ENABLEECCFEATURE),
    ...
};
```

- Locate a structure element definition. These use the "BLDCFG_" prefix. For example, the console out filter enable control is: BLDCFG_CONOUTCTRL_ABL_CONSOLE_FILTER_ENABLE. The default value is declared in file:

```
AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\Include\ApcbDefaults.h
```

The new value is set in file:

```
AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\Include\ApcbCustomizedDefinitions.h
```

And the instance is declared in file:

```
AgesaPkg\Addendum\Apcb\RomeSp3Rdimm\ApcbDataDefaultRecovery
\CRB_EthanolX\ApcbData_SSP_GID_0x1704_Type_ConsoleOutControl.c
```

Data Precedence Summary

The above sections mention several include files and data files making token declarations. To summarize how these files interact and review how the values take precedence, this table is provided.

Table 20. Data Values Precedence

Path/File ⁴	Include Sequence	Effective Precedence ⁵	Comments/ Entry style / Handling Method
For the recovery image ...			
Include\ ApcbCustomizedDefinitions.h	1	3rd	#define APCB_TOKEN_UID_<Control>_VALUE <Value> #define BLDCFG_<Control> <Value>
Include\ ApcbCustomizedBoardDefinitions	2	2nd	#undef APCB_TOKEN_UID_<Token>_VALUE #define APCB_TOKEN_UID_<Token>_VALUE <NewValue>

⁴ path relative to AgesaPkg\Addendum\Apcb\<ProgramSolution>

⁵ low number value takes precedence over higher number value

Table 20. Data Values Precedence (continued)

Path/File ⁴	Include Sequence	Effective Precedence ⁵	Comments/ Entry style / Handling Method
For the recovery image ...			
Include\ ApcbDefaults	3	4th	<pre>#ifndef APCB_TOKEN_UID_<Control>_VALUE #define APCB_TOKEN_UID_<Control>_VALUE <DefaultValue> #endif //Handles earlier token BLDCFG_<TokenControl> definitions, //if user definition exists, then take earlier definition #ifdef BLDCFG_<TokenControl> #undef APCB_TOKEN_UID_<CorrespondingNewControl>_VALUE #define APCB_TOKEN_UID_<CorrespondingNewControl>_VALUE BLDCFG_<TokenControl> #endif</pre>
Include\ ApcbAutoGen.h	4	1st	#define APCB_TOKEN_UID_<Control>_VALUE <Value>

Table 20. Data Values Precedence (continued)

For a Board Specific image ...			
if exists ApcbDataDefaultRecovery\<BoardName>\ ApcbCustomizedDefinitions.h else Include\ ApcbCustomizedDefinitions.h	1	2nd	<pre>#define APCB_TOKEN_UID_<Control>_VALUE <Value> #define BLDCFG_<Control> <Value></pre>
if exists ApcbDataDefaultRecovery\<BoardName>\ ApcbCustomizedBoardDefinitions else Include\ ApcbCustomizedBoardDefinitions	2	1st	<pre>#undef APCB_TOKEN_UID_<Token>_VALUE #define APCB_TOKEN_UID_<Token>_VALUE <NewValue></pre>
Include\ ApcbDefaults	3	3rd	<pre>#ifndef APCB_TOKEN_UID_<Control>_VALUE #define APCB_TOKEN_UID_<Control>_VALUE <DefaultValue> #endif //Handles earlier token BLDCFG_<TokenControl> definitions, //if user definition exists, then take earlier definition #ifdef BLDCFG_<TokenControl> #undef APCB_TOKEN_UID_<CorrespondingNewControl>_VALUE #define APCB_TOKEN_UID_<CorrespondingNewControl>_VALUE BLDCFG_<TokenControl> #endif</pre>

4.4.9 Multiple Board Support

A single BIOS ROM image may support one or more distinct platform board variations. Depending on how close these variations match, each board will likely need some unique APCB settings. As noted earlier, each

⁴ path relative to AgesaPkg\Addendum\Apcb\<ProgamSolution>

⁵ low number value takes precedence over higher number value

board variant has its own sub-directory to contain its sub-set of settings that are different for that board. These board files do not need to contain a full set of setting declarations, just the ones that are unique for this board variant.

The first issue to decide when planning to support multiple boards is - how to identify, from code, which board is the one currently being used. See [Selecting one of Multiple Boards](#). This is the Board ID and can be a value from 0x00 to 0x0F.

Each Config Data file in the tree contains an APCB record header that indicates the data group, the data type and a Board Mask. The Board Mask indicates which board instances the data record will apply. The Board ID is translated to a bit mask (max 16 bits) and matched to (Anded with) the Board Mask. If the result is non-zero, the contents of the Data record will be applied to this board.

Plan the Data

When dealing with multiple board variants, you need to separate the config data controls into groups of 'common settings' and 'board specific' settings. Usually there is a first, or 'base', board from which the other variants are derived.

Place all of the common settings into the defaults files at the top directory level

```
<ProgramSolution>\ApcbDataDefaultRecovery\
```

For the discussion to follow, three boards are considered

- 'BaseBoard' - the first or most generic of the boards. It uses most, or all, of the common settings.
- 'VarBeta' - the first variant which has some special features.
- 'VarGamma' - another variant with extended features for debug.

These boards all belong to the same marketing family of products 'ToyBox'.

1. Choose a CRB with a full list of APCB data files as the pattern for your ToyBox folder.
2. Clone the CRB folder to a new folder named 'ToyBox'. Include the sub-directories.
3. Locate and edit the file: ToyBox\ApcbCreate.bat

- Find the line entry for "SET APCB_MULTI_BOARD_SUPPORT="
- Make sure its value is set =1 to enable the multi board support.
SET APCB_MULTI_BOARD_SUPPORT=1
- Make sure its value is set =0 if only a single board variant exists.
SET APCB_MULTI_BOARD_SUPPORT=0

4. Locate and edit file: ToyBox\Include\ApcbCustomizedBoardDefinitions.h

- Find the entry line for "#define BLDCFG_APCB_DATA_BOARD_MASK "
- Make sure this is set for APCB_BOARD_INSTANCE_ALL_KNOWN
- since this dir is for the common settings, it applies to all boards.
#define BLDCFG_APCB_DATA_BOARD_MASK APCB_BOARD_INSTANCE_ALL_KNOWN

5. Locate and edit the file: ToyBox\Include\ApcbCustomizedDefinitions.h

- Review the settings declared in this file. Make changes that will apply to all boards in the marketing family. These are the 'common settings' you separated early in the planning. Only the settings that differ from the defaults (ApcbDefaults.h) need to be listed in this file.

6. Locate the directory: ToyBox\ApcbDataDefaultRecovery\

This directory will have two important sub-directories; an example base board and 'CustomBoard_Example'. In the AMD Rome_PI, the marketing name used is 'RomeSp3Rdimmm' and the base board is 'CRB_EthanolX'. When you cloned the ToyBox directory, you got these sub-dirs too.

- Rename the CRB base board directory to 'ToyBox'

- Clone the directory 'CustomBoard_Example' to a new directory named 'VarBeta'. This will contain the settings for the Beta variant board.
- Clone, or rename, the directory 'CustomBoard_Example' to a new directory named 'VarGamma'. This will contain the settings for the Gamma variant board.

7. Locate and edit the file: ToyBox\ApcbDataDefaultRecovery\SetApcbBoardList.bat

- Find the entry for "SET APCB_DATA_BOARD_DIR_LIST="
- Make this entry contain the list of all board variants

```
SET APCB_DATA_BOARD_DIR_LIST= BaseBoard VarBeta VarGamma
```

8. Locate and edit file: ToyBox\ApcbDefaultRecovery\BaseBoard\ApcbData_<SOC>_GID_0x1704_Type_BoardIdGettingMethod.c

- Modify the code in this file to match the hardware method on your platform for reading the BoardID. See [Selecting one of Multiple Boards](#).
- Only one method is supported at a time.
- Set the Board Index in the mapping table for the variants.

For this discussion, the hardware provides a 3-bit BoardID value. 'BaseBoard' will respond with a HW value of 0 and be board index 0; 'VarBeta' will respond with a HW value of 4 and will be board index 1 and 'VarGamma' will respond with HW value 6 and be board index 2.

```
{ // Apcb_Mapping {HwMask, HwValue, board index}
{0x07, 0x00, 0}, // The Base Board Index = 0, Board Mask = Instance_1
{0x07, 0x04, 1}, // VarBeta, Board Index = 1, Board Mask = Instance_2
{0x07, 0x06, 2}, //VarGamma, Board Index = 2, Board Mask = Instance_3
{0x07, 0x07, 3}, //VarGamma2, Board Index = 3, Board Mask = Instance_4
}
```

9. Locate and edit file: ToyBox\ApcbDefaultRecovery\VarBeta\ApcbCustomizedBoardDefinitions.h

- Set the board instance for the variant
 - Find the entry for "#define BLDCFG_APCB_DATA_BOARD_MASK"
 - Make this entry declare APCB_BOARD_INSTANCE_2
 - Repeat for VarGamma and use APCB_BOARD_INSTANCE_3

```
#define BLDCFG_APCB_DATA_BOARD_MASK APCB_BOARD_INSTANCE_2
```

Note: a board settings can be made to apply to more than one board instance. For example, if the VarGamma board is modified for electrical reasons (VarGamma2) and responds with a BoardID of 7 but uses the same config data, then the Board Mask for the VarGamma directory should use

```
#define BLDCFG_APCB_DATA_BOARD_MASK APCB_BOARD_INSTANCE_3 \
+ APCB_BOARD_INSTANCE_4
```

- Add declaration lines for the platform specific settings for the variant
 - Use the example lines in the file and insert the controls and new values to be used on the platform variant

```
#undef APCB_TOKEN_UID_<CONTROL_NAME>
#define APCB_TOKEN_UID_<CONTROL_NAME> NewValue
```

For example, to set the ECC Enable control:

```
#undef APCB_TOKEN_UID_ENABLEECCFEATURE_VALUE
#define APCB_TOKEN_UID_ENABLEECCFEATURE_VALUE TRUE
```

10. Itemize the variant platform specific setting files

- The BaseBoard directory will contain a full list of config data files

- Copy the config data files (*_GID_0x170?_Type_*.c) that contain the platform specific setting you determined in the early planning to the VarBeta and/or VarGamma directories. Only the files that contain your target values need to be copied. This should be a small set.
- Edit the file: VarBeta\SetApcbDataFileList.bat
- Find the entry for "APCB_DATA_TYPE_FILE_LIST". There are many such entries using the "A=A+B" style of declaration.
- Make the list contain an entry for each config data file located in the variant's directory. Follow the examples in the file.

At this point you should have a directory structure like this:

```

ToyBox\ApcbCreate.bat
    \ApcbCreateInt.bat

ToyBox\Include\ApcbAutoGen.h
    \ApcbCustomizedBoardDefinitions.h
    \ApcbCustomizedDefinitions.h
    \ApcbDefaults.h

ToyBox\ApcbDataDefaultRecovery\SetApcbBoardList.bat

ToyBox\ApcbDataDefaultRecovery\BaseBoard\SetApcbDataFileList.bat
    \SetApcbDataFileListInternal.bat
    \ApcbData_SSP_GID_0x1000_Type_???.c
    \ApcbData_SSP_GID_0x170?_Type_??*.c
    \ApcbData_SSP_GID_0x3000_Type-Token*.c

ToyBox\ApcbDataDefaultRecovery\VarBeta\ApcbCustomizedBoardDefinitions.h
    \SetApcbDataFileList.bat
    \ApcbData_SSP_GID_0x1704_Type_??*.c

ToyBox\ApcbDataDefaultRecovery\VarGamma\ApcbCustomizedBoardDefinitions.h
    \SetApcbDataFileList.bat
    \ApcbData_SSP_GID_0x1704_Type_??*.c

ToyBox\ApcbDataEventLog\ApcbData_SSP_GID_0x1704_Type_DdrPostPackageRepair.c

ToyBox\ApcbDataUpdatable\ApcbData_SSP_GID_0x3000_Type-Token*.c
    \ApcbData_SSP_GID_0x3000_Type-Token*Debug.c

```

5 Platform Topics and Services

The Platform Driver is specific to the platform and is generated by and for the platform porting host BIOS. The Platform driver may perform several duties which require it to access AMD components within the system. This section is used to collect several features of the AGESA™ package that provide some services for use by the Platform driver to help it accomplish its duties; such as:

- Multi-Socket Considerations
- DDR4 Package Repair
- Dynamic Configuration Updates - see [APCB Service Protocol](#)
- Run-Time controls via ACPI

In addition to these, some features or services are enabled by the presence of an external Micro-Controller, such as:

- 4-corner memory training

Please see section 6.

5.1 Platform CMOS

The standard CMOS uses IO pair 0x70/71 for the Address and data port for accessing the non-volatile battery backed storage. The offsets 0x00 - 0x0D are not general storage, but specialized registers for Real Time Clock control (RTC). In a sense, the 14 bytes (0x00..0x0D) are 'wasted' as storage.

5.1.1 CMOS used by AGESA™

For AMD products, the CMOS RAM has 256 bytes and can be directly accessed through IO pair 0x72/0x73 (without bank swapping, or 'paging' 128 bytes at a time). When using this access to CMOS through the IO pair 0x72/0x73, the storage at offset 0x00 - 0x0D is now accessible and the RTC special purpose registers are not available. These 14 bytes are 'hidden' from general system users. AGESA reserves this shadow CMOS region for private use. The storage is allocated as below:

Table 21. CMOS storage usage

CMOS offset	Used by Driver	Description
0x00..0x01	FCH	Used to save SATA ports status, which will be saved when normal boot and used when resume from S3 to shutdown the clock to the unconnected SATA ports.
0x04	ABL	Reserved for overclock feature and defined in ApcbData_XX_GID_0x1704_Type_MemOverclockConfig.c
0x05	ABL	Reserved for overclock failing count.
0x06..0x07	ABL	Used to identify the validity of APCB, if the value is 0xA55A, ABL load the APCB from normal copy or else load from backup entry.
0x08..0x0B	FCH	Used as FchConfigPtr, which is passed to AGESA FCH ASL code.
0x0D[2:0]	ABL	Reserved for MemContextRestore flags. • bit 0 - CMOS_BITMAP_MEM_RESTORE_BOOT_FAIL • bit 1 - CMOS_BITMAP_DISCARD_MEM_CONTEXT

5.1.2 Power for CMOS

For many embedded systems the usage of a coin cell battery to supply G3 state power to the RTC-CMOS is not desirable for field maintenance reasons. The embedded team has published a recommended circuit on the DevHub web site to replace the coin cell battery.

There are some trade-offs for using the replacement circuit. When using this replacement circuit there is no back-up power in G3 HW state, only S5 state since the power is supplied by the S5 rail. This means that the RTC (time, date, etc.) and CMOS storage content WILL BE CORRUPTED across a G3 shutdown.

Certain features of the AMD AGESA code set depend upon the use of sustained CMOS content. When using the replacement circuit the CMOS content is not retained and thus the features that depend upon it are not available.

The known features that depend upon a sustained CMOS content are:

- Writable APCB instances, see [Platform Writes to Flash](#).
- APCB recovery, see [PcdAmdPspApcbRecoveryEnable](#).
- Memory Context Restore
- CBS options (not covered in this spec) These are SETUP User Interface selected options that may rely upon a sustained CMOS content to retain the user's selection.

Note: These are the known features at this time.

5.2 Platform Writes to Flash

Some customers have indicated their desire to have a static Flash ROM image. Either to satisfy industry inspection requirements or corporate security policy. AMD has published new Flash Write Protect information in the product [PPRs](#) to indicate how to implement write protections on the SPI bus.

There are some trade-offs. When using the ROM Protection options, care must be taken to leave the UEFI NV variable storage and APCB sections writable. However, this is not a requirement as a platform can run very well without writing to the Flash ROM; but, certain features of the UEFI based code set depend upon the use of a 'writable' APCB instance and hence will be unavailable.

The known features that depend upon a writable APCB instance are:

- [Post Package Repair](#).
- [Memory Context Restore](#).
- User options (not covered in this spec). These are SETUP User Interface selected options that rely upon a writable APCB instance to retain the user's selection.
- FCH SPI bus speed settings for boot time improvements.
- Updating system PCDs.

Note: These are the known features at this time.

This feature uses a CMOS flag to indicate updated instances; so, it is affected by the platform CMOS power design. Please see [Platform CMOS power](#).

5.3 DDR4 Post Package Repair

DDR4 Post Package Repair is a [Jedec](#) DDR4 feature that allows bad rows in a DRAM device to be replaced using a repair mechanism built into the DRAM device. This improves the reliability/availability and helps reduce replacement costs of DIMMs.

The BIOS maintains a list of all repairs for the system, stores the list in a nonvolatile storage area and passes this information to the ABL via the APCB.

The platform BIOS will implement a policy and method to determine if a detected error is one that can be repaired by DDR4 post package repair. If one or more errors are discovered, the errors are placed into a data structure called “DDR4 Post Package Repair List (DPPRL)”. The DPPRL contains several entries that contain the repair information. The DPPRL is placed in the APCB and made available to the ABL on the next boot. The ABL will extract the information from the APCB and perform the repair(s). After completing repairs, ABL

reports the results of the repair by passing the results to the AgesaMemoryPEIM Driver. This information is available to the platform BIOS via a PPI.

Note: the Post Package Repairs are not performed when the system or PSP is in its recovery mode, as the repair info may not be reliable. See section on [Platform CMOS power](#).

Note: This feature depends on the ability to write the repair list to Flash. See [Platform Writes to Flash](#).

DDR Post Package Repair List

The list consists of an array of DPPRL entries. It is described in the APCB as the “APCB_MEM_TYPE_DDR_POST_PACKAGE_REPAIR”. An example can be found in the file AgesaPkg\Adendum\Apcb\<Platform>\ApcbData\ApcbData_..._DdrPostPackageRepair.c.

```
APCB_DPPRCL_REPAIR_ENTRY DdrPostPackageRepairStruct[] =
{
    Entry1,
    EntryN
}
```

Please note that the “DdrPostPackageRepairStruct” is indexed by the ABL using the “Entry Number”. The “Entry Number” can be from 0-63 where 63 is the maximum number of entries that ABL can correct. The “Entry Number” is also used in the output reporting mechanism to report the status of the repair.

Each entry in the array describes a single repair to be made.

Related Definitions

APCB_DPPRCL_REPAIR_ENTRY

```
typedef struct _APCB_DPPRCL_REPAIR_ENTRY {
    :
} APCB_DPPRCL_REPAIR_ENTRY;
```

This is the same structure used in [ApcbAddDramPostPkgRepair\(\):DPPRCL_REPAIR_ENTRY](#).

Configuration Items

Enablement Category

- **APCB_TOKEN_UID_POST_PACKAGE_REPAIR_ENABLE [F17M30][F17M70]**

This item configures the feature of DRAM Post Package Repair.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Post Package Repair is enabled.
- FALSE - Post Package Repair is disabled.

- **PcdAmdMemCfgMaxPostPackageRepairEntries**

This PCD tokens is applicable during build time. It controls the maximum number of Post Package Repair entries supported in APCB. When PFEH (Platform First Error Handling) is initiated and if there are correctable or uncorrectable error occurring in memory, a repair request is made by updating the entries in APCB for Post Package Repair. This token ensures that the number of entries doesn't exceed the limit.

Permitted Choices: (Type: value <=63)(Default: 63)

5.4 Memory Context Restore [F17M10]

Memory context restore is a feature to speed up the booting process of a system by memorizing the trained context from the last boot. The context is saved in Flash NVstore and used on all subsequent boot cycles, including on both warm and cold resets and S3/S4/S5/G3. Memory training and PMU training is bypassed when this feature is enabled.

Customers do not need to provide any special storage, the context is saved to an area of Flash reserved by the AGESA™ software.

Note: This feature depends on the ability to write the context to Flash. See [Platform Writes to Flash](#).

Memory context restore is triggered only when:

- The APCB token (BLDCFG_MEMORY_RESTORE_CONTROL) is turned on.
- DIMM configuration has not changed. Determined by comparing the SPD content.
- CPU part has not changed. Determined by comparing the CPU Serial number.
- CMOS has not been cleared.
- The system powered up successfully in the last boot. Determined by a CMOS flag (see Platform CMOS).
- No APCB modification has been made. Determined by a CMOS flag (see Platform CMOS). This bit is set by any APCB runtime change made via the run time library.
- The training context is not expired. The context is valid for 30 days by default; but, can be changed with a build option (see below). Memory timings are susceptible to environment and may drift over time/temperature change, so a training pass is forced to occur at this interval.

At present, this feature supports UDIMM and SODIMM systems only and does not include ECC configurations.

Related Definitions

Configuration Items

Enablement Category

- **APCB_TOKEN_UID_MEMRESTORECTL**

This PCD token enables the Memory Context Restoration feature.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Memory context restore is enabled. PMU training is bypassed whenever possible.
- FALSE - Memory context restore is turned off. PMU training is performed each time the system boots.

- **APCB_TOKEN_UID_MEM_RESTORE_VALID_DAYS**

This PCD token sets the maximum interval the system is allowed run without a memory training pass.

Permitted Choices: (Type: Value)(Default: 30)

5.5 Multi-Socket Designs

In a multi-socket system, external GMI (xGMI) high speed links are used to connect between the sockets. High speed signals traveling through long trace lane are subject to high frequency and dielectric losses. To improve signal integrity and Bit Error Rates (BER), the platform designer is provided the ability to specify Transmitter Equalization Coefficients (TX EQ) that are needed to accommodate the unique trace lengths and layout on a given platform. Please refer to Processor Programming Reference (PPR) specification on section “DXIO Subsystem” where it describes PCS (Physical Coding Sub-layer) lane coefficients for more detail.

The following macros are used to specify the TX EQ values for the platform. These values are recorded into the APCB and used by the ABL to establish the socket to socket links. The macros are based on a attribute-data combination. The attribute selects a set of lanes on which to operate and the data indicates the TX EQ value(s) for those lanes in the set.

Note: For optimal settings for a given platform, a tool provided by AMD (AGT) features the xGMI tuning functionality and should be used to select data values for the TX EQ values. Please contact your AMD support representative.

The AGT tool will output final optimized data values for use in the xGMI macros. An example file can be seen in “AgesaPkg\Addendum\Apcb\<board>\ApcbData\ApcbData_GID_0x1703_Type_xGMITxEq.c”. The [APCB_Tool](#) will be used to incorporate this data into the APCB image.

Overview of steps:

1. Run the AGT tool. Instructions are downloaded with the tool from the AMD DEV site. (see your AMD representative).
 - The AGT tool produces upto 3 output files with names similar to "xgmi_apcb_settings_8.h"
2. Copy the AGT output files to the platform APCB directory
 - Destination: AgesaPkg\Addendum\Apcb\<board>\ApcbData\AGT_Data\Instance0
3. Edit the AGT_xGMIxEq_FileList.bat
 - This batch file sets an environment variable to contain a list of the AGT files to include.
 - Location: AgesaPkg\Addendum\Apcb\<board>\ApcbData\AGT_Data\Instance0
 - Content: change the file list to match the names just copied to the directory.

```
SET AGT_XGMI_TXEQ_FILE_LIST=xgmi_apcb_settings_8.h xgmi_apcb_settings_9.h \
xgmi_apcb_settings_10.h
```
4. The XGMI TXEQ data is pulled into the build via the file AgesaPkg\Addendum\Apcb\<board>\ApcbData\ApcbData_ZP_GID_0x1703_Type_xGMITxEq.c and built by ApcbCreate.bat

5.5.1 Normal Mode Macros

This set of macros establishes the basic building blocks for describing the xGMI values. Using these macros provides the most flexibility, but uses more APCB storage space.

5.5.1.1 MAKE_TX_EQ_ATTR(macro)

Summary

Specify an attribute for a specific set of Lanes.

Prototype

```
MAKE_TX_EQ_ATTR (SocketMask, DieMask, LaneMask, DataMode)
```

Parameters

SocketMask

Mask indicating the physical processor socket or sockets to which this control applies. Possible values are:

- XGMI_SOCKET0
- XGMI_SOCKET1
- XGMI_ALL_SOCKETS

DieMask

Values can be combined by using an OR function (e.g. (XGMI_SOCKET0 | XGMI_SOCKET1)) to specify multiple sockets. Mask indicating the die or dies to which this control applies. Possible values are:

- XGMI_DIE0
- XGMI_DIE1
- XGMI_DIE2
- XGMI_DIE3
- XGMI_DIE4
- XGMI_DIE5
- XGMI_DIE6
- XGMI_DIE7
- XGMI_ALL_DIES

These values may be combined by using logical functions as follows:

- (XGMI_DIE0 | XGMI_DIE4)
- (XGMI_ALL_DIES & (~XGMI_DIE7))

LaneMask

Mask indicating the lane or lanes to which this control applies. Possible values are:

- XGMI_LANE0
- XGMI_LANE1
- XGMI_LANE2
- XGMI_LANE3
- XGMI_LANE4
- XGMI_LANE5
- XGMI_LANE6
- XGMI_LANE7
- XGMI_LANE8
- XGMI_LANE9
- XGMI_LANE10
- XGMI_LANE11
- XGMI_LANE12
- XGMI_LANE13
- XGMI_LANE14
- XGMI_LANE15
- XGMI_ALL_LANES

These values may be combined as follows:

- (XGMI_LANE0 | XGMI_LANE4 | XGMI_LANE8 | XGMI_LANE12)
- (XGMI_ALL_LANES & (~XGMI_LANE7))

DataMode

Data mode indicates the form of the following TX EQ data. Possible values are:

- XGMI_TX_EQ_DATA_MODE_PAIR
 - Indicates there is only one setting following, which is described by a MAKE_TX_EQ_DATA macro. This setting applies to the combined

socket, die and lane stated by SocketMask, DieMask and LaneMask. This is known as 'pairing style'.

- XGMI_TX_EQ_DATA_MODE_LIST
 - Indicates there is a list of data settings to follow, which are described by MAKE_TX_EQ_DATA macros separately. These settings apply, in order, to the set of lanes indicated by combined masks of SocketMask, DieMask and LaneMask, in order of low to high on socket, die and lane. This is known as 'list style'.
- XGMI_TX_EQ_DATA_MODE_LIST_COMPACT
 - Like XGMI_TX_EQ_DATA_MODE_LIST, but the list of data settings that follow are described by MAKE_TX_EQ_LANE_DATA macros instead to assure the list of data settings is formed in compact mode.

Description

This macro is used to specify a set of lanes using a combination of socket, die, lane masks and the method of how to associate a data value to each lane in the set. Following macros will specify the data value(s), this macro address the selection process to choose which lanes on which to apply the xGMI transmitter equalization coefficients.

Related Macros

For several common cases, preset macros are available to quickly and easily select the sub-set of lanes. Please refer to the PPR for the processor to determine the number of die per socket and number of lanes per die. These quick style macros use MAKE_TX_EQ_ATTR to select the preset set of lanes:

- MAKE_TX_EQ_ALL_SYS_LANES()
 - This selects all lanes in the system to be set to a single value (e.g. pair style).
- MAKE_TX_EQ_EACH_SYS_LANES()
 - This selects all lanes in the system to be set to individual values (e.g. list style). This can generate a large lane list (see Data Ordering), for which an equal number of data values are expected to follow.
- For various socket (**m**, 0~1) and die (**n**, 0~7)
 - MAKE_TX_EQ_SOCKET**m**_DIE_LANE(DieMask, LaneMask, DataMode)
 - targets a specific socket using parameters that are the same as for MAKE_TX_EQ_ATTR.
 - MAKE_TX_EQ_SOCKET**m**_DIE_LANE_PAIR(DieMask, LaneMask)
 - targets a specific socket using pairing style; one data value applied to all lanes.
 - MAKE_TX_EQ_SOCKET**m**_DIE_LANE_LIST(DieMask, LaneMask)
 - targets a specific socket using list style; a separate data value for each lane is expected to follow.
 - MAKE_TX_EQ_SOCKET**m**_DIEn_ALL_LANES()
 - All lanes of the target socket-die use the same single value (pairing style).
 - MAKE_TX_EQ_SOCKET**m**_DIEn_EACH_LANES()
 - Each lane of the socket-die will use a individual data value (list style). This means that 16 data values must be matched with this selection.
 - MAKE_TX_EQ_SOCKET**m**_DIEn_EACH_LANES_COMPACT()
 - Similar to Each_Lanes, but the output is in compact mode.

Data value Ordering

The order of the data values that follow the list style Attribute macro is important. The lanes, and their associated data values, are ordered by socket, die, lane. The length of the data list would be as below.

$\text{NumberInDataSet} = \text{SocketCount} * \text{DieCount} * \text{LaneCount}$

SocketCount is the number of 1-bits in the SocketMask

DieCount is the number of 1-bits in the DieMask.

LaneCount is the number of 1-bits in the LaneMask, it is 16 for the ALL_LANES and EACH_LANES macros.

For example:

```
MAKE_TX_EQ_SOCKET0_DIE_LANE_LIST(
    (XGMI_DIE0 | XGMI_DIE2),
    (XGMI_LANE0 | XGMI_LANE1 | XGMI_LANE2)
)
```

This macro instance has SocketCount = 1 (Socket 0), DieCount = 2 (Die 0 & 2), LaneCount = 3 (Lane 0, 1 & 2); so, $\text{NumberInDataSet} = 1 * 2 * 3 = 6$. The data order should be from low to high (Socket/Die/Lane) as below.

1. Socket 0 Die 0 Lane 0
2. Socket 0 Die 0 Lane 1
3. Socket 0 Die 0 Lane 2
4. Socket 0 Die 2 Lane 0
5. Socket 0 Die 2 Lane 1
6. Socket 0 Die 2 Lane 2

For the case of MAKE_TX_EQ_EACH_SYS_LANES in a two socket system and assuming four die per socket; the count would be SocketCount = 2 (Socket 0 & 1), DieCount = 4 (Die 0~3), LaneCount = 16 (Lane 0 ~ 15), so $\text{NumberInDataSet} = 2 * 4 * 16 = 128$.

5.5.1.2 MAKE_TX_EQ_DATA(macro)**Summary**

Specify transmitter equalization coefficient values.

Prototype

```
MAKE_TX_EQ_DATA (X, Y, Z)
```

Parameters

X, Y, Z Transmitter Amplitude Adjustment Control values.

Description

This macro is used to describe a set of xGMI transmitter equalization coefficients (TX EQ) values to be applied to the target lane(s). The values for these data parameters are determined by the AMD tool for tuning the xGMI links. The tool will generate source level output for inclusion into the APGB input files.

5.5.1.3 MAKE_TX_EQ_PAIR(macro)**Summary**

Specify TX EQ value(s) with an Attribute-Data pair.

Prototype

```
MAKE_TX_EQ_PAIR(Attr, Data)
```

Parameters

Attr	Specifies the lane set upon which to apply the data setting. See Make_TX_EQ_ATTR macro quick select Related macros.
Data	The TX EQ value to apply. This value will be applied to the lane(s) in the selected set.

Description

This macro is a higher level grouping macro. It combines an attribute macro and a data macro into a 'pair' to clearly identify a group and strictly ensure the order.

Example

Apply one setting to all system lanes on all dies on all sockets.

```
MAKE_TX_EQ_PAIR(
    MAKE_TX_EQ_ALL_SYS_LANES(),    // selected set is all lanes
    MAKE_TX_EQ_DATA(40, 0, 0)      // TX EQ: main, post, pre
)
```

5.5.1.4 MAKE_TX_EQ_LIST(macro)**Summary**

Specify TX EQ value(s) with an Attribute set selection and a list of Data values.

Prototype

```
MAKE_TX_EQ_LIST(Attr, Data0, ...)
```

Parameters

Attr	Specifies the lane set upon which to apply the data settings. See Make_TX_EQ_ATTR macro quick select Related macros.
Data0 ...	The TX EQ values to apply to the set of lanes. The Data values will be applied to the lanes in (socket-die-lane) order.

Description

This macro is a higher level grouping macro. It combines one attribute macro to select the sub-set of lanes and then one data value macro for each lane of the set.

It is required to provide matched count of data settings, when Attr parameter describes a set of lanes that is more than one lane. An unmatched count will lead to undefined behavior and malfunction of xGMI transmitter equalization coefficients.

5.5.1.5 MAKE_TX_EQ_FREQ_TABLE(macro)**Summary**

Specify TX EQ values based on a lane speed.

Prototype

```
MAKE_TX_EQ_FREQ_TABLE(SpeedMask, TxEqPairOrList, ...)
```

Parameters**SpeedMask**

This mask indicates the lane speeds for which the following data values should be applied. Possible values are:

- XGMI_TX_EQ_SPEED_53
- XGMI_TX_EQ_SPEED_64
- XGMI_TX_EQ_SPEED_75
- XGMI_TX_EQ_SPEED_85
- XGMI_TX_EQ_SPEED_96
- XGMI_TX_EQ_SPEED_107
- XGMI_TX_EQ_SPEED_117
- XGMI_TX_EQ_SPEED_128
- XGMI_TX_EQ_SPEED_ALL

This value may be combined as follows:

- (XGMI_TX_EQ_SPEED_ALL & (~XGMI_TX_EQ_SPEED_128))
- (XGMI_TX_EQ_SPEED_85 | XGMI_TX_EQ_SPEED_107)

TxEqPairOrList

This is a grouping macro such as MAKE_TX_EQ_PAIR or MAKE_TX_EQ_LIST that further specifies lanes and the associated data values.

"..."

The TxEqPairOrList parameter may be repeated as needed to cover all lanes that may need special values for the selected speeds.

Description

This macro is a higher level grouping macro. The purpose is to segregate lanes by speed, applying different TX EQ values for different speeds. This is handy if the platform designers find a speed where the links have less performance in certain niches. The TX EQ values can be adjusted for when that lane is operating at that speed only.

5.5.1.6 MAKE_TX_EQ_END(macro)**Summary**

End of Table terminator.

Prototype

```
MAKE_TX_EQ_END( )
```

Description

This macro designates the end of the TX EQ macro tables. Any further macros used afterward will be ignored. This macro must be used in an xGMI transmitter equalization coefficients overrides configurations table.

5.5.2 Compact Mode Macros

This set of macros is provided to save APGB space for platforms having settings where all 16 lanes are to be set.

Compact mode is indicated by MAKE_TX_EQ_ATTR with the parameter *DataMode* set to XGMI_TX_EQ_DATA_MODE_LIST_COMPACT. When compact mode is specified, it is required to use the

macro MAKE_TX_EQ_LIST_COMPACT to provide all lanes' data settings of a die. Normally, this mode is enabled when using the series of macros below for various socket (m= 0~1) and die (n= 0~7)

```
MAKE_TX_EQ_SOCKETm_DIE n_EACH_LANES_COMPACT
```

5.5.2.1 MAKE_TX_EQ_LANE_DATA(macro)

Summary

Specify TX EQ values for compact mode.

Prototype

```
MAKE_TX_EQ_LANE_DATA (X, Y, Z)
```

Parameters

X, Y, Z Transmitter Amplitude Adjustment Control values.

Description

This macro is used to describe a set of xGMI transmitter equalization coefficients (TX EQ) values to be applied to the target lane(s). The values for these data parameters are determined by the AMD tool for tuning the xGMI links. The tool will generate source level output for inclusion into the APCB input files.

5.5.2.2 MAKE_TX_EQ_LIST_COMPACT(macro)

Summary

.

Prototype

```
MAKE_TX_EQ_LIST_COMPACT(Attr, Lane0, Lane1, Lane2, Lane3, Lane4, Lane5, Lane6,  
Lane7, Lane8, Lane9, Lane10, Lane11, Lane12, Lane13, Lane14, Lane15)
```

Parameters

Attr Specifies the lane set upon which to apply the data settings. See Make_TX_EQ_ATTR macro quick select Related macros.

Lane0 ... Lane15 The TX EQ values to apply to the set of lanes. The Data values for Lanes 0 to 15. The only macro that can be used here is:
MAKE_TX_EQ_LANE_DATA.

Description

This macro is a higher level compact macro for setting values of all 16 lanes of a die in proper order.

It is required to use the macro MAKE_TX_EQ_LANE_DATA to provide the data setting for all 16 lanes, in correct order. Using the normal mode macro MAKE_TX_EQ_DATA will lead to the loss of settings and incorrect programming on xGMI transmitter equalization coefficients.

5.5.2.3 MAKE_TX_EQ_FREQ_TABLE_COMPACT(macro)

Summary

Specify TX EQ values for the full system, based on a lane speed.

Prototype

```
MAKE_TX_EQ_FREQ_TABLE_COMPACT(SpeedMask, SD0, SD1, SD2, SD3, SD4, SD5,
```

SD6, SD7)

Parameters**SpeedMask**

This mask indicates the lane speeds for which the following data values should be applied. Possible values are:

- XGMI_TX_EQ_SPEED_53
- XGMI_TX_EQ_SPEED_64
- XGMI_TX_EQ_SPEED_75
- XGMI_TX_EQ_SPEED_85
- XGMI_TX_EQ_SPEED_96
- XGMI_TX_EQ_SPEED_107
- XGMI_TX_EQ_SPEED_117
- XGMI_TX_EQ_SPEED_128
- XGMI_TX_EQ_SPEED_ALL

This value may be combined as follows:

- (XGMI_TX_EQ_SPEED_ALL & (~XGMI_TX_EQ_SPEED_128))
- (XGMI_TX_EQ_SPEED_85 | XGMI_TX_EQ_SPEED_107)

SD0... SD7

Socket/Die (SD) combination 0 to 7. Each one must be a list of xGMI transmitter equalization coefficients settings generated by a MAKE_TX_EQ_LIST_COMPACT macro.

Description

This macro is used to specify TX EQ values for all lanes of all Socket-die instances in the system. First the lanes are selected by their speed (SpeedMask) then each socket-die instance uses a MAKE_TX_EQ_LIST_COMPACT macro to set all lanes of that socket-die instance.

5.5.3 System Use Case Examples

To further aid the user, a series of use case examples is presented to illustrate how to use the structured xGMI macros.

1. Apply one setting to all xGMI lanes everywhere.

This example uses a single data value to be applied to all lanes, on all dies, on all sockets.

```
MAKE_TX_EQ_PAIR(
    MAKE_TX_EQ_ALL_SYS_LANES(),
    MAKE_TX_EQ_DATA(40, 0, 0)
)
```

2. Apply one setting to all lanes not using a certain speed.

This example uses one data value to set all lanes, on all dies, on all sockets for all speeds other than 12.8GT/s.

```
MAKE_TX_EQ_FREQ_TABLE(
    (XGMI_TX_EQ_SPEED_ALL & (~XGMI_TX_EQ_SPEED_128)),
    MAKE_TX_EQ_PAIR(
        MAKE_TX_EQ_ALL_SYS_LANES(),
        MAKE_TX_EQ_DATA(40, 0, 0)
    )
)
```

3. Apply a single setting to all lanes of a socket.

This example targets all of the xGMI lanes on socket 0, die 0.

```
MAKE_TX_EQ_PAIR(
    MAKE_TX_EQ_SOCKET0_DIE0_ALL_LANES(),
    MAKE_TX_EQ_DATA(40, 0, 0)
)
```

4. Apply a setting to one specific lane.

This example targets lane 3 on socket 1, die 2.

```
MAKE_TX_EQ_PAIR(
    MAKE_TX_EQ_ATTR(XGMI_SOCKET1,
                    XGMI_DIE2,
                    XGMI_LANE3,
                    MAKE_TX_EQ_DATA_MODE_PAIR
    ),
    MAKE_TX_EQ_DATA(40, 0, 3)
)
```

5. Apply settings to only two specific lanes.

This example makes a change to lane 3 and 7 on socket 1, die 0.

```
MAKE_TX_EQ_LIST(
    MAKE_TX_EQ_ATTR(XGMI_SOCKET1,
                    XGMI_DIE0,
                    (XGMI_LANE3 | XGMI_LANE7),
                    MAKE_TX_EQ_DATA_MODE_LIST
    ),
    MAKE_TX_EQ_DATA(40, 0, 3),
    MAKE_TX_EQ_DATA(40, 0, 7)
)
```

6. Apply different settings to each and every lane.

This example targets socket 0 die 0. Lane data values are listed in order, Lane 0 to 15).

```
MAKE_TX_EQ_LIST(
    MAKE_TX_EQ_SOCKET0_DIE0_EACH_LANES(),
    MAKE_TX_EQ_DATA(40, 0, 0), MAKE_TX_EQ_DATA(40, 0, 1),
    MAKE_TX_EQ_DATA(40, 0, 2), MAKE_TX_EQ_DATA(40, 0, 3),
    MAKE_TX_EQ_DATA(40, 0, 4), MAKE_TX_EQ_DATA(40, 0, 5),
    MAKE_TX_EQ_DATA(40, 0, 6), MAKE_TX_EQ_DATA(40, 0, 7),
    MAKE_TX_EQ_DATA(40, 0, 8), MAKE_TX_EQ_DATA(40, 0, 9),
    MAKE_TX_EQ_DATA(40, 0, 10), MAKE_TX_EQ_DATA(40, 0, 11),
    MAKE_TX_EQ_DATA(40, 0, 12), MAKE_TX_EQ_DATA(40, 0, 13),
    MAKE_TX_EQ_DATA(40, 0, 14), MAKE_TX_EQ_DATA(40, 0, 15)
)
```

7. Apply different settings to each and every lane in compact mode.

This example targets socket 0 die 0. Lane data values are listed in order, Lane 0 to 15). The compact mode will result in a smaller ROM size footprint.

```
MAKE_TX_EQ_LIST_COMPACT(
    MAKE_TX_EQ_SOCKET0_DIE0_EACH_LANES(),
    MAKE_TX_EQ_LANE_DATA(40, 0, 0), MAKE_TX_EQ_LANE_DATA(40, 0, 1),
    MAKE_TX_EQ_LANE_DATA(40, 0, 2), MAKE_TX_EQ_LANE_DATA(40, 0, 3),
    MAKE_TX_EQ_LANE_DATA(40, 0, 4), MAKE_TX_EQ_LANE_DATA(40, 0, 5),
    MAKE_TX_EQ_LANE_DATA(40, 0, 6), MAKE_TX_EQ_LANE_DATA(40, 0, 7),
    MAKE_TX_EQ_LANE_DATA(40, 0, 8), MAKE_TX_EQ_LANE_DATA(40, 0, 9),
    MAKE_TX_EQ_LANE_DATA(40, 0, 10), MAKE_TX_EQ_LANE_DATA(40, 0, 11),
    MAKE_TX_EQ_LANE_DATA(40, 0, 12), MAKE_TX_EQ_LANE_DATA(40, 0, 13),
    MAKE_TX_EQ_LANE_DATA(40, 0, 14), MAKE_TX_EQ_LANE_DATA(40, 0, 15)
)
```

8. Apply settings to two specific lanes based on lane speed.

This example targets two lanes (3 and 7), on socket 1, die 0. It applies the settings for the lanes when their speed is something other than 12.8GT/s.

```
MAKE_TX_EQ_FREQ_TABLE(
    (XGMI_TX_EQ_SPEED_ALL & (~XGMI_TX_EQ_SPEED_128)),
    MAKE_TX_EQ_LIST(
        MAKE_TX_EQ_ATTR(XGMI_SOCKET1,
                        XGMI_DIE0,
                        (XGMI_LANE3 | XGMI_LANE7),
                        XGMI_TX_EQ_DATA_MODE_LIST
        ),
        MAKE_TX_EQ_DATA(40, 0, 3),
        MAKE_TX_EQ_DATA(40, 0, 7)
    )
)
```

5.6 APCB Library

The APCB library is provided to the platform to support the use case of making dynamic modifications to the APCB content during POST (DXE phase). These library routines must be integrated into, and called from the platform driver. They provide the ability to read the APCB image, make modifications and re-write the image back to the Flash ROM.

Dynamic Configuration Alterations

The platform may decide to export some SETUP option that requires updating the APCB. It can be implemented via adding a hook to the time point when the SETUP browser saves content to non-volatile (NV) storage. If the action is to replace an existing value, `ApcbReplaceType` can be used. If the new value is of different size, then the hook routine, needs to read the APCB and create a local image. Then use the SETUP values to compose a new APCB type blob. The driver must then replace the APCB blob in the flash ROM using a call to `PspWriteApcbEntry` which saves the full APCB data back to the flash ROM.

It should be noted that since the ROM write takes place in the DXE phase, the modified values will not take effect until the next boot cycle. The platform driver may need to invoke a system reset if the changes are intended to take effect before continuing with the boot sequence.

All modifications are performed on the currently active APCB block.

5.6.1 ApcbReplaceType

Summary

Modify a block of data in the APCB binary image.

Prototype

```
EFI_STATUS
ApcbReplaceType (
    IN      UINT16      GroupId,
    IN      APCB_PARAM_TYPE ApcbParamType,
    IN      UINT16      InstanceId,
    IN      UINT8        *TypeDataStream,
    IN      UINT32        TypeDataSize,
    IN OUT  UINT8        *NewApcb
)
```

Parameters

GroupId

This specifies the APCB data group to which the target object belongs. The Type definitions can be found in the APCB.H file and will have a name similar to `APCB_GROUP_MEMORY`.

ApcbParamType

Identifies the APCB parameter data block, or 'type', that is to be updated. The Type definitions can be found in the APCB.H file, in or near the enum APCB_PARAM_TYPE.

InstanceId

Not presently used, this is for future expansion. Please set =0.

TypeDataStream

Pointer to the data buffer used to replace the given type. This data is the block to be inserted, so it must match the structure for the target Type. Below is an example of several types and their related structures.

Type	Data Stream Struct
APCB_PSP_TYPE _BOARD_ID_GETTING_METHOD	Depending upon your selection (see "Selecting one of Multiple APCBs") this may be one of: • PSP_GET_BOARD_ID_FROM_IO_STRUCT • PSP_GET_BOARD_ID_FROM_EEPROM_STRUCT • PSP_GET_BOARD_ID_FROM_SMBUS_STRUCT • PSP_GET_BOARD_ID_FROM_GPIO_STRUCT
APCB_MEM_TYPE _CONSOLE_OUT_CONTROL	PSP_CONSOLE_OUT_STRUCT
APCB_MEM_TYPE _DIMM_INFO_SMBUS	DIMM_INFO_SMBUS
APCB_MEM_TYPE _ERROR_OUT_EVENT_CONTROL	PSP_ERROR_OUT_CONTROL_STRUCT
APCB_MEM_TYPE _EXT_VOLTAGE_CONTROL	PSP_EXT_VOLTAGE_CONTROL_STRUCT
APCB_MEM_TYPE _PS_UDIMM_DDR4_CAD_BUS	array of PSCFG_CADBUS_ENTRY
APCB_MEM_TYPE _PS_UDIMM_DDR4_DATA_BUS	array of PSCFG_DATABUS_ENTRY_D4
APCB_MEM_TYPE _PS_UDIMM_DDR4_MAX_FREQ	array of PSCFG_MAXFREQ_ENTRY
APCB_MEM_TYPE _PS_UDIMM_DDR4_ODT_PAT	array of PSCFG_2D_ODTPAT_ENTRY
APCB_MEM_TYPE_PSO_DATA	PSP_PSO_STRUCT
APCB_MEM_TYPE _PS_UDIMM_DDR4_STRETCH_FREQ	PSCFG_MAXFREQ_ENTRY
APCB_MEM_TYPE_SPD_INFO	PSP_SPD_STRUCT

TypeDataSize

Size, in bytes, of data stream buffer.

NewApcb

Pointer to a memory buffer which will contain the new APCB data blob after type replacement.

Description This is the function used to update an APCB parameter of a given type.

This function consumes the APCB data and replaces a certain type data block specified by ApcbParamType with the one provided in TypeDataStream. The headers of APCB and its subcomponent gets updated automatically,

together with the content of the type data. The new APCB data will be provided in the buffer pointed to by NewApcb.

5.6.2 PspWriteApcbEntry

Summary

This is the function used to write the APCB blob to the Flash ROM.

Prototype

```
EFI_STATUS
PspWriteApcbEntry (
    IN      UINT8      *Apcb
);
```

Parameters

Apcb Pointer to the new APCB image intended to be written to the Flash ROM.

Description This function is used to write the APCB image to the Flash ROM. It also updates the BIOS Directory structure entry for the APCB block to the address pointer and size information of the new blob.

PspWriteApcbEntry will call AmdPspFlashAccLib (part of the Platform Driver) instance to write the flash ROM, where AmdPspFlashAccLib instance will utilized SMM_COMMUNICATION protocol to send R/W requests to SMM driver. The SMI handler of the SMM driver will finally call the platform implemented version of AmdPspFlashAccLib to access the flash ROM.

5.7 ACPI Library

Summary

Provided with the UEFI drivers is an ACPI library of routines to help implement run time services for the AMD products.

Description

The AGESA release package includes a set of ACPI routines provided to aid the platform BIOS in supporting several run time services. These routines are pre-compiled to the AML level as an SSDT which is linked to the UEFI ACPI sub-system by the NBIO driver. The services available are accessed through the method name, such as _SB.ALIB

Related Definitions - Common Data Structure Defs

Method: ALIB

```
\_SB.ALIB (Arg0, Arg1)
```

Arg0 This parameter designates the desired function code. Valid function codes are specified as procedures below.

Arg1 This parameter is an ACPI buffer which contains inputs to the method and outputs from the method. The inputs and outputs are specific to each function. The buffer is 256 bytes.

5.7.1 ALIB Function 01: Report AC/DC State

Summary

Report AC/DC state

Prototype

```
\_SB.ALIB (ALIB_FCN01, {Size, AC_DC_State} )
```

Parameters**ALIB_FCN01**

A constant with a value of 0x01.

Size (WORD)

The size of the parameter buffer for this function; should be set to 3 bytes.

AC_DC_State**(BYTE)**

This parameter indicates the current state of the power source:

- 0 - Current power source is AC, wall powered.
- 1 - Current power source is DC, a battery.

Description

This function is used by the system BIOS to report to the ACPI sub system the current status of the power source. This function is provided for those systems that need to track this state such as mobile devices, laptop units or units tracking the state of a power backup device, for example an Uninterruptible Power Supply (UPS). All systems should call this function once at early start-up (e.g. _SB.PCI0._INI or _SB._INI) to set the internal state. Systems wishing to track the state should also call this function upon a detected change of state.

Status Codes Returned

None.

5.7.2 ALIB Function 02: Performance Request**Summary**

Used to set or remove a performance limit for a PCIe® link.

Prototype

```
\_SB.ALIB (ALIB_FCN02, {Size, ClientID, ValidFlags, Flags, RequestType, PerformanceLevel} )
```

Parameters**ALIB_FCN02****Size (WORD)**

A constant with a value of 0x02.

The size of the parameter buffer for this function; should be set to 10 bytes.

ClientID (WORD)

This parameter indicates the target device of this request. The PCI address of the device is the ID. The address is contained in the parameter as:

- Bus number in bits [15:8]
- Device number in bits [7:3]
- Function number in bits [2:0]

ValidFlags**(WORD)**

This parameter is a mask indicating which bits in the Flags parameter are to be interpreted as valid:

- 0 - corresponding flag bit is to be ignored.
- 1 - Flag bit is valid.

Flags (WORD)

This parameter specifies operational characteristics for the performance request:

- bit 0 - Advertise capabilities.

**RequestType
(BYTE)**

- bit 1 - Wait for completion. Method will not return until the request has been processed.
- bits {31:2} - reserved, must be 0.

This parameter indicates the current state of the power source:

**PerformanceLevel
(BYTE)**

- 0 - Current power source is AC, wall powered.
- 1 - Current power source is DC, a battery.

This parameter indicates the desired level of performance to be allowed on the link:

- 0 - No limit, or remove previous limit.
- 1 - Force to low power mode.
- 2 - Level 1 (PCIe® Gen1 speed)
- 3 - Level 2 (PCIe® Gen2 speed)
- 4 - Level 3 (PCIe® Gen3 speed)

Description

This function is used by the system BIOS to establish a performance limit of individual PCIe® links. Setting a limit value cannot raise the link speed above the capabilities of the end point; it can only place a limit on the link at or below the end point capability.

Status Codes Returned

A status value is returned in the buffer.

**Size (WORD)
ReturnValue
(BYTE)**

Size of the returned buffer, should be 3 bytes.

Result of the requested limit application:

- 1 - Limit request failed.
- 2 - Limit request completed.
- 3 - Limit request is in progress. The 'wait for completion' flag was not set, so the method has returned while the request is still pending.

5.7.3 ALIB Function 03: PSPP Start/Stop**Summary**

Starts or stops the PCIe® Speed Power Policy (PSPP) policy enforcement on the PCIe® root bridge.

Prototype

```
\_SB.ALIB (ALIB_FCN03, {Size, Policy} )
```

Parameters**ALIB_FCN03
Size (WORD)**

A constant with a value of 0x03.

The size of the parameter buffer for this function; should be set to 3 bytes.

Policy (BYTE)

This parameter indicates the desired state for the PSPP feature:

- 0 - Stop the PSPP management.
- 1 - Start the PSPP management feature.

Description

This function is used by the system BIOS to set the operational state of the PCIe® Speed Power Policy. Please refer to the BKDG for details about the PSPP feature.

Status Codes Returned

A status value is returned in the buffer.

Size (WORD)	Size of the returned buffer, should be 3 bytes.
ReturnValue (BYTE)	Result of the requested operation: • 0 - request completed successfully. • 1 - request not supported. • 5 - request error. Processing the request resulted in an internal error.

5.7.4 ALIB Function 04: Set Bus Width

Summary

Set or reset power gating based on the number of requested PCIe® lanes.

Prototype

```
\_SB.ALIB (ALIB_FCN04, {Size, ClientID, Lanes} )
```

Parameters

ALIB_FCN04	A constant with a value of 0x04.
Size (WORD)	The size of the parameter buffer for this function; should be set to 5 bytes.
ClientID (WORD)	This parameter indicates the target device of this request. The PCI address of the device is the ID. The address is contained in the parameter as: • Bus number in bits [15:8] • Device number in bits [7:3] • Function number in bits [2:0]
Lanes (BYTE)	This parameter indicates the desired number of lanes to be active to this end point.

Description

This function is used by the system BIOS to .

Status Codes Returned

A status value is returned in the buffer.

Size (WORD)	Size of the returned buffer, should be ?? bytes.
Lanes (BYTE)	The number of active lanes. This may be more or less than the number requested.

5.7.5 ALIB Function 05: ALIB Initialize

Summary

Initialize the ACPI ASL library. Recommend this is called from _SB.PCI0._INI..

Prototype

```
\_SB.ALIB (ALIB_FCN05)
```

Parameters**ALIB_FCN05**

A constant with a value of 0x05.

Description

This function is used by the system BIOS to initialize the ACPI ASL library. It is recommended that this be called from _SB.PCI0._INI.

Status Codes Returned

None.

5.7.6 ALIB Function 06: Hot Plug Request**Summary**

Handle a Hot Plug event on an end point.

Prototype

```
\_SB.ALIB (ALIB_FCN06, {Size, PortID, State} )
```

Parameters**ALIB_FCN06**

A constant with a value of 0x06.

Size (WORD)

The size of the parameter buffer for this function; should be set to 5 bytes.

PortID (WORD)

This parameter indicates the target end point as specified by the PCI address:

- Bus number in bits [15:8]
- Device number in bits [7:3]
- Function number in bits [2:0]

State (BYTE)

This parameter indicates the type of event to process on the end point:

- 0 - handle a device eject event.
- 1 - handle a device insert event.

Description

This function is used by the system BIOS to handle a device insert or eject for a specific PCIe® port. This should be called from the SCI handler for the “PRSNT#” (Present) signal on the PCIe® port..

Status Codes Returned

A status value is returned in the buffer.

Size (WORD)

Size of the returned buffer, should be 4 bytes.

Status (BYTE)

Result of the requested operation:

- 0 - Operation completed successfully.
- 1 - Request is not supported.
- 5 - Error. The processing of this request caused an internal error.

**DeviceStatus
(BYTE)**

Status of the end point after the requested operation:

- 0 - Device is not present.
- 1 - Device is present.

5.7.7 ALIB Function 0A: Report Dock State**Summary**

Report the docked or undocked status.

Prototype

```
\_SB.ALIB (ALIB_FCN0A, {Size, DockState} )
```

Parameters**ALIB_FCN0A**

A constant with a value of 0x0A.

Size (WORD)

The size of the parameter buffer for this function; should be set to 3 bytes.

DockState (BYTE)

This parameter indicates the current state of docking:

- 0 - The unit is currently attached to a docking device.
- 1 - The unit is currently not docked.

Description

This function is used by the system BIOS to report the dock/undock state. All systems should perform this call early, from _SB.PCI0.INI or from _SB.INI and systems that support a dock device which provides supplemental cooling or thermal capabilities should call this function when processing a dock/undock event. This function is independent of the AC/DC function.

Status Codes Returned

None.

5.7.8 ALIB Function 0B: Battery Status**Summary**

Report the status of the battery.

Prototype

```
\_SB.ALIB (ALIB_FCN0B, {Size, BatteryID, PowerUnits, BatteryTotalCapacity,  
BatteryRemainingCapacity, BatteryVoltage} )
```

Parameters**ALIB_FCN0B**

A constant with a value of 0x0B.

Size (WORD)

The size of the parameter buffer for this function; should be set to 16 bytes.

BatteryID (BYTE)

This parameter indicates the ID of the battery being reported. This must be a unique value for each battery in the system.

PowerUnits (BYTE)

This parameter indicates the units of measure for the battery capacity:

- 0 - Battery capacity is in mAh (milli-Amp-hours).

**BatteryTotalCapacity
(DWORD)**

- 1 - Battery capacity is in mWh (milli-Watt-hours). This parameter indicates the total capacity of the battery when fully charged, in the units specified by PowerUnits. This should match the last reported value by the ACPI _BIF object.

**BatteryRemainingCapacity
(DWORD)**

Indicates the remaining capacity of the battery, in the units specified by PowerUnits.

BatteryVoltage (DWORD)

This parameter indicates the voltage currently provided by the battery, in mV. For systems reporting the remaining capacity in mAh the voltage is used to calculate the battery capacity in mWh units.

Description

The system BIOS should report battery status information for management of power and performance states while operating in DC mode. This information should be reported periodically as the battery status is measured by the system and should be reported independently for each battery in the system.

Status Codes Returned

None

5.7.9 ALIB Function 0C: Dynamic Power and Thermal Configuration**Summary**

Configure the operating values for the processor's Power and Thermal controller.

Prototype

```
\_SB.ALIB (ALIB_FCN0C, {Size, ControlBlock, [ControlBlock, ...]} )

ControlBlock {
    ControlID    BYTE;
    ControlValue DWORD;
};
```

Parameters**ALIB_FCN0C**

A constant with a value of 0x0C.

Size (WORD)

The size of the parameter buffer for this function. This function may specify multiple controls for modification in one call. These are packed into the buffer, 5 bytes each, in an array. The size, in bytes, is therefore 2 + (5 * number of control values passed).

**ControlID
(BYTE)**

For each control block, this parameter indicates which control is to be modified. Various controls will be available on different processors which are indicated by Family-Model tags on each control, for example "[F17M10]" indicates Family 17h Models 10-1Fh. For further detail on the specific parameters, please see the appropriate Infrastructure Roadmap (IRM) document.

- 0x0 - cTDP. [F15M65] Set the configurable TDP. This is the target TDP the thermal controller will attempt to maintain .
- 0x1 - STAPM Time Constant. [F15M65][F17M10][F17M60] Set the TimeConstant used by the power monitor.
- 0x2 - Skin Control Scalar. [F15M65] Set the temperature limit scalar is represented in a percent (100% being the default).

Passing in 110 to the argument will increase the Total Significant Power (TSP) power limit by 10%.

- 0x3 - Thermal Control Limit. [F15M65][F17M10][F17M60] This specifies the die temperature limit, on the Tctl temperature scale, to which the processor manages activity. The processor controls the activity level that contributes to power consumption, such as Pstates and DPM-states, so that the die temperature remains within this limit. Reducing this value is a way to address chassis skin temperature issues. It is recommended that this value be set at least 5C below the HTC temperature limit.
- 0x4 - Package Power Limit. [F15M65][F17M00] This specifies the package power limit. This function splits the value parameter into two 16-bit field values.

```
[ BYTE - ControlID (= 4)
  WORD - PkgPwrLimit
  WORD - PkgPwrLimitDC ]
```

PkgPwrLimit **[F15M65]**

This item specifies the upper limit on package power to be used when the unit is powered by an AC source.

PkgPwrLimit **[F17M00]**

This item specifies the upper limit on package power, regardless of power source.

PkgPwrLimitDC **[F15M65]**

This item specifies the upper limit on package power to be used when the unit is powered by an DC (battery) source. This value is not used by [F17Mxx].

- 0x5 - Sustained Power Limit. [F17M10][F17M60] This values specifies the sustained power limit that can be supported by the thermal solution and chassis thermal design. This limit is managed within the STAPM time constant. This a numeric value and must be in milliwatts.
- 0x6 - Fast PPT Limit. [F17M10][F17M60] This specifies the power source peak power limit. This is the amount of power the power source can provide for a short time (~<= 10ms). This a numeric value and must be in milliwatts.
- 0x7 - Slow PPT Limit. [F17M10][F17M60] This is the amount of power the power source can provide for a moderate amount of time. This a numeric value and must be in milliwatts.
- 0x8 - Slow PPT Time Constant. This indicates how long the power source can maintain the moderate power level. This is a time value specified in seconds.

- 0x9 - PROCHOT_L Deassertion Ramp Time [F17M10]
[F17M60] This specifies the time it takes to ramp the CCLK/
GFXCLK clocks up to Fmax from Fmin when PROCHOT is
deasserted. This a numeric value and must be in milliseconds.
- 0xA – System Temperature Tracking. [F17M10] As described in
the IRM, hotspots may occur outside the APU which can be
monitored by platform specific sensors. For platforms that
support this type of chassis temperature measurements, this
control tells the APU thermal management controller the
'margin' to use when managing system temperature. The 'margin'
indicates the amount of heat measured at the hot spots and
specifies the number of degrees C the temperature manager
should reduce its target temperature so as to keep the whole
system under the T_{Skin} limit.

This value is specific to the platform and is relative to the T_{Skin} limit. This is a 16-bit signed integer numeric value, with the lower 8-bits being a fractional part. Bit 15 is the sign bit, bits 14:8 are integer, and bits 7:0 are fractional meaning bit 7 is ½ degree, bit 6 is ¼, bit 5 is 1/8, bit 4 is 1/16, etc. For example, 2.875 0x02E0; -2.06250xFDF0 (2's complement of 0x0210).

Example, if the limit is 40 and the sensors currently read “38”, then this function should use “2” for the value. In this case the value should be 0x0200 (for +2.00).

- 0xB VRM_CURRENT_LIMIT [F17M10][F17M60] This value indicates the maximum current that the voltage regulator is capable of providing to VDD on a continuous basis. This a numeric value and must be in milliamperes. For example, a current limit of 12.6 amperes is represented as 12600.
- 0xC - VRM_MAXIMUM_CURRENT_LIMIT [F17M10]
[F17M60] This value indicates the maximum current that the voltage regulator is capable of providing to VDD for short amounts of time. This a numeric value and must be in milliamperes. For example, a current limit of 14.5 amperes is represented as 14500.
- 0xD - VRM_LOW_POWER_THRESHOLD [F17M10]
[F17M60] This value indicates the maximum current that the voltage regulator for VDDCR_VDD can deliver in PSI1 mode. This a numeric value and must be in milliamperes. For example, a current limit of 12.6 amperes is represented as 12600.
- 0xE - VRM_SOC_CURRENT_LIMIT [F17M10][F17M60] This value indicates the maximum current that the voltage regulator is capable of providing to the SOC on a continuous basis. This a numeric value and must be in milliamperes. For example, a current limit of 12.6 amperes is represented as 12600.
- 0xF - VRM_SOC_LOW_POWER_THRESHOLD [F17M10]
[F17M60] This value indicates the maximum current that the voltage regulator for VDDCR_SOC can deliver in PSI1 mode. This a numeric value and must be in milliamperes. For example, a current limit of 12.6 amperes is represented as 12600.

- 0x10 - DGPU_CONTROL [FM17M10][F17M60] This value is used to indicate a change in the power state of the dGPU for a "Ryzen plus Radeon" configuration. The APU SMU establishes a link to the AMD dGPU SMU to manage the balance of power consumption between the APU and the dGPU. The platform BIOS must call this function before powering off the dGPU and after powering on the dGPU to notify the APU SMU of the change. Failure to make these calls could result in undetermined operations. The control value for this feature must be "0" to stop dGPU control and "1" to re-start dGPU control.
- 0x11 - VRM_SOC_MAXIMUM_CURRENT_LIMIT [F17M10][F17M60] This value indicates the maximum current that the voltage regulator is capable of providing to VDD_SOC for short amounts of time. This is a numeric value and must be in milliamperes. For example, a current limit of 14.5 amperes is represented as 14500.
- 0x20 - STT_ALPHA_APU> [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttAlphaApu](#). See the PCD definition for values and format.
- 0x21 - STT_ALPHA_GPU [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttAlphaHS2](#). See the PCD definition for values and format.
- 0x22 - STT_SKIN_TEMPERATURE_LIMIT_APU [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttSkinTemperatureLimitApu](#). See the PCD definition for values and format.
- 0x23 - STT_SKIN_TEMPERATURE_LIMIT_GPU [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttSkinTemperatureLimitHS2](#). See the PCD definition for values and format.
- 0x24 - STT_ERROR_COEFF [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttErrorCoefficient](#). See the PCD definition for values and format.
- 0x25 - STT_ERROR_RATE_COEFF [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttErrorRateCoefficient](#). See the PCD definition for values and format.
- 0x26 - [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttM1](#). See the PCD definition for values and format.
- 0x27 - STT_M2 [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttM2](#). See the PCD definition for values and format.
- 0x28 - STT_M3 [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttM3](#). See the PCD definition for values and format.
- 0x29 - STT_M4 [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttM4](#). See the PCD definition for values and format.

- 0x2A - STT_M5 [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttM5](#). See the PCD definition for values and format.
- 0x2B - STT_M6 [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttM6](#). See the PCD definition for values and format.
- 0x2C - STT_C_APU [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttCApu](#). See the PCD definition for values and format.
- 0x2D - STT_C_GPU [F17M60]. This command provides the ability to dynamic modify the value set by the PCD [PcdSttCHS2pu](#). See the PCD definition for values and format.

ControlValue (DWORD)

For each control block, this parameter specifies the new value for the indicated control. Interpretation of this value is control dependent.

Description

This function provides platform BIOS the capability to dynamically change the operating parameters for a subset of power and thermal management features. This allows the platform BIOS to adjust parameters based on any platform status change rather than limiting it to specific functions (such as AC/DC or docked/undocked). The interface allows for single or multiple parameters to be passed in a single ALIB call.

The controls modified by this function are explained in the Infrastructure Roadmap (IRM) document. The controls supported by a specific processor may vary. Check the IRM for those supported by your processor.

Status Codes Returned

None.

5.7.10 AWAK Method: Apu Wake

Summary

Wake the processor from s sleep state.

Prototype

```
\_SB.AWAK (Arg0)
```

Parameters

Arg0 An integer indicating the sleep state from which the system is waking. This should be the same value passed to the `_WAK` method. The S1 sleep state is represented as 1, S2 as 2, etc.

Description

AWAK is a method which provides processor specific configuration after the system wakes from a sleep state. The host environment should invoke AWAK from its `_WAK` method.

Status Codes Returned

The AWAK method does not return a value.

5.7.11 APTS Method: Apu Sleep

Summary

Prepare the processor for Sleep state.

Prototype

```
\_SB.APTS (Arg0)
```

Parameters

Arg0 An integer indicating the sleep state to which the system is preparing to enter. This should be the same value passed to the `_PTS` method. The S1 sleep state is represented as 1, S2 as 2, etc.

Description

APTS is a method which provides processor specific configuration as the system prepares to enter a sleep state. The host environment should invoke APTS from its `_PTS` method.

Status Codes Returned

The APTS method does not return a value.

5.8 Firmware Managed PCIe® Resets

In systems with a very large number of PCIe® lanes, the lane training process can extend beyond the regular hardware hold-off time of the `*PCIE_RESET` signal. This can lead to situations where a device can be requesting lanes before the host controller is fully configured. Symptoms of this have been seen where a device starts functioning with fewer lanes of throughput than is normal for the device; e.g. an 8 lane device is running at 1 or 2 lanes.

Firmware management of PCIe® device and slot resets provides improved timing of the reset controls versus the hardware default PCIe® reset. The AMD AGESA™ libraries provide a capability to reprogram and assert the PCIe® reset signal very early in the boot process to insure that endpoint devices remain in reset until the PCIe® subsystem is fully configured for operation.

Multi-Die Considerations

The AMD library for multi-die implementations is designed with the assumption that the port or ports from any given die utilize the PCIe® Reset signal from that same die.

The implementation of the DXIO initialization completes the configuration and training of PCIe® ports on each die sequentially. The DXIO subsystem of die 0 is fully initialized and trained before any initialization occurs on die1 (or die 2, or die 3 ...). The reset control for the die being trained is released prior to the training.

Care must be taken in the design process to not cross the reset lines to the ports/devices on the various die. Since the reset for die N+1 will not be released until die N has finished; any port/device on die N using a reset signal from die N+1 would not be trained since its reset was not released. It is the responsibility of the platform BIOS to identify the mapping of the reset signals and ensure that the reset signal is de-asserted at an appropriate time to allow devices on a given die to train.

The firmware control capability uses three concepts within the system:

- ABL configuration
- Reset Identification
- Platform call-out to activate a reset signal

5.8.1 ABL PCIe® Reset Handling

The AGESA Boot Loader (ABL) will force the *PCIe_RESET signal to its active state by re-defining the multi-function pin to its GPIO26 definition and setting the signal to its output low state. This will keep the PCIe® reset asserted until it is de-asserted at the appropriate time by a callout to a platform function. If the PCIe® Reset signal is not used as the reset for PCIe® slots or endpoints, the feature can be disabled through a BLDCFG setting in the APCB; see BLDCFG_PCIE_RESET_CONTROL.

Early BMC link

AMD provides the capability for the ABL to train a single link to a single device (usually a Baseboard Management Controller (BMC) in a server system) ; see PcdEarlyBmcLinkTraining. In the case where an early train link is specified, the PCIe® Reset for the die connected to that link will not be asserted by ABL. This allows the BMC device to avoid the PCIe® reset delay of the other devices.

For the die to which the BMC is attached, the PCIe® Reset will be left to operate in its hardware default state. Implementers should notice that this hardware reset will apply to ALL devices attached to the same die as the BMC; which could lead to the failure symptoms noted earlier.

The default behavior on many platforms is to use GPIO26 for the early BMC reset pin. The APCB_TOKEN_UID_PCIE_RESET_PIN_SELECT APCB configures one of the other available GPIO pins for use as the reset for the early BMC link.

5.8.2 Reset GPIO specification

The Reset-GPIO control has two execution centers: the DXIO and the platform GPIO manager. Each firmware controlled reset must be assigned a unique identifier to allow for the two centers to mutually identify the same control signal. The Reset ID is associated with a specific device or slot. This association is maintained throughout the DXIO subsystem initialization process. The Reset ID is provided by the platform BIOS and is only utilized by AGESA and DXIO firmware to communicate control requests (i.e. “de-assert” or “assert”) to the platform BIOS. There is no specific requirement for assignment of the Reset ID, except that it be unique. The AMD reference platforms will use the CPM module to perform GPIO control. See reference: [CPM](#).

Topology Descriptor

The Reset ID is specified as an entry in the PCIe® Topology Descriptor. Each port is identified by an “engine” descriptor using the “DXIO_ENGINE_DATA_INITIALIZER” macro. For details see [DXIO Macros](#).

The following is an example descriptor table entry for a PCIe® engine.

```
// P2 - x16 slot
DXIO_ENGINE_DATA_INITIALIZER (
    DxioPcieEngine,    // Engine Type
    0,                 // Start Lane
    15,                // End Lane
    DxioHotplugDisabled, // Hot switchable
    MySlot2ResetId     // Reset ID
);
```

Avoiding Redundant Calls

In many cases a single reset signal may be shared across multiple devices and across multiple die. Generally the DXIO firmware within a given die will not make multiple requests to de-assert the same Reset ID. If the same Reset ID is used across multiple die, a de-assert request will be made for that Reset ID on each die as it enters configuration and training.

Redundant requests to de-assert a specific Reset ID may be expressly avoided by checking for that unique ID (along with the bus number of the die) in the platform GPIO control function.

5.8.3 Platform GPIO Control

Management of the reset controls is generally the responsibility of the platform BIOS. The reset control should be implemented by the platform BIOS as a UEFI library to be built along with the AMD library DxioLibV1. The function `AgesaGpioSlotResetControl`, as defined in the [CPM](#) reference, provides the platform control of the GPIO associated with a specific Reset ID. The library is generally provided within the CPM module or in the `AgesaPkg` as an Addendum.

The inputs to the function uniquely identify the specific reset signal using the Reset ID and indicate a control action request (de-assert, assert). Note that the DXIO implementation will not request an assertion of the reset signal. It is assumed that the ABL will force GPIO26 active or the platform designer has chosen a GPIO that powers-on in the active state.

The reference library will reassign the behavior of the signal as PCIe® Reset when it is called from AGESA to request that the reset be de-asserted. As the state of the PCIe® Reset signal is already “de-asserted”, reconfiguring the pin as PCIe® Reset will essentially de-assert the reset to the endpoint device. This has the added function of restoring the default hardware behavior of the PCIe® Reset signal during any subsequent cold/warm reset or S3/S5 power states.

6 Tools

6.1 APCB Tool (V2)

The APCB tool is used to process platform configuration settings into a binary block set into the flash ROM for the AGESA Boot Loader to process. The platform BIOS uses the APCB Tool (APCBToolV2.exe) to create the binary block.

- `ApcbData_<Platform>_ExtParams.c`—This file contains configuration items that describe the platform and the ABL operating environment.
- `ApcbData_<Platform>_ErrorReportingControl.c`—This file contains configuration items that describe how the ABL reports errors and interacts with any debug device.
- `ApcbData_<Platform>_MbistControl.c`—This file contains configuration items that describe how to run the MBIST tests.
- `ApcbData_<Platform>_PsoOverride.c`—This file contains configuration items that describe the memory topology and special considerations for this platform.
- `ApcbData_<Platform>_SpdInfo.c`—This file contains the platform descriptions of the memory layout.

6.1.1 Running the APCBTool

The APCB build procedure requires that Microsoft Visual Studio 2010 or newer is installed on your build computer. Earlier versions of Visual Studio may work but have not been validated. The APCB build tool runs from a Windows Command Prompt with Administrator privileges.

- Open a Windows command prompt window
- navigate to the following directory: `\Program Files (x86)\Microsoft Visual Studio XX.0\VC\bin`
- Run the `VCvars32.bat` file for each APCB build session you initiate.
- Verify that this batch file completes successfully. This batch file configures the necessary environment variables for the APCB build tool.

The AMD reference platform APCB source code is located in the following directory: `\\AGESA\AgesaPkg\Addendum\Apcb<platform>`. If you wish to build your APCB from this example platform APCB directory, then no change is needed. But if you want to move this build to your platform BIOS folder, then please verify you have corrected the path for `ApcbToolV2.exe` by updating the path in the `ApcbCreate.bat` file as noted below, to match your build environment. also, you can change the name of the output destination file in this same .bat file.

```
SET TOOL_DIR=%CD%\..\..\..\..\AgesaModulePkg\AMDTools\ApcbToolV2\Tools
REM -----
REM Target APCB binary will be put in RELEASE_DIR
REM -----
SET APCB_BIN_FILE_BASE_NAME=OutFileName           // Change output name here
```

The APCBToolV2 is operated using the following steps:

1. Set the current path to the location where the APCB files are located.
2. The user edits the input data files as “C” files to establish the appropriate settings.
3. Build the APCB binary by executing the following command:

```
Apcbcreate.bat CLEAN BUILD
```

4. The resulting APCB binary gets placed in the 'release' sub-directory with the specified filename: `\release\OutFileName.bin`

6.2 APB Tool (V3)

The APBv3 tool is used to process platform configuration settings into a binary block set into the flash ROM for the AGESA Boot Loader to process. The platform BIOS uses the APB Tool (APBToolV3.exe) to create the binary block.

6.3 AGESA Boot Loader Debug

The ABL collects the data and passes it to the AGESA™ V9 memory drivers via an output data structure stored in memory. DDR Training results are always produced and reported. The MBIST operation does use some input parameters as listed in the “ABL MBIST Configuration Items”. The AGESA Boot loader (ABL) can report errors using two methods:

- via UEFI Driver interface - The x86 AGESA V9 memory drivers provide an interface that allows platform BIOS EDK2 drivers to retrieve results that were produced during ABL execution (see the “AMD_MEMORY_MBIST_RESULTS_HOB_PPI” for more details). This method relies on the successful completion of memory initialization and is applicable for a “Normal Boot”.
- via an output port - System IO ports are used to pass codes. This method can provide more detail of situations occurring during the algorithm execution. This method is discussed in more detail in the next section.

Note: The two methods can be used at the same time. It is possible that the ABL can report its errors during execution using the output port method, then continue execution with memory initialized, released the x86 cores and then report errors via the UEFI Driver interface. If neither method is enabled, the ABL will indicate status or errors via POST codes output to the debug IO port or beep codes.

Execution Flow

The execution flow of the ABL error testing and reporting proceeds as follows:

1. ABL executes memory initialization
2. ABL executes DDR Training
3. ABL records DDR Training results
4. ABL executes MBIST tests
5. ABL records MBIST test results
6. ABL collects MBIST data in AGESA PSP Output Buffer (APOB) for more details)
7. ABL stores APOB in memory
8. X86 Cores are released (See “AMD_PSP_BIOS_Arch_Design_Guide_Rev” more details)
9. AGESA V9 PSP Driver retrieves APOB
10. AGESA V9 Memory Driver retrieves “MBIST Results” and “DDR Training Results” from APOB
11. Platform BIOS EDK2 Driver retrieves the results from a PPI.

The ABL will generate standard check point codes during its operation. These will be sent to the debug output port as enabled and defined in the APBC configuration items. A listing of these codes can be found in AgesaModulePkg\Include\AblPostCode.h file.

Console versus Interactive>

The ABL can be connected to an external uCtonroller, in which case the Interactive Debug topic will apply. For the other cases where 'POST code' output is desired, then the Console feature will apply. The main characteristics are:

- 'Interactive debug'
 - targeted for long term use - monitoring by an external agent/device (such as a BMC).
 - Permanent connection via dedicated IO ports
 - Reports logged errors during ABL process
- 'console debug'
 - Targeted for short term debug
 - Temporary serial port connection – port may be used for something else by full system or no-popped on production boards
 - Selectable verbosity of spew

Configuration items for these should be set appropriately for these separate and independent features.

Interactive Debug

The ABL Debug can be operated using an external device to receive the message outputs and provide some input control. This method allows the ABL to report errors during the execution and does not rely on memory being available. It is applicable for situations where there is an error during boot and the PSP cannot launch the x86 core.

This method supports two options that are selectable via the ABL configuration settings.

- Continuous Reporting - The ABL reports error information using an IO port but does not required an external agent. This is also known as mono-directional (e.g. output only).
- Handshake Protocol - The ABL reports error information using a handshake protocol that uses an IO output port and a separate IO input/output port.

Both methods only report a summary of Error information.

'Heart Beat' Feature

The ABL supports a special feature of the error log system for generating interactive 'heart beat' messages to indicate progress during execution of processes that may take a period of time. When enabled, this feature will generate information 'errors' that will be sent to the reporting port just like the other errors. The Heartbeat records are listed in [Table 22](#) and will use a major code of 'Alert'.

The major difference of the Heartbeat records are that they will not be recorded and reported to the x86 core via the UEFI Driver interface reporting method. They are only sent to the communications port.

This feature is enabled via the ABL Error Reporting Configuration Items and uses the same ports and protocols as the interactive debug feature. Messages are available to indicate progress for these processes:

- MBIST - initial memory diagnostic process
- Post Package Repair - DRAM device errors are attempting to be repaired
- NVDIMM Restore - NV type DIMM's data is being restored
- Mem Clear - zeroing of all memory as preparation for the ECC process

The progress message will be sent at intervals separated by `ErrorLogOutputDwordDelay` and repeated until the process completes.

Related Definitions

ERROR_LOG_STRUCT

```
typedef struct _ERROR_LOG_STRUCT {
    UINT32      TotalNumberOfLogEntries;
    LOG_ENTRY   LogEntry[MAX_LOG_ENTRIES];
} ERROR_LOG_STRUCT;
```

TotalNumberOfLogEntries

Total number of log entries in the queue.

LogEntry

Array of entries.

LOG_ENTRY

```
typedef struct _LOG_ENTRY {
    UINT16      MajorError;      // Major error number
    UINT16      MinorError;     // Minor error number
    UINT32      IdInfo;         // ID Information
    UINT32      DataA;          // Error Data A
    UINT32      DataB;          // Error Data B
} LOG_ENTRY;

#define MAX_LOG_ENTRIES 32

#define ABL_LOG_IO_DATA_BEGIN 0xDEADEAAA /* a.k.a. {begin} */
#define ABL_LOG_IO_DATA_END  0xDEADCA5E /* a.k.a. {end} */
#define ABL_HANDSHAKE_DATA_ACK 0xDEAD5555 /* a.k.a. {ACK} */
```

MajorError

Each error entry contains a major error code which classifies the error severity. The defined major error codes are as follows:

- `AGESA_SUCCESS` - The service completed normally.
- `AGESA_UNSUPPORTED` - The service is unimplemented
- `AGESA_BOUNDS_CHK` - A dynamic parameter was out of range and the service was not provided. Example, memory address not installed, heap buffer handle not found. Not Logged.
AGESA_STATUS of greater severity (the ones below this line), always have a log entry available.
- `AGESA_ALERT` - An observed condition, but no loss of function.
- `AGESA_WARNING` - Possible or minor loss of function.
- `AGESA_ERROR` - Significant loss of function, boot may be possible.
- `AGESA_CRITICAL` - Continue boot only to notify user.
- `AGESA_FATAL` - Halt booting. See Log, however Fatal errors pertaining to heap problems may not be able to reliably produce log events.
- `AGESA_SYNC_MORE_DATA` - More data is available from PSP communications
- `AGESA_SYNC_SLAVE_ASSERT` - Slave is at an ASSERT

MinorError Error codes that identify the component and specific error. For the list of reported errors, refer to the table [Table 22](#).

IdInfo Identifier of the source processing unit.

- Bits 15:8 - Socket number
- Bits 7:0 - Die number

DataA & DataB Information pertinent to the specific error. For the per error meanings, refer to the table [Table 22](#).

6.3.1 ABL Error Codes

The Minor error codes indicate a specific error and the DataA and DataB values add qualifying information about that error. The following table lists the errors.

Table 22. ABL Error Codes

Error code	Title/Description	Data A	Data B
0x3000 Fatal	ABL_GEN_ERROR_LOG_HEAP_ERROR Error Log heap Error		
0x3001	ABL_GEN_OPN_MISMATCH_ERROR - Indicates that an OPN mismatch was detected		
0x3002 Fatal	ABL_ERROR_SYNC_COMMUNICATION_WITH_DATA_SENT_TO_MASTER_ERROR - Indicates that a SVC communication error was detected on a slave when data was sent from the slave to the master		
0x3003 Fatal	ABL_ERROR_SLAVE_BROADCAST_COMMUNICATION_FROM_MASTER_TO_SLAVE_ERROR - Indicates that a SVC communication Error was detected on a slave when data was broadcast from the master to the slave		
0x3004 Fatal	ABL_MSG_SLAVE_SYNC_FUNCTION_EXECUTION_ERROR - Indicates that an error occurred while executing a slave sync function		
0x3005	ABL_ABL1_FATAL_ERROR - Indicates that an error was detected in ABL1		
0x3006	ABL_ABL2_FATAL_ERROR - Indicates that an error was detected in ABL2		
0x3007	ABL_ABL3_FATAL_ERROR - Indicates that an error was detected in ABL3		
0x3008	ABL_ABL6_FATAL_ERROR - Indicates that an error was detected in ABL	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x3009	ABL_ASSERT_ERROR - Indicates that an assert occurred	File code	Line number
0x4000	ABL_MEM_INIT_ERROR - Memory Init Errors	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4001 Fatal	ABL_MEM_PMU_TRAIN_ERROR - PMU Training Errors	• Bit 7:0 - Socket • Bit 15:8 - Channel • BIT 16 - DIMM 0, AGESA memory test detected an error on this DIMM. • BIT 17 - DIMM 1, AGESA memory test detected an error on this DIMM. • Bit 31:18 - Reserved	• Bit 0 - PMU FW Load Error • Bit 1 - PMU Training Error • Bit 31:2 - Reserved
0x4002 Fatal	ABL_MEM_ODT_HEAP_ALLOCATION_ERROR	Bit 31:0 - Reserved	Bit 31:0 - Reserved

Table 22. ABL Error Codes (continued)

Error code	Title/Description	Data A	Data B
0x4003 Fatal	ABL_MEM_AGESE_MEMORY_TEST_ERROR - An error occurred during the memory test execution, causing a test abort.	- Bit 7:0 - Socket - Bit 15:8 - Channel - BIT 16 - DIMM 0, AGESA memory test detected an error on this DIMM. - BIT 17 - DIMM 1, AGESA memory test detected an error on this DIMM. - Bit 31:18 - Reserved	Bit 31:0 - Reserved
0x4004	ABL_MEM_MEMORY_EXECUTING_TEST_ERROR - An error was detected by the post-test analysis of the test results.	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4005 Fatal	ABL_MEM_DPPR_MEM_AUTO_HEAP_ALOC_ERROR - Error with DDR Post Package repair Heap allocation	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4006 Fatal	ABL_MEM_DPPR_NO_APOB_HEAP_ALOC_ERROR - Error with DDR Post Package repair No APOB Heap allocation	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4007 Warning	ABL_MEM_DPPR_NO_PPR_TB4_HEAP_ALOC_ERROR - Error with DDR Post Package repair No PPR Table Heap allocation	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4008 Fatal	ABL_MEM_ECC_MEM_AUTO_HEAP_ALOC_ERROR - Error with ECC Mem Auto Heap allocation	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4009 Fatal	ABL_MEM_SOC_SCAN_HEAP_ALOC_ERROR - Memory Socket Scan Heap allocation	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x400A Fatal	ABL_MEM_SOC_SCAN_NO_DIE_ERROR - Memory Socket Scan No Die Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x400B Fatal	ABL_MEM_NO_NB_CONST_ERROR - No NB Constructor Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x400C Fatal	ABL_MEM_ABL1B_MEM_AUTO_ALOC_ERROR - ABL1B Mem Auto Allocation Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x400D Fatal	ABL_MEM_ABL1B_NO_NB_CONST_ERROR - ABL1B No NB Constructor Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x400E Fatal	ABL_MEM_ABL2_MEM_AUTO_ALOC_ERROR - ABL2 Mem Auto Allocation Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x400F Fatal	ABL_MEM_ABL2_NO_NB_CONST_ERROR - ABL2 No NB Constructor Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4010 Fatal	ABL_MEM_ABL3_MEM_AUTO_ALOC_ERROR - ABL3 Mem Auto Allocation Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4011 Fatal	ABL_MEM_ABL3_NO_NB_CONST_ERROR - ABL3 No NB Constructor Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4012	ABL_MEM_ABL1B_GEN_ERROR - Memory ABL1B General Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4013	ABL_MEM_ABL2_GEN_ERROR - Memory ABL2 General Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4014	ABL_MEM_ABL3_GEN_ERROR - Memory ABL3 General Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4015 Fatal	ABL_MEM_SPD_GET_TARET_SPEED_ERROR - Memory SPD Get Target Speed Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4016 Fatal	ABL_MEM_FLOW_P1_FAMILY_SUPPORT_ERROR - Mem Flow P1 Family Support Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4017 Fatal	ABL_MEM_NO_VALID_DDR4_DIMMS_ERROR - Mem no valid DDR4 DIMMs Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved

Table 22. ABL Error Codes (continued)

Error code	Title/Description	Data A	Data B
0x4018 Fatal	ABL_MEM_ERROR_NO_DIMM_FOUND_ON_SYSTEM_ERROR - Mem no DIMM found on System Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4019 Fatal	ABL_MEM_FLOW_P2_FAMILY_SUPPORT_ERROR - Mem Flow P2 Family Support Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x401A Fatal	ABL_MEM_ERROR_DDR_RECOVERY_ERROR - DDR Recovery Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x401B	ABL_MEM_ERROR_RRW_TEST_ERROR - Memory RRW Test Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x401D Fatal	ABL_MEM_ERROR_MODULE_TYPE_MISMATCH_RDIMM - Memory Module Type Mismatch RDIMM Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x401E Fatal	ABL_MEM_ERROR_MODULE_TYPE_MISMATCH_LRDIMM - Memory Module Type Mismatch LRDIMM Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x401F Fatal	ABL_MEM_AMD_MEM_AUTO_NVDIMM_ERROR - Memory AMD Mem Auto NVDIMM Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x4020 Fatal	ABL_MEM_ERROR_MIXED_ECC_AND_NON_ECC_DIMM_IN_SYSTEM - Mixing of ECC and non-ECC is not permitted within a system.	- Bit 7:0 - Socket - Bit 15:8 - Channel	Bit 31:0 - Reserved
0x4021 Fatal	ABL_MEM_ERROR_MIXED_3DS_AND_NON_3DS_DIMM_IN_CHANNEL - Mixing of 3DS and non-3DS is not permitted within a channel.	- Bit 7:0 - Socket - Bit 15:8 - Channel	Bit 31:0 - Reserved
0x4022	ABL_MEM_ERROR_MIXED_X4_AND_X8_DIMM_IN_CHANNEL - Mixing of x4 and x8 devices is not permitted within a channel.	- Bit 7:0 - Socket - Bit 15:8 - Channel	Bit 31:0 - Reserved
0x4023	ABL_MEM_ABL4_INIT_ERROR - Memory ABL4 INIT Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4024	ABL_MEM_OVERCLOCK_ERROR_RRW_TEST - Memory Over Clock Error RRW Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4025 Fatal	ABL_MEM_OVERCLOCK_ERROR_PMU_TRAINING - Memory Over Clock Error PMU Training Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	- Bit 7:0 - Chip Select - Bit 31:8 - Reserved
0x4026 Fatal	ABL_MEM_OVERCLOCK_ERROR_MEM_INIT - Memory Over clock Mem Init Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4027 Fatal	ABL_MEM_OVERCLOCK_ERROR_MEM_OTHER - Memory Over clock Mem other Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4028	ABL_MEM_ABL6_INIT_ERROR - Memory ABL6 Init Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x402E Fatal	ABL_MEM_FLOW_P3_FAMILY_SUPPORT_ERROR - Memory flow P3 famils support Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x402F Fatal	ABL_MEM_MBIST_HEAP_ALLOC_ERROR - Memory MBIST HEAP ALLOC Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4030 Fatal	ABL_MEM_ERROR_MBIST_RESULTS_ERROR - Memory MBIST Results Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4031 Fatal	ABL_MEM_NB_TECH_MEM_HEAP_ALOC_ERROR - Memory NB or Tech Heap Allocation Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4032 Fatal	ABL_MEM_ERROR_NO_DIMM_SMBUS_INFO_ERROR - Memory No Dimm Smbus Info Error Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved

Table 22. ABL Error Codes (continued)

Error code	Title/Description	Data A	Data B
0x4033	ABL_MEM_ERROR_LRDIMM_MIXMFG - For LRDIMMs, this is a notification message indicating that 2T timing is being used on the channel.	- Bit 7:0 - Channel - Bit 15:8 - Socket - Bit 31:16 - Reserved	Bit 31:0 - Reserved
0x4035 Fatal	ABL_MEM_ERROR_HEAP_DEALLOCATE_FOR_PMU_SRAM_MSG_BLOCK - Memory Heap Deallocate for PMU SRAM MSG Block Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4036 Fatal	ABL_MEM_ERROR_UNSUPPORTED_DIMM_CONFIG - Memory Unsupported DIMM Config Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4037 Fatal	ABL_MEM_ERROR_HEAP_ALLOCATE_FOR_DCT_STRUCT_AND_CH_DEF_STRUCTs - Memory Heap allocate for DCT STRUCR AND CH DEF Structs Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4038 Fatal	ABL_MEM_ERROR_HEAP_ALLOCATE_FOR_PMU_SRAM_MSG_BLOCK - Memory Heap Allocate for PMU SRAM MSG BLOCK Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x4039 Fatal	ABL_MEM_ERROR_HEAP_LOCATE_FOR_PMU_SRAM_MSG_BLOCK - Memory Heap locate for PMU SRAM MSG Block Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x403A Fatal	ABL_MEM_ERROR_PHY_PLL_LOCK_FAILURE - Memory Phy PLL Lock Failure Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x403B Fatal	ABL_MEM_ERROR_PMU_TRAINING - Memory PMU Training Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x403D Fatal	ABL_MEM_ERROR_FAILURE_TO_LOAD_OR_VERIFY_PMU_FW - Memory Failure to Load or Verify PMU FW Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x403E Fatal	ABL_MEM_ERROR_FAILURE_BIOS_PMU_FW_MISMATCH_AGESA_PMU_FW_VERSION - Memory Failure BIOS PMU FW Mis-match AGESA PMU FW Version Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x403F Error	ABL_MEM_ERROR_CAD_BUS_TMGM_NOT_FOUND - Memory CAD BUS timing Not found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x4040 Error	ABL_MEM_ERROR_DATA_BUS_CFG_NOT_FOUND - Memory Data Bus Configuration Not found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x4041 Error	ABL_MEM_ERROR_LR_IBT_NOT_FOUND - Memory LR IBT Not Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	- Bit 7:0 - Channel number - Bit 31:8 - Reserved
0x4043 Error	ABL_MEM_ERROR_MR0_NOT_FOUND - Memory MR0 Not found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	- Bit 7:0 - Channel number - Bit 31:8 - Reserved
0x4044 Error	ABL_MEM_ERROR_ODT_PATTERN_NOT_FOUND - Memory ODT Pattern Not found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x4045 Error	ABL_MEM_ERROR_RC10_OP_SPEED_NOT_FOUND - Memory RC10 OP Not found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	- Bit 7:0 - Channel number - Bit 31:8 - Reserved
0x4046 Error	ABL_MEM_ERROR_RC2_IBT_NOT_FOUND - Memory RC2 IBT Not found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved

Table 22. ABL Error Codes (continued)

Error code	Title/Description	Data A	Data B
0x4047 Error	ABL_MEM_ERROR_RTT_NOT_FOUND - Memory RTT found Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	- Bit 7:0 - Channel number - Bit 31:8 - Reserved
0x4048 Error	ABL_MEM_ERROR_CHECKSUM_NV_SPDCHK_RESTRT_ERROR - Memory Checksum NV SPD Chk restart Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x404A Error	ABL_MEM_ERROR_NO_CHIPSELECT - Memory No Chipselect Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x404B Error	ABL_MEM_ERROR_NO_COMMON_CAS_LATENCY - Memory No Common CAS Latench Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x404C Error	ABL_MEM_ERROR_CAS_LAT_XCEEDS_TAA_MAX - Memory CAS Latency exceeds TAA Max Error	- Bit 7:0 - DCT number - Bit 31:8 - Reserved	Bit 31:0 - Reserved
0x404D Error	ABL_MEM_ERROR_NVDIMM_ARM_MISMATCH_POWER_POLICY - Memory NVDIMM ARM Missmatch Power Policy Error	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x404E Fatal	ABL_MEM_ERROR_NO_DIMM_ON_ANY_CHANNEL_IN_SYSTEM_ERROR	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x404F Alert	ABL_MEM_ALERT_PMU_MAJOR_MSG - ALERT Indicating the Major PMU messages system. [F17M00] [F17M08]	Bit map of PMU progress. Each bit represents a stage in the PMU training. a bit=0 means the stage has not been reached/performed. a bit=1 means the stage is completed. The stages are: - Bit 0 - initialization - Bit 1 - fine write leveling - Bit 2 - read enable training - Bit 3 - read delay center optimization - Bit 4 - write delay center optimization - Bit 5 - 2D read delay/voltage center optimization - Bit 6 - 2D write delay /voltage center optimization - Bit 7 - Training has run successfully - Bit 8 - max read latency training - Bit 9 - read DQ de-skew training - Bit 10 - LCDL offset calibration - Bit 11 - LRDIMM Specific training (DWL, MREP, MRD and MWD) - Bit 12 - MPR read delay center optimization - Bit 13 - Write leveling coarse delay - Bit 14 - FATAL ERROR	- Bit 7:0 - Channel number - Bit 15:8 - Die number - Bit 23:16 - Socket number - Bit 31:24 - Reserved
0x4050 Alert	ABL_MEM_ALERT_PMU_RESULTS_TX_TMGM - ALERT Indicating the PMU Results for Dram Training Margin Tx Timing [F17M00][F17M08]	Eye width - These are distances from the trained center of the 'eye' to the closest failing region, in DLL steps. These values are the minimum of all eyes in the timing group. The groups are: - Bits 7:0 - Rank 0 - Bits 15:8 - Rank 1 - Bits 23:16 - Rank 2 - Bits 31:24 - Rank 3	- Bit 7:0 - Channel number - Bit 15:8 - Die number - Bit 23:16 - Socket number - Bit 31:24 - Reserved

Table 22. ABL Error Codes (continued)

Error code	Title/Description	Data A	Data B
0x4051 Alert	ABL_MEM_ALERT_PMU_RESULTS_TX_VREF - ALERT Indicating the PMU Results for Dram Training Margin Tx Vref [F17M00][F17M08]	Eye width - These are distances from the trained center of the 'eye' to the closest failing region, in PHY DAC steps. These values are the minimum of all eyes in the timing group. The groups are: - Bits 7:0 - Rank 0 - Bits 15:8 - Rank 1 - Bits 23:16 - Rank 2 - Bits 31:24 - Rank 3	- Bit 7:0 - Channel number - Bit 15:8 - Die number - Bit 23:16 - Socket number - Bit 31:24 - Reserved
0x4052 Alert	ABL_MEM_ALERT_PMU_RESULTS_RX_TMGM - ALERT Indicating the PMU Results for Dram Training Margin Rx Timing [F17M00][F17M08]	Eye width - These are distances from the trained center of the 'eye' to the closest failing region, in DLL steps. These values are the minimum of all eyes in the timing group. The groups are: - Bits 7:0 - Rank 0 - Bits 15:8 - Rank 1 - Bits 23:16 - Rank 2 - Bits 31:24 - Rank 3	- Bit 7:0 - Channel number - Bit 15:8 - Die number - Bit 23:16 - Socket number - Bit 31:24 - Reserved
0x4053 Alert	ABL_MEM_ALERT_PMU_RESULTS_RX_VREF - ALERT Indicating the PMU Results for Dram Training Margin Rx Vref [F17M00][F17M08]	Eye width - These are distances from the trained center of the 'eye' to the closest failing region, in PHY DAC steps. These values are the minimum of all eyes in the timing group. The groups are: - Bits 7:0 - Rank 0 - Bits 15:8 - Rank 1 - Bits 23:16 - Rank 2 - Bits 31:24 - Rank 3	- Bit 7:0 - Channel number - Bit 15:8 - Die number - Bit 23:16 - Socket number - Bit 31:24 - Reserved
0x5000	ABL_DF_GMI_ERROR - Data Fabric GMI Errors		
0x6000	ABL_HEART_BEAT_MBIST	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x6001	ABL_HEART_BEAT_POST_PACKAGE_REPAIR	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x6002 Alert	ABL_HEART_BEAT_NVDIMM_RESTORE	Bit 31:0 - Reserved	Bit 31:0 - Reserved
0x6003 Alert	ABL_HEART_BEAT_MEM_CLEAR	Bit 31:0 - Reserved	Bit 31:0 - Reserved

6.3.2 Mono-Directional Protocol

The mono-directional reporting sends data packets out to the IO port. In this mode, the ABL will transmit the error information continuously with a delay between the transmissions of the data. This method does not require an external agent.

The IO port addresses and time values used are defined in the ABL configuration controls.

Transmission Protocol

Once an error has been detected, the ABL will initiate the error transmission protocol. To begin the transmission, the ABL sends the 32-bit {Begin} flag. The ABL waits for the delay period then starts sending the next block. The data begins with {ERROR_LOG_STRUCT} as the initial 32-bit block transmitted containing the TotalNumberOfLogEntries value. It is immediately followed by the {LOG_ENTRY} elements.

Each ABL log entry is submitted as a set of 4 consecutive 32-bit blocks. The first 32-bit block consists of the MajorError and MinorError (these two values will be combined into a single 32-bit value for transmission). The second 32-bit block of an error entry is the processor identification data, padded to 32-bits with zeroes. The third 32-bit block is the DataA element; then the fourth 32-bit block is the DataB element.

The ABL completes the error reporting by sending the {End} Flag. When the ABL completes its error reporting, it repeats it indefinitely.

1. Send {Begin}
2. Wait for period
3. Send # of Entries
4. Wait
5. --For each log entry --
 - a. Send {block 1} (Maj/Min error code)
 - b. Wait
 - c. Send {block 2} (ID Info)
 - d. Wait
 - e. Send {block 3} (Data A)
 - f. Wait
 - g. Send {block 4} (data B)
 - h. Wait
6. Send {End}
7. Repeat to step 1

6.3.3 Bi-Directional Protocol

The Bi-directional reporting sends data packets out to the IO port and implements a handshake protocol with an active agent.

The IO port addresses and time values used are defined in the ABL configuration controls.

Transmission Protocol

Once an error has been detected, the ABL will initiate the error transmission protocol. To begin the transmission, the ABL sends the 32-bit {Begin} flag. The ABL then waits for the external agent to respond with {ACK}. Once received, the ABL then starts sending the next block. The data begins {ERROR_LOG_STRUCT} with the initial 32-bit block transmitted containing the TotalNumberOfLogEntries value. It is immediately followed by the {LOG_ENTRY} elements. The ABL will wait for the {ACK} response after each block.

Each ABL log entry is submitted as a set of 3 consecutive 32-bit blocks. The first 32-bit block consists of the MajorError and MinorError (these two values will be combined into a single 32-bit value for transmission). The second 32-bit block of an error entry that is the DataA element; then the third 32-bit block is the DataB element.

The ABL completes the error reporting by sending the {End} Flag.

The ABL records errors until its error buffer is full. At that time it will stop and initiate the error reporting protocol; reporting all error found. If ErrorStopOnFirstFatalErrorEnable = TRUE and MajorError = AGESA_FATAL, then the ABL execution will stop immediately and initiate the error reporting protocol.

table

Table 23. ABL Bi-Directional Handshake Protocol

	ABL	Agent
1	Send {Begin}	→
2		← Send {ACK}
3	clears ErrorLogInputPort, Send #Entries	. →
4		← Send {ACK}
5	clears ErrorLogInputPort, Send Block1 (maj/min error code)	. →
6		← Send {ACK}
7	clears ErrorLogInputPort, Send Block 2 (ID Info)	. →
8		←
9	clears ErrorLogInputPort, Send Block 3 (Data A)	. →
10		← Send {ACK}
11	clears ErrorLogInputPort, Send Block 4 (Data B)	. →
12		← Send {ACK}
13		< Repeat to step 5 send next error Entry >
14	clears ErrorLogInputPort, Send {End}	. →
15		← Send {End}

6.4 MakeFile Generator

Although the AGESA™ code set is UDK-2010 and EDK-II compatible, the BIOS vendors tend to use proprietary build control files. Mostly these are based off of the UDK defined files, but some variances occur. The AGESA code source package contains the ArchV9 tool and a set of configuration files. The tool uses the configuration files to generate the build control files. Using the configuration files as master files eliminates the potential of mis-synchronized of build control file sets. The output build control files should not be modified by the porting engineer; any changes will be lost the next time the ArchV9 tool is executed.

Summary

Use this tool to generate new build control files. It is a Perl script used to dynamically generate build control files for multiple build systems. It extracts file and relationship information from the configuration files then generates build control files for all of the build systems at once.

This tool requires Perl scripting language support. It is the responsibility of the developer to install the proper support package(s). Version: Perl v5.20.1 or later is required.

Prototype

```
perl ArchV9.pl -o [programName] -p [socketName] -i [configFilePath] -e [outFilePath]
```

Command Line Options

-h Print the help and exit.
-o Set the name(s) of the program(s) to be included in the output,
[programName] separated by "|" e.g. -o "Ap|Bt". The configuration files will

contain information for several programs. This parameter selects which of those programs will be included in the output build control files.

-p [socketName] Set the name(s) of the socket(s) to be included in the output, separated by "|" e.g. -o "Am4|Sp3". The configuration files will contain information for several sockets. This parameter selects which of those sockets will be included in the output build control files.

-i [configFilePath] Specifies the configuration file path. If not present, the current directory will be assumed.

-e [outFilePath] Specifies the output file path. If not present, the current directory will be assumed. The name of output file will be "AGESA" + [SocketName] + [ProgramName] + "ModulePkg" + [suffix].
Example: AgesaAm4ApBtModulePkg.dxe.inc.fdf

Examples

To Display the help information, use this command:

```
perl ArchV9.pl -h
```

To generate a combo bios combining the product 'Alpha'(Ap) and the product 'Beta'(Bt) support for the AM4 socket:

```
perl ArchV9.pl -o "Ap|Bt" -p Am4
```

Note: the tools does not 'validate' combinations, so the user must be careful to assure any specified combinations are valid.

To designate where the configuration file is located:

```
perl ArchV9.pl -o Bt -p Am4 -i %CD%\Config
```

Related Files

These files are included in the AGESA release package and are used by the ArchV9 tool.

ArchV9.pl	Perl script to extract information from config files and generate build file.
Products.config	Configuration file used to define mapping of the programs and tags (e.g. the product names and socket names used on the command line). This file is maintained by the AGESA team and should not be modified.
AgesaModulePkg.<suffix>	Configuration file that contains information on how to generate corresponding suffix build files, where 'suffix' is one of: • .dx.e.inc.fdf • .inc.dsc • .pei.inc.fdf This file is maintained by AGESA and should not be modified.

6.5 ABL Version Checker

The build process for the AGESA™ package generates several binary components. As is normal for make systems, components are only re-built when their source files change. At times, this can lead to mis-matched

version of the various components. This tool is provided to check and verify that the components match in their versions.

The version checker, CheckAblVer, is used to confirm the ABL binary, PSP binary, APCB binary and APCB/ APPB/APOB headers are synchronized with each other. It should be run before BIOS building is started. If the version check fails, BIOS building must stop and indicate an error to the user so that corrections can be made.

Note: This check should be repeated for each instance of the ABL that is created for the platform. Some platforms may use more than one instance.

Prototype

```
CheckAblVer.bat [AgesaBootloader] [HeaderFilePath] [APCB] [PspBootloader]
```

Command Line Options

AgesaBootloader	Complete path to the compiled Agesa Bootloader Binary. Each time CheckAblVer.bat is run, the path of an ABL binary is passed, which is usually located under \Firmwares directory within the release package.
HeaderFilePath	The release folder containing the APCB.h , APPB.h and APOB.h files.
APCB	Complete path to the APCB Binary. The standard released version is located under the Firmware release folder. Customized versions will be located elsewhere and this parameter needs to identify that location.
PspBootloader	Complete path to the compiled PSP Bootloader Binary, usually located under the Firmware release folder.

Examples

```
CheckAblVer.bat \Firmwares\xxxAgesaBootloader\AgesaBootloader0_prod_xxx.csbin
                \Firmwares\xxxAgesaBootloader
                \Firmwares\xxxAgesaBootloader\APCB_xxx.bin
                \Firmwares\xxxAgesaBootloader\PspBootLoader_prod_xxxSCM.sbin
#//repeat for next instance of ABL
CheckAblVer.bat \Firmwares\xxxAgesaBootloader\AgesaBootloader1_prod_xxx.csbin
                \Firmwares\xxxAgesaBootloader
                \Firmwares\xxxAgesaBootloader\APCB_xxx.bin
                \Firmwares\xxxAgesaBootloader\PspBootLoader_prod_xxxSCM.sbin
```

Note: 'xxx' is replaced by the product name abbreviation. For the Product name 'Alpha'(Ap) the directory would be ApAgesaBootloader.

7 External MicroControllers

Some system designs will include a System Management Controller (SMC) or Board Management Controller (BMC). Though these may have various titles, they generally provide the system with services and features beyond the standard personal computer and are typically proprietary to the system manufacturer. The presence of such an external controller (external to the APU/CPU) enables certain advanced features, such as "4-corner memory training" and the Advanced Platform Management Link (APML). For these features, the external controller must communicate directly with the APU/CPU PSP on-board controller to perform certain services.

A BMC micro-controller is typically connected via a PCIe® link.

7.1 BMC via PCIe® Link

A BMC attached to the CPU via a PCIe® link acts like a standard PCI device and is configured in that manor. The lane used for this connection must be accounted for in the PCI config description macros defined in [Installation Macros](#).

For some platform configurations, an XMGI link can be used for the BMC connection. For rules and guidance for these types on connections, please see the [MBDG for SP3](#).

Using the standard PCI enumeration process, the BMC to CPU link will be connected 'late' in the POST time line. A special provision is available to have this specific link trained early, so as to establish the connection as soon as possible after power-on.

7.1.1 BIOS x86 Controls

These controls are used by the UEFI NBIO Driver. The values used for these controls must match the values used for the APCB token if/when the early init option is used.

PcdEarlyBmcLinkTraining	This value notifies AGESA if the early BMC link training feature is enabled in the AGESA boot loader. AGESA utilizes this notification to clear link training status before DXIO initialization. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - Early link training is enabled at the location specified by PcdEarlyBmcLinkSocket and PcdEarlyBmcLinkDie • FALSE - Early link training is turned off
PcdEarlyBmcLinkSocket	This value identifies the socket number to which the BMC link is connected if early BMC training is enabled. Permitted Choices: (Type: List)(Default: 0xFF) • 0 - BMC is connected to socket 0 • 1 – BMC is connected to socket 1 • 0xFF – BMC early training is not supported
PcdEarlyBmcLinkDie	This value identifies the die number to which the BMC link is connected if early BMC training is enabled. Permitted Choices: (Type: List)(Default: 0xFF) • 0 – BMC is connected to die 0 (lanes 0 – 31) of the socketed identified by PcdEarlyBmcLinkSocket

- 1 – BMC is connected to die 1 (lanes 32 – 63) of the socketed identified by PcdEarlyBmcLinkSocket
- 2 – BMC is connected to die 2 (lanes 64 – 95) of the socketed identified by PcdEarlyBmcLinkSocket
- 3 – BMC is connected to die 3 (lanes 96 – 127) of the socketed identified by PcdEarlyBmcLinkSocket
- 0xFF – BMC early training is not supported

**PcdEarlyBmc
LinkLaneNum**

This value identifies the lane number use for the BMC link trained in ABL. If the lane number is 128 or 129 then the link does not need to be disabled for reconfiguration. In this case the link will remain active. Note that any value below 128 will cause the BMC link to be reconfigured in PEI.

Permitted Choices: (Type: Value)(Default: 0x0, No BMC link present)

- 0 – 127 - Link will be disabled and restored during PEI
- 128 – 129 - Link will not be disabled during PEI

7.1.2 BMC APCB Controls

These controls are used by the ABL for early link initialization of the BMC. The default values are those when the 'early training' option is false.

**APCB_TOKEN_UID_PCIE
RESET_PIN_SELECT
[F17M30][F17M70]**

This control specifies which system GPIO is being used on the platform for reset of the BMC link. This will be used during the early initialization for the BMC link.

Permitted Choices: (Type: List)(Default: GPIO266)

- Disable - ABL will not attempt to control early BMC link reset
- GPIO26
- GPIO266
- GPIO267
- GPIO27

**APCB_TOKEN_UID_
BMC_SOCKET_NUMBER**

This Token indicates the socket being used on the platform for the BMC link. This will be used during the early initialization for the BMC link.

Permitted Choices: (Type: Value)(Default: 0x0F)

**APCB_TOKEN_UID_
BMC_DEVICE**

This Token indicates the PCI Device number of the link being used on the platform for the BMC link. This will be used during the early initialization for the BMC link.

Permitted Choices: (Type: Value)(Default: 0)

**APCB_TOKEN_UID_
BMC_FUNCTION**

This Token indicates the PCI Function number of the link being used on the platform for the BMC link. This will be used during the early initialization for the BMC link.

Permitted Choices: (Type: Value)(Default: 0)

**APCB_TOKEN_UID_
BMC_START_LANE**

This Token indicates the PCIe controller start lane of the link being used on the platform for the BMC link. This will be used during the early initialization for the BMC link.

**APCB_TOKEN_UID_
BMC_END_LANE**

Permitted Choices: (Type: Value)(Default: 0x80)

This Token indicates the PCIe controller end lane of the link being used on the platform for the BMC link. This will be used during the early initialization for the BMC link.

Permitted Choices: (Type: Value)(Default: 0x80)

**APCB_TOKEN_UID_
BMC_LINK_SPEED**

This Token is being used on the platform to select the Link speed for the BMC link. This will be used during the early initialization for the BMC link.

Permitted Choices: (Type: List)(Default: Gen1)

- 1 - Gen1 speed will be used.
- 2 - Gen2 speed will be used.

**APCB_TOKEN_UID_
BMC_GEN2_TX_
DEEMPHASIS [F17M30]**

This token is being used to control PCIe GEN2 De-emphasis in ABL for Early training for BMC link.

Permitted Choices: (Type: List)(Default: 0xFF)

- 0 - Use de-emphasis from CSR.
- 1 - Use de-emphasis requested by upstream component.
- 2 - Use -6.0 dB
- 3 - Use -3.5 dB
- 0xFF - Skip override setting.

8 Support for Family 15h

To promote end user upgrade paths for their platforms, new processors are often packaged to fit into platform sockets used by earlier processor families, such as the AM4 infrastructure. Combination BIOS can be built to support both the older and the newer processors.

This section collects the notes and special features that are unique to the Family 15h processors support.

8.1 The meaning of 'Auto'

There are many configuration options and several have inter-dependencies on other settings or the particular processor installed at the moment. It is not reasonable to expect the customer to know the best setting for all of these items; so, the control option of 'Auto' is present to simplify the choices. This setting is usually the default and generally means that the AGESA™ software will make the best choice, given the inter-dependencies, with the target of optimizing for:

1. Power usage
2. Performance

Once a system is operational, settings may be adjusted to meet individual platform requirements.

8.2 Memory Driver for Family 15h

Summary

The Memory Driver for Family 15h does the memory initialization in the system. This duty has moved to the Platform Security Processor for Family 17h processors.

Description

The main Memory Driver is described in [Memory Driver](#). This is a description of the controls unique to the Family 15h processors.

8.2.1 Configuration Items for Family15h (Memory Driver)

The Family15h support uses this set of configuration items. These only apply to Family 15h products and do not apply to Family 17h products. For Family 17h products these controls have been moved to the AGESA Parameters Control Block (APCB). Items that are configurable by the host system are manifested as UEFI Platform Configuration Data elements (PCDs). The operation of PCDs are well defined in the UEFI specifications. The PCD elements used by the AGESA™ Memory Driver are listed below.

8.2.1.1 Enablement Category

PcdAmdMem UmaEnable	Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMem DmiEnable	Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg EnableOnLineSpareCtl	This feature is recommended only for expert users and is described in the BIOS and Kernel Developer's Guides (BKDG).

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMem EccEnable

This turns on the correction action and enables the ability of the MCA subsystem to report errors. It does not activate the MCA error report interrupts. If the build option is set to include the code for the ECC feature, then this setting activates the feature. If the feature code is removed from the build, then this element has no affect.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg EnableBankIntlv

Specifies if the system should use DRAM bank (also known as chip-select) interleaving. If the build option is set to include the code for this feature, then this setting activates the feature. If the feature code is removed from the build, then this element has no effect. The AMD-recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg EnableNodeIntlv

Specifies if the system should use memory node interleaving. The AMD-recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg EnableChannelIntlv

Specifies if the system should use memory channel interleaving for performance tuning. If the build option is set to include the code for this feature, then this setting activates the feature. If the feature code is removed from the build, then this element has no effect. Also, this option has noeffect if the channel ganged mode is selected. The AMD-recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg EnableParity

Parity is an error-detection tool available on some DIMMs. This control specifies whether or not the memory controller should activate parity checking. The feature is invoked only if the DIMMs present are all capable of supporting the parity detection.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg EnableMemClr

Memory Clear functionality control

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg MemoryBusFrequencyLimit

This is the maximum memory clock at which the platform memory busses are capable of performing. This setting will restrict the frequency even is the DIMMs support a higher frequency. This platform restriction may occur due to layout difficulties.

Permitted Choices: (Type: List)(Default: DDR1066_FREQUENCY)

- DDR400_FREQUENCY
- DDR533_FREQUENCY
- DDR667_FREQUENCY
- DDR800_FREQUENCY
- DDR1066_FREQUENCY
- DDR1333_FREQUENCY
- DDR1600_FREQUENCY
- DDR1866_FREQUENCY
- DDR2100_FREQUENCY
- DDR2133_FREQUENCY
- DDR2400_FREQUENCY

PcdAmdMemCfg AmpEnable

AMP functionality control

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Enable, platform BIOS requests to enable memory overclocking function, and AGESA detects if memory is capable of it.
- FALSE - Disable, there is no request to enable memory overclocking function.

8.2.1.2 Platform Category

PcdAmdMemCfg UmaMode

Supply the UMA memory allocation mode build time customization, if any. The default mode is Auto.

Permitted Choices: (Type: List)(Default: UMA_AUTO)

- UMA_NONE — no UMA memory will be allocated.
- UMA_SPECIFIED — up to the requested UMA memory will be allocated.
- UMA_AUTO — allocate the optimum UMA memory size for the platform. For APUs with integrated graphics, this will provide the

optimum UMA allocation for the platform and for other platforms will be the same as NONE.

PcdAmdMemCfg UmaSize

Provide a build time customization of the UMA allocation size input. If the PcdAmdMemCfgUmaMode is SPECIFIED, up to this amount of UMA memory will be allocated, otherwise, it will be ignored. The size must be a multiple of 64 KBytes. Addr[47:16]

Permitted Choices: (Type: Value)(Default: 0x0000)

PcdAmdMemCfg UmaVersion

This is a new methodology in the selection of the UMA video memory buffer. It works in conjunction with the other UMA controls to allow the user to set the UMA buffer size.

Permitted Choices: (Type: List)(Default: UMA_NON_LEGACY)

- UMA_LEGACY - The traditional UMA allocation method will be used. The user will specify a size using the controls:PcdAmdMemCfgUmaMode and PcdAmdMemCfgUmaSize.
- UMA_NON_LEGACY - The size of the UMA buffer will be determined by the table in the BKDG, based on available memory and the expected maximum display resolution. See “BLDCFG_RESOLUTION” .

PcdAmdMemCfg UmaAbove4G

Video devices and their associated drivers may require that the UMA buffer be located in the 32-bit address space. This control provides that selection.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - The UMA buffer may be placed above the 4GB boundary, in the 64-bit address space.
- FALSE - Keep the UMA buffer in the 32-bit address range (e.g. below the 4GB boundary).

PcdAmdMemCfg UmaAlignment

This control specifies any alignment requirements for the UMA buffer dictated by the video device.

Permitted Choices: (Type: List)(Default: NO_UMA_ALIGNED)

- NO_UMA_ALIGNED - There are no imposed restrictions.
- UMA_4MB_ALIGNED - The UMA buffer must be aligned to a 4MB boundary.
- UMA_128MB_ALIGNED - The UMA buffer must be aligned to a 128MBboundary.
- UMA_256MB_ALIGNED - The UMA buffer must be aligned to a 256MB boundary.
- UMA_512MB_ALIGNED - The UMA buffer must be aligned to a 512MB boundary.

**PcdAmdMemCfg
DqsTrainingControl**

DQS signal timing training control is a platform selection to specify whether the automatic timing training routine is desired or a set of pre-defined timing values should be used. This can provide boot speed

	improvement by bypassing the active training algorithm and using previously stored values.
	Permitted Choices: (Type: Boolean)(Default: TRUE)
	• TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg CustomVddioSupport	Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg CustomVddioVoltage	CustomVddioSupport helps to customize the VDDIO voltage for DRAM. Permitted Choices: (Type: List)(Default: VOLT_INITIAL)
	• VOLT_INITIAL - AGESA™ code will determine best VDDIO based upon SPD Data. • VOLT1_5, - 1.5 Volt • VOLT1_35, - 1.35 Volt • VOLT1_25, - 1.25 Volt • VOLT_DDR4_RANGE_START, - Start of DDR4 Voltage Range • VOLT1_2 = VOLT_DDR4_RANGE_START, - 1.2 Volt • VOLT_TBD1, - TBD1 Voltage • VOLT_TBD2, - TBD2 Voltage • VOLT_UNSUPPORTED - No common voltage found
PcdAmdMemCfg ExternalVrefCtl	This item specifies whether to use an external, platform specific, memory Vref control. Permitted Choices: (Type: Boolean)(Default: FALSE)
	• TRUE - Use the callout AgesaExternal2dTrainVrefChange • FALSE - Use internal Vref control
PcdAmdMemCfg Signature	"ASTR" for AMD Suspend-To-RAM Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg Version	S3 Params version number Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg Flags	Indicates operation Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg NvStorage	Pointer to memory critical save state data Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg NvStorageSize	Size in bytes of the NvStorage region Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg VolatileStorage	Pointer to remaining AMD save state data Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg VolatileStorageSize	Size in bytes of the VolatileStorage region

	Permitted Choices: (Type: Value)(Default: 0x0000)
PcdAmdMemCfg DataEyeEn	Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - Enable to get the 2D data eye • FALSE - The 2D data eye is not enabled
PcdAmdMemCfg BottomIo	Bottom of 32-bit PCI MMIO space (Addr[31:24]) Permitted Choices: (Type: Value)(Default: 0x00E0)
PcdAmdMemCfg UserTimingMode	Allows the platform to select how the determination is made for the memory clock that is to be used in the system. (See also PcdAmdMemCfgMemClockValue.) Permitted Choices: (Type: List)(Default: TIMING_MODE_AUTO) • TIMING_MODE_AUTO - The AGESA™ software calculates the best memory clock rate. • TIMING_MODE_LIMITED - The AGESA™ software calculates the best memory clock, restricted by a maximum user limit provided in PcdAmdMemCfgMemClockValue. • TIMING_MODE_SPECIFIC - The platform specifies the clock rate, which is provided in PcdAmdMemCfgMemClockValue.
PcdAmdMemCfg MemClockValue	This is the specified memory clock to be used in the system as determined by the selected mode. (See also PcdAmdMemCfgUserTimingMode.) Permitted Choices: (Type: List)(Default: see note) • DDR400_FREQUENCY • DDR533_FREQUENCY • DDR667_FREQUENCY • DDR800_FREQUENCY • DDR1066_FREQUENCY • DDR1333_FREQUENCY • DDR1600_FREQUENCY • DDR1866_FREQUENCY • DDR2100_FREQUENCY • DDR2133_FREQUENCY • DDR2400_FREQUENCY The default value used depends upon the type of memory present. DDR2 memories use a default value of DDR400_FREQUENCY. DDR3 memories use a default value of DDR800_FREQUENCY. DDR4 memories use a default value of DDR1866_FREQUENCY.
PcdAmdMemCfg DimmTypeUsedInMixedConfig	Specifies how to handle installations of mixed memory technology. If the processor supports more than one memory technology, only channels with the specified memory technology will be enabled and other channels will be disabled. Both sets of channels must contain supported memory technology DIMMs. Specifying a memory type that is not supported by

the processor is treated the same as specifying
UNSUPPORTED_TECHNOLOGY.

Permitted Choices: (Type: List)(Default: DDR3_TECHNOLOGY)

- DDR3_TECHNOLOGY - Enable only DDR3 memory channels
- DDR4_TECHNOLOGY - Enable only DDR4 memory channels
- GDDR5_TECHNOLOGY - Enable only GDDR5 memory channels
- UNSUPPORTED_TECHNOLOGY - Do not support mixed installations

Note: DDR2 is no longer supported.

PcdAmdMemCfg EccRedirection DRAM ECC redirection is a data-protection feature. Redirection is a special ECC feature that enables the scrubber to immediately scrub any address in which a correctable error is discovered. The AMD recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

PcdAmdMemCfg MemRestoreCtl Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - restore memory context and skip training.
- FALSE - perform memory init as normal

**PcdAmdMemCfg
SaveMemContextCtl**

Control switch to save memory context at the end of MemAuto

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - AGESA will setup MemContext block before exit AmdInitPost
- FALSE - AGESA will not setup MemContext block. Platform is, expected to call S3Save laterPOST if it wants to use memory context restore feature.

**PcdAmdMemCfg
MemoryModeUnganged**

The platform is designed to use the memory channel unganged mode.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
MemoryQuadRankCapable**

This is a platform-specific setting indicating that the memory slots are capable of supporting Quad Rank DIMMs.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
MemoryRDimmCapable**

Specifies if the platform is designed to be capable of supporting RDIMMs.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
MemoryLRDimmCapable**

Specifies if the platform is designed to be capable of supporting LRDIMMs.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
MemoryUDimmCapable**

Specifies if the platform is designed to be capable of supporting UDIMMs.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
MemorySODimmCapable**

Specifies if the platform is designed to be capable of supporting SoDIMMs.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
DimmTypeDddr3Capable**

Specifies if the platform is designed to be capable of supporting DDR3.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
DimmTypeDddr4Capable**

Specifies if the platform is designed to be capable of supporting DDR4.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
IgnoreSpdChecksum**

When the checksum of the SPD record fails, the typical action is to drop the DIMM and not attempt to configure it into the system. The software indicates via return code that this condition has occurred. Under certain conditions, the platform can decide to accept the associated risk and choose to ignore the SPD checksum.

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - Ignore faulty SPD checksum and continue DIMM initialization
- FALSE - If the checksum of an SPD is found to be invalid, then the memory initialization process is terminated for that DIMM

8.2.1.3 Power Management Category

PcdAmdMemCfg EnableDlIPDBypassMode	This element enables a low power DDR bus operation mode, which is useful for soldered down memory subsystems that follow low power design guidelines. This feature will limit the maximum DDR rate and should be disabled for high performance memory subsystem designs. This feature should be disabled for systems with DIMM slots rather than soldered down memory. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg EnableExtended TemperatureRange	This feature will maintain a refresh rate appropriately higher for the DIMM based upon the temperature. This may cause higher power consumption and a slight loss of performance. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - The device may support the extended temperature range (0-95oC). The refresh rate will be adjusted as the temperature on the DIMM changes. • FALSE - the feature is disabled. Normal (0-85oC) range is expected.
PcdAmdMemCfg DramTempControlled RefreshEn	Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg MemoryPowerPolicy	This value provides for advanced performance tuning by controlling the power saving features. Permitted Choices: (Type: List)(Default: A) • Performance - (Default) Maximize performance; disables all power saving features. • BatteryLife - Maximize battery life; some features may cause response latency. • Auto - A balanced approach; save power without causing most latencies.
PcdAmdMemCfg EnablePowerDown	This feature conserves power in the DIMMs by placing them into self-refresh when the memory on a channel is not actively being accessed. They quickly exit self-refresh upon the next processor access. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg MemoryAllClocksOn	This is a power-usage control. To save power, unused memory clocks are disabled. Some platforms may not prefer to use this feature. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - Enable all memclocks whether they are used or not. • FALSE - Unused memclocks are disabled

**PcdAmdMemCfg
PowerDownMode**

This is a power-usage control that performs memory clock enable-based power-down control. If the platform enables power-down capability, then this element describes the platform method chosen. If the platform is not indicated to have power-down capability, then this element has no affect.

Permitted Choices: (Type: List)(Default: POWER_DOWN_MODE_AUTO)

- POWER_DOWN_BY_CHANNEL
- POWER_DOWN_BY_CHIP_SELECT
- POWER_DOWN_MODE_AUTO

8.2.1.4 Performance Category**PcdAmdMemCfg
DramMacDefault**

This is a performance and security optimization setting that indicates a limit on how fast back-to-back accesses are permitted. This is a new feature to the JEDEC DDR3 and DDR4 specifications to prevent a security vulnerability referred to as ‘hammering’. The addition was a MAC parameter (Maximum Activity Count). This control works in conjunction with the PcdAmdMemCfgDramDoubleRefreshRateEn control above. For AMD APU, this feature is implemented as a Trcpge timing parameter. For APU without Trcpge:

```

        if (PcdAmdMemCfgDramMacDefault == MAC_UNTESTEDMAC)
            Tref is set according to
PcdAmdMemCfgDramDoubleRefreshRateEn
        else {
            if (PcdAmdMemCfgDramMacDefault != MAC_UNRESTRICTEDMAC)
                if (SPD.MAC == 'Unrestricted')
                    Tref is set according to
PcdAmdMemCfgDramDoubleRefreshRateEn
            else
                Tref is always set to double refresh rate;
        }

```

For APU with Trcpge:

```

        //Tref is always set according to
BLDCFG_DRAM_DOUBLE_REFRESH_RATE.
        if (SPD.MAC == 'untested')
            { // (0x00)defined by JEDEC as backward compatible SPD
entry
            if (PcdAmdMemCfgDramMacDefault == MAC_UNTESTEDMAC)
                'Unrestricted MAC' is applied to Trcpge
            else
                PcdAmdMemCfgDramMacDefault is applied as the platform
specified MAC to Trcpge.
            }

```

Permitted Choices: (Type: List)(Default: MAC_UNTESTEDMAC)

- MAC_UNTESTEDMAC - Use the SPD setting or the double rate refresh per the above equations.
- MAC_700k - Allow 700K accesses within a single refresh period
- MAC_600k - Allow 600K accesses within a single refresh period
- MAC_500k - Allow 500K accesses within a single refresh period
- MAC_400k - Allow 400K accesses within a single refresh period
- MAC_300k - Allow 300K accesses within a single refresh period.
- MAC_200k - Allow 200K accesses within a single refresh period

	• MAC_UNRESTRICTEDMAC - Unrestricted - allow unlimited accesses
PcdAmdMemCfg AmpWarningMsgEnable	Specifies to provide warning events if the AMP capability does not match the desired AMP mode. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - Generate warning events to the event log • FALSE - Suppress warning events
PcdAmdMemCfg MemHoleRemapping	Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg LimitMemoryToBelow1Tb	This item provides for ensuring that the top of memory is limited to below 1TByte. This may be needed for certain operating systems. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - Top of memory will be below 1 TByte no matter how much memory is installed; any additional memory will not be accessible. • FALSE - Not limited. Top of memory will be greater than 1 TByte if sufficient memory is installed. Memory may be hoisted above 1 TByte to make it accessible.
PcdAmdMemCfg EnableBankSwizzle	Address swizzle is a performance fine-tuning element that swaps some address lines. See the BKDG for further description. Permitted Choices: (Type: Boolean)(Default: TRUE) • TRUE - This option is active • FALSE - This option is turned off
PcdAmdMemCfg EnableBankSwapOnly	This is an advanced performance setting. It specifies if the system should use DRAM BankSwapOnly mode. The AMD recommended setting is indicated by the default setting. Changes to this setting must be based on individual platform testing. Permitted Choices: (Type: List)(Default: Auto) • Disable • Enable • Auto
PcdAmdMemCfg DramDoubleRefreshRateEn	This is a performance optimization setting and indicates whether or not to refresh the memories at twice the rate calculated or as indicated in the SPD information. This faster frequency refresh is likely to be employed in cases where extended temperature support is desired or smaller geometry DRAM technologies are utilized. Permitted Choices: (Type: Boolean)(Default: FALSE) • TRUE - Double the refresh rate (cut period in half). {Example: 3.9us} • FALSE - Use the standard refresh rate. {Example: 7.8us}

**PcdAmdMemCfg
PmuTrainMode**

For processors which provide a Phy Micro-controller Unit (PMU), this parameter selects the memory data bus training mode which the PMU will perform.

Permitted Choices: (Type: List)(Default: PMU_TRAIN_AUTO)

- PMU_TRAIN_1D - perform 1D training only
- PMU_TRAIN_1D_2D_READ - perform both 1D and 2D read only training
- PMU_TRAIN_1D_2D - perform both 1D and 2D training
- PMU_TRAIN_AUTO - The training mode will be automatically selected based on configuration

**PcdAmdMemCfg
ForceTrainMode**

Permitted Choices: (Type: List)(Default: Auto)

- Force 1D Training for all configurations
- Force 2D Training for all configurations
- Auto - AGESA will control 1D or 2D

**PcdAmdMemCfg
ScrubDramRate**

This value selects how often the ECC background scrubber makes a pass through the DRAM. This is a numeric value and the value can vary from processor family to family.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0 = Disabled
- X - see BKDG for available scrub rates

**PcdAmdMemCfg
ScrubL2Rate**

This value selects how often the ECC background scrubber makes a pass through the L2 cache. This is a numeric value and the value can vary from processor family to family.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0 = Disabled
- X - see BKDG for available scrub rates

**PcdAmdMemCfg
ScrubL3Rate**

This value selects how often the ECC background scrubber makes a pass through the L3 cache. This is a numeric value and the value can vary from processor family to family.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0 = Disabled
- X - see BKDG for available scrub rates

**PcdAmdMemCfg
ScrubIcRate**

This value selects how often the ECC background scrubber makes a pass through the IC (Instruction Cache). This is a numeric value and the value can vary from processor family to family.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0 = Disabled
- X - see BKDG for available scrub rates

**PcdAmdMemCfg
ScrubDcRate**

This value selects how often the ECC background scrubber makes a pass through the DC (Data Cache). This is a numeric value and the value can vary from processor family to family.

Permitted Choices: (Type: Value)(Default: 0x0000)

- 0 = Disabled
- X - see BKDG for available scrub rates

**PcdAmdMemCfg
EccSyncFlood**

This control provides the ability to cause system fatal errors (sync flood) when an ECC error occurs. This is a security feature to help prevent unauthorized access to DRAM.

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - This option is active
- FALSE - This option is turned off

**PcdAmdMemCfg
EccSymbolSize**

This is an error-detection control. Please see the BKDG. for a description. This is a numeric value of 0, 4, or 8. Zero means to use the BKDG recommendation.

Permitted Choices: (Type: List)(Default: 4)

- 0
- 4 - 4-bit ECC symbols
- 8 - 8-bit ECC symbols

**PcdAmdMemCfg
IsCapsuleMode**

Permitted Choices: (Type: Boolean)(Default: FALSE)

- TRUE - This is a capsule reboot
- FALSE - This is not a capsule reboot

8.3 Core Complex Driver for Family 15h

Summary

The Core Complex Driver (CCX) is responsible for starting the cores and configuring the power management systems.

Description

The main CCX Driver is described in [Core Complex Driver](#). This is a description of the controls unique to the Family 15h processors.

8.3.1 Configuration Items for Family 15h (CCX Driver)

These configuration items pertain only to the Family 15h processors.

Enablement Category

Platform Category

PcdAmdNumberOfIoApics This element provides the number of IO APICs that are used on the motherboard. This is used by the software to appropriately assign APIC IDs to the processors, leaving room for the motherboard devices.

Permitted Choices: (Type: Value)(Default: 0x03)

Power Management Category

PcdAmdPowerCeiling Specifies a maximum power limit for the platform. This indicates the capability of the thermal solution (e.g. heat sink) implemented on this platform to dissipate power. For processors with BAPM support, this limit will be used as the target BAPM TDP. In addition, P-State capping may be used to implement the TDP limit. This value is the integer number, in milliwatts, to be used as the TDP limit.

Permitted Choices: (Type: Value)(Default: 0)

Performance Category**8.4 NBIO Driver for Family 15h****Summary**

The NBIO driver is responsible for initializing the PCIe® root complex.

Description

The NBIO Driver is new to AGESA V9 and was previously covered as part of the GNB. The main portion of the NBIO Driver is described in [NBIO Driver](#). This section describes the controls unique to the Family 15h processors.

8.4.1 Configuration Items for Family 15h (NBIO Driver)

These configuration items pertain only to the Family 15h processors.

Enablement Category

PcdAmdCfg GnbPcieSSID This 32-bit value defines the subsystem ID assigned to PCIe® bridges for Family 15h.

Permitted Choices: (Type: Value)(Default: 0x12341022)

PcdAmdCfg GnbIGPUSSID This 32-bit value defines the subsystem ID assigned to internal graphics for Family 15h processors.

Permitted Choices: (Type: Value)(Default: 0x00000000)

PcdAmdCfg GnbHDAudioSSID This 32-bit value defines the subsystem ID assigned to internal HD audio device for Family 15h processors.

Permitted Choices: (Type: Value)(Default: 0x00000000)

Platform Category

Power Management Category

Performance Category

8.4.2 PEI_AMD_NBIO_PCIE_COMPLEX_FM15_PPI

Summary

Call-out to the Host BIOS for services to obtain information about the PCIe® device assignment and layout.

GUID

```
#define AMD_NBIO_PCIE_COMPLEX_FM15_PPI_GUID
{ 0xdc2770d, 0xede5, 0x41d0, { 0xa8, 0x84, 0xa8, 0x16, 0x8a, 0xcc, 0xa1, 0x5d }}
```

PPI Interface Structure

```
typedef struct _PEI_AMD_NBIO_PCIE_COMPLEX_FM15_PPI {
    UINT32                Revision;
    AMD_NBIO_PCIE_GET_COMPLEX_FM15_STRUCT  PcieGetComplex;
} PEI_AMD_NBIO_PCIE_COMPLEX_FM15_PPI;
```

Parameters

Revision

Revision identifier for this PPI interface.

PcieGetComplex

Function to obtain a pointer to the PCIe® complex descriptor tables.

Description

The Host BIOS must create and publish this PPI. The AGESA™ NBIO driver will use this PPI to obtain the platform dependent configuration information for the PCIe® ports, lanes and devices.

For backward compatibility, the Family 15h support code in AGESA V9 supports the same data structure for the PCIe® Port Descriptor List and DDI Link Descriptor list as that used in AGESA Arch2008. Platform BIOS must publish the PEI_AMD_NBIO_PCIE_COMPLEX_FM15_PPI when the topology structure is initialized.

8.4.2.1 PcieGetComplex

Summary

Request a pointer to the PCIe® complex descriptor tables.

Prototype

```
typedef
EFI_STATUS
PcieGetComplex (
    IN      PEI_AMD_NBIO_PCIE_COMPLEX_FM15_PPI  *This,
    OUT     DXIO_COMPLEX_DESCRIPTOR_FM15        **UserConfig
);
```

Parameters

This

A pointer to this PPI entry.

UserConfig

Pointer to a caller provided storage space for a pointer to the PCIe® complex data structures.

Description

This Function is used by the AGESA™ DXIO driver to obtain a pointer to the platform PCIe® complex descriptor tables. This procedure will fill the caller provided pointer location with the address of the PCIe® complex table created for this platform.

Related Definitions

Please refer to the [DXIO common structures](#) section.

Status Codes Returned

status code	meaning
AGESA_SUCCESS	The function has completed successfully.

8.5 Data Fabric for Family 15h

Summary

The Data Fabric Driver for Family 15h is responsible for initializing controls used by the 'Northbridge' component of those processors.

Description

The main Data Fabric Driver is described in [Data Fabric Driver](#). This is a description of the controls unique to the Family 15h processors.

8.5.1 Configuration Items for Family 15h (Data Fabric Driver)

These configuration items pertain only to the Family 15h processors.

Enablement Category

Platform Category

Power Management Category

PcdAmdFabricPstateSupport This value indicates whether to enable processor northbridge (NB) P-States (as opposed to processor core P-States).

Permitted Choices: (Type: Boolean)(Default: TRUE)

- TRUE - Enable NB P-States, if supported by the processor.
- FALSE - Disable NB P-States.

Performance Category